



Teradata Vantage Administration

Version 17.10.3

57040
Student Guide

Trademarks

The product or products described in this book are licensed products of Teradata Corporation or its affiliates.

Teradata, Applications-Within, Aster, BYNET, Claraview, DecisionCast, Gridscale, MyCommerce, QueryGrid, SQL-MapReduce, Teradata Decision Experts, "Teradata Labs" logo, Teradata ServiceConnect, Teradata Source Experts, WebAnalyst, and Xkoto are trademarks or registered trademarks of Teradata Corporation or its affiliates in the United States and other countries.

Adaptec and SCSISelect are trademarks or registered trademarks of Adaptec, Inc.

Amazon Web Services, AWS, [any other AWS Marks used in such materials] are trademarks of Amazon.com, Inc. or its affiliates in the United States and/or other countries.

AMD Opteron and Opteron are trademarks of Advanced Micro Devices, Inc.

Apache, Apache Avro, Apache Hadoop, Apache Hive, Hadoop, and the yellow elephant logo are either registered trademarks or trademarks of the Apache Software Foundation in the United States and/or other countries. Apple, Mac, and OS X all are registered trademarks of Apple Inc.

Axeda is a registered trademark of Axeda Corporation.

Axeda Agents, Axeda Applications, Axeda Policy Manager, Axeda Enterprise, Axeda Access, Axeda Software Management, Axeda Service, Axeda ServiceLink, and Firewall-Friendly are trademarks and Maximum Results and Maximum Support are servicemarks of Axeda Corporation.

CENTOS is a trademark of Red Hat, Inc., registered in the U.S. and other countries.

Cloudera, CDH, [any other Cloudera Marks used in such materials] are trademarks or registered trademarks of Cloudera Inc. in the United States, and in jurisdictions throughout the world.

Data Domain, EMC, PowerPath, SRDF, and Symmetrix are registered trademarks of EMC Corporation.

GoldenGate is a trademark of Oracle.

Hewlett-Packard and HP are registered trademarks of Hewlett-Packard Company.

Hortonworks, the Hortonworks logo and other Hortonworks trademarks are trademarks of Hortonworks Inc. in the United States and other countries.

Intel, Pentium, and XEON are registered trademarks of Intel Corporation.

IBM, CICS, RACF, Tivoli, and z/OS are registered trademarks of International Business Machines Corporation.

Linux is a registered trademark of Linus Torvalds.

LSI is a registered trademark of LSI Corporation.

Microsoft, Active Directory, Windows, Windows NT, and Windows Server are registered trademarks of Microsoft Corporation in the United States and other countries.

NetVault is a trademark or registered trademark of Dell Inc. in the United States and/or other countries.

Novell and SUSE are registered trademarks of Novell, Inc., in the United States and other countries.

Oracle, Java, and Solaris are registered trademarks of Oracle and/or its affiliates.

QLogic and SANbox are trademarks or registered trademarks of QLogic Corporation.

Quantum and the Quantum logo are trademarks of Quantum Corporation, registered in the U.S.A. and other countries.

Red Hat is a trademark of Red Hat, Inc., registered in the U.S. and other countries. Used under license.

SAP is the trademark or registered trademark of SAP AG in Germany and in several other countries.

SAS and SAS/C are trademarks or registered trademarks of SAS Institute Inc.

SPARC is a registered trademark of SPARC International, Inc.

Symantec, NetBackup, and VERITAS are trademarks or registered trademarks of Symantec Corporation or its affiliates in the United States and other countries.

Unicode is a registered trademark of Unicode, Inc. in the United States and other countries.

UNIX is a registered trademark of The Open Group in the United States and other countries.

Other product and company names mentioned herein may be the trademarks of their respective owners.

The information contained in this document is provided on an "as-is" basis, without warranty of any kind, either express or implied, including the implied warranties of merchantability, fitness for a particular purpose, or non-infringement. Some jurisdictions do not allow the exclusion of implied warranties, so the above exclusion may not apply to you. In no event will Teradata Corporation be liable for any indirect, direct, special, incidental, or consequential damages, including lost profits or lost savings, even if expressly advised of the possibility of such damages.

The information contained in this document may contain references or cross-references to features, functions, products, or services that are not announced or available in your country. Such references do not imply that Teradata Corporation intends to announce such features, functions, products, or services in your country. Please consult your local Teradata Corporation representative for those features, functions, products, or services available in your country.

Information contained in this document may contain technical inaccuracies or typographical errors. Information may be changed or updated without notice. Teradata Corporation may also make improvements or changes in the products or services described in this information at any time without notice.

Teradata Vantage Administration

Version 17.10.3

Module 0 – Course Overview

Course Materials	0-2
PDF Comment and Markup Tools.....	0-3
Example of Highlighter and Sticky Note Tools.....	0-4
Example of Add Text Tool	0-5
Open PDF File at Last Page Viewed	0-6
Course Description.....	0-7
Who Should Attend and Prerequisites	0-8
Class Format and Rules	0-9
Outline of the Class.....	0-10
Teradata Certification Exams	0-11

Module 1 – Database Administration and Building the Database Environment

Objectives	1-2
Database Administration.....	1-3
Initial Teradata Database	1-4
Administrative User	1-5
Owners, Parents, and Children.....	1-6
Creating New Users and Databases	1-7
Transfer of Ownership	1-8
DELETE / DROP Statements	1-9
Teradata Studio – New System.....	1-10
Teradata Studio – Hierarchy	1-11
Summary	1-12
Module 1: Review Questions.....	1-13

Module 2 – Teradata Studio

Objectives	2-2
Teradata SQL Tools.....	2-3
Defining a Connection	2-4
Defining a Connection (Transcend).....	2-6
Connecting to a Teradata System	2-7
Query Development: Main Window.....	2-8
Data Source Explorer Tree.....	2-9
Creating and Executing a Query	2-11
Dragging Object Names to the Query Window	2-12
Query Options	2-13
Viewing Query Results	2-14
SQL Query Builder	2-15
Teradata SQL History Window	2-16
General Options	2-17
Connecting to Multiple Data Sources	2-18
Teradata Studio Administration – Initial Display	2-19
Teradata Studio Administration –Show Databases.....	2-20
Teradata Studio Administration – Object List Viewer	2-21
Teradata Studio Administration – Object Details	2-22
Teradata Studio Summary.....	2-23
Module 2: Review Questions.....	2-24
If Using Transcend, Hands-on Setup	2-25
Lab Exercise 2-1	2-27

Module 3 – Index Considerations

Objectives	3-2
Primary Index Choice Criteria.....	3-3
Partitioning with RANGE_N – Example.....	3-4
ALTER TABLE – NO RANGE is not Defined.....	3-9
ALTER TABLE – NO RANGE is Defined.....	3-10
ALTER TABLE – Example.....	3-11
ALTER TABLE TO CURRENT.....	3-12
ALTER TABLE TO CURRENT – Example.....	3-13
ADD Partitions Option – Example	3-14
What is a NoPI Table?	3-15
Loading Data into a NoPI Table (SQL).....	3-16
Loading Data into a NoPI Table (Utility).....	3-17
Secondary Index Usage.....	3-18
Value-Ordered NUSIs.....	3-19
Range Accesses for a NUSI.....	3-20
Hash-Ordered NUSIs	3-21
Hash-Ordered NUSI Example	3-22
Compressed and Non-Compressed Join Indexes.....	3-23
Non-Compressed Multi-Table Join Index	3-24

ALTERing a Join Index to CURRENT	3-25
Partitioning a Sparse Join Index	3-26
Global Index as a “Hashed NUSI”.....	3-27
Creating a Global Index (“Hashed NUSI”)	3-28
Column Partitioned Tables – Primary AMP	3-29
TableSize View.....	3-30
Summary	3-31
Module 3: Review Questions.....	3-32
Lab Exercise 3-1	3-33
Lab Exercise 3-2	3-35
Lab Exercise 3-3	3-39

Module 4 – The Data Dictionary

Objectives	4-2
Data Dictionary / Directory	4-3
Fallback Protected Data Dictionary Tables	4-4
Non-Hashed Data Dictionary Tables	4-5
Updating Data Dictionary Tables	4-6
Supplied Data Dictionary Views	4-7
Restricted Views	4-8
Suffix Options with Views.....	4-9
Selecting Information about Created Objects	4-10
Children View	4-11
Databases View.....	4-12
Users View.....	4-13
Tables View	4-14
Columns View	4-15
Indices View	4-16
Indices View (Second Example).....	4-17
Partitioning Constraints View	4-18
Show Table Checks View	4-19
Show Column Checks View	4-20
Triggers View	4-21
All Temporary Tables View	4-22
Referential Integrity Views.....	4-23
Using the DBC.Tables View.....	4-24
Referential Integrity States	4-25
DBC.All_RI_Children View.....	4-26
DBC.Databases2 View	4-27
Time Stamps in Data Dictionary.....	4-28
Summary	4-29
Module 4: Review Questions.....	4-30
Lab Exercise 4-1	4-31
Lab Exercise 4-2 (optional: 1 of 2).....	4-34
Lab Exercise 4-2 (optional: 2 of 2).....	4-35

Module 5 – Space Allocation and Usage

Objectives	5-2
Permanent Space Terminology	5-3
Spool Space Terminology	5-4
Temporary Space Terminology	5-5
Resetting Peak Values	5-6
Assigning Perm and Spool Limits	5-7
Giving One User to Another	5-9
Teradata Studio – Move Space	5-10
Reserving Space for Spool	5-11
Views for Space Allocation Reporting	5-12
DiskSpace View	5-13
TableSize View	5-14
AllSpace View	5-15
DataBaseSpace Table	5-16
Different Views — Different Results	5-17
Global Space Accounting	5-18
Global Space Accounting Example	5-19
Additional Utilities to View Space Utilization	5-20
Teradata Studio – Object Viewer Options	5-21
Teradata Viewpoint – Space Usage	5-22
Ferret Utility	5-23
Ferret SHOWSPACE Command	5-24
Ferret SHOWBLOCKS Command	5-25
Module 5: Review Questions	5-26

Module 6 – Users, Accounts, and Accounting

Objectives	6-2
Creating New Users and Databases	6-3
CREATE DATABASE Statement	6-4
CREATE USER Statement	6-5
CREATE USER and the Data Dictionary	6-6
MODIFY USER Statement	6-8
Teradata Studio – Create User	6-9
Creating and Using Account IDs	6-10
Dynamically Changing an Account ID	6-11
Account String Expansion	6-12
ASE Accounting Example	6-13
System Accounting Views	6-14
AccountInfo View	6-15
AMPUsage View	6-16
AMPUsage View – Example	6-17
Summary	6-18
Module 6: Review Questions	6-19
Lab Exercise 6-1	6-20

Module 7 – Profiles

Objectives	7-2
Example of Simplifying User Management	7-4
Implementing Profiles.....	7-5
Impact of Profiles on Users.....	7-6
CREATE/MODIFY PROFILE Statement.....	7-7
Password Attributes (CREATE/MODIFY PROFILE).....	7-8
Teradata Password Control	7-9
Teradata Password Control Options	7-11
CREATE PROFILE Example.....	7-12
Teradata Studio – Create Profile Example.....	7-13
DROP PROFILE Statement.....	7-18
ProfileInfo View	7-19
Miscellaneous SQL Functions	7-20
Summary	7-21
Module 7: Review Questions.....	7-22
Lab Exercise 7-1	7-23

Module 8 – Access Rights

Objectives	8-2
Privileges/Access Rights.....	8-3
Access Rights Mechanisms.....	8-4
CREATE TABLE – Automatic Rights.....	8-5
CREATE USER – Automatic Rights	8-6
Implicit, Automatic, and Explicit Rights	8-7
GRANT Command (SQL Form)	8-8
Granting Rights at Database Level	8-9
GRANT Rights at the Table or Column Level	8-10
REVOKE Command (SQL Form).....	8-11
Revoking Non-Existent Rights	8-12
Removing a Level in the Hierarchy	8-13
Inheriting Access Rights	8-14
The GIVE Statement and Access Rights	8-15
Access Rights and Views.....	8-16
Access Rights and Nested Views.....	8-17
System Views for Access Rights	8-18
AllRights and UserRights Views	8-19
UserGrantedRights View	8-20
Teradata Studio – GRANT/REVOKE Privileges	8-21
Summary	8-22
Module 8: Review Questions.....	8-23

Module 9 – Roles

Objectives	9-2
What is a Role?	9-3
Access Rights Without Roles.....	9-4
Access Rights Using a Role.....	9-5
Implementing Roles.....	9-6
Current or Active Roles	9-7
Nesting of Roles.....	9-8
Example of Using "Nested Roles"	9-9
Access Rights Validation and Roles	9-10
SQL Statements to Support Roles.....	9-11
GRANT Command (SQL Form)	9-12
REVOKE Command (SQL Form).....	9-13
GRANT and REVOKE Commands (Role Form).....	9-14
System Hierarchy (Used in Following Examples).....	9-15
Example of Using Roles	9-16
Roles for Directory-Based Users	9-19
Roles for Proxy Users	9-20
RoleInfo View.....	9-21
RoleMembers View	9-22
DBC.AccessRights and "Rights" Views.....	9-23
AllRoleRights and UserRoleRights Views.....	9-24
Steps to Implementing Roles	9-25
Summary	9-26
Module 9: Review Questions.....	9-27
Lab Exercise 9-1	9-28
Lab Exercise 9-2	9-30

Module 10 – System Access Controls

Objectives	10-2
System Access Control Levels.....	10-3
Teradata Access Control Mechanisms.....	10-4
Teradata Password Encryption	10-5
Password Security Features	10-6
Teradata Connectivity	10-7
Host Logon Processing	10-8
Objects Used in Host Logon Processing.....	10-9
GRANT/REVOKE LOGON Statements.....	10-10
GRANT/REVOKE LOGON Examples.....	10-11
Session Related Views.....	10-12
LogonRules View	10-13
LogOnOff View	10-14
SessionInfo View	10-15
Additional Utilities to View Sessions	10-16
Viewpoint – Query Monitor.....	10-17
Viewpoint – Remote Console – Query Session.....	10-18
A Recommended Structure Using Roles	10-19

A Recommended System Hierarchy	10-20
Teradata Secure Zones	10-21
Multi-tenancy Secure Zones Example	10-22
Summary	10-23
Module 10: Review Questions.....	10-24

Module 11 – Access and Query Logging

Objectives	11-2
Access Logging.....	11-4
Objects Used in Access Logging	11-5
BEGIN LOGGING Statement	11-6
END LOGGING Statement	11-7
Setting Up Access Logging	11-8
Access Log Views.....	11-9
AccLogRules View	11-10
BEGIN LOGGING – Example.....	11-11
AccessLog View	11-12
AccessLog View – Example.....	11-13
END LOGGING – Example.....	11-14
Query Logging (DBQL) Concepts.....	11-15
Objects Used in Defining Rules for DBQL	11-16
Objects Used in DBQL	11-17
BEGIN QUERY LOGGING	11-18
BEGIN QUERY LOGGING WITH	11-19
BEGIN QUERY LOGGING LIMIT	11-21
BEGIN QUERY LOGGING Examples.....	11-22
END QUERY LOGGING Statement	11-25
REPLACE QUERY LOGGING.....	11-26
FLUSH QUERY LOGGING.....	11-27
PARAMINFO, UTILITYINFO Options	11-28
DBQL Lock Logging Enhancement / PLL	11-29
DBQLRules View	11-30
DBC.QryLog View – Example.....	11-31
QryLogSummary View – Example	11-32
Summary	11-33
Module 11: Review Questions.....	11-34
Lab Exercise 11-1	11-35
Lab Exercise 11-2	11-37

Module 12 – Workload Management

Objectives	12-2
What is Workload Management?.....	12-3
Workload Management Benefits	12-4
Workload Management.....	12-5
Workload Management Offering Comparison	12-6
Workload Management Capabilities.....	12-7
Query Management Architecture.....	12-8
Workload Management Architecture.....	12-9
Filters	12-10
Throttles	12-11
Workload Designer Example.....	12-12
Workload Definitions.....	12-13
Arrival Rate Meter	12-14
Workload Designer – Workloads.....	12-15
Levels of Workload Management.....	12-16
Viewpoint – Monitoring and Reporting.....	12-17
Virtual Partitions	12-18
Control Groups and Workloads	12-19
Summary	12-20
Module 12: Review Questions.....	12-21

Module 13 – Teradata Viewpoint

Objectives	13-2
Viewpoint Architecture.....	13-4
Data Collection Service	13-5
Viewpoint Portal and Portlets	13-6
Logging onto Viewpoint.....	13-7
Updated User Interface	13-8
Adding Portlets	13-9
Example of Initial Session and Portlets	13-10
Viewpoint Portlet Controls	13-11
Viewpoint Rewind	13-12
Query Monitor	13-13
Viewpoint Query Monitor Detail View	13-14
My Queries.....	13-15
PJI and UII Metrics.....	13-16
Remote Console	13-17
Viewpoint Alert Viewer.....	13-18
Viewpoint SQL Scratchpad	13-19
Viewpoint Administration	13-20
Viewpoint Dashboard Feature	13-21
Viewpoint Mobile Feature	13-22
Summary	13-23
Module 13: Review Questions.....	13-24

Lab Exercise 13-1	13-25
Lab Exercise 13-2	13-27
Lab Exercise 13-3 (optional)	13-28

Module 14 – ResUsage – Performance Monitoring

Objectives	14-2
Performance Monitoring Tools.....	14-3
Why Collect Performance Data?	14-4
Resource Usage Data	14-5
Logging ResUsage Data	14-6
Specifying ResUsage Tables and Logging Rates	14-7
Resource Usage Tables	14-8
Resource Usage Views	14-9
Resource Usage Macros.....	14-10
Example Output from DBC.ResNode.....	14-11
PM/API and Viewpoint.....	14-12
Teradata System Emulation Tool.....	14-13
Summary	14-14
Module 14: Review Questions.....	14-15

Module 15 – System Restarts

Objectives	15-2
Types of Restarts.....	15-3
Scheduled Restarts	15-4
Restart Using the “tpareset” Command	15-5
PDE States	15-6
Unscheduled Restarts.....	15-7
TPA Reset – Crashdumps	15-10
Allocating Crashdumps Space	15-11
TPA Dump Maintenance	15-12
Module 15: Review Questions.....	15-13

Module 16 – System and Maintenance Utilities

Objectives	16-2
Starting Teradata System Utilities	16-3
Using the Interfaces to Access Utilities	16-4
DBS Control Utility	16-5
DBS Control Record – General Fields.....	16-6
DBS Control Record – File System Fields	16-8
DBS Control Record – Performance Fields.....	16-9
Modifying DBS Control Fields.....	16-10
Checking Data Integrity	16-11
Ferret – Scandisk Utility	16-12
Checktable Utility	16-13
Checktable – Levels of Checking	16-14
Checktable – Example	16-15
Table Rebuild Utility	16-16
Recovery Manager Utility.....	16-17
Recovery Manager Commands.....	16-18
Rcvmanager – List Status	16-19
Rcvmanager – List Locks	16-20
Rcvmanager – List Status (2nd Example)	16-21
Rcvmanager – List Rollback Tables	16-22
Rcvmanager – Cancel Rollback on Table.....	16-23
Showlocks Utility.....	16-24
Orphan or Phantom Spool Issues.....	16-25
Update Space Utility	16-26
Vprocmanager.....	16-27
Summary	16-28
Module 16: Review Questions.....	16-29

Module 17 – MAPS

Objectives	17-2
MAPS High Level Description.....	17-3
MAPS System Expansion Concept.....	17-4
MAPS Multiple Reconfigurations	17-5
Special Maps	17-6
Moving a Table to a New Map	17-7
System Expansion & Table Migration.....	17-8
What is TDMaps?	17-9
Steps to Move Tables.....	17-10
ALTER TABLE Possible Issues.....	17-11
GRANT MAP	17-12
Sparse Maps	17-13
Colocation Name	17-14
Miscellaneous SQL and Hash Functions	17-15
Summary	17-16

Module 17: Review Questions.....	17-17
Lab Exercise 17-1	17-19

Module 18 – Recap

Survey Question.....	18-2
Objectives	18-3
Teradata DBA Review	18-4
Dictionary Tables to Maintain	18-5
Plan and Follow-up	18-6
Things You Never Have to do with Teradata	18-7
Teradata Certification Exams	18-9

Appendix A:	Review Question Solutions
Appendix B:	Lab Exercise Solutions
Appendix C:	Acronyms
Appendix D:	Lab Environment
Appendix E:	Data Dictionary Views
Appendix F:	Permanent Journals



Module 0: Course Overview

Teradata Vantage Administration

Copyright © 2007–2023 by Teradata. All Rights Reserved.

Course Materials

The primary course material for this 5-day VILT (Virtual Instructor Led Training) course is:

- Electronic copy (PDF file) of the Student Manual

The course materials that are provided on a USB flash drive are listed on this slide. These materials are provided at the beginning of the class.

The Student Manual and the Lab Workbook have been created as PDF files which can be viewed using Adobe® Reader®.

These PDF files were created using Adobe Acrobat® and commenting has been enabled for both files. This allows you to use Adobe® Reader® Comment and Markup tools to place your own notes and comments within the files.

View and search a PDF

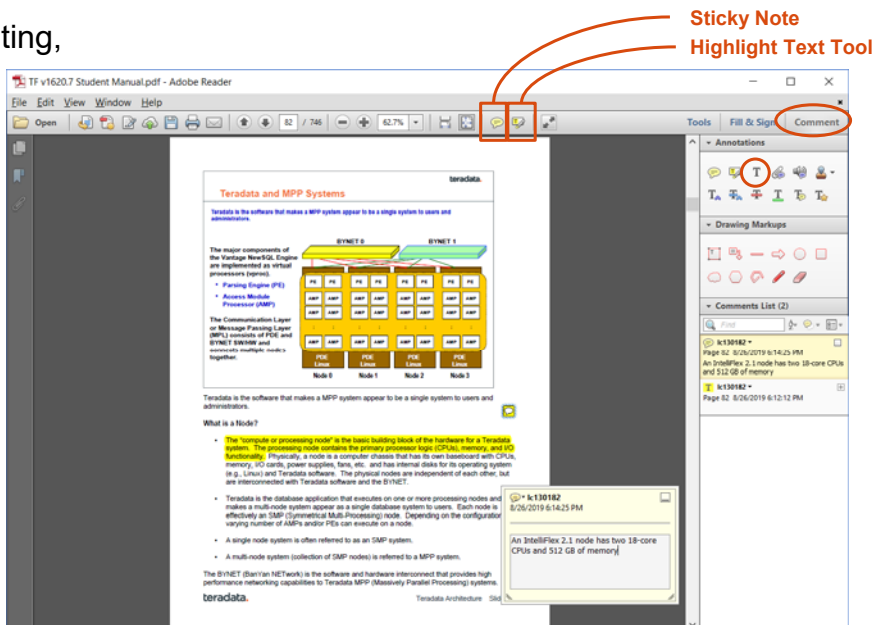
In the Adobe Reader toolbar, use the Zoom tools and the Magnification menu to enlarge or reduce the page. Use the options on the View menu to change the page display. There are various options in the Tools menu to provide you with more ways to adjust the page for better viewing (Tools > Select & Zoom).

PDF Comment and Markup Tools

You can add comments, highlighting, and use other markup tools.

Examples are:

- Sticky Note
- Highlight Text Tool
- Add Text



These course materials have "commenting" enabled. Therefore, you can make comments in these files using the commenting and markup tools. Of the many commenting and markup tools that are available, you may find it easier to use the following tools (highlighted on this slide).

- Add Sticky Note
- Highlight Text Tool
- Add Text

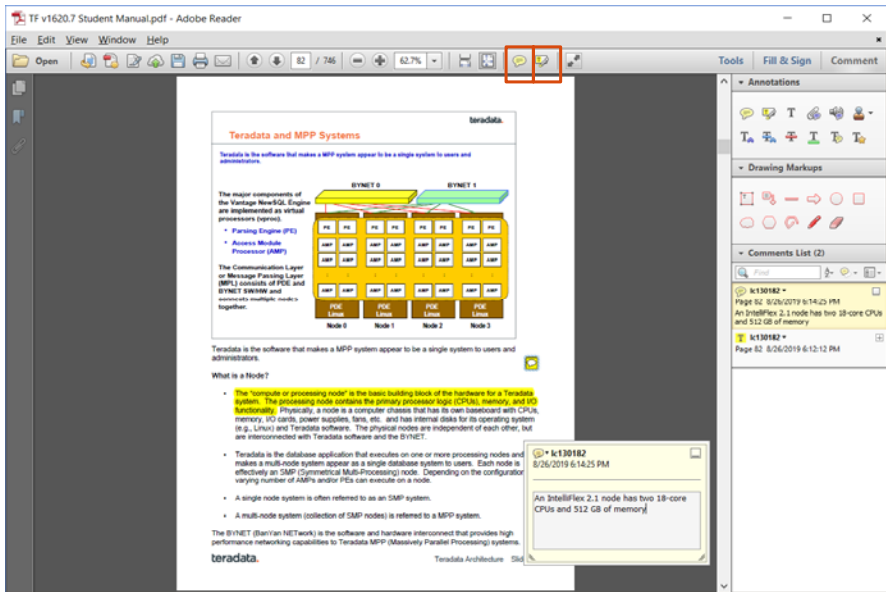
Comments can include both notes and drawings (if you have the time during class). You can enter a text message using the Sticky Note tool. You can use a drawing tool to add a line, circle, or other shape and then type a note in the associated pop-up note.

You can enable the Comment & Markup Toolbar or you can simply select the tools using the pull-down menus

After you add a note or comment, it stays selected until you click elsewhere on the page. A selected comment is highlighted by a blue halo to help you find the markup on the page.

Example of Highlighter and Sticky Note Tools

After including highlights and comments in your PDF file, a recommendation is to use the Save AS option and save the updated version with a new name – maybe include your initials.



This slide illustrates an example of using the Highlighter and Sticky Note tools.

Select a commenting or markup tool.

Choose Tools > Comment & Markup > *Highlighter or Sticky Note (or another tool)*

Note: After you make an initial comment, the tool changes back to the Select tool so that you can move, resize, or edit your comment. (The Pencil, Highlight Text, and Line tools stay selected.)

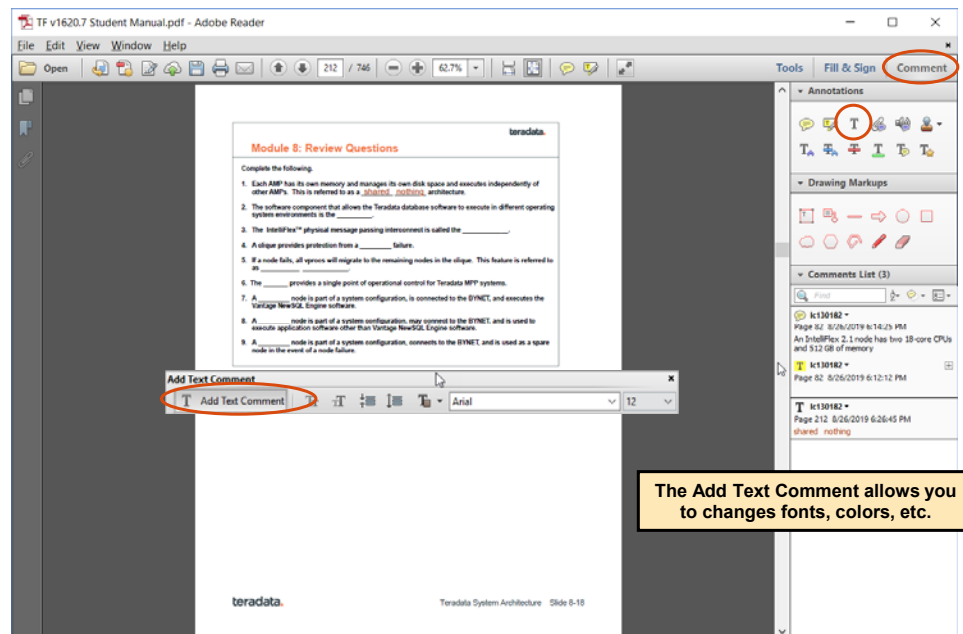
To keep a commenting tool selected so you can add multiple comments without reselecting the tool, do the following:

- Select the tool you want to use (but don't use it yet).
- Choose View > Toolbars > Properties Bar.
- Select Keep Tool Selected.

You can change the font of a text in a sticky note. Open the sticky note, choose View > Toolbars > Properties Bar, select the text in a note, and then change the font size in the Properties Bar.

Example of Add Text Tool

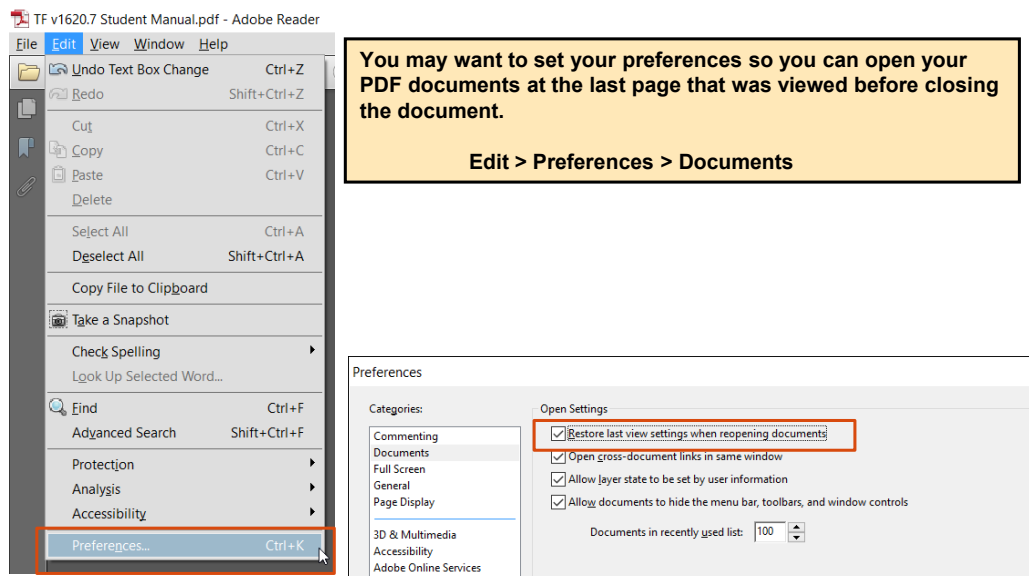
The Add Text Comment tool can be used to add text at any location in the PDF file.



The Add Text Toolbar may be useful when completing review questions as shown on this slide. You already have the answer to one of hundreds of questions in this course.

After making notes and comments, save your changes. You may want to save your changes to a different PDF file name in order to preserve the original PDF file.

Open PDF File at Last Page Viewed



If you are reading a 1500+ page PDF manual and you get to page 500, you may want to start on page 500 the next time you open the PDF file. The default is to return you to the beginning of the PDF file. There are multiple options that can be used to open your PDF file at the last page you viewed.

One option is to set a bookmark and select the bookmark when you reopen the file.

Another option is set your preferences so you open every PDF document at the last page that was viewed before closing it.

The next time you reopen your PDF file, you will be able to continue reading from exactly where you were when you closed the PDF document.

This setting applies to all of the PDF files you that you open.

Course Description

The primary focus of this 5-day VILT course is to teach you about Teradata database administration topics.

The major topics covered in this course include:

- How to perform common administrative functions for the Teradata database and setting up the Teradata database
- How to use Teradata Studio for database administration
- Index considerations for DBAs
- Data Dictionary Tables and Views
- Creating Databases, Users, Accounting, and Profiles
- Space Allocation and Usage
- Access Rights and Roles
- Access and Query Logging
- Using Teradata Utilities such as dbcontrol, vprocmanager, etc.
- Viewpoint
- MAPS

Who Should Attend and Prerequisites

Who Should Attend

This course is designed for ...

- Teradata Professional Services Consultants
- Channel Partners
- Customers

Prerequisites

Required:

- A technical understanding of the Teradata database, indexing and data distribution concepts as well as general data processing concepts is necessary.
- The recommended course to attend is Teradata Factory Xpress – #56749.

Useful, but not required:

- Experience with relational databases and SQL
- Experience with large systems used with Teradata

Class Format and Rules

Class Format

This class consists of ...

- Instructor presentations
- Class discussions
- Workshop exercises

Classroom Rules

The session rules are ...

- Mute your session whenever possible to limit the amount of background noise.
- During lecture, only use your laptop to follow the class materials
- Come to class on time when the sessions start and after breaks
- Enjoy the class

This class will be conducted as a series of lectures with classroom discussions, review questions, and workshops.

Outline of the Class

1. Database Administration Overview
2. Using Studio as Query and Administration Tool
3. Index Considerations
4. Data Dictionary
5. Allocation of Space
6. Creating Users and Databases
7. Using Profiles to simplify user management
8. Access Rights
9. Using Roles to simplify access right management
10. Access Control
11. Access and Query Logging
12. Introduction to Workload Management
13. Viewpoint
14. ResUsage Tables, Views, and Macros
15. Restarting Teradata
16. System Utilities
17. MAPS
18. Recap

Teradata Certification Exams

Teradata Vantage
Associate
Certification

Teradata Vantage
Administration
Certification

Teradata Vantage
Architecture
Certification

Teradata Vantage
Data Engineering
Certification

Teradata Vantage
Analytics
Certification

Teradata Vantage
Data Science
Certification

Teradata Vantage
Master 2.0

Teradata Vantage
Data Science
Master

Depending upon the tests that are completed, you can earn various Teradata Certified designations. The Teradata Certification exams require knowledge plus experience with Teradata release 16 and higher. This manual will help you prepare for these Teradata tests, but many of the test questions are scenario-based and Teradata experience is needed to answer these types of questions. The websites listed below provide much more information. The first website listed below can be used to understand the concepts and objectives that are on each exam. These objectives will give you a better idea of what to study.

Teradata Certification (TCPP) Website: www.teradata.com/University/Certification

Bypass Exams_with_Prior_Teradata_14_Certification information:

www.teradata.com/University/Certification/Vantage-Certifications#BypassExams

Registration Information: www.pearsonvue.com/teradata

TCCP Contact Information: email: certification.Teradata@Teradata.com
phone: 1-800-845-2273

Disclaimer: The Teradata Certification tests include questions from a mix of sources and **require experience**, especially on tests other than the Associate Exam.



Module 1: Database Administration and Building the Database Environment

Teradata Vantage Administration

Copyright © 2007–2023 by Teradata. All Rights Reserved.

Objectives

After completing this module, you should be able to:

- Describe the purpose and function of an administrative user
- Differentiate between creators, owners (parents), and children
- Describe how to transfer ownership of databases and users

Database Administration

Some of the functions of a Teradata Database Administrator (DBA) include:

- User and Database Management
- Space Allocation and Usage
- Access of Objects (e.g., tables, views, macros, etc.)
- Access Control and Security
- System Maintenance
- System Performance and Resource Monitoring
- Data Archives, Restores, and Recovery

Examples of tools available to the Teradata DBA include:

- Use of Data Dictionary/Directory tables and views to manage the system
- **Teradata Studio – Administration perspective**
- **Teradata Viewpoint – set of portlets for database monitoring and administration**
- Teradata Logging facilities – access logging, query logging (DBQL), ResUsage
- Teradata Workload Analyzer
- Remote Console and System utilities – e.g., dbscontrol, ferret, rebuild, TSET, etc.
- User scripts and 3rd party applications

To do these functions, it is important to understand key concepts such as ...

- Teradata hierarchy and the concepts of ownership (parents and children)

Some of a Teradata DBA's functions include:

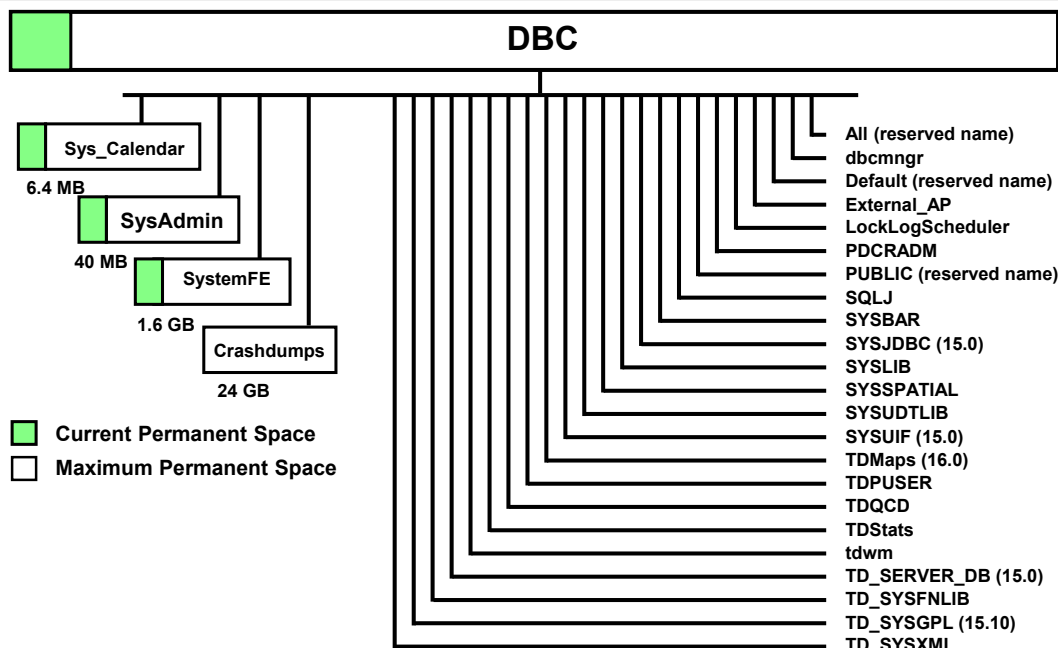
- User Management – creation of databases, users, accounts, roles, and profiles
- Space Allocation and Usage – perm, spool, and temporary space
- Access of Objects (e.g., tables, views) – access rights, roles, use of views, etc.
- Access Control and Security – logon access, logging access, etc.
- System Maintenance – specification of system defaults, restarts, data integrity, etc.
- System Performance – use of Priority scheduler, job scheduling, etc.
- Resource Monitoring – use of ResUsage tables/views, query capture (DBQL), etc.
- Data Archives, Restores, and Recovery – ARC facility, Permanent Journals, etc.

Examples of tools available to the Teradata DBA include:

- Use of Data Dictionary/Directory tables and views to manage the system
- Teradata Studio – Administration perspective
- Teradata Viewpoint – set of portlets for database monitoring and administration
- Various Teradata logging facilities – access, query, and ResUsage
- System utilities – e.g., dbscontrol, ferret, rebuild, etc.
- User scripts and 3rd party applications

To do these functions, it is important to understand key concepts such as the Teradata hierarchy and the concepts of ownership (parents and children). The hierarchy and the concept of ownership will be discussed in this module.

Initial Teradata Database



DBC

This system user named DBC initially owns all usable disk space. DBC's space includes dictionary tables, views and macros and is the overall parent of a Teradata database.

Sys_Calendar

This user is used to hold the system calendar table and views.

SysAdmin

SysAdmin is a system user with a minimum of space for table storage. SYSADMIN contains several supplied views and macros as well as a restart table for FastLoad jobs.

SystemFE User

This system user contains special macros used to generate diagnostic reports for Customer Engineers (field support personnel) logged on as this user. The default password is "service".

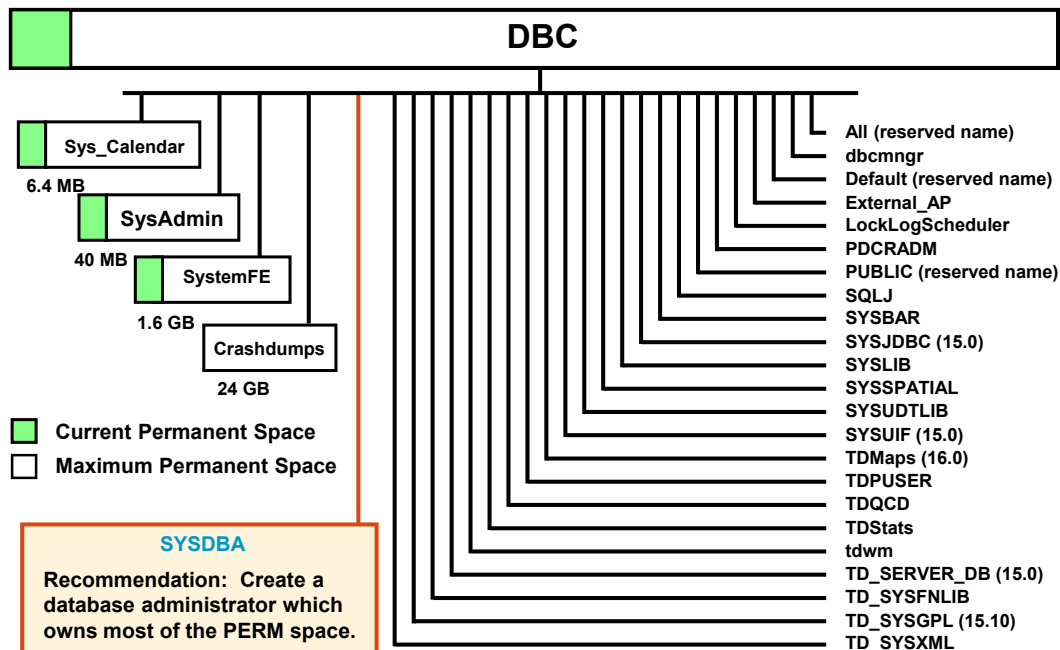
TDStats

This database contains collected statistics starting with Teradata 14.0.

TDMAPS

This database supports the Teradata MAPS feature (multiple hash maps..

Administrative User



System user DBC contains all Teradata Database software components and all system tables. Before you define application users and databases, you should first use the CREATE USER statement to create a special administrative user to complete these tasks.

The amount of space for the administrative user is allocated from DBC's current PERM space. DBC becomes the owner of your administrative user and of all users and databases you subsequently create. Be sure to leave enough space in DBC to accommodate the growth of system tables and logs.

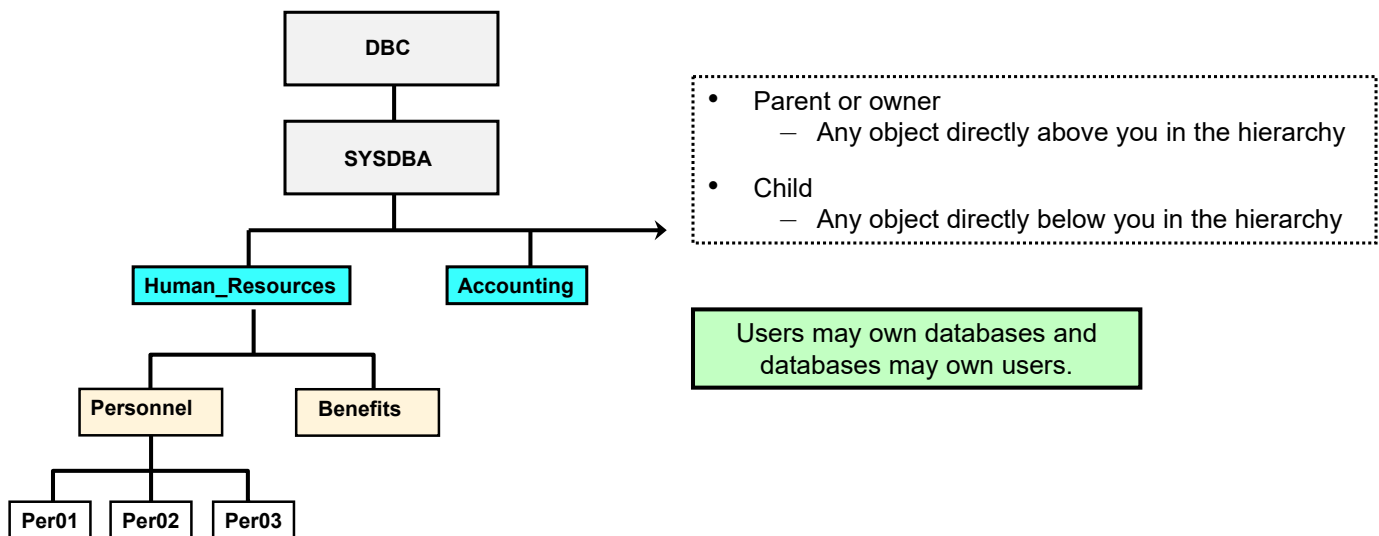
You can name the user anything you would like. A common database administrator name is user SYSDBA. Create the administrative user, and then logon as that user to protect sensitive data in DBC. In addition, change and secure the DBC password.

To ensure perm space is from the administrative user, logon as that user to add other users and databases.

Notes:

- All space in the Teradata Database is owned. No disk space known to the system is unassigned or not owned.
- Think of a user as a database with a password. Both may contain (or "own") tables, views and macros.
- Both users and databases may hold privileges.
- Only users may logon, establish a session with the Teradata Database, and submit requests.

Owners, Parents, and Children



As you define users and databases, a hierarchical relationship among them will evolve.

When you create new objects, you subtract permanent space from the assigned limit of an existing database or user. A database or user that subtracts space from its own permanent space to create a new object becomes the immediate owner of that new object.

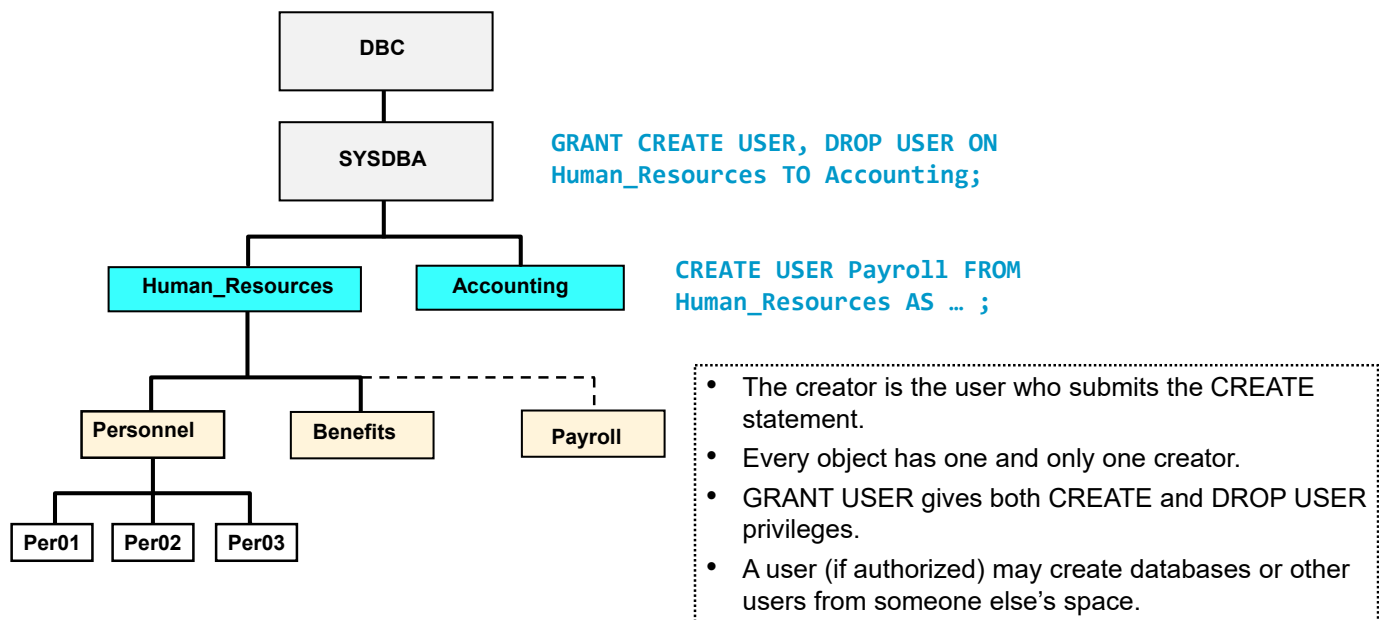
An “owner” or “parent” is any object above you in the hierarchy through which a direct line of ownership is established during the creation process. (Note that you can use the terms owner and parent interchangeably.) A “child” is any object below you in the hierarchy through which a direct line of ownership is established during the creation process.

The term “immediate parent” is sometimes used to describe a database or user just above you in the hierarchy.

Example

The illustration on this slide illustrates Teradata system hierarchy. System user DBC is the owner, or parent, of all the objects in the hierarchy. The administrative user (SYSDBA) is the owner of all objects below it, such as Human Resources, Accounting, Personnel and Benefits. These objects are also children of DBC, since DBC owns SYSDBA.

Creating New Users and Databases



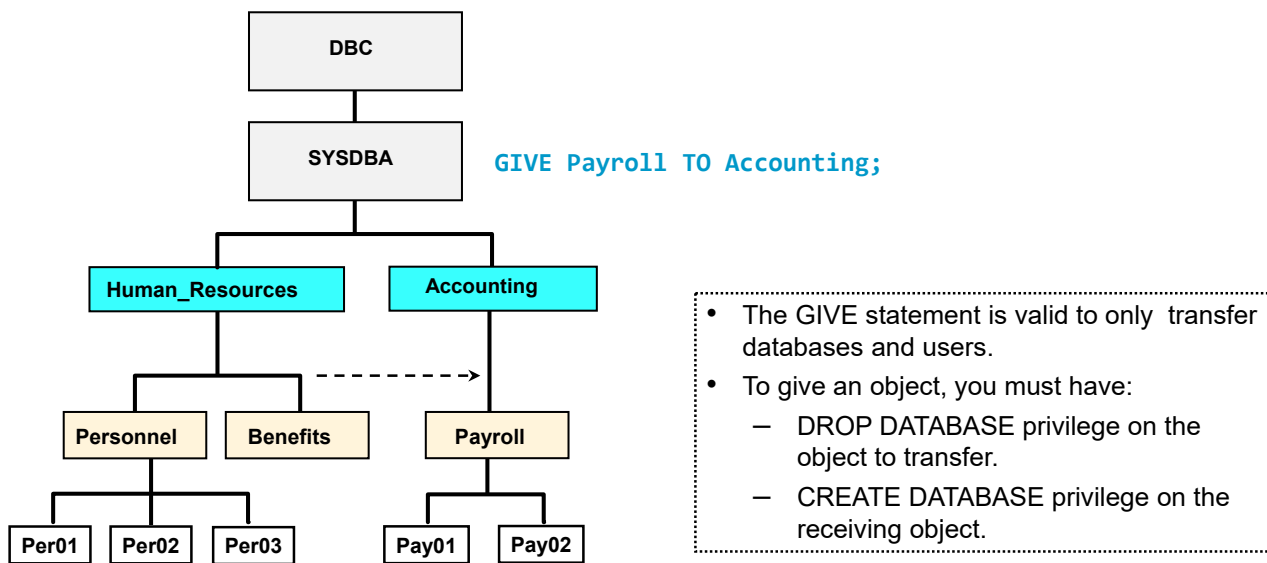
The “creator” of an object is the user who submitted the CREATE statement.

Every object has one and only one creator. If you are the creator of a new space, you automatically have access rights to that space and anything created in it.

Notes:

- While you may be the creator of an object, you are not necessarily the owner of the database or user that contains the object.
- You are the owner of an object if the new object is directly below you in the hierarchy.
- As a creator, you can submit a CREATE statement that adds a new object somewhere else in the hierarchy, assuming you have the appropriate privileges. In this instance, the creator (you) and the owner are two different users or databases.
- If authorized, you may create databases or users FROM someone else's space.
- You can transfer databases and users from one owner to another.

Transfer of Ownership



The GIVE statement transfers a database or user space to a recipient you specify. The GIVE statement also transfers all child databases and users as well as the tables, views and macros owned by the transferred object.

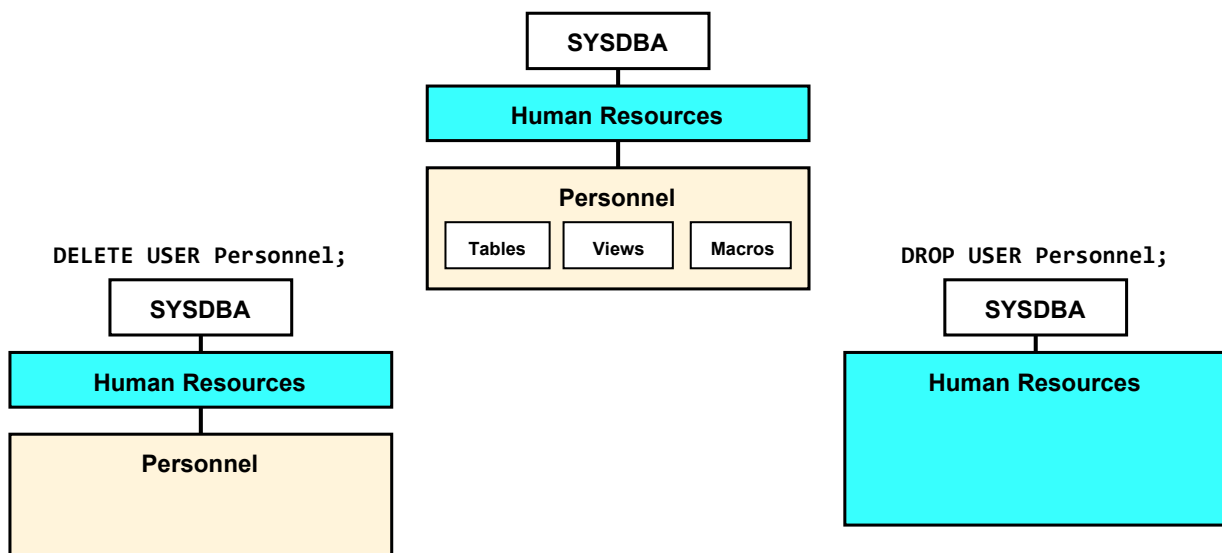
Rules affecting transfer of ownership:

- Use the GIVE statement to transfer databases and users only. (You cannot use the GIVE statement to transfer tables, views, and macros from one database to another.)
- To transfer an object, you must have DROP DATABASE privilege on the object to transfer and CREATE DATABASE privilege on the receiving object.
 - Even though you may be transferring a USER to another user, you need the CREATE DATABASE privilege on the user that is going to get the transferred user. A CREATE USER privilege will not work.
- You cannot give an object to one of its children.
- During a transfer, you transfer all objects the object owns.
- Transfer of ownership affects space ownership and access right privileges. When you transfer an object, the space the object owns is also transferred. The implications of how access rights are affected will be described in more detail later in this course.

Example

In the illustration on this slide, the administrative user, SYSDBA, GIVES the user, Payroll, to Accounting. The new owner, Accounting, acquires ownership of Payroll. All objects that belong to Payroll are transferred as well, including its tables, views, macros and children databases.

DELETE / DROP Statements



`DELETE DATABASE` and `DELETE USER` statements delete all data tables, views, and macros from a database or user. The database or user remains in the Teradata Database as a named object and retains the available space. None of that space is any longer in use. All space used by the deleted objects becomes available as spool space until it is reused as perm space. You must have `DROP DATABASE` or `DROP USER` privilege on the referenced database or user to delete objects from them. The database or user that you are dropping cannot own other databases or users.

DELETE USER Example

The diagram on this slide illustrates a `DELETE USER` statement. Assume the user `Personnel` has three tables: `TB01`, `TB02`, and `TB03`. `Human Resources` logs on to the system and submits the `DELETE USER` statement on user `Personnel`. All tables are deleted from the user space owned by `Personnel`. The `DELETE DATABASE/USER` command does NOT delete a permanent journal, join indexes, or hash indexes.

DROP USER Example

The `DROP DATABASE` or `DROP USER` statement drops empty databases or users only. You must delete all objects associated with the database or user before you can drop the `DATABASE` or `USER`. When you drop a database or user, its perm space is credited to the immediate owner.

This slide illustrates the `DROP USER` statement. `Human Resources` submits the `DROP USER` statement on user `Personnel`. The user `Personnel` is dropped from the hierarchy. The user space that belonged to user `Personnel` is returned to its parent, `Human Resources`.

Teradata Studio – New System

Teradata Studio Administration

- GUI interface to Teradata hierarchy and objects.
- This example shows the default users and databases in a newly initialized Teradata 16.10 system.

Which user initially has the majority of the Perm space?

Database Name	Type	Owner Name	Perm Space	Spool Space	Temp Space
All	User	DBC	0	428,577,440,392	428,577,440,392
Crashdumps	User	DBC	48,075,000,000	428,577,440,392	428,577,440,392
DBC	User	DBC	325,887,324,104	428,577,440,392	428,577,440,392
dbcmngr	Database	DBC	100,000,000	428,577,440,392	428,577,440,392
Default	User	DBC	0	428,577,440,392	428,577,440,392
External_AP	User	DBC	640,000,000	428,577,440,392	428,577,440,392
EXTUSER	User	EXTUSER	0	428,577,440,392	428,577,440,392
LockLogShredder	User	DBC	19,200,000	428,577,440,392	428,577,440,392
PUBLIC	User	DBC	0	428,577,440,392	428,577,440,392
SQLJ	Database	DBC	25,600,000	428,577,440,392	428,577,440,392
Sys_Calendar	User	DBC	102,400,000	428,577,440,392	428,577,440,392
SysAdmin	User	DBC	640,000,000	428,577,440,392	428,577,440,392
SYSBAR	Database	DBC	1,280,000,000	428,577,440,392	428,577,440,392
SYSIDBC	Database	DBC	64,000,000	428,577,440,392	428,577,440,392

Teradata Studio provides an Administration perspective that you can use to perform database administration tasks on the associated Teradata Database system.

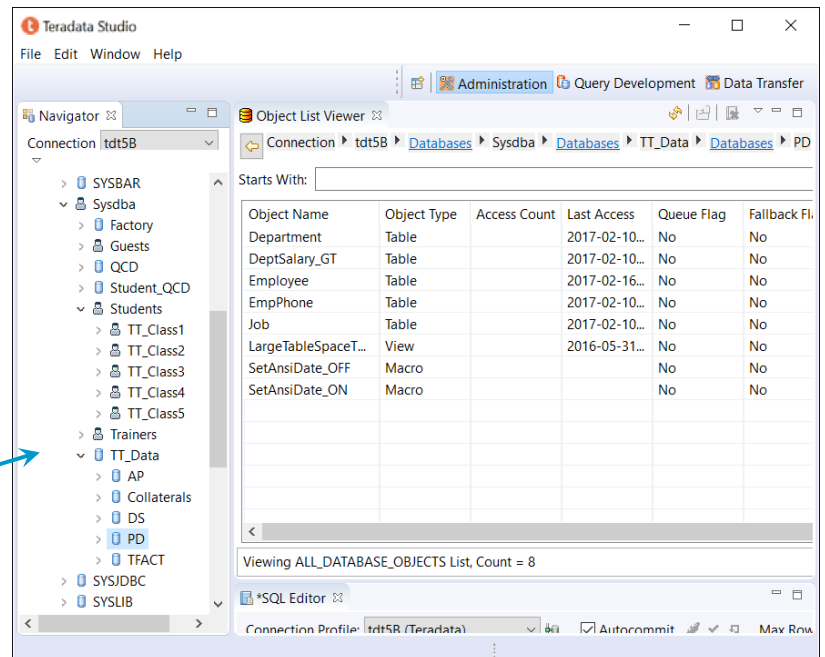
This slide contains an example of the users and databases that exist in a newly initialized Teradata 16.10 system.

Teradata Studio – Hierarchy

Teradata Studio

- This example shows the hierarchy of a Teradata system and the objects in one of the databases.
- This utility also provides drag and drop capabilities.

The hierarchy of the Teradata database is shown in the left pane.



Teradata Studio is a client-based graphical interface for performing database administration, query development, and management tasks on Teradata Databases, Teradata Aster Databases, and Hadoop systems. Teradata Studio is built on the Eclipse Rich Client Platform (RCP) which takes advantage of the RCP framework for building and deploying native GUI applications. It extends the Eclipse Data Tools Platform (DTP) to provide enhancements for accessing objects and data stored in Teradata-supported databases.

Teradata Studio supports the Teradata Unified Data Architecture by seamlessly integrating administration, data management and transfer, and SQL query development across the Teradata, Aster, and Hadoop platforms

Manage Teradata Database, Teradata Aster Database, and Hadoop objects

- Create Databases, Users, Tables, Views, Stored Procedures, User Defined Functions, and User Defined Types objects.
- Modify and DROP Database and User objects.
- Copy Tables, Views, Stored Procedures, Macros, or User-Defined Types definitions to another database, or to another system.
- Drop or rename Tables, Views, Stored Procedures, Macros, or User Defined Functions
- Display information about Database, Users, Tables, Views, Macros, Stored Procedures, User-Defined Functions, User-Defined Types

Additional examples of Teradata Studio displays and capabilities will be shown in various modules throughout this course.

Summary

- Initially, system user DBC owns all space in the Teradata Database except that owned by system users and databases.
- The database administrator should create a special administrative user containing most of the space available which will become the owner of all administrator-defined application databases and users.
- Everyone directly higher in the hierarchy is a *parent* or *owner*. Everyone directly lower in the hierarchy is a *child*.
- Every object has one and only one creator. The creator is the user who executes the CREATE statement.
- The GIVE statement enables you to transfer a database or user. The following privileges are necessary:
 - DROP DATABASE on the given object.
 - CREATE DATABASE on the receiving object.
- You cannot DROP databases or users that own objects (tables, views, macros, journals or children databases/users).
- The Studio Administrator perspective provides an easy-to-use Windows-based graphical interface to the Teradata Database Data Dictionary.

This slide summarizes some important concepts regarding this module.



Module 1: Review Questions

1. True or False: You should use system user DBC to create application users and databases.
2. True or False: A database or user can have multiple Owners, but only one Creator.
3. True or False: An Owner and a Parent are two different terms that mean the same thing.
4. True or False: An Owner and a Creator are two different terms that mean the same thing.
5. True or False: An administrative user (e.g., Sysdba) will never have more permanent space than DBC.
6. True or False: The GIVE statement transfers a database or user space to a recipient you specify. It does not automatically transfer all child databases.

Check your understanding of the concepts discussed in this module by completing the review questions as directed by your instructor.



Module 2: Teradata Studio

Teradata Vantage Administration

Copyright © 2007–2023 by Teradata. All Rights Reserved.

Objectives

After completing this module, you will be able to:

- Define a Data Source Connection for Teradata
- Submit SQL using Teradata Studio or Teradata Studio Express
- Utilize Explorer Tree to simplify creation of queries
- Use Teradata Studio or Teradata Studio Express to access a table with a LOB

Teradata SQL Tools

Teradata Studio Express is also an information discovery tool that retrieves data from Teradata, Hadoop, and Aster databases using a JDBC interface.

Teradata Studio extends the Studio Express capabilities by allowing you to also perform database administration tasks and has replaced Teradata Administrator.

Both of these tools provide the following capabilities:

- Provide access via a graphical user interface.
- Submit SQL to Teradata – support DDL, DML, and DCL commands
- Provides a Data Source Explorer Tree to easily view database objects.
- Permits retrieval of previously used queries (History).
- Studio also provides a database administration perspective to facilitate the management of roles, profiles, etc.
- This module is intended to provide an introduction to establishing Teradata connections and submitting SQL using Teradata Studio or Teradata Studio Express.

Note: Teradata SQL Assistant was another information discovery tool to retrieve data from Teradata using either an ODBC or Teradata .NET interface.

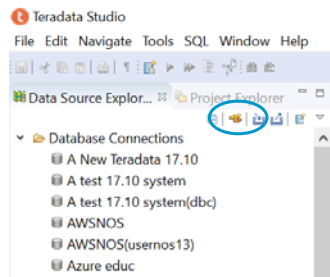
Teradata SQL Assistant is an information discovery tool designed for the Windows operating system (e.g., Windows 10). Teradata SQL Assistant retrieves data from any ODBC-compliant database server. The data can then be manipulated and stored on the desktop PC.

Teradata Studio Express is the base level product. It is client-based graphical interface and provides an information discovery tool for retrieving and displaying data from your Vantage Advanced SQL Engine systems. Its main use is for running SQL statements, displaying result set data, and storing SQL execution history.

Teradata Studio, in addition to the features provided in Teradata Studio Express, includes administration dialogs and wizards. Its main use is for administrators to create and administer database objects. Teradata Studio is a tool for DBAs, Data Scientists, and Power Users. Teradata Studio also provides a client-based graphical interface for performing SQL, database administrative/management tasks on Vantage Advanced SQL Engines, Teradata Aster Databases, and Hadoop systems.

Teradata Studio is built on the Eclipse Rich Client Platform (RCP) which takes advantage of the RCP framework for building and deploying native GUI applications. It extends the Eclipse Data Tools Platform (DTP) to provide enhancements for accessing objects and data stored in Teradata-supported databases. Teradata Plug-in for Eclipse, in addition to the features provided in Teradata Studio, plugs into your existing Eclipse IDE and provides wizards to help users create Java database objects such as Java Stored Procedures and Java User Defined Functions. Teradata Plug-in for Eclipse is a tool for developers creating Java applications and database objects.

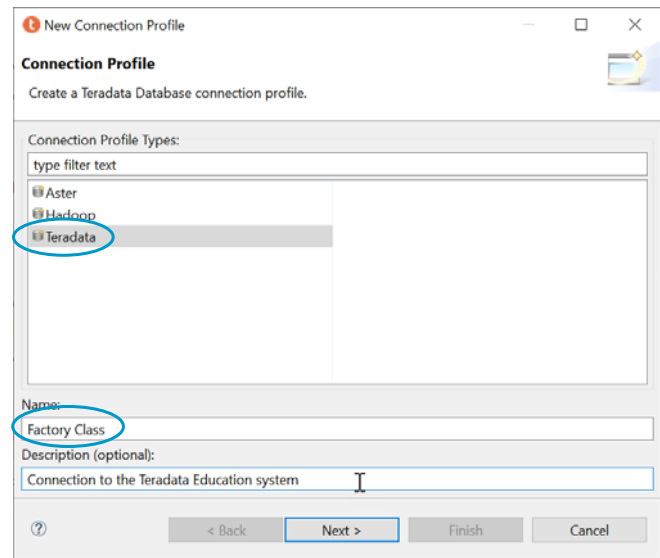
Defining a Connection



When you first install Teradata Studio, you will get prompted for a new connection.

If not, then Click “New Connection Profile” icon.

Select **Teradata Database** as Connection Profile type, enter a name and then click on **Next** button.



Defining a Connection

To begin using Teradata Studio, create a Connection Profile for your database.

Connection to any database must be made through a JDBC (Java-based data access technology) connection. The Connection Profile details the JDBC information required to connect to an instance of a Vantage Advanced SQL Engine.

Defining a Connection (cont.)

Enter Database Server name or IP address.

Then enter User Name and Password.

Default is ANSI mode so click **Add** and select TMODE as JDBC property and select value **TERA** for Teradata Mode and click **OK**. Also include the JDBC properties of CHARSET=UTF8 and TCP=KEEPALIVE.

Use the **Test Connection** option to verify the connection.

Click **OK** to complete this dialog. Then click **Finish** to complete connection setup.

When connecting to the Vantage Advanced SQL Engine, determine which mode you will be using. Studio defaults to ANSI mode rather than Teradata mode. Therefore, you may need to change it in the data connection profile.

Use the Test Connection option to verify the connection with the username and password. If you choose to save the password, do this after testing the connection.

Defining a Connection (Transcend)

Enter Database Server name (tdprd).

Then enter User Name, password, and specify **Authentication Mechanism as LDAP**.

Default is ANSI mode so click **Add** and select TMODE as JDBC property and select value **TERA** for Teradata Mode and click **OK**.

Also include the JDBC properties of CHARSET=UTF8 and TCP=KEEPALIVE.

Use the **Test Connection** option to verify the connection.

Click **OK** to complete this dialog. Then click **Finish** to complete connection setup.

To create a Transcend connection, specify the following:

Database Server Name:	tdprd2.td.teradata.com
User Name:	your Quicklook ID
Password:	your Teradata password
Authentication Mechanism:	LDAP

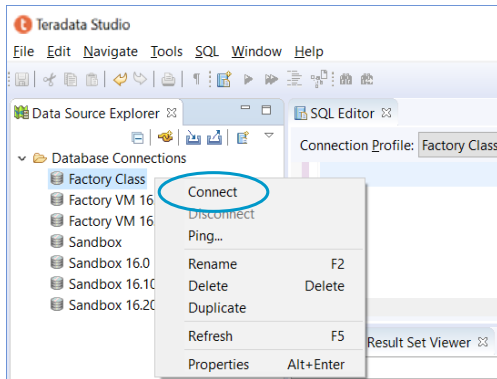
Also specify the following JDC properties:

CHARSET = UTF8
TCP = KEEPALIVE
TMODE = TERA

Use the Test Connection option to verify the connection with the username and password.

Although there is another Transcend system (tdprd.td.teradata.com), you should use the connection to "tdprd2" for training and education purposes. You have additional privileges on "tdprd2" that do not exist on the "tdprd" system.

Connecting to a Teradata System

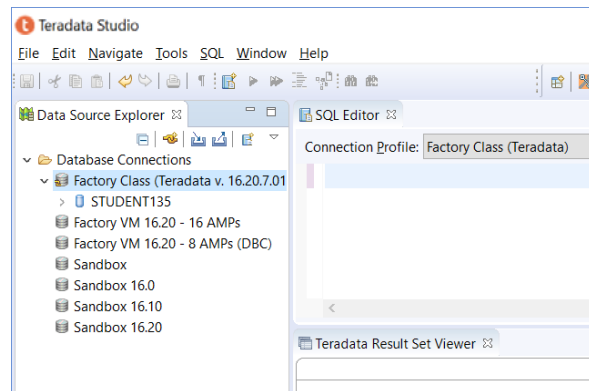


You should connect as part of an initial setup.

If NOT then:

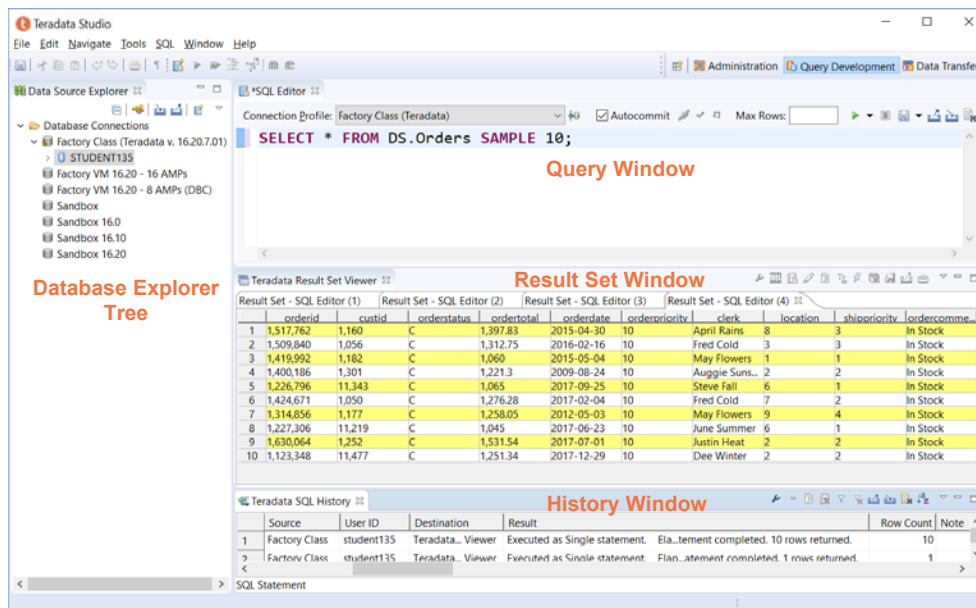
1. Right Click on the connection name and SELECT Connect.
or Double-click on the connection name.
2. Re-enter password

This display should occur when connected.



Connecting to a data source is the equivalent of “logging on” with Teradata Studio. You may choose from any previously defined data connection profile. Just right mouse click on the defined data connection and select “Connect”.

Query Development: Main Window



You can customize the look and feel of the interface by moving view panels within the display window.

Main Window

The **SQL Editor** (query window) provides an integrated environment that enables you to develop SQL statements for execution. The editor enables you to validate SQL syntax, save files to a project or export to system files, and execute all SQL in a file or execute the SQL as individual statements. You must specify a Connection Profile before you can execute an SQL query statement. The SQL Editor displays when you open an SQL/DDI file or when you select the Query Development Perspective. Double-clicking an SQL/DDI file in the Project Explorer opens that file in the editor.

The **Result Set Viewer** (result set window) provides an enhanced view of the result set, including options to hide columns, filter on column values, find column values, copy data to the clipboard, format cell data, and switch between grid and text view. The result set is displayed in its own view with its own toolbar.

You can change preferences to customize the view by, for example, displaying alternating row background colors, enabling or disabling row numbers, column headers, table grid lines, and thousand separators on numeric columns.

The **SQL History View** (history window) contains the results when executing SQL statements and running database routines, user-defined functions, and triggers.

The **Data Source Explorer** (database explorer tree) enables you to connect to a specific database system using a Connection Profile and then view and manage the database objects.

Data Source Explorer Tree

To change the Explorer Tree display settings, use the following menus

Window → Preferences

For class purposes,

Check

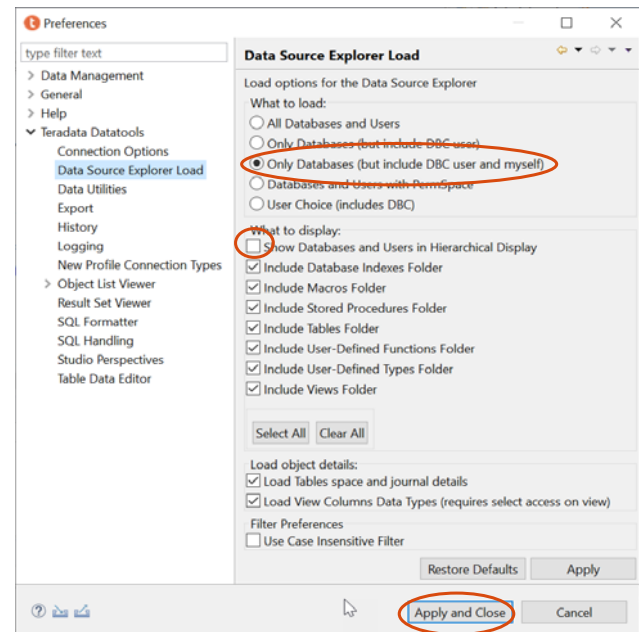
Only Databases (but include DBC user and myself)

Uncheck

Show Databases and Users in Hierarchical Display

Note:

Unchecking "Show Databases and Users in Hierarchical Display" can speed up dialogs such as Tools→Create→Database, User and Macro.



The **Data Source Explorer Load Preferences** enables you to configure your preference settings for loading database and user objects and to determine what folders are shown in the hierarchical display. You can set preferences to:

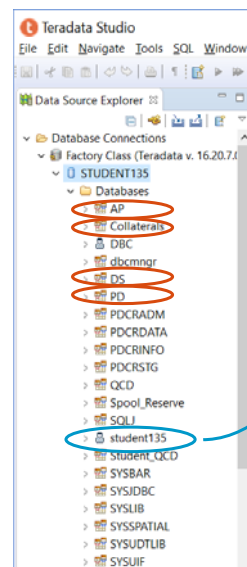
- Load databases and users:
 - All databases and all users
 - Load only databases and certain users
 - Load all databases and users with PermSpace configured
- Configure hierarchical display to show:
 - All databases and all users
 - Only certain folders you specify

Data Source Explorer Tree (cont.)

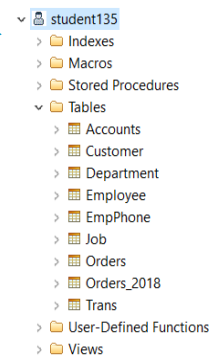
- The Database Explorer Tree displays an alphabetical listing of databases and objects of the connected user.
- To refresh a database, right-click on the database/user name and select "Refresh".

For this course, some of the users/databases we will be using are the following:

- TRNG_AP
- TRNG_Collaterals
- DBC
- TRNG_DS
- TRNG_PD
- TRAINEE_XX



To expand an item/object, click on the icon or double-click on the object name.

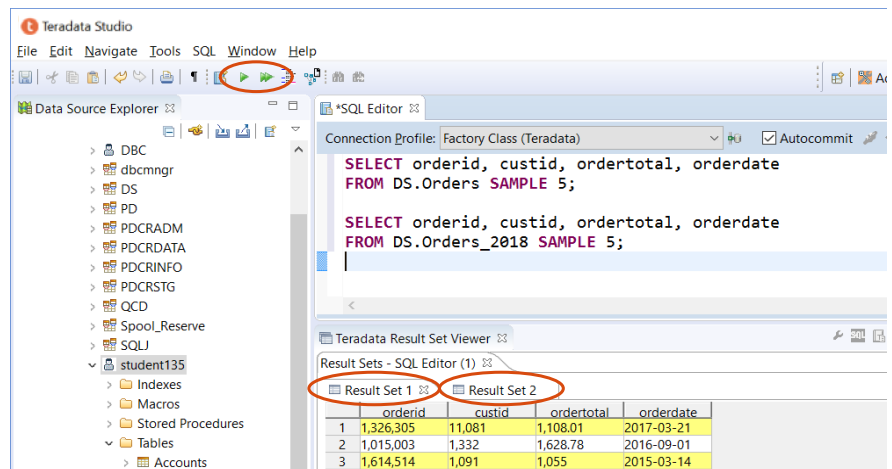


The database objects display in a navigation pane in which you can explore the structure of a database's data and manage stored data sets. You can view a list of tables, macros, stored procedures, views, and other objects in a selected database, and view additional details about each object type. For example, from the Databases node you can expand a Tables node to view and list of tables in a database, and the names and types of columns in the table.

Creating and Executing a Query

1. Create a query in the Query Window.
2. To execute a query use either the “Execute as Individual Statements” icon / button (or F5) **serially** executes all statements in the query window.

Or the “Execute All” drop down selection executes all statements in the query window in a single multi-statement request. These queries are effectively executed **in parallel**.



The SQL Editor provides a scratchpad for you to enter your SQL statements. Simply type the query text into the query window. It is not necessary to add a semi-colon at the end of a command, unless you are entering multiple commands in a single window. As an option, you can enable the auto activation of code assist by selecting it from the SQL menu on the toolbar. If this option is enabled, as you type your SQL statement, code assist will pop up annotations showing a list of expected keywords. If a database name is entered followed by a dot, the content assist will display the list of tables from your database for you to select from. By default, auto activation of code assist is disabled.

Parameterized queries can also be created in the SQL Editor. The query contains parameters indicated by question marks. When the query is executed, the user is prompted for the parameter value(s) in the Configure Parameters dialog. Named parameters (appending backslash, '\', <parameter name>) allows the user to specify a name for the parameter. When the Configure Parameters dialog is presented, the parameter names will be used to reference the parameters.

Creating statements (single and multi-queries)

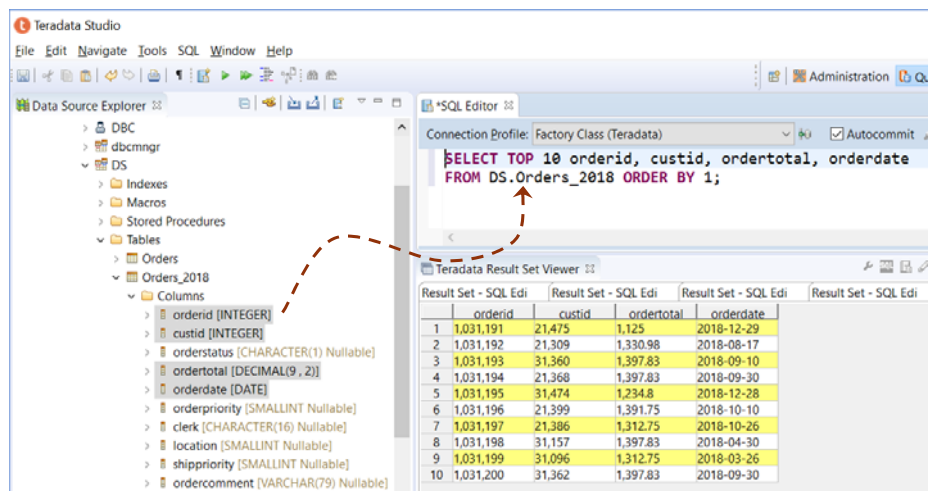
When right clicking in the SQL Editor window, a SQL Editor Context menu appears with options to execute the SQL statement(s) in your SQL file.

An Autocommit checkbox is provided in the SQL Editor to allow the user to toggle Autocommit on or off. By default, Autocommit is ON. If the option is turned off, then the Commit and Rollback options are enabled for the user to manually control the commit of their SQL statements to the database or rollback if an error occurs.

Dragging Object Names to the Query Window

You can click and drag object(s) from the Database Explorer tree to the Query window.

- By default, qualified names are automatically included. Use the Window > Preferences > Teradata Datatools Preferences > SQL Handling window to turn off this option if desired.
- The **Shift** and **Ctrl** keys can also be used to select multiple objects in the Database Explorer tree for the purpose of dragging multiple objects to the Query Window.



You can drag object names from the Database Explorer tree to the Query pane.

Click and drag the object from the Explorer tree to the Query pane. The name of the object appears in the Query window.

Use the Ctrl key to add a comma after the object name when it is dragged to the Query Window.

Dragging Multiple Objects

Use the Shift and Ctrl keys to select more than one object from the Database Explorer Tree that can be dragged to the Query window.

- Use the Ctrl key to select additional objects.
- Use the Shift key to select a range of objects.

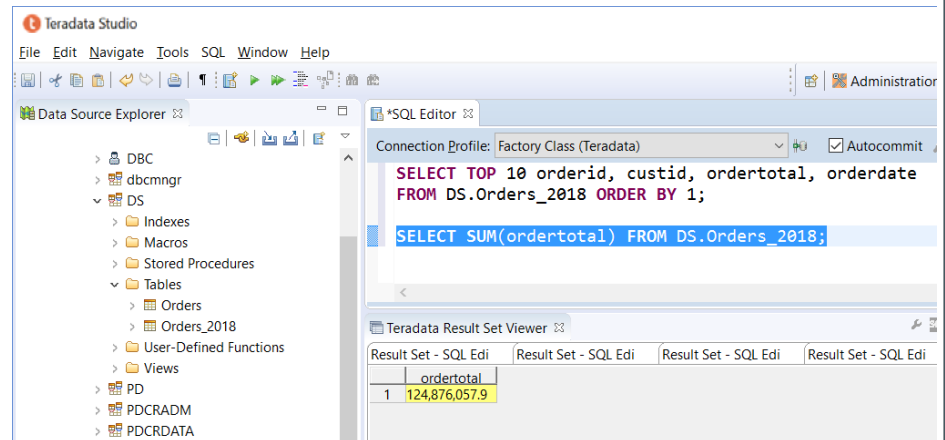
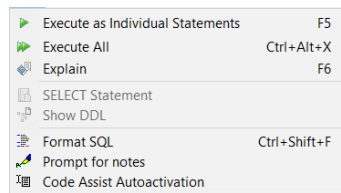
Query Options

To submit any part of a query:

- ▶ Highlight the text in the query window and execute.

To format a query, select the **Format SQL** option from either the main Edit menu or SQL menu. (Ctrl+Shift+F)

There is also a **Prompt for notes** and **Code Assist Autoactivation** options from the SQL menu.



To submit any part of any query

1. Highlight the SQL text you want to run.
2. Click the execute play button.
3. Or Right mouse click and select "Execute selected text".
4. Or Press Keys "Alt-x".

Formatting a Query

The query formatting feature adds line breaks and indentation before certain keywords, making SQL that comes from automatic code generators or other sources more readable.

To Format a Query

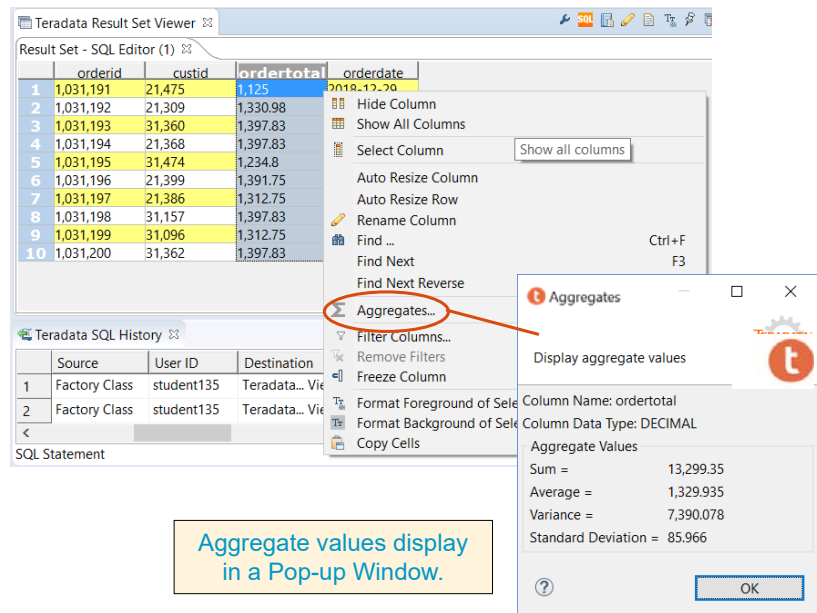
1. Ensure a statement exists in the Query window.
2. Do one of the following:
 - Select menu SQL > Format SQL
 - Or Press Ctrl+Shift+F
 - Or Select menu Edit > Format Query

Indentation

When you press the **Enter** key, the new line will automatically indent to the same level as the line above.

Viewing Query Results

- The Teradata Result Set Viewer window is a table that displays the results.
- The output table has many options by right mouse clicking in Result Set:
 - Aggregates of selected numeric values (see below causes a pop-up window with result)
 - Auto Resize Columns and Rows
 - Data can be filtered using the Filter Columns... option (funnel) 



Aggregate values display in a Pop-up Window.

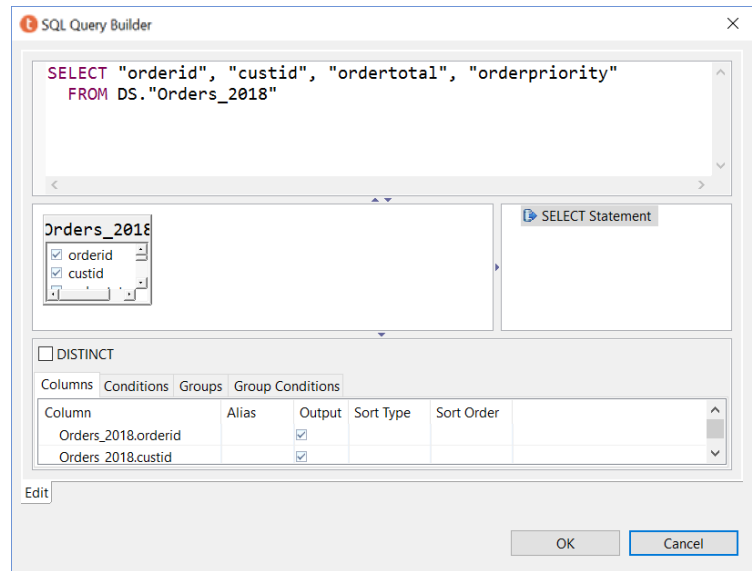
The Result Set Viewer displays the result set or parameter data from executing SQL statements and running database routines, such as stored procedures, macros, user-defined functions, and triggers.

The Result Set Viewer provides an enhanced display of the Result Set data. It displays the result sets as tabs within the Teradata Result Set Viewer, allowing the user to select rows or cells from the Result Set data table grid.

SQL Query Builder

SQL Query Builder provides the user with the ability to visually create SQL commands.

1. To launch just Right click in SQL Editor and Select **Edit in SQL Query Builder** (Alt+q).
2. Right mouse click to **Add tables**. Once you have multiple tables you can right click and create a join if you wish.



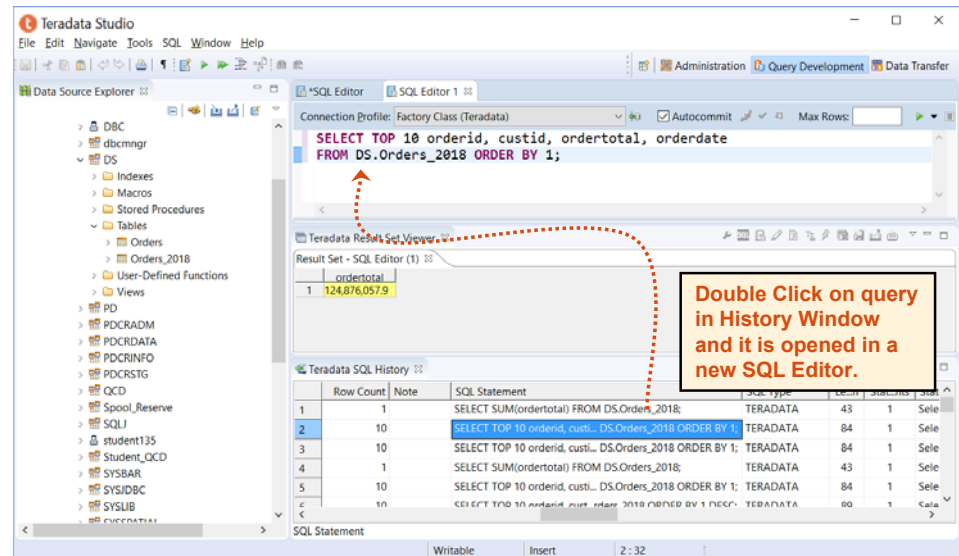
Teradata Studio provides a graphical interface for creating SQL statements for Teradata, referred to as the **SQL Query Builder**. You can edit an existing statement using the SQL Query Builder by selecting the statement within the SQL Editor and choosing the 'Edit in SQL Query Builder...' menu option.

Teradata SQL History Window

Historical record of the submitted SQL queries with timestamps, results status, record counts, etc. History data can be exported and shared with other Teradata Studio users.

Note:

If you close the view and need to re-open it go to [Window > Show View > Other > Teradata > Teradata SQL History](#).



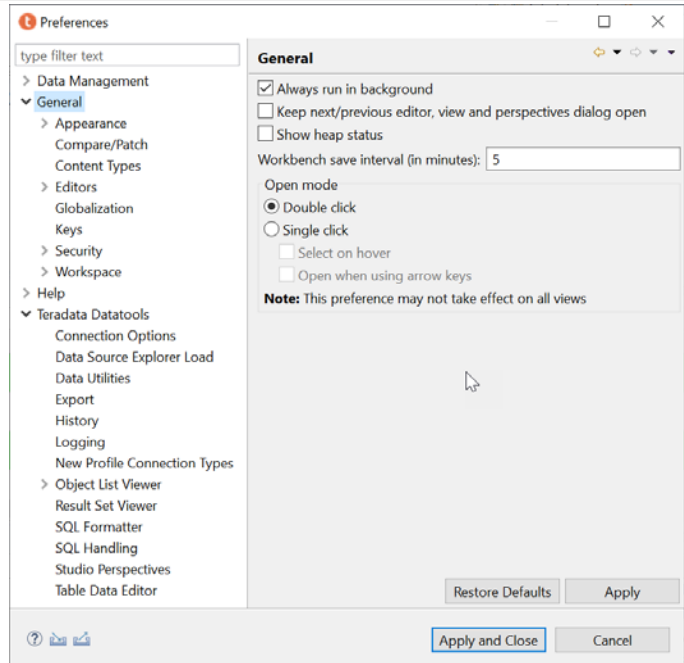
The SQL History View provides an enhanced display of the SQL History information. As SQL statements are executed from the SQL Editor or functions are run from the Data Source Explorer, an entry is placed in the SQL History. The SQL History view displays the data in a grid format, allowing the user to select rows and cells.

The SQL History provides extensive information about the SQL execution, such as execution elapsed time, DBMS time, and fetch time, database server, row count, parameter display for macros and stored procedures, SQL statement, and SQL statement type. It provides options to copy cells or rows, edit the 'Note' or SQL for the SQL History entry, search for result history data, and re-execute SQL statements. Choosing the 'Edit SQL' option will open a SQL Editor file with the selected history entry's SQL statement. This allows users to further fine tune their SQL statements.

General Options

General options (Window > Preferences > General):

- **Appearance:** Customize the appearance of the display for colors, fonts, and label decorations.
- **Compare/Patch:** Customize the display when comparing text files.
- **Content Types:** Add content types and file associations.
- **Editors:** Customize editors, file associations, content types, and editor appearance.
- **Globalization:** Customizes global settings for Unicode and text/graphic support.
- **Keys:** Customize the keyboard bindings.
- **Security:** Customize the secure storage.
- **Workspace:** Change the text file encoding for the workspace.



To set general program preferences:

1. Select the menu **Window > Preferences > General**.

Connecting to Multiple Data Sources

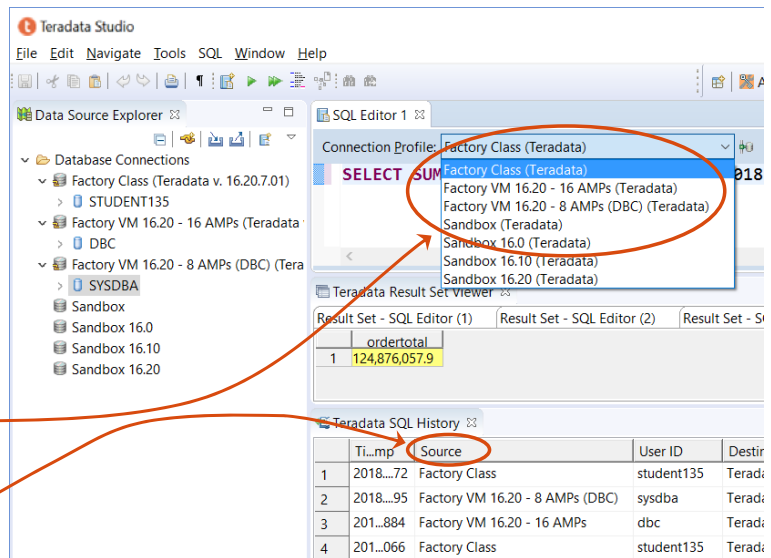
You can connect to multiple systems via the Database Connections.

Connections have been made to three systems:

- Factory Class
- Factory VM – 16 AMPs
- Factory VM – 8 AMPs

You can select which system to run query against

History includes the Source name for queries.

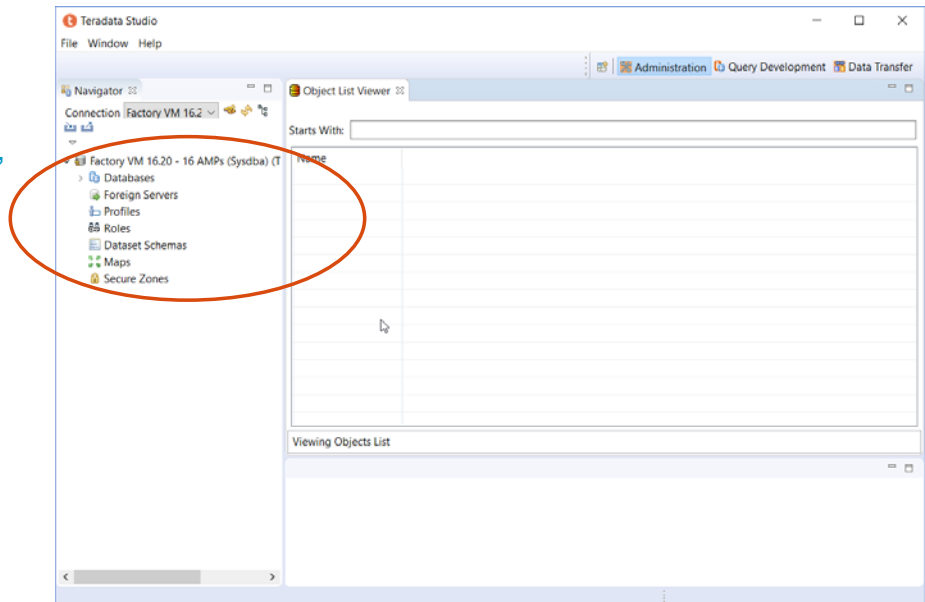


You can connect to multiple data sources via the data connection profiles. Just right mouse click the profile and select connect. Then in the SQL editor you can pick the system you want to run the query against.

This example show three connects to three different systems.

Teradata Studio Administration – Initial Display

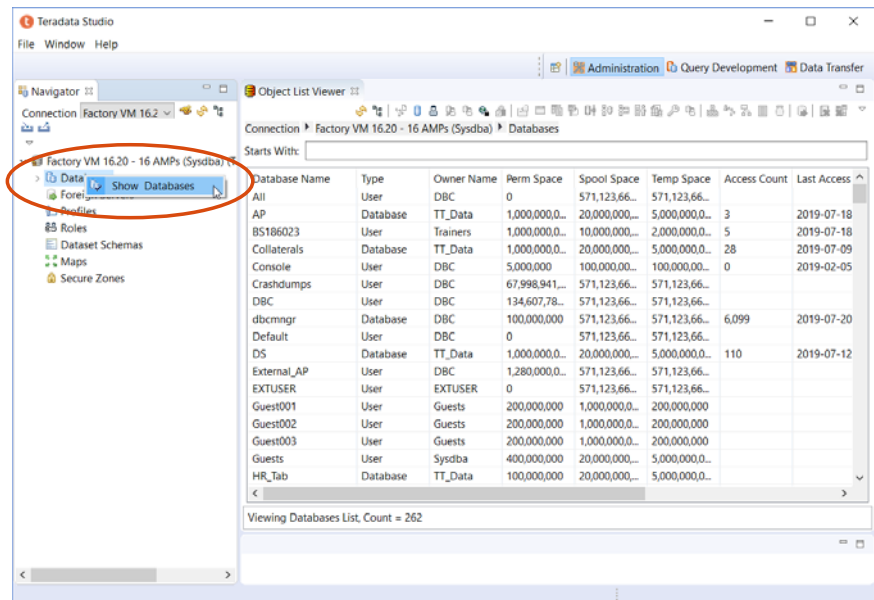
The Administration perspective provides the ability to manage databases, foreign servers, profiles, roles, and secure zones.



This slide illustrates the initial display using the Administration perspective of Teradata Studio.

Teradata Studio Administration –Show Databases

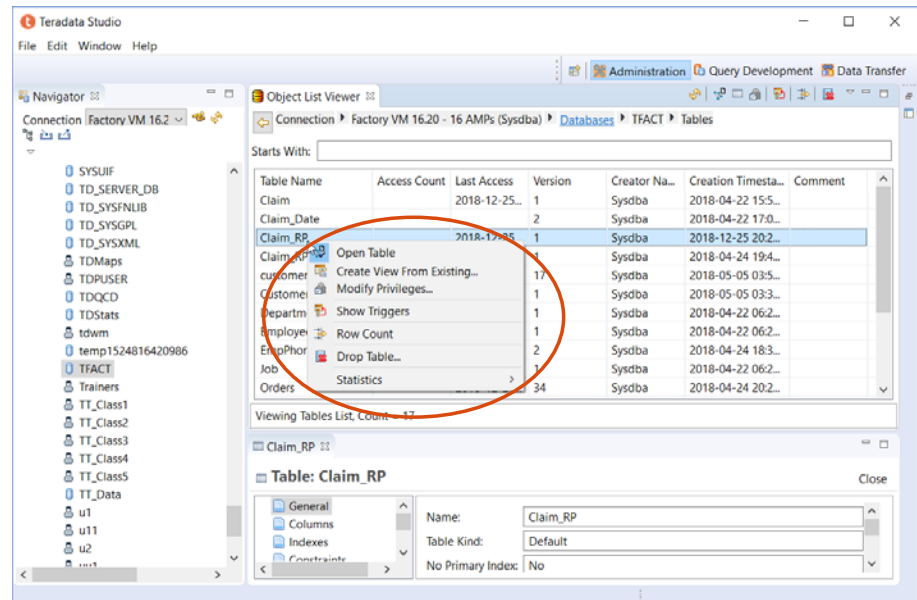
The Show Databases option provides information about space used by users and databases.



This slide illustrates the initial display using the Show Databases option in the Administration perspective of Teradata Studio.

Teradata Studio Administration – Object List Viewer

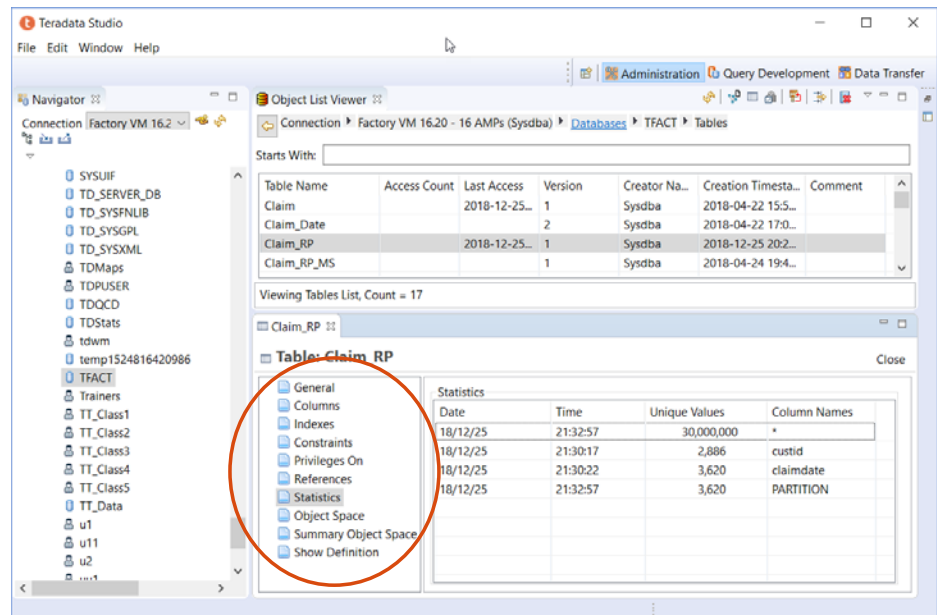
By double-clicking on an object (e.g., table), additional details are provided.



This slide illustrates the initial display using the Object List Viewer option in the Administration perspective of Teradata Studio.

Teradata Studio Administration – Object Details

By double-clicking on an object (e.g., table), additional details are provided.



This slide illustrates the initial display using the Table Details option in the Administration perspective of Teradata Studio.

Teradata Studio Summary

Characteristics of both Teradata Studio and Teradata Studio Express include:

- Client-based graphical interface to submit SQL for the Teradata Advanced SQL Engine.
- Provides a Database Explorer Tree to easily view database objects.
- Supports DDL, DML and DCL commands.
 - SQL Query Builder feature allows for easy creation of SQL statements.
- Provides the retrieval of previously used queries (History).
- Saves information about previous query result sets.
- Provides both import and export capabilities to files on a PC.
- Provides database administration capabilities.

The Teradata Studio or Teradata Studio Express tools can be of great value to you.



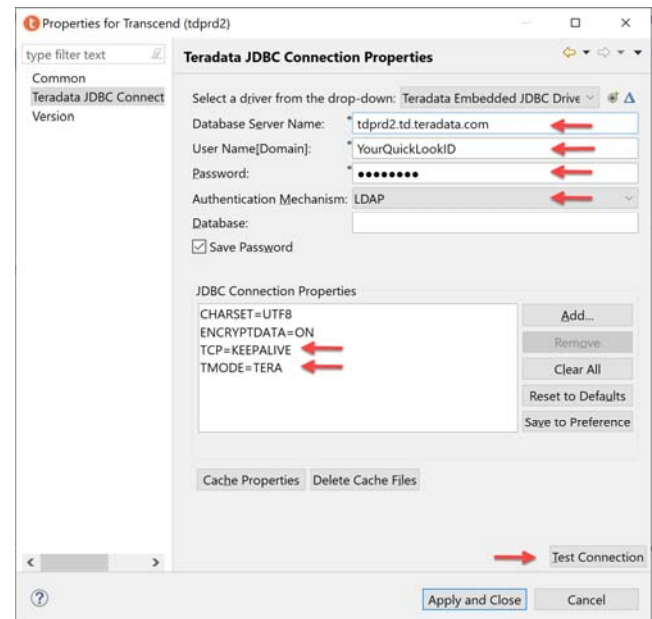
Module 2: Review Questions

1. Which data interface is available with Teradata Studio?
 - a. CLIV2
 - b. JDBC
 - c. ODBC
 - d. Teradata .Net
2. Which piece of query information is not available in the History Window for Teradata Studio?
 - a. User name
 - b. Query band
 - c. Elapsed time
 - d. Data source name
 - e. Number of rows returned
3. What is a technique to execute multiple statements as a multi-statement request in Teradata Studio? _____

Check your understanding of the concepts discussed in this module by completing the review questions as directed by your instructor.

If Using Transcend, Hands-on Setup (1 of 2)

Teradata Vantage system: you can use **tdprd2.td.teradata.com** (preferred for training purposes).



To create a Transcend connection, specify the following:

Database Server Name:	tdprd2.td.teradata.com
User Name:	your Quicklook ID
Password:	your Teradata password
Authentication Mechanism:	LDAP

Also specify the following JDC properties:

```
CHARSET = UTF8
TCP = KEEPALIVE
TMODE = TERA
```

Use the Test Connection option to verify the connection with the username and password.

Although there is another Transcend system (tdprd.td.teradata.com), you should use the connection to "tdprd2" for training and education purposes. You have additional privileges on "tdprd2" that do not exist on the "tdprd" system.

If Using Transcend, Hands-on Setup (2 of 2)

- ▼ TRNG_TDU_FACTORY
 - > Indexes
 - > Macros
 - > Stored Procedures
 - > Tables
 - > User-Defined Functions
 - > Views
- > TRNG_AP
- > TRNG_Collaterals
- > TRNG_DS
- > TRNG_PD

On the Transcend system (tdprd2.td.teradata.com), **use the following databases and CREATE TABLE statements to set up your user environment.**

```
CREATE SET TABLE Accounts      AS TRNG_AP.Accounts      WITH NO DATA;
CREATE SET TABLE Customer     AS TRNG_AP.Customer     WITH NO DATA;
CREATE SET TABLE Trans        AS TRNG_AP.Trans        WITH NO DATA;
CREATE SET TABLE Department   AS TRNG_PD.Department   WITH NO DATA;
CREATE SET TABLE Employee     AS TRNG_PD.Employee     WITH NO DATA;
CREATE SET TABLE Job          AS TRNG_PD.Job          WITH NO DATA;
CREATE SET TABLE EmpPhone     AS TRNG_PD.EmpPhone     WITH NO DATA;
CREATE SET TABLE Orders       AS TRNG_DS.Orders       WITH NO DATA;
CREATE SET TABLE Orders_2021  AS TRNG_DS.Orders_2021  WITH NO DATA;
```

Copy and paste the CREATE TABLE statements into your Studio Query window to establish the set of empty tables used in lab exercises in the Teradata database classes.



Lab Exercise 2-1 (1 of 4)

Purpose

In this lab, you will use either Teradata Studio or Teradata Studio Express to define a data source and execute some simple SQL commands.

What you need

Teradata Studio or Teradata Studio Express installed on the laptop or PC.

Tasks

1. If this has not been already defined, create a connection to a Teradata system based on the following:

Complete the dialog box with the following information:

Name – Factory

Description – Teradata Training for your name

Name or IP Address – _____ (supplied by instructor)

Username – _____ (supplied by instructor)

Password – fill in your password and test the connection

If your connection ping succeeds, **before connecting, verify the following options are set properly.**

CHARSET = UTF8

ENCRYPTDATA = ON

TMODE = TERA

TCP = KEEPALIVE

Check your understanding of the concepts discussed in this module by completing the lab exercise as directed by your instructor.



Lab Exercise 2-1 (2 of 4)

2. Connect to the data source you just created and logon with your username and password.
3. Use the Window > Preferences > Teradata Datatools tab, for class exercises, you may want to set the following options as indicated:

Connection Options

Teradata – JDBC Connection Properties:

CHARSET = UTF8, TMODE = Tera, TCP = KEEPALIVE

Data Source Explorer Load Preferences

Check – Only Databases (but include DBC and myself)

Uncheck – Show Databases and Users in Hierarchical Display

Result Set Viewer

Check – Display alternate Result Set rows in color – choose a color

Check – Display Column Titles rather than Column Names

Check – Reuse Result Set Viewer Window

SQL Handling

Uncheck – Qualify Names for drag and drop in SQL Editor

4. If the Explorer Tree pane is not visible, mouse click > next to your login name and then Databases
Find the following databases: TRNG_AP, TRNG_DS, TRNG_PD, TRNG_Collaterals
5. Using the Explorer Tree, view the table objects in your database.



Lab Exercise 2-1 (3 of 4)

6. Using the Query Window, execute the following query.

```
CREATE TABLE Orders_1Q2021 AS Orders_2021 WITH NO DATA;
```

Does the new table object appear in the table object list? ____ If not, "refresh" the database.

7. Execute the following query to populate Orders_1Q2021 with orders for January of 2021.

```
INSERT INTO Orders_1Q2021
SELECT * FROM TRNG_DS.Orders_2021
WHERE orderdate BETWEEN DATE '2021-01-01' AND DATE '2021-01-31';
```

How many rows are in Orders_1Q2021? ____

8. Using the History window, recall the query from step #7 and modify it to add orders for February and March of 2021 into Orders_1Q2021.

How many rows are now in Orders_1Q2021? ____

9. Execute the following query by using the drag and drop object feature of the SQL tool you are using.

```
SELECT custid, SUM (ordertotal)
FROM Orders_1Q2021
WHERE custid = 21072
GROUP BY 1
ORDER BY 1;
```

What is the SUM of the orders (ordertotal) in the 1Q2021 for custid = 21072? ____

If using the TRANSCEND (tdprd) system for your lab exercises, the base Orders and Orders_2021 tables are in the TRNG_DS database.

```
CREATE TABLE Orders_2021 AS TRNG_DS.Orders_2021 WITH DATA;
```

```
CREATE TABLE Orders_1Q2021 AS Orders_2021 WITH NO DATA;
```

```
INSERT INTO Orders_1Q2021 AS Orders_2021
WHERE orderdate BETWEEN DATE '2021-01-01' AND DATE '2021-01-31';
```

Use the following SQL to determine a count of rows in a table.

```
SELECT COUNT(*) FROM tablename;
```

Step 7 Hint: With the Orders_1Q2021 table and January 2021 orders, the count should be 12,021.

Step 8 Hint: with the Orders_1Q2021 table and first quarter 2021, the count should be 36,075.

Step 9 Hint: The sum of Orders (ordertotal) for custid = 21072 should be 67,125.66.



Lab Exercise 2-1 (4 of 4)

10. Create a view (that utilizes an Access Lock) named "Orders_Jan2021_v" on the Orders_1Q2021 table that includes the following columns and only includes orders for the January of 2021.
- orderid, custid, ordertotal, orderdate
- SELECT the total count of orders from "Orders_Jan2021_v".
- How many orders are available using this view? _____
11. Create a parameterized macro named "Orderid_m" which accepts an orderid as a value, accesses the view, and returns a single order from "Orders_Jan2021_v".
- Execute the macro to provide order details for orders 550013 and 551550.
- What is the ordertotal for orderid #550013 _____
- What is the ordertotal for orderid #551550? _____
12. (Optional) Use the Collaterals database to access the Photos table to display various JPG files.
- Execute the following: `SELECT * FROM Collaterals.Photos ORDER BY 1;`
- Note: if using Transcend, `SELECT * FROM TRNG_Collaterals.Photos ORDER BY 1;`
- Note: Set the file type to JPG and check the option "Display as picture in Answerset".
- What is shown in photo #7? _____

Use the following SQL to create a view.

```
CREATE VIEW viewname AS
LOCKING ROW FOR ACCESS
SELECT column1, column2
FROM table_or_view_name
WHERE condition;
```

To create a parameterized macro, you can use the following SQL:

```
CREATE MACRO orderid_m (oid_value INTEGER)
AS (SELECT * FROM table_or_view_name
WHERE Orderid = :oid_value;);
```

Use the following SQL to execute a parameterized macro.

```
EXEC orderid_m (550013);
EXEC orderid_m (551550);
```



Module 3: Index Considerations

Teradata Vantage Administration

Copyright © 2007–2023 by Teradata. All Rights Reserved.

Objectives

After completing this module, you will be able to:

- Create single-level partitioned table using date ranges and CURRENT_DATE option
- Use ALTER TABLE to add and drop ranges from a row partitioned table
- CREATE and populate NoPI tables with the HASH BY options for distribution purposes
- CREATE a column and row partitioned table using the PRIMARY AMP option
- Describe how value-ordered and hash-ordered secondary indexes might be used to improve performance

Primary Index Choice Criteria

ACCESS	Maximize one-AMP operations: • Choose the column(s) most frequently used for access. • Consider both join and value access.
DISTRIBUTION	Optimize parallel processing: • Choose the column(s) that provides good distribution.
VOLATILITY	Reduce maintenance resource overhead (I/O): • Choose the column(s) with stable data values.

Note: Data distribution has to be balanced with Access usage in choosing a PI.

General Notes:

A good logical model identifies the Primary Key for each table or entity.

- Do not assume that the Primary Key will become the Primary Index.
 - It is common for many tables in a database to have a Primary Index that is different than the Primary Key.
- This module will first cover PI tables then cover details of the NO PRIMARY INDEX option. The general assumption in this course is that tables will have a PI.

There are three Primary Index Choice Criteria: **Access Demographics**, **Distribution Demographics**, and **Volatility**.

- Access demographics are the first of three Primary Index Choice Criteria. Access columns are those that would appear (with a value) in a WHERE clause in an SQL statement. Choose the column most frequently used for access to maximize the number of one-AMP operations.
- Distribution demographics are the second of the Primary Index Choice Criteria. The more unique the index, the better the distribution. Optimizing distribution optimizes parallel processing.
- In choosing a Primary Index, there is a trade-off between the issues of access and distribution. The most desirable situation is to find a PI candidate that has good access and good distribution. Many times, however, index candidates offer great access and poor distribution or vice versa. When this occurs, the physical designer must balance these two qualities to make the best choice for the index.
- The third of the Primary Index Choice Criteria is volatility, or how often the data values will change. The Primary Index should not be very volatile. Any changes to Primary Index values may result in heavy I/O overhead, as the rows themselves may have to be moved from one AMP to another. Choose a column with stable data values.

Partitioning with RANGE_N – Example

For example, partition a Claim table by "claimdate".

```
CREATE TABLE Claim_RP
( claimid INTEGER NOT NULL
, custid  INTEGER NOT NULL
, claimdate DATE NOT NULL
... )
PRIMARY INDEX (claimid)
```

```
PARTITION BY RANGE_N (claimdate BETWEEN
DATE '2012-01-01' AND DATE '2021-12-31'
EACH INTERVAL '1' DAY, NO RANGE);
```

This table has 3654 logically defined partitions. As INSERTs place new rows into the Claim table, the date is evaluated and the rows are placed into the appropriate partitions.

```
INSERT INTO Claim_RP VALUES (1028111,1009,'2012-01-13',...); placed in partition #13
INSERT INTO Claim_RP VALUES (24286111,1020,'2021-01-13',...); placed in partition #3300
INSERT INTO Claim_RP VALUES (45451111,1025,'2022-01-01',...); placed in partition #3654
INSERT INTO Claim_RP VALUES (100039,1009, NULL,...); Error 3811 – NOT NULL violation
```

If the table did not have the NO RANGE partition defined, then the following error occurs:

```
INSERT INTO Claim_RP VALUES (45451111,1025,'2022-01-01',...); (5728 – Partitioning violation)
```

Note: claimid must be defined as a NUPI because claimdate is not part of PI.

One of most common partitioning expression is to use RANGE_N partitioning to partition the table based on a group of dates (e.g., month partitions). A range is defined by a starting boundary and an optional ending boundary. If an ending boundary is not specified, the range is defined by its starting boundary, inclusively, up to but not including the starting boundary of the next range. The list of ranges must specify ranges in increasing order, where the ending boundary of a range is less than the starting boundary of the next range.

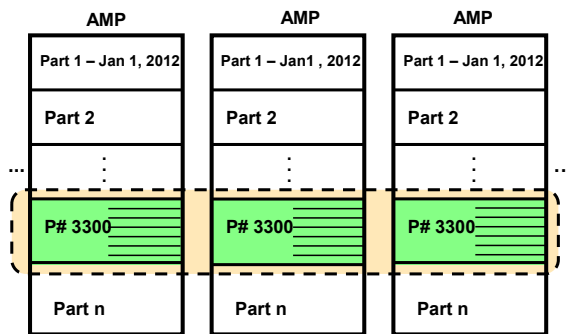
RANGE_N limitations include:

- Multiple test values are not allowed in a RANGE_N function.
- Range value and range size in a RANGE_N function must be constant.
- Ascending ranges only and ranges must not overlap with each other.

For example, this CREATE TABLE statement can be used to establish the monthly partitioning.

```
CREATE SET TABLE Claim_RP
(claimid          INTEGER      NOT NULL
, custid          INTEGER      NOT NULL
, claimdate       DATE         NOT NULL
:
PRIMARY INDEX (claimid)
PARTITION BY RANGE_N (claimdate
BETWEEN          DATE '2012-01-01' AND DATE '2021-12-31'
EACH INTERVAL '1' MONTH, NO RANGE);
```

Access Using Partitioned Data – Example (cont.)



QUERY – row partitioned

```
SELECT *
FROM Claim_RP
WHERE claimdate = DATE '2021-01-13';
```

Explain plan

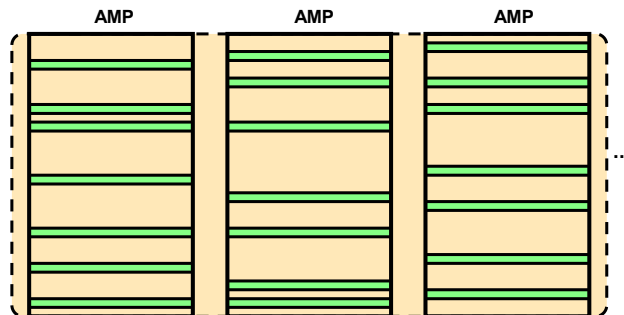
ALL-AMPs – Single Partition Scan with a Partition Read Lock. This date (partition) has 17,293 rows. EXPLAIN estimated cost – 0.04 sec.

QUERY – not row partitioned

```
SELECT *
FROM Claim
WHERE claimdate = DATE '2021-01-13';
```

Explain plan

ALL-AMPs – Full Table Scan with a Table Read Lock – table has 30M rows. EXPLAIN estimated cost – 16.11 sec.



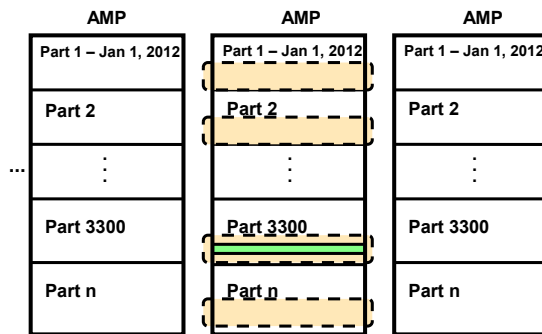
Both Claim_RP and Claim tables have 30,000,000 rows. Statistics for custid and claimdate were collected on both tables. In addition, claimid and PARTITION statistics were collected on the Claim_RP table.

The EXPLAIN text for these queries is shown below.

```
EXPLAIN SELECT *
FROM Claim_RP
WHERE claimdate = DATE '2021-01-13';
```

- 1) First, we lock TFACT.Claim_RP in TD_MAP1 for read on a reserved RowHash in a single partition to prevent global deadlock.
 - 2) Next, **we lock TFACT.Claim_RP in TD_MAP1 for read on a single partition.**
 - 3) We do an all-AMPs RETRIEVE step in TD_MAP1 from a single partition of TFACT.Claim_RP with a condition of ("TFACT.Claim_RP.claimdate = DATE '2021-01-13'") with a residual condition of ("TFACT.Claim_RP.claimdate = DATE '2021-01-13'") into Spool 1 (group_amps), which is built locally on the AMPs. The size of Spool 1 is estimated with high confidence to be 17,293 rows (4,029,269 bytes). The estimated time for this step is 0.04 seconds.
 - 4) Finally, we send out an END TRANSACTION step to all AMPs involved in processing the request.
- > The contents of Spool 1 are sent back to the user as the result of statement 1. **The total estimated time is 0.04 seconds.**

Access Using Primary Index – Example (cont.)



QUERY – row partitioned

```
SELECT *
FROM Claim_RP
WHERE claimid = 1028111;
```

Explain plan

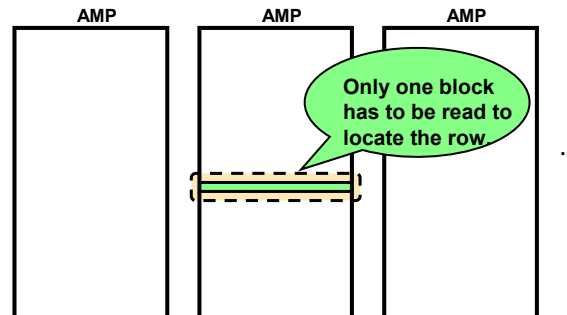
One AMP – All Partitions are probed
EXPLAIN estimated cost – 0.82 sec.

QUERY – not row partitioned

```
SELECT *
FROM Claim
WHERE claimid = 1028111;
```

Explain plan

One AMP – UPI Access
EXPLAIN estimated cost – 0.00 sec.



```
EXPLAIN SELECT * FROM Claim_RP
WHERE claimid = 1028111;
```

- 1) First, we do a single-AMP RETRIEVE step from all partitions of TFACT.Claim_RP by way of the primary index "TFACT.Claim_RP.claimid = 1028111" with a residual condition of ("TFACT.Claim_RP.claimid = 1028111") into Spool 1 (one-amp), which is built locally on that AMP. The size of Spool 1 is estimated with high confidence to be 1 row (233 bytes). The estimated time for this step is 0.82 seconds.
- > The contents of Spool 1 are sent back to the user as the result of statement 1. **The total estimated time is 0.82 seconds.**

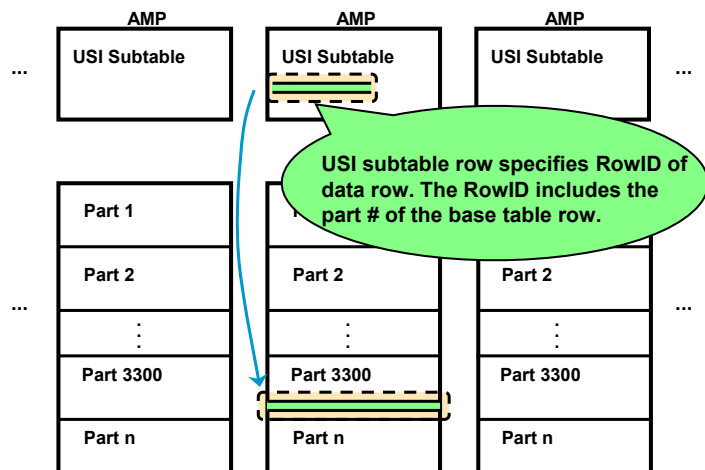
The table named Claim is similar to Claim_RP except it does not have a Partitioned Primary Index, but does have "claimid" as a UPI.

```
EXPLAIN SELECT * FROM Claim
WHERE claimid = 1028111;
```

- 1) First, we do a **single-AMP RETRIEVE step** from TFACT.Claim by way of the unique primary index "TFACT.Claim.claimid = 1028111" with no residual conditions. The estimated time for this step is 0.00 seconds.
- > The row is sent directly back to the user as the result of statement 1. **The total estimated time is 0.00 seconds.**

Place a USI on NUPI – Example (cont.)

- If the partitioning column(s) are not part of the Primary Index, the Primary Index cannot be unique (e.g., claimdate is not part of the PI).
- To maintain uniqueness on the Primary Index, you can create a USI on the PI (e.g., claimid). This is a two-AMP operation.



```
CREATE UNIQUE INDEX (claimid) ON
Claim_RP;
```

```
SELECT *
FROM   Claim_RP
WHERE  claimid = 1028111;
```

USI Considerations:

- Eliminate partition probing
- Row-hash locks
- 2-AMP operation
- Can only be used if values in PI column(s) are unique
- Will maintain uniqueness

```
CREATE UNIQUE INDEX (claimid) ON Claim_RP;
```

```
EXPLAIN  SELECT *
        FROM   Claim_RP
        WHERE  claimid = 1028111;
```

- 1) First, we do a **two-AMP RETRIEVE** step in TD_MAP1 from TFACT.Claim_RP by way of **unique index # 4** "TFACT.Claim_RP.claimid = 1028111" with no residual conditions. The estimated time for this step is 0.00 seconds.
- > The row is sent directly back to the user as the result of statement 1. The total estimated time is 0.00 seconds.

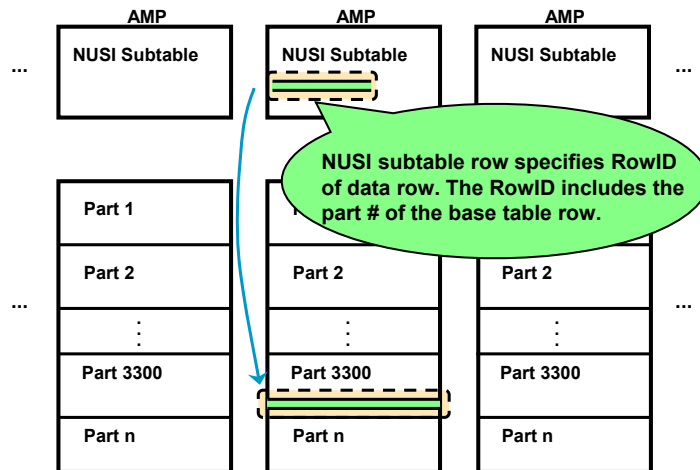
As an alternative, the SELECT can include the Primary Index values and the partitioning information. If you knew that the claimid of 1028111 occurred on 2021-01-13, you could include both claimid and claimdate in the query. This allows the PE to build a request that has the AMP scan a specific partition.

```
EXPLAIN  SELECT *
        FROM   Claim_RP
        WHERE  claimid = 1028111
        AND    claimdate = DATE '2021-01-13';
```

However, the user may not know the claimdate in order to include it in the query.

Place a NUSI on NUPI – Example (cont.)

- You can optionally create a NUSI on the same columns as the Primary Index (e.g., claimid). The PI may be unique or not.
- Optimizer generates a plan for a [single-AMP NUSI access](#) with [row-hash locking](#) (instead of table-level locking).



```
CREATE INDEX (claimid) ON Claim_RP;
```

```
SELECT *
FROM   Claim_RP
WHERE  claimid = 1028111;
```

NUSI Considerations:

- Eliminate partition probing
- Row-hash locks
- 1-AMP operation
- Can be used with unique or non-unique PI columns
- Must be equality condition
- NUSI Single-AMP operation are only supported on row partitioned tables

You can use a NUSI on the same columns that make up the PI and actually get a single-AMP access operation. This feature only applies to a NUSI created on the same columns as a PI on row partitioned table. Additionally, instead of table level locks (typical NUSI), row hash locks will be used.

Reasons to choose a NUSI for your PI may include:

- The primary index is non-unique (cannot use a USI) and you need faster access than do a look-up or a probe into multiple partitions on a single AMP.
- TPT Update (MultiLoad protocol) can be used to load a table with a NUSI without using the MultiLoadX protocol.
- The access time for a USI and NUSI will be similar (each will access a subtable block) – however, the USI is a 2-AMP operation and requires BYNET message passing.

```
CREATE INDEX (claimid) ON Claim_RP;
EXPLAIN  SELECT * FROM Claim_RP
        WHERE claimid = 1028111;
```

- 1) First, we do a **single-AMP RETRIEVE step** from TFACT.Claim_RP by way of **index # 4** "TFACT.Claim_RP.claimid = 1028111" with no residual conditions into Spool 1 (one-amp), which is built locally on that AMP. The size of Spool 1 is estimated with high confidence to be 1 row (233 bytes). The estimated time for this step is 0.00 seconds.
- > The contents of Spool 1 are sent back to the user as the result of statement 1. The total estimated time is 0.00 seconds.

ALTER TABLE – NO RANGE is not Defined

Assume a table is partitioned:

```
PARTITION BY RANGE_N (salesdate BETWEEN
DATE '2012-01-01' AND DATE '2021-12-31' EACH INTERVAL '1' MONTH);
```

To drop/add partitions and **DELETE** the old data:

```
ALTER TABLE Sales MODIFY PRIMARY INDEX
DROP RANGE BETWEEN DATE '2012-01-01' AND DATE '2012-12-31'
ADD RANGE BETWEEN DATE '2022-01-01' AND DATE '2022-12-31' EACH INTERVAL '1' MONTH
WITH DELETE;
```

To drop/add partitions and **COPY** the old data to another table:

```
ALTER TABLE Sales MODIFY
DROP RANGE BETWEEN DATE '2012-01-01' AND DATE '2012-12-31'
ADD RANGE BETWEEN DATE '2022-01-01' AND DATE '2022-12-31' EACH INTERVAL '1' MONTH
WITH INSERT INTO SalesHistory;
```

Notes assuming NO RANGE is not defined:

- The WITH DELETE or WITH INSERT syntax is required if the partitions have data.
- Dropping partition ranges means the data in the dropped partitions is also deleted from the table.
- The MODIFY syntax does not require you to also include the words PRIMARY INDEX.
- The SalesHistory table must already exist when using the WITH INSERT option.
- SalesHistory may optionally be partitioned; recommended for the table to be empty and have a matching PI as the Sales table for performance.

The DROP RANGE option is used to drop a range set from the RANGE_N function on which the partitioning expression for the table is based. You can only drop ranges if the partitioning expression for the table is derived only from a RANGE_N function.

You can drop empty partitions without specifying the WITH DELETE or WITH INSERT option. However, if the partitions to be dropped have data, you must include either the WITH DELETE or WITH INSERT syntax.

Some of the ALTER TABLE statement options include:

DROP RANGE WHERE *conditional_expression* – a conditional partitioning expression used to drop a range set from the RANGE_N function on which the partitioning expression for the table is based.

You can only drop ranges if the partitioning expression for the table is derived only from a RANGE_N function.

You must base *conditional_partitioning_expression* on the system-derived PARTITION column.

ALTER TABLE – NO RANGE is Defined

Assume a table is partitioned:

```
PARTITION BY RANGE_N (salesdate BETWEEN
DATE '2012-01-01' AND DATE '2021-12-31' EACH INTERVAL '1' MONTH, NO RANGE);
```

To drop/add partitions from the table definition:

```
ALTER TABLE Sales MODIFY
DROP RANGE BETWEEN DATE '2012-01-01' AND DATE '2012-12-31'
ADD RANGE BETWEEN DATE '2022-01-01' AND DATE '2022-12-31' EACH INTERVAL '1' MONTH;
```

If WITH DELETE or WITH INSERT is included, it is effectively ignored.

```
ALTER TABLE Sales MODIFY
DROP RANGE BETWEEN DATE '2012-01-01' AND DATE '2012-12-31'
ADD RANGE BETWEEN DATE '2022-01-01' AND DATE '2022-12-31' EACH INTERVAL '1' MONTH
WITH INSERT INTO SalesHistory;
```

Notes assuming NO RANGE is defined:

- If dropped partitions have data, then the data is moved to the NO RANGE partition.
- If WITH DELETE or WITH INSERT syntax is included, it is effectively ignored.
- The WITH DELETE or WITH INSERT syntax is not needed and is confusing if included.
- If new partitions are added, the NO RANGE partition is checked and rows that apply to the newly added partitions are moved from NO RANGE into the new partitions.
- If you want to delete the data, use SQL DELETE before using ALTER TABLE.
- General recommendation is to not use NO RANGE unless it is specifically needed.

When using the ALTER TABLE to drop existing ranges, if the table has NO RANGE defined, it is not necessary to include the WITH DELETE or WITH INSERT syntax.

If NO RANGE is defined and the WITH DELETE or WITH INSERT syntax is used, the WITH DELETE or WITH INSERT is effectively ignored.

This simply means the rows are not deleted or copied to another table.

In this case (table has a NO RANGE partition), rows are copied from dropped partitions into the NO RANGE partition. If new partitions are added, the NO RANGE partition is checked and rows that apply to the newly added partitions are moved from NO RANGE into the new partitions.

Note: When dropping Ranges, the word "Interval" is optional.

```
ALTER TABLE Sales MODIFY
DROP RANGE BETWEEN DATE '2012-01-01' AND DATE '2012-12-31'
EACH INTERVAL '1' MONTH
ADD RANGE BETWEEN DATE '2022-01-01' AND DATE '2022-12-31'
EACH INTERVAL '1' MONTH;
```

You can add the NO RANGE and/or UNKNOWN partitions to an already partitioned table.

```
ALTER TABLE Sales MODIFY ADD RANGE NO RANGE OR UNKNOWN;
```

ALTER TABLE – Example

In this example, NO RANGE has not been defined for the Sales tables.

```
SELECT PARTITION, COUNT(*)
FROM Sales
GROUP BY 1
ORDER BY 1;
```

Result:

<u>PARTITION</u>	<u>COUNT(*)</u>
1	10850
2	10150
:	:
119	14800
120	14950

Part #1 - January 2012

Part #120 - December 2021

```
ALTER TABLE Sales MODIFY
DROP RANGE BETWEEN DATE '2012-01-01' AND DATE '2012-12-31'
ADD RANGE BETWEEN DATE '2021-01-01' AND DATE '2021-12-31' EACH INTERVAL '1' MONTH
WITH DELETE;
```

```
SELECT PARTITION, COUNT(*)
FROM Sales
GROUP BY 1
ORDER BY 1;
```

Result:

<u>PARTITION</u>	<u>COUNT(*)</u>
1	12400
2	11200
:	:
107	14800
108	14950

Part #1 - January 2013

Part #108 - December 2021

This page contains notes on the internal implementation. The important point is to understand that dropping or adding partitions (to the “ends” of an already partitioned table with data) does not cause changes to the internal partitioning numbers that are currently implemented. The logical partition numbers change, but the internal partition numbers do not. For this reason, dropping or adding partitions does not cause an undue amount of work.

The following table shows the same result as the slide, but also identifies the internal partition #'s as allocated.

<u>PARTITION</u>	<u>Count(*)</u>	
1	10850	internally mapped to partition #3 (Jan, 2012)
2	10150	internally mapped to partition #4 (Feb, 2012)
:	:	:
13	12400	internally mapped to partition #15 (Jan, 2012)
14	11200	internally mapped to partition #16 (Feb, 2012)
:	:	:

After the 12 partitions were dropped (as in this example), the logical and internal partition numbers are:

<u>PARTITION</u>	<u>Count(*)</u>	
1	12400	internally mapped to partition #15 (Jan, 2013)
2	11200	internally mapped to partition #16 (Feb, 2013)
:	:	:

ALTER TABLE TO CURRENT

This option allows you to periodically resolve the CURRENT_DATE (or DATE) and CURRENT_TIMESTAMP of a row partitioned table to their current values.

Benefits include:

- You do not have to change the partitioning expression to update the value for CURRENT_DATE or CURRENT_TIMESTAMP.
- To partition on a **new** CURRENT_DATE or CURRENT_TIMESTAMP value, simply submit an ALTER TABLE TO CURRENT request.

Considerations:

- The **ALTER TABLE TO CURRENT** request causes the CURRENT_DATE and/or CURRENT_TIMESTAMP to effectively repartition the rows in the table.
- If RANGE_N specifies CURRENT_DATE or CURRENT_TIMESTAMP in a partitioning expression, **you cannot use ALTER TABLE to add or drop ranges for the table**. You must use the ALTER TABLE TO CURRENT statement to achieve this function.

```
ALTER TABLE table_name  
join_index_name TO CURRENT WITH DELETE  
INSERT [INTO] save_table;
```

You can specify CURRENT_DATE and CURRENT_TIMESTAMP functions in a partitioned primary index for base tables and join indexes. Teradata also provides a new option with the ALTER TABLE statement to modify a partitioned table that has been defined with a moving CURRENT_DATE (or DATE) or moving CURRENT_TIMESTAMP. This new option is called ALTER TABLE TO CURRENT.

When you specify CURRENT_DATE and CURRENT_TIMESTAMP as part of a partitioning expression for a partitioned table, these functions resolve to the date and timestamp when you define the PPI. To partition on a new CURRENT_DATE or CURRENT_TIMESTAMP value, submit an ALTER TABLE TO CURRENT request.

The ALTER TABLE TO CURRENT syntax is shown on this slide.

The WITH DELETE option is used to delete any row whose partition number evaluates to a value outside the valid range of partitions.

The WITH INSERT [INTO] *save_table* option is used to insert any row whose partition number evaluates to a value outside the valid range of partitions into the table specified by *save_table*.

The WITH DELETE or INSERT INTO *save_table* clause is sometimes referred to as a null partition handler. You cannot specify a null partition handler for a join index. *Save_table* and the table being altered must be different tables with different names.

ALTER TABLE TO CURRENT – Example

This example creates a partitioning expression to maintain the last eight years of historical data, data for the current year, and data for one future year for a total of 10 years.

Assuming the current year is 2021, an equivalent definition using `CURRENT_DATE` is:

```
CREATE TABLE Sales
(   storeid          INTEGER NOT NULL,
    itemid           INTEGER NOT NULL,
    salesdate        DATE FORMAT 'YYYY-MM-DD',
    :
PRIMARY INDEX (storeid, itemid)
PARTITION BY RANGE_N (salesdate BETWEEN
DATE '2013-01-01' AND DATE '2022-12-31' EACH INTERVAL '1' MONTH);
```

```
PRIMARY INDEX (storeid, itemid)
PARTITION BY RANGE_N
(salesdate BETWEEN
CAST ((EXTRACT(YEAR FROM CURRENT_DATE) - 8 ) || '-01-01' AS DATE) AND
CAST ((EXTRACT(YEAR FROM CURRENT_DATE) + 1 ) || '-12-31' AS DATE)
EACH INTERVAL '1' MONTH );
```

In 2022, execute `ALTER TABLE Sales TO CURRENT WITH DELETE;`

- Teradata deletes the rows from 2013 because they are no longer needed.
- To view the date when the table was last resolved, the `DBC.PartitioningConstraintsV` provides columns named "ResolvedCurrent_Date" and "ResolvedCurrent_TimeStamp".

This slide assumes partitioning begins on a year boundary. Using this example, consideration for the two options are:

- With hard-coded dates in the `CREATE TABLE` statement, you must compute the new dates and specify them explicitly in the `ADD RANGE` clause of the request. This requires manual intervention every year you submit the request.
- With `CURRENT_DATE` in the `CREATE TABLE` statement, you can schedule the `ALTER TABLE TO CURRENT` request to be submitted annually or simply execute the next year. This request rolls the partition window forward by efficiently dropping and adding partitions.

As a result of executing the `ALTER TABLE TO CURRENT WITH DELETE`, Teradata deletes the rows from the table because they are no longer needed. An alternative partitioning expression is:

```
PARTITION BY RANGE_N (salesdate BETWEEN
CAST ((EXTRACT(YEAR FROM CURRENT_DATE) - 8 - 1900) * 10000 + 0101) AS
DATE) AND
CAST ((EXTRACT(YEAR FROM CURRENT_DATE) + 1 - 1900) * 10000 + 1231) AS
DATE)
EACH INTERVAL '1' MONTH);
```

Note: If you specify multiple ranges using a `DATE` or `CURRENT_DATE` function in one of the ranges, and then later reconcile the partitioning, the range specified using `CURRENT_DATE` might overlap one of the existing ranges. If so, reconciliation aborts the request and returns an error to the requestor. If this happens, you must recreate the table with a new partitioning expression based on `DATE` or `CURRENT_DATE`. Because of this, you should design a partitioning expression that uses a `DATE` or `CURRENT_DATE` function in one of its ranges with care.

ADD Partitions Option – Example

The ADD option allows you to define additional partitions for a sub-level – these partitions can be added at a later time via the ALTER table command.

```
CREATE TABLE Claim_MLRP2
( claimid      INTEGER NOT NULL,
  custid       INTEGER NOT NULL,
  claimdate    DATE     NOT NULL FORMAT 'YYYY-MM-DD',
  :
  district     SMALLINT,
  :
  PRIMARY INDEX (claimid)
  PARTITION BY (
    RANGE_N (claimdate BETWEEN DATE '2012-01-01' AND DATE '2021-12-31'
             EACH INTERVAL '1' MONTH, NO RANGE),
    RANGE_N (district BETWEEN 1 AND 100 EACH 1, NO RANGE OR UNKNOWN) ADD 200)
  UNIQUE INDEX (claimid);
```

An INSERT of a new claim into district 101 will cause the new row to be placed in the NO RANGE partition.

The following ALTER TABLE can be used to add an additional 75 district partitions.

```
ALTER TABLE Claim_MLRP2 MODIFY ADD RANGE#L2 BETWEEN 101 AND 175 EACH 1;
```

Note the syntax to add or drop ranges for sub-levels (RANGE#Ln).

If the table or join index is defined with multiple partitioning levels, you can (and possibly should consider) explicitly specify the number of partitions per partitioning level using the ADD option.

This slide has an example of using the ADD option for the "district" sub-partition.

Note that the following ALTER TABLE will fail and return the 5734 error message:

```
ALTER TABLE Claim_MLRP2 MODIFY
  ADD RANGE#L2 BETWEEN 1 AND 175 EACH 1;
```

This fails because the ranges (i.e., districts) 1 to 100 already exist in the table.

If you want to change the special partitions for District from NO RANGE OR UNKNOWN to just NO RANGE, the following SQL can be used:

```
ALTER TABLE Claim_MLRP2 MODIFY
  DROP RANGE#L2 NO RANGE OR UNKNOWN
  ADD RANGE#L2 NO RANGE
  WITH DELETE;
```

What is a NoPI Table?

What is a No Primary Index (NoPI) Table?

- It is simply a table without a primary index
- As rows are inserted into a NoPI table, rows are always appended at the end of the table and never inserted in a middle of a hash sequence.
 - Organizing/sorting rows based on row hash is therefore avoided.

Basic Concepts

- Rows will still be distributed between AMPs. NoPI uses Random Generator code for SQL inserts or round robin logic for data blocks (e.g., TPT LOAD) to determine which AMP will receive rows or blocks of rows.
- Within an AMP, rows are simply appended to the end of the table. Rows will have a unique RowID – the Uniqueness Value is incremented.

Benefits

- A NoPI table will reduce skew in intermediate ETL tables which have no natural Primary Index.
- Loads using TPT Load (FastLoad) or TPT Stream (TPump) Array Insert into a NoPI staging table are faster.

A NoPI Table is simply a table without a primary index. NoPI stands for No Primary Index.

Without a PI, the hash value as well as AMP ownership of a row is arbitrary. Within the AMP, there are no row-ordering constraints and therefore rows can be appended to the end of the table as if it were a spool table. Each row in a NoPI table has a hash bucket value that is internally generated. A NoPI table is internally treated as a hashed table; it is just that typically all the rows on one AMP will have the same hash bucket value.

Loading Data into a NoPI Table (SQL)

Simple INSERTs

- For a simple INSERT, the PE selects a random AMP where the row is sent to. That AMP then turns the row into proper internal format and appends it to the end of the NoPI table.

INSERT-SELECT

- When inserting data from a source table (PI or NoPI) table into a NoPI target table, data from the source table will NOT be redistributed and will be locally appended into the target table.

Example: `INSERT INTO target_nopi_table
SELECT * FROM skewed_table;`

With INSERT-SELECT, the NoPI table can be skewed if the source table is skewed.

If the source table is skewed, the "HASH BY" option can be used to randomly (more evenly) distribute the data between the AMPs for a NoPI table.

- The HASH BY RANDOM clause will redistribute data blocks randomly between the AMPs.

```
INSERT INTO target_nopi_table
SELECT * FROM skewed_table      HASH BY RANDOM;
```

- The HASH BY RANDOM (1, 2000000000) is useful to redistribute individual rows and provides a more even distribution than the HASH BY RANDOM clause provides.

```
INSERT INTO target_nopi_table
SELECT * FROM source_table      HASH BY RANDOM (1,2000000000);
```

This slide summarizes various techniques of inserting data into a NoPI table using SQL INSERT or INSERT-SELECT statements.

However, the HASH BY and/or LOCAL ORDER BY options can be used to redistribute the data before copying the data into the NoPI table.

Note: You can only specify HASH BY and LOCAL ORDER BY clauses if the target table or the underlying target view table has no primary index (for example, a column-partitioned table). If the table has a primary index, the request is aborted and an error is returned.

If you specify a HASH BY RANDOM clause, data blocks of selected rows are first randomly redistributed. If you also specify a LOCAL ORDER BY clause, then the rows are locally ordered and inserted locally into the target table or underlying target view table.

HASH BY RANDOM (1, 2000000000) is useful to redistribute each individual selected row when there is no particular column set on which to hash distribute the rows, and when a more even distribution is needed than the HASH BY RANDOM clause provides.

Loading Data into a NoPI Table (Utility)

TPT Load or FastLoad

- Blocks of data are sent to the AMP load sessions and the AMP randomly distributes the blocks between the AMPs usually resulting in even distribution of the data between AMPs.
- The blocks are distributed between the AMPs using a round-robin technique.

TPT Stream or TPump

- With TPT Stream (TPump) Array INSERT, rows are packed together in a request and distributed to an AMP and then appended to the NoPI table on that AMP.
- Different requests are distributed to different AMPs by the PE. This will usually result in even distribution of the data between the AMPs.

This slide summarizes techniques of loading data into a NoPI table using the TPT Load (FastLoad) or TPT Stream (TPump) operators.

Secondary Index Usage

Unique Secondary Index (USI) Usage

- A USI is used to maintain uniqueness in a column or columns.
- Usage is determined by specifying the USI value in an equality condition in the WHERE clause or ON clause.
- Unique Secondary Indexes support ...
 - Nested Joins
 - Row-hash locking

Non-unique Secondary Index (NUSI) Usage

- Usage is determined by specifying the NUSI value in an equality condition in the WHERE clause or ON clause.
- Non-Unique Secondary Indexes support Nested Joins and Merge Joins
- Optimizer can choose to use bit-mapping for weakly selective (>10%) NUSIs which can alleviate limitations associated with composite NUSIs.
- In some cases, it may be better to use multiple single-column NUSIs (City, State) instead a single composite NUSI.
 - User has to balance the overhead of multiple NUSIs as compared to a single composite NUSI.
- Can be used to “cover” a query, avoiding base table access.
- Can significantly reduce base table I/O during value and join operations.

The facing lists common usage for a USI and a NUSI.

Value-Ordered NUSIs

A Value-Ordered NUSI is limited to a single numeric (4-byte) or DATE column.

Some benefits of using value-ordered NUSIs:

- Index subtable rows are sorted (sequenced) by data value rather than hash value.
- Optimizer can search only a portion of the index subtable for a given range of values.
- Can provide major advantages in performance of range queries.
- Even with a row partitioned table, the Value-Ordered NUSI is still a valuable index selection for other columns in a table.
 - An Orders table may be partitioned on Orderdate and have a VONUSI on Shipdate.

Example of creating a Value-Ordered NUSI:

```
CREATE INDEX (shipdate) ORDER BY VALUES (shipdate)
ON Orders;
```

```
SELECT shipdate
       ,SUM(ordertotal)
FROM Orders
WHERE shipdate
BETWEEN DATE '2021-01-01' AND DATE '2021-01-31'
GROUP BY 1
ORDER BY 1;
```

The optimizer may choose to transverse the NUSI using a range constraint rather than do an FTS.

NUSIs are maintained as separate subtables on each AMP. Their index entries point to base table or Join Index rows residing on the same AMP as the index. The row hash for NUSI rows is based on the secondary index column(s). Unlike row hash values for base table rows, this row hash does not determine the distribution of subtable rows; only the local sort order of each subtable.

Enhancements have been made to support the user-specified option of sorting the index rows by data value rather than by hash code. This is referred to as "value ordered" indexes and is presented to the user in the form of new syntax options in the CREATE INDEX statement.

By using the "value-ordered" indexes feature, this option can be specified to sort the index rows by data value rather than by hash code. Value-ordered NUSIs, on the other hand, are useful for processing range conditions and conditions with an inequality on the secondary index column set.

By sorting the NUSI rows by data value, it is possible to search only a portion of the index subtable for a given range of key values. The major advantage of a value-ordered NUSI is in the performance of range queries.

Value-ordered NUSIs have the following limitations.

- The sort key is limited to a single numeric column.
- The sort key column must be four or fewer bytes.

Range Accesses for a NUSI

If a query has range conditions and the NUSI is NOT value-ordered, the system may do a FTS of the NUSI subtable depending on the selectivity of the query.

- Retrieve Row IDs of qualifying base table rows into spool.
- Access the base table rows from the spooled Row IDs.
- The Optimizer needs collected statistics in order to make this choice.

If a query has range conditions and the NUSI is ordered by values, then the NUSI subtable is more likely be used to locate matching base table row.

```
CREATE INDEX (hiredate) ORDER BY VALUES (hiredate) ON Employee;

SELECT lastname, firstname, hiredate
FROM Employee
WHERE hiredate BETWEEN DATE '2021-01-01' AND DATE '2021-03-31';

SELECT lastname, firstname, hiredate
FROM Employee
WHERE EXTRACT(YEAR FROM hiredate) = EXTRACT(YEAR FROM Current_Date)
AND EXTRACT(MONTH FROM hiredate) = EXTRACT(MONTH FROM Current_Date);
```

Use EXPLAIN to see if, and how, indexes are being used.

Trivial Note: A Value-Ordered NUSI actually reserves two subtable IDs and this counts as 2 secondary indexes in the maximum count of 32 for a table.

Hash-Ordered NUSIs

- If a NUSI consists of multiple columns (and is not value or hash ordered), then the hash order of the NUSI subtable is based on the composite hash of the set of columns.
- The '**ORDER BY HASH**' clause provides the ability to create a multi-column index, but have the NUSI hashed based on a single attribute within the index, not the entire composite value.
 - A Hash-Ordered NUSI is useful for equality searches based on a single attribute and can be used to also cover queries.
 - Example: A NUSI may contain 10 columns for covering purposes and a single value '**ORDER BY HASH**' for equality searches on that NUSI value.
- This is effectively an example of vertical partitioning with a NUSI.
- Optimizer is much more likely to use a hash-ordered NUSI if you have collected statistics on the hash-ordered columns.

When a multi-column NUSI is created (without using the Order by Values or Hash option), the hash of the NUSI is based on the set of NUSI values. This is effectively a composite hash. Therefore, in order the hash to be used, an *equality* condition is needed on all of the secondary index column(s).

The Hash-Ordered NUSI provides the ability to create a multi-column index, but have the NUSI hashed based on a single attribute within the index, not the entire composite value.

When a multi-column NUSI is ordered by hash on a single column, each AMP creates a subtable row with a local row hash based on the hash of the single column. When the table is accessed via this column, the value can be hashed and this hash is used to access its portion of the index subtable to see if a corresponding row(s) exists.

Hash-Ordered NUSI Example

Assume a table has 26 columns. However, 95% of the queries use a subset of seven columns (F, A, C, T, O, R, Y). A NUSI subtable is all that needs to be accessed for these queries.

```
CREATE INDEX HO_Idx (F,A,C,T,O,R,Y) ORDER BY HASH (F) ON TableX;
```

This NUSI will cover the query and provide hash access for a specific value of column F.

TableX

A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z

H-O NUSI

F	A	C	T	O	R	Y

The Optimizer can build a plan to just access the NUSI subtable for queries that use a subset of the columns (F, A, C, T, O, R, Y). This can result in significant I/O savings.

Example of query that is covered by doing an FTS of the index subtable.

```
SELECT F, T, SUM(R) FROM TableX ORDER BY 1 GROUP BY 1, 2;
```

Example of query that is covered by this index and only rows with the hash and value of F are accessed.

```
SELECT F, T, SUM(R) FROM TableX WHERE F=748102 ORDER BY 1 GROUP BY 1, 2;
```

This slide illustrates the use of a Hash-Ordered NUSI to reduce I/O for a large percentage of the queries using a subset of the 26 columns from the table.

Compressed and Non-Compressed Join Indexes

- Recommendation: Normally use non-compressed join indexes except when creating a Global Join Index used as a "hashed NUSI" (covered later).
- A non-compressed join index provides more flexibility than a compressed join index and may not use more storage space because MVC on a base table carries into a non-compressed join index.
- Logical example of compressed and non-compressed join indexes.

```
CREATE JOIN INDEX Cust_Ord_JI1 AS
SELECT      (custid, lastname, firstname),
            (orderid, orderstatus, ... )
FROM        Customer C
INNER JOIN   Orders O
ON          C.custid = O.custid
PRIMARY INDEX (custid);
```

Compressed Join Index					
Join Index Row ID	Fixed Columns	Repeating Columns (Orders)			
RH	5001 ...	Order1	Order2	Order3	
RH	5002 ...	Order4	Order5		

```
CREATE JOIN INDEX Cust_Ord_JI2 AS
SELECT      custid, lastname, firstname
            orderid, orderstatus, ...
FROM        Customer C
INNER JOIN   Orders O
ON          C.custid = O.custid
PRIMARY INDEX (custid);
```

Non-Compressed Join Index			
Join Index Row ID	Join Index Column Values		
RH	5001 ...	Order1	
RH	5001 ...	Order2	
RH	5001 ...	Order3	
RH	5002 ...	Order4	
RH	5002 ...	Order5	

This slide illustrates the difference between a compressed and a non-compressed join index.

PERM Space Required

The amount of PERM space used by the compressed multi-table join index (previous example) and the non-compressed join index can vary.

In this example, the sizes (in bytes) of the two join indexes are:

<u>Join Index Name</u>	<u>Permanent Space</u>	<u>Type of Join Index</u>
Cust_Ord_JI1	20,848,640	non-compressed multi-table
Cust_Ord_JI2	16,949,248	compressed multi-table
Cust_Ord_JI3	26,558,464	non-compressed partitioned multi-table

Non-Compressed Multi-Table Join Index

```
CREATE JOIN INDEX Cust_Ord_JI1 AS
  SELECT      C.custid as c_custid, lastname, firstname,
             orderid, orderstatus, ordertotal, orderdate
  FROM        Customer C
  INNER JOIN   Orders O
  ON          C.custid = O.custid
  PRIMARY INDEX (c_custid);
```

Max # columns per
referenced table is 64

Maximum # of columns for
non-compressed join index is 2048.

Fixed Portion			Variable Portion			
c_custid	lastname	firstname	orderid	orderstatus	ordertotal	orderdate
111443	Federer	Roger	142292	C	500.25	01/10/2021
111443	Federer	Roger	145893	C	1393.95	01/12/2021
111443	Federer	Roger	157093	O	1025.45	01/31/2021
111443	Federer	Roger	135993	C	1025.37	12/14/2020
114000	Djokovic	Novak	142000	C	745.76	12/20/2020
114000	Djokovic	Novak	149600	C	945.40	01/05/2021
114000	Djokovic	Novak	154798	C	1540.50	01/07/2021
115809	Gauff	Cori	149698	C	2081.43	12/31/2020
115809	Gauff	Cori	156599	C	680.91	01/05/2021
115809	Gauff	Cori	101199	C	845.33	12/24/2020
115809	Gauff	Cori	158999	O	1002.50	01/23/2021
115809	Gauff	Cori	152399	C	945.20	01/07/2021

Within the join index, each order is represented by a separate join index row.

Options that are available with a non-compressed JI.

- Partitioning
- MVC on base tables is carried into a non-compressed JI.
- Unique Primary Index (13.10 feature)

This slide includes an example of creating a Multiple Table Join Index without using the repeating group option.

The storage space in this example for the non-compressed multi-table join index will be higher.

For a non-compressed multi-table join index, the maximum number of columns defined per referenced table is 64. The total maximum number of columns in this type of join index is 2048.

ALTERing a Join Index to CURRENT

This option allows you to periodically resolve the CURRENT_DATE (or DATE) and CURRENT_TIMESTAMP of a join index to their current values.

Benefits include:

- You do not have to use ALTER TABLE to change to partitioning (drop and/or add partitions) for a partitioned join index.
- To partition on a **new** CURRENT_DATE or CURRENT_TIMESTAMP value, simply submit an ALTER TABLE TO CURRENT request.

ALTER TABLE join_index_name TO CURRENT;

Example: **ALTER TABLE Cust_Ord_SJI TO CURRENT;**

Considerations:

- The options WITH DELETE and WITH INSERT INTO *save_table* are not available for a join index.
- If RANGE_N specifies CURRENT_DATE or CURRENT_TIMESTAMP in a partitioning expression, **you cannot use ALTER TABLE to add or drop ranges for the join index**. You must use the ALTER TABLE TO CURRENT statement to achieve this function.

You can also specify CURRENT_DATE and CURRENT_TIMESTAMP functions within a join index.

The ALTER TABLE statement can be used to modify a join index that has been defined with a moving CURRENT_DATE (or DATE) or moving CURRENT_TIMESTAMP. This option is called ALTER TABLE TO CURRENT.

When you specify CURRENT_DATE and CURRENT_TIMESTAMP as part of a partitioning expression for a join index, these functions resolve to the date and timestamp when you define the PPI. To partition on a new CURRENT_DATE or CURRENT_TIMESTAMP value, submit an ALTER TABLE TO CURRENT request.

The options WITH DELETE and WITH INSERT [INTO] *save_table* option are not available for a join index.

Partitioning a Sparse Join Index

A sparse non-compressed join index can also be partitioned.

- This allows a partition scan of the join index instead of full join index.
- This example creates a sparse partitioned join index for the current year.
 - Teradata allows the use of CURRENT_DATE (or DATE) and CURRENT_TIMESTAMP in a join index definition.
 - Use ALTER TABLE *join_index* TO CURRENT; to resolve the dates in the next year.
- This example also creates a NUSI on the join index.

```
CREATE JOIN INDEX Orders_RP_JI2 AS
  SELECT     orderid, custid, orderstatus, ordertotal, orderdate, clerkid
  FROM Orders_RP
  WHERE EXTRACT (YEAR FROM orderdate) = EXTRACT (YEAR FROM Current_Date)

PRIMARY INDEX (custid)
  PARTITION BY RANGE_N (orderdate BETWEEN
    CAST ((EXTRACT(YEAR FROM CURRENT_DATE)) || '-01-01' AS DATE) AND
    CAST ((EXTRACT(YEAR FROM CURRENT_DATE)) || '-12-31' AS DATE)
    EACH INTERVAL '1' DAY)

INDEX (clerkid);
```

To resolve the dates in the next year, ALTER TABLE Orders_RP_JI2 TO CURRENT;

The example on this slide contains an example of creating a partitioned sparse non-compressed join index and also includes a NUSI on the same partitioned sparse non-compressed join index.

This example also illustrates the use of CURRENT_DATE in the sparse portion and in the partitioning expression of the join index.

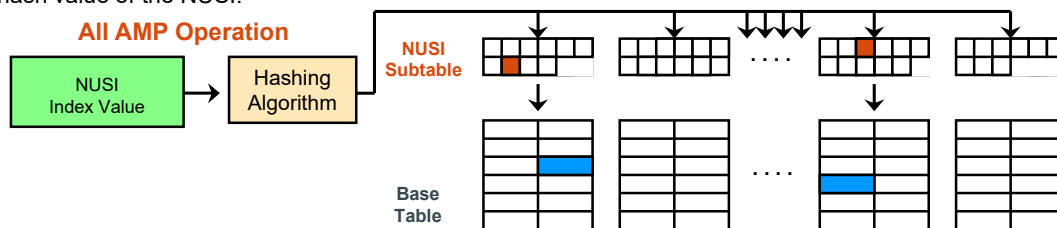
An alternative partitioning expression (to this slide) is:

```
PRIMARY INDEX (custid)
  PARTITION BY RANGE_N (orderdate BETWEEN
    CAST(((EXTRACT(YEAR FROM CURRENT_DATE) - 1900) * 10000 + 0101)
    AS DATE) AND
    CAST(((EXTRACT(YEAR FROM CURRENT_DATE) - 1900) * 10000 + 1231)
    AS DATE)
    EACH INTERVAL '1' DAY)
```

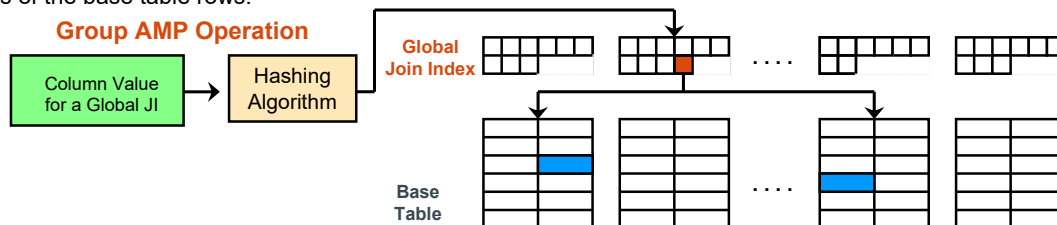
Global Index as a “Hashed NUSI”

Assume 200 AMPs and the typical rows per value is two for a column referenced in queries.

- A NUSI on this column avoids a FTS, but it is an **all-AMP operation** and every AMP is required to look in the NUSI subtable for the hash value of the NUSI.



- A Global Join Index (“Hashed NUSI”) on this column utilizes a **Group AMP operation**. The GJI subtable row contains the Row IDs of the base table rows.



Another use for a Global Join Index for a single table is that of a Hashed NUSI. An actual NUSI accesses all AMPs, whereas this index only accesses the AMPs that have rows matching the value. The Global Join Index is hashed, and the system uses the hash to access a single AMP, and then uses the Row IDs in the subtable row to then access only those AMPs that have rows.

ODS (Operational Data Store) or tactical queries that involve an equality condition on a fairly unique column can use a global index which will change:

- Table-level locks to row hash locks
- All-AMP steps to group-AMP steps

A “hashed NUSI” global index offers the advantages of a NUSI (it supports duplicate rows per value) combined with the advantages of a USI (its index rows are hash-distributed on the indexed value) and is often able to offer group AMP capabilities. As with all Teradata join indexes, its use is transparent to the query and will be determined by the optimizer.

Its main benefit is for situations where you are only getting a few rows for one value, and you can avoid an all AMP operation that a NUSI always requires. This may not have a huge impact on a system with a modest number of AMPs. However, for very large systems, with hundreds or thousands of AMPs, a group AMP operation that engages a small percentage (e.g., only 1% of the total AMPs or less), when done often enough, may increase overall throughput of the platform, as well as faster query response.

Creating a Global Index (“Hashed NUSI”)

```
CREATE JOIN INDEX Orders_JI7 AS
SELECT      (custid),
            (ROWID)
FROM        Orders
PRIMARY INDEX (custid);
```

Fixed portion of Join Index contains index value

Repeating portion of Join Index contains Row IDs

One Global Index row can contain multiple Row IDs.

- If the typical rows for a value is less than 50% of the number of AMPs, this global index will yield performance gains.
This effectively means that the number of Row IDs in the subtable row is less than 50% of the number of AMPs.
– EXPLAIN plan will indicate “group-AMPs” operation and “row hash locks”.
- If the typical rows for a value is 50% or greater than the number of AMPs, this global index will result in an all-AMP operation (like a NUSI).
This effectively means that the number of Row IDs in the subtable row is 50% or more than the number of AMPs.
– EXPLAIN plan will indicate “all-AMPs” operation and “table level locks”.

This slide contains an example creating a Global Join Index as a “Hashed NUSI”.

If the number of Row IDs within the Global Index subtable row is less than 50% of the number of AMPs, you will see an EXPLAIN plan with “group-AMPs” operations.

If the number of Row IDs within the Global Index subtable row is more than 50% of the number of AMPs, you will see an EXPLAIN plan with “all -AMPs” operations.

Note: When the Row ID is included in a Join Index, each Row ID is 10 bytes long. The partition number is included even if the base table is not partitioned. The partition number is 0 for non-partitioned tables.

Creating the Global Join Index without “Repeating Row Ids”

If the Global Join Index (that is to be used as a hashed NUSI) is created as follows – without the parenthesis, multiple Row IDs will not be included in a single join index subtable row.

```
CREATE JOIN INDEX  Orders_JI8
AS
SELECT            custid, ROWID
FROM              Orders
PRIMARY INDEX     (custid);
```

Column Partitioned Tables – Primary AMP

Column Partitioning with [Primary AMP](#) Index (CPPA)

```
CREATE TABLE Super_Bowl_CPPA
  (Winner      CHAR(25) NOT NULL,
   Loser       CHAR(25) NOT NULL,
   Game_Date   DATE     NOT NULL,
   Game_Score  CHAR(7)  NOT NULL,
   Attendance  INTEGER)
PRIMARY AMP INDEX ("Winner") PARTITION BY COLUMN; /* INDEX is optional */
```

- Rows are distributed to AMPs based on hash value of PA column(s):
 - Column partition values are appended on AMP to a combined partition and hash bucket where the hash bucket is chosen the same as for Column-Partitioned No Primary Index (CP NoPI)
 - Can also be row partitioned.
- Similar to a PI for row redistribution between AMPs, but rows are appended to the table on the AMP (similar to a NoPI table).
- [Single-AMP access when value for PA column\(s\) is specified.](#)
- More efficient aggregations with GROUP BY on PA columns.
- [More efficient joins, which is a key advantage.](#)
 - Allows local dynamic hash or product join when equality join on PA columns and other relation has same PA or PI columns.
 - Allows dynamic hash or product join with redistribution instead of duplication of other relation when equality join on PA columns and other relation does not have same PA or PI columns

Starting with Teradata 15.10, a column-partitioned table or join index may have a primary index or primary AMP index as an alternative to having no primary index. For a delete, subrows for a column partition with ROW format are physically deleted. For an update of only column partitions with ROW format, an update-in-place of subrows may occur.

A CP table may have a primary index (PI), primary AMP index (PA), or no primary index (NoPI).

However for PA or NoPI table, the rowid includes the internal partition number, hash bucket (not a full hash value) and uniqueness; for a PA table this hash bucket is unrelated to the hash bucket portion of the hash of the index value for the row. Instead for a PA or NoPI table, all rows owned by an AMP initially use the lowest hash bucket associated to that AMP by the NoPI hash map in their rowid. The NoPI hash map only includes hash buckets from the regular PI hash map where the AMP owning this hash bucket does not change if the number of AMPs increases; this avoids having to move rows to a different AMP when doing a Reconfig or restore/copy.

A primary AMP index (PA) allows for single-AMP access when the values of the primary AMP index column(s) are specified or to provide better distribution of the rows. Also, a PA allows various local join and aggregation methods if the join or GROUP BY is on the primary AMP index column(s). Row header compression and auto-compression effectiveness is similar to a CP table with no primary index. A PA also allows column partition elimination to occur in the same manner as for a CP table with a primary index or no primary index. This can occur with single-AMP access as discussed in the preceding paragraph or with other forms of access such as all-AMP access when not all the values of the primary AMP index are specified. If the table is also row partitioned, row partition elimination may be possible.

TableSize View

Provides AMP Vproc disk space usage at table level.

`DBC.TableSize[V][X]`

Vproc	DatabaseName	AccountName
TableName	CurrentPerm	PeakPerm

Example:

Display table distribution across AMPs of a NoPI table.

```
SELECT  Vproc
        ,CAST (TableName AS CHAR(15))
        ,CurrentPerm
FROM    DBC.TableSizeV
WHERE   DatabaseName = USER
AND     TableName = 'Customer_NoPI'
ORDER BY Vproc;
```

Result:

<u>Vproc</u>	<u>TableName</u>	<u>CurrentPerm</u>
0	Customer_NoPI	783,360
1	Customer_NoPI	812,032
2	Customer_NoPI	792,576
3	Customer_NoPI	803,840
4	Customer_NoPI	796,672
5	Customer_NoPI	802,304
6	Customer_NoPI	790,016
7	Customer_NoPI	793,088
8	Customer_NoPI	796,160
9	Customer_NoPI	800,768
10	Customer_NoPI	805,888
11	Customer_NoPI	801,792
12	Customer_NoPI	806,400
13	Customer_NoPI	781,212
:	:	:
29	Customer_NoPI	806,400

The TableSize[V][X] views are Data Dictionary views that provides AMP Vproc information about disk space usage at a table level, optionally for tables the current User owns or has SELECT privileges on.

Example

The SELECT statement on this slide looks for poorly distributed tables by displaying the CurrentPerm figures for a single table on all AMP vprocs.

Summary

Note with all indexing and partitioning choices, **there are tradeoffs** due to performance impacts on table access, joins, maintenance, and other operations.

General Recommendations for row partitioning:

- Collect statistics on system-derived column PARTITION.
- Do not define or name a column PARTITION in a PPI table – you won't be able to reference the system-derived column PARTITION for the table.
- If possible, avoid use of NO RANGE, NO RANGE OR UNKNOWN, or UNKNOWN options with RANGE_N partitioning for DATE columns.
- Consider only having as many date ranges as needed currently plus some for the future – helps the optimizer cost plans better, especially when partitioning column is not included in the Primary Index.

Reasons to use a Join Index:

- May be used to pre-join multiple tables.
- May be used as an aggregate index.
- The WHERE clause (**Sparse Index**) can be used to limit the number of rows in the join index.
- Row ID(s) of table (or tables) can be included to create a **"Global Index"**.



Module 3: Review Questions

1. With a populated table, which two actions are allowed with the ALTER TABLE command.
 - a. Drop all of the ranges
 - b. Add or drop ranges from the partition "ends"
 - c. Change the columns that comprise the primary index
 - d. Add or drop special partitions (NO RANGE, UNKNOWN)
2. What are two advantages of partitioning a table?
 - a. Fast delete of rows in partitions
 - b. Fewer AMPs are involved when accessing data
 - c. Range queries can be executed without doing an all-rows scan
 - d. Faster access for a PI value access than for a non-partitioned table
3. For NoPI tables, what are two ways in which the Random Generator is executed?
 - a. At the PE level for ad hoc SQL requests
 - b. At the TPT Stream client level for array insert operations
 - c. At the PE level with TPT Stream for array insert operations
 - d. At the AMP level for INSERT-SELECT into an empty NoPI table
4. Which two types of usage normally apply to a composite NUSI that is hash-ordered?
 - a. Covering index
 - b. Equality value access
 - c. Inequality value access
 - d. Non-covering range access

Check your understanding of the concepts discussed in this module by completing the review questions as directed by your instructor.



Lab Exercise 3-1 (1 of 2)

Purpose

In this lab, you will create tables with primary indexes partitioned in various ways.

What you need

Populated TRNG_DS tables and empty tables in your database

Tasks

1. Using INSERT/SELECT, populate your Orders and Orders_2021 tables from the TRNG_DS.Orders and TRNG_DS.Orders_2021 tables, respectively. Your Orders table will have data for the years 2012 to 2020 and the Orders_2021 table will have data for 2021. Verify the number of rows in your tables.

```
SELECT COUNT(*) FROM Orders;           Count = _____ (should be 450,000)
SELECT COUNT(*) FROM Orders_2021;      Count = _____ (should be 150,000)
```

2. Use the SHOW TABLE for Orders to help create a new, similar table (same column names and definitions, etc.) named "Orders_RP" that is row partitioned based on the following:

Primary Index – orderid

Partitioning column – orderdate

- From '2012-01-01' through '2021-12-31', partition by month
- Include the NO RANGE option (the UNKNOWN option is not needed for orderdate)
- Do not create any secondary indexes for this table

How many partitions are logically defined for the Orders_RP table? _____

Check your understanding of the concepts discussed in this module by completing the lab exercises as directed by your instructor.

SQL hints:

```
INSERT INTO table_1 SELECT * FROM table_2;
```

```
SELECT COUNT(*) FROM table_name;
```

```
SHOW TABLE table_name;
```

A count of rows in the Orders table is 450,000.

A count of rows in the Orders_2021 table is 150,000.



Lab Exercise 3-1 (2 of 2)

3. Use the SHOW TABLE for Orders to help create a new, similar table (same column names and definitions, etc.) named "Orders_RP2" that is row partitioned based on the following:

Primary Index – orderid

Partitioning column – orderdate

- Specify the date range '2012-01-01' through '2021-12-31' using CURRENT_DATE
- partition interval 1 day
- Include the NO RANGE option (the UNKNOWN option is not needed for orderdate)
- Do not create any secondary indexes for this table

4. Using INSERT/SELECT, populate the [Orders_RP](#) table with all rows from your [Orders](#) and [Orders_2021](#) tables.

Verify the number of rows in your table. Count = _____

5. Using INSERT/SELECT, populate the [Orders_RP2](#) table with all rows from your [Orders_RP](#) table.

Verify the number of rows in your table. Count = _____

SQL hints:

```
SELECT    PARTITION, MIN (orderdate), MAX (orderdate), COUNT(*)
FROM      table_name
GROUP BY  1
ORDER BY  1;
```

To determine the number of partitions in a table.

```
SELECT    COUNT(DISTINCT(PARTITION)) FROM table_name;
```

To determine the number of orders (rows) for months in a specific year:

```
SELECT    PARTITION, COUNT(*)
FROM      table_name
WHERE     EXTRACT(YEAR FROM date_column_name) = 2021
AND       EXTRACT(MONTH FROM date_column_name) between 1 and 12
GROUP BY  1
ORDER BY  1;
```

To determine the number of orders (rows) for a specific month:

```
SELECT    COUNT(*)
FROM      table_name
WHERE     EXTRACT(YEAR FROM date_column_name) = 2021
AND       EXTRACT(MONTH FROM date_column_name) = month_number;
```



Lab Exercise 3-2 (1 of 4)

Purpose

In this lab, you will determine data distribution on a per AMP basis for NoPI tables.

What you need

Populated Orders and Orders_2021 tables

Tasks

1. Use the SHOW TABLE for Orders to help create a new, similar table (same column names and definitions, etc.) named "Orders_custid" which has custid as the primary index.

Using INSERT/SELECT, populate the Orders_custid table with all rows from the Orders and Orders_2021 tables.

2. Determine the AMP space usage of the Orders_custid table using DBC.TableSizeV.

What is the least amount of permanent space on one of the AMPs? _____

What is the greatest amount of permanent space on one of the AMPs? _____

What is the skew factor for this table? _____

Distribution of table space by AMP:

```
SELECT      Vproc, TableName (CHAR(15)), CurrentPerm
FROM        DBC.TableSizeV
WHERE       DatabaseName = DATABASE
AND         TableName = 'table_name' ORDER BY 1;
```

To determine the minimum and maximum space used by AMP:

```
SELECT      MIN(CurrentPerm) AS Min_Size,
            MAX(CurrentPerm) AS Max_Size
FROM        DBC.TableSizeV
WHERE       DatabaseName = DATABASE
AND         TableName = 'table_name';
```



Lab Exercise 3-2 (2 of 4)

3. Use the SHOW TABLE for Orders to help create a new, similar table (same column names and definitions, etc.) named "Orders_NoPI" and does NOT have a primary index.

Using INSERT/SELECT, populate the Orders_NoPI table with all rows from your Orders_custid table.

4. Determine the AMP space usage of the Orders_NoPI table using DBC.TableSizeV.

What is the least amount of permanent space on one of the AMPs? _____

What is the greatest amount of permanent space on one of the AMPs? _____

What is the skew factor for this table? _____

Is the distribution similar to the Orders_custid table (yes or no)? _____

Why or why not? _____

To display sum of space, minimum space used by an AMP, maximum space used by an AMP in MB (megabytes) AND calculate a Skew Factor for all the tables in your database:

```
SELECT      TableName
            ,SUM(CurrentPerm)/(1024**2) AS CurentPerm_MB
            ,MIN(CurrentPerm)/(1024**2) AS Min_Size_MB
            ,MAX(CurrentPerm)/(1024**2) AS Max_Size_MB
            ,(100 - (CAST(AVG(CurrentPerm) AS FLOAT) /
              NULLIFZERO (MAX(CurrentPerm))* 100)) AS SKEW_FACTOR
FROM        DBC.TableSizeV
WHERE       DatabaseName = DATABASE
GROUP BY    1
ORDER BY    1;
```



Lab Exercise 3-2 (3 of 4)

5. DELETE all the rows in Orders_NoPI.

Using INSERT/SELECT with the **HASH BY RANDOM** option, populate your **Orders_NoPI** with all rows from your **Orders_custid** table.

6. Determine the AMP space usage of your **Orders_NoPI** table using DBC.TableSizeV.

What is the least amount of permanent space on one of the AMPs? _____

What is the greatest amount of permanent space on one of the AMPs? _____

What is the skew factor for this table? _____

Is the space distribution the same as the **Orders_custid** table (yes or no)? _____

Is the table distributed as evenly as expected (yes or no)? _____

Why or why not? _____

To display sum of space, minimum space used by an AMP, maximum space used by an AMP in MB (megabytes) AND calculate a Skew Factor for all the tables in your database.

This is an alternative to a previous query which displayed the results in MB.

```
SELECT      TableName
            ,SUM(CurrentPerm)/(2**20) AS CurentPerm_MB
            ,MIN(CurrentPerm)/(2**20) AS Min_Size_MB
            ,MAX(CurrentPerm)/(2**20) AS Max_Size_MB
            ,(100 - (CAST(AVG(CurrentPerm) AS FLOAT) /
              NULLIFZERO (MAX(CurrentPerm))* 100)) AS SKEW_FACTOR
FROM        DBC.TableSizeV
WHERE       DatabaseName = DATABASE
GROUP BY    1
ORDER BY    1;
```



Lab Exercise 3-2 (4 of 4)

7. DELETE all the rows in Orders_NoPI.

Using INSERT/SELECT with the `HASH BY RANDOM (1, 2000000000)` option, populate Orders_NoPI with all the rows from your `Orders_custid` table.

8. Determine the AMP space usage of your Orders_NoPI table using `DBC.TableSizeV`.

What is the least amount of permanent space on one of the AMPs? size _____

What is the greatest amount of permanent space on one of the AMPs? size _____

What is the skew factor for this table? _____

Is the space distribution similar to the `Orders_custid` table (yes or no)? _____

Is the table distributed as evenly as expected (yes or no)? _____

Why or why not? _____

Was row level vs. block level distribution useful in this case? _____

This query is simply repeated for your convenience:

```
SELECT      TableName
            ,SUM(CurrentPerm)/(1024**2) AS CurentPerm_MB
            ,MIN(CurrentPerm)/(1024**2) AS Min_Size_MB
            ,MAX(CurrentPerm)/(1024**2) AS Max_Size_MB
            ,(100 - (CAST(AVG(CurrentPerm) AS FLOAT) /
              NULLIFZERO (MAX(CurrentPerm))* 100)) AS SKEW_FACTOR
FROM        DBC.TableSizeV
WHERE       DatabaseName = DATABASE
GROUP BY    1
ORDER BY    1;
```



Lab Exercise 3-3

1. Use SHOW TABLE for Orders_RP2 to create a column partitioned table named "Orders_CPPA_RP based on the following:
 - Incorporate row partitioning with orderdate as the partitioning column
 - From '2012-01-01' through '2021-12-31', partition by day using CURRENT_DATE
 - Do not use the NO RANGE or UNKNOWN options
 - Each column is created as a separate partition (COLUMN partitioning is the second level)
 - Use the PRIMARY AMP INDEX option with custid
 - Utilize auto compression for every column
2. Populate the Orders_CPPA_RP table (via INSERT/SELECT) from the Orders_NoPI table.
3. Use the PARTITION key word to determine the number of partitions defined in the Orders_CPPA_RP.
How many partitions actually have data? _____

SQL hints:

INSERT INTO *table_1* SELECT * FROM *table_2*;

SELECT COUNT(*) FROM *table_name*;

**SELECT PARTITION, COUNT(*) FROM *table_name*
GROUP BY 1
ORDER BY 1;**

SELECT COUNT(DISTINCT(PARTITION)) FROM *table_name*;



Module 4: The Data Dictionary

Teradata Vantage Administration

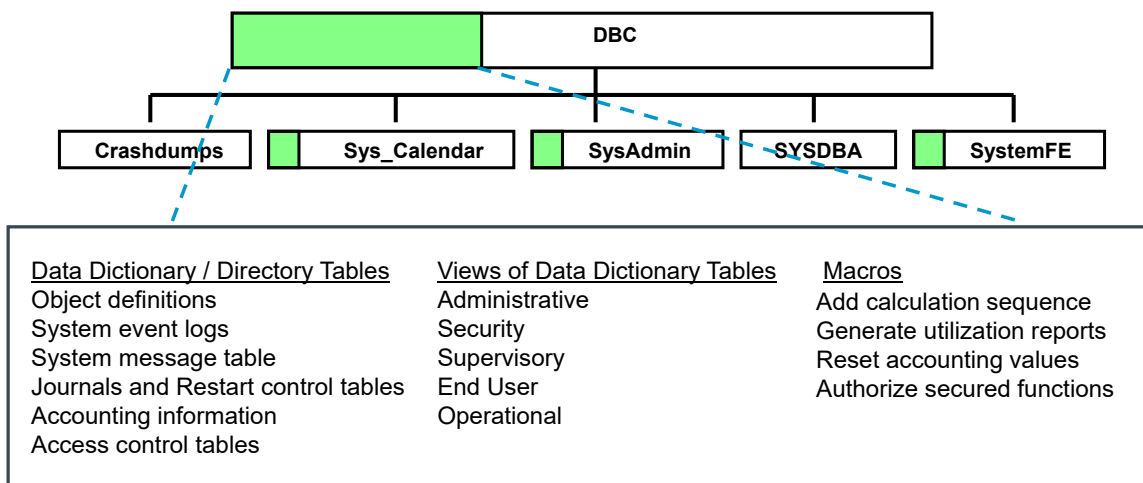
Copyright © 2007–2023 by Teradata. All Rights Reserved.

Objectives

After completing this module, you will be able to:

- Summarize information contained in the Data Dictionary tables
- Differentiate between restricted and unrestricted views
- Use the supplied Data Dictionary views to retrieve information about created objects

Data Dictionary / Directory



The data dictionary/directory is a complete database composed of system tables, views, and macros that reside in system user DBC. It is referred to as a Dictionary / Directory because it provides two functions:

- Dictionary – information you can view (e.g., you can view the columns and their attributes of a table)
- Directory – information to control the system (e.g., table names are converted to table IDs for the software to use)

The Teradata Data Dictionary / Directory may be referred to as simply the Teradata Data Dictionary.

Data dictionary views reference data dictionary tables. The system views and macros are created by running the Database Initialization Program (DIP) scripts. When a system is first installed, the **start dip** utility is executed by the installation person/team.

Data dictionary tables are used to:

- Store definitions of objects you create (e.g., databases, tables, indexes, etc.).
- Record system events (e.g., logon, console messages, etc.).
- Hold system message texts.
- Control system restarts.
- Accumulate accounting information.
- Control access to data.

Fallback Protected Data Dictionary Tables

(Partial list of Data Dictionary tables in Teradata 17.10)

AccessRights Users Rights on objects	AccLogRuleTbl Specifies events to be logged	AccLogTbl Logged User-Object events	Accounts Account Codes by user
ALL (Dummy) Represents all tables	ConstraintNames Named indexes or referential	DBase Database and User Profiles	DBCInfoTbl Software Release & Version
ErrorMsgs Message Codes and text	EventLog Session logon/logoff history	Hosts To override default char. sets	IdCol Maintains Identity column data
Indexes Defines indexes on tables	LogonRuleTbl Users Rights on objects	Next Internal ID for next create	ObjectUsage Object Use Count
Owners Hierarchy (Downward)	Parents Hierarchy (Upward)	Profiles Users and logon attributes	RCConfiguration Archive/Recovery Config
RCEvent Archive/Recovery events	RepGroup Replication Groups for Tables	ResUsage Resource Usage tables	ReferencedTbls Referential Integrity (PK)
ReferencingTbls Referential Integrity (FK)	RoleGrants Users/Roles to Roles assigned	Roles Defined Roles	SecConstraints Security Constraint Objects
SessionTbl Current logon information	SW_Event_Log Database Console Log	SysSecDefaults Logon security options	TVFields Table/View column description
TVM Tables, Views and Macros	TableConstraints Table Constraints	TempTables Materialized Temporary Tables	TriggersTbl Stores trigger information

Most data dictionary tables are fallback protected.

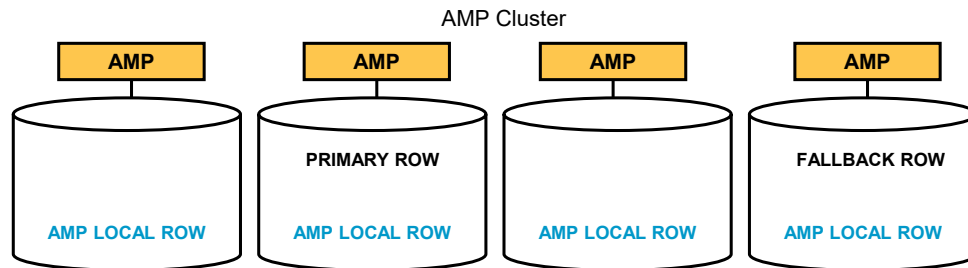
Fallback protection means that a copy of every table row is maintained on a different AMP vproc in the configuration. Fallback-protected tables are always fully accessible and are automatically recovered by the system.

Note: Every system database and user includes a dummy table named “ALL” (with an internal TableID of binary zeros). This table represents all the tables in a system database or user when, for example, privileges are granted or disk space is summarized at the database level.

Starting with Teradata 14.10, there is a reduced risk of lock contention on popular dictionary tables – this is done via a new dictionary table DBC.ObjectUsage.

Non-Hashed Data Dictionary Tables

Acctg <i>Resource usage by user/account</i>	ChangedRowJournal <i>Down-AMP Recovery Journal</i>	DatabaseSpace <i>Database and Table space accounting</i>
LocalSessionStatus <i>Last request status by AMP</i>	LocalTransactionStatus <i>Last TXN Consensus status</i>	OrdSysChngTable <i>Table-level recovery</i>
RecoveryLockTable <i>Recovery session locks</i>	RecoveryPJTable <i>Permanent Journal recovery</i>	SavedTransactionStatus <i>AMP recovery table</i>
SysRcvStatJournal <i>Recovery, reconfig, startup information</i>	TransientJournal <i>Actually implemented in WAL Log</i>	UtilityLockJournalTable <i>Host Utility Lock records</i>



The data dictionary tables on the following page contain rows that are *not* distributed using hash maps.

Rows in these tables are stored AMP-locally. For example, the DBC.Acctg table rows represent CPU time and I/O counts and are stored on the same AMP where the CPU time is used and I/Os are executed.

Note: User-defined table rows are *always* hash distributed ... either with or without a fallback copy.

Updating Data Dictionary Tables

```
EXPLAIN CREATE TABLE Orders
( order_id      INTEGER NOT NULL,
  order_date    DATE FORMAT 'yyyy-mm-dd',
  cust_id       INTEGER )
UNIQUE PRIMARY INDEX (order_id);
```

17.10 Explain

-
- 1) First, we lock TFACT.Orders in TD_MAP1 for exclusive use.
 - 2) Next, we lock a distinct DBC."pseudo table" in TD_DATADictionaryMAP for write on a RowHash for deadlock prevention, we lock a distinct DBC."pseudo table" in TD_DATADictionaryMAP for write on a RowHash for deadlock prevention, we lock a distinct DBC."pseudo table" in TD_DATADictionaryMAP for read on a RowHash for deadlock prevention, and we lock a distinct DBC."pseudo table" in TD_DATADictionaryMAP for write on a RowHash for deadlock prevention.
 - 3) We execute the following steps in parallel.
 - 1) We do a single-AMP ABORT TEST step from DBC.DBase by way of the unique primary index "Field_1 = 'TFACT'" with a residual condition of ("'00003207'XB= DBC.DBase.Field_2").
 - 2) We do a single-AMP ABORT TEST step from DBC.TVM by way of the unique primary index "Field_1 = '00003207'XB, Field_2 = 'ORDERS'".
 - 3) We do an INSERT step into DBC.Indexes (no lock required).
 - 4) We do an INSERT step into DBC.TVFields (no lock required).
 - 5) We do an INSERT step into DBC.TVFields (no lock required).
 - 6) We do an INSERT step into DBC.TVFields (no lock required).
 - 7) We do an INSERT step into DBC.ObjectUsage (no lock required).
 - 8) We do a two-AMP ABORT TEST step in TD_DATADictionaryMAP from DBC.Maps by way of unique index # 4 "Field_3 = 1025" with no residual conditions.
 - 9) We do an INSERT step into DBC.TVM (no lock required).
 - 10) We INSERT default rights to DBC.AccessRights for TFACT.Orders.
 - 6) We create the table header in TD_MAP1.
 - 7) Finally, we send out an END TRANSACTION step to all AMPs involved in processing the request.
- > No rows are returned to the user as the result of statement 1.

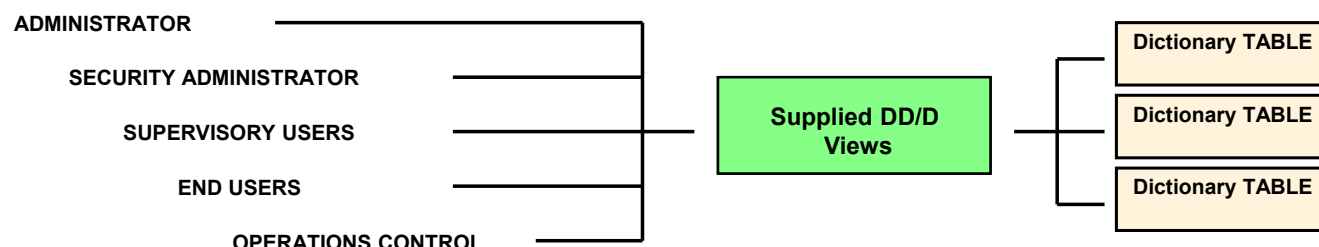
Whenever you submit a data definition or data control statement, Teradata system software automatically updates data dictionary tables.

When you use the EXPLAIN modifier to describe a DDL statement, you can view updates to the data dictionary tables.

The EXPLAIN modifier is a helpful function that allows you to understand what happens when you execute an SQL statement.

- The statement is not executed.
- The type of locking used is described.
- At least five different tables are updated when you define a new table.

Supplied Data Dictionary Views



- Views are representations of data accessed/derived from DD/D tables.
 - Clarify tables – re-title tables and/or columns; reorder and format columns, etc.
 - Simplify operations – supply join operation syntax; select and project relevant rows and columns.
 - Limit access to data – exclude certain rows and/or columns from selection.
- View definitions are stored in DBC.TVM.
- View column information is stored in DBC.TVFields.
- DIP scripts install the dictionary views.
- Disclaimer: The view descriptions in this module may not include all the columns in the view. The complete view and column definitions are in an appendix of this manual.

System views are supplied from the data dictionary for frequently used data. The system views do not contain data. They are stored as entries in the data dictionary until you submit an SQL statement that uses them. Views of data dictionary tables are provided for the same reasons that views are defined for any database application.

Data dictionary table column names are re-titled and formatted. Derived values are computed from data dictionary tables. Most supplied views reference more than one table and have the join syntax included. Supplied views also allow you, as the database administrator, to limit access to data dictionary information and provide a consistent image of the data stored in the data dictionary. In practice, as the administrator you may grant permission to the appropriate members of your organization to use any supplied view.

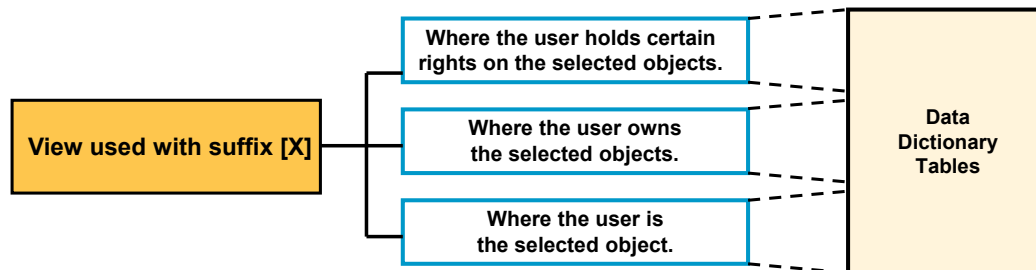
The installation script for the standard views is contained in the supplied Database Initialization Program (DIP) scripts which are normally executed by the installation teams.

The DIP installation screen lists each DIP script separately. You are given the option of choosing “All” to execute all the DIP scripts with one command, or you can choose and run each script separately. There are specific DIP scripts that can be executed to enable specific functions.

Restricted Views

These views limit the scope of what a user can access in the DD/D.

Views with an [X] suffix typically make the following three tests before returning information to the user:



For example,

- The following query returns information about ALL parents and children.
- The restricted [x] version of this view selects only information on objects owned by the executing user or that the requesting user has access to:

```
SELECT Child, Parent FROM DBC.ChildrenV;
```

```
SELECT Child, Parent FROM DBC.ChildrenVX;
```

There are two versions of the system views: restricted [x] and non-restricted [non-x]. The system administrator can load either or both versions. Non-X views are named according to the contents of their underlying tables. DBC.DiskSpaceV, DBC.TableSizeV, and DBC.SessionInfoV are examples of Non-X views. X Views are the same views with an appended WHERE clause. The WHERE clause limits the information returned by a view to only those rows associated with the requesting user.

Granted Rights

By default, the SELECT privilege is granted to PUBLIC User on most views in X and non-X versions. This privilege allows any user to retrieve view information via the SELECT statement. The system administrator can use GRANT or REVOKE statements to grant or revoke a privilege on any view to or from any user at any time.

Special Needs Views

Some views are applicable only to users who have special needs. For example, the administrator, a security administrator, or a Teradata field engineer may need to see information that other users do not need. Access to these views is granted only to the applicable user.

Access Tests

Limited views typically run three different tests before returning information from data dictionary tables to a user. Each test focuses on the user and his or her current privileges. It can take longer to receive a response when a user accesses a restricted view.

Suffix Options with Views

Views may optionally include a suffix of [V] and/or [X].

- Views without the V are the "Compatibility" version of the views, where object name columns have a data type of CHAR(30).
- In general, the X version is for restricted views.
- Most views have a "V" version which is the newer version of the view and defines object name columns with a data type of VARCHAR(128). This version can display information associated with Unicode.

The extended object names feature allows object names up to 128 characters.

For example, the recommended views to use for account information are:

- DBC.AccountInfoV – displays global information using VARCHAR(128)
- DBC.AccountInfoVX – displays information associated with the requesting user only using VARCHAR(128).

The older "compatibility" views (without the V) are:

- DBC.AccountInfo – displays global information using CHAR(30)
- DBC.AccountInfoX – displays information associated with the requesting user only using CHAR(30).

The following are the view forms:

- Without the V, the views display information associated with the Compatibility version, where object name columns have a data type of CHAR(30).
- With the V, the views display information associated with the Unicode version, where object name columns have a data type of VARCHAR (128). Even though objects are limited to a max of 127 characters, the tables in the DD have 128 Character columns defined.
- Without the X, the views display global information.
- With the X, the views display information associated with the requesting user only.

Operations that use restricted views may take longer to run because these views access more data dictionary tables. By contrast, operations that use unrestricted views may run faster but return more rows. To control access to data dictionary information, you can grant users permission to access only restricted views.

Teradata 14.10 extended object names feature

- Extends all ANSI SQL object names (e.g., User/Database/Table/Column Names, Accounts, and Passwords).
- Increases the length of names to 127 NFD characters.
- Extends the repertoire of names to any Teradata supported Unicode character.
- Name Validation Rules added.
- Mechanism added to display objects from any session character set.
- Further enhancements of this feature are expected but minor.

Selecting Information about Created Objects

DBC.Children[V][X]	Hierarchical relationship information.
DBC.Databases[V][X]	Database, user and immediate parent information.
DBC.Users[V]	Similar to Databases view, but includes columns specific to users.
DBC.Tables[V][X]	Tables, views, macros, triggers, and stored procedures information.
DBC.ShowTblChecks[V][X]	Database table constraint information.
DBC.ShowColChecks[V][X]	Database column constraint information.
DBC.Columns[V][X]	Information about columns/parameters in tables, views, and macros.
DBC.Indices[V][X]	Table index information.
DBC.PartitioningConstraints[V][X]	Provides information about index constraints, e.g., PPI definition.
DBC.AllTempTables[V][X]	Information about global temporary tables materialized in the system.
DBC.Triggers[V][X]	Information about event-driven, specialized procedures attached to a single table and stored in the database.

These views return information about created objects:

Note: The table “Indexes” is referenced by a view spelled “Indices.”

Children View

Provides the names of all databases, users and their owners.

[DBC.Children\[V\]\[X\]](#)

Child	Parent
-------	--------

Example:

Using the unrestricted form of the view and a WHERE clause, list your parents.

```
SELECT *
FROM   DBC.ChildrenV
WHERE  CHILD = USER;
```

Results:

Child	Parent
student230	DBC
student230	Students
student230	Sysdba
student230	TT_Class2

The Children view lists the names of databases and users and their parents in the hierarchy.

Column

Definition

Child

Name of a child database or user

Parent

Name of a parent database or user

Example

This slide has an SQL statement that lists the parents of the current user. The SQL keyword USER causes the parser to substitute the "User Name" of the user who has logged on and submitted the statement. The results of the request show one child, student230, and four parents.

Databases View

Provides information about [databases](#) and [users](#).

[DBC.Databases\[V\]\[X\]](#)

DatabaseName	CreatorName
OwnerName	AccountName
ProtectionType	JournalFlag
PermSpace	SpoolSpace
TempSpace	CommentString
CreateTimeStamp	LastAlterName
LastAlterTimeStamp	DBKind
AccessCount **	LastAccessTimeStamp **

** Only updated if Logging Object Use Counts is enabled

Example:

For databases/users with a name of "tfact%", find the creator name, when it was created, its max perm space, and the type (database or user).

```
SELECT DatabaseName (CHAR(10)) AS "Name"
,CreatorName (CHAR(10)) AS "Creator"
,CreateTimeStamp
,PermSpace (FORMAT 'zzz,zz9,999')
,DBKind
FROM DBC.DatabasesV
WHERE DatabaseName LIKE 'TFACT%'
ORDER BY 1;
```

Results:

<u>Name</u>	<u>Creator</u>	<u>CreateTimeStamp</u>	<u>PermSpace</u>	<u>DBKind</u>
TFACT	DBC	2019-03-31 20:57:43	20,000,000	D
tfact01	Sysdba	2019-03-31 21:09:52	10,000,000	U
tfact02	Sysdba	2019-03-31 21:09:55	10,000,000	U
tfact03	Sysdba	2019-03-31 21:10:09	10,000,000	U

The Databases view returns information about databases and users from the DBC.DBase table.

Notes:

- Only the *immediate* owner is identified in this view. Use the parent column of the Children view to select *all* owners.
- The data dictionary records the name of the creator of a system user or database, as well as the date and time the user created the object. This information is not used by the software, but is recorded in DBC.DBase for historical purposes.

Column definitions in this view include:

<u>Column</u>	<u>Definition</u>
OwnerName	The IMMEDIATE parent (owner)
ProtectionType	Default protection type for tables created within this database: F = Fallback N = No Fallback
JournalFlag	Two characters (before and after image) where: S = Single D = Dual N = None L = Local
CreatorName	Name of the user who created the object.
CreateTimeStamp	Date and time the user created the object.

Users View

Provides information about the users that the requesting user owns or to which he or she has modify rights. This is a restricted view ... there is no [x] version.

[DBC.Users\[V\]](#)

UserName	CreatorName	PasswordLastModDate	PasswordLastModTime	OwnerName
PermSpace	SpoolSpace	TempSpace	ProtectionType	JournalFlag
StartupString	DefaultAccount	DefaultDatabase	CommentString	DefaultCollation
PasswordChgDate	LockedDate	LockedTime	LockedCount	TimeZoneHour
TimeZoneMinute	DefaultDateForm	CreateTimeStamp	LastAlterName	LastAlterTimeStamp
DefaultCharType	RoleName	ProfileName	AccessCount **	LastAccessTimeStamp **

Example:

Find your default account code, the name of your immediate owner, and max spool space.

```
SELECT  UserName
        ,DefaultAccount
        ,OwnerName
        ,SpoolSpace (FORMAT 'zzz,zz9,999')
FROM    DBC.UsersV
WHERE   UserName = USER;
```

** Only updated if Logging Object Use Counts is enabled

Results:

<u>UserName</u>	<u>DefaultAccount</u>	<u>Owner</u>	<u>SpoolSpace</u>
Tfact02	\$M00EDUC&D&H	Factory	20,000,000,000

The Users view is a subset of the Databases view and:

- Limits rows returned from DBC.DBase to only USER rows (e.g., where there is a password).

Restricts rows returned to:

- The current users' information.
- Information about owned users (i.e., children).
- Information about users on which the current user has DROP USER or DROP DATABASE rights.
- Date and time a user is locked due to excessive erroneous passwords and the number of failed attempts since the last successful one.

The view features CreatorName and CreateTimeStamp columns that display the user name who created an object and the date and time he or she created it. The LastAlterName and LastAlterTimeStamp columns list the name of the last user to modify an object, as well as the date and time.

Note: The DBC.Users view is already a restricted view; there is no [X] version.

Tables View

Provides information about objects – tables, views, macros, journals, join indexes, hash indexes, triggers, stored procedures, etc.

[DBC.Tables\[V\]\[X\]](#)

DataBaseName	TableName	Version	TableKind	ProtectionType
JournalFlag	CreatorName	RequestText	CommentString	ParentCount
ChildCount	NamedTblCheckCount	UnnamedTblCheckExist	PrimaryKeyIndexId	RepStatus
CreateTimeStamp	LastAlterName	LastAlterTimeStamp	RequestTxtOverflow	AccessCount **
LastAccessTimeStamp **	UtilVersion	QueueFlag	CommitOpt	

(this is not a complete list of columns)

** Only updated if Logging Object Use Counts is enabled

Example:

List the objects that contain the characters "rolerights" in their name.

```
SELECT TRIM(DatabaseName) || '.' ||
       TableName AS "Qualified Name"
       ,TableKind
FROM   DBC.TablesV
WHERE  TableName LIKE '%rolerights%'
ORDER BY 1, 2 ;
```

Results:

Qualified Name	TableKind
DBC.AllRoleRights	V
DBC.AllRoleRightsV	V
DBC.UserRoleRights	V
DBC.UserRoleRightsV	V

The Tables view accesses the data dictionary table, DBC.TVM, which contains descriptions of objects – tables, views, macros, journals, join indexes, triggers, stored procedures, etc..

The view features a TableKind column that allows you to specify the *kind* of object to reference. The view also features CreatorName and CreateTimeStamp columns that display the name of the user who created an object and the date and time he or she created it. The LastAlterName and LastAlterTimeStamp columns list the name of the last user to modify an object, as well as the date and time.

The PrimaryKeyIndexID column identifies the columns used as the primary index. As the administrator, use this view to find NO FALLBACK tables (where ProtectionType = 'N').

TableKind	T = Table
	M = Macro
	V = View
	G = Trigger
	P = Stored Procedure
	F = User-defined Function
	I = Join index
	N = Hash Index
	J = Journal table
	O = No Primary Index

Columns View

Provides information about columns in tables and views, and parameters in macros and stored procedures.

[DBC.Columns\[V\]\[X\]](#)

DatabaseName	TableName	ColumnName	ColumnFormat	ColumnTitle
SPPParameterType	ColumnType	UDTColumnName	ColumnLength	DefaultValue
Nullable	CommentString	DecimalTotalDigits	DecimalFractionalDigits	ColumnId
UpperCaseFlag	Compressible	CompressValue	ColumnConstraint	ConstraintCount
CreatorName	CreateTimeStamp	LastAlterName	LastAlterTimeStamp	CharType
IdColType	AccessCount **	LastAccessTimeStamp **	CompressValueList	TimeDimension
VTCheckType	TTCheckType			

** Only updated if Logging Object Use Counts is enabled

Results:

Example:

Use this view to show the parameters of the DBC.ResCPUbyAMP macro.

```
SELECT ColumnName, ColumnFormat, DefaultValue
FROM   DBC.ColumnsV
WHERE  DatabaseName = 'DBC' AND
       TableName = 'ResCPUbyAMP';
```

ColumnName	ColumnFormat	DefaultValue
FromDate	YYYY-MM-DD	Date
FromNode	X(6)	'000-00'
FromTime	99:99:99	0.000000000000000E000
ToTime	99:99:99	9.999990000000000E005
ToDate	YYYY-MM-DD	Date
ToNode	X(6)	'999-99'

The Columns view returns information from the DBC.TVFields table.

This data dictionary table includes information about:

- Table and view columns
- Macro and stored procedure parameters

Like several other views in this module, the Columns view features CreatorName and CreateTimeStamp columns that display the name of the user who created an object and the date and time he or she created it. The LastAlterName and LastAlterTimeStamp columns list the name of the last user to modify an object, as well as the date and time.

As an administrator, you may use this view to enforce domain constraints. The following SELECT statement provides an example:

```
SELECT  DatabaseName, TableName FROM DBC.TVFields
WHERE   ColumnTitle LIKE 'amount'
AND     ColumnType NE 'D'
OR      DecimalTotalDigits NE 7
OR      DecimalFractionalDigits NE 2
ORDER BY 1, 2;
```

Indices View

Provides information about each indexed column defined for each table.

[DBC.Indices\[V\]\[X\]](#)

DatabaseName	TableName
IndexNumber	IndexType
UniqueFlag	IndexName
ColumnName	ColumnPosition
CreatorName	CreateTimeStamp
LastAlterName	LastAlterTimeStamp
IndexMode	AccessCount **
LastAccessTimeStamp **	UniqueOrPK
VTConstraintType	TTConstraintType
SystemDefinedJI	

** Only updated if Logging Object Use Counts is enabled

Example:

Select information about the Employee Phone table indices in the current database.

```
SELECT      ColumnName (CHAR(15)) AS "Column Name"
           ,UniqueFlag          AS "Unique"
           ,IndexType           AS "Type"
           ,IndexName           AS "Name"
           ,IndexNumber         AS "IndNo"
           ,ColumnPosition      AS "ColPos"
FROM        DBC.IndicesV
WHERE       TableName = 'Emp_Phone'
AND         DatabaseName = DATABASE
ORDER BY IndNo, ColPos ;
```

Results:

Column Name	Unique	Type	Name	IndNo	ColPos
employeenumber	N	P	?	1	1
areacode	N	S	ac_phone	4	1
phonenumber	N	S	ac_phone	4	2

The Indices view returns information about each indexed column from the DBC.Indices table. (A compound index returns multiple rows.) Use the view to list tables with non-unique primary indexes (NUPI). (These tables may be subject to skewed data distribution.)

Column IndexType

Definition

P = Primary (non-partitioned table)
 Q= Primary (partitioned table)
 S = Secondary
 U = Unique constraint (USI with NOT NULL)
 K = Primary Key
 J = Join
 V = Value-ordered secondary
 H = Hash-ordered ALL (covering) secondary
 O = Value-ordered ALL (covering) secondary
 I = Ordering column of a composite secondary index
 M = Multi-Column Statistics
 D = Derived column partition statistics
 Q = Partitioned Primary Index

UniqueFlag

Y = Unique N = Non-unique

Indices View (Second Example)

Example:

Select information about the Department table indices in the current database.

SQL of how this Join Index was created:

```
CREATE JOIN INDEX Dept_JnIx AS
  SELECT deptnumber, deptname,
         deptmgrnumber
  FROM   Department
  PRIMARY INDEX (deptmgrnumber);
```

```
SELECT   ColumnName (CHAR(15)) AS "Column Name"
        ,UniqueFlag      AS "Unique"
        ,IndexType       AS "Type"
        ,IndexName       AS "Name"
        ,IndexNumber     AS "IndNo"
        ,ColumnPosition  AS "ColPos"
FROM     DBC.IndicesV
WHERE    TableName = 'Department'
AND      DatabaseName = DATABASE
ORDER BY IndNo, ColPos ;
```

Results:

Column Name	Unique	Type	Name	IndNo	ColPos
deptnumber	Y	P	?	1	1
deptname	Y	U	?	4	1
deptnumber	N	J	Dept_JnIx	8	1
deptname	N	J	Dept_JnIx	8	2
deptmgrnumber	N	J	Dept_JnIx	8	3

Column definitions for this view include:

Column

Definition

IndexType

P = Primary
 S = Secondary
 K = UPI or USI is the index is create via PRIMARY KEY constraint
 U = UPI or USI (U is used if the index is created via UNIQUE constraint)
 J = Join
 V = Value-ordered secondary
 H = Hash-ordered ALL (covering) secondary
 O = Value-ordered ALL (covering) secondary
 I = Ordering column of a composite secondary index

UniqueFlag

Y = Unique
 N = Non-unique

Column Position

Position of a column within an index. This value may be greater than one if the column is part of a composite index.

1 = Field 1 column of a join index
 2 = Field 2 column of a join index

Partitioning Constraints View

This view is the [recommended view to use to display information about partitioning constraints](#) for tables or join indexes.

- The previous view was named DBC.IndexConstraintsV and is still available.

[DBC.PartitioningConstraintsV\[X\]](#)

DatabaseName	TableName
IndexName	IndexNumber
ConstraintType	ConstraintText
ConstraintCollation	CollationName
CreatorName	CreateTimeStamp
CharSetID	SessionMode
ResolvedCurrent_Date	ResolvedCurrent_TimeStamp
DefinedMaxPartitions	MaxCombinedPartitions
PartitioningLevels	ColumnPartitioningLevel

Example:

List all of the partitioning expression constraints for all tables in the current database.

```
SELECT TableName           AS "Table Name"
      ,ColumnPartitioningLevel AS "CP"
      ,PartitioningLevels   AS "# PLevels"
      ,ConstraintText       AS "Constraint Text"
FROM   DBC.PartitioningConstraintsV
WHERE  DatabaseName = USER;
```

Results:

Table Name	CP	# PLevels	Constraint Text
Orders_RP	0	1	CHECK ((RANGE_N(orderdate BETWEEN DATE '200
Orders_RP_ML	0	2	CHECK (/*02*/ RANGE_N(orderdate BETWEEN DATE
Orders_CP	1	1	CHECK (/*01 02 01*/ PARTITION#L1 /*1 12+65522*/ =1)
Orders_CP_RP	2	2	CHECK (/*02 08 02*/ RANGE_N(orderdate BETWEEN

DBC.PartitioningConstraintsV provides information about index constraints, specifically a partitioning expression constraint. A partitioning expression is an implied index constraint.

A ConstraintType = Q indicates a partitioned primary index.

The ConstraintText indicates the partitioning constraint. The general format of the text will be:

CHECK ((<partitioning-expression>) BETWEEN 1 AND <max>)

```
CHECK ((RANGE_N("salesdate" BETWEEN DATE '2015-01-01' AND DATE '2019-
12-31' EACH INTERVAL '1' DAY)) BETWEEN 1 and 65535)
```

The DBC.PartitioningConstraintsV is new starting with Teradata 14.0. The previous view was named DBC.IndexConstraints and is still available. However, it is recommended to use the new view as it contains additional information.

Show Table Checks View

Provides information about check constraints at the table level and “named” column constraints.

[DBC.ShowTblChecks\[V\]\[X\]](#)

DatabaseName	TableName	CheckName
TblCheck	CreatorName	CreateTimeStamp

Example:

Display table constraint information.

Results:

```
SELECT  TableName (CHAR(10))
        ,CheckName (CHAR(10))
        ,TblCheck
FROM    DBC.ShowTblChecksV
WHERE   DatabaseName = 'TFACT';
```

TableName	CheckName	TblCheck
DEPARTMENT	Dept_Chk1	CONSTRAINT "Dept_Chk1" CHECK ("Deptnum
EMPLOYEE	Emp_Chk1	CONSTRAINT "Emp_Chk1" CHECK ("Employee
JOB	?	CHECK ("Jobcode" >= 3000)

Note:

The first two are named constraints and the third is an unnamed constraint. All three of these constraints were created at the table level.

The ShowTblChecks view displays database table constraint information. The view features CreatorName and CreateTimeStamp columns that display the name of the user who created an object, and the date and time he or she created it. The LastAlterName and LastAlterTimeStamp columns list the name of the last user to modify an object, as well as the date and time.

Column definitions for this view include:

<u>Column</u>	<u>Definition</u>
DatabaseName	Names of databases that contain tables with table-level checks.
TableName	Table names that have table-level constraints.
CheckName	Table-level check name.
TblCheck	Returns the text for the table level or named column level check constraint.
CreatorName	Name of the user who created the object.
CreateTimeStamp	The time the object was created.

Show Column Checks View

Provides information about unnamed column check constraints.

[DBC.ShowColChecks\[V\]\[X\]](#)

DatabaseName	TableName	ColumnName
ColCheck	CreatorName	CreateTimeStamp

Example:

Show information about column constraints for a database.

```
SELECT  TableName    (CHAR(10))
        ,ColumnName  (CHAR(10))
        ,ColCheck
FROM    DBC.ShowColChecksV
WHERE   DatabaseName = 'TFACT';
```

Results:

TableName	ColumnName	ColCheck
EMP_2	employeenumber	CHECK ("employeenumber" >= 100000)
DEPT_2	deptnumber	CHECK ("deptnumber" >= 1000)
JOB_2	jobcode	CHECK ("jobcode" >= 3000)

Note:

A second set of Employee, Department, and Job tables were created with unnamed CHECK constraints at the column level.

The ShowColChecks view displays database column constraint information for unnamed column level constraints. The view features CreatorName and CreateTimeStamp columns that display the name of the user who created an object and the date and time he or she created it.

Column definitions for this view include:

<u>Column</u>	<u>Definition</u>
DatabaseName	Names of databases that contain tables with column-level checks.
TableName	Table names that have column-level constraints.
ColumnName	Names of columns that contain column-level check constraints.
ColCheck	Returns text for the column-level check condition.
CreatorName	Name of the user who created the object.
CreateTimeStamp	The time the object was created.

Triggers View

Provides information about event-driven triggers attached to a single table and stored in the database.

[DBC.Triggers\[V\]\[X\]](#)

DatabaseName	SubjectTableDataBaseName	TableName	TriggerName	EnabledFlag
ActionTime	Event	Kind	OrderNumber	TriggerComment
RequestText	CreatorName	CreateTimeStamp	LastAlterName	LastAlterTimeStamp
AccessCount **	LastAccessTimeStamp **	CreateTxtOverflow	VTEventType	TTEventType

** Optionally updated

Example:

Show if the trigger is enabled, when it fires, the type of statement that fires it, and the kind.

```
SELECT  TableName      (CHAR(10)) AS TName
        ,TriggerName   (CHAR(12))
        ,EnabledFlag   AS "Enabled"
        ,ActionTime    AS Action
        ,Event
        ,Kind
FROM    DBC.TriggersV
WHERE   DatabaseName = DATABASE;
```

Results:

<u>Tname</u>	<u>TriggerName</u>	<u>Enabled</u>	<u>Action</u>	<u>Event</u>	<u>Kind</u>
Employee	Raise_Trig	Y	A	U	R

The Triggers view provides information about event-driven, specialized procedures attached to a single table and stored in the Teradata database.

Note: A positioned (updateable cursor) UPDATE or DELETE is not allowed to fire a trigger and generates an error. In addition, the FastLoad and MultiLoad utilities return an error if you have any triggers enabled on the target table.

Column definitions for this view include:

<u>Column</u>	<u>Definition</u>
ActionTime	Indicates when the triggered action fires. B = Before trigger statement changes the table. A = After trigger statement changes the table.
Event	Indicate which of the following SQL statements fires the trigger: U = UPDATE I = INSERT D = DELETE
Column Position	Position of a column within an index. This value may be greater than one if the column is part of a composite index.

All Temporary Tables View

Provides information about all global temporary tables materialized in the system.

[DBC.AllTempTables\[V\]\[X\]](#)

HostNo	SessionNo
UserName	B_DatabaseName
B_TableName	E_TableID

Example:

Show all temporary tables materialized in the system.

```
SELECT HostNo
       ,SessionNo
       ,UserName (CHAR(10))
       ,B_DatabaseName AS "DataBase"
       ,B_TableName AS "Table Name"
FROM   DBC.AllTempTablesV;
```

Results:

HostNo	SessionNo	UserName	Database	Table Name
01	20887	TFACT02	PD	GT_DEPTSALARY
01	20908	TFACT01	PD	GT_DEPTSALARY

This view provides information about all global temporary tables materialized in the system.

Global Temporary Tables

Use global temporary tables to store temporary, immediate results from multiple queries into working tables. To create a global temporary table, you must state the keywords GLOBAL TEMPORARY in the CREATE TABLE statement. The temporary table defined during the CREATE TABLE statement is referred to as the base temporary table.

When referenced in an SQL session, a local temporary table is materialized with the exact same definition as the base table. Once the temporary table is materialized, subsequent DML statements referring to that table are mapped to the materialized instance.

Note: After you create a global temporary table definition, use the INSERT statement to create a local instance of the global temporary table to use during the session.

Temporary versus Permanent Tables

Temporary tables are different than permanent tables in the following ways:

- They are always empty at the start of a session.
- Their contents cannot be shared by other sessions.
- You can empty them at the end of each transaction.
- The system automatically drops them at the end of each session.

Referential Integrity Views

Views that can be used to provide information about Referential Integrity are:

View	Description
DBC.Tables[V][X]	Can be used to list parent and child counts for tables.
DBC.All_RI_Children[V][X]	Information about tables in child-parent order. Also identifies if RI constraint is consistent or inconsistent .
DBC.All_RI_Parents[V][X]	Similar to above view but provides Information in parent-child order.
DBC.Databases2[V][X]	Can be used to identify child tables with unresolved reference constraints .

Note: Additional RI views are listed on this slide.

This slide identifies a number of views that can be used to list tables with referential integrity and the state of the referential integrity on the tables. Additional views that specifically provide referential integrity information are listed below. The effects of referential integrity on the database can be seen in the series of views identified with the letters "RI". The X views were implemented in Teradata V2R6.0.

VIEW NAME

DESCRIPTION

DBC.All_RI_Children[V][X]	This view shows the Referential Integrity Constraints defined in the database, from the Child-Parent point of view.
DBC.All_RI_Parents[V][X]	This view shows the same information as the above view, but from the Parent-Child perspective.
DBC.RI_Distinct_Children[V][X]	Provides information about tables in child-parent order without the duplication that could result from multi-column foreign keys.
DBC.RI_Distinct_Parents[V][X]	Provides information about tables in parent-child order without the duplication that could result from multi-column foreign keys.
DBC.RI_Child_Tables[V][X]	Provides information about tables in child-parent order. It is similar to the All_RI_Children view but returns the internal Ids of databases, tables, and columns.
DBC.RI_Parent_Tables[V][X]	Provides information about all tables in parent-child order. It is similar to the All_RI_Parents view but returns the internal IDs of databases, tables, and columns instead of names.

Using the DBC.Tables View

The DBC.Tables[V][X] views can be used to list parent and child counts for tables.

- **ParentCount** – how many foreign keys does a table have or how many parents does the table reference?
- **ChildCount** – how many foreign keys reference this table or how many children does the table have?

Example:

List the tables objects in the database
TFACT and identify parent and child counts.

```
SELECT      TableName
           ,TableKind
           ,ParentCount
           ,ChildCount
FROM        DBC.TablesV
WHERE       DatabaseName = 'TFACT'
AND         TableKind = 'T'
ORDER BY    2, 1 ;
```

Results:

<u>TableName</u>	<u>TableKind</u>	<u>ParentCount</u>	<u>ChildCount</u>
Department	T	1	1
Employee	T	3	3
EmpPhone	T	1	0
Job	T	0	1
SalaryLog	T	0	0

The DBC.Tables[V][X] views (described previously) provide parent and child counts for tables within a database. The examples on the following pages are based on the following ALTER TABLE statements.

```
ALTER TABLE Employee ADD CONSTRAINT emp_dept_ref
FOREIGN KEY (deptnumber) REFERENCES
    Department (deptnumber);
```

```
ALTER TABLE Employee ADD CONSTRAINT emp_job_ref
FOREIGN KEY (jobcode) REFERENCES
    Job (jobcode);
```

```
ALTER TABLE Employee ADD CONSTRAINT emp_mgr_ref
FOREIGN KEY (empmgrnumber) REFERENCES
    Employee (employeenumber);
```

```
ALTER TABLE Department ADD CONSTRAINT dept_mgr_ref
FOREIGN KEY (deptmgrnumber) REFERENCES
    Employee (employeenumber);
```

```
ALTER TABLE EmpPhone ADD CONSTRAINT phone_emp_ref
FOREIGN KEY (employeenumber) REFERENCES
    Employee (employeenumber);
```

Referential Integrity States

The status of a Referential integrity constraint is classified as follows:

- Unresolved reference constraint – the FK exists, but the PK does not.
 - Creating a table with an FK before creating the table with Parent Key (PK).
 - Restoring a table with an FK and the table with the PK does not exist or hasn't been restored.
- Inconsistent reference constraint – both the FK and the PK exist, but the constraint is marked as inconsistent.
 - When either the child or parent table is restored, the reference constraint for the child table is marked as inconsistent.
 - no inserts, updates, deletes or table changes are allowed
- Consistent reference constraint – both the FK and the PK exist and are considered consistent, but FK values without a corresponding PK value are identified as “invalid rows”.
 - Typically occurs when the reference constraints are created on already populated tables or ...
 - Following a REVALIDATE REFERENCES command after a restore of either the child or parent table.

This slide identifies three states that are associated with a Referential Integrity constraint.

DBC.All_RI_Children View

This view is used to identify tables with RI in child-parent order and can also be used to show if the RI constraint is consistent or **inconsistent**.

[DBC.All_RI_Children\[V\]\[X\]](#)

IndexID	IndexName	ChildDB	ChildTable
ChildKeyColumn	ParentDB	ParentTable	ParentKeyColumn
InconsistencyFlag	CreatorName	CreateTimeStamp	

Example:

```
SELECT IndexID (FORMAT 'z9') AS ID
,IndexName ,ChildTable ,ChildKeyColumn
,ParentTable ,ParentKeyColumn
,InconsistencyFlag AS ICF
FROM DBC.All_RI_ChildrenV
WHERE ChildDB = 'PD' ORDER BY 3, 4 ;
```

Options to handle inconsistent references:

- [ALTER TABLE tname DROP INCONSISTENT REFERENCES](#); – FK constraints are dropped – use the ALTER TABLE command to create new references constraints.
- Use the [REVALIDATE REFERENCES](#) command (ARC facility).

Results:

ID	IndexName	ChildTable	ChildKeyColumn	ParentTable	ParentKeyColumn	ICF
0	dept_mgr_ref	Department	dept_mgr_number	Employee	employee_number	N
0	emp_dept_ref	Employee	dept_number	Department	dept_number	Y
8	emp_mgr_ref	Employee	emp_mgr_number	Employee	employee_number	Y
4	emp_job_ref	Employee	job_code	Job	job_code	Y
0	phone_emp_ref	Emp_Phone	employee_number	Employee	employee_number	N

The DBC.All_RI_Children[X] views provide information about all tables in child-parent order.

A table can have many referential constraints defined. When either the child or parent table is restored, these constraints are marked **inconsistent**. The DBC.All_RI_Children view provides information about reference constraints.

The REVALIDATE REFERENCES FOR statement validates these inconsistent constraints against the target table.

If inconsistent constraints remain after a REVALIDATE REFERENCES FOR statement has been executed, the SQL statement ALTER TABLE DROP INCONSISTENT REFERENCES must be used to remove them.

REVALIDATE REFERENCES FOR creates error tables containing information about data rows that failed referential constraint checks.

DBC.Databases2 View

The DBC.Databases2[V][X] views provide information about [unresolved reference constraints](#).

Unresolved reference constraints are caused by:

- Creating a table with a Foreign Key before creating the table with Parent Key (Primary Key).
- Restoring a table with a Foreign Key and the Parent Key (Primary Key) table does not exist or hasn't been restored.

DBC.Databases2

DatabaseName	DatabaseID	UnresolvedRCount
--------------	------------	------------------

Example:

List all databases with unresolved references.

```
SELECT DatabaseName
       ,DatabaseId
       ,UnresolvedRCount
FROM   DBC.Databases2V
WHERE  UnresolvedRCount > 0;
```

Results:

DatabaseName	DatabaseID	UnresolvedRCount
PD	0000FE03	2

Restore the Parent Table to “resolve” the references constraint, but the references constraint is marked as inconsistent.

The DBC.Databases2[V][X] views provide ID definition information about databases and provide a count of unresolved reference constraints for any tables within the database.

It is similar to the Databases view but returns the ID of the database and Referential Integrity (RI) information instead of the other information (creator name, owner name, etc.) provided by the Databases view. You can control who has access to internal ID numbers by limiting the access to the Databases2 view while allowing more users to access the names via the Databases view.

Example

The SQL request on this slide uses the Databases2 view to find databases that have tables with unresolved references.

The columns selected are:

DatabaseName	Returns the name of a database with the indicated count of unresolved references.
DatabaseId	Returns the ID of the database with the indicated count of unresolved references.
UnresolvedRCount	Returns the total number of unresolved Referential Integrity (RI) constraints in the database.

If one table has 2 unresolved reference constraints, then the count is 2.

Time Stamps in Data Dictionary

The Teradata Database features a time stamp in the Data Dictionary tables.

- CreateTimeStamp, CreatorName, LastAlterTimeStamp, LastAlterName

This feature can help system administration tasks by providing a means to identify objects recently updated, obsolete objects, etc. and who altered the objects.

Example:

List all tables that have been altered in January of 2019.

```
SELECT TRIM(DatabaseName) || '.' || TableName
      AS "Qualified Name"
      ,LastAlterName      AS "User Name"
      ,LastAlterTimeStamp AS "Last Alter Date & Time"
FROM   DBC.TablesV
WHERE  EXTRACT (YEAR FROM LastAlterTimeStamp) = 2019
AND    EXTRACT (MONTH FROM LastAlterTimeStamp) = 1
ORDER BY 1, 2 ;
```

Results:

Qualified Name	User Name	Last Alter Date & Time
HR_Tab.Job	tfact03	2019-01-23 09:35:36
HR_Tab.Location	tfact03	2019-01-23 13:29:49
HR_Tab.Raise_Trig	Sysdba	2019-01-20 10:07:19
HR_Tab.Salary_Log	Sysdba	2019-01-20 10:07:19
TFACT.New_Sales	tfact03	2019-01-24 18:35:47

The Teradata database includes a time stamp in most of the data dictionary tables.

The time stamp feature is meant to facilitate and enhance your system administration tasks by providing a means to identify obsolete objects, and clean up and recapture space. Time stamps also help you determine when a change to an object occurred for system maintenance activities and problem investigations.

Time Stamp Field Definitions:

Create Time Stamp	The time the object was created in ANSI TimeStamp Format.
CreateUID	User ID of the user who created the object.
CreatorName	Name of the user who created the database, table, or the name of the user's creator.
LastAlterName	Name of the user who last updated the object.
LastAlterTimeStamp	The time the object was last updated in ANSI TimeStamp format.
LastAlterUID	User ID of the user who last updated the object.
AccessCount	The number of times the object was accessed. (Only used if requested.)
LastAccessTimeStamp	The time the object was last accessed in ANSI TimeStamp format.

Summary

- The data dictionary consists of tables, views, and macros stored in system user DBC.
- The Teradata Database software automatically maintains the dictionary/directory tables as you create or drop objects.
 - Database and users
 - Tables, views, macros, triggers, stored procedures, and user-defined functions
 - Columns and indexes
 - Hierarchies
- You should access data dictionary tables via the supplied views.
- Note: To access information about individual objects stored in the data dictionary, use the HELP and SHOW commands.

This slide summarizes some important concepts regarding this module.



Module 4: Review Questions

1. True or False: The DBC.DatabasesV view only contains information about databases; users are not included in this view.
2. True or False: The DBC.UsersV view only contains information about users; databases are not included in this view.
3. True or False: The DBC.ShowTblChecksV view provides information about table level check constraints, unnamed and named column check constraints.
4. True or False: All of the data dictionary tables are Fallback protected.
5. If a child table exists and the parent table doesn't, the reference constraint is marked as _____.
 - a. Invalid
 - b. Missing
 - c. Unresolved
 - d. Inconsistent
6. After executing the ALTER TABLE ... ADD FOREIGN KEY ... statement, Foreign Key values that are missing in the parent table are marked in an error table and are known as _____ rows.
 - a. Invalid
 - b. Missing
 - c. Unresolved
 - d. Inconsistent

Check your understanding of the concepts discussed in this module by completing the review questions as directed by your instructor.



Lab Exercise 4-1 (1 of 3)

teradata.

Purpose

In this lab, you will view information in the data dictionary using various Data Dictionary views.

What you need

SELECT Access to the Data Dictionary views.

Tasks

1. Using the DBC.DBCInfoV view, find the release and version of the system you are logged on:

Release (PDE) _____ Version (Teradata) _____

2. Using the DBC.ChildrenV view, list your parents' user names.

3. Using the DBC.DatabasesV view, find your:

Immediate parent's name _____
Creator's name _____
Default account code _____
Perm space limit _____
Spool space limit _____
Temp space limit _____

Notes about DBC.DBCInfoV columns:

- The Release column provides the PDE release number.
- The Version column provides the Teradata software version number.



Lab Exercise 4-1 (2 of 3)

4. Using the DBC.UsersV view, find your:

Default database name _____
Default collation sequence _____
Default date format _____
Create time stamp _____
Last password modification date _____

OPTIONAL: SHOW this view. Note the WHERE conditions. (Remember, this is a restricted view, even though it does not have an [X] suffix.)

5. Using the DBC.TablesV view, find the number of tables in the DD/D (user DBC) that are:

Fallback protected _____
Not Fallback protected _____

Modify the query to find the number of tables OTHER THAN DD/D (not DBC tables) that are:

Fallback protected _____
Not Fallback protected _____

How many tables (entire system) are defined as No Primary Index tables? _____

SQL Hints:

```
SELECT      Count (*)  
FROM        DBC.TablesV  
WHERE       DatabaseName = 'DBC'  
AND         TableKind IN ('T', 'O') AND ProtectionType = 'F';
```

```
SELECT      Count (*)  
FROM        DBC.TablesV  
WHERE       DatabaseName = 'DBC'  
AND         TableKind IN ('T', 'O') AND ProtectionType = 'N';
```



Lab Exercise 4-1 (3 of 3)

6. Using the DBC.PartitioningConstraintsV view, answer the following questions; hint, use the ColumnPartitioningLevel column to determine if a table is column partitioned or not.

For your user, how many tables have row partitioning and no column partitioning? ____

For your user, how many tables have column partitioning? ____

For the system, how many tables have row partitioning and no column partitioning?? ____

For the system, how many tables have column partitioning? ____

For the system, how many tables have column and row partitioning? ____

What is the constraint type for row partitioned tables? ____

7. Using the DBC.IndicesV view, find the number of tables OTHER THAN Dictionary tables that have non-unique primary indexes (NUPI):

Number of NUPI tables _____

8. Using the DBC.ColumnsV view, find the number of columns in the entire system defined with *default values*:

Number of columns _____

Optional: Modify the query to find the number of objects that have columns defined with *default values*:

Number of tables _____

```
SELECT      TableName, ColumnPartitioningLevel AS "CP",  
            PartitioningLevels AS "# Levels", ConstraintText AS "Text"  
FROM        DBC.PartitioningConstraintsV  
WHERE       DatabaseName = USER;
```

```
SELECT      COUNT(*)  
FROM        DBC.PartitioningConstraintsV  
WHERE       ColumnPartitioningLevel = 0;
```

```
SELECT      COUNT(*)  
FROM        DBC.PartitioningConstraintsV  
WHERE       ColumnPartitioningLevel <> 0  
AND         PartitioningLevels GE 2;
```

```
SELECT      COUNT (DISTINCT(DatabaseName || '.' || TableName))  
FROM        DBC.IndicesV  
WHERE       IndexType IN ('P', 'Q')  
AND         UniqueFlag = 'N'  
AND         DatabaseName NE 'DBC';
```

```
SELECT      COUNT (*)  
FROM        DBC.ColumnsV  
WHERE       DefaultValue IS NOT NULL;
```



Lab Exercise 4-2 (optional: 1 of 2)

Purpose

In this lab, you will establish References constraints between 4 populated tables and view the associated data dictionary entries.

What you need

Populated PD tables and empty tables in your database

Tasks

1. Use INSERT/SELECT to place all rows from the populated PD tables into your empty tables. Verify the number of rows in your tables.

PD.Employee	to populate	Employee	Count = _____
PD.Department	to populate	Department	Count = _____
PD.Job	to populate	Job	Count = _____
PD.EmpPhone	to populate	EmpPhone	Count = _____

2. Use the GRANT statement to GRANT yourself the REFERENCES access rights on the tables.

```
GRANT REFERENCES ON tablename TO username;
```

Use the following SQL for this exercise.

To populate a table:

```
INSERT INTO tablename1
SELECT * FROM tablename2;
```

To grant an Access Right on a table:

```
GRANT REFERENCES ON tablename TO username;
```

To grant an Access Right on a specific column:

```
GRANT REFERENCES (column_name) ON tablename TO username;
```




Lab Exercise 4-2 (optional: 2 of 2)

3. Create a References constraint between the Employee.DeptNumber column and the Department.DeptNumber column.

What is the name of the Employee RI error table? _____

How many rows are in this table? _____

Which department is not represented in the department table? _____

What is the name of the employee with an invalid department number? _____

4. Use the DBC.ALL_RI_ChildrenV view (qualify the ChildDB to your database) and verify this References constraint.

What is the IndexID of this constraint? _____

Use the following SQL for this exercise.

To create a References constraint:

```
ALTER TABLE      child_tablename
  ADD CONSTRAINT  constraint_name
    FOREIGN KEY   (child_name)
    REFERENCES    parent_tablename (parent_column);
```

To select from the DBC.ALL_RI_ChildrenV view:

```
SELECT    Indexid (FORMAT 'z9') AS ID
          ,IndexName
          ,ChildTable
          ,ChildKeyColumn
          ,ParentTable
          ,ParentKeyColumn
FROM      DBC.ALL_RI_ChildrenV
WHERE     ChildDB = USER
ORDER BY  1;
```



Module 5: Space Allocation and Usage

Teradata Vantage Administration

Copyright © 2007–2023 by Teradata. All Rights Reserved.

Objectives

After completing this module, you will be able to:

- Define permanent space, spool space and operating system space requirements
- Estimate system capacity
- Use the AllSpace, DiskSpace, and TableSize views to monitor disk space utilization
- Use Teradata Studio to view database and table space utilization

Permanent Space Terminology

MaxPerm

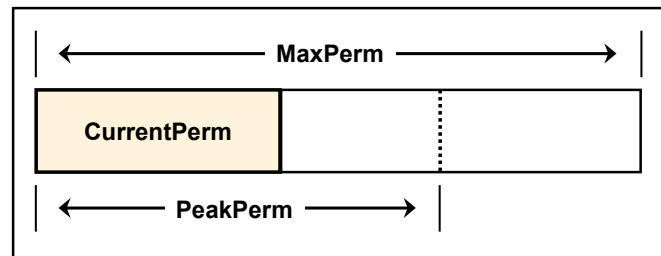
The maximum number of bytes *available* for table, index, permanent journal, stored procedure, and UDF storage in a database or user.

CurrentPerm

The total number of bytes *in use* to store the tables, indexes, permanent journal, stored procedures, and UDFs contained in the database or user.

PeakPerm

The *largest* number of bytes actually used to store data in this database/user since the value was last reset.



MaxPerm

MaxPerm is the maximum number of bytes available for a table, subtables, join indexes, hash indexes, etc. in a system database or user. The number of bytes specified is divided by the number of AMP vprocs in the system. The result is recorded on each AMP vproc and may not be exceeded on that vproc.

CurrentPerm

CurrentPerm is the total number of bytes (including table headers) in use on the database to store the tables, subtables, etc. in a database or user. This value is maintained on each AMP vproc.

PeakPerm

PeakPerm is the largest number of bytes ever actually used to store data in a database or user. This value is maintained on each AMP vproc.

Reset the PeakPerm value to zero by using the ClearPeakDisk Macro supplied in the user DBC.

Note: Space limits are enforced at the database/user level. A database or user may own several small tables or a few large tables as long as they are within the MaxPerm limit set on each AMP.

* Minor exceptions to this rule may occur occasionally. For example, utilities that write data to disk a block at a time (such as FastLoad) check space limits after a block is written.

Spool Space Terminology

MaxSpool

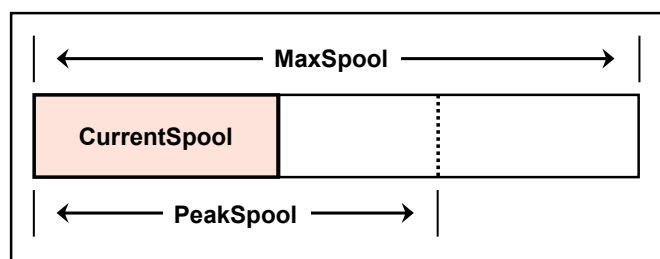
A value used to limit the number of bytes the system will consume to create spool files (including volatile temporary tables) for a user.

CurrentSpool

The number of bytes currently in use for running transactions and/or volatile temporary tables.

PeakSpool

The maximum number of spool bytes used for this user since the value was last reset.



MaxSpool

MaxSpool is a value used to limit the number of bytes the system will allocate to create spool files for a user.

Limit the spool space you allocate to users to reduce the impact of "runaway" transactions, such as accidental product joins.

CurrentSpool

CurrentSpool is the number of bytes in use for running transactions. This value is maintained on each AMP vproc for each user.

PeakSpool

PeakSpool is the maximum number of bytes used by a transaction run for a user since the value was last reset by the ClearPeakDisk Macro (supplied in system user DBC).

Temporary Space Terminology

MaxTemp

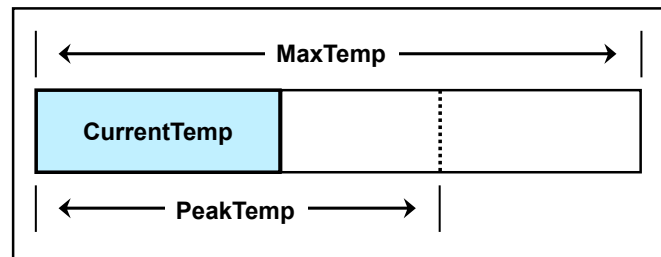
A value used to limit the number of bytes the system will use to store data for global temporary tables for a user.

CurrentTemp

The number of bytes in use for global temporary tables.

PeakTemp

The maximum number of bytes used by global temporary tables for a user since the value was last reset.



Temporary space is released when the user terminates the session or when the user frees the temporary space (e.g., deleting the rows in a global temporary table).

MaxTemp

MaxTemp is a value used to limit the number of bytes the system will use to store data for global temporary tables for a user.

CurrentTemp

CurrentTemp is the number of bytes in use for global temporary tables. This value is maintained on each AMP vproc for each user.

PeakTemp

PeakTemp is the maximum number of bytes used by global temporary tables for a user since the value was last reset by the ClearPeakDisk Macro (supplied in system user DBC).

Resetting Peak Values

The DBC.ClearPeakDisk macro resets PeakPerm, PeakSpool, and PeakTemp values in the DBC.DatabaseSpace table. Starting in Teradata 16.00, this macro also resets the peak values in the DBC.GlobalDBSpace table.

SHOW MACRO DBC.ClearPeakDisk;

```
REPLACE MACRO DBC.ClearPeakDisk
AS ( UPDATE DataBaseSpace
      SET PeakPermSpace = 0,
          PeakSpoolSpace = 0,
          PeakTempSpace = 0 ALL
    ;UPDATE GlobalDBSpace
      SET PeakPermSpace = 0,
          PeakSpoolSpace = 0,
          PeakTempSpace = 0 ALL; );
```

This macro may be used to reset peak values for the next data collection period.

To reset the "peak" values:

EXEC DBC.ClearPeakDisk;

```
*** Update completed. 3911 rows changed.
*** Time was 4 seconds.
```

From time to time, the administrator needs to clear out the values accumulated in the DBC.DatabaseSpace table. These values must be reset to restart the data collection process.

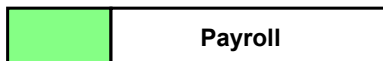
DBC.ClearPeakDisk

The Teradata software provides a macro to reset the PeakPerm, PeakSpool, and PeakTemp values in the DBC.DatabaseSpace table.

Starting in Teradata 16.00, this macro also resets values in the DBC.GlobalDBSpace table.

This macro is normally used to reset the peak values for the next collection period.

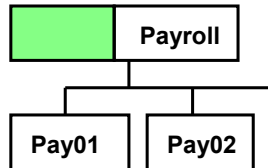
Assigning Perm and Spool Limits



MaxPerm = 25E9, MaxSpool = 50E9 (via a Profile)

Payroll creates the following two users as children of itself.

```
CREATE USER Pay01 AS PASSWORD = abc123, PERM = 6E9, SPOOL = 35E9;
CREATE USER Pay02 AS PASSWORD = xyz456, PERM = 4E9;
```

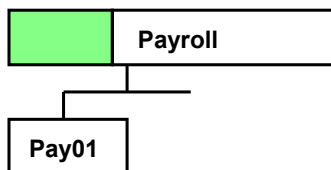


MaxPerm = 15E9 MaxSpool = 50E9

The maximum Spool value may not exceed that of the creator when a new user/database is created.

```
DROP USER Pay02;
```

What is the maximum Spool limit of Pay02?



MaxPerm = 19E9 MaxSpool = 50E9

The Perm space from Pay02 is returned back to the immediate owner.

You define permanent and spool space limits at the database or user level, not at the table level. When you create databases or users, perm space limits are deducted from the available (unused) space of the immediate owner.

Assuming a creator is assigned a SPOOL limit via a Profile, then the spool space limit cannot exceed that of the creator at the time you create a database or user.

The SPOOL and TEMPORARY limits for a new user are determined by the following order of preference (top to bottom).

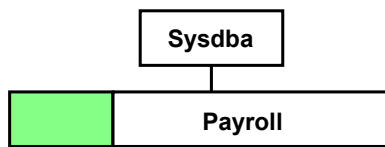
1. SPOOL limit in the PROFILE identified in the CREATE USER statement.
2. SPOOL limit explicitly stated in the CREATE USER statement.
3. SPOOL limit in the PROFILE of the CREATOR.
4. If the creator is not assigned to a PROFILE, then the SPOOL limit is limited to the immediate OWNER of the newly created user (maintained for legacy purposes).

Example

The illustration on this slide illustrates how Teradata manages permanent and spool space. Since there is no statement of spool space for Pay02, its maximum Spool defaults to the limit of its creator which is 50 GB.

The amount of maximum Perm increases and decreases as the owner creates and drops new users. The spool space figure remains constant even when the owner adds and drops users.

Assigning Perm and Spool Limits (cont.)

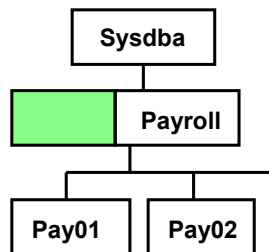


MaxPerm = 10E9, MaxSpool = 100E9 (via a Profile)

MaxPerm = 25E9, MaxSpool = 50E9 (via a Profile)

Sysdba creates the following two users as children of Payroll.

```
CREATE USER Pay01 FROM Payroll AS PASSWORD = abc123, PERM = 6E9, SPOOL = 75E9;
CREATE USER Pay02 FROM Payroll AS PASSWORD = xyz456, PERM = 4E9;
```



The Perm space for Pay01 and Pay02 is deducted from which user?

What is the maximum Spool limit of Pay01?

What is the maximum Spool limit of Pay02?

In this example, Sysdba creates 2 new users as children of Payroll. These two new users are not assigned to a Profile. However, Sysdba has a 100 GB spool space limit via a Profile.

A user, Payroll, has a 25 GB permanent space limit and a 50 GB spool space limit.

Sysdba creates two new users, Pay01 and Pay02, as children of Payroll. After Sysdba creates the new users, Payroll's maximum Perm space drops to 15 GB because Pay01 is assigned 6 GB of maximum Perm space and Pay02 is assigned 4 GB of maximum Perm space.

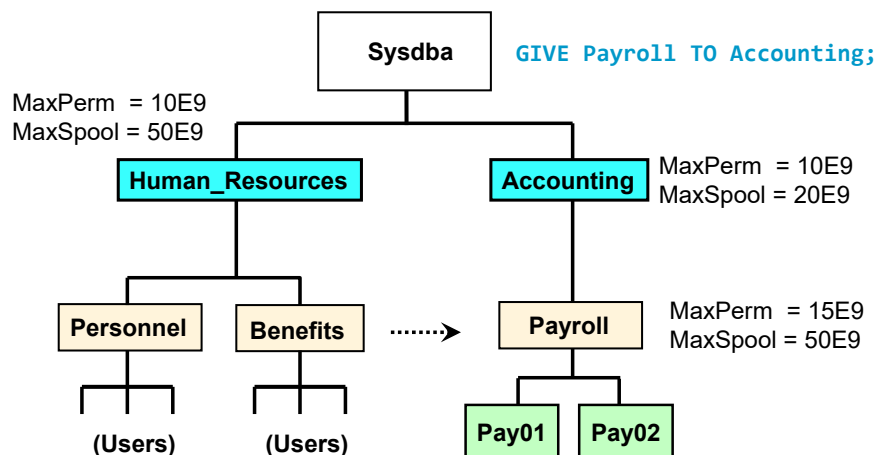
Assume Sysdba has a limit of 100 GB of maximum Spool (via a Profile), therefore, the Sysdba user can create new users and/or databases with a maximum of 100 GB of spool. When Sysdba creates user Pay01, it assigns 75 GB of maximum Spool which is more than Payroll – this is allowed.

Since there is no specification of spool space for Pay02, its maximum Spool defaults to the limit of its creator which is 100 GB.

There are two situations where a child user/database can have more Spool and/or Temporary space than their immediate parent. This is one case and the other is when a user/database is moved (GIVE command) in the system.

Giving One User to Another

- Payroll ownership is transferred (GIVE) to Accounting.
- All descendants (child users/databases, tables, views, etc.) of a “given” object remain with the “given” object.
- The GIVE command may also be used to move Permanent space from one database/user to another database/user.



When you give an object to another object in the hierarchy, all space allocated to that object goes with it. If you drop the object, its perm space is credited to its immediate owner. When you give databases or users, all descendants of the given object remain descendants of the given object.

When you give a database/user to a new owner, the transferred database/owner retains the space limits assigned to them. The only change in the data dictionary for the transferred database/user is the immediate owner's new name.

However, If you drop a database/user, its perm space limit is credited to its immediate owner.

This slide illustrates giving a database/user from one database/user to another such as giving Payroll from Human_Resources to Accounting.

Adjusting Perm Space Limits

You can easily adjust perm space limits. Using the illustration on this slide as an example, you could transfer 10 GB from Human_Resources to Accounting using the following technique:

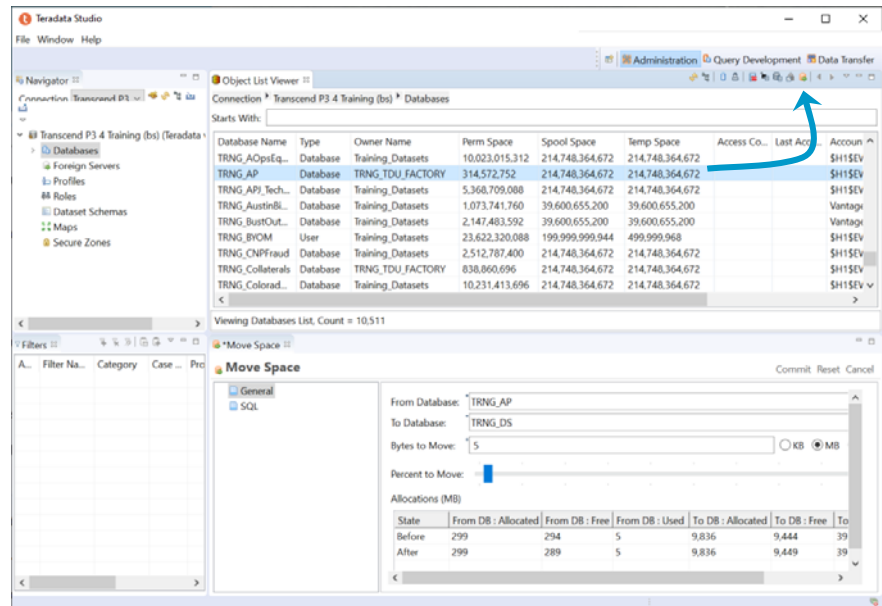
1. CREATE DATABASE TempDB FROM Human_Resources AS PERM = 10E9;
2. GIVE TempDB TO Accounting;
3. DROP DATABASE TempDB;

The database TempDB is NOT shown on this slide, but is used as an example of how space could be transferred from one database to another.

Teradata Studio – Move Space

Right-click and select “Show Child Databases / Users”.

Using the Object List Viewer, right click on a selected object, and then select “Move Space” or click “Move free space between databases” icon.



The Move Space function of Teradata Studio is shown on this slide.

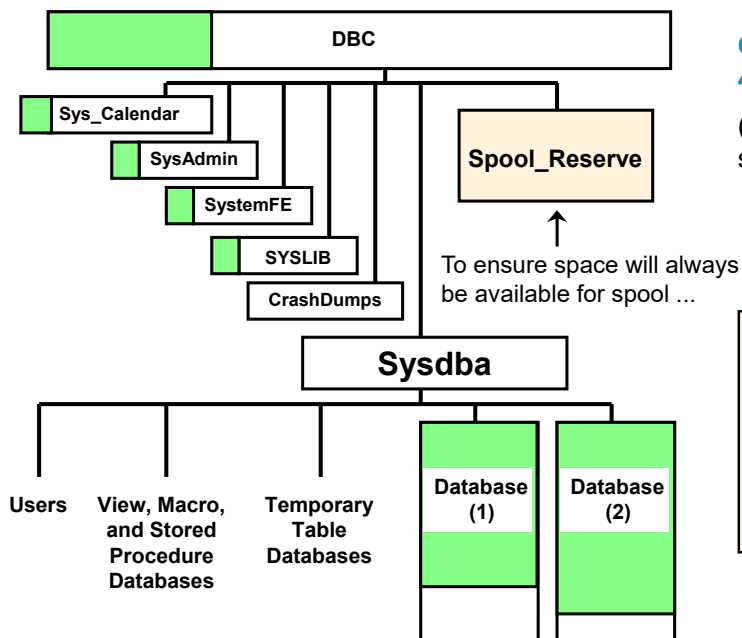
This example illustrates moving 5 MB of Permanent space from the TRNG_AP database to the TRNG_DS database.

The user requires the DROP DATABASE access right on TRNG_AP and the CREATE DATABASE access right on TRNG_DS in order to do this move space operation.

As discussed previously, the following commands would accomplish the same thing.

1. CREATE DATABASE Temp_DB FROM TRNG_AP AS PERM = 5E6 ;
2. GIVE Temp_DB TO TRNG_DS ;
3. DROP DATABASE Temp_DB ;

Reserving Space for Spool



```
CREATE DATABASE Spool_Reserve AS PERM = 40E9 *(HASHAMP()+1);
```

(This example reserves ~ 40 GB of space for spool on each AMP.)

Do not use the Spool_Reserve database to store tables.

User data tables/indexes may occupy up to 50 – 70% of the total disk space. Remember that DBC tables, WAL, etc. also needs space within the system.

Spool space serves as temporary storage for returned rows during transactions that users submit. To ensure that space is always available, you may want to set aside about 20 to 25% of total available space as spool space. To do this, you can create a special database called Spool_Reserve.

Decision support applications should reserve more of the total disk space as reserved spool space since their SQL statements generate larger spool files. Tactical queries/applications often use less spool space because their statements generate smaller spool files.

The above actions guarantee that data tables will never occupy more than 50% to 70% of the total disk space. There is no data stored in Spool_Reserve; therefore, this is space available for spool or temp.

Orphan or Phantom Spool Issues

Spool files (tables) are temporary work tables which are created and dropped as queries are executed. When a query is complete, all of the spool files it used should be dropped automatically.

Like all tables, spool tables require a Table ID. There is a range of tableids exclusively reserved for spool tables (C000 0001 thru FFFF FFFF) and the system cycles through them. If a spool table is not dropped correctly, it remains in existence (which is very unusual). Eventually, the system will cycle through all the table ids and reassign the tableid which is in use by our left over spool table. Usually, the presence of this table is detected, the query which was going to use the tableid is aborted – even though it is innocent of any wrongdoing – and the error message (Failure 2667) is returned to the user and put in the error log. This topic will be addressed in more detail later.

Views for Space Allocation Reporting

View Name

DBC.DiskSpace[V][X]

DBC.TableSize[V][X]

DBC.AllSpace[V][X]

Description

AMP vproc information about disk space usage (including spool) for any database or account.

AMP vproc information about disk space usage (excluding spool) for any database, table or account.

AMP vproc information about disk space usage (including spool) for any database, table, or account.

SUM (CurrentPerm) using the three views

DBC.DiskSpaceV

Table All – 100 MB

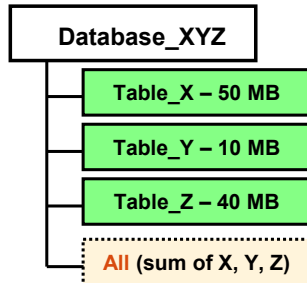
DBC.TableSizeV

Table_X – 50 MB
Table_Y – 10 MB
Table_Z – 40 MB
100 MB

DBC.AllSpaceV

Table_X – 50 MB
Table_Y – 10 MB
Table_Z – 40 MB
Table All – 100 MB
200 MB

Example:



Use the following system views to report current space allocation:

DBC.DiskSpace[V][X]

This view gives AMP vproc information about disk space usage for any database or account. It gets this information from the ALL table.

DBC.TableSize[V][X]

This view gives AMP vproc information about disk space usage (excluding spool) for any table or account.

DBC.AllSpace[V][X]

This view gives AMP vproc information about disk space usage (including spool) for any database, table, or account.

Each of these views references the non-hashed DBC.DataBaseSpace table.

DiskSpace View

Provides AMP Vproc disk space usage at the database level, including spool space.

[DBC.DiskSpace\[V\]\[X\]](#)

Vproc	DatabaseName
AccountName	MaxPerm
MaxSpool	MaxTemp
CurrentPerm	CurrentSpool
CurrentTemp	PeakPerm
PeakSpool	PeakTemp
MaxProfileSpool	MaxProfileTemp

This view selects values for TableName "ALL" (TableID = '000000000000' XB).

Example:

Calculate the percentage of permanent space used in databases and users.

```
SELECT DatabaseName
,CAST (SUM (MaxPerm) AS FORMAT 'zzz,zzz,zz9')
,CAST (SUM (CurrentPerm) AS FORMAT 'zzz,zzz,zz9')
,CAST (((SUM (CurrentPerm))/
        NULLIFZERO (SUM(MaxPerm)) * 100)
        AS FORMAT 'zz9.99%') AS "% Used"
FROM DBC.DiskSpaceV
GROUP BY 1
ORDER BY 4 DESC ;
```

Results:

DatabaseName	Sum(MaxPerm)	Sum(CurrentPerm)	% Used
DS	209,715,200	185,413,632	88.41%
TFACT	104,857,600	45,396,480	43.29%
AP	20,000,000	3,978,240	19.89%
Sys_Calendar	15,000,000	2,647,040	17.65%
SystemFe	1,000,000	123,904	12.39%

The DiskSpace[V][X] view provides AMP vproc information about disk space usage at the database level. This view includes spool space usage.

The PeakSpool column can be used to determine the maximum amount of spool space that a user has used (via DatabaseName and AccountName columns) .

Example

The SELECT statement on this slide calculates the percentage of disk space used in the owner's database. The result displays a partial report with five rows. The DS database has the highest percentage of utilized space at 88.41%. SystemFE has the lowest at 12.39%.

Note: In the statement, use NULLIFZERO to avoid a divide exception.

TableSize View

Provides AMP Vproc disk space usage at table level.

[DBC.TableSize\[V\]\[X\]](#)

Vproc	DatabaseName	AccountName
TableName	CurrentPerm	PeakPerm

Excludes TableName "All" (TableID = '000000000000' XB)

Example:

Look at table distribution across AMPs.

```
SELECT Vproc
      ,CAST (TableName AS FORMAT 'X(15)')
      ,CurrentPerm
      ,PeakPerm
FROM   DBC.TableSizeV
WHERE  DatabaseName = USER
ORDER BY TableName, Vproc ;
```

<u>Vproc</u>	<u>TableName</u>	<u>CurrentPerm</u>	<u>PeakPerm</u>
0	Table2	127,488	253,440
1	Table2	127,488	253,440
2	Table2	127,488	253,952
3	Table2	127,488	253,952
4	Table2	128,000	255,488
5	Table2	128,000	255,488
6	Table2	126,976	251,904
7	Table2	126,976	251,904
0	Table2_nupi	95,232	95,232
1	Table2_nupi	95,232	95,232
2	Table2_nupi	145,920	145,920
3	Table2_nupi	145,920	145,920
4	Table2_nupi	123,904	123,904
5	Table2_nupi	123,904	123,904
6	Table2_nupi	145,408	145,408
7	Table2_nupi	145,408	145,408

The TableSize view is a Data Dictionary view that provides AMP Vproc information about disk space usage at a table level, optionally for tables the current User owns or has SELECT privileges on.

Table Skew Percentage

The following SQL can be executed to determine the skew percentage for tables in a database.

```
SELECT
  DataBaseName,
  TableName,
  SUM(CurrentPerm) AS CurrentPerm,
  (1 - AVG(CurrentPerm) / MAX(CurrentPerm)) * 100 AS SkewFactor
FROM DBC.TableSizeV
WHERE DatabaseName = USER
GROUP BY 1, 2;
```

Example

The SELECT statement on this slide looks for poorly distributed tables by displaying the CurrentPerm figures for a single table on all AMP vprocs.

The result displays one table, **Table2**, which is evenly distributed across all AMP vprocs in the system. The CurrentPerm figure is nearly identical across all vprocs. **Table2_nupi**, is poorly distributed. The CurrentPerm figures range from 95,232 bytes to 145,920 bytes on different AMP vprocs.

AllSpace View

AMP vproc disk space usage at the database AND table level.

[DBC.AllSpace\[V\]\[X\]](#)

Vproc	DatabaseName	AccountName
TableName	MaxPerm	MaxSpool
MaxTemp	CurrentPerm	CurrentSpool
CurrentTemp	PeakPerm	PeakSpool
PeakTemp	MaxProfileSpool (13.0)	MaxProfileTemp (13.0)

Includes TableName "All" (TableID = '000000000000' XB)

Example:

List all tables (by Vproc) contained in the user's space.

```
SELECT      Vproc
            ,CAST (TableName AS FORMAT 'X(15)')
            ,MaxPerm
            ,CurrentPerm
FROM        DBC.AllSpaceV
WHERE       DatabaseName = USER
ORDER BY    TableName, Vproc;
```

<u>Vproc</u>	<u>TableName</u>	<u>MaxPerm</u>	<u>CurrentPerm</u>
0	All	1,250,000	222,720
1	All	1,250,000	222,720
2	All	1,250,000	273,408
3	All	1,250,000	273,408
:	:	:	:
0	Table2	0	127,488
1	Table2	0	127,488
2	Table2	0	127,488
3	Table2	0	127,488
:	:	:	:
0	Table2_nupi	0	95,232
1	Table2_nupi	0	95,232
2	Table2_nupi	0	145,920
3	Table2_nupi	0	145,920
:	:	:	:

The AllSpace[V][X] view provides AMP vproc information about disk space usage at the database and table level. This information includes the "All" table.

Example

The SELECT statement on this slide lists the MaxPerm and CurrentPerm figures for each table in the user's space. The result displays three table names: **All**, **Table2**, and **Table2_nupi**.

The "All" table represents all tables that reside in the user's space. The MaxPerm figure for "All" is the amount of permanent space defined for that user. There are only two tables in this user's defined space.

Note: The "Table2 and Table2_nupi" tables display zero bytes in the MaxPerm column. This is because tables do not have MaxPerm space, only databases and users do, as represented by the "All" table.

DataBaseSpace Table

Four columns from
DBC.DataBaseSpace:

Notes:

	1,019,904
+	1,020,928
	<u>2,040,832</u>
	2,040,832
	1,019,904
+	1,020,928
	<u>4,081,664</u>

DatabaseID	TableID	Vproc	CurrentPermSpace	
UPI				
00001404	000000000000	0	}Table “All” Sum is 2,040,832	222,720
00001404	000000000000	1		222,720
00001404	000000000000	2		273,408
00001404	000000000000	3		273,408
00001404	000000000000	4		251,904
00001404	000000000000	5		251,904
00001404	000000000000	6		272,384
00001404	000000000000	7		272,384
00001404	0000260C0000	0	}Table2 Sum is 1,019,904	127,488
00001404	0000260C0000	1		127,488
00001404	0000260C0000	2		127,488
00001404	0000260C0000	3		127,488
00001404	0000260C0000	4		128,000
00001404	0000260C0000	5		128,000
00001404	0000260C0000	6		126,976
00001404	0000260C0000	7		126,976
00001404	0000270C0000	0	}Table2_nupi Sum is 1,020,928	95,232
00001404	0000270C0000	1		95,232
00001404	0000270C0000	2		145,920
00001404	0000270C0000	3		145,920
00001404	0000270C0000	4		123,904
00001404	0000270C0000	5		123,904
00001404	0000270C0000	6		145,408
00001404	0000270C0000	7		145,408

The DataBaseSpace table tracks and stores information about disk space usage for objects in the Teradata system. The information is updated as users create new databases and add tables to them. This slide illustrates four columns from DataBaseSpace.

DataBaseSpace Columns

These columns are used by the AllSpace, DiskSpace and TableSize views to provide space details.

DatabaseID

A DatabaseID is a unique identification number assigned to a database when the CREATE DATABASE/USER statement is issued. In this example, **DatabaseID='00001404'XB**

TableID

TableID is a unique identification number assigned to a table when the CREATE TABLE statement is issued. Each database has a table ID 000000000000 (or table ALL).

Vproc

Vproc is the **logical vproc number** where a table is stored. Since tables are evenly distributed across all AMP vprocs, a single table is stored on several different vprocs.

CurrentPermSpace

CurrentPermSpace is the number of bytes of permanent space taken up on a specific vproc by that table. Table ALL represents the entire database.

Different Views — Different Results

```
SELECT MAX(CurrentPerm)
      ,SUM(CurrentPerm)
FROM   DBC.DiskSpaceV
WHERE  DatabaseName = USER;
```

Maximum (CurrentPerm)	Sum (CurrentPerm)
273,408	2,040,832

Values only represent table ALL.

```
SELECT MAX(CurrentPerm)
      ,SUM(CurrentPerm)
FROM   DBC.TableSizeV
WHERE  DatabaseName = USER;
```

Maximum (CurrentPerm)	Sum (CurrentPerm)
145,920	2,040,832

Values represent all of the actual tables except table ALL.

```
SELECT MAX(CurrentPerm)
      ,SUM(CurrentPerm)
FROM   DBC.AllSpaceV
WHERE  DatabaseName = USER;
```

Maximum (CurrentPerm)	Sum (CurrentPerm)
273,408	4,081,664

Values represent all of the actual tables and table ALL.

Conflicting Results

It would seem logical that query results would be the same for any of the preceding views, since they all use the same underlying table. However, query results can differ depending upon which view is used.

When the MAX (CurrentPerm) and SUM (CurrentPerm) is selected from the AllSpace, DiskSpace, and TableSize views, our results will differ. The SUM (CurrentPerm) value from the DBC.AllSpace view represents the Sum of “All” tables (i.e., the database total) *plus* the sum of *each* table in the database. The results are misleading.

The recommendation is use the DBC.DiskSpace view for queries at the *database* level and use the DBC.TableSize view for queries at the *table* level. Avoid using aggregate functions (e.g., SUM) with the AllSpaceV view.

Sum(CurrentPerm)

The DiskSpace view displays 2,040,832 bytes of total permanent space used which reflects the total number of bytes stored for table ALL.

The TableSize view also displays 2,040,832 bytes of total CurrentPerm. This figure comes from the individual tables stored within the same database.

However, the AllSpace view displays 4,081,664 bytes which is the total of all tables plus table ALL. Since table name ALL already has the database total, the resulting sum is double what it should be.

Global Space Accounting

The space accounting and space management infrastructure was enhanced (Teradata 16.00) to manage the space limits at a [global level \(system level\)](#) and [augment the AMP-level space management](#). This was in preparation for the Teradata 16.10 MAPS feature.

What is New?

- [DBC.GlobalDBSpace](#) table
- [GlobalDBSpaceV\[X\]](#) and [AllGlobalSpaceV\[X\]](#) Views
- The SQL CREATE USER/DATABASE command is extended to specify a [skew limit percentage](#) that allows the max AMP space usage to be above the per-AMP quota.

A system soft limit can be established (DBS Control) as well as a SKEW factor at the database/user level. The two limits can be cumulative. For example:

- Soft limit: 10 percent (set with DBS Control field GlobalSpaceSoftLimitPercent, default is 0 percent)
- Database or global limit: 800 GB
- Per-AMP limit on a 4-AMP system: 200 GB
- Skew factor: 25 percent

Therefore, the database global soft limit is actually 880 MB (or 200 + 20 GB per AMP). Since the skew is 25%, the per AMP limit is 250 GB. However, the cumulation is:

- The global database limit is actually 880 GB
- The per-AMP limit of 250 + 20 = 270 GB.

The space accounting and space management infrastructure has been enhanced to manage the space limits at a global level (system level) and augment the AMP-level space management.

- Global space accounting does “need based” space allocations to AMPs and maintains overall limit. Limits may no longer be at AMP-level.
 - This is controlled by a new “skew” limit defined on space.
 - A “soft limit” is defined to allow space usage to exceed system level max values.
- The SQL CREATE USER/DATABASE command is extended to specify a SKEW limit percentage that allows maximum AMP space usage to be above the per-AMP quota. This can help load jobs complete without getting a per-AMP limit error.

The Skew factor and soft limits can be cumulative. For example, consider the following example:

- Database or global limit: 800 GB
- Soft limit: 10 percent (set with DBS Control)
- Per-AMP limit on a 4-AMP system: 200 GB
- Skew factor: 25 percent

With these numbers, the global limit can exceed the 800 GB limit by 10 percent (to 880 GB). In addition, an AMP can exceed the 200 GB limit by 25% (to 250 GB) if the total across all AMPs does not exceed 880 GB. Therefore, each of the AMPs can also receive an additional 20 GB because of the global soft limit. The cumulative effects can increase the maximum possible space allotment to an AMP to 250 + 20 = 270 GB, as long as the total space used across all AMPs does not exceed 880. An error message is given if either of the following limits are exceeded: the global limit of 880; the per-AMP limit of 270.

Global Space Accounting Example

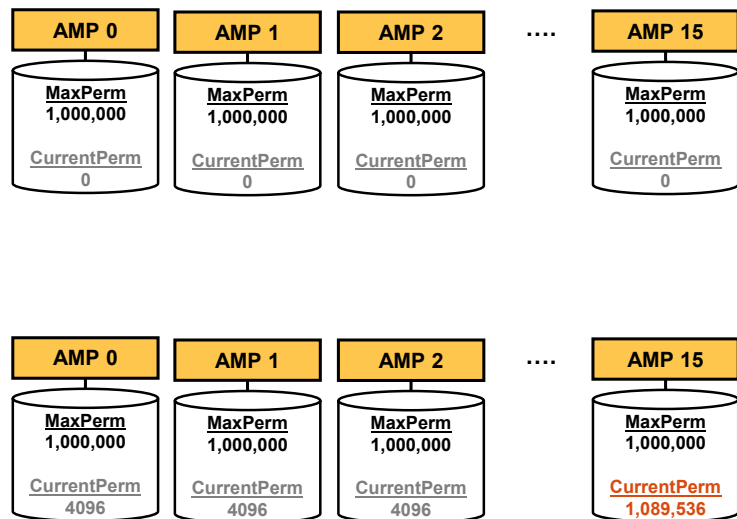
Assume a 16 AMP system.

```
CREATE DATABASE database1 AS PERM = 16E6
      SKEW = 20 PERCENT;
```

Note: Before Teradata 16.00 or with a SKEW = 0, the per-AMP maximum perm is 1,000,000 (1E6).

A new table is created and **all of the inserted rows have the same PI value** and are placed on a same AMP. The other AMPs only have the table header which is 4K.

Note: With a SKEW = 20 PERCENT, the per-AMP maximum perm of 1,000,000 (1E6) can be exceeded.



The following example assumes a 4 AMP system:

```
CREATE DATABASE database2 AS PERM = 40000 SKEW = 10 PERCENT;
CREATE MULTISET TABLE database2.t1 (a INT, b INT);
SELECT      AllocatedPerm, MaxPerm FROM DBC.GlobalDBSpaceV
WHERE       Databasename = 'database2';
```

	<u>AllocatedPerm</u>	<u>MaxPerm</u>
	36000	40000

```
SELECT      Vproc, CurrentPerm, AllocatedPerm, MaxPerm
FROM DBC.DiskSpaceV WHERE Databasename='database2' ORDER BY 1;
```

<u>Vproc</u>	<u>CurrentPerm</u>	<u>AllocatedPerm</u>	<u>MaxPerm</u>
0	4096	9000	10000
1	4096	9000	10000
2	4096	9000	10000
3	4096	9000	10000

Internal details: the system level space entry in DBC.GlobalDBSpace tries to maintain a reserve space in its AllocatedPerm column (the default is 10% as defined in DBS Control internal field #459. Initially the reserve amount was 4000 (40000 – 36000), but when an AMP crosses its AllocatedPerm, the reserve AllocatedPerm at global level drops to below 10% of its max limit. At this point, a de-allocation cycle is triggered in the background which collects all the unused AllocatedPerm from all the AMPs into global pool which decreases the AllocatedPerm at AMP level.

Additional Utilities to View Space Utilization

Teradata Studio – graphical tool to easily view space usage

- Object Viewer
 - Space Summary – current and peak perm usage of the specified table
 - Space by AMP – current and peak perm usage of the specified table by AMP

Viewpoint Space Usage Portlet (shown later in this course)

- By Database – space usage at the database level
- By Vproc – space usage by AMP

Ferret – system utility – started from Viewpoint Remote Console or Supervisor

- ShowSpace – space usage (perm, spool, and temporary) at the system level
- ShowBlocks – allocation of permanent space at the table and subtable level

Question – Why use ShowBlocks to determine space usage at a table level?

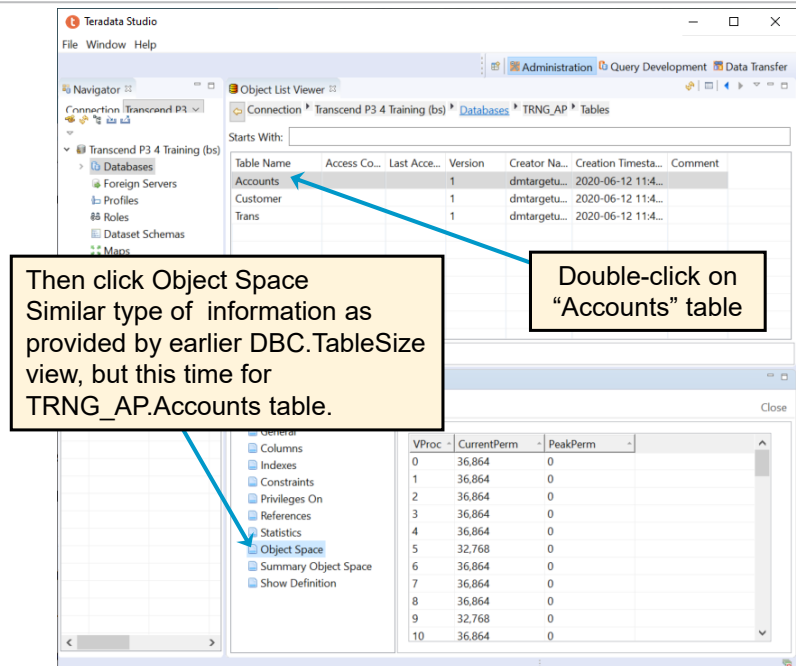
- To see block level detail and/or compression details for large tables.
- To determine how much perm space is currently being used by a secondary index.
 - This level of detail is not available with DBC.TableSizeV and Studio Administrator perspective – only provide current perm space usage at the table level.

Examples of additional tools that may be used to view database and table space utilization are provided in this module.

Teradata Studio – Object Viewer Options

Object Viewer:

- Select an object like a table, view or macro from the Data Source Explorer pane.
- Then select an item in the Object List Viewer pane.
- Right-click object and select, for example, “Show Object Space” Information.



The Object Viewer provides detailed information about database objects you select either in Data Source Explorer or in the Object Viewer. The Object Viewer automatically displays when you select the Administrator perspective. To open it in a different perspective, select Window > Show View in the menu toolbar.

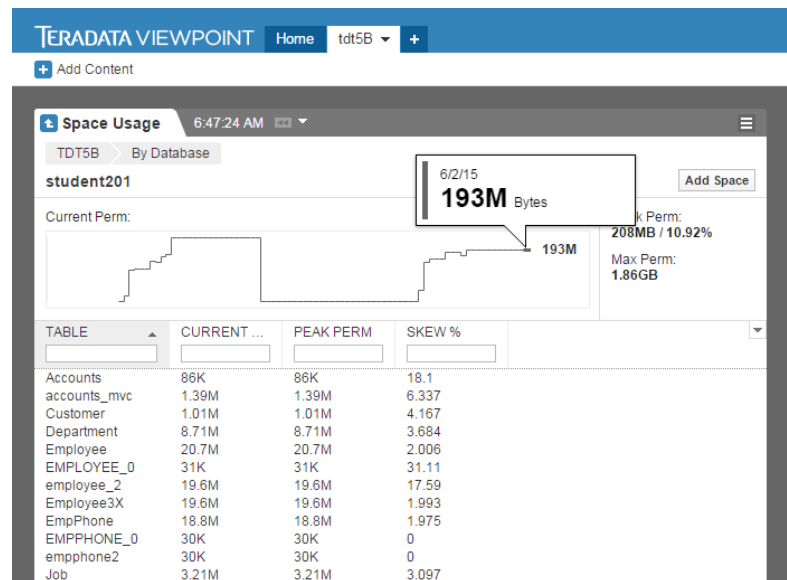
Tabbed Object Views

Each object you select has a view tab labeled with the name of the object. Open the tab to view object details and other related objects. For example, you can drill down on a Database object to review details for Table, View, Macro, Procedure, and Index objects. If you then drill down on the Table object, you are able to view Column, Index, and Trigger objects.

To drill down and view details, click the object name displayed in **Object Viewer** or select the object from the **Data Source Explorer** tree. A new tabbed view opens for that object.

Teradata Viewpoint – Space Usage

The Teradata Viewpoint Space Usage portlet allows a user to view space at the vproc or database level. This example illustrates the detail shown for a specific user.



The **Space Usage** portlet allows you to monitor database disk space usage for Teradata Database, Teradata Aster, and Teradata Hadoop systems. The views and the options vary depending on the system you choose.

For Teradata Database systems, you can monitor database data, such as the perm space, temp space, and spool space, and reallocate permanent disk space. The details view displays permanent space usage over time in information balloons. You can reallocate permanent disk space from one database to another using either the **Space Usage** view or the details view.

Ferret Utility

```
Input Supervisor Command:
> start ferret
start ferret

Started 'ferret' in window 1
at Thu Nov 26 11:17:29 2019

Ferret Utility

Teradata

Release 16.20.34.01 Version 16.20.34.01
Ferret

Waiting for 16 Ferret background tasks to start
All Background tasks have been started
FERRET will run in Workload Def: WD-Default
The SCOPE has been set

Ferret ==>
```

Functions of FERRET covered in this module:

- SHOWSPACE
- SHOWBLOCKS

16 background tasks were started – one for each AMP.

To maintain data integrity, the Ferret utility (File Reconfiguration Tool) enables you to display and set various disk space utilization attributes associated with the Teradata database.

When you select the Ferret utility attributes and functions, it dynamically reconfigures the data on the disks to correspond with the selections.

Depending on the functions, Ferret can operate at the vproc, table, subtable, disk, or cylinder level.

Start Ferret from the DBW connected to the Teradata database. Note that the Teradata database must be in the Logons Enabled state.

The commands within the Ferret utility that we will discuss in this module include:

- SCOPE
- SHOWSPACE
- SHOWBLOCKS

To start the Ferret utility, enter the following command in the Supervisor screen of the DBW:

START FERRET

You will be placed in the interactive partition where the Ferret utility was started.

Ferret SHOWSPACE Command

- What does the * represent after #cyl?

AMP

- What is the typical (per AMP) cylinder fill factor used for Permanent data?

```
> showspace /s
```

SHOWSPACE results for Each AMP

Vproc Num	DSU	Perm Data Cyls				Wal Cyls		Depot Cyls		Spool Cyls				Temp Cyls			
		Avg		% of		% of		Avg		Avg		% of		Avg		% of	
		total	utl	total	Space	total	Space	total	utl	total	utl	total	Space	total	utl	total	Space
		Avail	per	Avail	#Cyl	Avail	#Cyl	per	#Cyl	per	Avail	#Cyl	per	Avail	per	Avail	#Cyl
0	19495	80%	2%	68*	0%	21*	0%	3*	0%	1%	18*	0%	0%	1*			
1	19495	81%	2%	67*	0%	21*	0%	3*	0%	1%	18*	0%	0%	1*			
2	19495	82%	2%	67*	0%	21*	0%	3*	0%	1%	18*	0%	0%	1*			
3	19495	83%	2%	66*	0%	21*	0%	3*	0%	1%	18*	0%	0%	1*			
4	19495	82%	2%	67*	0%	21*	0%	3*	0%	1%	18*	0%	0%	1*			

Showspace options: `showspace /s` Summary listing
`showspace /l` Long listing

The SHOWSPACE command reports the amount of disk cylinder space currently in use and the amount of cylinder space that remains available. Use SHOWSPACE to determine if disk compaction or system expansion is required.

SHOWSPACE is a command you execute from within the Ferret utility. To start the utility, enter **START FERRET** in the Supervisor window. Within the Ferret application window, enter **SHOWSPACE** (upper or lowercase). The Showspace command reports on physical disk utilization.

This slide shows the results of a SHOWSPACE command. Notice the command displays the average utilization per cylinder for permanent and spool space. It displays the percentage of total available cylinders as well as the number of cylinders for all types of space.

Enter an **S** for a summary report that displays only subtotals for all AMP vprocs in the system. This slide shows an example of a Showspace summary report.

In this example, the typical percentage of cylinders used for Permanent data is only 2%.

Enter an **L** for a full report that displays Pdisk and Vdisk information for all AMP vprocs in the system. The full report format displays information separately for each of the Pdisks used by an AMP vproc, as well as total space utilization for the vproc.

Ferret SHOWBLOCKS Command

```
> tableid "TFACT.Claim"
```

```
The table id for TFACT.Claim is
0 7935 0 (0x0000 0x1EFF 0x0000)
```

```
> scope table 0 7935 0
```

```
The SCOPE has been set
```

```
Ferret ==>
```

```
> showblocks /m
```

```
Showblocks has been started on all AMP vprocs in the SCOPE.
Type 'ABORT' to stop the command before completion
```

Table ID	Compression Status	Estimated Compression Ratio	Estimated % of Blocks Un-compressed	Distribution of data block sizes (by range of number of sectors)																Data block size statistics (sectors)		
				1-8	9-24	25-64	65-120	121-168	169-216	217-256	257-360	361-456	457-512	513-760	761-1024	1025-1304	1305-1632	1633-2048	Min	Avg	Max	
0 7935	TFACT.Claim (DEFAULT) (MAP: TD_MAP1)																					
0	N			100%														8	8	8		
1024	C	82.64%	0.00%				100%											48	88	88		
2048	C	82.64%	0.00%				100%											48	88	88		

```
* Reported in units of Large Cylinders
(A Large Cylinder is 6 times the size of a small cylinder)
```

```
"Compression Status" : C = Fully Compressed
                        PC = Partially Compressed
                        U = Fully Uncompressed
                        N = Not Compressible
```

Note: The Summary information display only provides detailed size information about the primary data subtable of a table. Fallback and index subtables are not included.

This view also provides the compression status of the table.

Showblocks options:

- showblocks /s displays table information
- showblocks /m displays subtable information
- showblocks /l displays rows per block information

The Ferret utility includes a SHOWBLOCKS command that displays the data block size and/or the number of rows per data block for a defined scope.

The SHOWBLOCKS command displays the following disk space information for a defined range of data blocks and cylinders.

SHOWBLOCKS /M – displays subtable information.

SHOWBLOCKS /L – displays minimum, average, and maximum number of rows per block.

SHOWBLOCKS /S – displays table level information – doesn't display empty tables (tables with no rows).

Another option to set the scope for the table (example on this slide) is to use the following command:

- SCOPE TABLE ("TFACT.Claim" 0)

Ferret SHOWBLOCKS – Subtable Detail

The Ferret ShowBlocks utility also allows you to view block sizes down to the subtable level. The /m option provides this level of detail as shown on this slide.



Module 5: Review Questions

1. True or False: Space limits are enforced at the table level.
2. True or False: When you use the GIVE statement to transfer a user, the SPOOL and TEMPORARY max values are adjusted to the value of the new immediate owner.
3. True or False: A child user cannot have a SPOOL limit greater than the immediate parent.
4. The DBC._____ view provides disk space usage at the table level and excludes table ALL.
5. The DBC._____ view only provides disk space usage at the database level.

Check your understanding of the concepts discussed in this module by completing the review questions as directed by your instructor.



Module 6: Users, Accounts, and Accounting

Teradata Vantage Administration

Copyright © 2007–2023 by Teradata. All Rights Reserved.

Objectives

After completing this module, you will be able to:

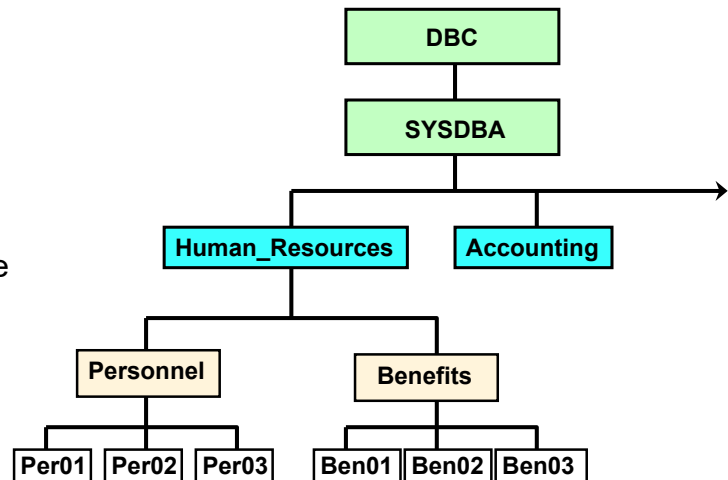
- Use Teradata accounting features to determine resource usage by user or account
- Explain how the database administrator uses system accounting to support administrative functions
- Use system views to access system accounting information

Creating New Users and Databases

You can grant CREATE DATABASE and/or CREATE USER authority to any user.

The user may then create other users and databases from:

- The user's own space, or
- The space of another user or database (if authorized).



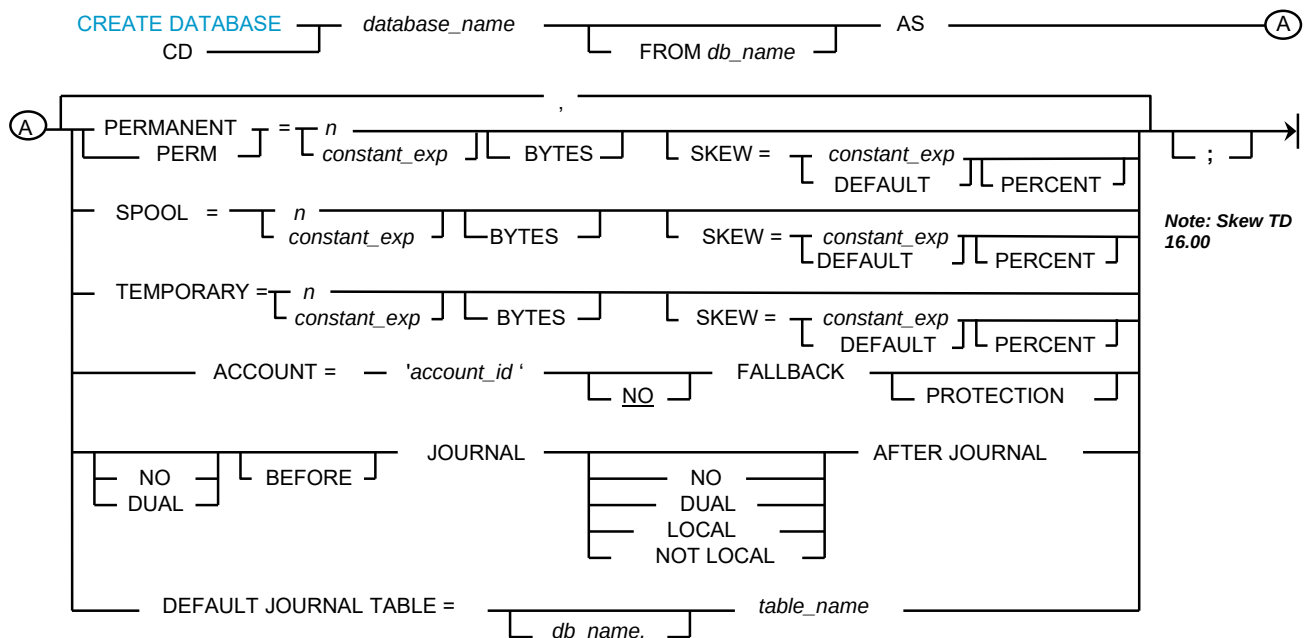
As the database administrator, you create system databases and tables and assign user privileges and access rights to tables.

To perform the above tasks, you must:

- Determine database information content and create macros to ensure the referential integrity of the database.
- Define authorization checks and validation procedures.
- Perform audit checks on the database for LOGON, GRANT, REVOKE and other privilege statements.

You can give the authority to use the CREATE DATABASE or CREATE USER statements to any application user. He or she may then create other system users or databases from his or her own space, or if specifically authorized, from the space of another system database or user.

CREATE DATABASE Statement



As the database administrator, you use the CREATE DATABASE statement to add new databases to the existing system. The permanent space for new databases you create comes from the immediate parent database or user.

Global Space Accounting (TD 16.00)

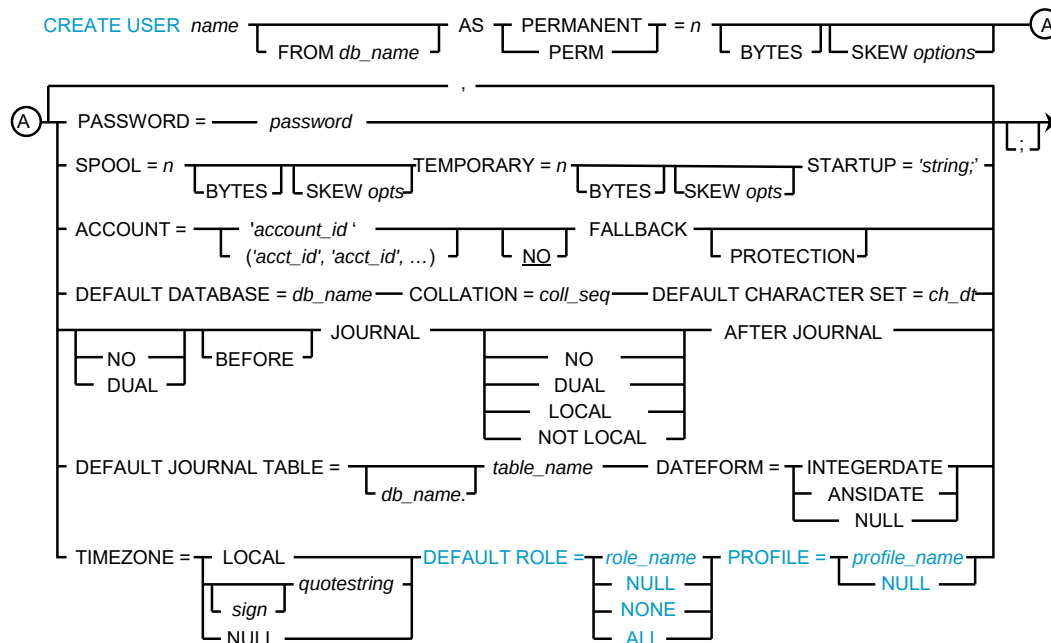
The space accounting and space management infrastructure was enhanced with Global Space Accounting to manage the space limits at a global level (system level) and augment the AMP-level space management.

- The SQL CREATE USER/DATABASE was extended to specify a skew limit percentage that allows maximum AMP space usage to be above the per-AMP quota.
- A non-zero or unlimited skew limit percentage implies the space accounting is at a global level instead of at an AMP level and thus, space usage in each AMP can be above the per-AMP quota but, for a non-zero (but not unlimited) skew limit percentage, no more than the defined skew limit percentage
- A zero skew limit percentage for a database/user implies the space accounting is at the AMP level with per-AMP quota as a maximum permissible AMP-level space as prior releases.
- A new option called skewlimit_option was added to create/modify user/database DDL under space options:

```

skewlimit_option ::= SKEW = { constant_expression | DEFAULT } [
  PERCENT ]
  
```

CREATE USER Statement



The `CREATE USER` statement enables you to add new users to the system. The permanent space for these new users comes from the immediate parent database or user.

Users have passwords while databases do not. User passwords allow users to log on to the Teradata database and establish sessions.

When you create a new user, you also create a password for the user. If the password expiration time period is not set to 0, when the user logs on for the first time, he or she is prompted to change the password.

If a user forgets the password, you can assign a new temporary password. As another option, you can set user passwords not to expire.

Acronym: `ch_dt` – Character Data Type

CREATE USER and the Data Dictionary

```
EXPLAIN CREATE USER tfact06 AS PERM = 10E9, SPOOL = 50E9, PASSWORD = secure1time;
```

Explanation

16.20 Explain

- 1) First, we lock data base tfact06 for exclusive use.
- 2) Next, we lock DBC.DataBaseSpace for write on a reserved RowHash in all partitions to prevent global deadlock.
- 3) We lock DBC.Parents for write on a reserved RowHash to prevent global deadlock.
- 4) We lock DBC.Owners for write on a reserved RowHash to prevent global deadlock.
- 5) We lock DBC.AccessRights for write on a reserved RowHash in all partitions to prevent global deadlock.
- 6) We lock DBC.DataBaseSpace in [TD_GLOBALMAP](#) for write, we lock DBC.Parents in [TD_DATADITIONARYMAP](#) for write, we lock DBC.Owners in [TD_DATADITIONARYMAP](#) for write, we lock DBC.GlobalDBSpace in [TD_DATADITIONARYMAP](#) for write on a RowHash, we lock [DBC.GlobalDBSpace](#) in [TD_DATADITIONARYMAP](#) for write on a RowHash, we lock DBC.DBase in [TD_DATADITIONARYMAP](#) for write on a RowHash, we lock DBC.DBase in [TD_DATADITIONARYMAP](#) for write on a RowHash, we lock DBC.UIFDEPENDENCY in [TD_DATADITIONARYMAP](#) for write on a RowHash, we lock DBC.Accounts in [TD_DATADITIONARYMAP](#) for write on a RowHash, and we lock DBC.AccessRights in [TD_DATADITIONARYMAP](#) for write.
- 7) We execute the following steps in parallel.
 - 1) We do a single-AMP ABORT TEST step from DBC.DBase by way of the unique primary index "Field_1 = 'TFACT06'" with no residual conditions.
 - 2) We do a single-AMP ABORT TEST step from DBC.Roles by way of the unique primary index "Field_1 = 'TFACT06'" with no residual conditions.
 - :
 - 5) We do an INSERT into DBC.DBase.
 - 6) We do a single-AMP UPDATE from DBC.DBase by way of the unique primary index "Field_1 = 'Sysdba'" with no residual conditions.
 - 7) We do a single-AMP RETRIEVE step from DBC.Parents by way of the primary index "Field_1 = '00000904'XB" with no residual conditions into Spool 1 (all_amps), which is redistributed by the hash code of (DBC.Parents.Field_2) to few AMPs in [TD_DataDictionaryMap](#). Then we do a SORT to order Spool 1 by row hash.
- 8) We do an all-AMPS MERGE step in [TD_DATADITIONARYMAP](#) into DBC.Owners from Spool 1 (Last Use).

A user is associated with a password and an account, and can log on, establish a session, and execute SQL statements. Note all of the data dictionary tables that are updated when creating a new user.

Default values associated with the CREATE USER statement are:

<u>Entry</u>	<u>Defaults to the value of</u>
FROM database	Current CREATOR
SPOOL	Same value as the CREATOR
TEMPORARY	Same value as the CREATOR
STARTUP	Null (no startup string)
ACCOUNT	CREATOR's first account ID
DEFAULT DATABASE	Username

There are also two types of rights granted automatically when you use the CREATE USER statement:

- The rights granted to a newly created user or database on itself.
- The rights granted on a newly created user, database, or object to the creating user.

As shown on this slide, the database administrator logged on as Sysdba and creates tfact06:

```
CREATE USER tfact06 AS PERM = 10E9, SPOOL = 50E9,
    PASSWORD = secure1time;
```

CREATE USER and the Data Dictionary (cont.)

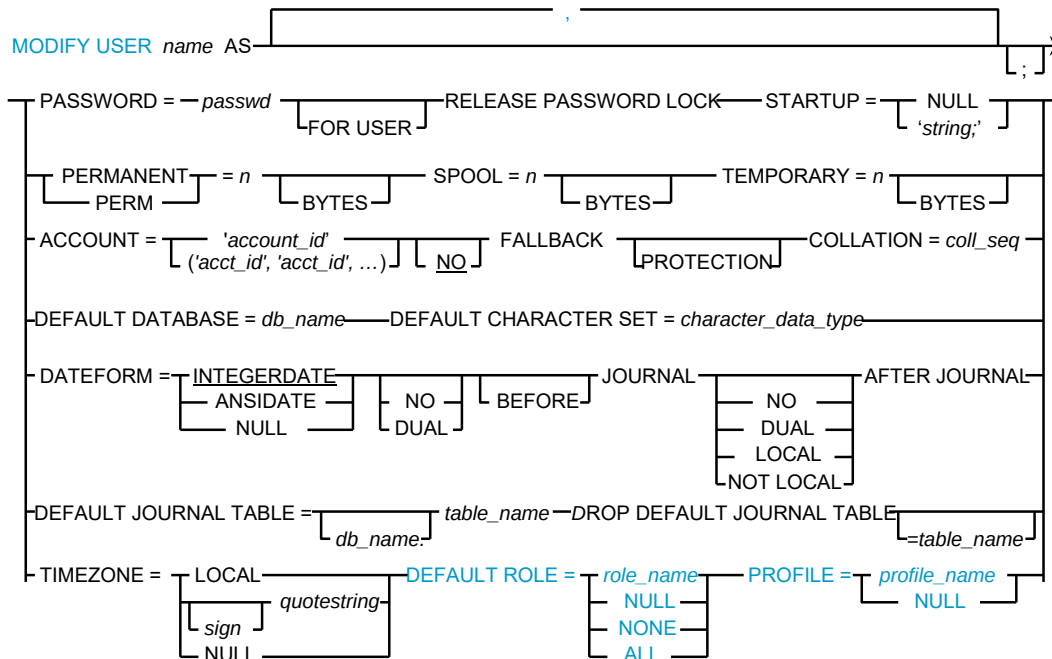
```

9) We execute the following steps in parallel.
  1) We do an INSERT into DBC.Owners.
  2) We do a single-AMP RETRIEVE step from DBC.Parents by way of the primary index "Field_1 = '00003104'XB" with no residual conditions into Spool 2 (all amps), which is redistributed by the hash code of ('00008804'XB) to few AMPs in TD DataDictionaryMap. Then we do a SORT to order Spool 2 by row hash.
10) We do an all-AMPs MERGE step into DBC.Parents from Spool 2 (Last Use).
11) We execute the following steps in parallel.
  1) We do an INSERT into DBC.Parents.
  2) We do an INSERT into DBC.Accounts.
  3) We do a single-AMP RETRIEVE step from a single partition of DBC.AccessRights by way of the primary index "Field_1 = '00000000'XB, Field_2 = '00000000'XB, Field_3 = '000000000000'XB" with a residual condition of ("DBC.AccessRights.Field_5 <> 'MD') AND ...
:
14) We execute the following steps in parallel.
  1) We do an all-AMPs MERGE step into DBC.AccessRights from Spool 3 (Last Use).
  2) We do an INSERT into DBC.UIFDEPENDENCY.
15) We flush the AMPUSAGE, DISKSPACE and Global DISKSPACE caches.
16) We do an all-AMPs ABORT TEST step from 2 partitions of DBC.DataBaseSpace by way of the primary index "Field_1 = '00003104'XB, Field_2 = '000000000000'XB" with a residual condition of ("2500000 <= (DBC.DataBaseSpace.Field_23 - DBC.DataBaseSpace.Field_18)").
17) We execute the following steps in parallel.
  1) We do an INSERT into DBC.DataBaseSpace.
  2) We do an INSERT into DBC.GlobalDBSpace.
  3) We do a single-AMP UPDATE from DBC.GlobalDBSpace by way of the primary index "Field_1 = '00003104'XB" with no residual conditions.
18) We do an all-AMPs UPDATE from 2 partitions of DBC.DataBaseSpace by way of the primary index "Field_1 = '00003104'XB, Field_2 = '000000000000'XB" with a residual condition of ("DBC.DataBaseSpace.Field_1 = '00003104'XB) AND (DBC.DataBaseSpace.Field_2 = '000000000000'XB)").
19) We flush the AMPUSAGE, DISKSPACE and Global DISKSPACE caches.
20) We spoil the parser's dictionary cache for the database.
21) Finally, we send out an END TRANSACTION step to all AMPs involved in processing the request.
-> No rows are returned to the user as the result of statement 1.

```

Several steps are performed in parallel during the CREATE USER statement.

MODIFY USER Statement



The MODIFY USER statement enables you to change the options of an existing user. In order to change the following options, the DROP DATABASE privilege is required:

- PERMANENT space limit
- SPOOL space limit
- TEMPORARY space limit
- Account codes
- Release password lock
- DROP DEFAULT JOURNAL TABLE
- Role
- Profile

The FOR USER option effectively established a temporary password that can be used to logon one time by the user. This option is only effective if the ExpirePassword attribute (set in DBC.SysSecDefaults or a profile) is set to a value greater than 0.

```
MODIFY USER RobertSmith AS PASSWORD = secret FOR USER;
```

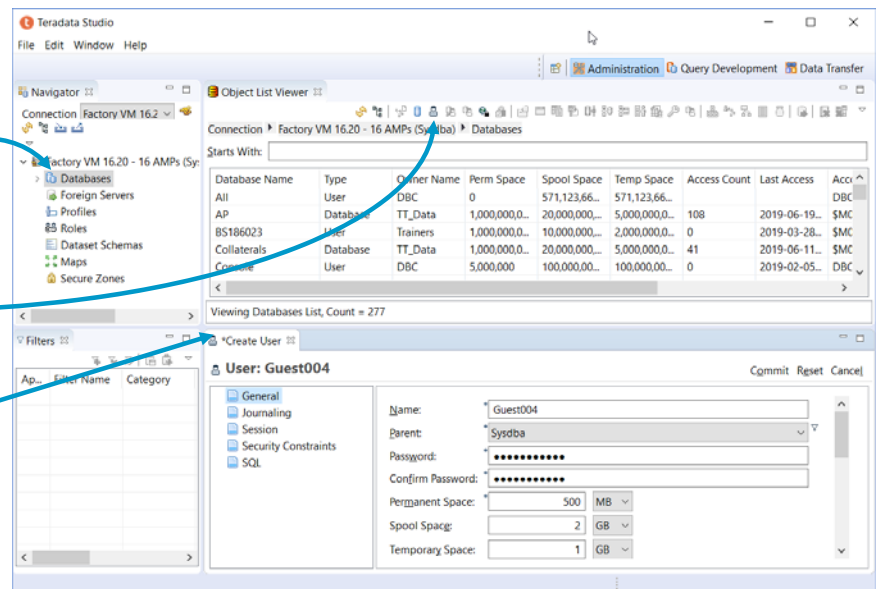
The existing password immediately expires and is replaced by “secret”. In this example, “secret” is effectively a temporary password that allows a one-time logon. The value for PasswordChgDate is reset to 0.

Note: This syntax does not have room to include the TD 16.0 “skewlimit_option” as shown earlier.

Teradata Studio – Create User

Teradata Studio can be used to create and manage users and databases.

- In the **Navigator**, right-click **Databases** and click **Show Databases**.
- In the **Object List Viewer** toolbar, click (database) or (user) icon.
- In the form, enter property values for the new database or user.
- Click **Commit**



Teradata Studio can be used to create users and other objects easily.

The following options are available for ease-of-use.

Selection

Create

Grant/Revoke

Administer Profiles

Administer Roles

Clone User

Modify User

Access Logging

Query Logging

Move Space

Query

Function / Options

Create an entirely new object – Database, Table, User, Profile, or Role.

Grant or revoke general access privileges to users. Options include Object Rights, System Rights, Logon Rights, or Column Rights.

Create and manage Profiles for users.

Create and manage Roles.

Create a new user either identical or closely related to an existing user.

Change the specifications of an existing user.

Create and manage Access Log rules.

Create and manager Query Log rules.

Reallocate permanent disk space from one database to another (efficient if not a direct descendant or parent).

Create, modify, test, or run SQL query scripts.

Creating and Using Account IDs

CREATE USER tfact07	Names user
FROM Sysdba	Name of immediate owner in hierarchy
AS	Amount of Permanent space
PERM = 1E9 *(HASHAMP()+1)	Maximum amount of Spool space
,SPOOL = 3E9 *(HASHAMP()+1)	Initial password
,PASSWORD = Secure12	Default account – medium priority
,ACCOUNT = ('\$M_9038_&D&H',	Opt. Account – high priority – recommended format
'\$H00EDUC&D&H',	Opt. Account – performance group M1
'\$M1\$LOAD&D&H',	Opt. account – medium priority – no ASE
'\$M_9038');	

A logon can optionally include an account ID; the first account ID is the default account ID.

batch logon syntax: `.LOGON tdpid/username,password,'account_id'`

Example: `.LOGON FACTORY/tfact07,Secure12,'$H00EDUC&D&H'`

BTEQ Interactive logon syntax: `.LOGON tdpid/username,, 'account_id'`

Example: `.LOGON FACTORY/tfact07,, '$H00EDUC&D&H'`
Enter password when prompted

SQL Assistant Notes:

- With an ODBC connection, single quotes for the account id are optional.
- With a .NET connection, the single quotes for the account id are not used.

When you create a user, you can specify one or more account IDs that a new user can specify. Account codes may be used to track system CPU, I/O usage, or space usage. When the user logs on, the user can specify a valid account ID, or let the first account ID in the user row (from CREATE or MODIFY USER) become the default. You should determine an account ID scheme for ease of accounting and priorities.

Account IDs may begin with the characters \$L, \$M, \$H, or \$R to identify the priorities low, medium, high, and rush, respectively.

Using Account IDs with Logon

All logons require an account ID. A user can submit an explicit account ID by including it in the logon string. It must be a valid ID specified in the last CREATE or MODIFY USER statement. If no ID is specified in the CREATE or MODIFY statements, it defaults to the ID of the immediate owner's database.

Note:

batch logon syntax:

`.LOGON tdpid/user_name, password, 'account_ID';`

BTEQ Interactive logon syntax:

`.LOGON tdpid/user_name,, 'account_ID' (note the two commas)`

Enter password when prompted

Dynamically Changing an Account ID

- You can change your Account ID without logging off. This may be done to re-prioritize a query. This is also referred to as “nicing a query”.
- You can change Account IDs for the next SQL statement you run, or for all jobs for the remainder of the current session.
- To change Account IDs, use the SET SESSION ACCOUNT statement:

Syntax:

For the next SQL statement : `SET SESSION ACCOUNT = 'Account_ID' FOR REQUEST;`

For the rest of the current session: `SET SESSION ACCOUNT = 'Account_ID' FOR SESSION;`

Example:

For the rest of the session: `SET SESSION ACCOUNT = '$H00EDUC&D&H' FOR SESSION;`

- Note: You can only use valid account IDs. Therefore, you cannot exceed the priority defined by the performance groups in your account ID.

You can dynamically change your Account ID without logging off and logging back on. One reason you may want to do this is to change your session's priority. This is also called “nicing a query”. “Nicing” is a UNIX term that means manipulating the scheduling priority of a “running” task. You typically “nice” a query to re-prioritize jobs. For instance, you could nice a query to a higher priority to run a business-critical job sooner than under its originally defined priority.

Use the SET SESSION ACCOUNT statement to change your performance group (account priority) for the next SQL query you run, or for all jobs for the remainder of the current session.

Syntax

For the next SQL statement:

```
SET SESSION ACCOUNT = 'Account_ID' FOR REQUEST;
```

For the remainder of the current session:

```
SET SESSION ACCOUNT = 'Account_ID' FOR SESSION;
```

Account String Expansion

- ASE is used to provide more detailed utilization reports and user accounting data.
 - ASE increases the granularity of information returned with the DBC.AMPUsageV.
 - For queries that span multiple hours, the time will be accumulated in its entirety to the query's start hour.
- You may add the following substitution variables to a user's account string. The system resolves the variables at logon or at SQL statement execution time.

&S	Session number	(SSSSSSSSSS)
&D	Date	(YYMMDD)
&H	Hour	(HH)
&I	Logon hostid, session number, request number	(LLLL SSSSSSSSS RRR RRR RRR)
&L	Logon timestamp	(YYMMDDHHMISS.hh)
&T	Time	(HHMISS)
- \$xxxWORK&D&H is the recommended account format where WORK is a 4-character work load type starting in the 5th position of an account id.
 - For \$L, \$M, \$H, and \$R in Resource Partition 0, then examples are:
\$M00EDUC&D&H or \$M00TACT&D&H (the 00 is simply a placeholder)
 - If a two-character performance group name like M1 (or MD) is used, then examples are:
\$M1\$TACT&D&H or \$MD\$LOAD&D&H

Account String Expansion (ASE) is an optional feature that enables you to use substitution variables in the account ID portion of the user's logon string. These variables enable you to include date and time information in the string. The variables are resolved at logon time or at actual SQL execution time. Account strings can be up to 128 characters. If, as a result of string expansion, you generate a string longer than 127 characters, the system truncates all characters to the right of position 127. Separation characters, such as colons in time fields and slashes in dates, are included in the character count.

ASE Variables:

- &L The **logon time stamp** variable causes the logon time stamp to be inserted into the account string. The full logon time stamp consists of 15 characters. The value inserted into AMPUsage is established at logon time.
- &D The **date** variable causes the date to be inserted into the account string.
- &T The **time** variable inserts the time of day into the account string.
- &H The **hour** variable inserts the hour of the day into the account string. The inserted value consists of two characters and becomes truncated if you place &H to the right of position 29.
- &I The **logon host ID/session number/request number** variable inserts the logon host ID, the session number and the request number into the account string.
- &S The **session number** variable inserts the current session number into the account string.

A past ASE recommendation has been to include &S&D&H in the account string. The newer recommendation is just use &D&H and omit the &S as it provides little value, increases space usage, and may degrade the performance of tactical queries.

ASE Accounting Example

- Users TFACT01 and TFACT02 each log on with \$M00EDUC&D&H account string.
 - Each hour a new row is possibly placed in Acctg.
- Impact of using ASE variables with Acctg.

ASE Variable	Performance Impact	Data Capacity Impact
none	Negligible	1 row per account per AMP.
&D	Negligible	1 row per account per day per AMP.
&H	Negligible	1 row per account per hour per AMP (all days go into 1 hour).
&D&H	Negligible	1 row per account per hour per day per AMP.
&S&D&H	Negligible	1 row per account per session per hour per day per AMP.
&L	Negligible	1 row per logon (LAN) or session pool.
&T	Potentially Non-negligible	1 row per query per AMP.

- Perform the following tasks to extract accounting information:
 - Step 1. Create AMPUsageSum table.
 - Step 2. Populate AMPUsageSum table.
 - Step 3. Create Usage view.
 - Step 4. Create billing and resource usage reports.

Background Information

Two existing users, TFACT01 and TFACT02, logged onto the system using an account string defined as &D&H. The DBC.Acctg table contains a number of rows generated by the ASE feature.

Logon example: `.LOGON DBC/tfact01, password, '$L00EDUC&D&H';`

Tasks

You need to create a table, view, and a number of reports that provide billing and resource usage information based on the statistics collected by the AMPUsage view.

- Step 1. Create AmpUsageSum table.
Create a table to hold the collected statistics from the AMPUsage view. This is a history table since it contains stored historical data.
- Step 2. Populate AMPUsageSum table.
After you build the AmpUsageSum table, use the INSERT command to populate it with row information from the DBC.AMPUsage view.
- Step 3. Create Usage view
Use the CREATE statement to combine columns from the DBC.AMPUsage view and DBC.LogOnOff view into the Usage view.
- Step 4. Create billing and resource usage reports.
Once the view is completed, construct SQL statements to SELECT information from the Usage view to create billing and resource usage reports.

System Accounting Views

<u>View</u>	<u>Description</u>
DBC.AccountInfo[V][X]	Returns each Account Name (Account ID) associated with a user (for users the requestor owns).
DBC.AMPUsage[V][X]	Provides information about I/O and AMP CPU usage by user and account.

AccountInfo views provide information about valid accounts, and AMPUsage views provide information about the usage of each AMP vproc by user and account.

AccountInfo Views

The DBC.AccountInfo[V][X] views provide information about valid accounts for a specific user. The information provided is based on data from the DBC.Accounts table in the data dictionary. Each time a CREATE or MODIFY statement indicates an account ID, a row is either inserted or updated in the DBC.Accounts table.

(When you use restricted views, you must be the requester or have modify rights turned on.)

AMPUsage Views

The DBC.AMPUsage[V][X] views provide information about the usage of each AMP vproc for each user and account. It is based on information in the DBC.Acctg table in the data dictionary and supplies information about AMP CPU time consumed, and the number of AMP to DSU read and write operations generated by a given user or account. It also tracks the activities of any console utilities.

Each time a user logs on or submits an SQL request; a row is either inserted or updated in the DBC.Acctg table. If the user_name/account_name does not exist, then a new row is inserted. If the row already exists in the DBC.Acctg table, then it is updated. The rows in this table track how much AMP usage the specific user_name/account_name generates. This information may be used to bill an account for system resource use.

AccountInfo View

Provides information about each user and the valid account codes associated with each. (X views – lists users and accounts the requestor owns or has modify rights to).

[DBC.AccountInfo\[V\]\[X\]](#)

UserName	AccountName	UserOrProfile
----------	-------------	---------------

Example:

Identify all users with a valid HIGH priority code.

```
SELECT *
FROM   DBC.AccountInfoV
WHERE  AccountName LIKE '$H%'
ORDER BY 1 ;
```

Results:

UserName	AccountName	UserOrProfile
AP	\$H_9038	User
Cust_Service_Gold	\$H00TACT&D&H	Profile
Employee	\$H00TACT&D&H	Profile
Students	\$H_9038	User
Sysdba	\$H_9038	User
TDPUSER	\$H	User
tfact01	\$H00EDUC&D&H	User
tfact07	\$H00EDUC&D&H	User

The DBC.AccountInfo[V][X] views shown on this slide provide information about each user and the valid account codes associated with each user. When the requesting user indicates the [X] view, they can only see information about users that they own or have modify rights on.

The UserOrProfile column indicates whether the user is an actual user or a profile.

Example

The SQL statement on this slide requests a list of all users with a valid HIGH priority account code.

AMPUUsage View

AMPUUsage views are updateable views that use the DBC.Acctg table to provide accounting information by username and account.

These views can be used to determine which users are consuming CPU and/or I/O resources.

AMPUUsage will accumulate CPU and I/O usage for every unique account.

DBC.AMPUsage[V][X]

AccountName	UserName	CPUTime	DiskIO
Vproc	VprocType	Model	

CPUTime: Total number of AMP CPU seconds used (increments of 1/100 second).

DiskIO: Total number of logical disk I/O operations.

Vproc: AMP Vproc number

VprocType: AMP

Model: Model number (e.g., 2850)

The DBC.AmpUsage[V][X] views use the underlying DBC.Acctg table to provide accounting information by username and account. You can update this view. This view provides CPU activity and logical I/O counts explicitly requested by the following two sources:

- AMP database software
- File system that is running in the context of an AMP worker task

This view can be used to determine which user or users are consuming CPU and I/O resources on a system.

The system requests I/Os to execute a step in the user's query. The DBC.AmpUsage views do not include I/Os the operating system performs for swapping or I/Os caused by parsing the user's query. The system charges a logical I/O even if the segment you request is cached and no physical I/O is done.

Column definitions in this view include:

<u>Column</u>	<u>Definition</u>
Vproc	The virtual processor ID
VprocType	AMP
Model	System model (e.g., 6800, etc.)

AMPUUsage View – Example

Example:

Show CPU time and I/O totals for a single user.

```
SELECT      UserName (CHAR(10))
           ,Substring (AccountName FROM 1 FOR 8) AS "Acct_Str"
           ,Substring (AccountName FROM 9 FOR 6) AS "Date"
           ,Substring (AccountName FROM 15 FOR 2) AS "Hour"
           ,SUM (CPUTime) AS "CPU Time"
           ,SUM (DiskIO) AS "Disk IO"
FROM        DBC.AMPUsageV
WHERE       UserName = 'student135'
GROUP BY    1, 2, 3, 4
ORDER BY    5 DESC;
```

The Account ID is \$M00EDUC&D&H

To reset counters for ALL rows or selected rows,
use the DBC.ClearAccounting macro.

```
SHOW MACRO DBC.ClearAccounting;
REPLACE MACRO DBC.ClearAccounting
AS ( UPDATE Acctg SET CPU = 0, IO = 0 ALL; );
```

UserName	Acct Str	Date	Hour	CPU Time	Disk IO
STUDENT135	\$M00EDUC	180115	09	53.97	1,894,531
STUDENT135	\$M00EDUC	180115	11	28.95	913,708
STUDENT135	\$M00EDUC	180112	14	19.04	912,218
STUDENT135	\$M00EDUC	180116	19	8.96	223,322
STUDENT135	\$M00EDUC	180108	09	4.88	92,933
STUDENT135	\$M00EDUC	171219	20	2.83	43,171
STUDENT135	\$M00EDUC	171218	07	1.52	22,880

The SQL statement on this slide requests totals for CPU time and I/O for a user named student140. The totals are aggregates of all resources used across all AMP vprocs.

Note that the account ID has been expanded and the SUBSTRING function is used to display the expanded account string in a readable format.

Summary

- To establish execution time priorities, use the first two characters in the account code and the performance groups.
- Your position in the hierarchy does not affect your priority.
- You can define accounting mechanisms:
 - Charge-back billing
 - System usage reporting
 - Capacity planning
 - Performance analysis
- To reset data dictionary tables used to collect accounting information, use:
 - DBC.ClearAccounting macro
 - DBC.ClearPeakDisk macro

This slide summarizes some important concepts regarding this module.



Module 6: Review Questions

1. True or False: You can only give the authority to use the CREATE DATABASE and CREATE USER statements to certain types of users.
2. True or False: AMPUsage views can be used to determine which users are consuming CPU and/or logical disk I/O resources.
3. True or False: A user can use the MODIFY USER statement to change their password, default database, and date format.
4. When creating a new user, which option defaults to the creator's value.
 - a. SPOOL
 - b. FALLBACK PROTECTION
 - c. All of the Account_IDs
 - d. DEFAULT DATABASE
5. When creating a new user, which three options are required with the CREATE USER command.
 - a. SPOOL
 - b. PERMANENT
 - c. User name
 - d. PASSWORD

Check your understanding of the concepts discussed in this module by completing the review questions as directed by your instructor.



Lab Exercise 6-1 (1 of 3)

Purpose

In this lab, you will view information in the data dictionary regarding space usage and accounting details.

Tasks

1. Using the DBC.DiskSpaceV view, find the total disk storage capacity of the system on which you are logged on:

Total capacity _____

2. Using the same view, find how much of the space is currently in use:

Current space utilization _____

Write a query to show what percentage of system capacity is currently in use. _____%

3. Write a query to show which databases/users are currently using (current perm) the largest percentage of their max perm space limit (group by database/user).

This slide describe the tasks for this lab exercise.

SQL Hint

```
SELECT    ((CAST(SUM(currentperm) AS FLOAT) /  
           NULLIFZERO(SUM(maxperm)) * 100)) AS "% Used Perm Space"  
FROM      DBC.DiskSpaceV;
```



Lab Exercise 6-1 (2 of 3)

4. Using the DBC.DatabasesV view, find the total number of databases and users defined in the system.

Total row count (databases and users) _____

Using this view, how many users are there? _____

Using this view, how many databases are there ? _____

Who is the creator of AP? _____

Who is the owner of AP? _____

5. Using the TableSizeV view, find the name and size of each table in the DBC user. List the Data Dictionary tables in DESCending order by size.

List the six largest tables:

1. _____ 2. _____ 3. _____

4. _____ 5. _____ 6. _____

SQL Hint:

```
SELECT      TOP 6 TableName, SUM(CurrentPerm)
FROM        DBC.TableSizeV
WHERE       DatabaseName = 'DBC'
GROUP BY    1
ORDER BY    2 DESC;
```




Lab Exercise 6-1 (3 of 3)

teradata.

6. Using the DBC.AMPUsageV view, find the number of AMP vprocs defined on your system.
(HINT: Use a WHERE condition to reduce the number of DD/D table rows considered.)

Number of AMPS _____

7. Using the DBC.AccountInfoV view, list all of your valid account codes.

8. Using the DBC.AMPUsageV view, write a query to show the number of AMP CPU seconds and logical disk I/Os that have been charged to your username:

Username _____ Seconds _____ I/Os _____

SQL Hint:

```
SELECT      UserName (CHAR(15))
            ,SUM(CPUTime)
            ,SUM(DiskIO)
FROM        DBC.AMPUsageV
WHERE       UserName = USER
GROUP BY    UserName;
```



Module 7: Profiles

Teradata Vantage Administration

Copyright © 2007–2023 by Teradata. All Rights Reserved.

Objectives

After completing this module, you should be able to:

- List two advantages of utilizing profiles
- Use profiles when creating new users
- Use system views to display profile information

Profiles

What is a “profile”?

- A profile is a set of common user attributes that can be applied to a group of users.
- Profile parameters include:
 - Account id(s)
 - Default database
 - Spool space allocation
 - Temporary space allocation
 - Password attributes (expiration, etc.)
 - Query Band (15.10)
 - Default Map (16.10)

What are advantages of using “profiles”?

- Profiles simplify user management.
 - A change of a common attribute requires an update of a profile instead of each individual user affected by the change.
 - Specify password security controls for groups of users.

How are “profiles” managed?

- Utilize the following DDL commands, tables, and views:
 - CREATE/MODIFY/DROP PROFILE and SELECT PROFILE
 - Profiles are maintained in the system table named [DBC.Profiles](#).
 - To view profiles and their attributes use system views – [DBC.ProfileInfo\[V\]\[X\]](#).

Profiles define system attributes. By assigning a profile to a group of users, you can ensure that all group members operate with a common set of attributes.

To manage system attributes for groups, a database administrator can:

- Create a different profile for each user group, based on system attributes that group members share.

You can define values for all or a subset of the parameters in a profile. If you do not set the value of a parameter, the system uses the setting defined for the user in a CREATE USER or MODIFY USER statement.

- Assign profiles to users.

The parameter settings in a user profile override the settings for the user in a CREATE USER or MODIFY USER statement.

Like roles, the concept of ownership and ownership hierarchy is not applicable to profiles.

This module covers the commonly used Profile attributes. There are additional Profile attributes that are described in detail in the reference manuals.

Example of Simplifying User Management

Example:

- **The problem:**
 - A customer has a group of 1,000 users that are assigned the same spool space, the same default database, and the same account ID.
 - Changing any of these parameters for 1,000 users can be a time-consuming task.
- **A solution using profiles:**
 - Create a profile that contains these parameters and assign that profile to the users.
 - This would simplify system administration because a parameter change requires updating only the profile instead of each individual user.

The profile concept provides a solution to the following problem.

A customer has a group of 1,000 users that are assigned the same amount of spool space, the same default database, and the same account ID. Changing any of these parameters for 1,000 users is a very time-consuming task for the database administrators.

The database administrators' task will be simplified if they can create a profile that contains one or more system parameters such as accounts ids, default database, spool space and temporary space. This profile is assigned to the group of users.

This would simplify system administration because a parameter change requires updating only the profile instead of each individual user.

In summary, a set of parameters may be assigned certain values in a profile and this profile may be assigned to a group of users and thereby have them share the same settings. This makes changing parameters for a group of users a single step instead of a multi-step (one for each user in the group) process.

Implementing Profiles

What access rights are used to support profiles?

- CREATE PROFILE – needed to create new profiles
- DROP PROFILE – needed to modify and drop profiles

Who is allowed to create and modify profiles?

- Initially, only DBC has the CREATE PROFILE and DROP PROFILE access rights.
- As DBC, give the “profile” access rights to the database administrators (e.g., Sysdba).
GRANT CREATE PROFILE, DROP PROFILE TO Sysdba WITH GRANT OPTION;

How are users associated with a profile?

- The CREATE PROFILE command is used to create a profile of desired attributes.
CREATE PROFILE Employee_P AS ... ;
- The PROFILE option is used with CREATE USER and MODIFY USER commands to assign a user to a specific profile.

**CREATE USER Emp01 AS ..., PROFILE = Employee_P;
 MODIFY USER Emp02 AS PROFILE = Employee_P;**

Profiles enable you to manage the following common parameters:

- Account strings, including ASE codes and Performance Groups
- Default database
- Spool space
- Temporary space
- Query Band (15.10)
- Default Map (16.10)
- Password attributes, including:
 - Expiration
 - Composition (length, digits, and special characters)
 - Allowable logon attempts
 - Duration of user lockout (indefinite or elapsed time)
 - Reuse of passwords

The key word PROFILE will give both the CREATE PROFILE and DROP PROFILE access rights.

GRANT PROFILE TO SYSDBA WITH GRANT OPTION;

Note that the PROFILE privileges can only be granted to a user and not to a role or database. Like roles, the concept of ownership and ownership hierarchy is not applicable to profiles.

To remove a profile from a user,

MODIFY USER *username* AS PROFILE = NULL;

Impact of Profiles on Users

The assignment of a profile to a group of users is a way of ensuring that all members of a group operate with a common set of parameters.

Profile definitions apply to every assigned user, overriding specifications at the system or user level.

- However, any profile parameter can be NULL or NONE.

All members inherit changed profile parameters. The impact on current users is as follows:

- SPOOL and TEMPORARY space allocations are imposed immediately.
- Password attributes take effect upon next logon.
- Database, Account IDs and Query Band are considered at next logon unless the member submits a SET statement.

Order of Precedence for parameters:

1. Specify database, query band or account ID at session level
2. Specified parameters in a Profile
3. CREATE USER or MODIFY USER statements

All members inherit changed profile parameters. The impact is immediate, or in response to a SET SESSION statement, or upon next logon, depending on the parameter:

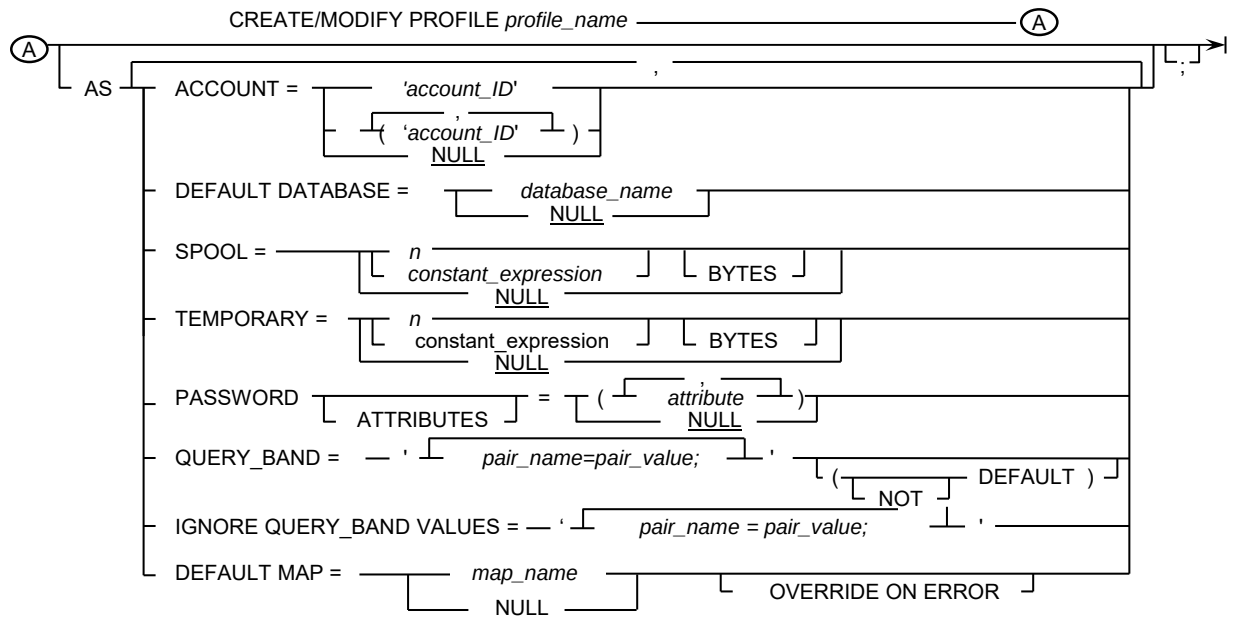
- SPOOL and TEMP space allocations are imposed immediately. This will affect the current session of any member who is logged on at the time his or her user definition is modified.
- Password attributes take effect upon next logon.
- Account IDs, Query Band and a default database are considered at next logon unless the member submits a SET statement. For SET SESSION ACCOUNT case the account ID must agree with the assigned profile definition.

Order of Precedence

With profiles, there are 3 ways of setting accounts and default database (2 for query band). The order of precedence (from high to low) is as follows:

1. The DATABASE statement is used to set the current default database or the SET SESSION ACCOUNT is used to set the account ID. However, a user can only specify a valid account ID. SET QUERY_BAND can be used to specify the query band for the session or transaction. Note: If the values in the profile query band are not default, the transaction or session query band cannot contain the same names as the profile query band. Basically, these values cannot be changed (restricted).
2. Specify them in a profile and assign the profile to a user.
3. Specify accounts or default database through the CREATE USER/MODIFY USER statements.

CREATE/MODIFY PROFILE Statement



Password Attributes (CREATE/MODIFY PROFILE)

Password Attributes

EXPIRE	=	$\left[\begin{array}{c} n \\ \text{NULL} \end{array} \right]$	(# of days; 0 doesn't expire)
MINCHAR	=	$\left[\begin{array}{c} n \\ \text{NULL} \end{array} \right]$	(range is 1 – 127 if extended object naming is enabled)
MAXCHAR	=	$\left[\begin{array}{c} n \\ \text{NULL} \end{array} \right]$	(range is 1 – 127 if extended object naming is enabled)
DIGITS	=	$\left[\begin{array}{c} c \\ \text{NULL} \end{array} \right]$	(options are Y, N, R)
RESTRICTWORDS	=	$\left[\begin{array}{c} c \\ \text{NULL} \end{array} \right]$	(options are Y, N)
SPECCHAR	=	$\left[\begin{array}{c} n \\ \text{NULL} \end{array} \right]$	(options are Y, N, ...)
MAXLOGONATTEMPTS	=	$\left[\begin{array}{c} n \\ \text{NULL} \end{array} \right]$	(# of attempts; 0 = never locked)
LOCKEDUSEREXPIRE	=	$\left[\begin{array}{c} n \\ \text{NULL} \end{array} \right]$	(# of minutes; 0 = not locked; -1 = locked indefinitely)
REUSE	=	$\left[\begin{array}{c} n \\ \text{NULL} \end{array} \right]$	(# of days; 0 - reuse immediately)

If a parameter is not specified in a profile and is not specified with the CREATE USER or MODIFY USER statement, the following list of defaults applies.

<u>Parameter</u>	<u>Default Value</u>
Account ID	The default account ID (first account ID of the immediate owner).
Performance Group	\$M
DEFAULT DATABASE	Username
SPOOL	The same SPOOL value as the creator (assumes creator has a profile)
TEMPORARY	The same TEMPORARY value as the creator (assume profile)

If the password attributes option is not specified in a profile, then the password attributes specified in the DBC.SysSecDefaults table are used for the users assigned to this profile.

DEFAULT MAP

You can specify an existing contiguous or sparse map as the default map for the profile. You must have been granted the specified map unless the map is the same as the default map of the profile creator. The map_name must be the name of an existing contiguous or sparse map. You cannot specify TD_DataDictionaryMap or TD_GlobalMap.

OVERRIDE ON ERROR

Specify this option to use the default map if an error occurs when the MAP clause is specified for a CREATE TABLE, CREATE JOIN INDEX, or CREATE HASH INDEX statement.

Teradata Password Control

Description

- At the system level, the DBC.SysSecDefaults table has password parameters to control system-wide password attributes. Similar parameters are available at the profile level to establish password attributes for a group of users.
- The DBS Password Control feature provides additional requirements on valid Teradata user passwords.
 - These requirements, which mainly consist of enforcing character variation within a password string, can be enabled or disabled by the DBA/Security Administrator on a system/user basis.
- You can specify whether or not a password is subject to the content restrictions defined in the table [DBC.PasswordRestrictions](#).

Customer Benefit

- These new features allow for the requirement and enforcement of stronger passwords.
- Many passwords would be relatively easy for an intruder to guess, especially if they contain common words or names. Forcing users to create passwords that do not use common words or names enhances password security.

Many passwords would be relatively easy for an intruder to guess, especially if some of the letters are known. Forcing users to create passwords with one or more digits enhances password security.

When specifying the maximum password length, keep in mind that some users may try to create a password of maximum length. Because it is more difficult to remember a long password, the user is more likely to write it down rather than memorize it.

Adding and Removing Restricted Words

The PasswordRestrictWords (DBC.SysSecDefaults) or RestrictWords (profiles) parameter determines whether or not a password is subject to the content restrictions defined in the DBC.PasswordRestrictions list. You can add words to the Restricted Words list as follows:

```
INSERT INTO DBC.PasswordRestrictions VALUES ('newrestrictedword');
```

Note: Although the default Restricted Words list is composed of English words, the words you add can be in any supported character set.

You can also remove words from the list as follows:

```
DELETE FROM DBC.PasswordRestrictions WHERE (RestrictedWords = 'delword');
```

The DBC.RestrictedWords (or DBC.RestrictedWordsV) views can be used to view restricted words.

Teradata Password Control (cont.)

Options

- The [PasswordSpecChar](#) (or [SpecChar](#)) parameter is used to establish the following password control rules.
 - Do not allow a password to contain the user name
 - Allow or require a mixture of upper/lower case characters
 - Allow or require at least one alpha character
 - Allow, not allow, or require at least one special character
- The [PasswordDigits](#) (or [Digits](#)) parameter is used to allow, not allow, or require a numeric digit in the password.
- The [PasswordRestrictWords](#) (or [RestrictWords](#)) parameter is used to indicate that a password cannot contain one of the "restricted words".
- The [DBC.SecurityDefaultsV](#) (view) can be used to view/update DBC.SysSecDefaults.

The PasswordDigits or Digits parameter determines if digits may be used in a password. The default value for the PasswordDigits or the Digits parameter is **Y**: Digits are allowed in a password.

The acceptable values for the PasswordDigits (DBC.SysSecDefaults – table or DBC.SecurityDefaults - view) or Digits (Profile) parameter are: Note: The values are not case sensitive.

- **Y** = Digits are allowed
- **N** = Digits are not allowed
- **R** = At least one digit is required

Password Special Characters

One of the key password parameters is "PasswordSpecChar" or "SpecChar". This parameter determines how ASCII special characters can be used in a password. It includes the following options:

- special characters are allowed/not allowed/required
- passwords must contain at least one alpha character
- no password can contain the database username
- passwords must contain a mixture of upper/lower case letters

The default value of this parameter is **Y**:

- special characters are allowed in a password
- username is allowed in the password string
- alpha characters are allowed but not required
- mixed upper and lower case characters are allowed but not required

Teradata Password Control Options

Options Table for PasswordSpecChar and SpecChar

Option Special Chars	N	Y	A	B	C	D	E	F	G	H	I	J	K	L	M	O	P	R
Rule Username	Y	Y	Y	Y	Y	Y	Y	Y	Y	N	N	N	N	N	N	N	N	N
Rule Upper/Lower	Y	Y	Y	Y	Y	Y	R	R	R	Y	Y	Y	Y	Y	Y	R	R	R
Rule One Alpha	Y	Y	Y	R	R	R	R	R	R	Y	Y	Y	R	R	R	R	R	R
Rule Special Chars	N	Y	R	N	Y	R	N	Y	R	N	Y	R	N	Y	R	N	Y	R

N – Not Allowed, Y – Allowed, but not required, R – Required
 Note: These options are not case sensitive. You can use "A" or "a".

The PasswordSpecChar parameter (DBC.SysSecDefaults) or SPECCHAR (Profile) determines how special characters can be used in a password. It includes the following options:

- special characters are allowed/not allowed /required
- passwords must contain at least one alpha character
- no password can contain the database username
- passwords must contain a mixture of upper/lower case letters

The default value of the PasswordSpecChar parameter is **Y**:

- special characters are allowed in a password
- username is allowed in the password string
- alpha characters are allowed but not required
- mixed upper and lower case characters are allowed but not required

A Password *can* contain ...

- 1 to **127** characters (UTF-8 or UTF-16 characters), letters **A** through **Z**, and/or **a** through **z**
- Digits **0** through **9** in single-byte or multi-byte form
- **Note:** A password can be all-numeric only if it is enclosed in quotes password = "12341234"
- The following special characters, in either single-byte or multi-byte form:
 - \$ (dollar sign)
 - _ (underscore)
 - # (pound sign)
 - Other special characters may be used (if they are not specifically prohibited by the rules in the reference manuals) *and* if the password is enclosed in quotes (" ").

CREATE PROFILE Example

Create a profile.

```
CREATE PROFILE Employee_P AS
  ACCOUNT      = ('$M00EDUC&D&H', '$L00EDUC&D&H')
  ,DEFAULT DATABASE = HR_VM
  ,SPOOL       = 10E9
  ,TEMPORARY   = 1E9,
  ,PASSWORD    = (EXPIRE = 90, MINCHAR = 8, MAXLOGONATTEMPTS = 3,
                  LOCKEDUSEREXPIRE = 1440, REUSE = 365,
                  DIGITS = 'R', RESTRICTWORDS = 'Y', SPECCHAR = 'P')
  ,DEFAULT MAP  = TD_Map1 OVERRIDE ON ERROR
  ,QUERY_BAND  = 'TVSTEMPERATURE=NONE;' /* NOT DEFAULT is the default */

CREATE USER Emp01 AS PERM = 0, PASSWORD = emp01pass,
  PROFILE = Employee_P,
  SPOOL = 20E9,
  ACCOUNT = '$M00FACT&D&H';
```

What is the spool space limit for Emp01?

What is the default account code for Emp01?

Assume this command is executed: `MODIFY USER Emp01 AS PROFILE = NULL;`

What is the spool space limit for Emp01?

What is the default account code for Emp01 for the current session?

What is the default account code for Emp01 for a new session?

This slide contains a simple example of creating a profile, assigning it to a user, and then removing it from the user with the MODIFY USER command.

Answers to first set of two questions on this slide:

10E9

\$M00EDUC&D&H

Answers to second set of three questions on this slide:

20E9

\$M00EDUC&D&H

\$M00FACT&D&H

QUERY_BAND = 'pair_name=pair_value;[...]' [(NOT) DEFAULT]

- **DEFAULT** – permits setting session and transaction query bands with same names.
- **NOT DEFAULT** – name-value pairs in session and transaction query bands with same names will be discarded. This is the default if not specified.

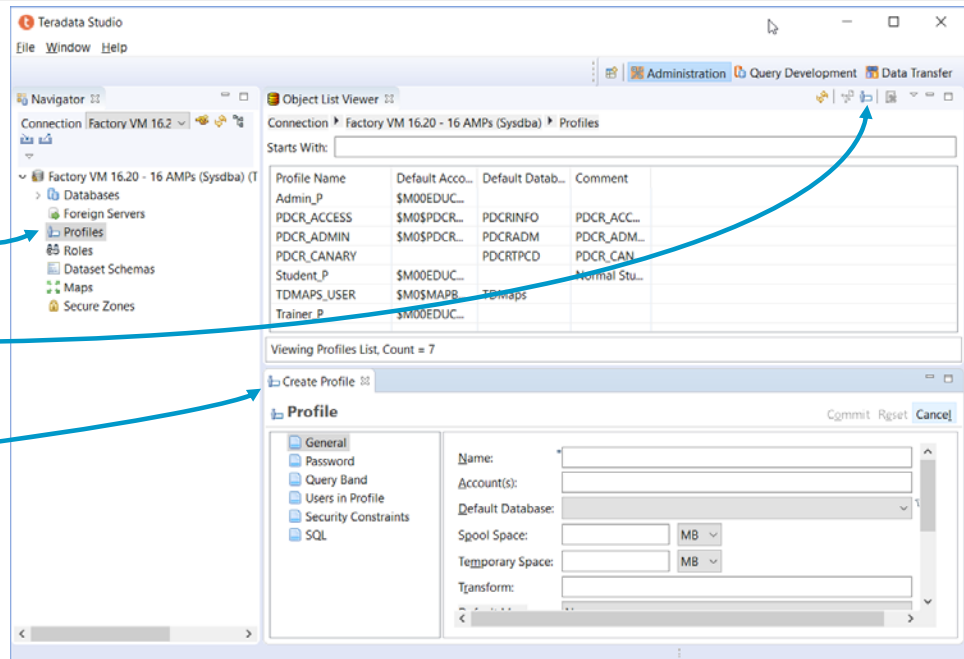
The option can also specify if the values in the profile query band are defaults. If defaults:

- They can be overridden for the session or transaction using a SET QUERY_BAND request only if DEFAULT is specified with the QUERY_BAND option.
- The session or transaction query band can contain the same names as the profile query band with different assigned values.

Teradata Studio – Create Profile Example

Teradata Studio can be used to create and manage profiles.

- In the **Navigator**, right-click **Profiles** and click **Show Profiles**.
- In the **Object List Viewer** toolbar, click the **Create Profile** icon.
- In the **Create Profile** window, enter property values for the new profile.
- After completing, click **Commit**.



This slide contains an example of creating a profile using Teradata Studio.

Teradata Studio – Create Profile Example (cont.)

The screenshot shows the 'Create Profile' dialog box in Teradata Studio. The 'General' tab is selected. The profile name is 'Employee_P'. The account(s) are '\$M00EDUC&D&H, \$L00EDUC&D&H'. The default database is 'HR_VM'. The spool space is '10 GB' and the temporary space is '1 GB'. The transform is 'TD_Map1'. The default map is 'TD_Map1'. The 'Override on error' checkbox is checked. The comment field is empty.

Create Profile > General tab

These two options are equivalent to:

**SP00L=10E9,
TEMPORARY=1E9**

```
CREATE PROFILE Employee_P AS  
  ACCOUNT = ('$M00EDUC&D&H', '$L00EDUC&D&H')  
  DEFAULT DATABASE = HR_VM, SP00L = 10E9, TEMPORARY = 1E9, ...
```

This slide continues the example of creating a profile using Teradata Studio.

Teradata Studio – Create Profile Example (cont.)

The screenshot shows the 'Create Profile' dialog box with the 'Password' tab selected. The profile name is 'Employee_P'. The 'Length' section shows 'Minimum Length' as 8 and 'Maximum Length' as 15. The 'Security' section shows 'Expiration (days)' as 90, 'Maximum Log On Attempts' as 3, 'Reuse (days)' as 365, and 'Lock User (minutes)' as 1440. The 'Restrictions' section has 'Allow User Name' unchecked and 'Search for Restricted Words' checked. Under 'Restrictions', there are four dropdown menus: 'Digits' (Required), 'Alpha Characters' (Required), 'Mixed Case Characters' (Required), and 'Special Characters (# \$ _ !)' (Allowed). A blue bracket groups these four dropdown menus.

Create Profile > Password tab

These options are equivalent to:

**DIGITS='R',
RESTRICTWORDS='Y',
SPECCHAR='P'**

```
CREATE PROFILE Employee_P ...
PASSWORD = (EXPIRE=90, MINCHAR=8, MAXCHAR=15, MAXLOGONATTEMPTS=3,
            LOCKEDUSEREXPIRE=1440, REUSE=365,
            DIGITS='R', RESTRICTWORDS='Y', SPECCHAR='P')...
```

This slide continues the example of creating a profile using Teradata Studio.

Teradata Studio – Create Profile Example (cont.)

Create Profile > Query Band tab

Name	Value
TVSTEMPERATURE	NONE

Then add Query Band Name/Value Pairs as desired.

Note: Once added, if you do not select “Set as Default Query Bands”, then they are restricted and can NOT be changed by user assigned to the profile.

```
CREATE PROFILE Employee_P ...
... QUERY_BAND = 'TVSTEMPERATURE=NONE;';
```

This slide continues the example of creating a profile using Teradata Studio.

If the values in the profile query band are not default, the transaction or session query band cannot contain the same names as the profile query band. In other words, they cannot be overridden by a SET QUERY_BAND for the session or transaction and are considered restricted. Attempted changes will just be discarded with no error other than a warning message. If the DEFAULT clause is omitted, the profile query band is set to NOT DEFAULT.

A QUERY_BAND=NULL option for the MODIFY PROFILE statement removes the query band from the profile.

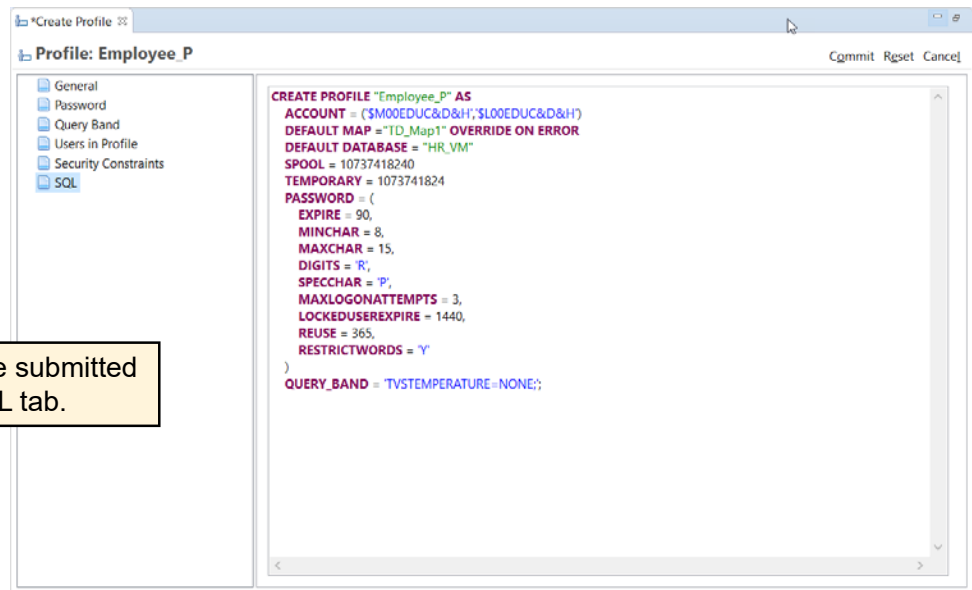
Note about: IGNORE QUERY_BAND VALUES

Defines the set of name-value pairs to ignore if specified in a SET QUERY_BAND statement for all users assigned the profile. When a user with the profile issues a SET QUERY_BAND statement containing a name-value pair matching a IGNORE QUERY_BAND VALUES name-value pair, the system performs the following actions:

- The matching name-value pair is ignored.
- The query band is set with the remaining name-value pairs.
- A warning message displays.

Teradata Studio – Create Profile Example (cont.)

Create Profile > SQL tab



To preview the SQL that will be submitted when you commit, use the SQL tab.

This slide continues the example of creating a profile using Teradata Studio.

DROP PROFILE Statement

Syntax:

```
DROP PROFILE profile_name;
```

Notes:

- If a profile is dropped, users who have the profile assigned to them continue to have that profile assigned to them.
- Effects on sessions logged on with the profile that has been dropped:
 - Spool and temporary space settings immediately change user's settings.
 - Account and database settings change to user's settings on next log on or if user explicitly changes the settings.
 - Users may only explicitly change to an account in the list of account IDs available to them.
- The effects of re-creating a profile with the same name as the dropped profile are not immediate.
 - The parameter settings in the re-created profile take effect the next time the users log on.

The DROP PROFILE statement drops the named profile. The DROP PROFILE access right is required in order to execute this command.

The syntax is simply: `DROP PROFILE profile_name;`

When a profile is dropped, users who have the profile assigned to them continue to have that profile assigned to them; the system does *not* reset the profile for the affected users to NULL. Affected users receive no warnings or errors the next time they log on.

The effects of re-creating a profile with the same name as the dropped profile are not immediate. The parameter settings in the re-created profile take effect the next time that users (who are assigned the profile) log on.

DROP PROFILE has the following effects on users (sessions) logged on with the profile:

- Spool and temporary space settings immediately change to the settings defined for the affected users.
- Account and database settings change to the settings defined for the affected users the next time the users log on or explicitly change the settings.

However, changes to the list of valid account IDs take effect immediately. Users may only explicitly change to an account ID in the list of account IDs available to them.

ProfileInfo View

DBC.ProfileInfo[V][X]

ProfileName	DefaultAccount
DefaultDB	SpoolSpace
TempSpace	ExpirePassword
PasswordMinChar	PasswordMaxChar
PasswordDigits	PasswordSpecChar
PasswordRestrictWords	MaxLogonAttempts
LockedUserExpire	PasswordReuse
CommentString	CreatorName
CreateTimeStamp	LastAlterName
LastAlterTimeStamp	Queryband (15.10)
QuerybandDefault (15.10)	

Provides information about profiles that exist in the system.

Example:

List profiles that exist in the system.

```
SELECT ProfileName
      ,DefaultAccount AS "Def Acct"
      ,DefaultDB
      ,SpoolSpace
      ,TempSpace
      ,QueryBand
      ,QuerybandDefault AS QD
FROM   DBC.ProfileInfoV
ORDER BY 1;
```

Results

ProfileName	Def Acct	DefaultDB	SpoolSpace	TempSpace	Queryband	QD
Cust_Service_P	\$M_&D&H	CS_VM	2000000000	1000000000	Group=CS;	D
Cust_Gold_P	\$H_&D&H	CS_VM	2000000000	1000000000	? ?	
Employee_P	\$M00EDUC&D&H	HR_VM	10000000000	5000000000	TVSTEMPERATURE=NONE;	R
Payroll_P	\$M_&D&H	Payroll_VM	2000000000	1000000000	? ?	

The DBC.ProfileInfo view will list all profiles and their parameter settings. The DBC.ProfileInfoX view will list the profile, if any, and its parameter settings for the current user.

Extension to COMMENT command:

```
COMMENT [ON] PROFILE <profile name> [ [AS] <comment string> ]
```

- inserts or retrieves comments in CommentString column of the DBC.Profiles table for the named profile.

Like accounts for the DBC.Dbase table, only the default account is stored in the DBC.Profiles table. All other accounts associated with the profile will be stored in DBC.Accounts table. The ProfileInfoV view provides the first or default account ID.

Queryband

The Queryband column contains the profile query band string that is set in the user's session at logon. NULL indicates that this profile option is not set.

Possible Values for QuerybandDefault

The value in this column indicates whether the session or transaction query band can be set with the same name as in the profile query band. D=Default (can be changed), R=Restricted (can NOT be changed).

Miscellaneous SQL Functions

Example 1: As Emp01, identify the current user, role, profile, and database information.

```
SELECT USER, ROLE, PROFILE, DATABASE;
```

Result 1:

User	Role	Profile	Database
EMP01	HR_R	EMPLOYEE_P	HR_VM

Example 2: As Emp01, list the profile attributes.

```
SELECT * FROM DBC.ProfileInfoVX;
```

Result 2:

ProfileName	DefaultAccount	DefaultDB	SpoolSpace	TempSpace	...
Employee_P	\$M00EDUC&D&H	HR_VM	10000000000	5000000000	...

Example 3: As Emp01, list account information.

```
SELECT * FROM DBC.AccountInfoVX;
```

Result 3:

Name	AccountName	UserOrProfile
Employee_P	\$M00EDUC&D&H	Profile
Employee_P	\$L00EDUC&D&H	Profile
Emp01	\$M_&D&H	User

Example 4: As Emp01, list QueryBand information.

```
SELECT GetQueryBand();
```

Result 4:

GetQueryBand() =S> Group=HR; =P> TVSTEMPERATURE=NONE;
--

As you may notice, the DBC.AccountInfoV view has a column named UserOrProfile. This corresponds to the column named RowType in the DBC.Accounts table. The RowType column is necessary because profile and user names come from separate name spaces. Since profile accounts are also stored in DBC.Accounts, they will be confused with accounts of a user with the same name. Hence, the RowType column is necessary to distinguish a user account from a profile account. This column will have a value of P (for Profile) or U (for User).

This slide contains 3 SQL queries that a user can execute to identify their information.

Miscellaneous notes:

- If you are accessing the Teradata database through a proxy connection, CURRENT_USER returns the proxy user name. Otherwise, it functions exactly like the USER built-in function and returns the session user name.
- If you are accessing the Teradata database through a proxy connection, then CURRENT_ROLE returns the current role of the proxy user.

```
SELECT GetQueryBand();
```

The GetQueryBand() function returns the following concatenated profile, transaction and session query band text:

```
=P> profile query band
=T> transaction query band
=S> session query band
```

Summary

- A profile is a set of common user parameters that can be applied to a group of users.
- The CREATE PROFILE command is used to create a profile of desired attributes.
 - `CREATE PROFILE profile_name AS ... ;`
- The PROFILE option (new) is used with CREATE USER and MODIFY USER commands to assign a user to a specific profile.
 - `CREATE USER user1 AS ..., PROFILE = prof_name;`
 - `MODIFY USER user2 AS PROFILE = prof_name;`

This slide summarizes some important concepts regarding this module.



Module 7: Review Questions

Answer the following questions:

1. List two advantages of utilizing profiles. _____
2. If a user's profile is set to NULL, which two are immediately affected in the current session?
 - a. SPOOL value
 - b. Session priority
 - c. Default database
 - d. TEMPORARY value
3. Which two attributes can be set in a Profile?
 - a. Default Role
 - b. Query_Band
 - c. Default Database
 - d. Permanent Space

Match each term to the definition.

- | | |
|-------------------------------|---|
| 4. ____ CREATE PROFILE | a. Lists all words that can not be included in passwords |
| 5. ____ DBC.ProfileInfoV | b. System access right needed to create a profile |
| 6. ____ DBC.RestrictedWordsV | c. Lists the profiles currently in the system |
| 7. ____ DBC.SecurityDefaultsV | d. May be used to change password rules at the system level |

Check your understanding of the concepts discussed in this module by completing the review questions as directed by your instructor.



Lab Exercise 7-1 (1 of 2)

Purpose

In this lab, use Teradata Studio to create a profile and multiple users. This lab also provides an opportunity to use the DBC.ProfileInfoV view.

What you need

A user account with system profile privileges.

Tasks

1. Create a user profile with a profile name that is the same as your user name (e.g., "studentxxx_P" where xxx is your student number). The attributes of your profile are:

```
ACCOUNT          = '$M00FACT&D&H',
DEFAULT DATABASE = studentxxx (i.e., your database),
SPOOL            = 500E6,
TEMPORARY        = 500E6,
PASSWORD         = (MINCHAR=6, MAXLOGONATTEMPTS=3, LOCKEDUSEREXPIRE=5,
                   DIGITS='R', RESTRICTWORDS = 'Y', SPECCHAR = 'P'),
QUERY_BAND       = 'Group=PS;' (DEFAULT);
```

2. Use the DBC.ProfileInfoV view to display information about profiles in the system.

How many profiles are defined in the system? _____

Check your understanding of the concepts discussed in this module by completing the lab exercises as directed by your instructor.



Lab Exercise 7-1 (2 of 2)

3. Create two new users in the system with the following attributes.

User name: studentxxx_A (where xxx is your student number)
Perm space: 0
Password: Temp1234 (note that the password is different from the username)
Profile: studentxxx_P

User name: studentxxx_B (where xxx is your student number)
Perm space: 0
Password: Temp1234 (note that the password is different from the username)
Profile: studentxxx_P

4. Logon to Teradata as "studentxxx_A".

As studentxxx_A, use MODIFY USER to change their password to a different value (and remember this password).

Logon off as studentxxx_A and logon using the new password that you just created.

As studentxxx_A, issue the following SELECT statement.

```
SELECT COUNT(*) FROM studentxxx.EmpPhone;
```

Note: You should get an access right error (3523) with this SELECT since studentxxx_A should not have access rights on studentxxx.

Check your understanding of the concepts discussed in this module by completing the lab exercises as directed by your instructor.



Module 8: Access Rights

Teradata Vantage Administration

Copyright © 2007–2023 by Teradata. All Rights Reserved.

Objectives

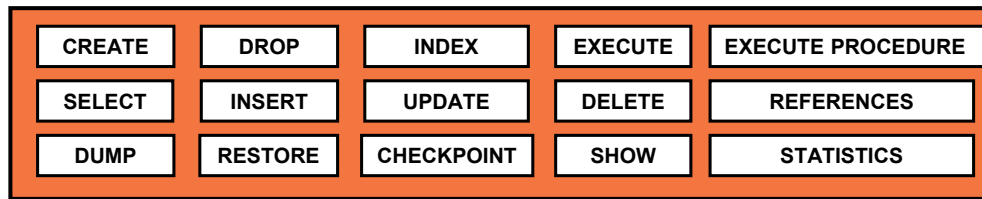
After completing this module, you will be able to:

- Use the DBC.AllRights, DBC.UserRights and DBC.UserGrantedRights views to obtain information about current users
- Use views and macros to access information about privileges
- Use the GRANT and REVOKE statements to assign and remove access rights
- Understand the impact of the GIVE statement with access rights

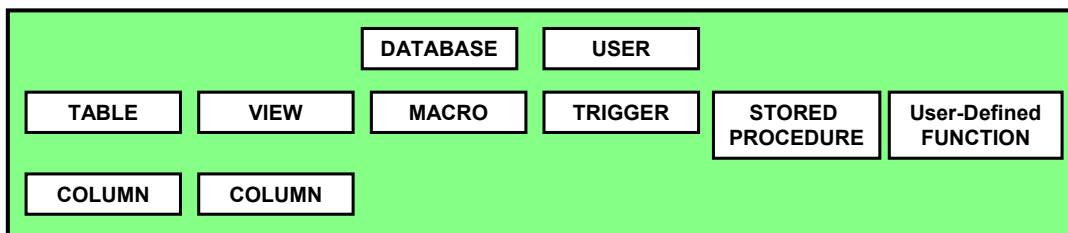
Privileges/Access Rights

A privilege (or access right) is the right of a specific user to perform a specified operation.

Note: Some access rights don't directly correspond to an SQL statement.



On a specified Object

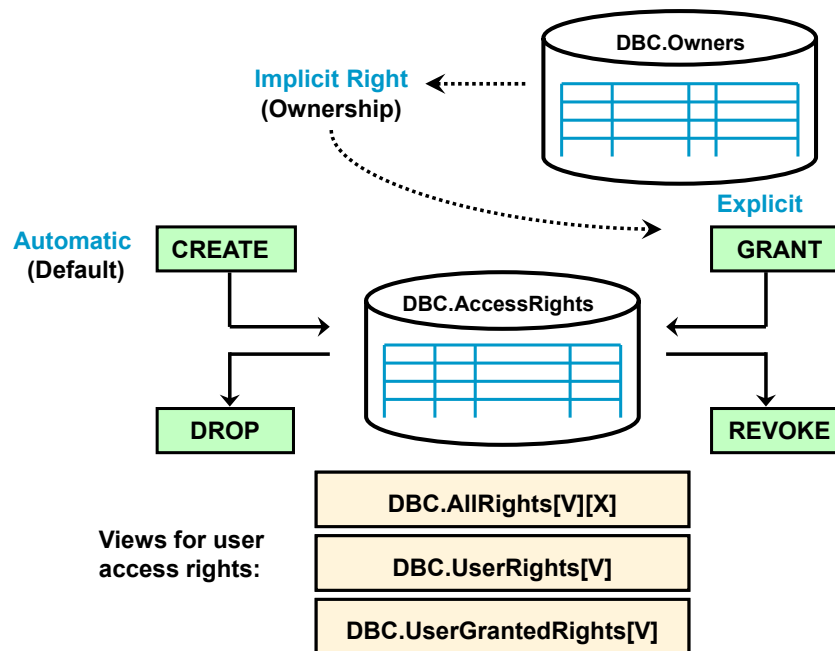


Your privileges (access rights) define the types of activities you can perform during a session. The top box identifies some of the common privileges that a user requires, but this is not an inclusive list of privileges.

Notes:

- To use UPDATE or DELETE commands, you must also have the SELECT right on the object.
- Additional rights you need to control access to performance monitoring functions are discussed in another module.
- A column can only be specified with the SELECT, INSERT, UPDATE or REFERENCES access right.
- SHOW privilege – this privilege enables you to have access to database object definitions and create text without having access to the data contained by the objects on which the privilege is granted. For example, SHOW permits a user to execute HELP and SHOW requests against an object while at the same time not being able to SELECT from it.
- STATISTICS privilege – allows a user to collect or drop statistics on an object (e.g., table). STATISTICS does not grant users the wider capabilities associated with those privileges. STATISTICS can be granted at both the table and database levels.

Access Rights Mechanisms



The `DBC.AccessRights` table contains information about the access rights assigned to existing users. The `DBC.AccessRights` table has NUPI on (UserId) and is a multilevel partitioned table.

Automatic Rights

Automatic rights are privileges given to creators and, in the case of users and databases, their created objects. When a user submits a `CREATE` statement, new rows are inserted in the `DBC.AccessRights` table. All rights are automatically removed for an object when it is dropped.

Explicit Rights

Explicit rights are privileges conferred by using a `GRANT` statement. This statement inserts new rows into the `DBC.AccessRights` table. Explicit rights can be removed using the `REVOKE` statement.

Ownership Rights

Owners (Parents) have the implicit right to grant rights on any or all of their owned objects (Children), either to themselves or to any other user or database. If an owner grants him or herself rights over any owned object, the parser will validate that `GRANT` statement even though the owner holds no other privileges. Ownership rights cannot be taken away unless ownership is transferred.

Inherited Rights

If an Owners (Parents) has the `ALL` option established, children will inherit the rights associated with the `ALL` flag. Inherited rights will be discussed in detail later in the module.

CREATE TABLE – Automatic Rights

```
EXPLAIN CREATE TABLE TFACT.Customer
  ( CustomerNumber INTEGER, LastName CHAR(30),
    FirstName CHAR(20), SocialSecurity INTEGER)
  UNIQUE PRIMARY INDEX (CustomerNumber)
  UNIQUE INDEX (SocialSecurity);
```

- 1) First, we lock TFACT.Customer in TD_MAP1 for exclusive use.
 - 2) Next, we lock a distinct DBC."pseudo table" in TD_DATADITIONARYMAP for write on a RowHash for deadlock prevention, we lock a distinct DBC."pseudo table" in TD_DATADITIONARYMAP for write on a RowHash for deadlock prevention, we lock a distinct DBC."pseudo table" in TD_DATADITIONARYMAP for read on a RowHash for deadlock prevention, and we lock a distinct DBC."pseudo table" in TD_DATADITIONARYMAP for write on a RowHash for deadlock prevention.
 - 3) We lock DBC.AccessRights in TD_DATADITIONARYMAP for write on a reserved RowHash in a single partition to prevent global deadlock.
 - 4) We lock DBC.DBase in TD_DATADITIONARYMAP for read on a RowHash, we lock DBC.Maps in TD_DATADITIONARYMAP for read on a RowHash, we lock DBC.TVFields in TD_DATADITIONARYMAP for write on a RowHash, we lock DBC.Indexes in TD_DATADITIONARYMAP for write on a RowHash, we lock DBC.TVM in TD_DATADITIONARYMAP for write on a RowHash, we lock DBC.ObjectUsage in TD_DATADITIONARYMAP for write on a RowHash, and we lock DBC.AccessRights in TD_DATADITIONARYMAP for write on a single partition.
 - 5) We execute the following steps in parallel.
 - 1) We do a single-AMP ABORT TEST step from DBC.DBase by way of the unique primary index "Field_1 = 'TFACT'" with a residual condition of ("'0000F906'XB= DBC.DBase.Field_2").
 - : :
 - 12) We INSERT default rights to DBC.AccessRights for TFACT.Customer.
 - 6) We create the table header in TD_MAP1.
 - 7) We create the index subtable on TFACT.Customer.
 - 8) We modify the table header TFACT.Customer.
 - 9) Finally, we send out an END TRANSACTION step to all AMPs involved in processing the request.
- > No rows are returned to the user as the result of statement 1.

In step 5 (parallel step 12 of this Explain plan), you can see that the system adds default access rights (automatic rights) to the AccessRights table.

The following 12 access rights are inserted in Teradata 16.20 and each right also with the grant authority when a new table is created.

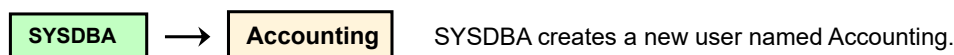
- SELECT (R)
- INSERT (I)
- UPDATE (U)
- DELETE (D)
- DROP TABLE (DT)
- INDEX (IX)
- REFERENCES (RF)
- CREATE TRIGGER (CG)
- DROP TRIGGER (DG)
- DUMP (DP)
- RESTORE (RS)
- STATISTICS (ST)

If a view is created, 5 access rights are added.

Creation of a macro causes 2 access rights to be added.

CREATE USER – Automatic Rights

By issuing a CREATE USER (or DATABASE) statement, the CREATOR causes Automatic privileges to be generated for both the created user or database AND the creator.



Both SYSDBA and Accounting are given the following privileges on the Accounting user.

CREATE Table	DROP Table	CREATE View	DROP View
CREATE Macro	DROP Macro	CREATE Trigger	DROP Trigger
SELECT	INSERT	UPDATE	DELETE
EXECUTE	DROP Procedure	DROP Function	DUMP
RESTORE	CHECKPOINT	CREATE Authorization	DROP Authorization
STATISTICS			

The creator (SYSDBA) is also given these 4 additional privileges on the Accounting user.

CREATE Database	DROP Database	CREATE User	DROP User
-----------------	---------------	-------------	-----------

When you create a new user or database, the system automatically generates access rights for the created object and the creator of the object. The system inserts this rights information into the DBC.AccessRights table when you submit a CREATE request. You can remove these rights from the DBC.AccessRights table with the REVOKE statement.

Example

In this example, user SYSDBA logs on to the system and creates a new user called Accounting. Both SYSDBA and Accounting have the 21 privileges (shown on this slide) written into the DBC.AccessRights table:

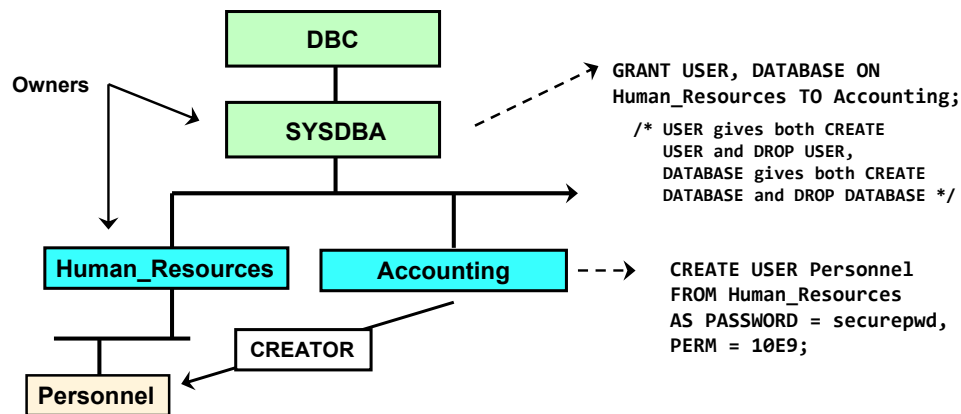
As the creator, SYSDBA is also given these 21 privileges PLUS the 4 following rights on Accounting:

- CREATE Database/User
- DROP Database/User

These access rights that are given to a new user and the creator are the same starting with Teradata 14.0 through Teradata 16.20.

The same number of access rights are also established if you are creating a new database.

Implicit, Automatic, and Explicit Rights



Where does the 10E9 space for Personnel come from?

How many automatic access rights are created for Personnel?

How many automatic access rights are created for Human_Resources?

How many automatic access rights are created for Accounting?

Implicit rights belong to the owners of objects. Owners do not require rows in the DBC.AccessRights table to grant privileges on owned objects. Ownership rights cannot be “revoked.” An owner has the implicit right to GRANT privileges over any owned object.

When you submit a CREATE statement, the system automatically adds new rows to the DBC.AccessRights table. You can remove automatic rights with the REVOKE or DROP statements.

GRANT and REVOKE statements control explicit rights. The GRANT statement adds new rows to the DBC.AccessRights table. The REVOKE statement removes them.

In the example, Accounting is the creator. The system automatically inserts rows for access rights in DBC.AccessRights for the creator (Accounting) and for the created user (Personnel). These rights can be revoked.

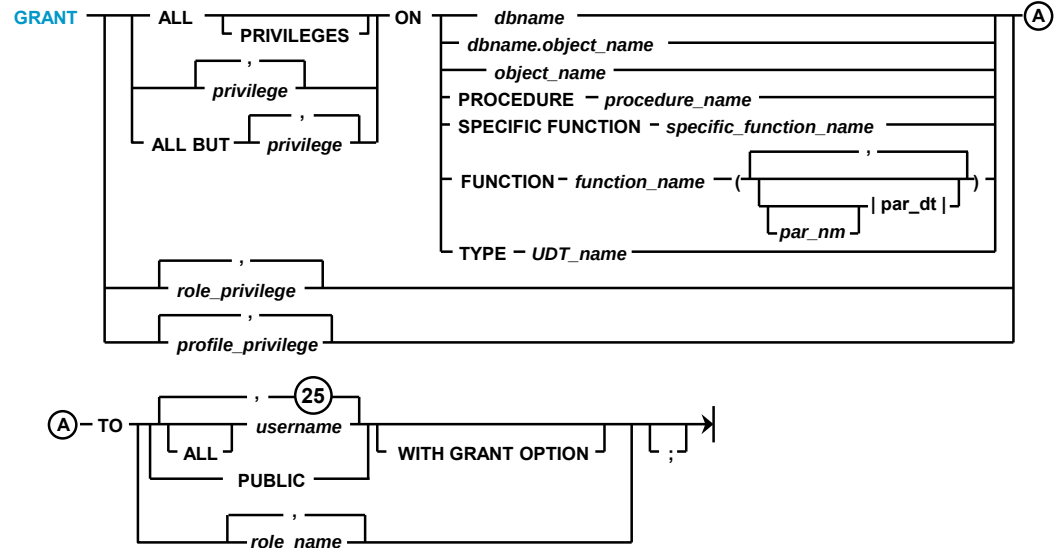
The user named Personnel is the created object. The user Personnel automatically receives all but four access rights on itself. These rights are inserted *automatically* in DBC.AccessRights. These rights can be revoked.

Assume that Human_Resources is a user and is the immediate owner. The system does not insert any rows in the Data Dictionary for Human Resources. If Human_Resources is a user, then Human_Resources has the owner’s implicit right to grant itself rights over Personnel. You cannot revoke the right to GRANT (or re-GRANT) rights over owned objects.

GRANT Command (SQL Form)

To GRANT a privilege, the user (grantor) must have one of the following:

- Have the privilege granted **AND** hold GRANT authority on the privilege.
- Be an owner of the object.



You can use the GRANT statement to give to users, databases, or roles one or more privileges on a database, user, table, view, macro, trigger, stored procedure, or user-defined function.

To grant a privilege, you must:

- Have the privilege itself and have GRANT authority OR
- Be an owner

The recipient of an explicitly granted privilege may be:

- Username The specific user(s) or database(s) named
- PUBLIC Every user in the DBC system (same as ALL DBC)
- ALL username The named user and ALL descendants
- Role Specified role or roles

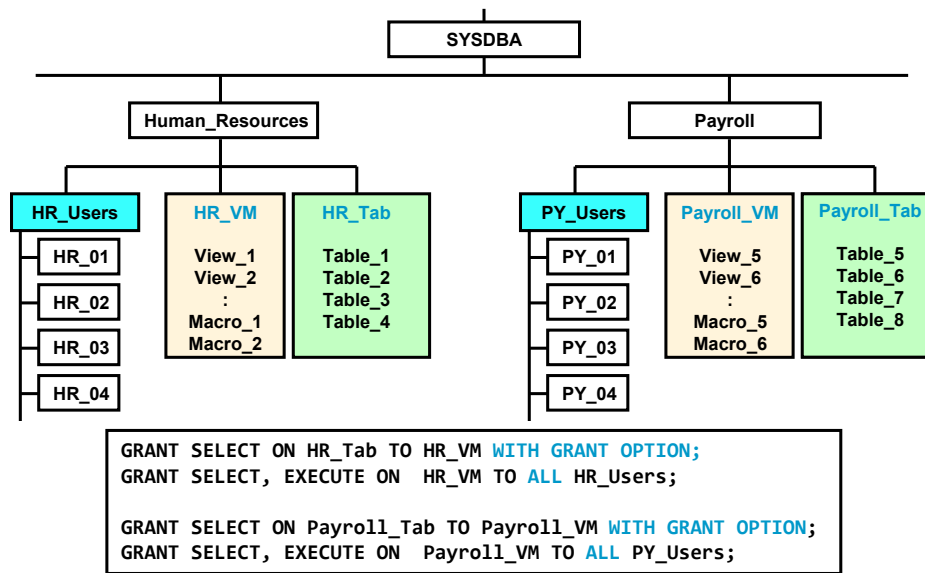
Access rights that a new user inherits because the ALL or PUBLIC option is used are referred to as “inherited rights”.

The **WITH GRANT OPTION** confers on the recipient “Grant Authority”. The recipient (or “Grantee”), holding this authority, may then grant the access right to other users, databases, or roles.

Example of syntax for REFERENCES or UPDATE access right for a column:

```
GRANT REFERENCES [(columnname list or ALL BUT column_name_list)] ...
GRANT UPDATE [(columnname list or ALL BUT column_name_list)] ...
```

Granting Rights at Database Level



The ALL option grants the SELECT and EXECUTE privileges to HR_Users and all of its current and future descendants on the database HR_VM.

This slide illustrates privileges granted at the database level.

A system structure for the Teradata database is shown on this slide and this hierarchy will be used in numerous examples.

Keys to the hierarchy on this slide are:

- HR_Users – users that require SELECT and EXECUTE access rights on the views and macros in the HR_VM database.
- PY_Users – users that require SELECT and EXECUTE access rights on the views and macros in the Payroll_VM database.

The database HR_VM will have the SELECT WITH GRANT OPTION access right on the database named HR_Tab.

The database Payroll_VM will have the SELECT WITH GRANT OPTION access right on the database named Payroll_Tab.

GRANT Rights at the Table or Column Level

The **SELECT**, **INSERT**, **UPDATE**, and **REFERENCES** privileges can be granted at the table or at the column(s) level.

To allow a user to **UPDATE** specific columns or restrict updating specific columns in a table:

```
GRANT UPDATE (LastName, FirstName) ON Employee TO tfact01;
GRANT UPDATE (ALL BUT Rank, SalaryAmount) ON Employee TO tfact01;
```

To allow a user to **CREATE** or **ALTER** a table with foreign key references:

```
GRANT REFERENCES ON Employee TO tfact01;
GRANT REFERENCES (Emp#, Dept#, JobCode) ON Employee TO tfact01;
```

To allow a user to only **INSERT** values for specific columns into a table:

```
GRANT INSERT (LastName, FirstName, Dept#) ON Employee TO tfact01;
```

The user must specify the allowed columns as part of the **INSERT** statement – other columns would require **DEFAULT** values when the table was created.

```
INSERT INTO Employee (LastName, FirstName, Dept#) VALUES ('Sands', 'Carter', 23);
```

The **INDEX** privilege is granted to allow a user to **CREATE** or **DROP** indexes on a table and the **STATISTICS** privilege is granted to allow the user to collect statistics on the table.

```
GRANT INDEX, STATISTICS ON Employee TO tfact01;
```

Access Rights and Triggers

To create or replace a trigger, specific access rights are required.

Access Rights to Create Triggers:

- **CREATE TRIGGER** privilege on the subject table or the database.
- **SELECT** privilege on any column referenced in a **WHEN** clause or a triggered SQL statement subquery.
- **INSERT**, **UPDATE**, or **DELETE** privileges on the triggered SQL statement target table, depending on the triggered SQL statement.

Access Rights to Replace Triggers:

- **DROP TRIGGER** privilege on the subject table or the database. The exception is when you use the **REPLACE TRIGGER** statement when no target trigger exists and you instead create a new trigger.
- **SELECT** privilege on any column referenced in a **WHEN** clause or a triggered SQL statement subquery.
- **INSERT**, **UPDATE**, or **DELETE** privileges on the triggered SQL statement target table, depending on the triggered SQL statement.

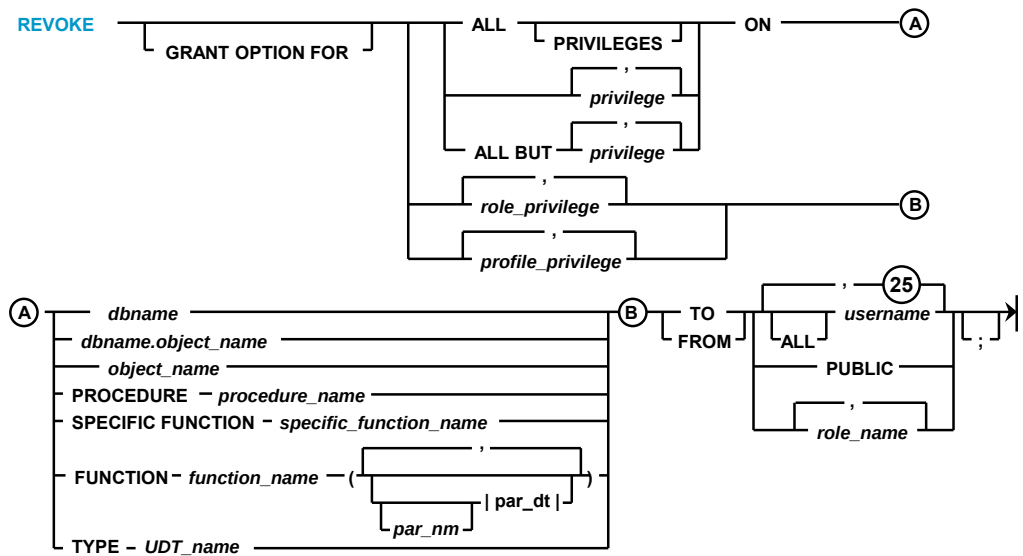
Example:

```
CREATE TRIGGER trigger1
  AFTER UPDATE OF (col1) ON table1 FOR EACH ROW
  WHEN NEW col1 > 100
  INSERT INTO log_table VALUES ...
```

REVOKE Command (SQL Form)

To REVOKE a privilege, the user must have one of the following:

- Have the privilege granted **AND** hold GRANT authority on the privilege
- Be an owner of the object.



REVOKE is passive in that it:

- Does *not* add rows to DBC.AccessRights.
- Removes rows from the DBC.AccessRights table *only* if the privileges specified exist.
- Does *not* cascade through the hierarchy unless you specify the “ALL username” option.
- Is not automatically issued for privileges granted by a grantor dropped from the system.

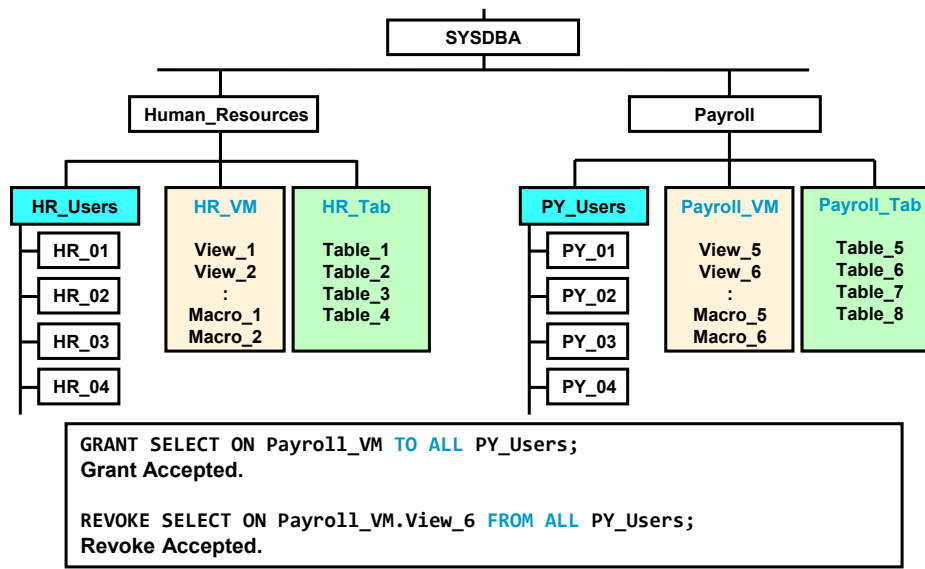
The REVOKE statement removes rights inserted in the DBC.AccessRights table by a CREATE statement. It can also remove explicit rights inserted in the DBC.AccessRights table by the GRANT statement.

REVOKE Recipients

The REVOKE statement can remove privileges from one of the following:

- | | |
|----------------|---|
| • username | A specific named user(s) |
| • PUBLIC | Every user in the DBC system |
| • ALL username | The named user and all of his descendants |
| • Role | Specified role or roles |

Revoking Non-Existent Rights



REVOKE is passive. It does not add rows to DBC.AccessRights, but removes rows if they exist.

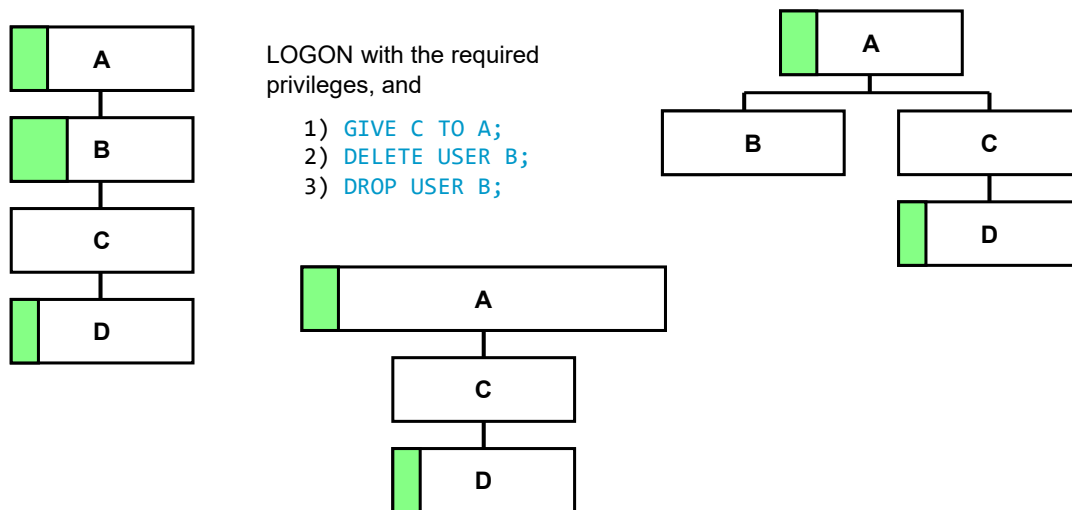
A REVOKE statement at the object level cannot remove privileges from that object that were granted at the database or user level because there is no correlating row in the DBC.AccessRights table for the individual object.

Example

The illustration on this slide illustrates privileges granted at the database level. User Payroll logs on to the system, and grants the SELECT privilege to user PY_Users and ALL of its descendants on the database Payroll_VM.

Later, Payroll REVOKES the SELECT privilege from ALL PY_Users only on View_6 that resides in Payroll_VM. Although the system returns the message “Revoke Accepted,” nothing actually happened. The user PY_Users and its descendants still have the SELECT privilege on all views residing in database Payroll_VM because the DBC.AccessRights table does not have a row correlating to View_6. Since the row granting select at the database level is still intact, all access rights remain in effect.

Removing a Level in the Hierarchy



Although B no longer exists as a user, the privileges granted or caused to be granted are not automatically revoked.

The example on this slide demonstrates how to remove a level from an existing hierarchy. In the first diagram, user A is the owner of users B, C, and D. User A no longer needs user B. He wants to keep users C and D.

Transfer Ownership

The first thing user A needs to do is transfer ownership of user C to A. When user A submits the GIVE statement, both user C and user D will be transferred. That is because the GIVE statement transfers the named object and all of its children. Since user D is a child of user C, both objects are transferred under user A.

Delete User or Database

In order to DROP user B, user A must first delete all objects from user B. The DELETE USER command will delete all data tables, views, triggers, stored procedures, and macros from a database or user. This command will not remove a Permanent Journal, Hash Indexes, or Join Indexes from a user or database.

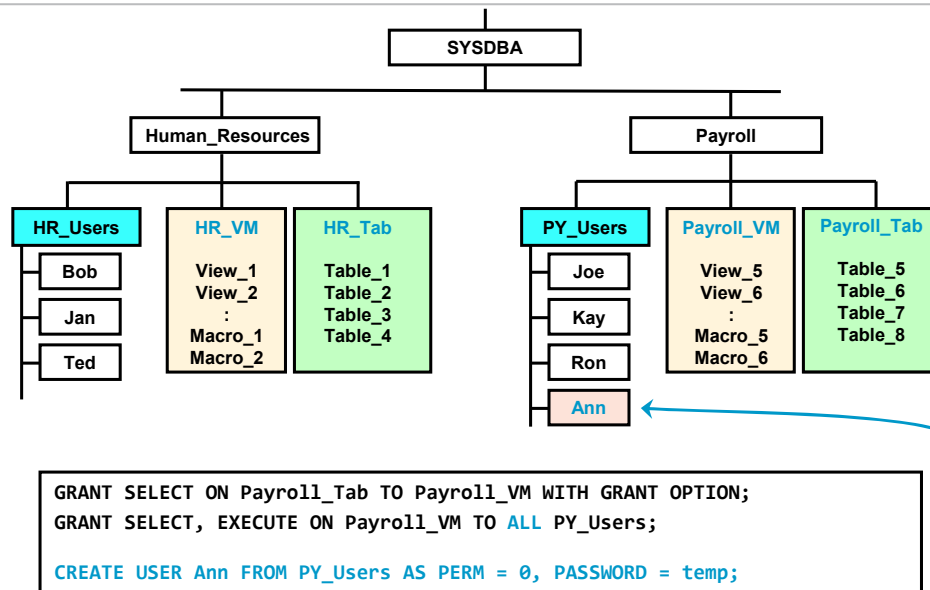
Drop User

After user A removes all objects from user B, user A can submit the DROP statement.

Access Rights

The privileges for user C and user D remain intact. Although user B, their original creator, no longer exists, the privileges granted or caused to be granted are not automatically revoked. Note that user A has recovered the perm space held by user B.

Inheriting Access Rights



Ann "inherits" the SELECT and EXECUTE access rights for the database Payroll_VM.

You may inherit access rights by the placement of your user in the hierarchy. As an administrator, you can set up access rights so that any new object added to an existing user or database inherits specific access rights. Doing so saves time since you do not need to submit a GRANT statement each time you add a new user.

The immediate owner (user or database) of a view or table that is referenced by another must have the right on the referenced object that is specified (SELECT, EXECUTE, etc.) and must have that right with the GRANT option.

Example

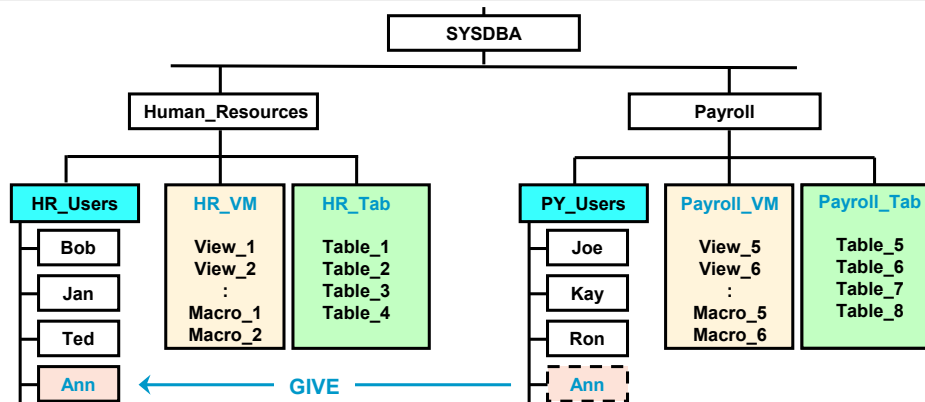
The example on this slide illustrates a user inheriting access rights.

The user Human_Resources logs on the system and grants the SELECT and EXECUTE privileges to user HR_Users and all of its current and future descendants on the database HR_VM.

The user Payroll also logs on the system and grants the SELECT and EXECUTE privileges to user PY_Users and all of its current and future descendants on the database Payroll_VM.

Later, Payroll creates a new user called Ann from the space owned by user PY_Users. Ann inherits the SELECT and EXECUTE privileges on database Payroll_VM database.

The GIVE Statement and Access Rights



NOT Recommended

```
.LOGON sysdba, password;
GIVE Ann TO HR_users;
```

The GIVE command transfers ownership, but does not change any access rights.

Possible, but recommended
to utilize Roles

```
.LOGON sysdba, password;
DROP USER Ann;
CREATE USER Ann FROM HR_Users ...;
```

The DROP will cause Ann's access rights to be removed for Payroll_VM. The CREATE will allow Ann to inherit access rights for HR_VM.

When you give a user to another owner, privileges are not altered. **The GIVE statement does not alter DBC.AccessRights.** No rights on the given database or user are granted to the new ownership hierarchy as a result of the GIVE statement. The database or user that you GIVE does not receive any access rights from its new owner. The new owner gains *implicit* access rights over the transferred object and the old owner loses them.

Example

In the example on this slide, Sysdba logs on to the system and gives user Ann to HR_Users. Ann retains the privileges that she inherited from PY_Users when she was created. Ann does not inherit any access privileges from the new owner, HR_Users, or from Human_Resources

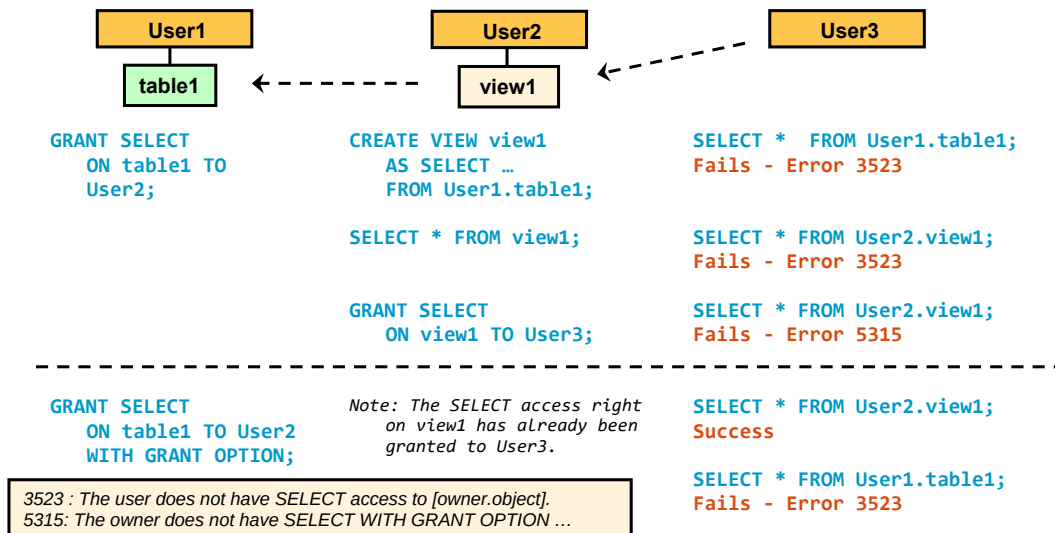
HR_Users is Ann's new owner. It has ownership rights over Ann. PY_Users loses ownership rights over Ann when she is transferred.

The syntax of the GIVE statement is as follows:

```
GIVE database_name TO recipient_name;
```


Access Rights and Views

- View names are fully expanded (resolved) at creation time.
- The system checks access rights at creation time, and validates them again at execution time.



Views may be nested up to 64 levels. View names are fully expanded (resolved) at creation time. This slide shows an example of a nested view.

The system checks access rights at creation time, and validates them again at execution time. Any database referenced by the view requires access rights on all objects accessed by the view.

You can create a view with the intention of read access only, or for controlled UPDATES use. For read access, the SELECT right is needed. For updates, the UPDATE right is needed.

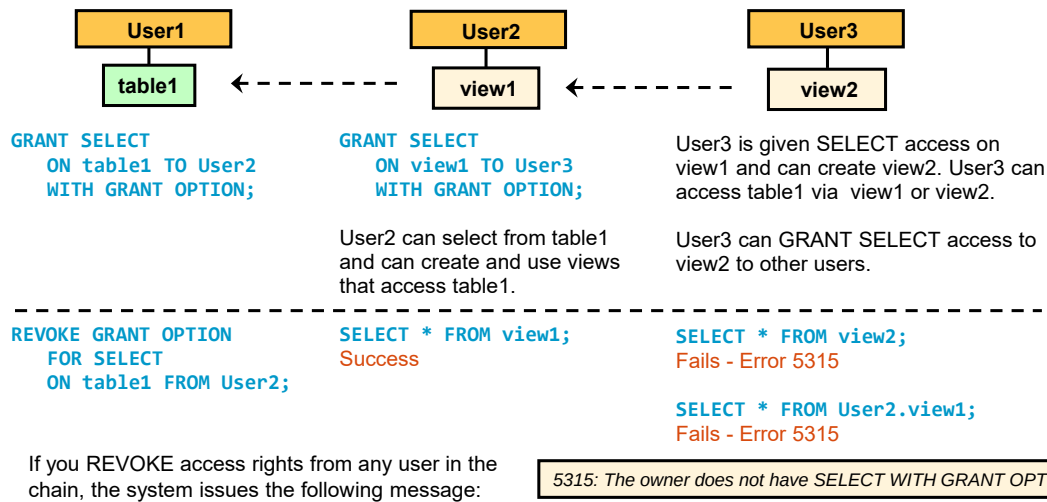
For other users to access a view, the owner must grant the appropriate rights on the view and must have the appropriate rights WITH GRANT OPTION.

The system verifies that the creator has the appropriate right on the objects being referenced when a view is created. It also verifies that the creator has the rights needed to execute the statements defined in a macro. To grant to another user any privilege on a view or macro that references objects owned by a third user, the owner of the view or macro must have the appropriate rights with GRANT OPTION.

Teradata also verifies that the appropriate privileges exist on the objects being referenced for any user who attempts to access a view or execute a macro. This ensures that a change to a referenced object does not result in a violation of access rights when the view or macro referencing that object is invoked.

Access Rights and Nested Views

- Views that reference other views are called nested views. Views may be nested up to 64 levels.
- The system validates access rights at execution time.



The previous example is continued on this slide.

Views that reference other views are sometimes called nested views. Views may be nested up to 64 levels. View names are fully expanded (resolved) at creation time.

The system checks access rights at creation time, and validates them again at execution time. Any database referenced by the view requires access rights on all objects accessed by the view.

Note: Assume User2 only has the SELECT access right on table1 without WITH GRANT OPTION). This would be the case if the REVOKE GANT OPTION FOR SELECT has been executed.

If User3 has also been granted the SELECT access right on table1 (not shown) and also has SELECT on User2.view1, you might think that User2 can access table1 via User2.view1. However, this will not work and the following statements indicate this.

- User3 can SELECT directly from table1.
- If User3 executes a SELECT on User2.view1 (effectively to access table1), this query will fail because Teradata validates if User2 has SELECT WITH GRANT and since User2 does not, the query will file

System Views for Access Rights

<u>View</u>	<u>Description</u>
DBC.AllRights[V][X]	Provides information about all rights that have been automatically or explicitly granted.
DBC.UserRights[V]	Provides information about all rights the user has acquired, either automatically or explicitly.
DBC.UserGrantedRights[V]	Provides information about rights which the current user explicitly has granted to other users.

There are three system views you can use to obtain information about access rights. (These views access the DBC.AccessRights table to obtain needed information.) They are:

- DBC.AllRights[V][X]
- DBC.UserRights[V]
- DBC.UserGrantedRights[V]

DBC.AllRights[V][X]

The DBC.AllRights[X] views provide information about all rights that have been automatically or explicitly granted.

DBC.UserRights[V]

This view provides information about all rights that the user has acquired, either automatically or explicitly.

DBC.UserGrantedRights[V]

This view provides information about rights that the current user has explicitly granted to other users.

AllRights and UserRights Views

Provides information about the objects on which all users (DBC.AllRights), or the current user (DBC.UserRights), have automatically or explicitly been granted privileges.

DBC.AllRights[V][X]

UserName	DatabaseName
TableName	ColumnName
AccessRight	GrantAuthority
GrantorName	AllnessFlag
CreatorName	CreateTimeStamp

DBC.UserRights[V]

DatabaseName	TableName
ColumnName	AccessRight
GrantAuthority	GrantorName
CreatorName	CreateTimeStamp

Example:

All rights held by the user at the database level (for user tfact07).

```
SELECT  CAST (DatabaseName AS CHAR(15))
        ,AccessRight
        ,CAST (GrantorName AS CHAR(15))
FROM    DBC.UserRightsV
WHERE   Tablename = 'ALL'
ORDER BY 1, 2;
```

DatabaseName	AccessRight	GrantorName
AP	R	DBC
PD	D	SYSDBA
PD	I	SYSDBA
PD	CT	SYSDBA
PD	CG	SYSDBA
tfact07	CP	SYSDBA

Examples of Privileges/Access Rights (not an inclusive list) and their codes include:

AE = ALTER EXTERNAL PROCEDURE	AF = ALTER FUNCTION
AP = ALTER PROCEDURE	AS = ABORT SESSION
CA = CREATE AUTHORIZATION	CD = CREATE DATABASE
CE = CREATE EXTERNAL PROCEDURE	CF = CREATE FUNCTION
CG = CREATE TRIGGER	CM = CREATE MACRO
CO = CREATE PROFILE	CP = CHECKPOINT
CR = CREATE ROLE	CT = CREATE TABLE
CU = CREATE USER	CV = CREATE VIEW
D = DELETE	DA = DROP AUTHORIZATION
DD = DROP DATABASE	DF = DROP FUNCTION
DG = DROP TRIGGER	DM = DROP MACRO
DO = DROP PROFILE	DP = DUMP
DR = DROP ROLE	DT = DROP TABLE
DU = DROP USER	DV = DROP VIEW
E = EXECUTE	EF = EXECUTE FUNCTION
I = INSERT	IX = INDEX
MR = MONITOR RESOURCE	MS = MONITOR SESSION
NT = NONTEMPORAL	PC = CREATE PROCEDURE
PD = DROP PROCEDURE	PE = EXECUTE PROCEDURE
RO = REPLICATION OVERRIDE	R = RETRIEVE/SELECT
RF = REFERENCE	RS = RESTORE
U = UPDATE	UU = UDT Usage

UserGrantedRights View

Provides information about objects on which the current user has explicitly granted privileges to other users.

DBC.UserGrantedRights[V]

DatabaseName	TableName
ColumnName	Grantee
AccessRight	GrantAuthority
AllnessFlag	CreatorName
CreateTimeStamp	

Example:

List the rights explicitly granted by the current user.

```
SELECT  CAST (DatabaseName AS CHAR(12))
        ,CAST (TableName   AS CHAR(15))
        ,CAST (Grantee     AS CHAR(10))
        ,AccessRight
        ,AllnessFlag
FROM    DBC.UserGrantedRightsV
ORDER  BY 1, 2, 3, 4;
```

Results:

<u>DatabaseName</u>	<u>TableName</u>	<u>Grantee</u>	<u>AccessRight</u>	<u>AllnessFlag</u>
AP	All	tfact07	R	N
DS	Daily_Sales	tfact03	R	N
DS	Daily_Sales	tfact03	RF	N
DS	Order_Item_JI	tfact03	IX	N
PD	All	Students	R	Y

The DBC.UserGrantedRights[V] view provides information about objects on which the current user has explicitly granted privileges. When you submit the GRANT statement, the system stores explicit privileges as rows in the DBC.AccessRights table.

Column definitions in this view include:

<u>Column</u>	<u>Definition</u>
Grantee	The recipient of the access right.
AllnessFlag	Y (Yes) indicates the privilege was granted to all. N (No) indicates the privilege was not granted to all.

Teradata Studio – GRANT/REVOKE Privileges

Teradata Studio can be used to easily grant or revoke access rights (privileges).

- Select the object name and right mouse click “Modify Privileges”
- Select Object Level Privileges
- Select Grantee type of “User”
- Select Grantees then a Display Privileges
- Click Display Selected
- Then you can modify by clicking. Here added Show and revoked Statistics.

Table Name	Access Co...	Last Acce...	Version	Creator Na...	Creation Timesta...	Comment
Employee			1	BS186023	2021-11-09 17:4...	
Employee_2			1	BS186023	2021-11-09 18:3...	
Super_Bowl_CP...			1	BS186023	2021-12-20 18:0...	
tableA			1	BS186023	2021-12-15 14:4...	

Viewing Tables List, Count = 4

*Modify Privileges

Privileges for Table Employee

Commit Reset Cancel

Object Level Privileges

Column Rights

SQL

Grantee Type: User

Users Root Database: BS186023

Grantees: BS186023

Display Privileges

☒ Access

☐ Create

☐ Dictionary

☐ Drop

☐ Host Utilities

☐ Row Level Security

Grant Options

☐ All Children

☐ With Grant

Display Selected

Legend: ☐ Not granted ☒ Granted ☒ Granted with grant ☐ Revokes ☒ Grants

Grants with grant

Name	Delete	Insert	Nontemporal	Select	Show	Statistics	Update
BS186023	☒	☒	☐	☒	☒	☒	☒

This slide contains an example of the Grant/Revoke dialog box that is provided when using the menus of Teradata Studio.

Click SQL side tab to see code.

GRANT Show ON "BS186023"."Employee" TO "BS186023";
REVOKE Statistics ON "BS186023"."Employee" FROM "BS186023";

The help in Teradata Studio can provide additional information regarding Granting/Revoking Privileges.

Summary

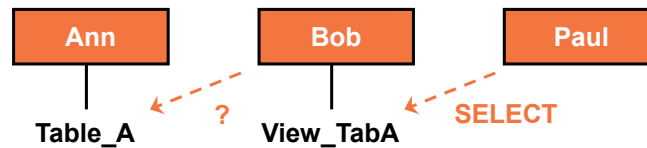
- Access Rights (Privileges) are maintained in the data dictionary.
- Rows are inserted into or removed from DBC.AccessRights by:
 - CREATE or DROP statements
 - GRANT or REVOKE statements
- Creators are given automatic rights on created objects.
- A newly created user or database is given all rights on themselves except:
 - CREATE Database/User
 - DROP Database/User
- Owners have the right to grant privileges on their owned objects.
- The GIVE command affects ownership, but not information in the DBC.AccessRights table.

This slide summarizes some important concepts regarding this module.



Module 8: Review Questions

1. True or False: There are only two ways used to create new access rights: explicit and implicit.
2. True or False: The primary statements you use to manage access rights are GRANT, REVOKE, and GIVE.
3. The _____ option on the GRANT command grants privileges to a database or user and all of its current and future descendants.
4. The _____, _____, _____, and _____ access rights can be granted at the column level.
5. Which user name is used when you want to grant an access right to every user? _____
6. Given the following: Ann owns Table_A, Bob creates View_TabA and grants SELECT on View_TabA to Paul.
What access right does Ann give Bob on Table_A so Paul can use View_TabA to access Table_A?



Check your understanding of the concepts discussed in this module by completing the review questions as directed by your instructor.



Module 9: Roles

Teradata Vantage Administration

Copyright © 2007–2023 by Teradata. All Rights Reserved.

Objectives

After completing this module, you should be able to:

- List three advantages of utilizing roles
- Identify the access rights needed to create roles
- Number the steps of access right validation
- Specify the order of precedence of user/session parameters
- Specify the default role when creating new users
- Use system views to display role information

What is a Role?

A Role is an administration/security features which can help simplify the management of access rights.

What is a “role”?

- A role is simply a collection of access rights.
 - Rights are first granted to a role and the role is then granted to users.
- A DBA can create different roles for different job functions and responsibilities.

What are the advantages of using “roles”?

- Simplify access rights management by allowing grants and revokes of multiple rights with one request.
 - useful when an employee changes job function (role) within the company.
 - if a job function needs a new access right, grant it to the role and it is effective immediately.
- The number of access rights in the DBC.AccessRights table is reduced.
- Improves performance and reduces dictionary contention for DDL, especially CREATE USER.
 - Removal of hierarchical inherited rights improves DDL performance and reduces dictionary contention.

A role can be viewed as a pseudo-user with privileges on a number of database objects. Any user granted a role can take on the identity of the pseudo-user and access all of the objects it has rights to. A database administrator can create different roles for different job functions and responsibilities, grant specific privileges on database objects to these roles, and then grant these roles to users.

Advantages of Roles

- **Simplify access rights administration** – a database administrator can grant rights on database objects to a role and have these rights automatically applied to all users assigned to that role. When a user's function within his organization changes, it is easier to change his/her role than deleting old rights and granting new rights that go along with the new function.
- **Reduce disk space usage** – maintaining rights on a role level rather than on an individual level makes the size of the DBC.AccessRights table much smaller. Instead of inserting one row per user per right on a database object, one row per role per right is placed in the DBC.AccessRights table.
- **Better performance** – roles can improve performance and reduces dictionary contention for DDL. If roles are fully utilized on a system, roles will reduce the size of the AccessRights table and improve the performance of DDL commands that do full-table scans of this table.
 - Faster DROP/DELETE USER/DATABASE, DROP TABLE/VIEW/MACRO due to shorter scans of the DBC.AccessRights table.
 - Faster CREATE USER, DATABASE - remove copy of hierarchical inherited rights.
 - Less dictionary contention during DDL operations because the commands use less time.

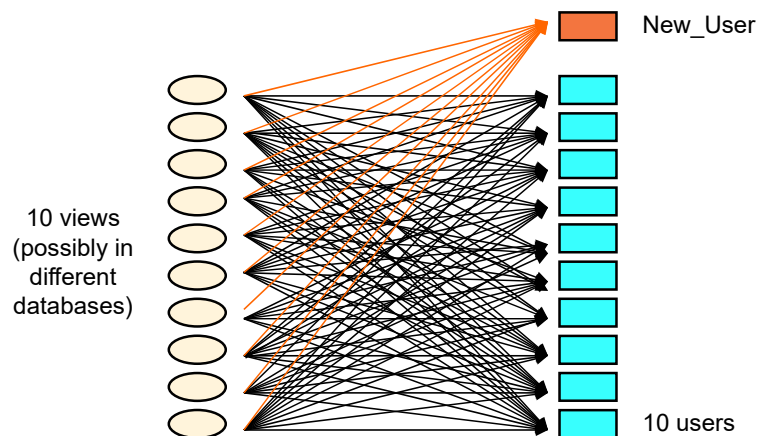
Access Rights Without Roles

```
GRANT SELECT ON View1 TO New_User;
GRANT SELECT ON View2 TO New_User;
```

...

When a new user is given the SELECT access right to these 10 views, 10 new access right rows are added to the DBC.AccessRights table.

In this simple example, these 10 views and 11 users would place 110 access right rows in the DBC.AccessRights table.



This slide illustrates the following:

- If 10 users have the SELECT access right on each of 10 views, there would be 100 rows in the DBC.AccessRights table for these 10 users.
- What if there were 50,000 users in the system and there were 500 views needed by each user? The DBC.AccessRights table would have 25 million rows.

When a new user is added in this example, 10 rows have to be added to the DBC.AccessRights table.

Access Rights Issues (prior to Roles)

The role concept provides a solution to the following problem. Without using roles, there are typically 2 ways of granting rights to a large user base:

1. Use the ALL option of the GRANT statement to grant rights on the shared object(s) to a parent database. Sometimes this is referred to as a “parent” database or a “group” database without using roles.

```
GRANT SELECT ON database_object TO ALL parent_database;
```

Then, create users under the parent database. The system will automatically grant all rights held by the parent database to each user created under the parent database. This is referred to as “inherited rights”.

2. Grant the rights to users individually – an administrative nightmare.

Access Rights Using a Role

First, create a role and grant privileges to the role.

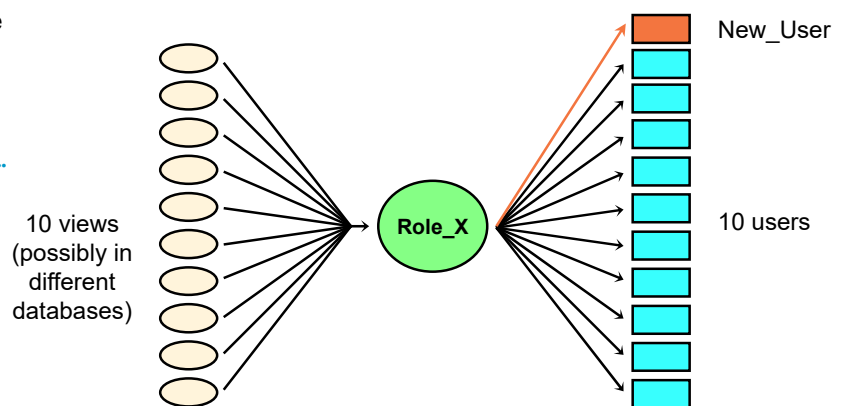
```
CREATE ROLE Role_X;
GRANT SELECT ON View1 TO Role_X;
GRANT SELECT ON View2 TO Role_X; ...
```

When creating a new user, only one right to use a role needs to be granted.

```
GRANT Role_X TO New_User;
```

This command places a row in the DBC.RoleGrants table, not DBC.AccessRights.

With 10 views, 1 role, and 11 users, there would be 10 rows in the DBC.AccessRights table and 11 rows in the DBC.RoleGrants table.



When creating a new user, only one right to use a role needs to be granted, as opposed to a right for every table/view/macro/stored procedure that the user needs to access.

As mentioned earlier, a role can be viewed as a pseudo-user with privileges on a number of database objects. Any user granted a role can take on the identity of the pseudo-user and access all of the objects it has rights to.

A database administrator can create different roles for different job functions and responsibilities, grant specific privileges on database objects to these roles, and then grant these roles to users.

In the example on this slide, the GRANT Role_X to New_User places a row in the DBC.RoleGrants table, not the DBC.AccessRights table.

Note: When an access right is granted to a role, a row is placed in the DBC.AccessRights table. The DBC.AllRights system view only shows access rights associated with users, not roles. The DBC.UserRoleRights system view shows access right rows associated with roles.

Implementing Roles

What access rights are used to create new roles?

- CREATE ROLE – needed to create new roles
- DROP ROLE – needed to drop roles

Who is allowed to create and modify roles?

- Initially, only DBC has the CREATE ROLE and DROP ROLE access rights.
- As DBC, give the “role” access rights to the database administrators (e.g., Sysdba).

GRANT CREATE ROLE, DROP ROLE TO Sysdba WITH GRANT OPTION;

How are access rights associated with a role?

- First, create a role.

CREATE ROLE HR_Role;

A newly created role does not have any associated rights until grants are made to it.

- Use the GRANT (or REVOKE) command to assign (or take away) access rights to (or from) the role.

GRANT SELECT, EXECUTE ON HR_VM TO HR_Role;

Roles define access privileges on database objects. When you assign a default role to a user, you give the user access to all the objects that the role has been granted privileges to. A default role that has a role as a member gives the user additional access to all the objects that the nested role has privileges to. The syntax to create a new role is simply: **CREATE ROLE *role_name*;**

A newly created role does not have any associated privileges until grants are made to it. To manage user access privileges, you can:

- Create different roles for different job functions and responsibilities.
- Grant specific privileges on database objects to the roles.
- Assign default roles to users.
- Add members to the role.
- Members of a role can be users or other roles.
- Roles can only be nested one level. Thus, a role that has a role member cannot also be a member of another role.

The CREATE ROLE and DROP ROLE access rights are system rights. These rights are not on a specific database object. Note that the ROLE privileges can only be granted to a user and not to a role or database.

This example identifies the CREATE ROLE and DROP ROLE rights for Sysdba. You can also simply use the key word ROLE to represent both CREATE ROLE and DROP ROLE privileges.

GRANT ROLE TO SYSDBA WITH GRANT OPTION;

Current or Active Roles

How are users associated with a role?

- The role needs to be granted to the user.

```
GRANT HR_Role TO user1;
```

The current or active role can be set to a specific role or to the key word ALL.

- A specific role can be established as the [current role](#).
 - A user has the access rights of the current role plus any nested roles.

- SET ROLE ALL** allows a user to have all valid roles (for that user) to be active.
 - Most users can have their DEFAULT ROLE = ALL; this is the simplest implementation so the users will have automatic access to all of their granted roles.

- At logon, the current role is determined by the DEFAULT ROLE value for the user.

```
CREATE/MODIFY USER user1 AS ... , DEFAULT ROLE = HR_Role;
```

or

```
CREATE/MODIFY USER user1 AS ... , DEFAULT ROLE = ALL;
```

- A user may change roles by executing the following command within their session.

```
SET ROLE role_name;
```

or

```
SET ROLE ALL;
```

- ANSI Note: A session that has only one current role complies with the ANSI standard.

If a specific role_name is specified, that only that one role is the session's current or active role. The effective roles for that user are the session's current role plus any nested roles. At logon time, the current role will be the user's default role.

The SET ROLE ALL option is also available and this allows a user to have all valid roles (granted for that user) to be active or available.

Create User or Modify User

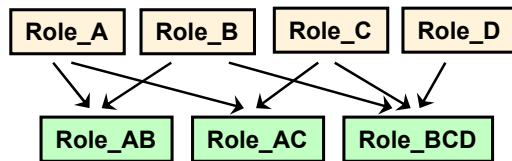
The user executing the CREATE USER command with the DEFAULT ROLE option must have ADMIN privileges on a specified role. The default role is automatically granted to the newly created user.

The user executing the MODIFY USER command with the DEFAULT ROLE option must also have ADMIN privileges on a specified role. The new default role must have first been directly granted to the user before modifying the DEFAULT ROLE with the MODIFY USER command.

Nesting of Roles

You can GRANT a role to another role – referred to as “[nesting of roles](#)”.

Allowed: 1 level of nesting

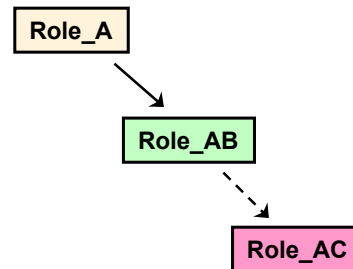


```

GRANT Role_A TO Role_AB;
GRANT Role_B TO Role_AB;
GRANT Role_A TO Role_AC;
GRANT Role_C TO Role_AC;
GRANT Role_B TO Role_BCD;
GRANT Role_C TO Role_BCD;
GRANT Role_D TO Role_BCD;
  
```

A user that is granted access to Role_AB also has all of the access rights associated with Role_A, Role_B, and Role_AB.

Not Allowed: 2nd level of nesting



```

GRANT Role_A TO Role_AB; /*accepted*/
GRANT Role_AB TO Role_AC; /*fails*/
  
```

Roles define access privileges on database objects. When you assign a default role to a user, you give the user access to all the objects that the role has been granted privileges to. It is possible to grant a role to another role. This is referred to as “nesting”. Teradata supports one level of nesting.

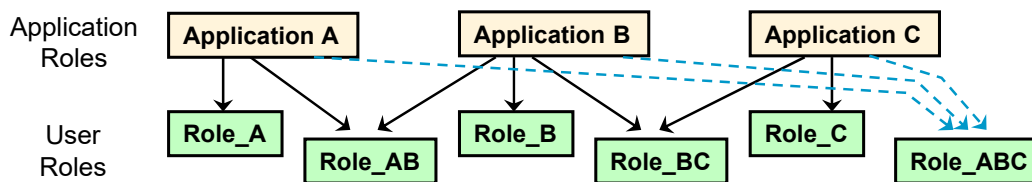
If a role has another role as a member (a role has been granted to a role) and the role is the active role for a user, then a user gets additional access to all the objects that the nested role has privileges to.

For example:

- Assume Role_A and Role_B is granted to Role_AB and assume that Role_AB is the current role of a user. The user then has the following access rights:
 - Access rights directly assigned to the user
 - Access rights assigned to Role_A
 - Access rights assigned to Role_B
 - Access rights assigned to Role_AB

Example of Using "Nested Roles"

Access rights required by an Application are assigned to "application" roles.



Users are assigned to a role at this level based on job requirements.

Characteristics include:

- Users are only assigned to one "role" – ANSI standard.
- Provides a logical separation between application access rights and user access rights.
 - Access rights for an application are only assigned to a single "application" role.
 - For example, if a user needs to use Applications A, B, and C, then the user is granted access to Role_ABC.

This slide contains an example of using nested roles.

Performance Note: If numerous roles are nested within the current role, there may have noticeable performance impact on "short requests or tactical queries". However, a few more access right checks won't be noticed queries that execute for a minute or longer.

The following indexes are placed on the DBC.AccessRights, DBC.RoleGrants, and DBC.Roles tables and are used by Teradata software.

DBC.AccessRights

PI – (NUPI) – (UserId), partitioned by databaseid and tvmid (sub-partition)

DBC.RoleGrants

PI – (NUPI) – (GranteeId)

SI – (USI) – (GranteeId, RoleId)

SI – (NUSI) – (RoleId)

DBC.Roles

PI – (UPI) – (RoleNameI)

SI – (USI) – (RoleId)

Access Rights Validation and Roles

Validation of access rights for accessing a given database object will be carried out in the following steps.

Order of access right validation is:

1. If the user has a current role, check the DBC.AccessRights table for the required right [at the role level](#).
2. [Retrieve all roles nested within the current role from the DBC.RoleGrants table](#). For each nested role, check the DBC.AccessRights table for the required right.
3. Check the DBC.AccessRights table for the required right [at the individual level](#).
4. Check if required right is a [PUBLIC right](#).

Prior to Teradata 15.00, the access rights checking scheme first checked access rights granted to individual user ids. The user direct rights were always checked first, and then the user role rights (plus the public rights) were checked later. This was less efficient as more rights are managed through roles.

Therefore, the default (starting in Teradata 15.00) in checking access rights was changed to:

- 1) If the user has a current role, search the AccessRights table for RoleId-ObjectId pair entry for the required right.
- 2) If not found, retrieve all roles (e.g., ALL) or nested roles associated within the current role from the RoleGrants table. For each role or nested role, search the AccessRights table for RoleId-ObjectId pair entry for the required right.
- 3) If the access right is not yet found, search the AccessRights table for a UserId-ObjectId pair entry for the required right. In this step, the system will check for rights at the database/user level and at the object (e.g., table, view) level.
- 4) If not yet found, check if the right is a Public right.

DBSControl internal field (#330) can be used to disable this feature and have userid access rights checked first (#3 in above list). However, it is recommended that it stays enabled.

330. DisableChkCachedARsFirst – This field enables or disables the enhancement on the Access Rights Checking order when the logon user activates roles.

FALSE - Enabled (default)

TRUE - Disabled

SQL Statements to Support Roles

Command Syntax:

```
CREATE ROLE role_name;
GRANT access_rights TO role_name;
GRANT role_name TO user_name [WITH ADMIN OPTION];
    – ADMIN OPTION allows grantee the right to grant or drop the role.
SET ROLE role_name / NONE / NULL / ALL;
    – Assigns/changes current role for session.
    – Role must be granted to user before statement is valid.
    – SET ROLE ALL; All valid roles for user are available to user.
CREATE/MODIFY USER user1 AS DEFAULT ROLE = NULL or ALL or role_name;
    – When the user logs on, the default role will become the session's initial current role.
```

Other commands:

```
REVOKE ... role_name ... ;
DROP ROLE role_name ;
SELECT ROLE ;
```

Some miscellaneous rules concerning roles include:

- Roles may only be granted to users and other roles.
- There is no limit on the number of roles that can be granted to a grantee.
- The default role for a user will automatically be made the current role for the session when he first logs on. The default role can be established with the CREATE USER or MODIFY USER commands.
- A role grantor can only be a user, but a role grantee can be a user or another role. A role may share the same name as a profile, table, column, view, macro, trigger, or stored procedure. However, a role name must be unique amongst users, databases and roles.
- The role creator is automatically granted membership to the newly created role WITH ADMIN OPTION, which makes the role creator a member of the role who can grant membership to the role to other users and roles.

Dropping a Role

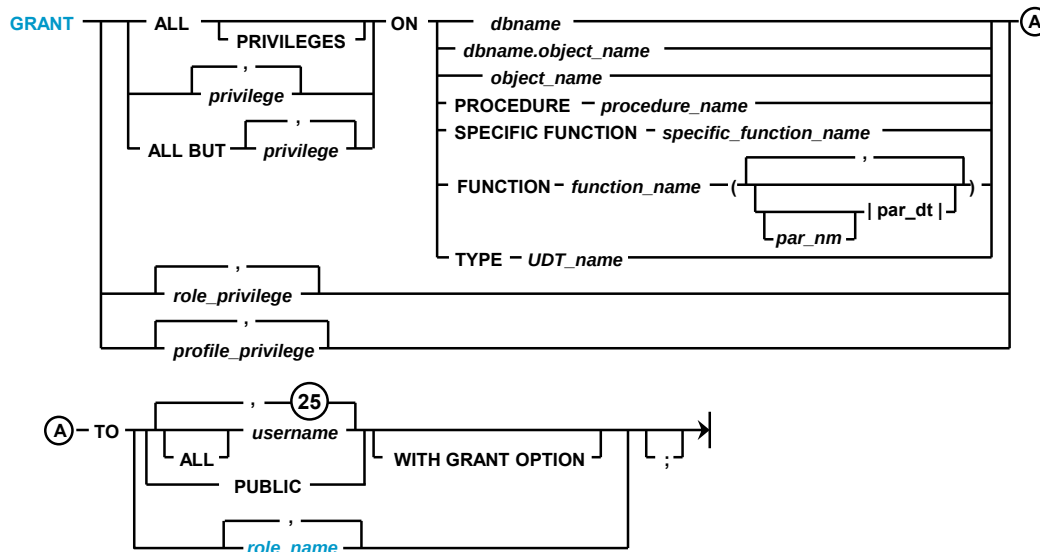
The following users can drop a role:

1. DBC
2. Any user given the system right DROP ROLE
3. Any user granted the role WITH ADMIN OPTION
4. A user whose current role has the specified role as a nested role, and the nested role was granted to the current role WITH ADMIN OPTION

The creator does not have the implicit right to drop a role. If WITH ADMIN OPTION and DROP ROLE rights are revoked from him/her, he/she will not be able to drop the role.

GRANT Command (SQL Form)

The GRANT command may be used to grant access rights to roles.



Roles may not be granted on the following access rights (or functions).

- CREATE ROLE and DROP ROLE
- CREATE PROFILE and DROP PROFILE
- CREATE USER and DROP USER
- CREATE DATABASE and DROP DATABASE
- CTCONTROL – Grants the privilege to connect as a proxy permanent or proxy application user through the specified trusted user, storing the information in *DBC.ConnectRulesTbl*. Authorizes a user to grant or revoke the CONNECT THROUGH privilege using the GRANT CONNECT or REVOKE CONNECT statements. You can only grant CTCONTROL to specific users
- REPLCONTROL (controls two separate functions)
 - The privilege to define and manage replication groups.
 - The ability to run SQL statements that change columnar data values for a table when that table is in a state that would not otherwise allow changes to be made.

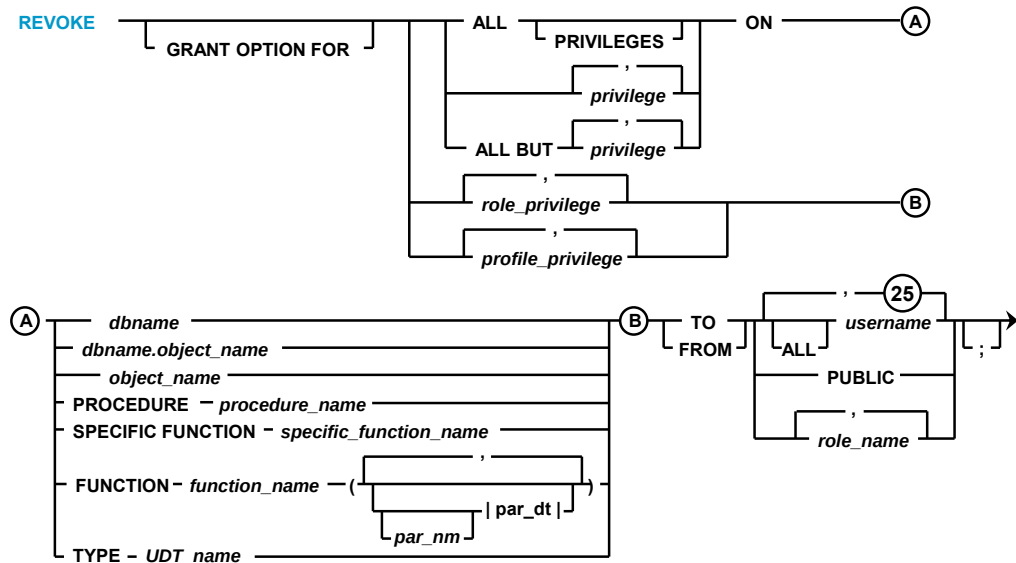
Exceptions

A role cannot have descendants, i.e., the ALL option of a GRANT/REVOKE statement cannot be applied to a role. ANSI also disallows a right to be granted to a role with the GRANT option. **The following statements both fail and are not allowed:**

```
GRANT <right> ON <database object> TO ALL <role name>;
GRANT <right> ON <db object> TO <role name> WITH GRANT OPTION;
```

REVOKE Command (SQL Form)

The REVOKE command may be used to revoke access rights from roles.



This slide shows the syntax for the REVOKE Command.

GRANT and REVOKE Commands (Role Form)

The syntax to grant a role to a user (or role) is:

```
GRANT [role_name] TO [user_name, role_name] WITH ADMIN OPTION;
```

WITH ADMIN OPTION

- Gives the role grantee(s) the right to use DROP ROLE, GRANT, and REVOKE statements to administer the roles to which they are becoming members.

The syntax to revoke a role from a user (or role) is:

```
REVOKE [ADMIN OPTION FOR] [role_name] FROM [user_name, role_name];
```

ADMIN OPTION FOR

- The role members maintain membership status, but lose the right to administer the roles to which they are members.
- If this option is not used, the system removes the specified roles or users as role members.

GRANT (Role Format) is used to grant role membership to users or other roles.

role_name

This is a list of one or more comma-separated names of roles to which membership or administrative ability is being granted

TO *user_name* or *role_name*

This is a list of names of role grantees. Grantees can be users or roles; however, a role cannot be granted membership to itself.

WITH ADMIN OPTION

The role grantees have the right to use DROP ROLE, GRANT, and REVOKE statements to administer the roles to which they are becoming members.

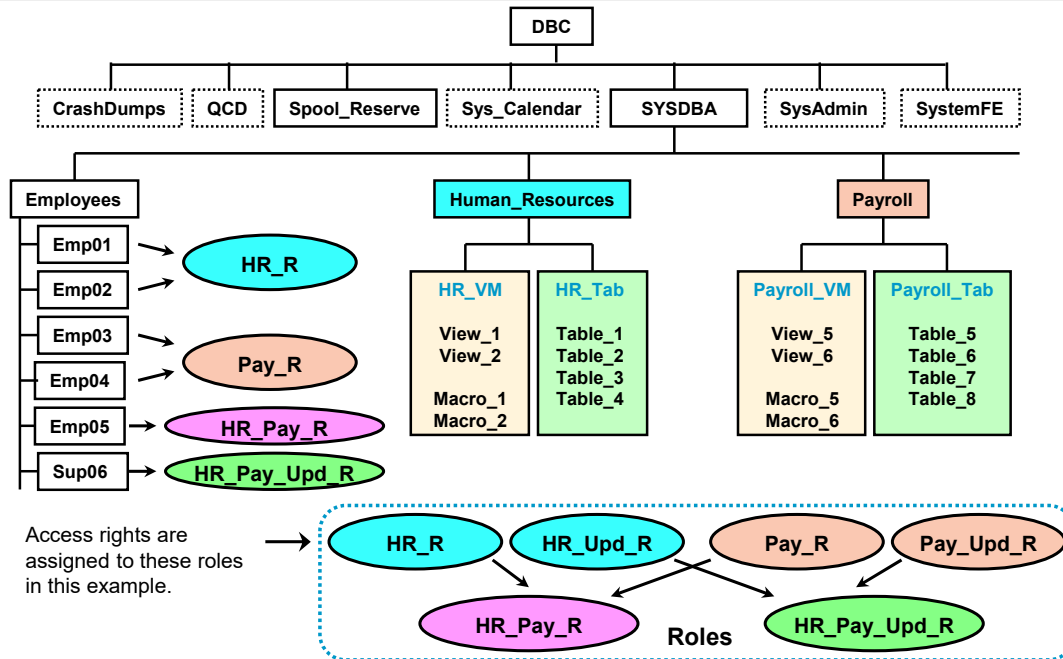
A GRANT statement that does not include WITH ADMIN OPTION does not revoke a previously granted WITH ADMIN OPTION privilege from grantee.

REVOKE (Role Format) is used to revoke role membership to users or other roles.

ADMIN OPTION FOR

- The role members maintain membership status, but lose the right to use GRANT, REVOKE, and DROP ROLE statements to administer the roles to which they are members.
- If ADMIN OPTION FOR does not appear in the REVOKE statement, the system removes the specified roles or users as role members.

System Hierarchy (Used in Following Examples)



This slide shows a Teradata system hierarchy structure which will be used in numerous examples.

- Roles will have a _R at the end of the role_name. For example, HR_R represents the Human Resources Role.
- Inquiry Users – users that require SELECT and EXECUTE access rights on the views and macros in the VM databases. These users will be assigned either to the HR_R, Pay_R, or the HR_PAY_R.
- Update Users – users that require SELECT, EXECUTE, INSERT, UPDATE, and DELETE access rights on the views and macros in the VM databases. These users will be assigned either to the HR_Upd_R, Pay_Upd_R, or the HR_PAY_Upd_R.

The database HR_VM will have SELECT, EXECUTE, INSERT, UPDATE, and DELETE privileges WITH GRANT OPTION on the database named HR_Tab. Likewise for Payroll_VM and Payroll_Tab.

Dropping a User

When a DROP USER command is issued, both individual rights and role rights granted to the user being dropped will be deleted from the DBC.AccessRights and the DBC.RoleGrants tables. Deletions of database objects within the user space prior to the DROP USER command will cause corresponding deletions of DBC.AccessRights rows for rights granted on these objects to roles and other users/databases. However, rights granted by the dropped user that are not on objects within its space will remain in the system. This would include role rights. Roles and profiles created by the dropped user will remain in the system.

Example of Using Roles

Create roles.

```
CREATE ROLE HR_R;           CREATE ROLE HR_Upd_R;
CREATE ROLE Pay_R;         CREATE ROLE Pay_Upd_R;
CREATE ROLE HR_Pay_R;      CREATE ROLE HR_Pay_Upd_R;
```

Assign access rights to the roles (partial listing).

```
GRANT SELECT, EXECUTE                ON HR_VM TO HR_R;
GRANT SELECT, EXECUTE, INSERT, UPDATE, DELETE ON HR_VM TO HR_Upd_R;
```

Grant users permission to use the roles (partial listing).

```
GRANT HR_R          TO Emp01, Emp02;
GRANT Pay_R         TO Emp03, Emp04;
GRANT HR_R          TO HR_Pay_R;      /*nested role*/
GRANT Pay_R         TO HR_Pay_R;      /*nested role*/
GRANT HR_Pay_R      TO Emp05;
GRANT HR_Pay_Upd_R  TO Sup06 WITH ADMIN OPTION;
```

Modify the user to set the default role or set it to ALL.

```
MODIFY USER Emp01 AS DEFAULT ROLE = ALL;
MODIFY USER Emp02 AS DEFAULT ROLE = ALL;
MODIFY USER Emp03 AS DEFAULT ROLE = ALL;
MODIFY USER Emp04 AS DEFAULT ROLE = ALL;
MODIFY USER Emp05 AS DEFAULT ROLE = ALL;
MODIFY USER Sup06 AS DEFAULT ROLE = ALL;
```

This slide contains a simple example of creating a role, assigning access rights to it, and granting the role to users.

The default role for a user will automatically be made the current role for the session when the user first logs on. **The role must be currently granted to the user (otherwise, it is ignored).**

Only a partial listing of the access rights that would be assigned to roles is shown on this slide. Additionally, these commands would also be executed to complete the example.

- `GRANT SELECT, EXECUTE ON Payroll_VM TO Pay_R;`
- `GRANT SELECT, EXECUTE, INSERT, UPDATE, DELETE ON Payroll_VM TO Pay_Upd_R;`
- `GRANT HR_Upd_R TO HR_Pay_Upd_R; /* nested role */`
- `GRANT Pay_Upd_R TO HR_Pay_Upd_R; /* nested role */`

Example of Using Roles (cont.)

Emp01 – is granted to HR_R role and the current role is set to ALL.

```
SELECT *
FROM   HR_VM.Employee_v
WHERE  EmpNumber = 100001;    (success)

UPDATE HR_VM.Employee_v
SET    DeptNumber=1001
WHERE  EmpNumber=100001;    (success or fail?)
```

Does this statement succeed or fail for Emp01?

Emp05 – is granted to HR_Pay_R role and the current role is set to ALL.

```
SELECT *
FROM   HR_VM.Employee_v
ORDER BY 1;    (success or fail?)

SELECT *
FROM   Payroll_VM.Salary_v
ORDER BY 1;    (success or fail?)
```

Do both of these statements succeed for Emp05?

This slide continues the example.

Answer to first question on this slide:

Emp01 does not have UPDATE permission to update the Employee table via the Employee_v view.
The error returned is:

5315: The user does not have UPDATE access to HR_VM.Employee_v.Dept_Number.

Answer to second question on this slide:

Both SQL statements work for Emp05 because the access rights for HR_R and Pay_R are nested within HR_Pay_R.

Example of Using Roles (cont.)

Sup06 – is granted to HR_Pay_Upd_R role WITH ADMIN OPTION.

```
GRANT HR_Pay_Upd_R TO Emp05; (success)
```

Emp05 – has not changed their current role – it is still set to ALL in the session.

```
SELECT *  
FROM   HR_VM.Employee_v  
WHERE  EmpNumber=100001; (success)  
UPDATE HR_VM.Employee_v  
SET    DeptNumber=1001  
WHERE  EmpNumber=100001; (success or fail?)
```

Does this statement succeed for Emp05?

If Emp05 current role is set to HR_PAY_R, will the previous UPDATE command work?

Emp05 – executes the following GRANT command.

```
GRANT HR_Pay_Upd_R TO Emp01;
```

Will this GRANT statement succeed?

Why or why not?

This slide continues the example.

Answer to first question: Yes - the statement succeeds because Emp05's Default role is ALL and Emp05 has been granted to the update role.

Answer to second question: No, because the HR_PAY_R does not have the UPDATE privilege.

Answer to third question: No.

Answer to third question of why: The statement fails succeeds because Emp05 does not have the WITH ADMIN OPTION on the update role.

Additional Note: If a user tries to use the SET ROLE command to specify a role they have not been granted access, the user will get the following error:

5621: User has not been granted a specified role.

Roles for Directory-Based Users

What is the difference between an internal role and an external role?

- External roles are mapped to users on the directory server (e.g., LDAP).
- Internal roles are granted to users with the GRANT command.

How do roles work for directory users?

- An assigned external role overrides the default role of the permanent user on Teradata.
- To associate rights with an external role, first create the role.

```
CREATE EXTERNAL ROLE HR_Ext_Role;
```

- Use the GRANT (or REVOKE) command to assign (or take away) access rights to (or from) the role, as with internal roles.

```
GRANT SELECT, EXECUTE ON HR_VM TO HR_Ext_Role;
```

- A user may change roles to any external or internal role available to them by executing the SQL command:

```
SET ROLE role_name;
```

- External roles are identified in the DBC.Roles table. However, when you map an external role to a directory user, the system does not insert a row in DBC.RoleGrants.

There are a couple of options in providing access rights to directory-based users.

- Map each directory user to one or more database users that already have database privileges.
- You can optionally create external roles and grant privileges to them. Then map each directory user to one or more of the external roles.

The use of roles by directory users depends on the setting of the AuthorizationSupported property:

- When the AuthorizationSupported property is set to no, directory users can log on using a username that matches a database username. They have access to roles in which the matching database username is a member.
- When the AuthorizationSupported property is set to yes, directory users are authorized privileges according to the roles (and users) they are mapped to in the directory.

Implementing Roles for Directory Authorization of Database Privileges

1. Create external roles.
2. Review directory user management options and select a user provisioning strategy.
3. Create one or more directory role objects with names that match Teradata Database external roles and map the roles to directory group objects.

Since roles are assigned by mapping instead of role grants, assignments cannot include WITH ADMIN OPTION.

The system records external roles in the data dictionary, along with database roles, but when you map an external role to a directory user, the system does not insert a row in the DBC.RoleGrants table.

Roles for Proxy Users

Proxy users use standard internal roles, if these are specified in the WITH ROLE clause of the GRANT CONNECT THROUGH statement or associated with the permanent proxy user.

- A proxy user is an end user that logs on to the database through a trusted user application. The system identifies and authorizes the user an individual.
- Proxy users can be either permanent database users or other end users unknown to the database.

How are roles associated with a proxy user?

- Use the GRANT CONNECT THROUGH statement to grant the application the privilege to connect as the application users:

```
GRANT CONNECT THROUGH Application1_User TO App_End_User1    [WITH ROLE HR_Role];
```

- Each time the application performs a request for an application user, it must issue the SET QUERY_BAND statement before its SQL queries:

```
SET QUERY_BAND = 'PROXYUSER=App_End_User1;[PROXYROLE=HR_Role;]' FOR SESSION;
SELECT FirstName, LastName FROM HR_VM.Employee;
. . .
```

```
SET QUERY_BAND = NONE FOR SESSION;
```

For proxy users that are either permanent database users or users unknown to the database, you can specify one or more roles in the GRANT CONNECT THROUGH statement that defines the proxy. For proxy users that are also permanent database users:

- You can specify WITHOUT ROLE to use the privileges granted to the permanent user.
- You can assign row level security constraints to the permanent user or the user profile. Proxy user sessions use the profile constraints, if assigned. If no constraints are assigned in the profile, the session uses the user constraints. The user can also use the SET SESSION CONSTRAINT command to access any assigned security constraints.

CONNECT THROUGH option: With roles

SET QUERY_BAND option	Proxy connection role
PROXYROLE = rolename	Rolename, if it has been specified in GRANT CONNECT
PROXYROLE = ALL	All roles in the GRANT privilege
PROXYROLE = NONE or	Not permitted
PROXYROLE = NULL	
PROXYROLE is not used	All roles in the GRANT privilege

CONNECT THROUGH option: Without roles

PROXYROLE = rolename	Rolename, if it is granted to the permanent user
PROXYROLE = ALL	All permanent user's roles
PROXYROLE = NONE or	NULL
PROXYROLE = NULL	
PROXYROLE is not used	Permanent user's default role

RoleInfo View

Provides information about roles.

`DBC.RoleInfo[V][X]`

RoleName	CommentString	CreatorName
CreateTimeStamp	ExtRole	

Example: List role names that exist in the system.

```
SELECT  RoleName, CreatorName, CreateTimeStamp
FROM    DBC.RoleInfoV
ORDER BY 1;
```

Results:

<u>RoleName</u>	<u>CreatorName</u>	<u>CreateTimeStamp</u>
HR_Pay_R	Sysdba	2019-03-24 17:25:41
HR_Pay_Upd_R	Sysdba	2019-03-24 17:25:44
HR_R	Sysdba	2019-03-24 17:25:02
HR_Upd_R	Sysdba	2019-03-24 17:25:19
Pay_R	Sysdba	2019-03-24 17:25:34
Pay_Upd_R	Sysdba	2019-03-24 17:25:37

The DBC.RoleInfo (or DBC.RoleInfoV) views list all of roles, their creators, and the creation timestamp in the system. This information is taken from the DBC.Roles and the DBC.Dbase tables.

The DBC.RoleInfoX (or DBC.RoleInfoVX) views only return roles that a user has created. Users that can create roles need the system access right – CREATE ROLE.

Extension to COMMENT command:

```
COMMENT [ON] ROLE <role_name> [ [AS] <comment string> ]
```

Inserts or retrieves comments in CommentString column of the DBC.Roles table for the named role.

Example:

```
COMMENT ON ROLE HR_R AS 'SEL and EXE rights for HR_VM';
```

RoleMembers View

Provides information about roles and its members.

DBC.RoleMembers[V][X]

RoleName	Grantee	GranteeKind	Grantor
WhenGranted	DefaultRole	WithAdmin	

Example: List roles and the members that have access to the HR database.

```
SELECT  RoleName, Grantee, GranteeKind, DefaultRole, WithAdmin
FROM    DBC.RoleMembersV
WHERE   RoleName IN ('HR_Pay_R', 'HR_Pay_Upd_R', 'HR_R', 'HR_Upd_R')
ORDER BY 1, 2;
```

Partial
Results:

<u>RoleName</u>	<u>Grantee</u>	<u>GranteeKind</u>	<u>DefaultRole</u>	<u>WithAdmin</u>
HR_Pay_R	Emp05	User	N	N
HR_Pay_Upd_R	Emp05	User	N	N
HR_Pay_Upd_R	Sup06	User	N	Y
HR_R	Emp01	User	N	N
HR_R	Emp02	User	N	N
HR_R	HR_Pay_R	Role	N	N
HR_Upd_R	HR_Pay_Upd_R	Role	N	N

Note that Emp05 has been granted to two separate roles.

Note that the Default Role = N for all of the Employees and the Supervisor since the DEFAULT ROLE = ALL.

The DBC.RoleMembers (or DBC.RoleMembersV) views list each role and all of its members.

The DBC.RoleMembersX (or DBC.RoleMembersVX) views list all roles, if any, directly granted to the user.

For example, Emp05 executes the following statement:

```
SELECT  RoleName, Grantor, WhenGranted, DefaultRole, WithAdmin
FROM    DBC.RoleMembersX
ORDER BY 1;
```

The result is:

<u>RoleName</u>	<u>Grantor</u>	<u>WhenGranted</u>	<u>DefaultRole</u>	<u>WithAdmin</u>
HR_Pay_R	Sysdba	2017-01-24 17:32:51	Y	N

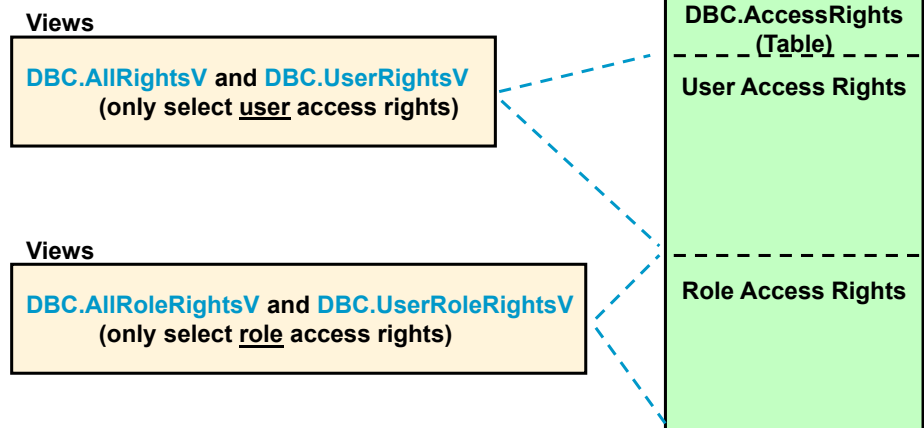
DBC.AccessRights and “Rights” Views

`AllRights[V][X]` – lists all rights granted to users in the system.

`UserRights[V]` – lists all rights granted to the current user.

`AllRoleRights[V]` – lists all rights granted to roles in the system.

`UserRoleRights[V]` – lists all rights granted to the enabled roles of the user.



This slide illustrates the difference between the various “Rights” views of the `DBC.AccessRights` table.

AllRoleRights and UserRoleRights Views

[AllRoleRights\[V\]](#) – lists all rights granted to roles in the system.

[UserRoleRights\[V\]](#) – lists all rights granted to the enabled roles of the user.

[DBC.AllRoleRights](#) and
[DBC.UserRoleRights](#)

RoleName	DatabaseName	TableName	ColumnName
AccessRight	GrantorName	CreateTimeStamp	

Example: List current role rights.

```
SELECT RoleName, DatabaseName, TableName, ColumnName, AccessRight
FROM   DBC.UserRoleRightsV
ORDER BY 1;
```

Results for Emp01:

<u>RoleName</u>	<u>DatabaseName</u>	<u>TableName</u>	<u>ColumnName</u>	<u>AccessRight</u>
HR_R	HR_VM	All	All	R
HR_R	HR_VM	All	All	E

Results for Emp05:

Emp05 has been granted to 2 roles
each of which has 2 nested roles.

<u>RoleName</u>	<u>DatabaseName</u>	<u>TableName</u>	<u>ColumnName</u>	<u>AccessRight</u>
HR_R	HR_VM	All	All	R
HR_R	HR_VM	All	All	E
HR_Upd_R	HR_VM	All	All	R
HR_Upd_R	HR_VM	All	All	E
HR_Upd_R	HR_VM	All	All	I
:	:	:	:	:

The DBC.AllRoleRights[V] and DBC.UserRoleRights[V] views provide information about role and access rights granted to roles in the system.

DBC.UserRoleRights[V] view lists all rights granted to the current role of the user and its nested roles.

Steps to Implementing Roles

1. Identify individual rights to be converted into role rights.
2. Create roles and grant appropriate rights to each role.
3. Grant roles to users and assign users their default roles.
4. Revoke from users individual rights that have been replaced by role rights.
5. Test that users have access via the roles.

Sample query to identify individual rights that may be good candidates for conversion to roles.

```
SELECT DatabaseName,
       TVMName,
       COUNT(*)      AS RightsCount
FROM   DBC.AccessRights AR,
       DBC.TVM         TVM,
       DBC.DBase       DBASE
WHERE  AR.tvmid = TVM.tvmid
AND    AR.databaseid = DBASE.databaseid
AND    AR.fieldid = 0
GROUP BY DatabaseName, TVMName
ORDER BY 3 DESC;
```

Results:

<u>DatabaseName</u>	<u>TVMName</u>	<u>RightsCount</u>
Student_QCD	AnalysisLog	592
Student_QCD	DataDemographics	592
PUBLIC	All	491
Student_QCD	TableStatistics	447
:	:	:

This slide identifies a sequence of steps to consider when implementing roles. A sample query and results are also provided.

Summary

- A role is simply a collection of access rights.
- Rights are first granted to a role and the role is then granted to users.
 - `CREATE ROLE role_name;`
 - `GRANT access_right ON object TO role_name;`
 - `GRANT role_name TO user_name;`

A summary of the rules for using roles are as follows:

- You can grant one or more roles to one or more users and/or roles; thus:
 - A role can have many members
 - A user or role can be a member of more than one role
- Only single-level nesting is allowed; that is, a role that has a member role cannot also be a member of another role.
- An access privilege granted to an existing role immediately affects any user and/or role that is specified as a recipient in the GRANT statement and currently active within a session.
- The privileges of a role granted to another role are inherited by every user member of the grantee role.
- When a user logs on, the assigned default role is the initial current role for the session and is used to authorize access after all checks against individually granted rights have failed.
- Once the session is active, the user can submit a SET ROLE statement to change or nullify the current role.

DROP ROLE considerations: Default role settings for all users with the dropped role as their default role do *not* reset to NULL. Affected users receive no warnings or errors the next time they log on. The system does not use the obsolete default role for privileges validation.

If a dropped default role is subsequently recreated, it reassumes its status as the default role, but it has a different role ID number than it had before being dropped.



Module 9: Review Questions

1. What are two advantages of utilizing Roles? (choose two)
 - a. Reduces the number of rows in the access rights table.
 - b. Eliminates the need for the access right checks in the PE.
 - c. Helps reduce the number of required profiles in a system.
 - d. Simplifies assigning access rights for an application to a user.
2. If a user executes this command (SET ROLE Applx_R;), which two statements are true? (choose two)
 - a. Access rights granted to PUBLIC are still available to the user.
 - b. Access rights granted to the user are not available to the user.
 - c. Access rights assigned to Applx_R and any nested roles are available to the user.
 - d. Access rights granted to any role granted to the user are still available to the user.
3. How many levels of role nesting are allowed? ____
4. True or False: Roles may only be granted to users and other roles.

Match each term to the definition.

- | | |
|-----------------------------|---|
| 5. ____ WITH ADMIN OPTION | a. Established by the SET ROLE command |
| 6. ____ CREATE ROLE | b. Lists the roles currently in the system |
| 7. ____ DBC.RoleInfoV | c. System access right needed to create a role |
| 8. ____ DBC.UserRoleRightsV | d. Lists all of a user's role rights – including nested roles |
| 9. ____ DEFAULT ROLE | e. Allows the user to assign other users to the role |
| 10. ____ Current role | f. Option with the MODIFY USER statement |

Check your understanding of the concepts discussed in this module by completing the review questions as directed by your instructor.



Lab Exercise 9-1 (1 of 2)

Purpose

In this lab, you will work with access rights and roles. This lab will also allow you to use various system views.

Tasks

1. Using the DBC.AllRightsV view, find the total number of rows in the DBC.AccessRights table assigned to users/databases.

Total number of user rights (AllRightsV) _____

Using the DBC.AllRoleRightsV view, find the total number of rows in the DBC.AccessRights table assigned to roles.

Total number of role rights (AllRoleRightsV) _____

2. Using the DBC.UserRightsV view, how many access rights do you currently have?

Total number of your user rights (UserRightsV) _____

How do you think most of these access rights were granted? _____

Execute the following SQL command and then recheck the number of Access Rights you have.

```
CREATE TABLE EmpPhone2 AS TRNG_PD.EmpPhone WITH NO DATA;
```

What is the total number of your user rights? _____

How many new access rights were created? _____

Check your understanding of the concepts discussed in this module by completing the lab exercises as directed by your instructor.



Lab Exercise 9-1 (2 of 2)

3. If EmpPhone is not populated, use INSERT/SELECT to populate the EmpPhone table from TRNG_PD.EmpPhone with 350,000 rows.

Use the GRANT command to give the SELECT WITH GRANT OPTION access right on EmpPhone to studentxxx_A.

Use the GRANT command to give the INSERT access right on EmpPhone2 to studentxxx_A.

Check the total number of user rights returned for studentxxx _____

Did this count change? _____

If not, why not? _____

4. Logon as studentxxx_A and issue the following statement.

```
INSERT INTO studentxxx.EmpPhone2 SELECT * FROM studentxxx.EmpPhone;
```

Does this INSERT complete successfully? _____

As studentxxx_A, issue the following command.

```
SELECT COUNT(*) FROM studentxxx.EmpPhone2;
```

Does this SELECT complete successfully? _____

This slide continues this lab exercise.



Lab Exercise 9-2 (1 of 4)

teradata.

Purpose

In this lab, you will utilize roles and access the RoleInfoV, RoleMembersV, and UserRoleRightsV views.

What you need

Tables from previous exercises and a user account with system role and profile privileges.

Tasks

1. Logon as your normal student user (studentXXX). Three roles are available for your use. The role names have your user name incorporated into them. Your role names will be unique in the system. For example, if your user name is "student142", then your role names are:

Role1_142	Role1_xxx (note - this role may be referred to as "Role1" throughout this lab).
Role2_142	Role2_xxx
Role3_142	Role3_xxx

Using the DBC.RoleInfoV view, what is the total number of roles defined in the system? _____

Using the DBC.RoleInfoVX view, what is the number of roles that you have created? _____

Using the DBC.RoleMembersVX view, what is your (studentxxx) default role? _____

Using the DBC.RoleMembersVX view, which roles do you have the "With Admin" option?

Check your understanding of the concepts discussed in this module by completing the lab exercises as directed by your instructor.

DBC.RoleInfo[V][X]

RoleName	CommentString	CreatorName
CreateTimeStamp	ExtRole	



Lab Exercise 9-2 (2 of 4)

2. Grant the following access rights to the specified roles as follows:

<u>Access Rights</u>	<u>Tables</u>	<u>Role Name</u>
SELECT	Orders, Orders_NoPI	Role1_xxx
SELECT	Orders_RP	Role2_xxx
INSERT, UPDATE, DELETE	Orders_RP	Role3_xxx

3. Grant these roles as following:

```
Grant Role1_xxx to studentxxx_A;  
Grant Role2_xxx to studentxxx_B;  
Grant Role2_xxx to Role3_xxx;  
Grant Role3_xxx to studentxxx_B;
```

4. Modify your two users to set their default role as follows:

User name: studentxxx_A (where xxx is your student number)
Default Role: Role1_xxx

User name: studentxxx_B (where xxx is your student number)
Default Role: Role2_xxx

This slide continues this lab exercise.

DBC.ProfileInfo[V][X]

ProfileName	DefaultAccount	DefaultDB
SpoolSpace	TempSpace	ExpirePassword
PasswordMinChar	PasswordMaxChar	PasswordDigits
PasswordSpecChar	PasswordRestrictWords	MaxLogonAttempts
LockedUserExpire	PasswordReuse	CommentString
CreatorName	CreateTimeStamp	LastAlterName
LastAlterTimeStamp		



Lab Exercise 9-2 (3 of 4)

5. Logon as "studentxxx_A" and execute the following SQL statements and indicate if SELECT is allowed or not.

SELECT COUNT(*) FROM Orders;	Permitted or not? _____
SELECT COUNT(*) FROM Orders_NoPI;	Permitted or not? _____
SELECT COUNT(*) FROM Orders_RP;	Permitted or not? _____

6. As "studentxxx_A", use the DBC.RoleMembersVX and DBC.UserRoleRightsV views to view role information about this user.

How many roles are available to studentxxx_A?	_____
What is the default role for studentxxx_A?	_____
Does studentxxx_A have the "With Admin" option on any roles?	_____
How many user role rights are available to studentxxx_A?	_____

OPTIONAL

7. Logon to Teradata as "studentxxx_B". If prompted, set the password to a new value.

This slide continues this lab exercise.



Lab Exercise 9-2 (4 of 4)

OPTIONAL

8. As "studentxxx_B", execute the following SQL statements and indicate if SELECT is allowed or not.

SELECT COUNT(*) FROM Orders;	Permitted or not? _____
SELECT COUNT(*) FROM Orders_NoPI;	Permitted or not? _____
SELECT COUNT(*) FROM Orders_RP;	Permitted or not? _____
DELETE Orders_RP;	Permitted or not? _____

9. As "studentxxx_B", use the DBC.RoleMembersVX and DBC.UserRoleRightsV views to view the current role of the user, any nested roles, and access rights for the roles.

How many roles are available to studentxxx_B? _____
What is the default role for studentxxx_B? _____
Does studentxxx_B have the "With Admin" option on any roles? _____
How many user role rights are available to studentxxx_B? _____

10. As "studentxxx_B", use the SET ROLE command to set the current role to "Role3_xxx".

SELECT COUNT(*) FROM Orders;	Permitted or not? _____
SELECT COUNT(*) FROM Orders_NoPI;	Permitted or not? _____
SELECT COUNT(*) FROM Orders_RP;	Permitted or not? _____
DELETE Orders_RP;	Permitted or not? _____

11. Log off as "studentxxx_A" and "studentxxx_B". Using your initial user logon name, DROP the two users and the profile you created.

This slide continues this lab exercise.

DBC.UserRoleRights[V]

RoleName	DatabaseName	TableName
ColumnName	AccessRight	GrantorName
CreateTimeStamp		

DBC.RoleMembers[V][X]

RoleName	Grantee	GranteeKind
Grantor	WhenGranted	DefaultRole
WithAdmin		



Module 10: System Access Controls

Teradata Vantage Administration

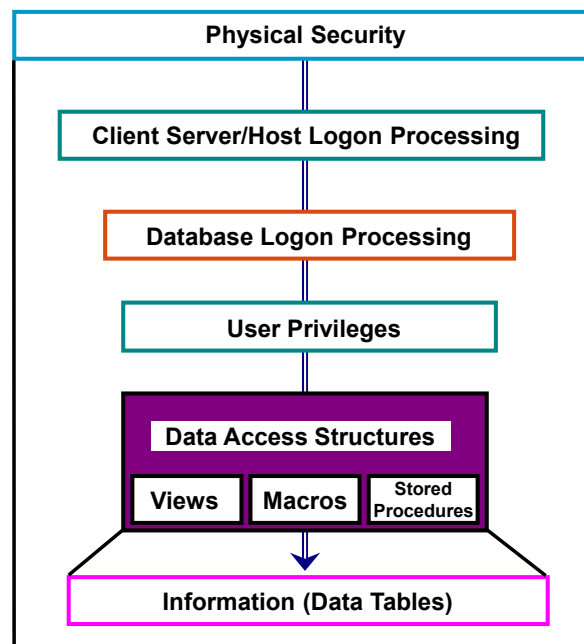
Copyright © 2007–2023 by Teradata. All Rights Reserved.

Objectives

After completing this module, you will be able to:

- Describe where and how to control and log user access to the Teradata database.
- Use views and macros to limit user access to data.
- Design your system hierarchy structures for better security and easier maintenance.

System Access Control Levels



The mission of security administration on a Teradata system is to:

- Prevent unauthorized persons from gaining access to the RDBMS and its resources.
- Permit legitimate users access to only those resources they are authorized to use.

A variety of mechanisms provide security to the data stored on a Teradata system.

Access Control Levels

There are three levels of access controls for the Teradata database:

Physical Security

Physical security pertains to the actual building or computer room in which the Teradata system resides. The system's owner designs and implements physical security.

Host Logon Processing

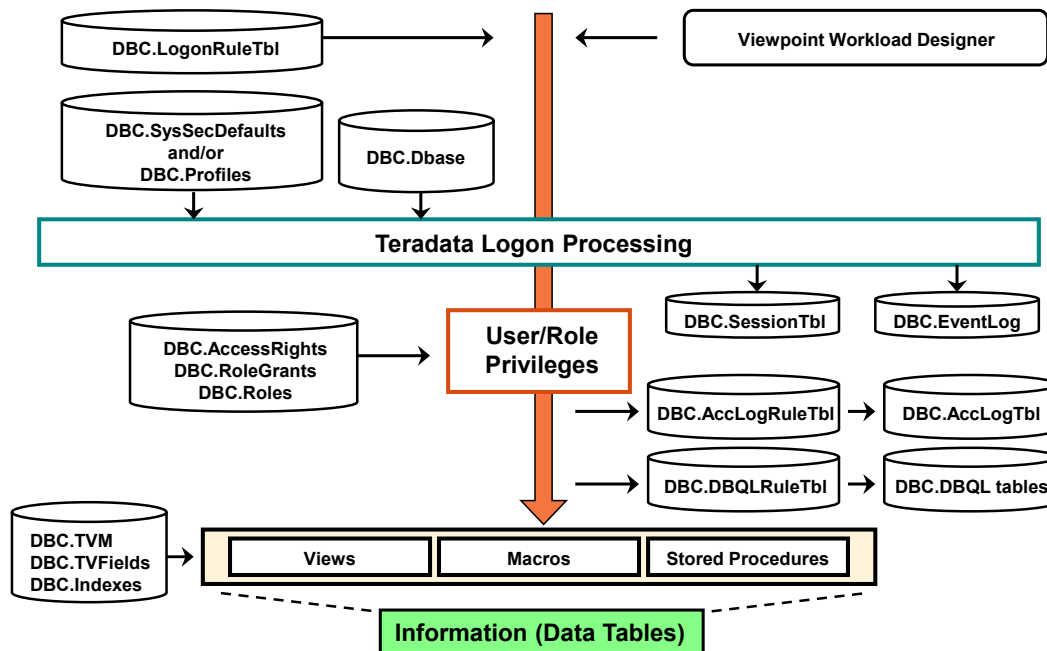
Host logon processing is the first level of access control and allows or disallows connection between the host and database systems. It involves a host ID and password. This level controls access to the host system.

Database Logon Processing

Database logon processing is the second level of access control and determines access to the Teradata system. This level employs a username and password. It controls access to the Teradata system itself.

Data access structures (views, macros and tables) are discussed later in this module.

Teradata Access Control Mechanisms



This slide shows a diagram of access control mechanisms in Teradata.

You can control user access by granting access to specific views and macros. Views limit user access to table columns or rows that may contain sensitive information. Macros limit the types of actions a user can perform on the columns and rows.

User Privileges

During a session, the Teradata Database system accesses the user's default database to search for or store the occurrence of an object whose reference in the SQL statement is not qualified with a database name. At any time during the session, the user can override the current default by executing the SQL DATABASE statement. The system uses the space associated with the specified or default database as the default until the user executes another DATABASE statement or logs off.

An arrangement of predefined privileges or access rights control the user's activities during a session. Access rights are associated with a user, a database, and an object (table, view, or macro).

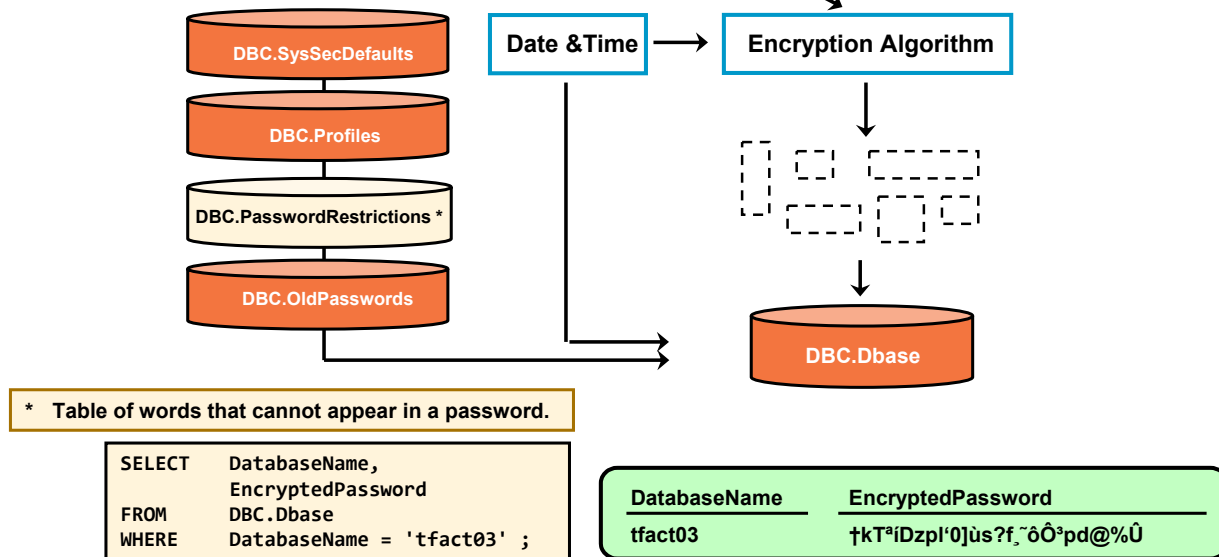
The system verifies a user's access rights when the user attempts to access or execute a function that accesses an object. Teradata stores access rights information in the system table DBC.AccessRights. You can retrieve this information by querying the DBC.UserRights view.

As the administrator, you can use to limit user access to the Teradata database via:

- Views, Macros, and/or Stored Procedures

Teradata Password Encryption

```
CREATE USER tfact03 AS PERM=0, SPOOL=50E9, PASSWORD = secure123 ;
```



You can give access to the Teradata database with the CREATE USER statement, which identifies a username and a password. Although the username is the basis for identification to the system, it is not usually protected information. Often the username is openly displayed during interactive login, on printer listings, and when session information is queried. To protect system access, associate a password with the username. Teradata does not display or print passwords on listings, terminals or PC screens.

Teradata stores password information in encrypted form in the DBC.Dbase system table. Information stored in the table includes the date and time a user defined a password, along with the encrypted password. As the administrator, you may modify passwords temporarily when the PasswordLastModDate plus a fixed number has been reached. This allows you to ensure that users change their passwords regularly.

To establish a session on the Teradata system, a user must enter a username at login. Upon successful login, the username is associated with a unique session number until the user logs off. To supervise and enforce users' access rights to stored data, the system associates each username with a default storage area and an arrangement of access rights.

Displaying Passwords

The PasswordString column from the DBC.Dbase table displays encrypted passwords. Note that a password cannot be decrypted.

Password Security Features

The [DBC.SecurityDefaults\[V\]](#) (view) can be used to view/update [DBC.SysSecDefaults](#) table.

ExpirePassword	Number of days to elapse before the password expires. Zero (0) indicates passwords do not expire; default is 0.
PasswordMinChar	Minimum number of characters in a valid password string; default is 1.
PasswordMaxChar	Max number of characters in a valid password string; default is 127 (if EON is enabled).
PasswordDigits	Indicate if digits are to be allowed in the password (Y, N, or R); default is Y; (R – one or more digits are required in password).
PasswordSpecChar	Indicate if special characters are allowed in the password (Y or N); default is Y; (Options – A to P, R – options provide for more secure passwords).
PasswordRestrictWords	Indicate whether or not a password is subject to the content restrictions (Y or N); default is N.
MaxLogonAttempts	Number of erroneous logons allowed before locking user. Zero (0) indicates that user is never locked; default is 0 - max is 32,767.
LockedUserExpire	Number of minutes to elapse before a locked user is unlocked. Zero (0) indicates immediate unlock; -1 = locked indefinitely; default is 0 - max is 32,767.
PasswordReuse	Number of days to elapse before a password can be reused. Zero (0) indicates immediate reuse; default is 0 - max is 32,767.

To change system-wide password security features:

1. UPDATE this view or table with the desired values
2. Restart Teradata (required)

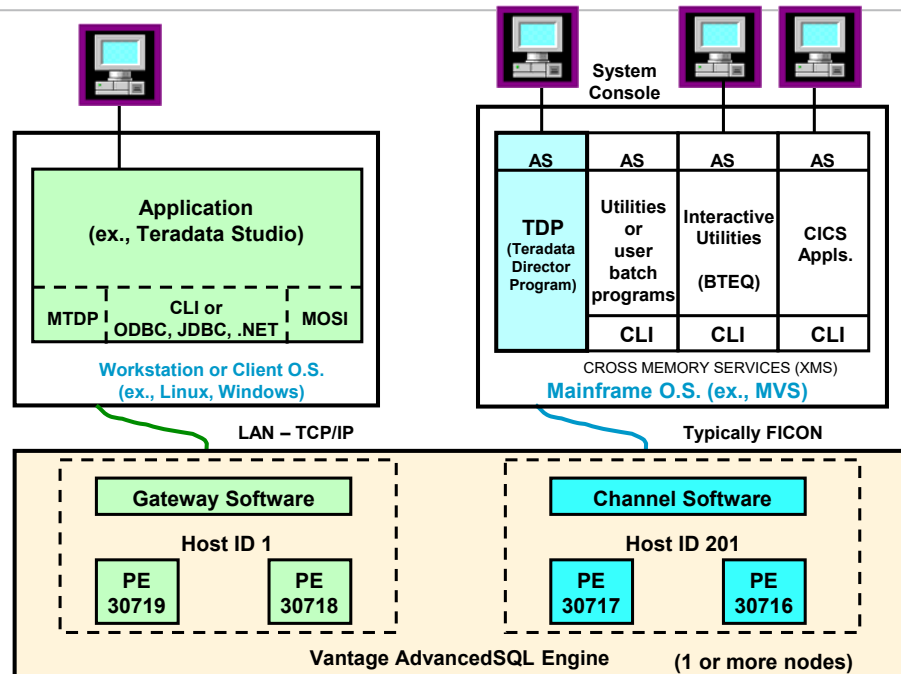
The DBC.SysSecDefaults table is used to establish system-wide or system default password requirements. These same options can be specified at the Profile level, and if specified at the Profile level and a user is assigned to the Profile, the Profile attribute takes precedence.

You can enable password features by updating the appropriate row in the DBC.SysSecDefaults table as shown on this slide. The DBC.SecurityDefaults[V] view can also be used to view/update this table. After modifying this table, it is necessary to restart Teradata for the changes to be in effect.

Notes:

- If you attempt to set the PasswordMinChar attribute equal to 0, Teradata will assume a value of 1.
- If MaxLogonAttempts is set to a value other than zero, and if the time interval for locking users after erroneous attempts is left at zero, then the user is never locked.

Teradata Connectivity



Teradata utilities and software programs support Teradata database access in both mainframe and LAN environments. Utilities and programs run under the client's operating system and provide the functionality for a user to access the database system. When a system is configured, host numbers are assigned to different channel and LAN connections. It is possible to enable/disable user access from specific host numbers.

Channel Environment

The Teradata Channel Interface is an architecture that enables communication between a mainframe client and a Teradata server using a FICON channel interface. A Teradata Channel node can use a PCI-X FICON Adapter (PXFA) to connect to a FICON channel. TDP software executes on the mainframe and communicates with the PE software executing within Teradata.

LAN Environment and Teradata Gateway Software

In a LAN environment, each network client will have the utilities and programs needed to access the Teradata database. These connections provide the workstation operating system access to the gateway software in the node. The gateway software runs on the Teradata server that is running the Teradata database. Client programs that communicate through the gateway to the Teradata Database may be resident on the system, or may be installed and running on network-attached workstations.

When a system is configured, it is possible to assign different hostids to different Ethernet LANs connected to a system. By having multiple hostids (for LANs), a customer can enable/disable a specific LAN. An example might be that you have east/west coast users on different LANs.

Host Logon Processing

The default is that any authorized user can access Teradata through any identified client connection only if they provide a valid password.

Optionally, an administrator can ...

- grant or deny users permission to logon to Teradata from specific client connections.
- give users permission to logon to Teradata from specific host connections using a NULL password.

The following statements are used to control access from specific “host ids”.

- **GRANT LOGON statement**
 - Gives users permission to logon to Teradata from specific client connections and optionally use a pre-validated logon request.
- **REVOKE LOGON statement**
 - Denies users permission to logon to Teradata from client system(s).

The Teradata system default is that any defined user with a valid password who is logged on to a host machine has permission to access the Teradata server through any identified client connection. After installing the software, you may restrict access to the server by associating individual users with specific hosts.

GRANT/REVOKE LOGON Statements

Use the GRANT LOGON statement to give users permission to log on to the Teradata RDBMS from one or more specific client systems. Use the REVOKE LOGON command to retract permission to log on to the Teradata database from one or more specific client systems. These two commands store rows in the DBC.LogonRuleTbl.

You must have EXECUTE privileges on the macro DBC.LogonRule to execute either of these commands.

After installation, use the REVOKE LOGON statement to change the system default by first removing access privileges from all users from all hosts. Then, you can submit the GRANT LOGON statement to assign individual users to specific host IDs.

You can execute the GRANT or REVOKE LOGON statements any time after installation to add or remove user names on individual host connections as needed.

Objects Used in Host Logon Processing

Users who are granted the EXECUTE permission on the following macro can use the GRANT LOGON and REVOKE LOGON statements.

Example:

```
GRANT EXECUTE ON DBC.LogonRule TO SecAdmin;
```

This allows “SecAdmin” to execute the GRANT LOGON and REVOKE LOGON statements.

Execution of GRANT LOGON or REVOKE LOGON statements causes rows (representing the rules) to be added or updated in ...

To view the rules in this table, SELECT from this view.

DD/D Macro

DBC.LogonRule

DD/D Table

DBC.LogonRuleTbl

DD/D View

DBC.LogonRules[V]

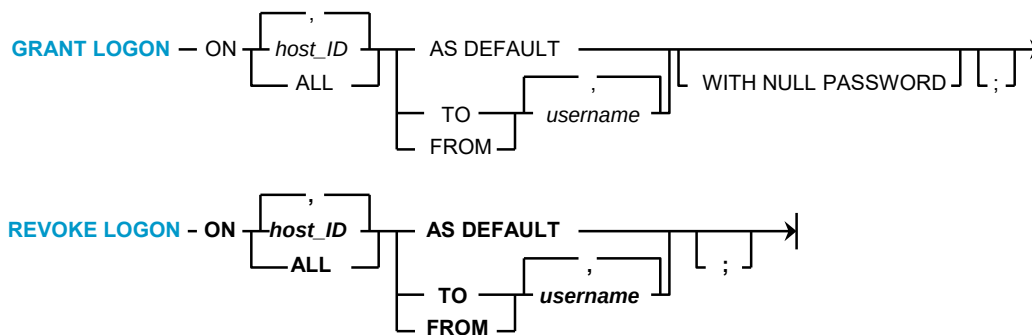
You must have EXECUTE privileges on the macro DBC.LogonRule to execute the GRANT LOGON and REVOKE LOGON statements.

Note: DBC.LogonRule is a “dummy macro”. It only has a semicolon (;) in it.

The GRANT LOGON and REVOKE LOGON statements store rows in the DBC.LogonRuleTbl.

To view the rows in this table use the DBC.LogonRules view.

GRANT/REVOKE LOGON Statements



host_ID	Host number from configuration data. The database console is host number "0" (zero). ALL is represented as "1024".
AS DEFAULT	Changes the default for the specified host.
username	You can specify up to 25, but not "DBC".
WITH NULL	When used in conjunction with a TDP exit or with single sign-on (LDAP),
PASSWORD	overrides the system default that a password is required.

To execute a GRANT or REVOKE LOGON statement, you must hold execute privileges on the DBC.LogonRule macro.

GRANT LOGON and REVOKE LOGON are flagged as non-ANSI statements.

Keywords

HostID

Identifies a mainframe channel connection or a local area network connection that is currently defined to the Teradata RDBMS by the hardware configuration data. The host ID for the Teradata database console is zero (0). For any other connector, the host ID is a value from 1 to 1023.

ALL

The ALL keyword, used in place of a host ID, applies to any source through which a logon is attempted, including the Teradata database console. This is shown as host ID 1024.

AS DEFAULT

Specifies that the current default for the specified host ID(s) is to be changed as defined in this GRANT LOGON statement. A statement with AS DEFAULT has no effect on the access granted to or revoked from particular user names.

TO or FROM username(s)

Overrides the current default for the specified username(s) on the specified host ID(s). The name DBC cannot be specified as a username in a GRANT LOGON statement. A statement that includes this name will return an error message.

WITH NULL PASSWORD

The initial Teradata database default is that all logon requests must include a password. The WITH NULL PASSWORD option, in conjunction with a TDP security exit procedure, permits a logon string that has no password to be accepted on a Teradata system.

GRANT/REVOKE LOGON Examples

GRANT LOGON ON 1 TO Sysdba;

This **GRANT LOGON** creates a specific logon rule for Sysdba.

Example:

For maintenance purposes, Sysdba decides to disable logons for all users except Sysdba from the LAN which has a default hostid of 1.

REVOKE LOGON ON ALL AS DEFAULT;

Note that Sysdba (and DBC) can still logon since individual rules override AS DEFAULT.

Teradata BTEQ 16.20.15.00 for LINUX.

Enter your logon or BTEQ command:

.logon FACTORY/sysdba

Password:

*** Logon successfully completed.

To allow all users to logon to the system via the LAN, execute the following GRANT LOGON statement.

GRANT LOGON ON ALL AS DEFAULT;

REVOKE LOGON ON 1 TO Guest001, Guest002, Guest003;

This **REVOKE LOGON** creates three specific logon rules for three guest users.

Example:

Guest users are only allowed access to the system when granted logon access.

Guest001 attempts to log on while its REVOKE LOGON rule is in effect.

Teradata BTEQ 16.20.15.00 for LINUX.

Enter your logon or BTEQ command:

.logon FACTORY/guest001

Password:

*** Failure 8017 The UserId, Password or Account is invalid.

*** Error: Logon failed!

To allow guest001 to logon to the system via the LAN, execute the following GRANT LOGON statement.

GRANT LOGON ON 1 TO Guest001;

This slide contains an example of using the REVOKE and GRANT LOGON statements.

COP Entries for LAN Connections

CLI clients work a little differently than ODBC clients. Any CLI based utility will dynamically generate a set of "cop" names at the time you try to make a connection. Example of entries in a "hosts" file:

141.206.28.01	SMP001-7	educ1	educcop1
141.206.28.02	SMP001-8	educ2	educcop2
141.206.28.03	SMP001-9	educ3	educcop3
141.206.28.04	SMP001-10	educ4	educcop4

For example, if you have a 4-node system, you can have four entries for the hostid: TDPIDcop1, TDPIDcop2, TDPIDcop3, TDPIDcop4. Where you put these "cop" entries for address resolution is up to you. COP entries for multiple hosts can be placed in the local hosts file OR in the DNS server file. Most people use DNS, since it is a central repository. If you have "cop" entries in the local hosts file AND they are also in the DNS server file, which are used?

The usual order is to first look in the local /etc/hosts file and then look at DNS server files. You can specify the order of resolution in the "/etc/netconfig" file.

Typically, the place to manage these cop entries is definitely the DNS server. When there are changes, it is much easier to do them in one place rather than on every machine that connects to Teradata.

Session Related Views

DBC.LogonRules[V]

Returns a list of logon rules generated by GRANT and REVOKE statements.

DBC.LogOnOff[V][X]

Provides information about logon attempts (successful or unsuccessful) and logoffs.

DBC.SessionInfo[V][X]

Provides information about the current user or all users currently logged on.

There are three system views that you can use to monitor database access. They are:

DBC.LogonRules[V]

Retrieves information about logon rules generated as a result of successfully processed GRANT/REVOKE LOGON statements. This view uses columns from the DBC.LogonRuleTbl.

DBC.LogOnOff[V][X]

Supplies information about logon and logoff activity. This view uses columns from the DBC.EventLog table, which records both successful and unsuccessful logon attempts.

DBC.SessionInfo[V][X]

Provides information about users who are currently logged on. This view uses columns from DBC.SessionTbl.

Dictionary Tables accessed include:

- DBC.LogonRuleTbl
- DBC.EventLog
- DBC.SessionTbl

LogonRules View

Provides information about logon rules that are created by GRANT LOGON and REVOKE LOGON statements.

Returns rules from the DBC.LogonRuleTbl.

DBC.LogonRules[V]

UserName	LogicalHostID	LogonStatus
NullPassword	CreatorName	CreateTimeStamp

Example:

List logon rules for Host ID 1.

```
SELECT Username, LogicalHostID,
       LogonStatus, NullPassword
FROM   DBC.LogonRulesV
WHERE  LogicalHostID = 1
ORDER BY Username;
```

Results:

Username	LogicalHostID	LogonStatus	NullPassword
Default	1024	G	F
Guest001	1	G	F
Guest002	1	R	F
Guest003	1	R	F
Sysdba	1	G	F

The LogonRules view retrieves information about logon rules generated as a result of successfully processed GRANT LOGON statements. This information is stored as rows in the system table DBC.LogonRuleTbl.

This view returns information about the defined rules that you, as the administrator, specify with the GRANT LOGON statement. This statement controls access to the Teradata Database from any server or host.

System administrators or security administrators must specifically authorize user logon requests without passwords.

Example

The SQL statement on this slide requests a list of the logon rules sorted by username. The response displays that user "Guest001" can log on using host id 1.

Not shown: If a system had LDAP users such as ldapuser1 and ldapuser2 and they were allowed to log on to the database without a password, they would appear in this display with NullPassword = T.

LogOnOff View

Provides information about logon and logoff activity, including bad logon attempts and sessions forced off.

[DBC.LogOnOff\[V\]\[X\]](#)

LogDate	LogTime
UserName	AccountName
Event	LogicalHostId
IFPNo	SessionNo
LogonDate	LogonTime
LogonSource	

Partial list of columns in this view

Example:

List "bad" logon attempts and sessions forced off during the last seven days.

```
SELECT CAST (LogDate AS FORMAT 'YYYY-MM-DD')
,LogTime
,CAST (UserName AS FORMAT 'X(12)')
,Event
FROM DBC.LogOnOffV
WHERE (Event LIKE 'Bad%'
OR Event LIKE 'Forced%')
AND LogDate > CURRENT_DATE - 7
ORDER BY LogDate, LogTime ;
```

Results:

LogDate	LogTime	UserName	Event
2019-09-12	11:12:54.34	student30	Bad User
2019-09-12	11:15:25.14	student118	Bad Password
2019-09-12	12:20:19.11	student105	Bad Account
2019-09-13	08:28:51.58	student113	Forced Off

The DBC.LogOnOff[V][X] views provide information about users who have logged on and off. You can also use this view when you need to know about a user's failed attempts to logon. This slide shows an example of the DBC.LogOnOff view.

DBC.LogOnOff event column definitions include:

	<u>Event</u>	<u>Result</u>
Logon	Bad User Bad Password Bad Account	Logon failed.
Logoff	Forced Off	User had their session aborted.

SessionInfo View

Returns information about the current users or all users currently logged on.

[DBC.SessionInfo\[V\]\[X\]](#)

Partial list of columns in this view

UserName	AccountName	SessionNo	DefaultDataBase	IFPNo	Partition
LogicalHostId	HostNo	CurrentCollation	LogonDate	LogonTime	LogonSequenceNo
LogonSource	ExpiredPassword	TwoPCMode	Transaction_Mode	ProfileName	CurrentRole
LogonAcct	LDAP	AuditTrailID	CurlsolationLevel	QueryBand	ProxyUser
ProxyCurRole					

Example: List all users currently logged on, their session source, the logon date, and connect time.

```
SELECT  UserName, 'from ' || CAST (LogonSource AS CHAR(55)) AS "Logon Info"
        ,CAST (LogonDate AS FORMAT 'YYYY-MM-DD')
        ,CAST (TIME - LogonTime AS FORMAT '99:99:99') AS ConnectTime
FROM    DBC.SessionInfoV
ORDER BY UserName ;
```

UserName	Logon Info	LogonDate	ConnectTime
DBC	from (TCP/IP) BCB6 127.0.0.1 FACTORY 13866 ROOT BTEQ	2019-09-17	00:25:40
STUDENT102	from (TCP/IP) 8A84 127.0.0.1 FACTORY 3091 LINUX102 B	2019-09-17	03:16:21
STUDENT103	from (TCP/IP) 0521 153.65.42.41 153.64.112.7 3624 IM	2019-09-17	03:33:11
STUDENT103	from (TCP/IP) 0507 153.65.42.41 153.64.112.7 4824 IM	2019-09-17	03:35:34
STUDENT104	from (TCP/IP) 04ED 153.65.42.202 153.64.112.7 4460 M	2019-09-17	03:19:43
VIEWPOINT	from (TCP/IP) 0540 153.65.42.35 FACTORY 864 SYSTEM IS	2019-09-17	03:04:22

This view provides information about users who are currently logged on the system.

LogonSource May contain up to 11 fields, depending on the values returned.
 Operating system name (e.g., VM or MVS) followed by:
 TDP name
 VM user ID or MVS job name
 Environment name (e.g., TSO, CICS) etc.

Partition
 DBC/SQL = an SQL session
 EXPORT = a FASTEXPORT session
 FASTLOAD = a FASTLOAD session
 HUTPARSE = an ARC data session
 MLOAD = a MULTILoad session
 MONITOR = sessions running in a performance monitoring application
 NONE = session is recognized but not yet assigned

To get a count of load jobs that are currently executing, you can use the following SQL.

```
SELECT COUNT(DISTINCT LogonSequenceNo) AS Utility_Cnt
FROM    DBC.SessionInfoV
WHERE    Partition IN ('Fastload', 'Export', 'MLoad');
```

The LogonAcct and AccountName columns will usually have the same account id for a user. If a user changes the account id within a session, the AccountName column will reflect the current account id.

Additional Utilities to View Sessions

Viewpoint – Query Monitor and My Queries

- Provides functions to view sessions and details about user sessions
- Optionally, sessions can be aborted or have their priority changed

Older (and less used) utilities that can be used to view sessions include:

- **QrySessn** – utility started via Supervisor (e.g., Viewpoint – Remote Console)
 - Provides display of sessions; Supervisor is used to abort sessions
- **gtwglobal** – system utility
 - Not as commonly used because Viewpoint provides an easier interface
 - Only monitors/abort LAN-based sessions
 - gtwglobal or xgtwglobal (X Windows)
- **Teradata Manager** and/or **Performance Monitor** – Windows utilities
 - No longer supported with current releases of Teradata
 - provided functions to view, abort sessions and change a session's priority

Gateway Global Utility

Gateway Global is not as commonly used as it once was because the Query Monitor display of Viewpoint is much easier to use. However, you can still access the Gateway Global Utility by invoking the following commands: **gtwglobal** or X-version **xgtwglobal**

Session Control

The Gateway Global utility allows you to monitor and control Teradata database network-attached users and their sessions. For example, by starting the utility and issuing utility commands with this utility, you can monitor network sessions and traffic, disable logons, force users off the Teradata database and diagnose gateway problems.

Disconnect and Kill Commands

The Disconnect User/Session and Kill User/Session commands are similar in that they both disconnect sessions from the database. The Kill command will abort one session immediately or all sessions of a particular user, then log the user off. The Disconnect command simply puts the sessions in a disconnect state and does not log the user off. The database is still aware of the sessions, and if the user re-establishes the connection from their client workstation, the sessions are allowed to re-connect.

Examples of Gateway Global Commands

DISPLAY GTWALL	Displays all sessions connected to the gateway.
DISPLAY SESSION	Displays information about a specific session on the gateway.
KILL SESSION	Terminates a specific session. Must know session number.

Viewpoint – Query Monitor

The Query Monitor portlet of Viewpoint allows you to view and control sessions.

SESSION ID	STATE ICON	ΔCPU	ΔI/O	CPU SKEW	PJI	DURATION	USERNAME
419981	⊞	0.002	0			0:00:00	DBC
422211	⊞	0.004	0			0:00:00	VIEWPOINT
425122	⇒	0	0	80.77	0	0:00:00	VIEWPOINT
468184	⊞	0	0			0:00:00	STUDENT103
468187	⊞	0	0			0:00:00	STUDENT103
472440	⊞	0.004	0			0:00:00	STUDENT105
472573	⊞	0	0			0:00:00	STUDENT108
475394	⇐	0	0	92.31	0	0:19:45	DBC
475444	⊞	0	0			0:00:00	VIEWPOINT
475482	⊞	0	0			0:00:00	STUDENT105_A
475486	⊞	0	0			0:00:00	STUDENT106_A
475497	⊞	0	0			0:00:00	STUDENT104_A
475513	⊞	0	0			0:00:00	STUDENT103
475524	⊞	0	0			0:00:00	STUDENT102_A
475535	⊞	0	0			0:00:00	STUDENT101_A
475541	⊞	0	0			0:00:00	STUDENT107
475551	⊞	0.004	0			0:00:00	STUDENT101
475582	⊞	0	0			0:00:00	STUDENT108_A
475611	⊞	0	0			0:00:00	VIEWPOINT
475622	⊞	0	0			0:00:00	DBC
475644	⊞	0.004	0			0:00:00	VIEWPOINT
475656	⊞	0.002	0			0:00:00	DBC
475670	⊞	0.004	0			0:00:00	DBC
475681	⇐	0	0	88.46	0	0:09:51	DBC

31 rows total

Teradata Viewpoint enables database and system administrators and business users to monitor and manage Teradata Database systems from anywhere using a standard web browser.

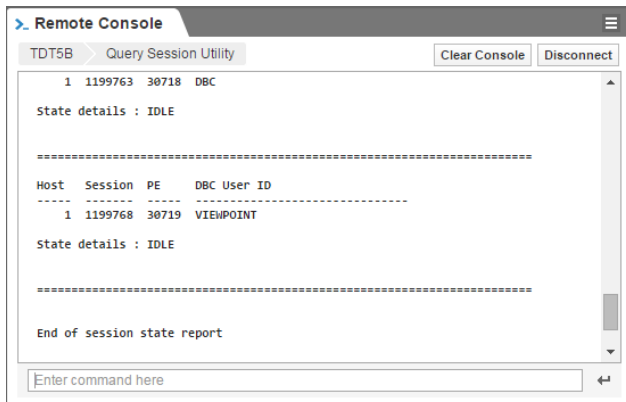
Teradata Viewpoint allows users to view system information, such as query progress, performance data, and system saturation and health through preconfigured portlets displayed from within the Teradata Viewpoint portal. Portlets can also be customized to suit individual user needs. User access to portlets is managed on a per-role basis.

Database administrators can use Teradata Viewpoint to determine system status, trends, and individual query status. By observing trends in system usage, system administrators are better able to plan project implementations, batch jobs, and maintenance to avoid peak periods of use. Business users can use Teradata Viewpoint to quickly access the status of reports and queries and drill down into details.

The Query Monitor portlet allows you to view information about queries running in a Teradata Database system so you can spot problem queries. You can analyze and decide whether a query is important, useful, and well written. After you have identified a problem query, you can take action to correct the problem by changing the priority or workload, releasing the query, or aborting the query or session. You can take these actions for one query or session, or multiple queries or sessions at a time.

Viewpoint – Remote Console – Query Session

From Remote Console, Select Query Session



Query Session prompts for:

- Please enter logical host id? 1
- Please enter session ids? *
- Is detail information needed? Y

From the Viewpoint Operator Console, you can also abort sessions.

- The ABORT SESSION command aborts the currently running SQL transaction in progress.
- The LOGOFF option also terminates the user session.

From Viewpoint Operator Console, the ABORT SESSION options include:

```
ABORT SESSION hostid:ses# [LOGOFF]
               hostid.username
               *.username
               hostid.*
               *.*
```

To abort a large number of sessions quickly:

```
ABORT SESSION *.* LOGOFF
```

The Query Session utility provides information about all Teradata sessions. To start the utility, enter **START QRYSESSN** in the Supervisor window. Possible Teradata session states include:

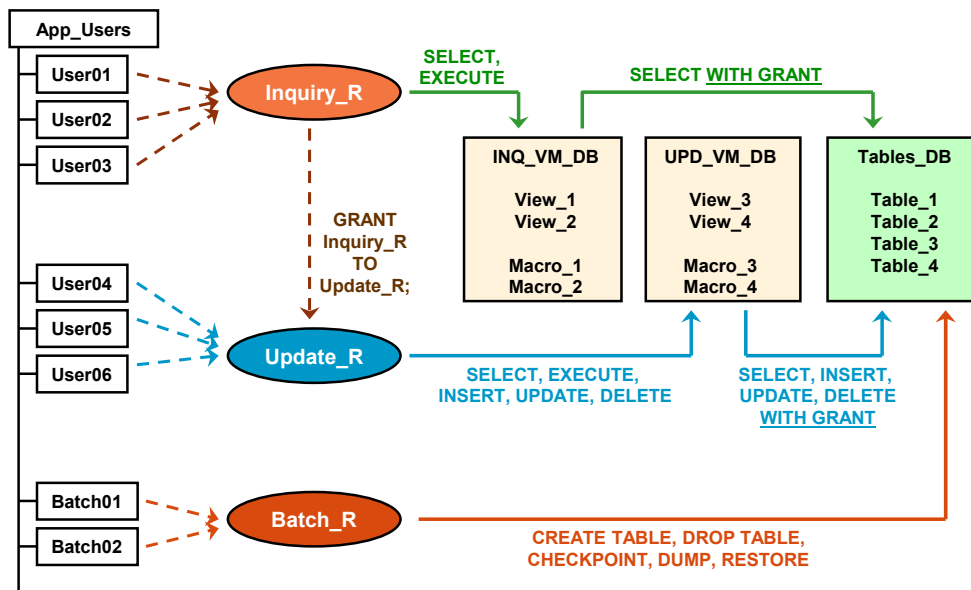
<u>State</u>	<u>Description</u>
Unknown	The session number is not recognized.
Idle	Process is not taking place at this time.
Delay	The query is on a Teradata DWM Delay queue.
Parsing	Session is in the DBC/SQL parser phase.
Active ¹	Session has sent steps to the dispatcher/AMPs
Aborting ¹	Session is aborting the latest request.
Blocked ²	Session is waiting for a database lock to release.
Response	Response to session request is in process.

Example

The example on this slide illustrates using the Remote Console portlet of Viewpoint to access the Query Session utility. A sample Query Session Report and the headings displayed by Query Session for this report. You can use an asterisk (*) as a wild card symbol to depict all hosts and/or all sessions. If a session is idle, only the session identifier information would be displayed.

Answering yes to this prompt provides more details for the archive and load utilities.

A Recommended Structure Using Roles



Access rights are granted to roles instead of being inherited by users.

Roles give the option of grouping different types of users under a single database.

Managing access rights is important for:

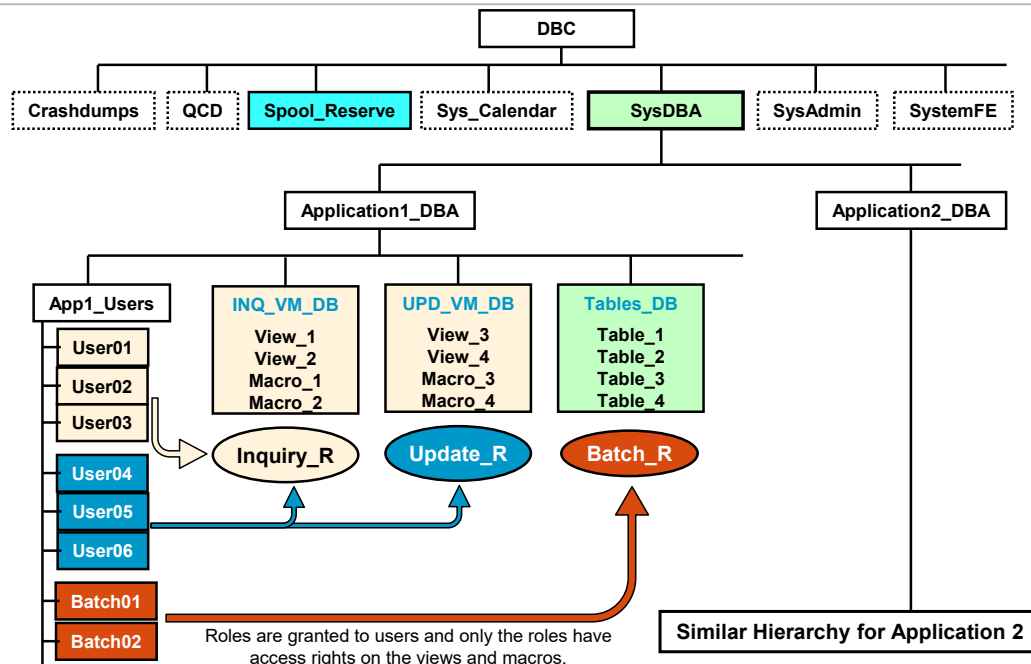
- New user creation
- Security rule enforcement
- Data maintenance
- Training and documentation
- Archiving and recovery

An access rights structure recommended for the Teradata database has the following characteristics:

- Users do not have direct access to data tables, unless they are performing batch operations.
- Users access databases that contain only views and macros.
- VM_DB databases contain *only* views and macros.
- TABLE databases contain *only* tables.
- Access rights are *only* extended at the database or user level, not at the individual table level.

The example on this slide illustrates an example of the suggested Teradata access rights scheme utilizing roles.

A Recommended System Hierarchy



A system structure recommended for the Teradata database is shown on this slide. Each major application function has an associated administrator that would have control of the users and databases within that application function.

Keys to the hierarchy on this slide are:

- INQ_Users** database or set of “inquiry” users
- UPD_Users** database or set of “update” users
- INQ_VM_DB** database of views and macros that access tables in Tables_DB
- UPD_VM_DB** database of views and macros that update tables in Tables_DB
- Tables_DB** database of user data tables
- Batch_Users** database or set of “batch” users; operational users that execute utilities that directly access the tables (e.g., TPT Load)

Optionally roles can be used to easily maintain access rights and reduce the number of access rights.

Teradata Secure Zones

- Secure Zones (15.10 feature) supports grouping of user/database hierarchies into secure zones, and restriction of user access to database objects within a secure zone.
 - Consolidation of multiple tenants into one instance of a database system
 - Isolation of tenants from each other as if they were running on physically segregated databases
 - Users within a tenancy have no access or visibility to objects within other tenancies
- Addresses regulatory compliance requirements to separate administration duties from access control. Data within a zone is secure from being accessed by database or zone administrators.
 - Tenant Zones are managed by a system-level Zone Administrator
 - Administration of objects within a tenancy is done by tenant DBAs as required
- Syntax
 - New DDLs: CREATE ZONE, ALTER ZONE, DROP ZONE
 - New DCLs: GRANT ZONE, REVOKE ZONE
 - Extensions to existing CREATE USER, MODIFY USER syntaxes
- System tables: DBC.Zones, DBC.ZoneGuests

Secure Zones Elements:

Zone - 'zone' is a secure area which restricts unauthorized outside users from accessing data and objects within the zone. A zone can be created by a database administrator who has the 'Create Zone' privilege.

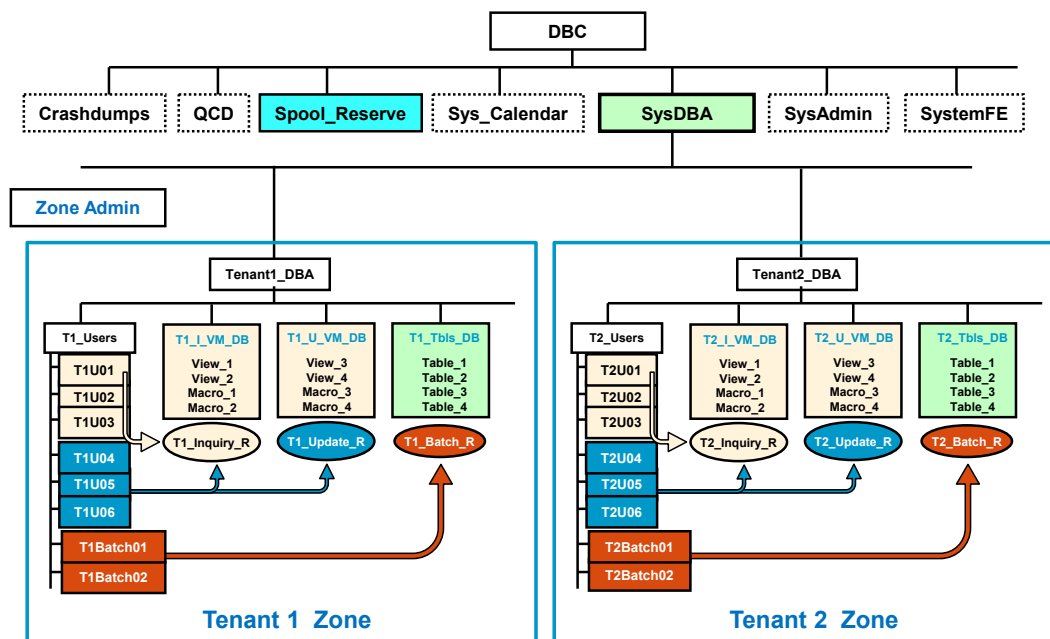
Zone Creator – a Zone creator creates a zone and assigns a user or a database as the zone root. Zone creator will be prevented from accessing the objects or data within the zone he creates.

Rules concerning a zone creator are as follows:

1. Only the zone creator can add a root and primary DBA to the zone.
2. Only the zone creator can delink the root and primary DBA from the zone.
3. Zone creator may grant zone access to users or roles outside the zone. Note: Zone grants do not allow grantees to access database objects within the zone. The zone's primary DBA must also grant Discretionary Access Control (DAC) privileges to zone guests before access is permitted.
4. Zone creator is also responsible for revoking zone access to a zone.
5. Zone creator must be a user having 'Create Zone' and 'Drop Zone' privileges and additionally have all privileges on the user which will become a root of the zone.
6. Zone creator cannot be dropped until the zone is dropped.

Zone Root - Zone root is a database or a user on which a zone is created

Multi-tenancy Secure Zones Example



Zone Guest

A zone guest is a role or user outside the zone who may be granted privileges to create/access objects in the zone where he is a guest. To make an external LDAP user as zone guest, an external role may be granted a zone and privileges to zone objects. If an external user logs on with that role, he will be able to access zone objects he has privileges on.

Rules concerning a zone guest:

1. Only zone members can grant privileges on database objects within the zone to zone guests.
2. Zone members cannot grant privileges to the zone guest with "WITH GRANT OPTION".
3. A zone can have more than one zone guest.
4. A user/role can be a guest of more than one zone.
5. Zone guest with required privileges can create users, databases, and TVM objects inside the zone.
6. Zone guest can create views, triggers, macros etc. on the zoned objects in their perm space.
7. Zone guest doesn't have the privilege to add another guest to a zone.
8. A zone member cannot be a zone guest in another zone [when nested zones are supported in future, a zone guest may either be a non-zone user or a member of the outer zone].

Use Case: Multi-tenancy

Consolidation of multiple tenants onto one instance of a database system while isolating the tenants from each other as if they were running on physically segregated databases.

- Users within a tenancy have no access or visibility to objects within other tenancies.

Summary

- The mission of security administration is to prevent unauthorized user access to the database and its resources.
- To protect system access:
 - Associate passwords with usernames.
 - Associate application users with specific hosts.
- You can control database access by granting access to views and macros.
 - Views can limit access to certain columns and rows.
 - Macros can limit the actions a user can perform.
- Good access rights management facilitates your role as system administrator in security rule enforcement, data maintenance, archive and recovery, and other areas.
- Characteristics of a good database structure include:
 - Users belong to a database group and get their access rights from roles.
 - Users do not have direct access to tables.
 - Access rights are extended at the database, user, or role level (not at the individual table level).

This slide summarizes some important concepts regarding this module.



Module 10: Review Questions

1. True or False: Although usernames are the basis for identification in the system, username information usually is not protected information.
2. True or False: A REVOKE LOGON command for a user removes a GRANT LOGON row in the DBC.LogonRuleTbl for the that user.
3. True or False: To change the minimum number of characters in a valid password from 6 to 8, you would update the DBC.LogonRules table.
4. What does 1024 represent in the DBC.LogonRules view? _____
5. Which two choices can be used to view host or mainframe sessions?
 - a. Sessions utility
 - b. QrySessn utility
 - c. Gtwwglobal utility
 - d. DBC.SessionInfoV view
6. Which view is used to determine why a user logon has failed?
 - a. DBC.LogonsV
 - b. DBC.LogOnOffV
 - c. DBC.AccessLogV
 - d. DBC.SessionInfoV
 - e. DBC.LogonEventsV

Check your understanding of the concepts discussed in this module by completing the review questions as directed by your instructor.



Module 11: Access and Query Logging

Teradata Vantage Administration

Copyright © 2007–2023 by Teradata. All Rights Reserved.

Objectives

After completing this module, you will be able to:

- Describe how the BEGIN/END LOGGING statements capture information about user access to Teradata.
- Describe how to set up user access logging.
- Use system views to gather information about data access.
- Identify the reasons for using the Database Query Log.
- Identify the tables and views that make up the DBQL facility.

Access and Query Logging

There are two logging facilities available to the database and/or security administrator.

- **Access Logging Facility**
 - Used for access and security audit analysis.
 - May be used to monitor data access requests (via access rights checks) and log entries for requests that are granted and/or denied.
- **Query Logging Facility (DBQL)**
 - Used for query activity and workload analysis.
 - Can be used to track processing behavior and/or capture detailed information about the queries that are running on a system.
 - Workloads can be utilized with Teradata Analyst tools such as Teradata Index Wizard.

Access Logging Facility

The Access Logging facility provides an administrator with the capability to monitor data access requests in the system and log granted and/or denied requests.

With Access Logging, logged rows are written immediately to disk before the SQL is executed, which can slow down short query work. At the same time, this approach to logging does offer higher reliability and can register negative accesses (attempts to view data that did not succeed), which would not show up in DBQL.

Query Logging Facility

The Database Query Log (DBQL) is a feature that you can employ to log query processing activity for later analysis. Query counts and response times can be charted and SQL text and processing steps can be compared to fine-tune your applications for optimum performance.

DBQL provides a series of predefined tables that can store, based on rules you specify, historical records of queries and their duration, performance, and target activity. DBQL is flexible enough to log information on the variety of SQL requests that run on Teradata, from short transactions to longer-running analysis and mining queries. You begin and end collection for a user or group of users and/or one or a list of accounts. DBQL is streamlined for collection efficiency and provides more detail about the query than the Access Log. On the other hand, because DBQL caches data in memory before writing it periodically to disk, there is some delay in seeing the logged data, and it is possible to lose data that is still in the cache on a restart.

Access Logging

An administrator can ...

- use the Access Logging facility to monitor data access requests and log entries for requests that are granted and/or denied.
- optionally capture the SQL text along with the access right check.

The following statements are used to specify objects and/or SQL requests to monitor for specific or all users.

- **BEGIN LOGGING**
 - Starts the monitoring of data access requests by Teradata.
- **END LOGGING**
 - Ends the monitoring of data access requests by Teradata.

The Access Logging facility provides an administrator with the capability to monitor data access requests in the system and log granted and/or denied requests.

The DDL statements **BEGIN LOGGING** and **END LOGGING** are used to control the monitoring of access rights checks performed by the Teradata Database. Each time you execute a **BEGIN LOGGING** statement, the system table `DBC.AccLogRuleTbl` receives applicable rule entries. (The system view `DBC.AccLogRules` offers access to the contents of this table.)

When a user named in a **BEGIN LOGGING** statement attempts to execute a specified action against a specified object, the Teradata Database checks the access rights necessary to execute the statement according to the rules in `DBC.AccLogRuleTbl`. The privilege checks made and/or the access results are logged in the system table `DBC.AccLogTbl`. (The system view `DBC.AccessLog` offers access to the contents of this table.)

A logging entry does not indicate that a statement was executed; rather, it indicates that the system checked the privileges necessary to execute the statement.

You can terminate logging by submitting an **END LOGGING** statement for any action, user, or object for which logging is currently active. Note that you cannot end logging begun for a specific username by omitting the *BY username* option.

Objects Used in Access Logging

Users who are granted EXECUTE permission on the following macro can use the BEGIN LOGGING and END LOGGING statements.

Example:

```
GRANT EXECUTE ON DBC.AccLogRule TO SecAdmin;
```

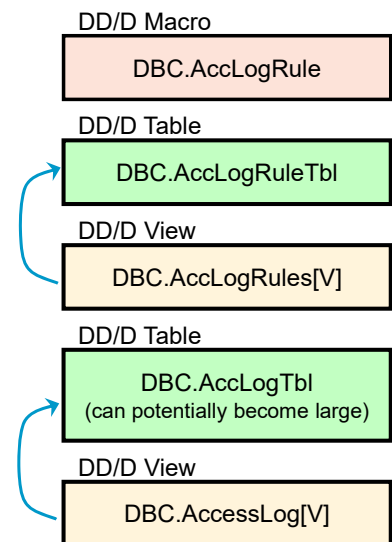
This allows "SecAdmin" to execute the BEGIN LOGGING and END LOGGING statements.

Execution of **BEGIN LOGGING** or **END LOGGING** statements causes rows (representing the rules) to be added or updated in ...

To view the rules in this table, **SELECT** from these views.

Based on the rules, access of specified objects or SQL statements cause entries to be placed in ...

To view the log of entries, **SELECT** from these views.



You must have EXECUTE privileges on the macro DBC.AccLogRule to execute the BEGIN LOGGING and END LOGGING statements.

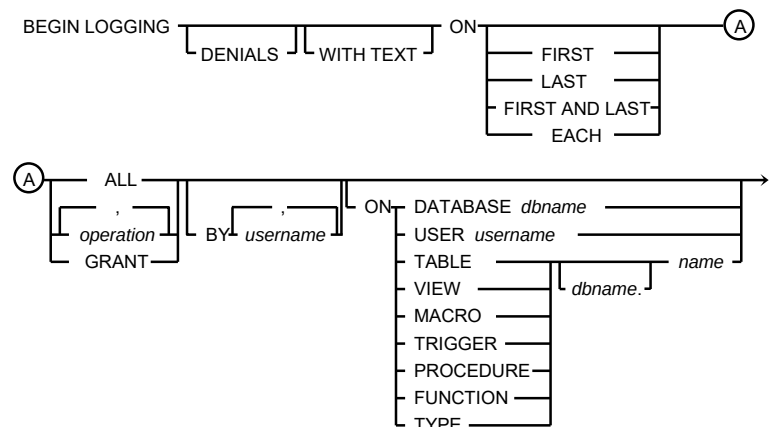
Note: DBC.AccLogRule is a “dummy macro”. It only has a ; in it.

The BEGIN LOGGING and END LOGGING statements start and stop the auditing of data access requests. The BEGIN LOGGING and END LOGGING statements store rows in the DBC.AccLogRuleTbl. To view the rows in this table, use the DBC.AccLogRules[V] views.

If the user does not submit a BEGIN LOGGING statement, then by default the system does not generate any entries on any user action.

When an object or SQL request (identified in one of the access log rules) is accessed, an entry is logged in the DBC.AccLogTbl. To view the rows in this table, use the DBC.AccessLog[V] views.

BEGIN LOGGING Statement



Operation Any function for which an access right can be granted (e.g., GRANT).

BY *username* – implies all users, if not specified.

ON *object-name* – implies all entities, if not specified. Valid object-names are:

DATABASE	<i>database_name</i>	USER	<i>user_name</i>	TABLE	<i>table_name</i>	VIEW	<i>view_name</i>
MACRO	<i>macro_name</i>	PROCEDURE	<i>procedure_name</i>	TRIGGER	<i>trigger_name</i>	FUNCTION	<i>function_name</i>

Teradata verifies a user's access rights when the user attempts to access an object. As the administrator, you can capture information about checks performed on a user's access rights with the BEGIN LOGGING statement. When you activate logging, the specified privilege check performed by the Teradata Database generates a row in the DBC.AccLogTbl.

The BEGIN LOGGING statement has a number of options.

DENIALS — Tracks only those entries when statement execution fails because the user does not have the privilege(s) necessary to execute the statement.

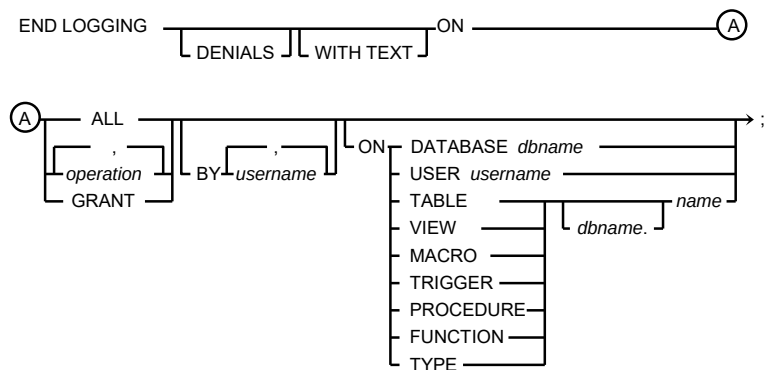
FIRST, LAST, EACH — Defines the frequency with which log entries are made. The default for BEGIN LOGGING is FIRST.

ALL — Tells the system to make a log entry when the user attempts any action against the specified object.

BY username — Lists the users for which the system will make log entries. The default is all users.

ON keyword object-name — Defines which objects will generate rows in the log table when a user attempts to access them. Specific keyword object-name combinations can be specified as see in the syntax diagram on this slide. Absence of the ON keyword object name option implies all entities that the user attempts to access. A single logging statement may contain up to 20 objects.

END LOGGING Statement



Operation Any function for which an access right can be granted (e.g., GRANT).

BY *username* – implies all users, if not specified.

ON *object-name* – implies all entities, if not specified. Valid object-names are:

DATABASE	<i>database_name</i>	USER	<i>user_name</i>	TABLE	<i>table_name</i>	VIEW	<i>view_name</i>
MACRO	<i>macro_name</i>	PROCEDURE	<i>procedure_name</i>	TRIGGER	<i>trigger_name</i>	FUNCTION	<i>function_name</i>

Stops the auditing of SQL requests that attempt to access data that was started with a BEGIN LOGGING statement. The END LOGGING statement erases only the frequency or text flags for the specified actions and user or object. However, if erasing a frequency leaves all logging blank for a particular user, database, and table, then the row is deleted from the AccLogRuleTbl table.

Use of the END LOGGING statement results in an error if BEGIN LOGGING is not currently in effect for the community for which logging is to be ended.

The END LOGGING statement has a number of options. Several are described below:

DENIALS — tracks only those entries when statement execution fails because the user does not have the privilege(s) necessary to execute the statement.

ALL — tells the system to end the logging when the user attempts any action against the specified object.

BY username — lists the users for which to end logging on. The default is all users.

ON keyword object-name — Defines which objects will generate rows in the log table when a user attempts to access them. Specific keyword object-name combinations can be specified as see in the syntax diagram on this slide. Absence of the ON keyword object name option implies all entities that the user attempts to access. A single logging statement may contain up to 20 objects

Setting Up Access Logging

STEP 1 Install Access Logging on the system:

1. Run the DIP script DIPACC to install the DBC.AccLogRule macro.
2. Restart the database to activate the code.

STEP 2 Create and empower a security administrator:

```
CREATE USER SecAdmin AS PASSWORD=secpasswd, PERM=0, SPOOL=5E9;
GRANT EXECUTE ON DBC.AccLogRule TO SecAdmin;
GRANT EXECUTE ON DBC.LogonRule TO SecAdmin;
GRANT EXECUTE ON DBC.DBQLAccessMacro TO SecAdmin;
```

STEP 3 Define access logging rules. Examples:

1. Log all attempts to access security macros.
2. Log all denied attempts to access DBC User.
3. Log any CREATE or DROP USER/DATABASE/ROLE/PROFILE or GRANT commands.

```
BEGIN LOGGING WITH TEXT ON EACH ALL
ON MACRO DBC.LogonRule,
MACRO DBC.AccLogRule,
MACRO DBC.DBQLAccessMacro;

BEGIN LOGGING DENIALS WITH TEXT ON EACH ALL
ON USER DBC;

BEGIN LOGGING WITH TEXT ON EACH DATABASE, USER, ROLE, PROFILE, GRANT;
```

As part of the installation process, a DIP script is executed which creates a special security macro called DBC.AccLogRule and enables this software capability. After you run the script, you must reset the system to initialize the logging software.

Example

Step 3 on this slide is an example of three access logging rules. These 3 rules provide a good start in logging access information at the system database level. Additional rules may be needed for specific applications.

- Log all attempts to access the security macros — Creates a new row in the log table each time a user attempts to access DBC.LogonRule, DBC.AccLogRule, and the DBC.DBQLAccessMacro macros. The WITH TEXT option stores SQL statement contents in the log table.
- Log all denied attempts to access system user DBC — Logs all denied attempts made by any user to access system user and Teradata stores the SQL text in the log table.
- Log any SQL statements that involve CREATE or DROP USER/DATABASE/ROLE/PROFILE or GRANT.

Access Log Views

DBC.AccLogRules[V]

Contains current logging rules generated by BEGIN and END LOGGING statements.

DBC.AccessLog[V]

Contains log entries collected as a result of applying access log rules.

Dictionary Tables Accessed:

- DBC.AccLogRuleTbl
- DBC.AccLogTbl

There are actually four system-supplied views that provide information about the entries in the DBC.AccLogRuleTbl[V] and the DBC.AccLogTbl[V]. They are:

DBC.AccLogRules[V]

Teradata maintains entries in this view's underlying table as the result of executing BEGIN/END LOGGING statements. The system uses these entries to determine which privilege checks should generate rows in the DBC.AccLogTbl.

DBC.AccessLog[V]

The underlying table for this view is DBC.AccLogTbl. Each entry in DBC.AccLogTbl indicates the results of a privilege check performed against a Teradata SQL request, based on the criteria defined by the BEGIN LOGGING statement.

Underlying DD/D Tables

- DBC.AccLogRuleTbl
- DBC.AccLogTbl

AccLogRules View

DBC.AccLogRules[V] views – return information about current access logging rules.

UserName	DatabaseName	TVMName	AcrAlterFunction (AFN)	AcrCheckpoint (CPT)
AcrCreateDatabase (CDB)	AcrCreateFunction (CFN)	AcrCreateMacro (CMC)	AcrCreateTable (CTB)	AcrCreateUser (CUS)
AcrCreateView (CVW)	AcrCreateProcedure (CSP)	AcrCreExtProcedure (CXP)	AcrDelete (DEL)	AcrDropDatabase (DDB)
AcrDropFunction (DFN)	AcrDropMacro (DMC)	AcrDropTable (DTB)	AcrDropUser (DUS)	AcrDropView (DVW)
AcrDropProcedure (DSP)	AcrDump (DMP)	AcrExecute (EXE)	AcrExecuteFunction (EFN)	AcrExecuteProcedure (ESP)
AcrGrant (GRT)	AcrIndex (IDX)	AcrInsert (INS)		

```
SELECT  UserName      (CHAR (6)) AS "User"
        ,DatabaseName (CHAR (8)) AS "Database"
        ,TVMName      (CHAR (15)) AS "TVM_Name"
        ,AcrCreateDatabase, AcrCreateUser, AcrCreateRole, AcrCreateProfile, AcrDropDatabase
        ,AcrDropUser, AcrDropRole, AcrDropProfile, AcrGrant, AcrSelect, AcrExecute
FROM    DBC.AccLogRulesV;
```

User	Database	TVM Name	CDB	CUS	CRO	CPR	DDB	DUS	DRO	DPR	GRT	SEL	EXE
All	All	All	E +	E +	E +	E +	E +	E +	E +	E +	E +		
All	DBC	All	E -	E -			E -	E -			E -	E -	E -
All	DBC	LogonRule									E +		E +
All	DBC	AccLogRule									E +		E +
All	DBC	DBQLAccessMacro									E +		E +

Position #1 = How often to log requests (F, L, B, E, blank = First, Last, Both, Each, None)
 Position #2 = How often to log denials (F, L, B, E, blank = First, Last, Both, Each, None)
 Position #3 = How often to save text (+ All entries, - Denials, = All Specified)

The SQL statement on this slide requests a list of the current rules stored in the DBC.AccLogRules table. It limits the rules to CREATE and DROP database and user, GRANT, SELECT, and EXECUTE.

The response produces four rows. Each contains a series of codes under each privilege column. There are three positions under each privilege. The first position indicates how often to log privilege checks. The second position indicates how often to log denials. The third position indicates when to save text. The following codes are used for positions 1 and 2:

- B Log FIRST and LAST occurrences.
- E Log each occurrence.
- F Log the FIRST occurrence.
- L Log the LAST occurrence.
- Blank No logging

The third position for text uses the following codes:

- Save text only for Denial entries.
- + Save text for all entries.
- = Save text for all entries specified in multiple BEGIN LOGGING Statements

The code E+ means insert a row for each occurrence and save the text. The code E- means insert a row for each occurrence but only save the text for denials.

BEGIN LOGGING – Example

```

BEGIN LOGGING DENIALS WITH TEXT ON EACH SELECT ON TABLE PD.Employee;      1
BEGIN LOGGING          WITH TEXT ON FIRST INSERT ON TABLE PD.Employee;    2
BEGIN LOGGING          WITH TEXT ON FIRST AND LAST DELETE ON TABLE PD.Employee; 3
BEGIN LOGGING          WITH TEXT ON FIRST UPDATE ON TABLE PD.Employee;      } 4
BEGIN LOGGING DENIALS WITH TEXT ON LAST UPDATE ON TABLE PD.Employee;

SELECT      UserName          (CHAR (6))      AS "User//Name"
           ,DatabaseName      (CHAR (6))      AS "Dbase//Name"
           ,TVMName           (CHAR (10))     AS "TVM//Name"
           ,AcrSelect, AcrInsert, AcrDelete, AcrUpdate
FROM        DBC.AccLogRulesV
WHERE       DatabaseName = 'PD';

```

		1		2		3		4	
User Name	Dbase Name	TVM Name	SEL	INS	DEL	UPD			
All	PD	Employee	E-	F +	B +	FL=			

Note: This is an academic example, not intended as a best practice example.

Position #1 = How often to log requests (F, L, B, E, blank = First, Last, Both, Each, None)

Position #2 = How often to log denials (F, L, B, E, blank = First, Last, Both, Each, None)

Position #3 = How often to save text (+ All entries, - Denials, = All Specified)

This slide provides an additional example of entries that may appear in the DBC.AccLogRules view.

This view provides information about logging rules currently in effect on the system. These rules were put into effect by successfully processing BEGIN LOGGING statements.

This example is not meant to be a best practice, but simply to illustrate the capability and flexibility in establishing access logging rules.

AccessLog View

These views display entries made to DBC.AccLogTbl.

DBC.AccessLog[V]

LogDate	LogTime	LogonDate	LogonTime	LogicalHostID	IFPNo
SessionNo	UserName	AccountName	OwnerName	AccessType	Frequency
EventCount	AccLogResult	Result	DatabaseName	TVMName	ColumnName
StatementType	StatementText	QueryBand	ProxyUser		

Access Type The same codes are used that indicate an access right.
 Frequency F, L, B, E = First, Last, Both or Each.

To delete entries in the DBC.AccLogTbl, use the [DBC.DeleteAccessLog\[V\]\[X\]](#) views.

- **DELETE FROM DBC.DeleteAccessLogV;**
 – Deletes entries older than 30 days.
- **DELETE FROM DBC.DeleteAccessLogV WHERE LOGDATE < (CURRENT_DATE - 90);**
 – Deletes entries older than 90 days.

The DBC.AccessLog[V] views display the entries made in the DBC.AccLogTbl system table. It returns information on the results of privilege checks performed against user requests to access data, which are logged as determined by the access logging rules.

- Access Type
 The same codes are used to indicate an access right, but with CUS and DUS for CREATE/DROP USER, AN for any privilege (validated for HELP and SHOW commands), HR for HOST UTILITY LOCK, and WL for WRITE LOCK.
- Frequency
 F, L, B, E = First, Last, Both or Each.

Once logging begins, the access log grows very quickly. To keep space consumption under control, you should archive and empty the log regularly using the DBC.DeleteAccessLog view.

Examples:

```
DELETE FROM DBC.DeleteAccessLogV;
```

- Deletes entries from DBC.AccLogTbl older than 30 days.

```
DELETE FROM DBC.DeleteAccessLogV WHERE LOGDATE < (DATE - 90);
```

- Deletes entries from DBC.AccLogTbl older than 90 days.

AccessLog View – Example

Example:

List all of the entries in the Access Log table for the current date.

```
SELECT    LogTime
          ,UserName (CHAR (10)) AS "User//Name"
          ,AccessType      AS "Access//Type"
          ,Frequency       AS "Log//Freq"
          ,AccLogResult    AS "Granted//Denied"
          ,StatementText   AS "Statement//Text"
FROM      DBC.AccessLogV
WHERE     LogDate = CURRENT_DATE
ORDER BY LogDate, LogTime ;
```

Results:

LogTime	User Name	Access Type	Log Freq	Granted Denied	Statement Text
09:04:25	TFACT01	S	E	D	SELECT * FROM PD.Empl...
09:10:17	SYSDBA	I	F	G	INSERT INTO PD.Employee ...
09:12:22	SYSDBA	D	f	G	DEL FROM PD.Employee ...
09:12:31	SYSDBA	D	I	G	DEL FROM PD.Employee ...
09:15:54	SYSDBA	U	F	G	UPDATE PD.Employee SE ...
09:17:04	TFACT01	U	f	D	UPDATE PD.Employee SE ...
09:32:50	TFACT01	U	I	D	UPDATE PD.Employee SE ...

Note: This slide contains the SQL that generated this report.

The SELECT statement on this slide requests the contents of the DBC.AccLogTbl via the DBC.AccessLog view. The response shows seven separate entries.

User TFACT01 executed the following SQL statements at the listed times:

```
09:04:25  SELECT * FROM PD.Employee;
09:17:04  UPDATE PD.Employee SET SalaryAmount = 51000 WHERE Emp# = 100996;
09:24:31  UPDATE PD.Employee SET SalaryAmount = 51000 WHERE Emp# = 100995;
09:32:50  UPDATE PD.Employee SET SalaryAmount = 51000 WHERE Emp# = 100994;
```

Note that only the last UPDATE denial for TFACT01 is logged.

User Sysdba executed the following SQL statements at the listed times:

```
09:10:17  INSERT INTO PD.Employee VALUES
          (101001, 1060, 100991, 3054, 'Scott', 'Bill', 50000.00);
09:12:22  DELETE FROM PD.Employee WHERE Emp# = 100900;
09:12:25  DELETE FROM PD.Employee WHERE Emp# = 100901;
09:12:31  DELETE FROM PD.Employee WHERE Emp# = 100902;
09:15:54  UPDATE PD.Employee SET SalaryAmount = 51000      WHERE Emp# =
          101001;
09:57:20  UPDATE PD.Employee SET SalaryAmount = 51000      WHERE Emp# =
          100800;
```

Note that only the first and last DELETES are based on the Access Log Rules. Also notice that only the first UPDATE appears on this slide. When a FIRST and LAST are both specified for an entry, a lower case 'l' or 'f' are used. An uppercase 'L' and 'F' is used if one is specified without the other.

END LOGGING – Example

Previously, these rules were created for logging on PD.Employee table.

```
BEGIN LOGGING DENIALS WITH TEXT ON EACH SELECT ON TABLE PD.Employee;
BEGIN LOGGING WITH TEXT ON FIRST INSERT ON TABLE PD.Employee;
BEGIN LOGGING WITH TEXT ON FIRST AND LAST DELETE ON TABLE PD.Employee;
BEGIN LOGGING WITH TEXT ON FIRST UPDATE ON TABLE PD.Employee;
BEGIN LOGGING DENIALS WITH TEXT ON LAST UPDATE ON TABLE PD.Employee;
```

To end the logging for PD.Employee table, the following statements can be executed:

```
END LOGGING DENIALS ON SELECT, UPDATE ON TABLE PD.Employee;
END LOGGING ON INSERT, DELETE, UPDATE ON TABLE PD.Employee;
```

To verify the rules have been removed, use the DBC.AccLogRules view:

```
SELECT UserName (CHAR (6)) AS "User//Name"
, DatabaseName (CHAR (6)) AS "Dbase//Name"
, TVMName (CHAR (10)) AS "TVM//Name"
, AcrSelect, AcrInsert, AcrDelete, AcrUpdate
FROM DBC.AccLogRulesV
WHERE DatabaseName = 'PD';
```

User Name	Dbase Name	TVM Name	SEL	INS	DEL	UPD

Rules for PD.Employee have been removed.

This slide provides the END LOGGING statements to remove the logging for the PD.Employee table.

The BEGIN LOGGING statements (that were previously executed) are:

```
BEGIN LOGGING DENIALS WITH TEXT
ON EACH SELECT
ON TABLE PD.Employee;
```

```
BEGIN LOGGING WITH TEXT
ON FIRST INSERT
ON TABLE PD.Employee;
```

```
BEGIN LOGGING WITH TEXT
ON FIRST AND LAST DELETE
ON TABLE PD.Employee;
```

```
BEGIN LOGGING WITH TEXT
ON FIRST UPDATE
ON TABLE PD.Employee;
```

```
BEGIN LOGGING DENIALS WITH TEXT
ON LAST UPDATE
ON TABLE PD.Employee;
```

Query Logging (DBQL) Concepts

- DBQL is a feature that is used to log historical query information.
 - DBQL caches and eventually stores query information in multiple Teradata Data Dictionary tables as the queries are executed.
 - Key use is to track SQL for query and workload analysis.
 - Not intended for live review of queries and query steps.
- Logging is invoked and revoked via SQL statements – `BEGIN QUERY LOGGING`, `END QUERY LOGGING`, and `REPLACE QUERY LOGGING`.
 - Logging can be invoked for an application name (e.g., FASTLOAD), all users, a list of users, a list of accounts, or a specific user/account.
 - You can also selectively exclude users, applications, and so on from logging through the use of the `WITH NONE` option.
 - `REPLACE QUERY LOGGING` allows you to replace a logging rule set.
- By default, 1 row per query is logged that contains user id information and some statistics for that query.
 - Options are available to expand the amount and kind of information to be logged.
- DBSControl option (`DBQLFlushRate`) – determines the frequency (default is 10 minutes) for writing DBQL cache entries to DBQL dictionary tables.

Provides Collection of Historical records based on Rules

DBQL provides a series of predefined tables that can store, based on rules you specify, **historical records** of queries and their duration, performance, and target activity. DBQL is flexible enough to log information on the variety of SQL requests that run on Teradata, from short transactions to longer-running analysis and mining queries. **You begin and end collection for a user or group of users and/or one or a list of accounts or specific applications.**

Performance

The performance of collecting query activity with DBQL is much better than attempting to capture query activity with the Access Logging facility because DBQL stores information in cache memory and will write to disk only when cache is full or when you use `END QUERY LOGGING`.

Notes about performance:

- The impact of turning on DBQL for all-AMP operations is negligible in comparison to the time an all-AMP operation takes to complete.
- The impact of turning on DBQL for single-AMP (PI) and two-AMP (USI) operations does impact the response time of the operation by a small amount. These types of operations are best suited for summary logging where the overhead is negligible.

A single or few AMP queries will be best suited for summary logging, and will exhibit negligible overhead if logging is at that level.

Objects Used in Defining Rules for DBQL

Users who are granted EXECUTE permission on the following macro can use the BEGIN QUERY LOGGING, END QUERY LOGGING, and REPLACE QUERY LOGGING statements.

Example:

```
GRANT EXECUTE ON DBC.DBQLAccessMacro TO Sysdba;
```

Initially, only DBC and SystemFE users are allowed to issue BEGIN/END QUERY LOGGING statements.

Execution of **BEGIN QUERY LOGGING**, **END QUERY LOGGING**, or **REPLACE QUERY LOGGING** statements causes rows (representing the rules) to be added or updated in ...

To view the rules in this table, **SELECT** from these views.

Note: There are additional tables and views that are used to hold and view captured query data – these are shown on next page.

DD/D Macro

DBC.DBQLAccessMacro

DD/D Tables

DBC.DBQLRuleTbl

DD/D Views

DBC.DBQLRules[V]

The DBQL logs are a series of system tables created in database DBC during the Teradata Database installation process. The suite of DBQL components includes a security macro and a view for each table, which are created in database DBC by the DIP utility during installation.

Rules

You define rules that identify for which users and how much data to log for queries. For instance, you can log the first 5,000 characters of any query that runs during a session invoked by a specific user under a specific account. This rule can also be qualified so that only queries that exceed a specified time threshold are logged and those queries that execute in less than the threshold time are simply counted.

The DBC.DBQLRuleTbl table stores the rules resulting from each BEGIN QUERY LOGGING statement. One row exists for each set of specifications, which are made up of *user* and/or *account* plus any options or limits set for the user.

The DBC.DBQLRuleCountTbl table is an internal table that stores the cardinality of DBC.DBQLRuleTbl table.

The DBC.DBQLRules[V] views are used to display DBQL rules that are in effect.

Objects Used in DBQL

Key views and tables used to hold query data are ...

DD/D Views	DD/D Tables	DBQL Purpose
DBC.QryLog[V]	DBC.DBQLLogTbl	Stores default rows (key table)
DBC.QryLogSteps[V]	DBC.DBQLStepTbl	One row per step
DBC.QryLogObjects[V]	DBC.DBQLObjTbl	One row per object referenced in query
DBC.QryLogSQL[V]	DBC.DBQLSqlTbl	Stores full SQL text – multiple rows may be needed
DBC.QryLogSummary[V]	DBC.DBQLSummaryTbl	Queries meeting Summary or Threshold rules
DBC.QryLogExplain[V]	DBC.DBQLExplainTbl	Stores EXPLAIN steps of query
DBC.QryLogXMLV	DBC.DBQLXMLTbl	Stores Optimizer query plan for the logged query as an XML document

Examples of additional DBQL views that provide TDWM information include:

- DBC.QryLogEvents[V] – TDWM events that could affect system performance
- DBC.QryLogEventHis[V] – history of TDWM events
- DBC.QryLogExceptions[V] – query exceptions as defined by Workload Definitions (WD)

The DBQL logs are a series of system tables created in database DBC during the Teradata database installation process. The suite of DBQL components includes a security macro and a view for each table, which are created in database DBC by the DIP utility during installation. Like other system tables, the predefined DBQL logs are created as relational tables in database DBC during normal Teradata database installation. However, while most system tables are populated automatically, you can choose whether you want to populate the DBQL tables.

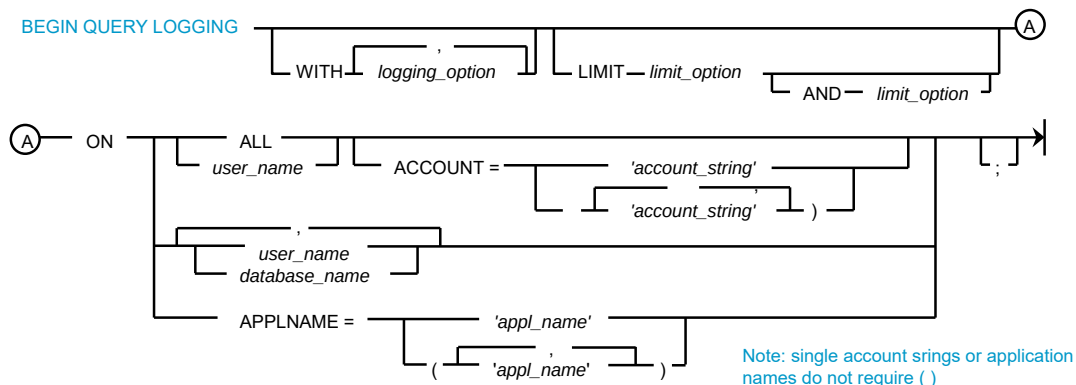
If you choose not to use the feature, the tables remain empty. If you want to use the feature, simply submit a BEGIN/END QUERY LOGGING statement, with or without options, to control the start, magnitude, and end of logging activity. \

The key or foundation table in DBQL is the DBC.DBQLLogTbl table that holds the default rows. When you specify options that result in more information being captured, a default row is still generated in this table plus one or more additional logs (tables) will get rows.

- Exceptions to this are when you use the SUMMARY option or a query completes within a THRESHOLD. In these cases, default rows won't be placed into DBC.DBQLLogTbl.

This slide summarizes some of the key tables and views used by DBQL to hold query data. Details on these views can be found in Appendix F.

BEGIN QUERY LOGGING



- A BEGIN QUERY LOGGING statement without the WITH or LIMIT options causes default rows to be placed in the DBQLogTbl. [A default row contains:](#)
 - User name, account string (expanded), time stamp information
 - Unique ID for process, session, and client (host) connection
 - First 200 characters of SQL statement
- Use of the [WITH and LIMIT option\(s\)](#) are discussed on following pages.
 - When the WITH option is used, the default row is logged in addition to the additional information based on the options specified on the WITH option.

The DBQL facility is controlled by the Teradata SQL statements BEGIN QUERY LOGGING and END QUERY LOGGING. There are numerous collection options using the WITH and LIMIT options.

BEGIN QUERY LOGGING

When submitted by a user with EXECUTE privileges on DBC.DBQLAccessMacro, enables logging for the named users and/or accounts. For active sessions, logging begins when the next query is received. (Teradata recommends a maximum of 100 user/account pairs per statement.)

When you do not specify a LIMIT option, one default row of query-level information is logged in DBQLogTbl for each query processed during a session that is initiated by any user for whom a query logging rule exists.

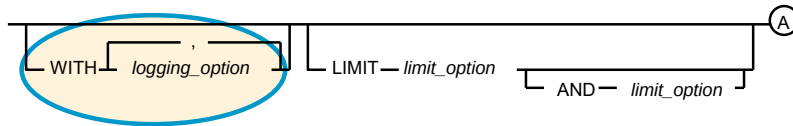
Default rows are stored in DBQLogTbl, the foundation of the DBQL feature. If you specify options that result in more detailed information, a default row is still generated in DBQLogTbl (except with the SUMMARY option or a query that completes within the limit specified with the THRESHOLD option), plus one or more additional logs are populated with one or more additional rows.

Examples of valid application names (Teradata 13.0 feature) include FASTLOAD, FASTEXP, MULTLOAD, ARC, etc. You can determine the application name used in the LogonSource string for a running application by querying DBC.SessionInfo as follows:

```
SELECT DISTINCT (LogonSource) FROM DBC.SessionInfo;
```

BEGIN QUERY LOGGING WITH ...

BEGIN QUERY LOGGING



WITH options

- **NONE** – turns off query logging for the specified user, account, or application name.
- **ALL** – logs the default row plus includes EXPLAIN, OBJECTS, SQL, and STEPINFO options.
- **EXPLAIN** – logs the default row plus the unformatted EXPLAIN text for the query in DBQLExplainTbl.
- **OBJECTS** – logs one row per target object per query in DBQLObjTbl plus default row in DBQLogTbl.
- **SQL** – logs the entire SQL for each request DBQLSqlTbl plus default row in DBQLogTbl.
- **STEPINFO** – inserts one row per step per query in DBQLStepTbl plus default row.

NONE - you use this option to turn off logging for any of the following items with NONE:

- account:user pair or account:user list
- application name or application name list
- user name list
- ALL:account name or account name list or ALL (which specifies all accounts)

ALL - logs all information for EXPLAIN, OBJECTS, SQL, & STEPINFO and the default row.

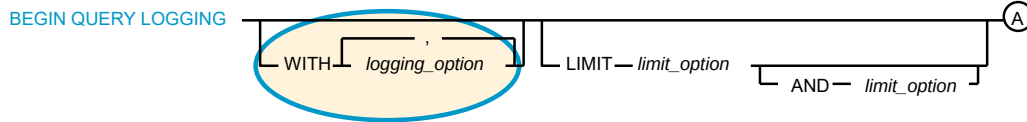
EXPLAIN - this option generates and logs the unformatted EXPLAIN text for each query. It does not generate EXPLAIN text for queries preceded by the EXPLAIN modifier. This option logs one or more rows into DBC.DBQLExplainTbl.

OBJECTS - object data is useful for analyzing queries that make heavy use of join indexes and indexed access. Inserts one row per target object per query in DBC.DBQLObjTbl.

SQL - this option logs the entire SQL statement in the DBC.DBQLSqlTbl table. Large statements can cause multiple rows to be written in order to log the full query text.

STEPINFO - use this option selectively. Although step data is useful for analyzing queries, this option can generate many rows. Inserts one row per step per query in the DBC.DBQLStepTbl.

BEGIN QUERY LOGGING WITH ... (cont.)



WITH options

- **STATSUSAGE** – logs statistics usage data and detailed query plan information for all DML and selected DDL requests as an XML document in DBQLXMLTbl.
- **XMLPLAN** – stores Optimizer query plan for the query in DBQLXMLTbl as an XML document.
- **USECOUNT** – logs Object Use Count (OUC) detail in the DD table ObjectUsage.
- **PARAMINFO** – captures values of the parameters for a parameterized query in DBQLParamTbl.
- **UTILITYINFO** – logs information for load, export, and Data Stream Architecture (DSA) utilities at the job level in DBQLUtilityTbl.
- **LOCK=*n*** – logs any Lock contention longer than *n* centiseconds in XML format into DBQLXMLLockTbl. Enhanced in 15.10 to include all Waiting Transactions, up to 30 transactions in each AMP.

XMLPLAN - logs the query plan generated by the Optimizer for all SQL DML requests as an XML document in system table DBC.DBQLXMLTbl. Because the XML document includes the query and step text, you generally do not need to specify the EXPLAIN and SQL options if you specify XMLPLAN. You should also specify a value of 0 for SQLTEXT to avoid redundant logging when you specify XMLPLAN. This option logs one or more rows into DBC.DBQLXMLTbl.

STATSUSAGE - logs statistics usage data and detailed query plan information for all DML and selected DDL requests as an XML document in DBC.DBQLXMLTbl. If you specify both XMLPLAN and STATSUSAGE, Teradata logs the collected data into a single integrated document.

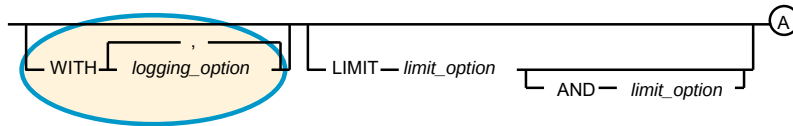
USECOUNT - improvements to Object Use Count (OUC) by adding support for statistics access usage and providing detailed usage context via Update, Delete, and Insert (UDI) object usage counts. Reduce risk of lock contention on popular dictionary tables, via new table DBC.ObjectUsage.

UTILITYINFO, PARAMINFO (15.0) - capture information for load, export, and Data Stream Architecture (DSA) utilities at the job level or capture parameter values in parameterized query requests.

LOCK=*n* (14.10, 15.10) - logs any Lock contention into DBQLXMLLockTbl

BEGIN QUERY LOGGING LIMIT ...

BEGIN QUERY LOGGING



LIMIT options

- **SQLTEXT** – specify the amount of SQL text to capture in the default row of DBQLogTbl. (Default is 200 char., 0 = off, max = 10,000 characters)
 - This option does NOT impact the SQL table if the WITH SQL option is used.
- **THRESHOLD** (May be used for short high-volume queries – example tactical queries)
 - Similar to SUMMARY, but default rows are generated in DBQLogTbl.
 - Threshold, in seconds, determines whether to log a query or just count it.
 - Query that complete <= threshold (sec.) are counted in DBQLSummaryTbl.
 - Query that complete > threshold (sec.), DBQL logs the default row.
- **SUMMARY** (used to simply count high volume queries or tactical queries – doesn't log default rows)
 - Counts queries; count is written in DBQLSummaryTbl every 10 min

SQLTEXT - use this option if you want to capture less than or more than the first 200 characters in the default row. The maximum limit is 10,000 characters. If you specify the option keyword but not a value, up to 10,000 characters are logged in DBQLogTbl.

To store the complete statement regardless of length, specify the SQL option, as many rows as needed to contain the full text will be logged in DBQLSqlTbl. If you do this, define LIMIT SQLTEXT=0 to avoid redundant logging in both the default row and DBQLSqlTbl.

SUMMARY - SUMMARY is useful for tracking voluminous short queries, such as for tactical queries, because it does not grow the DBQLogTbl. It simply counts queries based on specified time differentials in DBQLSummaryTbl. The SUMMARY option is unique in that it:

- Does not generate default rows in DBQLogTbl
- Summary information is flushed at system-controlled intervals of 10 minutes
- If no data has been collected for a summary category in a 10-minute interval, no rows will be written for it.

THRESHOLD - THRESHOLD also is useful for short, high-volume queries, but in addition to incrementing a count for qualifying queries, THRESHOLD logs a default row for any query that exceeds the specified time. This enables you examine the processing timestamps and the query structure. You can combine THRESHOLD with SQLTEXT if you want to capture more than the first 200 characters of a query that runs longer than THRESHOLD seconds for identification of the longer running queries.

BEGIN QUERY LOGGING Examples

```
BEGIN QUERY LOGGING WITH SQL, OBJECTS ON ALL;
```

- Creates a system rule to log default query, SQL, and OBJECT information for all users/accounts.

```
BEGIN QUERY LOGGING WITH SQL, OBJECTS ON ALL;
BEGIN QUERY LOGGING WITH NONE          ON tacticaluser1, tacticaluser2;
```

- Creates a system rule to log default query, SQL, and OBJECT information for all users/accounts.
- Creates two additional rules to disable query logging for tacticaluser1 and tacticaluser2.

```
BEGIN QUERY LOGGING ON tfact03 ACCOUNT = ('$L00EDUC&D&H', '$M00EDUC&D&H');
```

- Creates two rules for a specific user – each rule identifies a specific account id.

```
BEGIN QUERY LOGGING ON APPLNAME = ('MULTLOAD', 'TPTUPD');
```

- Creates a rule to enable logging for any MultiLoad or TPT Update job. (no 'I' in MULTLOAD)

```
BEGIN QUERY LOGGING WITH SQL, OBJECTS ON ALL;
BEGIN QUERY LOGGING WITH NONE          ON APPLNAME = ('FASTLOAD', 'TPTLOAD');
```

- Creates a rule to log default query information and full SQL on all users and accounts.
- Creates a rule to disable logging for any FastLoad or TPT Load job.

Hierarchy of Applying Database Query Logging Rules

Database Query Logging works from a hierarchical foundation that allows BEGIN QUERY LOGGING requests to be submitted for individual users even if a rule exists for ALL users.

However, if a rule exists for a specific account:user pair, you must submit an appropriate END QUERY LOGGING request to delete the rule before you can issue a new rule for that account:user pair. Teradata Database applies the rules in the following order:

Order	Type of Rule
1	A rule based on an application name.
2	A rule for this specific user and specific account.
3	A rule for this specific user and any account.
4	A rule for all users and this specific account.
5	A rule for all users and any account.

DBQL first searches for a rule based on an application name. If no such rule exists, DBQL then looks for a rule specific to the user and account, and so on down the hierarchy.

For example, you can submit a BEGIN QUERY LOGGING request for default logging on ALL users, and DBQL can also be enabled for *user1* with objects and steps. If *user1* logs on and executes queries, DBQL collects objects and steps. When users other than *user1* log on and execute queries, DBQL only logs default row information for them.

BEGIN QUERY LOGGING Examples (cont.)

```
BEGIN QUERY LOGGING WITH SQL, OBJECTS LIMIT SQLTEXT=0 ON ALL;
```

- Sets a system rule to log default query, SQL, and OBJECT information for all users/accounts.
- This example captures full SQL text in the SQL table, but does not capture the default first 200 characters of a query in the default row.

```
BEGIN QUERY LOGGING WITH SQL,OBJECTS          ON ALL;  
BEGIN QUERY LOGGING WITH OBJECTS, SQL, EXPLAIN ON adhoc01,adhoc02;
```

- Default rows are logged as well as complete SQL text and objects used in queries for all users.
- The default row has the first 200 bytes of SQL text.
- For adhoc01 and adhoc02 users, Explain text is also captured.

```
BEGIN QUERY LOGGING WITH SQL, OBJECTS LIMIT THRESHOLD=60 ON ACCOUNT= 'acctname';
```

- If a query runs for less than or equal to 60 seconds of wall clock time, increment the count.
- If a query runs longer than 1 minute, log a default row.

```
BEGIN QUERY LOGGING LIMIT SUMMARY = 5, 60, 600 ON ALL;
```

- Default Summary option only counts running queries based on elapsed time. 3 values (in sec.) are required. 4 count intervals are logged (<=5, <=60, <=600, >600)
- Summary limit cannot be used with any other limits.

In these examples, the ON option can specify users, account ids, or application names.

SQLTEXT: option to control the number of SQL statement characters to log

- 200 characters of SQL logged in default row
- SQLTEXT values range from 0 to 10,000
- "SQLTEXT" without a value logs 10,000 characters

THRESHOLD: option to limit the queries logged by elapsed time

- THRESHOLD values are specified in seconds.
- "THRESHOLD" without a value results in a 5 second value.
- Queries greater than Threshold value generate a default row.
- Maximum THRESHOLD value is 32,767 seconds.

Notes about Threshold logging

- You can use THRESHOLD logging along with the SQL, STEPINFO, and EXPLAIN options.
- With threshold logging, DBQL cannot log to separate Explain and XML tables, even for those queries taking longer than the specified criteria. SQL, STEPINFO, and OBJECTS can be logged during threshold logging, even for those queries taking longer than the specified clock seconds.
- By filtering logging with THRESHOLD will lower the performance overhead of detailed logging of unnecessary queries.
- For example, log queries that run > 1 CPU second into DBQLLogTbl, DBQLSqlTbl, DBQLStepTbl and DBQLObjTbl and summarize all other queries in the DBQLSummaryTbl:

BEGIN QUERY LOGGING Examples (cont.)

The THRESHOLD and SUMMARY Limit options include:

- **ELAPSEDSEC (default)** – values are in units of 1.0 seconds; counts the number of requests that fall into elapsed wall-clock time intervals
- **ELAPSEDTIME** – values are specified in units of 0.01 sec – provides finer granularity than ELAPSEDSEC
- **CPUTIME** – uses actual CPU time to set ranges and summarize – represented in units of 0.01 sec.
- **CPUTIMENORM** – similar to CPUTIME, but with normalized time intervals (use with coexistence systems)
- **IOCOUNT** – uses the I/O count for the query to set ranges and to summarize

```
BEGIN QUERY LOGGING WITH SQL, OBJECTS LIMIT THRESHOLD = 100 CPUTIME ON ...;
```

- If a query uses less than or equal to 1 CPU second, increment the count in the Summary table.
- If a query uses more than 1 CPU second, log a default row, SQL text, and OBJECT details.

```
BEGIN QUERY LOGGING LIMIT SUMMARY = 50,100,200 CPUTIME ON ACCOUNT='$H00TACT&D&H';
```

- For tactical users assigned to this account, this Summary example only counts queries based on actual CPU time.
- 50 is .5 CPU second, 100 is 1 CPU second, and 200 is 2 CPU seconds. 3 values are required. 4 count intervals are logged (<=50, <=100, <=200, >200)
- Summary limit cannot be used with any other limits.

The SUMMARY and THRESHOLD limits have additional options. The default for each of these limits is elapsed time (ELAPSEDSEC).

Options for both SUMMARY and THRESHOLD

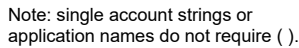
ELAPSEDSEC (default) – use this option to set ranges and to summarize counts of the number of requests that fall into an elapsed time interval. This value is expressed in units of 1.00 seconds.

ELAPSEDTIME – use this option to set ranges and to summarize counts of the number of requests that fall into an elapsed time interval. The value is expressed in units of 0.01 seconds so it provides finer granularity for elapsed time than ELAPSEDSEC.

CPUTIME – use the CPU time for the query to set ranges and to summarize. The value is in units of 0.01 second. For example, if you specify 500 for one of the intervals, then the value used to make the determination is 5 CPU seconds.

CPUTIMENORM – use this option to set ranges and to summarize counts of the number of requests that fall into a normalized CPU time interval. This option is designed for use with coexistence systems to aid in managing mixed nodes more efficiently, but it can be used with any system. The value is expressed in units of 0.01 second.

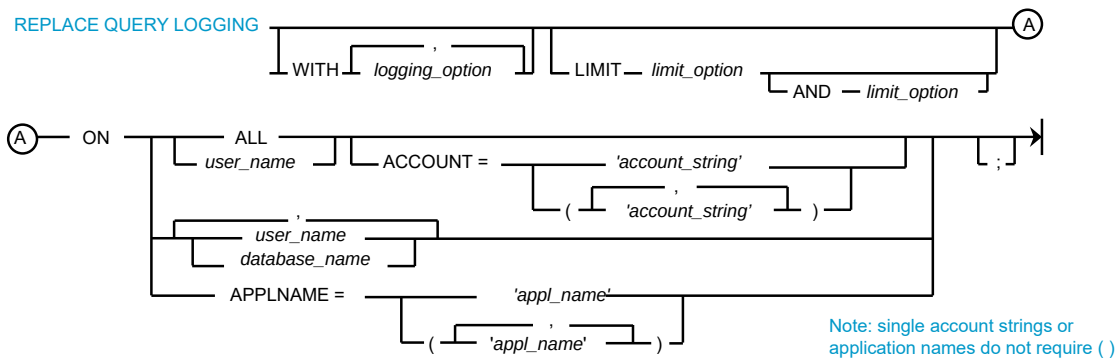
IOCOUNT – use the I/O count for the query to set ranges and to summarize.



- | | |
|---|---|
| END QUERY LOGGING ON ALL RULES; | Removes all rules. |
| END QUERY LOGGING ON tfact01; | (You can end logging for a specific user.) |
| END QUERY LOGGING ON tfact03 ACCOUNT='\$L00EDUC&D&H'; | (You can end logging for a specific account of a user.) |

- The data is committed immediately
- One or more DBQL rows are written (but may be incomplete); the cache is flushed
- Subsequent queries during that session are not logged

REPLACE QUERY LOGGING



- A `REPLACE QUERY LOGGING` (13.10) statement has the same options as the `BEGIN QUERY LOGGING` statement.
 - Replaces an existing rule. If no rule exists, it will be created.
- This statement allows you to replace a logging rule set without using two statements: `END QUERY LOGGING` and `BEGIN QUERY LOGGING`.
 - This will help prevent missing logging data between the end logging and begin logging sequence of statements.
 - This statement also avoids the unnecessary flushing of DBQL caches.

The `REPLACE QUERY LOGGING` statement allows the customer to modify their query logging for active users without having to end query logging and begin query logging. This will help prevent missing logging data between the end logging and begin logging sequence of statements.

This statement can also be used to avoid the end query logging and begin query logging statement pairs and unnecessary flushing of DBQL caches.

The `REPLACE QUERY LOGGING` statement will replace an existing rule or will create a new rule if one did not exist.

Additional Enhancement

An additional enhancement is that `TypeOfUse` column in the `DBQLObjTbl` table is populated.

The `TypeOfUse` column contains the following numeric values:

- 1 = Found in the resolver
- 2 = Accessed during query processing
- 4 = Found in a conditional context
- 8 = Found in inner join condition
- 16 = Found in outer join condition
- 32 = Found in a sum node
- 64 = Found in a full outer join condition

FLUSH QUERY LOGGING

FLUSH QUERY LOGGING WITH *flush_option*;

The *flush_option* feature in which query logging or Teradata database workload management cache is flushed to disk. Options include:

(ALL, ALLDBQL, ALLTDWM, DEFAULT, EXPLAIN, LOCK, OBJECTS, SQL, STATUSAGE, STEPINFO, SUMMARY, TDWMEVENT, TDWMEXCEPTION, TDWMHISTORY or XMLPLAN)

Note: This SQL statement is only valid in Teradata session mode and requires the EXECUTE privilege on DBC.DBQLAccessMacro.

Example:

- To flush all the caches of DBQL logs, but not flush any of the TDWM caches.

FLUSH QUERY LOGGING WITH ALLDBQL;

To flush one, several, or all database query log (DBQL) caches or Teradata Database Workload Management (TDWM) caches to disk. Note: This SQL statement is only valid in Teradata session mode and requires the EXECUTE privilege on DBC.DBQLAccessMacro.

```
FLUSH QUERY LOGGING WITH flush_option;
```

Where *flush_option* (14.10 feature) is which database query logging or Teradata database workload management cache to flush: (ALL, ALLDBQL, ALLTDWM, DEFAULT, EXPLAIN, LOCK, OBJECTS, SQL, STATUSAGE, STEPINFO, SUMMARY, TDWMEVENT, TDWMEXCEPTION, TDWMHISTORY or XMLPLAN).

- ALL is mutually exclusive to all other options.
- ALLDBQL is mutually exclusive to all other DBQL options.
- ALLTDWM is mutually exclusive to all other TDWM options.

Examples

This example flushes the caches for all DBQL and TDWM logs.

```
FLUSH QUERY LOGGING WITH ALL;
```

This example flushes all the caches of DBQL logs but does not flush any of the TDWM caches.

```
FLUSH QUERY LOGGING WITH ALLDBQL;
```

PARAMINFO, UTILITYINFO Options

DBQL Logging **WITH PARAMINFO**

- Logs parameter information along with values into the DBQL table named DBC.DBQLParamTbl.
- Holds all necessary parameter data in a BLOB column
- DBC View, DBC.QryLogParamV can be used to access the information in the DBQL parameter table.

SELECT QueryID, RowNum, ParamJSON from QryLogParamV; *(converts all the rows in DBQLParamTbl to JSON documents)*

DBQL Logging **WITH UTILITYINFO**

- Each utility job logs one row to DBC.DBQLUtilityTbl.
- Captures information for the following utilities at the **job** level:
 - **FastLoad Protocol**: FastLoad, TPT Load, and JDBC FastLoad
 - **MLOAD Protocol**: MultiLoad and TPT Update
 - **MLOADX Protocol**: TPT Update operator
 - **FastExport Protocol**: FastExport, TPT Export, and JDBC FastExport
 - **Data Stream Architecture (DSA)**
- Note: This feature does not support the legacy ARCMAN utility.
- Initial infrastructure for long term goal of TDWM management by SLG.

Both PARAMINFO and UTILITYINFO option are 15.0 features. A parameterized query uses placeholders for parameters, and the parameter values are supplied in a separate statement at execution time. Because this type of query distinguishes between code and data, it prevents attackers from changing the intent of a query by inserting SQL commands.

Show Parameters logs the values for parameterized queries in a new 15.0 Data Dictionary table, DBC.DBQLParamTbl. For each parameter, DBC.DBQLParamTbl logs the:
Name, Type, Position, Value

Values can have these data types:

- Standard Teradata data types and their ANSI equivalents
- BLOB/CLOB of up to 64 KB
- UDTs

DBQL - Show Parameters is off by default, but can be enabled with the PARAMINFO option for the BEGIN/REPLACE QUERY LOGGING statements. For example:

Begin Query Logging with PARAMINFO on user01;

DBQL Lock Logging Enhancement / PLL

DBQL Logging **WITH LOCK=*n***

- The original 14.10 feature limits the description of the Lock Contention to only 2 parties, the Blocking and Blocked Transactions. This 15.10 feature expands the Blocked party to include all Waiting Transactions, up to 30 transactions in each AMP.

PLL incorporated

- Incorporating Partition Level Lock (PLL) feature (15.10) into DBQL Lock Logger.
- New entities in lock contentions
 - Granter's lock kind
 - Waiter's lock kind
 - Waiter's Start Partition number, Internal Partition Number (IPN),
 - Waiter's End Partition number (IPN)
 - Waiter's Start RowHash value
 - Waiter's End RowHash value Partition range involved in Lock request
 - Rowhash involved in the Lock request
 - 2 byte and 8 byte Partition Numbers

Note: Granter is the blocker session Waiter is the blocked session

This 15.10 feature expands the Blocked party to include all Waiting Transactions, up to 30 transactions in each AMP.

There is a need to clarify the 2631 error as well as things like Deadlocks and Timeouts using actual Transactional/Session Data from the users. The goal is not necessarily to substitute for the error 2631 or other Contention error/failure. By the time we get into those situations it would have been too late. Rather, this feature supplements those errors. This additional information could be used to forecast and/or explain the length of lock waiting time, deadlocks and timeout due to Database Lock contention.

PLL incorporated

This enhancement is an extension of the original DBQL Lock Logging Feature, first introduced in Teradata Database 14.10, by now incorporating Partition Level Lock (PLL) feature (Teradata Database 15.10) into DBQL Lock Logger.

PLL provides the ability to apply a lock at the partition level on the database object. The aim of PLL is to reduce lock contention & thus improve the performance of the partitioned tables. PLL locks will be reflected in both new DBQL Lock Logger and the old Dumplocklog utility and needs to be properly displayed to differentiate from the older lock types.

New PLL Kinds

DBQL lock logger has been upgraded to support the new PLL types: RowHashAllParts, RowKey, RowKeyRg (depict a RowKey range), RowHashPartRg (a rowhash and a partition range), PartRg is a partition range, RowKeyRg, RowKeyRg2, & RowKeyRg3.

DBQLRules View

[DBC.DBQLRules\[V\]](#) views – return information about current query logging rules.

UserName	AccountString	AppName	TypeofRule	ExplainFlag	ObjFlag
SqlFlag	StepFlag	XMLPlanFLag	SummaryFlag	ThresholdFlag	TextSizeLimit
SummaryVal1	SummaryVal2	SummaryVal3	ThreshValue	TypeofCriterion	

Example:

```
SELECT      UserName      (CHAR (8)) AS "User"
           ,AccountString (CHAR (8)) AS "Acct_ID"
           ,SqlFlag       AS "Sql", TextSizeLimit AS "Size"
           ,ThresholdFlag AS "T_Flag", SummaryFlag AS "S_Flag"
           ,TypeofCriterion AS "Type"
           ,SummaryVal1   AS "V1" ,SummaryVal2 AS "V2"
           ,SummaryVal3   AS "V3"
FROM        DBC.DBQLRulesV
ORDER BY 1;
```

Notes:

BEGIN QUERY LOGGING statements are in student manual.

Type of Criterion:

- 0 – Elapsed (1.0 sec)
- 1 – CPUTIME
- 2 – IOCOUNT
- 3 – CPUTIMENORM
- 4 – Elapsed (0.01 sec)

User	Acct_ID	Sql	Size	T_Flag	S_Flag	Type	V1	V2	V3
All	\$H	F	200	F	F	?	?	?	?
tfact01		F	500	F	F	?	?	?	?
tfact02		F	500	F	F	?	?	?	?
tfact03	\$M_&D&H	F	200	F	F	?	?	?	?
tfact04		F	200	F	T	0	5	30	60
tfact05		F	200	T	F	1	200	?	?
tfact06		T	0	F	F	?	?	?	?

The BEGIN QUERY LOGGING statements that generated the rules on this slide are as follows:

```
BEGIN QUERY LOGGING ON ALL ACCOUNT = ('$H');
BEGIN QUERY LOGGING LIMIT SQLTEXT=500 ON tfact01, tfact02;
BEGIN QUERY LOGGING ON tfact03 ACCOUNT = ('$M_&D&H');
BEGIN QUERY LOGGING LIMIT SUMMARY = 5, 30, 60 ON tfact04;
BEGIN QUERY LOGGING LIMIT THRESHOLD = 200 CPUTIME ON tfact05;
BEGIN QUERY LOGGING WITH SQL LIMIT SQLTEXT=0 ON tfact06;
```

The END QUERY LOGGING statements that correspond to these BEGIN QUERY LOGGING statements are as follows:

```
END QUERY LOGGING ON ALL ACCOUNT = ('$H');
END QUERY LOGGING ON tfact01, tfact02;
END QUERY LOGGING ON tfact03 ACCOUNT = ('$M_&D&H');
END QUERY LOGGING ON tfact04;
END QUERY LOGGING ON tfact05;
END QUERY LOGGING ON tfact06;
```

Note: Simply replacing the BEGIN with END will also remove the rules.

DBC.QryLog View – Example

DBC.QryLog[V] views – return information about default rows in the DBQLogTbl.

Query to return the default row with a QueryID of 307191272404571608.

```
SELECT ProcID, SessionID, UserName, QueryID, UserID, AcctString,
       ExpandAcctString, ClientID, ClientAddr, StartTime, FirstStepTime,
       ElapsedTime, CacheFlag, NumResultRows, SpoolUsage, QueryText
FROM   DBC.QryLogV
WHERE  QueryID = 307191272404571608;
```

ProcID	30719
SessionID	977674
UserName	student240
QueryID	307191272404571608
UserID	00007B04
AcctString	\$M00EDUC&D&H
ExpandAcctString	\$M00EDUC16121514
ClientID	LC130182
ClientAddr	153.65.212.205
StartTime	12/15/2016 14:29:38
FirstStepTime	12/15/2016 14:29:38
ElapsedTime	0:03:52.120000
CacheFlag	/* blank indicates query is not cached */
NumResultRows	390,578
SpoolUsage	148,558,336
QueryText	SELECT D.DeptName, E.EmpNumber, E.LastName, E.FirstName, ...

Shown in BTEQ with
SIDETITLES and FOLDLINE on.

This slide contains an example of using the DBC.QryLog[V] view.

One of the columns is the CacheFlag column. This column can be one of the following:

- Blank indicates the query is not found in step cache.
- “S” if the query is a parameterized query and a Specific Plan is generated.
- “G” if the query is a parameterized query and a Generic Plan is generated.
- “T” if the query is found in step cache.
- “A” if the query is a parameterized query and a SpecAlways decision is taken. That is, each time a query submitted, the USING values are peeked at and the query is parsed.

The first time a parameterized request is seen, the parser peeks at the values in the USING clause, and a plan specific to those values is produced. This will cause the cache flag for that query's execution to be set to 'S' for 'specific plan'. At this point in time, no plan is actually cached for re-use, because a value-specific plan was produced.

After this first 'specific plan' execution is complete, metrics from the execution, as well as the estimates.

Assuming the same parameterized request is repetitively executed on the same PE, a common pattern that may occur, is a cache flag sequence of 'S', 'G', 'T', 'T', 'T'...

The first time the query is seen, a specific plan is produced ('S'), the second time a generic plan ('G'), and from that point on, the same generic plan is executed ('T') until that plan is flushed from the cache.

QryLogSummary View – Example

Returns information about summary rows in the DBQLSummaryTbl.

This example is based on the summary rule:

```
BEGIN QUERY LOGGING LIMIT SUMMARY = 5, 30, 60 ON tfact04;
```

```
SELECT UserID, CollectTimeStamp, QueryCount,
       QuerySeconds, LowHist, HighHist
FROM   DBC.QryLogSummaryV
ORDER BY 2, 3, 6;
```

For #2, the average time of queries can be calculated:

```
407 / 1060 = .38
748 / 41 = 18.24
351 / 8 = 43.88
2478 / 3 = 826.00
```

Result:

UserID	CollectTimeStamp	QueryCount	QuerySeconds	LowHist	HighHist
00003305	2014-09-12 19:32:06	60	18	0	5
00003305	2014-09-12 19:32:06	2	27	5	30
00003305	2014-09-12 19:42:06	1060	407	0	5
00003305	2014-09-12 19:42:06	41	748	5	30
00003305	2014-09-12 19:42:06	8	351	30	60
00003305	2014-09-12 19:42:06	3	2478	60	32767
00003305	2014-09-12 19:52:06	258	80	0	5
00003305	2014-09-12 19:52:06	16	251	5	30
00003305	2014-09-12 19:52:06	1	821	60	32767

1 – In this summary collection, no queries were executed that exceeded 30 seconds.

2 – In this summary collection, queries were executed in all 4 summary intervals.

3 – In this summary collection, no queries were executed that ran between 30 and 60 seconds.

An example of the output from the DBC.QryLogSummary[V is provided on this slide. This summary data was collected for the tfact04 user who had established multiple sessions on the system. The summary rule was created as follows:

```
BEGIN QUERY LOGGING LIMIT SUMMARY = 5, 30, 60 ON tfact04;
```

Summary

There are two logging facilities available to the database and/or security administrator.

- [Access Logging Facility](#)
 - Used for access and security audit analysis
- [Query Logging Facility \(DBQL\)](#)
 - Used for query activity and workload analysis

[Additional DBQL Enhancements:](#)

- Macros, views, triggers, stored procedures, and User-DefinedFunctions (UDFs) are logged in DBQLObjTbl if the WITH OBJECTS option is used.
- By querying DBQLObjTbl information, Database Administrators (DBAs) are able to see which views and macros users access.
- If the WITH OBJECTS option is used, FastLoad and MultiLoad target tables are also logged in the DBQLObjTbl.

This slide summarizes some important concepts regarding this module.



Module 11: Review Questions

1. In order to use the BEGIN/END LOGGING commands, what is the name of the system macro you need execute permission on? _____
2. How is this macro initially created? _____
3. What is a negative impact of the following statement?
BEGIN LOGGING WITH TEXT ON EACH
4. With DBQL, what is the size of the default text captured for queries? _____
5. True or False: With DBQL, the LIMIT SUMMARY option cannot be used with any other LIMIT.
6. True or False: With DBQL, the WITH SQL option only captures a maximum of 10,000 characters.
7. True or False: With DBQL, the option WITH ALL ON ALL is typically a good choice.
8. True or False: With DBQL, default rows are logged in the DBC.DBQLLogTbl.

Check your understanding of the concepts discussed in this module by completing the review questions as directed by your instructor.



Lab Exercise 11-1 (1 of 2)

Purpose

In this lab, you will use the views in the data dictionary to learn about security defaults and invalid logons in the system.

Tasks

1. What system security defaults are in effect for your system?

Number of days to expire password: _____
Minimum number of characters required: _____
Maximum number of characters required: _____
Are digits allowed? Yes _____ No _____
Are special characters allowed? Yes _____ No _____
Restrict words in passwords? Yes _____ No _____
Maximum failed logons permitted (0=never lock): _____
Minutes to elapse before unlocking: _____
Days to expire before password reuse: _____

2. Is a Profile in effect for your username? If so, what is the name of your Profile? _____

3. If a Profile is being used, which attributes in the Profile override the system security defaults?

Check your understanding of the concepts discussed in this module by completing the lab exercises as directed by your instructor.

Hints:

- To list your current profile, simply execute `SELECT PROFILE;`
- To view your profile attributes, select from the view named `DBC.ProfileInfoVX;`



Lab Exercise 11-1 (2 of 2)

4. Using the DBC.LogOnOffV view, list the "BAD" logon attempts on your system that have occurred during the last 2 weeks. Qualify the SELECT using LIKE 'BAD%'.

Number of Bad Logons (System) _____

Number of Bad Logons (Your UserName) _____

5. Using the DBC.SessionInfoV, list the sessions currently logged on your system.

Total number of Sessions (System) _____

Total number of sessions (Your UserName) _____

This slide continues this lab exercise.

Hint: Use the view named DBC.LogOnOffV and use the Event column to select 'Bad%' logons.



Lab Exercise 11-2 (1 of 3)

Purpose

In this lab, list the access and query logging rules that are being used and the entries in these logs for your username.

Tasks

1. Using the DBC.AccLogRulesV view, how many access logging rules are there? Count = _____

Using the DBC.AccLogRulesV view, complete the following access log information for macros – creating, dropping, and execution of macros.

<u>Username</u>	<u>Databasename</u>	<u>TVMName</u>	<u>CMC</u>	<u>DMC</u>	<u>EXE</u>
_____	_____	_____	_____	_____	_____
_____	_____	_____	_____	_____	_____
_____	_____	_____	_____	_____	_____
_____	_____	_____	_____	_____	_____
_____	_____	_____	_____	_____	_____

Check your understanding of the concepts discussed in this module by completing the lab exercises as directed by your instructor.



Lab Exercise 11-2 (2 of 3)

2. Execute the following statement.

```
CREATE DATABASE Testxxx FROM DBC AS PERM=0; (xxx is your student number)
```

(This command should fail – 3524: The user does not have CREATE DATABASE access to database DBC.)

3. Using the DBC.AccessLogV view, list the access log entries for the last 2 weeks for your username.

How many entries are in this log have been granted? _____

How many entries are in this log have been denied? _____

This slide continues this lab exercise.



Lab Exercise 11-2 (3 of 3)

4. Using the DBC.DBQLRulesV view, how many query logging rules are there? _____

Using the DBC.DBQLRulesV view, list the query log rules that are in effect for this system.

<u>Username</u>	<u>Application Name</u>	<u>TypeofRule</u>	<u>Explain</u>	<u>Object</u>	<u>SQL</u>	<u>Object UsageA</u>
_____	_____	_____	_____	_____	_____	_____
_____	_____	_____	_____	_____	_____	_____
_____	_____	_____	_____	_____	_____	_____
_____	_____	_____	_____	_____	_____	_____
_____	_____	_____	_____	_____	_____	_____

5. Using the DBC.QryLogV view, list the logged queries for your username and how many are there?

Count = _____

Using this view, how many queries are logged for all of the users with usernames like 'student%'?

Count = _____

This slide continues this lab exercise.

If threshold and/or summary log rules are used and it is necessary to determine the number of queries executed by a user, the following query can be used.

```
SELECT      SUM(S.QueryCount)
FROM        DBC.QryLogSummaryV S
INNER JOIN  SELECT  UserID
            FROM    DBC.QryLogV
            WHERE   UserName LIKE 'studentxxx%'
            GROUP BY 1) Q
ON          S.UserID = Q.UserID;
```




Module 12: Workload Management

Teradata Vantage Administration

Copyright © 2007–2023 by Teradata. All Rights Reserved.

Objectives

After completing this module, you will be able to:

- Describe the type of workload management rule to apply to limit certain kinds of queries.
- List the query attributes that are used to classify a query into a Workload Definition.
- Specify the exception actions that are possible for a workload definition.
- Place the workload management control options in the proper sequence as they are acted upon by Teradata software.

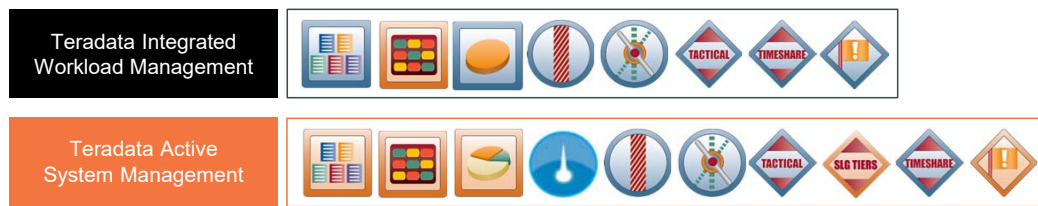
What is Workload Management?

- Goal-oriented, workload-centric, automated management of mixed workloads
- Provides for consistency of response times and throughput for high priority workloads
- Define a “Workload” for each type of work
 - Workloads are linked to priorities and concurrency limits
 - Service Level Goals may be defined on specific workloads
- Classify queries to workloads based on characteristics and resource consumption
- Automated exception handling
 - Queries that are running in an inappropriate manner can be automatically detected and managed
- Graphical Reporting allows you to monitor the workload arrival rates and the service level provided

The Workload infrastructure is a Goal-Oriented, Automatic Management and Advisement technology in support of performance tuning, workload management, capacity planning, configuration and system health management. Workload Management features greatly improves system management capabilities, with a key focus being to reduce the effort required by DBAs, application developers and support engineers through automation. In addition, Workload Management provides many more system management monitoring and analysis capabilities than previously available to Teradata users. Business-driven Service Level Goals can be specified and monitored against for a quick-and-easy evaluation of performance when using the new monitoring capabilities. Users of Workload Management will realize improved response time consistency and predictability of their workloads.

Workload Management Benefits

- Support business operations priority decisions
- Stabilize response times of the critical work
- Increase throughput
- Protect known, proven work from impact by unknown, adhoc unpredictable queries
- Give priority to proven, good performing queries
- Automatically manage prioritization based on processing periods or system health conditions



Benefits of Workload Management include:

- Support business operations priority decisions
- Stabilize response times of the critical work
- Increase throughput
- Protect known, proven work from impact by unknown, adhoc unpredictable queries
- Give priority to proven, good performing queries
- Automatically manage prioritization based on processing periods or system health conditions

Workload Management

What is Integrated Workload Management (IWM)?

- IWM consists of several tools to assist the DBA in defining the workload environment and rules to control the allocation of resources to workloads running on a system.
- IWM supports relatively high levels of concurrency, mixed workloads, and basic tactical work.

What is Teradata Active System Management (TASM)?

- TASM provides **additional workload management capabilities** beyond those provided by IWM.
- TASM is available with Enterprise Data Warehouse systems (e.g., 6000 platforms).

What are examples of rules that can be created?

- Rules include filters, throttles, workload definitions, as well as exceptions.
 - **Workload definitions are rules to control the allocation of resources to workloads.**

The key products that are used to create, manage, and monitor the rules/workloads are:

- Teradata Viewpoint Workload Designer, Workload Monitor and Workload Health
- Teradata Workload Analyzer

Integrated Workload Management (IWM) is made up of several products/tools that assist the DBA or application developer in defining and refining the rules that control the allocation of resources to workloads running on a system. These rules include filters, throttles, workload definitions, and exceptions.

Teradata Active System Management (TASM) provides additional capabilities above and beyond those provided with IWM. The capabilities of TASM have been greatly expanded to enhance the Integrated and Active data warehouse missions of these EDW platforms (e.g., IntelliFlex™).

Rules to control the allocation of resources to workloads are effectively represented as workload definitions. Tools are also provided to monitor workloads in real time and to produce historical reports of resource utilization by workloads. By analyzing this information, workload definitions can be adjusted to improve the allocation of resources.

The key products to implement “workloads” are:

- Viewpoint Workload Designer
- Teradata Workload Analyzer (TWA)

Workload Designer is a key supporting product component to create the rules. Functions include:

- Define Filters and Throttles
- Define Workloads
- Define general controls

Workload Management Offering Comparison

Key Features	Teradata IWM	TASM
 Workload Classification	Source, Target, Query Characteristics, QueryBand, Utility	Source, Target, Query Characteristics, QueryBand, Utility
 Virtual Partitions	One Partition	Multiple Virtual Partitions
 Prioritization	Tactical and Timeshare	Tactical, SLG Tiers , and Timeshare
 Resource Management	CPU and I/O	CPU and I/O
 Meters, Filters & Throttles	Filters, Workload and System Throttles; Flex Throttles	Filters, Arrival Rate Meters , Workload and System Throttles; Flex Throttles
 State Matrix	Planned Environments	by Planned Environment and by Health Conditions
 Exceptions	Tactical and Timeshare Decay	Tactical, Timeshare Decay, and Workload Exceptions

Teradata offers Workload Management bundled with all the platforms and it is called Teradata Integrated Workload Management. Teradata also offers advanced Workload Management on some platforms and it is called TASM.

With release 14.0/SLES11 all Teradata Platforms (that support Teradata 14.0 and beyond on SLES 11) will include “Teradata Integrated Workload Management” and TASM will be available on some platforms. See the Workload Management Support Matrix in the TASM 14.0 OCI for details.

This chart highlights the differences between the offerings for release 14.0 for SLES11. The key new features for SLES11 are highlighted in bold:

- All platforms now get full workload classification
- All platforms default to one virtual partition and TASM offers up to 10 virtual partitions
- Prioritization methods have been enhanced with all platforms utilizing Tactical and Timeshare methods while TASM adding SLG Tiers method
- All platforms utilize now also utilize workload throttles in addition to system throttles and filters
- TASM offers sophisticated operating period and health conditions state matrix
- Finally, all platforms utilize Tactical exceptions and Timeshare Decay and TASM adds Workload Exceptions

Workload Management Capabilities

Both IWM and TASM provide **Query Management (QM)** capabilities.

- A set of user-defined “**rules (or filters and throttles)**” is created to determine whether logon and query requests will be accepted by the Teradata Database.
 - These rules also determine whether the execution of some query requests should be “delayed” (internally queued).
- Query Management provides “non-workload” **filters and throttles**:
 - **Filters** – object access and query resource rules used to **reject** queries
 - **Throttles** – object and load utility rules used to **delay** or **reject** queries

Both IWM and TASM provide **Workload Management (WM)** capabilities.

- A set of user-defined “**workload definitions**” is created to control the allocation of resources to workloads.
- Queries are associated with a “workload” based on who, where, and what criteria.

Viewpoint Workload Designer provides a graphical user interface (GUI) for creating rules that manage database access, increase database efficiency, and enhance workload capacity.

What is Query Management?

- Query Management is a set of “rules” to determine whether logon and query requests will be accepted by Teradata, and further to determine whether the execution of some query requests should be “delayed” (internally queued). The purpose of “delaying” queries is to limit the number of database resources that are tied up in processing low priority and/or long running queries. Queries that are delayed are still perceived as executing within the user’s session.

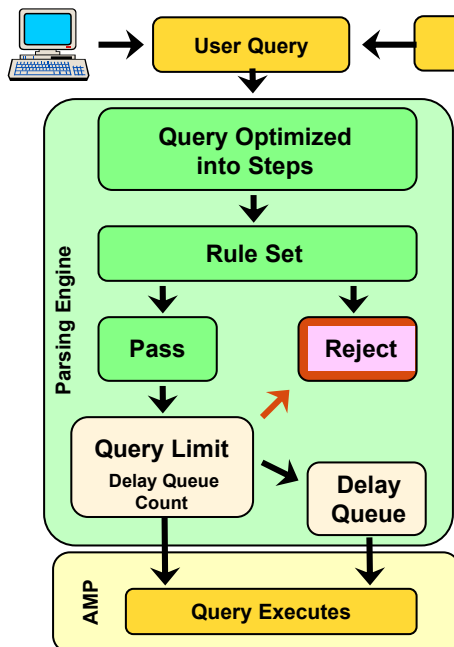
What is Workload Management?

- Workload management on a system yields improved workload distribution and customized delegation of resources among the various workloads. A workload represents a portion of the queries that are running on a system. To use workload management in Teradata, a set of workload definitions must be established and enabled.
- A Workload Definition (WD) is a workload grouping and its operating rules to assist in managing queries. The requests that belong to the same workload will share the same resource priority and exception conditions.

Why use a Workload Management facility?

- Assign queries to the correct workload before they start executing. Fix and prevent problems before they happen. Seamlessly and automatically manage resource allocation; removes the need for constant setup and adjustment as workload conditions change.

Query Management Architecture



User logs on a session with an Account ID and submits a query.

- A User and/or account may be associated with one or more Query Management Rules.
- Rules-checking happens in the PE (Dispatcher) before the query begins to execute.
- Queries that do not pass Filter Rules are rejected.
- Queries that pass Throttle Rules can also be delayed or rejected if the query would exceed a query limit.
- Delayed queries are managed by Query Workload Manager.
- Queries from all client sources are checked for compliance to Query Management Rules.

The rules you create are stored in tables in the Teradata database. Unless otherwise specified, every logon and every query in every Teradata database session is checked against the enabled rules.

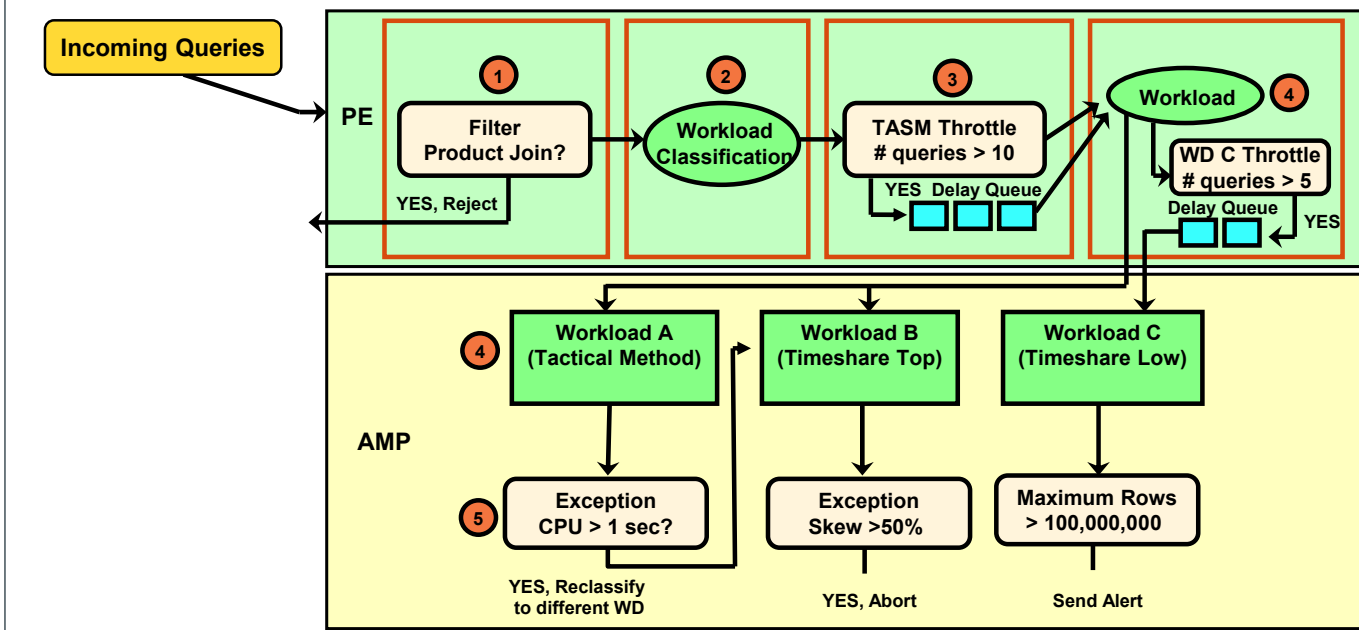
Workload Management rules are loaded into the Dispatcher components of the Teradata database. When a Teradata client application issues a request to the Teradata Database, the request is examined and checked by TASM functions in the Dispatcher before being forwarded to the AMPs to execute the request against the user database.

The Query Management component examines database log on and query requests. It also analyzes the resource criteria of those requests and the objects it references. TASM then compares the requests against the active rules to see if the requests should be accepted, rejected, or delayed.

Query Management rules have been generally classified into three groups: Filters, Throttles, and Workload Definitions (described in more detail on following pages). Query Management analyzes the incoming requests and compares the requests against the active rules to see if the requests should be accepted, rejected, or delayed.

- Queries that do not pass Filter Rules are rejected
- Queries that do not pass Throttle Rules can be delayed or rejected
- Queries that pass both Filter and Throttle rules are checked against Workload rules. Additional throttles can also be applied at the Workload Definition level.
- As queries execute within their assigned workload, they will be monitored for exceptions..
- Violations of exception rules can invoke several actions from changing workloads, abort the query, send alert or run a program.

Workload Management Architecture



The TDWM rules you create are stored in tables in the Advanced SQL Engine. Unless otherwise specified, every logon and every query in every Advanced SQL Engine session is checked against the enabled TDWM rules. That includes SQL queries from any supported Advanced SQL Engine interface, such as BTEQ, CLIV2, ODBC, and JDBC.

TDWM rules are loaded into the Dispatcher components of the Advanced SQL Engine. When a Vantage client application issues a request to the Advanced SQL Engine, the request is examined and checked by TDWM functions in the Dispatcher before being forwarded to the AMPs to execute the request against the user database.

Query Management analyzes the incoming requests and compares the requests against the active rules to see if the requests should be accepted, rejected, or delayed.

1. Queries that do not pass Filter Rules are rejected
2. Queries that pass Filter rules are classified into a Workload Definition.
3. Queries that do not pass Throttle Rules can be delayed or rejected. Additional throttles can also be applied at the Workload Definition level.
4. As queries execute within their assigned workload, they will be monitored against any exception rules.
5. Violations of exception rules can invoke several actions from changing workloads, abort the query, send alert or run a program.

Filters

[Object Access Filters](#)

Object Access Filters **reject** any access to database objects that you associate with the restriction.

Example:

If you associate a table (T1) with an access restriction for a group of users, then TDWM rejects any query that contains T1 within the defined operating environment (may be defined time period or a defined state such as LOAD).

[Query Resource Filters](#)

Query Resource Filters **reject** any access to database objects based on resource usage limits, as well as the dates/times or states that the resource usage limits apply.

Example:

On weekdays between 08:00 and 17:00, queries estimated to take longer than 30 minutes are not allowed to execute for users assigned to a specific performance group.

Note: You can configure how Query Resource filters are enforced by ignoring EXPLAIN estimates below a specified level of confidence.

Object Access filters are used to reject queries that attempt to access to all objects associated with the filter during the time period specified. When you define an Object Access filter, you can specify that only combinations of issuing and query objects are restricted. This lets you selectively limit access to the Teradata Database, tables, macros, and so on. For example, you could create a filter that never allows specific users access to specific database tables.

Defining Query Resource filters lets you reject queries based on database resource usage for any issuing object, query object, or object combinations associated with this type of filter. You define how resource usage is limited, as well as the dates and times the resource usage limits apply.

You can configure how Query Resource filters are enforced by ignoring EXPLAIN estimates below a specified level of confidence. For example, if the row count estimate on a query is generated with “no confidence” and the minimum explain confidence level is set to low confidence, then the row count estimate is not used.

SQL Types

For Object Access filters, Query Resource filters, and Object throttles, you can specify the types of SQL requests to which the rule applies. For example, you can specify ALL, DDL, DML, or SELECT.

Global

Global filters apply to all objects, and as a result to all logon and/or query requests during the specified time period. If a filter applies to all objects, you can specify it as a global rule.

Throttles

Object Throttles

Object Throttles can be used to limit the number of logon sessions and/or active queries.

Example:

On Weekdays between 8:00 and 17:00, account strings starting with \$M cannot have more than 200 simultaneous sessions on the entire Teradata Database.

On Weekends, account strings with \$H cannot have more than 5 simultaneous queries on the entire Teradata Database – delay queries, do not reject.

Load Utility Throttles

Load Utility Throttles allow you to control how many load utilities are simultaneously running on a Teradata Database at any given time.

Example:

On Weekdays between 8:00 and 17:00, the maximum number of simultaneous TPT Load and/or TPT Update jobs is 3.

Object Throttles

Defining Object Throttles lets you limit the number of logon sessions and/or queries active on for particular Teradata Database objects. You can define Object Throttles that apply to most types of Teradata Database objects. You cannot associate Object Combinations with Object Throttles. You can associate Macros and Stored Procedures with Object Throttles. However, they are treated like table objects in that we do not know that they are a “Macro” or a “Stored Procedure” per se. Workload Management will just know that the name in the rule matches the name on the object list.

You can set up this type of rule to reject or to delay any query that cannot be immediately processed. If more than one Object throttle applies to an object, the one with the lowest limit one is used.

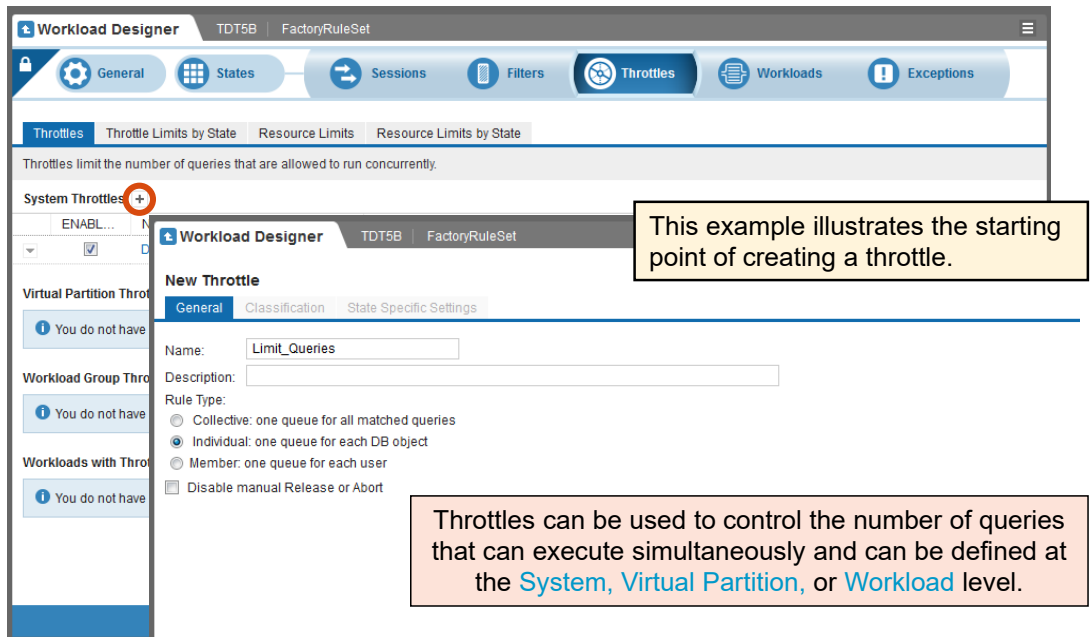
Load Utility Throttles

Defining Load Utility throttles lets you control how many load utilities are simultaneously running on a Teradata Database at any given time. Using this throttle type lets you override the MaxLoadTasks value set using the DBS Control Utility. Setting a throttling rate lets you override the value without having to change it using the DBS Control Utility.

You can specify limits for all load utilities as a group, and/or specify limits for each individual load utility. Because Load Utility throttles apply only to the kind and number of load utilities running on the Teradata Database, you cannot associate Teradata Database objects with them.

Workload Designer Example

Viewpoint's Workload Designer example.



The screenshot shows the Viewpoint Workload Designer interface. The top navigation bar includes tabs for General, States, Sessions, Filters, Throttles, Workloads, and Exceptions. The 'Throttles' tab is active, and the 'System Throttles' section is highlighted. A red circle with a plus sign is placed over the 'New Throttle' button. The 'New Throttle' dialog is open, showing the 'General' tab. The 'Name' field is set to 'Limit_Queries'. The 'Rule Type' section has three radio buttons: 'Collective: one queue for all matched queries', 'Individual: one queue for each DB object' (which is selected), and 'Member: one queue for each user'. There is also a checkbox for 'Disable manual Release or Abort'.

This example illustrates the starting point of creating a throttle.

Throttles can be used to control the number of queries that can execute simultaneously and can be defined at the [System](#), [Virtual Partition](#), or [Workload](#) level.

Viewpoint's Workload Designer is required to manage sessions, filters, throttles, workload definitions, and exceptions.

Workload Definitions

What is a Workload Definition?

- It is a description of rules that represent **who**, **where**, and **what** of a workload. A Workload Definition is assigned to a Priority Scheduler allocation group.
 - **WHO** – determines who is executing the query (e.g., Account string, Username, Profile, etc.)
 - **WHERE** – defines which database objects that are referenced by the query (e.g., table names)
 - **WHAT** – is based on optimizer estimates (# of AMPs, step row count, etc.)
- Each of the criteria are ANDed together.
 - Normally, you only need to have 1 or 2 classification criteria (maximum of 6).
 - A query is classified into a Workload definition (WD) if it satisfies all of the Classification criteria.

What are the benefits of Workload Definitions?

- **Improved Control of Resource Allocation** – resource priority is given on the basis of belonging to a particular workload from the start.
- **Improved Reporting** – workload definitions allow you to see who is using the system and how much of the various system resources – both real-time and long-term.
- **Automatic Exception Handling** – after a query has started executing, actions can be taken based on exception criteria that has been defined for the workload.

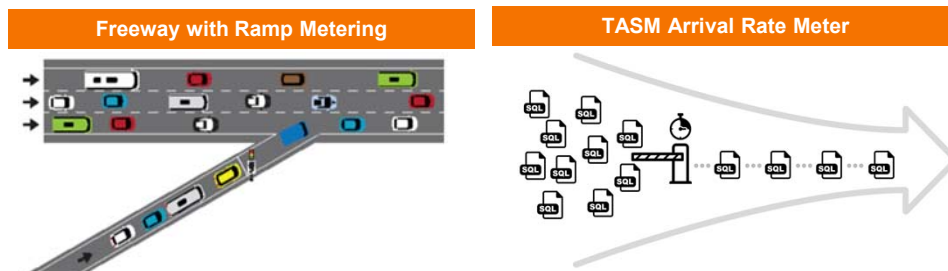
A workload represents a portion of the queries that are running on a system. A Workload Definition (WD) is a workload grouping and its operating rules to assist in managing queries. It consists of:

- **Classification Criteria:** criteria to determine which queries belong to the workload. This criteria defines characteristics which are detectable prior to query execution. This is also known as the "who", "where", and "what" criteria of a query. For example, "who" may be an account name, "where" is the database tables being accessed, and "what" may be the type of statement (UPDATE) being executed.
- **Exception Criteria:** criteria to specify "abnormal" behavior for queries in this workload. These criteria are only detectable after a query has begun execution. If the exception criteria are met, the request is subject to the specified exception action which may be to lower the priority or abort the query.
- **Operating Periods:** a description of hours of the day and/or days of the week (or month). Directives may be specified for exception handling and Priority Scheduler settings can be changed for each operating period.

You can specify up to six different criteria for a workload. A query is classified into a Workload Definition (WD) if it satisfies all of the Classification criteria. Normally, you will only need to specify one or two criteria for a workload definition.

Arrival Rate Meter

- An Arrival Rate Meter (ARM) is a new TASM rule type.
- ARM rules provide a means to regulate the flow of SQL requests admitted by TASM into the system.
- An ARM rule specifies the max number of SQL requests per time unit that TASM will admit into the system. Qualification criteria can be used to target specific types of requests (e.g., three queries per hour for queries with estimated processing time of more than one hour).



Workload Designer – Workloads

Workload Designer TDT5B FactoryRuleSet

General States Sessions Filters Throttles **Workloads** Exceptions

Workloads SLG Summary Evaluation Order Virtual Partitions Partition Resources Workload Distribution Console Utilities

Use workloads to group queries that have common characteristics or common requirements.

Workloads + Virtual Partition: (All) State: Base

ENABL...	NAME	METHOD	WORKLOAD THROTTLE
☒	H-WD	Timeshare	-
☒	L-WD	Timeshare	-
☒	M-WD	Timeshare	-
☒	T-WD	Timeshare	-
☒	WD-Default	Timeshare	-

WD-Default is a system workload used for any queries that do not classify to any previously defined workloads. It cannot be disabled, modified or deleted.

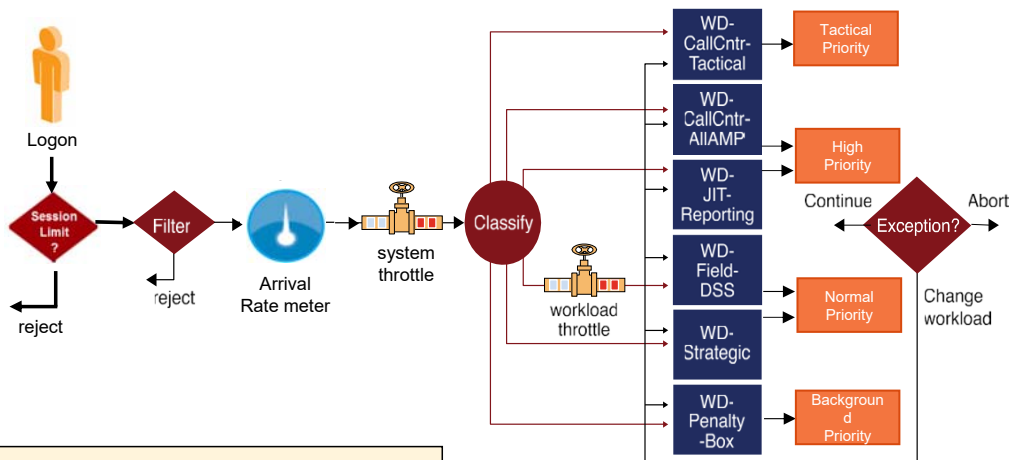
Save Reset

Viewpoint's Workload Designer can be used to create and modify workload definitions.

One workload that is always present is named **WD_Default**. The **WD-Default** workload definition is the default workload. It is automatically created and is used as a “No-Home WD”. Queries that do not match the characteristics of any other workload definition will run in this WD.

Note: The WD-Default definition cannot be deleted or edited.

Levels of Workload Management



Exceptions are used to detect misclassified queries executing within a workload.

There are several different methods of management offered, as illustrated below:

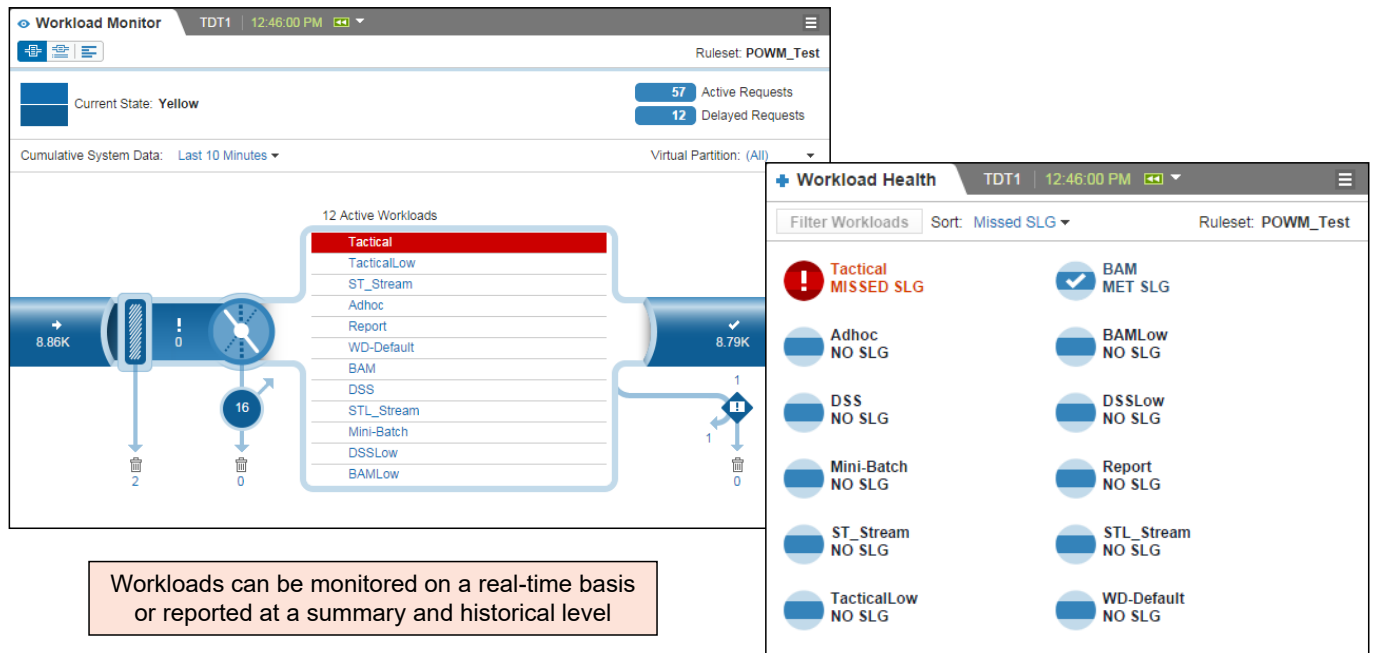
Methods regulated prior to the query beginning execution

1. Session Limits can reject Logons
2. Filters can reject requests from ever executing
3. Arrival Rate Meter can defer requests by managing the number of requests admitted into the system per specified time period
4. System Throttles can pace requests by managing concurrency levels at the system level.
5. Classification determines which workload's regulation rules a request is subject to
6. Workload-level Throttles can pace the requests within a particular workload by managing that workload's concurrency level

Methods regulated during query execution

1. Priority Management regulates the amount of CPU and I/O resources of individual requests as defined by its workload rules
2. Exception Management can detect unexpected situations and automatically act such as to change the workload the request is subject to or to send a notification

Viewpoint – Monitoring and Reporting



Workload management provides for monitoring and reporting by workloads via Teradata Viewpoint.

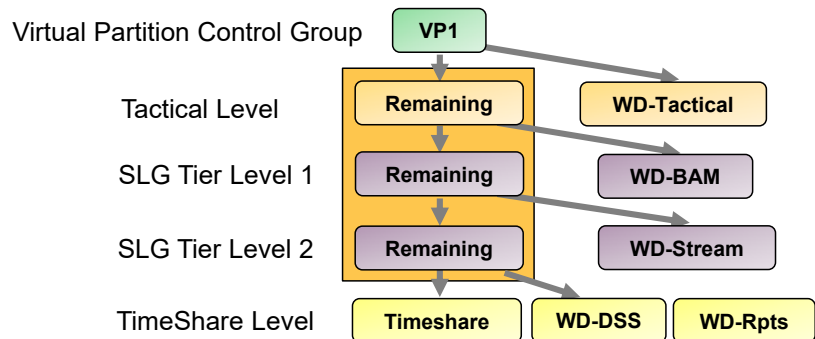
Workload Monitor Portlet enables monitoring of the system with a workload-centric perspective. It includes the ability to track workload performance against defined SLGs.

Workload Health Portlet displays workload health information for Service Level Goals, status (active, disabled, etc.) and various detailed metrics.

Virtual Partitions

- Virtual Partitions are similar to Resource Partitions in the previous Priority Scheduler.
- By Default, a single Virtual Partition exists named **Standard**.
- **Up to 10 Virtual Partitions** can be defined to Divide system resources in groups dedicated to a specific division of business units or geographic entities.
- Under each Virtual Partition there are levels (tiers) of service:
 - Tactical Tier (Tier #0)
 - 5 Service Level Goals (SLG) tiers
 - Time Share tier (last Tier)

Note: Resources “flow” from the top tier to the bottom tier (each tier will receive resources that are remaining/unused from the previous tier)



A single Virtual Partition exists for user work by default, but up to 10 may be defined, if needed. Due to improvements in Priority Scheduler capabilities, a single Virtual Partition is expected to be adequate to support most priority setups. Multiple virtual partitions are intended for platforms supporting several distinct business units or geographic entities that require strict separation.

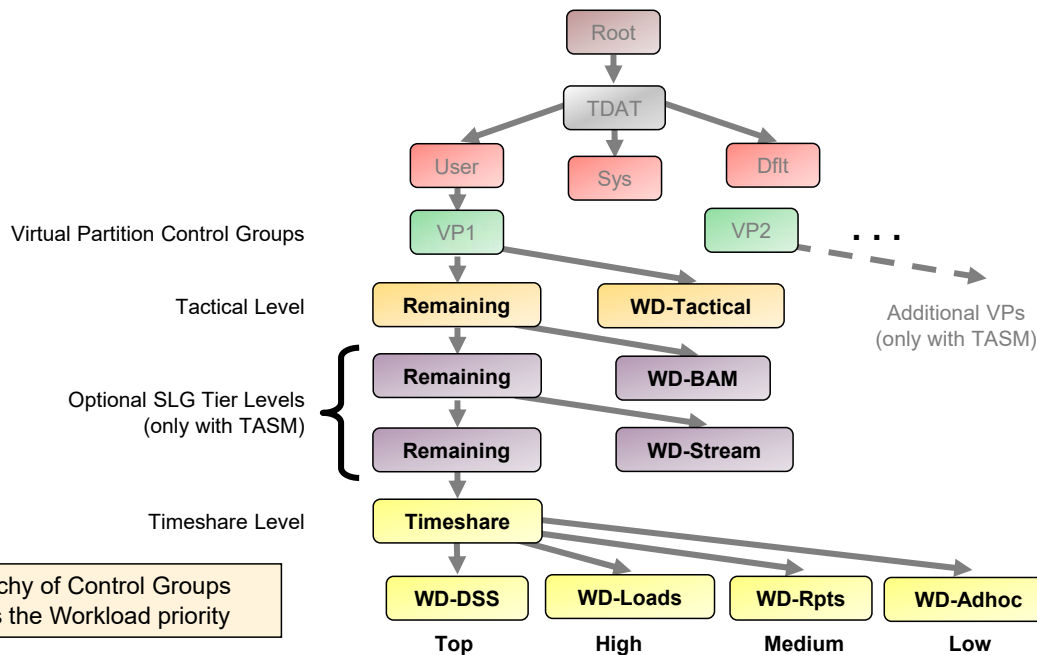
Virtual Partitions provide the ability to manage resources for groups of Workloads dedicated to specific divisions of the business. When a new Virtual Partition is defined, the administrator will be prompted to define a percentage of the Teradata Database resources that will be targeted for each, from the Viewpoint Workload Designer screens. This percent will be taken out of the percent of resources that flow down through the User Control Group.

Once defined, each of these Virtual Partitions can contain their own Control Group hierarchies. Each Virtual Partition hierarchy can include all allowable priority levels from Tactical to Timeshare.

The share percent given to a Virtual Partition will determine how the CPU is initially allocated across multiple Virtual Partitions. If there are spare cycles not able to be used within one Virtual Partition, another Virtual Partition will be able to consume more than its defined percent specifies.

For example, things like space accounting execute in the internal Virtual Partition, as do other activities not associated directly with user work, but that are important to get done. In addition, some internal activities that used to run in Performance Groups L and R in SLES 10 will now run in the internal Virtual Partition.

Control Groups and Workloads



Tactical Recommendations

- Only assign Workloads that support highly-tuned, very short queries to Tactical.
- Do not assign Workloads that support load utilities into Tactical.
- Rely on reasonable exception thresholds to demote queries with non-tactical characteristics.
- Monitor tactical exceptions regularly, adjust the exception thresholds, when needed.
- Don't increase the AMP worker task reserve count above zero unless a shortage of AMP worker tasks is impacting the tactical performance.
- If specifying reserved AWTs, set the reserved count for the worst case AMP usage by all tactical query Workloads combined, including tactical Workloads across all Virtual Partitions.
- Avoid placing tactical Workloads in Virtual Partitions with an inadequate percent of resources.

SLG Tier Recommendations

- SLG Tiers are intended for high priority work that is associated with response time expectations.
- Use only a single SLG Tier if only a few workloads fall into this category.
- If a large number of Workloads with widely-varying priorities fall into this category, place the Workloads with the more critical service level expectations on the higher SLG Tiers.
- If more than one SLG Tier supports Workloads, attempt to define smaller Workload Share Percents on higher Tiers, to allow a more predictable level of resources.

Timeshare Recommendations

- If SLG Tiers are not used, place Workloads in the Top Access Level that are very high priority.
- If a penalty box Workload exists when migrating to the new Priority Scheduler, add that Workload into the Timeshare Low Access Level.
- For more predictable priority differentiation in Timeshare, keep the decay option turned off.

Summary

- Workload Management is accomplished by using several tools that assist in defining the rules that control the allocation of resources to workloads running on a system.
 - These rules include **filters, throttles, workload definitions, and exceptions.**
 - Workload definitions are rules to control the allocation of resources to workloads.
- The benefit is to automate the allocation of resources to workloads.
- The key product that is used to create and manage these rules is **Teradata Viewpoint Workload Designer.**
- Other tools that facilitate workload management include:
 - **Teradata Workload Analyzer** – helps create workload definitions by analyzing existing PSF settings and DBQL information.
 - **Teradata Viewpoint** – contains portlets to monitor workloads.

This slide summarizes some important concepts regarding this module.



Module 12: Review Questions (1 of 2)

1. What type of filter or throttle rule is needed for the following restrictions?

Limit the number of concurrent sessions	_____
Reject queries based on max processing time	_____
Reject queries accessing a specific DB	_____
Limit the number of TPT Load jobs	_____
Delay more than 20 queries for a specific account	_____
2. What is the purpose of the default workload definition name "WD-Default"?
 - a. Default workload for any queries with an enforcement policy of normal.
 - b. Default workload for any queries that are not associated with a workload.
 - c. Default workload for any queries assigned to Default resource partition.
 - d. Default workload for any queries assigned to Standard resource partition.
3. Which query attribute is not used by the Parsing Engine software to classify a query into a Workload Definition (WD)?
 - a. User name
 - b. Account
 - c. User Role
 - d. User Profile
 - e. Optimizer estimates

Check your understanding of the concepts discussed in this module by completing the review questions as directed by your instructor.



Module 12: Review Questions (2 of 2)

4. Which Teradata application can be used to initially define workload definitions?
 - a. Teradata Manager
 - b. Workload Analyzer
 - c. Dynamic Workload Wizard
 - d. Priority Scheduler Administrator
5. Place the following control options in the proper sequence from 1 to 4 as they are acted upon by Teradata software.
 - a. ____ System throttles
 - b. ____ Exception criteria
 - c. ____ Workload throttles
 - d. ____ Filters

Check your understanding of the concepts discussed in this module by completing the review questions as directed by your instructor.



Module 13: Teradata Viewpoint

Teradata Vantage Administration

Copyright © 2007–2023 by Teradata. All Rights Reserved.

Objectives

After completing this module, you will be able to:

- List four portlets that are available with Viewpoint.
- Understand the purpose of various Viewpoint portlets
- Use Viewpoint to monitor sessions.

Teradata Viewpoint

Viewpoint is the cornerstone of Vantage Systems Monitoring and Management.

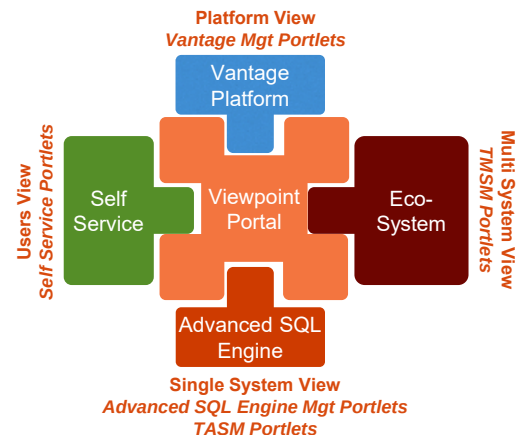
- Provides systems management via a web browser.
- Provides a single operational view (SOV) for a multi-system environment.
- Highly customizable and can be personalized.

Viewpoint Architecture

- Viewpoint Server – external server or appliance that executes the Viewpoint application
- Viewpoint Portal built on modern web technologies
- Data Collection Service (DCS) is performed by the Viewpoint server.

Viewpoint 17.10 Supported Browsers

- Mozilla Firefox 72+
- Google Chrome 79+
- Apple Safari 12.1+
- Microsoft Edge 41+
- Microsoft Internet Explorer 11
- Chrome Android mobile 70+
- Safari iOS mobile 9.3.5+



Teradata Viewpoint is intended as a Teradata customer's Single Operational View (SOV) for Teradata UDA, meaning it supports various Teradata systems in the UDA, including Teradata Vantage, Teradata Aster, Teradata QueryGrid, Teradata Presto, and Hortonworks and Cloudera Hadoop systems.

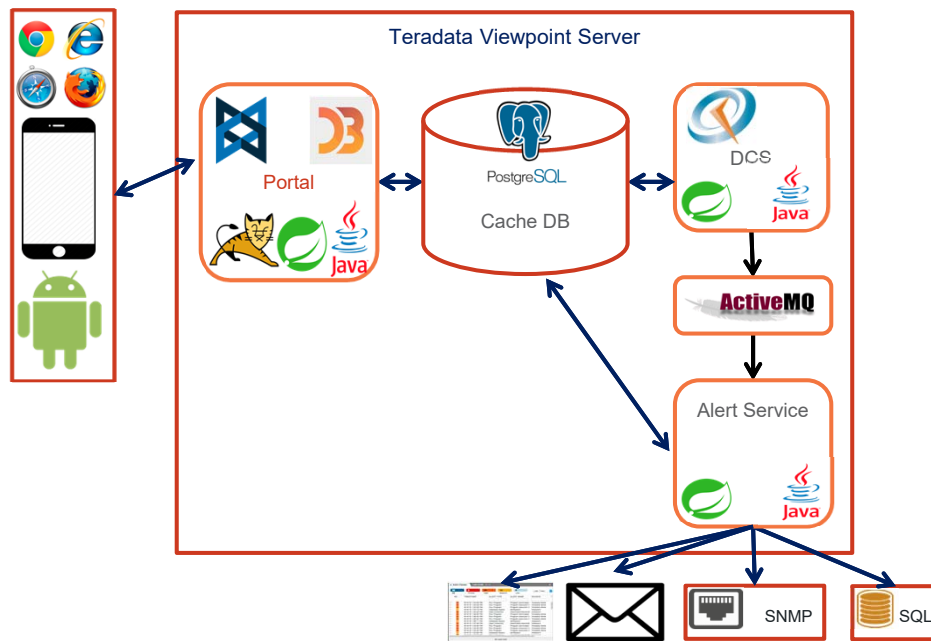
It provides web-based interface (a set of portals) for a wide range of capabilities and features, such as monitoring, management, alerting, and others. It serves both system administrators as well as business users. It also serves as user interface for other Teradata products, for example Teradata Data Lab.

Teradata Viewpoint provides systems management via a web browser which is extensible to Teradata end users and management, allowing them to understand the state of the system and make intelligent decisions about their work day.

Teradata Viewpoint allows users to view system information, such as query progress, performance data, and system saturation and health through preconfigured portlets displayed from within the Teradata Viewpoint portal. Portlets can also be customized to suit individual user needs. User access to portlets is managed on a per-role basis.

Administrators can use Teradata Viewpoint to determine system status, trends, and individual query status. By observing trends in system usage, system administrators are better able to plan project implementations, batch jobs, and maintenance to avoid peak periods of use. Business users can use Teradata Viewpoint to quickly access the status of reports and queries and drill down into details.

Viewpoint Architecture



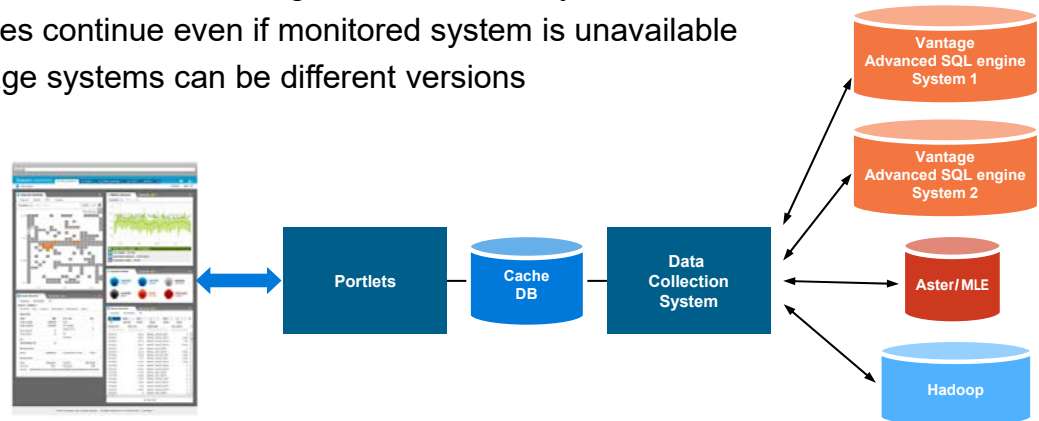
Characteristics of the current Viewpoint architecture are:

- At the heart of this arch is the TMS or VM hosting 3 components: DCS, Postgres and Portal
- DCS polls data periodically from different sources – TD, Aster, Hadoop
- DCS sends messages to the Alerting Service over JMS
- Tomcat Portal provides deployment framework for mini web apps, built on modern web technologies including Backbone, D3, JSON data exchange over REST API
- Finally user views data on Viewpoint from browser/ phone/ tablet

Data Collection Service

Viewpoint Data Collection Service (DCS)

- Collects monitoring and performance data
- Viewpoint portlets use data from cache database
- Offloads management and monitoring from monitored system
- Viewpoint services continue even if monitored system is unavailable
- Monitored Vantage systems can be different versions



The data collection service is a set of processes and a data cache that retrieve statistics from Teradata servers for display via Viewpoint portlets. The data collected from the Teradata servers is stored in the appliance database so you can observe historical trends graphically. But usually it is today's current performance people are most concerned with. So, the data collection service captures statistics from the EDWs, saves it, and displays them in various forms through portlets.

One advantage is that usually when a Teradata server is severely congested or locked up, any attempt to query it will be slow or will fail. Using Viewpoint, you can do some troubleshooting to find the congestion without adding more stress to the EDW server. Similarly, if the Teradata server is shut down or frozen, you can still query information from Viewpoint.

The Data Collection Server (DCS) is a component of the Viewpoint Framework and is responsible for retrieving performance and monitoring data for Teradata. Most of the data collected by the DCS is transient in Teradata so the DCS extracts it and persists it for historical analysis. The data is collected by communicating via JDBC to Teradata's SQL, Monitor and Console partitions. The DCS is capable of collecting data from multiple Teradata instances as well as Aster and Hadoop. Once the DCS collects the data, it persists it to a middle-tier database so that future access to the data does not affect the performance of the Teradata Vantage Advanced SQL engine. The end-user management and reporting modules in the Teradata Portal use the data to display near real time and historical monitoring and performance information for Teradata.

Viewpoint Portal and Portlets

- A Viewpoint portal represents a Web location that can be accessed or logged onto.
- Each [portal page](#) is a virtual work space where you decide which [portlets](#) to add/ display and how to arrange them on the page.
 - Viewpoint continually updates the information displayed on the portal page.
 - Viewpoint's applications or tools are called [portlets](#) and Add Content allows them to be added.

System Overview

- Alert Viewer
- Canary Response Times
- Elastic Performance
- Hadoop Services
- Productivity
- Space Usage
- System Health
- Today's Statistics
- TVS Monitor
- Viewpoint Monitoring

Node Overview

- Node Monitor
- Node Resources

Workload Management

- Workload Designer
- Workload Health
- Workload Monitor

Utilities

- Remote Console
- SQL Scratchpad
- Viewpoint Calendar

Applications

- Data Labs
- MAPS Manager
- QueryGrid
- Stats Manager
- Performance Data Collection

Session Management

- Application Queries
- Completed Queries
- Lock Viewer
- My Queries
- Query Groups
- Query Log
- Query Monitor
- Query Spotlight

Trends

- Metric Heatmap
- Metrics Analysis

Others

- Unity, Ecosystem Manager
- Data Mover
- DSA / BAR

Portal Basics

To help you work efficiently, Teradata Viewpoint uses a page metaphor as the framework for displaying and updating portlets. Each portal page is a virtual work space where you decide which **portlets** to display and how to arrange them on the page. Examples of ways to organize your work include defining a page for each system being monitored, or for each type of query or user.

As you work, Teradata Viewpoint continually updates the information displayed on the page that currently fills the Teradata Viewpoint portal. This page is called the *active page*.

Portlet Basics

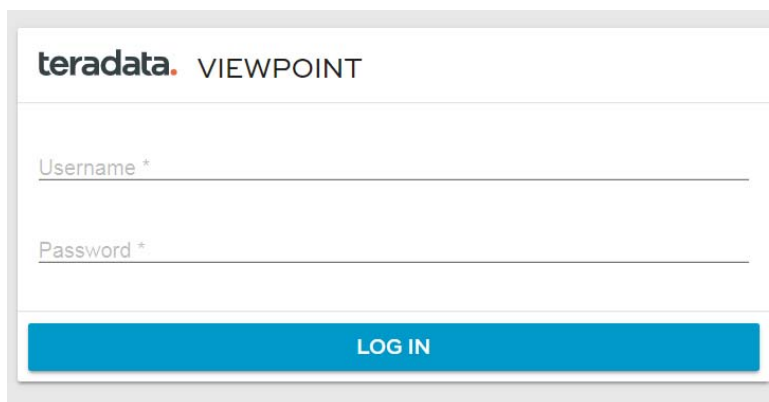
Teradata Viewpoint system management tools are called *portlets*. Select the portlets that you want to display and monitor from submenus, or categories, under Add Content. You can also search for a specific portlet name using the filter feature. Generally, every instance of a portlet:

- Has a frame that appears when the cursor moves over any part of the portlet on the page and disappears when the cursor moves off the portlet.
- Displays the portlet name or the current settings in the upper frame, depending on the activity.
- Has a width requirement to ensure proper display of its graphical information such as charts, spark lines, or graphs.
- Remains at a fixed size even when the browser window is re-sized. You can use the browser scroll bars to view the entire portlet view.
- Can be repositioned on the portal page.

Logging onto Viewpoint

Logging on to the Teradata Viewpoint portal begins your session so you can begin working with the Teradata Viewpoint portal.

1. Open a browser.
2. Enter the address for your Teradata Viewpoint portal.
3. Log on to the Teradata Viewpoint portal.

A screenshot of the Teradata Viewpoint login interface. At the top left, the 'teradata.' logo is followed by the word 'VIEWPOINT'. Below this, there are two input fields: 'Username *' and 'Password *'. At the bottom of the form is a blue rectangular button with the text 'LOG IN' in white capital letters.

Logging on to the Teradata Viewpoint portal begins your session so you can begin working with the Teradata Viewpoint portal.

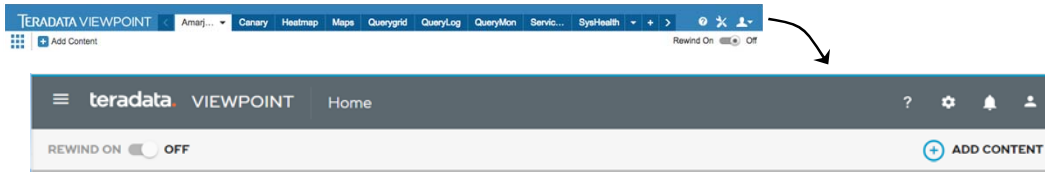
1. Open a browser.
2. Enter the address for your Teradata Viewpoint portal.

The Welcome page appears, with the portal version number shown at the bottom.

3. Log on to the Teradata Viewpoint portal.

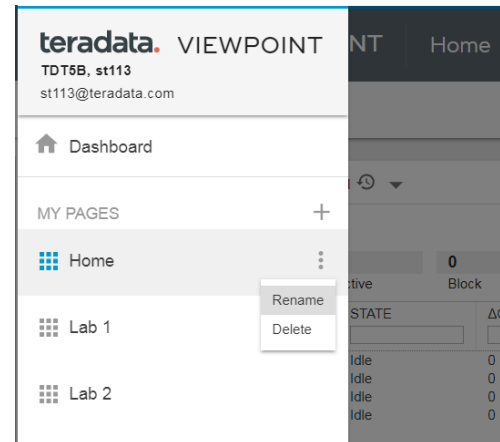
If your Teradata Viewpoint system is set up to create a user profile automatically, the username and password you enter are authenticated against your company-provided username and password the first time you log on to Teradata Viewpoint. Automatic profile creation is known as *auto-provisioning*.

Updated User Interface



What's updated with the navigation

- Portal tabs have been transformed into a page menu that slides out from the left of the page
- Page operation (delete and edit page name) are accessible via the page item menu
- Add new page button opens a New page dialog
- Edit page name menu items opens up an Edit page name dialog
- Page name rows are draggable



NOTE on the Updated UI

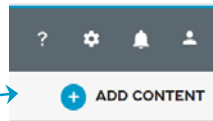
- The UI uses Teradata Covalent design standards, based on Material Design.
- However, the UI platform itself has NOT been changed to Covalent

Information on

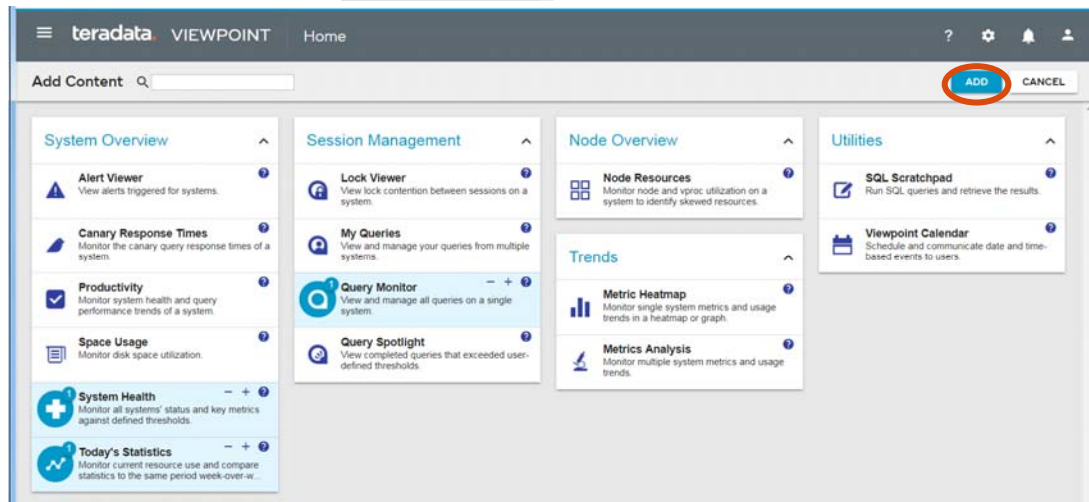
- External site on Teradata Covalent: <http://getcovalent.com>
- Internal only site on Covalent design process: <https://covalentcriteria.appcenter.ps.ac.uda.io/>
- Material Design: <https://material.io>

Adding Portlets

Portlets are added via the Add Content.



You can select several portlets to add to the page at once including multiple instances of a portlet.



Add Content: One can add one or more portlets in an operation including multiple instances of the same portlet if desired. There is also a search option at the top to assist in finding the right portlet. Portlet category groupings assist in search and also understanding of portlet relationships. Lastly, notice that all portlets have a unique representative icon.

Example of Initial Session and Portlets

This initial login shows three portlets that are executing.

The screenshot shows the Teradata Viewpoint portal interface. At the top, there is a navigation bar with a hamburger menu, the 'teradata' logo, and a 'Home' button. Below the navigation bar, there is a 'REWIND ON' toggle set to 'OFF' and an 'ADD CONTENT' button. The main content area displays three portlets: 'System Health' (showing 'Factory Healthy'), 'Query Monitor' (showing a table of session statistics), and 'Today's Statistics' (showing a table of system metrics). Annotations with red arrows point to the 'Add Pages' button in the top left, the 'Current Page' tab in the top right, and the 'Portlets' area in the center.

STATISTIC	LAST HOUR	SAME PERIOD 1 WEEK AGO	SINCE 12 AM TODAY	SAME PERIOD 1 WEEK AGO
Active Sessions	0	0	0	0
AMP CPU Skew	70.33%	64.35%	67.74%	60.13%
AMP I/O Skew	24.78%	22.29%	27.27%	24.67%
AWT	0.063	0.063	0.063	0.063
CPU	4.168%	3.939%	4.096%	4.218%
System CPU	3.302%	3.182%	3.282%	3.319%
Total Disk Space	3.502%	3.522%	3.509%	3.513%
User CPU	0.867%	0.756%	0.814%	0.9%
Wait I/O CPU	0.123%	0.132%	0.134%	0.137%

This slide illustrates an initial session with 3 portlets that have been started.

You can manage portal pages using the following guidelines:

- Add portal pages at any time during a Teradata Viewpoint session.
- Access any portal page by clicking its tab; only one page can be active at a time.
- Change the name of any tab, including the **Home** page tab; page names can be duplicated.
- Rearrange pages by dragging and dropping into a new location.
- Remove pages, along with any portlets contained on the page, with a single mouse-click.
- One page (tab) must remain, as well as the **Add Page** tab.

Adding a Portal Page

Organize your system management tools by adding pages to the Teradata Viewpoint portal. Multiple pages can be added or removed per session. The newest page is always the active page unless you click on another tab.

- In the Teradata Viewpoint portal, click Add Page.
- A New Page tab appears to the left of Add Page and becomes the active page.

Viewpoint Portlet Controls

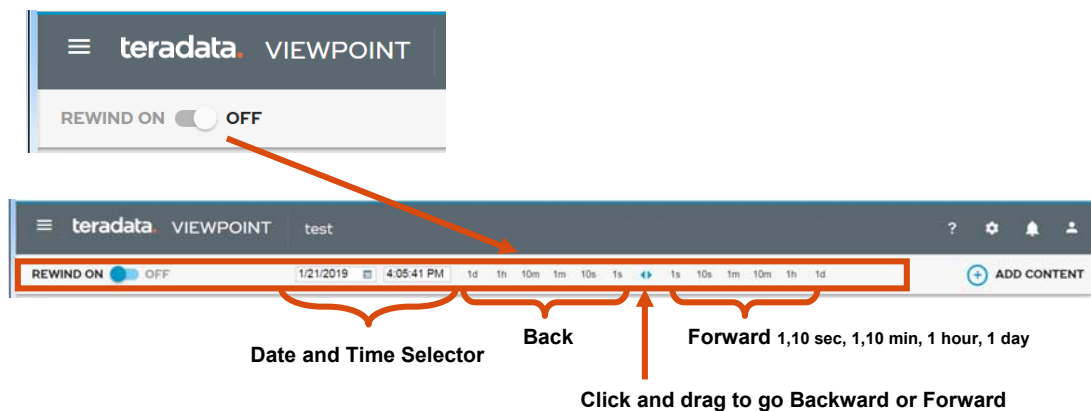
The screenshot displays the Teradata Viewpoint interface with several annotations highlighting portlet controls:

- Pages via hamburger menu:** Points to the hamburger menu icon in the top left.
- Viewpoint Administration:** Points to the 'Viewpoint Administration' link in the top right.
- User Profile:** Points to the user profile icon in the top right.
- Back:** Points to the 'Back' button in the top left of the portlet area.
- Pause/Resume:** Points to the pause/resume button in the top right of the portlet area.
- Portlet Menu:** A yellow box listing controls that vary by portlet:
 - Settings/Preferences
 - Maximize size
 - Share
 - Help
 - Remove Portlet
- Information Balloons:** Points to a data point on the 'Metrics Analysis' chart.
- In Rewind mode:** Points to the 'In Rewind mode' indicator at the bottom right.
- Adjust height:** Points to the 'Adjust height' button at the bottom center.
- Restore height:** Points to the 'Restore height' button at the bottom right.

Each portlet instance has its own settings and controls as well as the features common to all portlets. Not all portlets have all controls.

You can access the portlet features and controls as shown on this slide.

Viewpoint Rewind



Rewind, replay, fast forward Viewpoint portlets to review Teradata operations at past points in time.

The rewind feature allows you to view data that corresponds to dates and times in the past and compare it to data for a different date and time. You can rewind the data for some or all portlets on a portal page to a previous point in time, such as when a job failed. Rewinding portlet data is useful for identifying and resolving issues.

You can rewind data as far back as data is available. The rewind feature is not available for portlets that have portlet-specific methods for reviewing data over time.

Using the rewind toolbar, you can enter a specific date and time as well as scroll through the data in increments of seconds, minutes, hours, or days. All portlets on the page that are participating in rewind activities display data that corresponds to the selected rewind date and time each time a selection is made.

Query Monitor

The Query Monitor portlet allows you to view information about queries running in a Teradata Vantage Advanced SQL Engine.

Query Monitor 10:05:26 PM

Factory By Session All

44 8 8 0 0 0 0 36 0 0 0 0 0 0

All Not Idle Active Block Delay Abort Resp Idle Parse Unknown QTDelayed SesDelayed Response...

SESSION ID	ΔCPU	ΔI/O	SNAPSHOT...	PJI	DURATION	USERNAME	ACCOUNT	CPU USE
455470	0	0			0:00:00	STUDENT114	\$M00EDUC&D&H	0
455561	13.64	0	12.68	27.26	0:00:59	STUDENT114	\$M00EDUC&D&H	11.56
455567	0.002	0			0:00:00	STUDENT114	\$M00EDUC&D&H	0.002
455679	0.002	0			0:00:00	STUDENT105	\$M00EDUC&D&H	0.002
455735	0	0			0:00:00	STUDENT103	\$M00EDUC&D&H	0
455859	0	0			0:00:00	STUDENT113	\$M00EDUC&D&H	0
455913	0.004	0			0:00:00	STUDENT108	\$M00EDUC&D&H	0.003
455916	14.16	0	10.09	30.85	0:00:59	STUDENT108	\$M00EDUC&D&H	12
456037	0	0			0:00:00	STUDENT135	\$M00EDUC&D&H	0
456038	0	0			0:00:00	STUDENT135	\$M00EDUC&D&H	0
456076	15.33	19	3.79	163.1	0:05:00	STUDENT105	\$M00EDUC&D&H	12.99
456088	0	0			0:00:00	STUDENT105	\$M00EDUC&D&H	0
456117	14.58	64	7.768	103.7	0:04:00	STUDENT113	\$M00EDUC&D&H	12.36
456118	0.002	0			0:00:00	STUDENT104	\$M00EDUC&D&H	0.002
456119	14.79	7	6.427	172.1	0:05:00	STUDENT103	\$M00EDUC&D&H	12.53
456120	0	0			0:00:00	STUDENT110	\$M00EDUC&D&H	0
456121	14.82	0	6.275	43.74	0:00:59	STUDENT109	\$M00EDUC&D&H	12.56
456122	0	0			0:00:00	VIEWPOINT	\$M00EDUC&D&H	0

44 rows total

Selecting a SESSION ID will provide detailed information about the request.

The Query Monitor portlet allows you to view information about queries running in a Teradata Vantage Advanced SQL engine so you can spot problem queries. You can analyze and decide whether a query is important, useful, and well written. After you have identified a problem query, you can take action to correct the problem by changing the priority or workload, releasing the query, or aborting the query or session. You can take these actions for one query or session, or multiple queries or sessions at a time.

The summary view contains a table with one row allocated to each of the sessions, account strings, users, or utilities running.

The portlet allows you to filter queries in all of the session views. You can set thresholds for any column and when the threshold is exceeded, the information is highlighted in the sessions table.

Select a row to access session and query information in the details view. Using Query Monitor, you can also determine the types of utilities that are running most frequently on the system and then set utility limits. You can spot utilities that are using a large number of partition connections and, potentially, a high number of resources.

From the PREFERENCES view, you can set the criteria values used to display sessions in the My Criteria view and customize the information displayed in the views. For example, you can set criteria values to display only those sessions currently running on the selected system that exceed the specified criteria. You can troubleshoot problems to quickly explore details about queries such as the current state of a query or how long a query has been blocked.

Viewpoint Query Monitor Detail View

Depending on the current state of the request, by selecting the drop-down menu, the request can be:

- Aborted
- Changed to another Workload
- Released

Depending on Query State, available tabs include:

- Overview
- SQL
- Explain
- Skew
- Query Band

Query

Query Monitor 10:05:26 PM

Factory By Session All

Session: 456117

OVERVIEW SQL EXPLAIN SKEW QUERY BAND

QUERY INFO

State: Active
 Time in state: 0:04:00
 Total duration: 0:04:00
 Spool space: 43,319,296
 Hot AMP spool: 2,707,456
 Spool skew: -
 Temp space: 0
 Request CPU: 93.8
 Request I/O: 904
 PJI: 103.72
 Unnecessary I/O: 0.01

WORKLOAD INFO

Name: unknown (id=17)
 Method: -
 Classification mode: Auto
 Virtual partition: -

SESSION INFO

User: STUDENT113
 Account: \$M00EDUC&D&H
 Partition: DBC/SQL
 Requests: 5
 Source: (TCP/IP) e49a 172.16.41.108 10.25.252.18/10.25.252.18:1025 CID=3C4A710C BS186023 JDBC16.20.00.08;1.8.0_191 01 LSS

SNAPSHOT INFO

CPU use: 12.36%
 Impact CPU: 95.87
 Snapshot CPU: 7.8%
 Snapshot I/O: 0%

Session

The details view displays statistics and information about the selected session. This view can be accessed by clicking on a session row in the summary view.

When viewing a request, you can see detailed information from the following tabs:

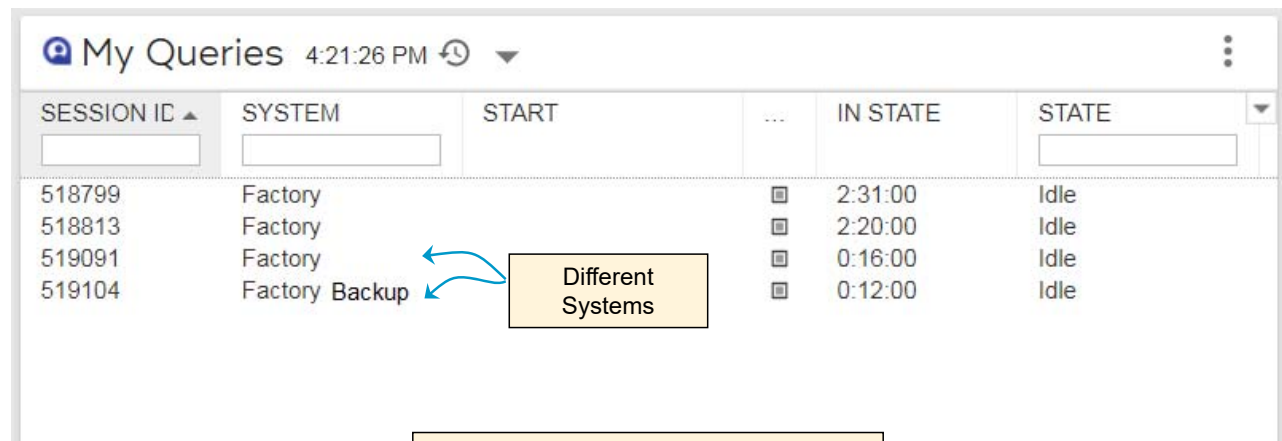
- **Overview** – key statistics for a session. Any value exceeding the thresholds is highlighted.
- **SQL** – SQL for the selected query.
- **Explain** – Explain steps for the query, including step statistics and explain text.
- **Blocked By** – details about other queries that are blocking this query.
- **Delay** – details about rules delaying this query.
- **Query Band** – displays the query band name and value for the selected query.

Use the **Tools** menu to change the priority or workload, release a query, or abort a query or session for one query or session at a time.

Use the **Next** and **Previous** buttons to move through sessions without returning to the summary view.

My Queries

My Queries allows a user to view their queries across multiple systems.



SESSION ID ▲	SYSTEM	START	...	IN STATE	STATE
518799	Factory		▣	2:31:00	Idle
518813	Factory		▣	2:20:00	Idle
519091	Factory		▣	0:16:00	Idle
519104	Factory Backup		▣	0:12:00	Idle

Selecting a SESSION ID will provide detailed information about the request.

The My Queries portlet allows you to view and manage your queries across multiple Teradata Vantage Advanced SQL engines. You can see if queries are queued or blocked, and you can see their impact on system resources.

Use the My Queries portlet to view information about queries in either the summary view or the details view. The summary view contains a table with one row allocated to each of the sessions logged on under one or more user names. Select a row in the summary view to see additional session and query information in the details view. Use the SQL, Explain, Blocked By, or Query Band tab in the details view to display information for the selected session.

The PREFERENCES view allows you to select one or more systems, and then select one or more users per system to monitor. From this view, you can also select a format for the SQL that appears in the query details view.

PJI and UII Metrics

Two important metrics that may appear in various Viewpoint displays are the **PJI** and **UII** metrics. The metrics are used as “Indicators” to analyze query efficiency towards consuming CPU and I/O resources.

- **PJI (Product Join Indicator)** – ratio of the CPU milliseconds per I/O. A high PJI number indicates system performance degradation and may indicate a bad product join has occurred.
 - A high PJI (> 5) for a query indicates that the query takes many CPU seconds for a given number of I/Os.
- **UII (Unnecessary I/O Indicator)** – all AMP I/O divided by all AMP CPU to reveal large amounts of I/O occurring over a period of time.
 - A high UII (> 5) for a query indicates that many I/O blocks are read, but relatively low number of rows are actually being processed.
- Example: If a query is performing a full table scan but returns very few rows then the UII will be relatively high.
- Creating an index or partitioning the table may possibly reduce the Unnecessary IO (UII).

This slide discusses the PJI and UII metrics (indicators) that are commonly used in query analysis.

A summary of these two indicators is:

PJI (Product Join Indicator)

CPU ms per I/O, if > 5, indicates a CPU intensive query such as bad Product Join.

UII (Unnecessary I/O Indicator)

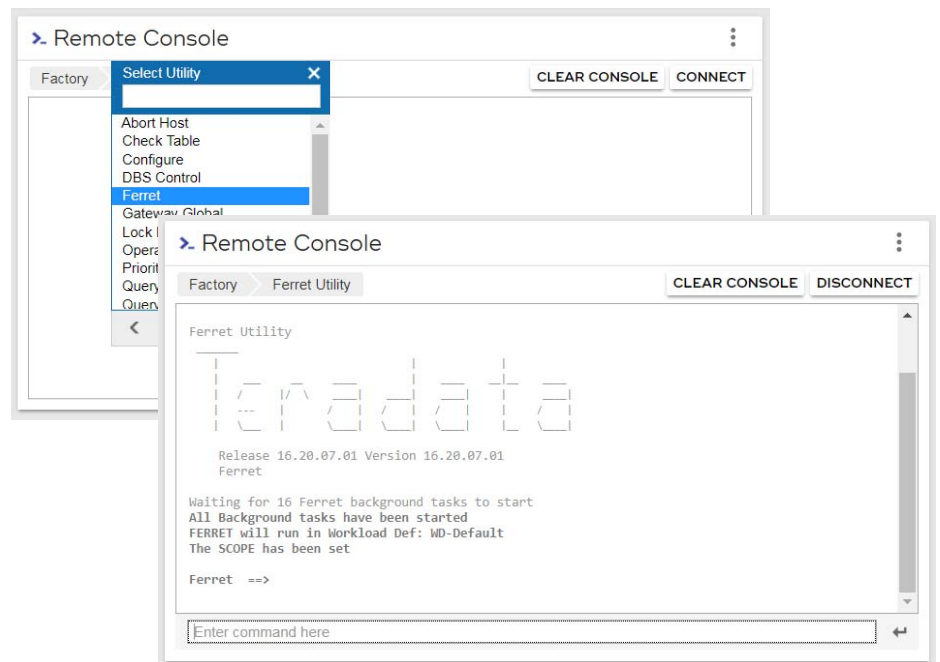
I/O per CPU ms, if > 5, then an Index or partitioning may help the query.

Remote Console

The [Remote Console](#) portlet allows you to execute system utilities.

Add Content > Tools > Remote Console

- Abort Host
- Check Table
- Configure
- DBS Control
- Ferret
- Gateway Global
- Lock Display
- Operator Console
- Priority Scheduler
- Query Configuration
- Query Session
- Recovery Manager
- Show Locks
- Teradata DWM Dump
- Update Space
- Vproc Manager



The Remote Console portlet allows you to run many of the Teradata Vantage Advanced SQL engine console utilities remotely from within the Teradata Viewpoint portal.

Using this portlet, you can:

- Select or search for a system.
- Select or search for a utility.
- Enter console utility commands.
- Display responses from the commands.

Teradata field engineers, Operators, System Administrators, and System Programmers use these utilities to administer, configure, monitor, and diagnose issues.

Remote Console activity requires special access rights, BUT does not require Linux Root authority.

Viewpoint Alert Viewer

SE...	TIMESTAMP	ALERT TYPE	ALERT NAME	SOURCE	SYSTEM NAME
	2/6/19 4:58:16 PM	User Session Summary	ThreeOrMore	Viewpoint	Viewpoint
	2/6/19 4:58:16 PM	User Session Summary	ThreeOrMore	Viewpoint	Viewpoint
	2/6/19 4:57:16 PM	User Session Summary	ThreeOrMore	Viewpoint	Viewpoint
	2/6/19 4:57:16 PM	User Session Summary	ThreeOrMore	Viewpoint	Viewpoint
	2/6/19 4:56:16 PM	User Session Summary	ThreeOrMore	Viewpoint	Viewpoint
	2/6/19 4:55:16 PM	User Session Summary	ThreeOrMore	Viewpoint	Viewpoint
	2/6/19 4:55:16 PM	User Session Summary	ThreeOrMore	Viewpoint	Viewpoint
	2/6/19 4:54:22 PM	System	MaxSessionsThreshold	Viewpoint	Factory
	2/6/19 4:54:16 PM	User Session Summary	ThreeOrMore	Viewpoint	Factory
	2/6/19 4:54:16 PM	User Session Summary	ThreeOrMore	Viewpoint	Factory
	2/6/19 4:53:16 PM	User Session Summary	ThreeOrMore	Viewpoint	Factory
	2/6/19 4:53:16 PM	User Session Summary	ThreeOrMore	Viewpoint	Factory
	2/6/19 4:52:16 PM	User Session Summary	ThreeOrMore	Viewpoint	Factory
	2/6/19 4:51:16 PM	User Session Summary	ThreeOrMore	Viewpoint	Factory
	2/6/19 4:51:16 PM	User Session Summary	ThreeOrMore	Viewpoint	Factory
	2/6/19 4:50:16 PM	User Session Summary	ThreeOrMore	Viewpoint	Factory
	2/6/19 4:49:16 PM	User Session Summary	ThreeOrMore	Viewpoint	Factory
	2/6/19 4:49:16 PM	User Session Summary	ThreeOrMore	Viewpoint	Factory
	2/6/19 4:48:16 PM	User Session Summary	ThreeOrMore	Viewpoint	Factory

30 rows total

The **ALERT VIEWER** portlet allows users to view alerts defined for the system. The alert information in the summary view is updated every 30 seconds. Every alert has a timestamp, displaying the date and time at which the alert was issued.

You can filter the alerts by for example severity, time period, type, or name. You can also combine the filters to narrow the results further.

The **ALERT DETAILS** view displays detailed information about what triggered the alert, the source of the alert, and any relevant messages.

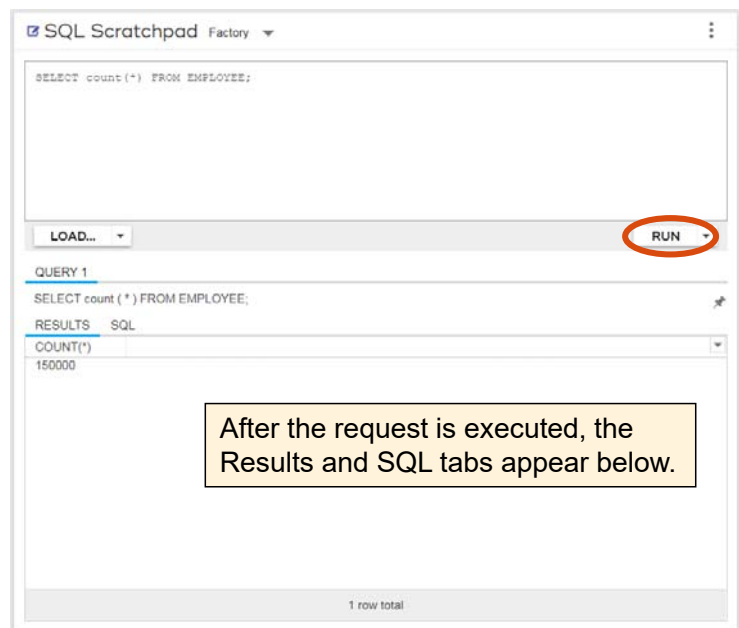
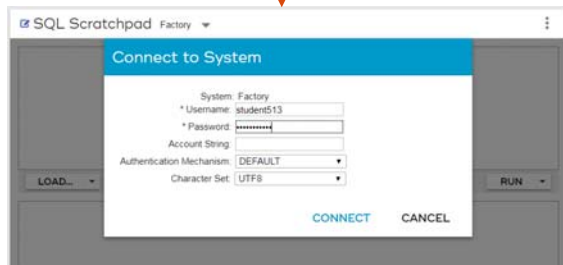
An alert is an event that the Teradata System Administrator defines as being significant. The Teradata System Administrator assigns alert severity levels to rank alerts, and can also include an explanatory message. The severity levels are: critical, high, medium, or low. The alerts displayed in the **ALERT VIEWER** portlet are specific to your system.

The Alert Viewer portlet basically **hides alerts**. As a new type of filter, a hide option has been built into the Alert Viewer portlet allowing certain alerts to be hidden from view. One may use this to selectively hide duplicates or possibly as part of tracking resolved issues. The hide option can be executed for an individual query or through a table's action menu bulk operation (via check boxes). There is a setting that then allows if hidden alerts should be displayed or not. If hidden alerts are displayed, they will have a strike-through representing the exception.

Viewpoint SQL Scratchpad

Click the Select System pull down button and select the system you want. You will be prompted to enter logon information.

Once connected then click Run to execute query.



After the request is executed, the Results and SQL tabs appear below.

The **SQL SCRATCHPAD** portlet allows you to enter queries and retrieve data from a Teradata Vantage Advanced SQL engine. Select a system to run your query, enter a single or multi-statement query, and then view the results.

After the executed query retrieves the data, the **RESULTS** section expands to display four tabs. Use the **Results** tab to see the results of your query. Use the **SQL** tab to display the SQL statements used to create the query. Use the **Explain** tab to see steps listed in chronological order. The steps are refreshed every 30 seconds so you can see how longer running queries are progressing. If there are delays in processing your query, you can cancel the query. If running a query takes longer than you expected, it might be blocked. Use the **Blocked By** tab to see information about queries that are blocking the currently running query.

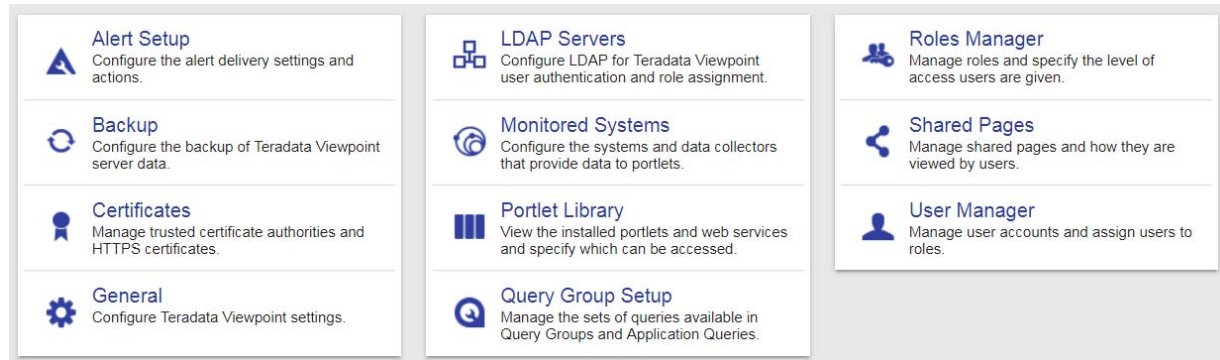
Use the **OBJECT BROWSER** to view a list of objects in the database. Also use the **OBJECT BROWSER** to insert an object into a query to reduce the time required to build a query and help reduce errors in object names.

The **SQL SCRATCHPAD** portlet also allows you to save queries or export results to a file where you can sort and analyze the information. You can *pin* query results (pinning allows you to temporarily save the query results). Saving the query allows you to use the query in the current session or in future sessions. Use the **History** tab to access previously run queries from your current session.

The **SQL SCRATCHPAD** portlet frame displays the system name on which the query is running.

Viewpoint Administration

- Portal/Portlet settings
- Viewpoint system configuration
- User/Roles & Authentication setup
- Backup and Alert settings
- And more...



Administration Portlets - allows access to Teradata Viewpoint resources and information.

Alert Setup - Configure the alert delivery settings and actions.

Backup - Configure the backup of Teradata Viewpoint server data.

Certificates - Manage trusted certificate authorities and HTTPS certificates.

General - Configure Teradata Viewpoint settings.

LDAP Servers - Configure the LDAP servers for Teradata Viewpoint to authenticate users and assign user roles.

Monitored Systems - Add the systems and configure the data collectors that provide data to portlets.

Portlet Library - View the installed portlets and specify which portlets can be enabled.

Query Group Setup - Manage the sets of queries available to users in the Query

Groups and Application Queries portlets - these can also be used to define the criteria that associate a query with a particular application in the Query Log portlet.

Roles Manager - Manage roles and specify the level of access users are given.

Shared Pages - Manage shared pages and how they are viewed by users.

User Manager - Manage user accounts and assign users to roles.

Security and Permissions - the Teradata Viewpoint administration portlets allow the Teradata Viewpoint Administrator to control portlet access at different levels by setting permissions.

Global - enable or disable access to portlets globally using **Portlet Library**. This setting takes precedence over all other portlet permission levels.

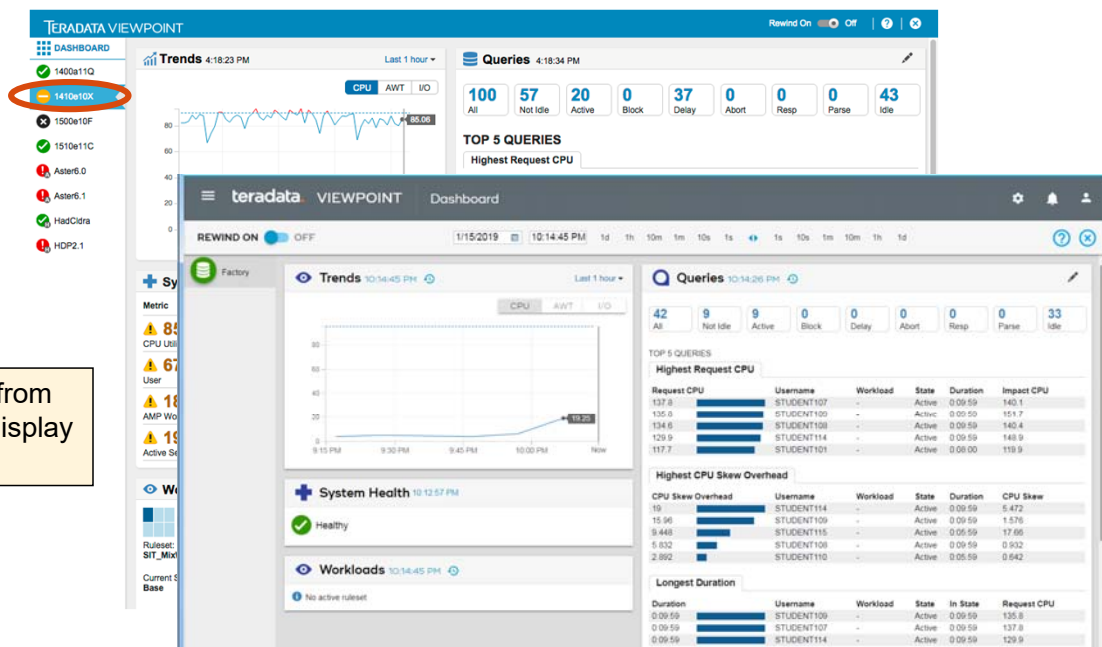
Role - Assign permissions to classes of users called roles using the **Roles Manager** portlet. Each tab controls access to features.

User - Assign available roles to users and set role precedence using **User Manager** portlet. Roles assigned higher priority take precedence over lower-priority roles.

Viewpoint Dashboard Feature

A portal feature that provides a high-level view of overall system activity

Select the System from Dashboard bar to display its system activity.



The Viewpoint Dashboard (released with Viewpoint 15.11) is designed to give the user an at-a-glance overview of monitored systems while they perform other everyday tasks. If something catches their eye they can quickly switch to the Dashboard for a system and begin a more in-depth investigation.

The Viewpoint Dashboard pulls together data from the *System Health*, *Query Monitor*, *Alert Viewer*, *Metrics Analysis*, and *Workload Monitor* portlets in order to support monitoring and management of Teradata, Aster, and Hadoop systems. To get started with the dashboard, be sure it is enabled in Roles Manager and then look for System Health status icons stacked vertically on the left side of the screen. Hover over a status icon to see the system name and health state. Then to see more detail about a system, click its health icon.

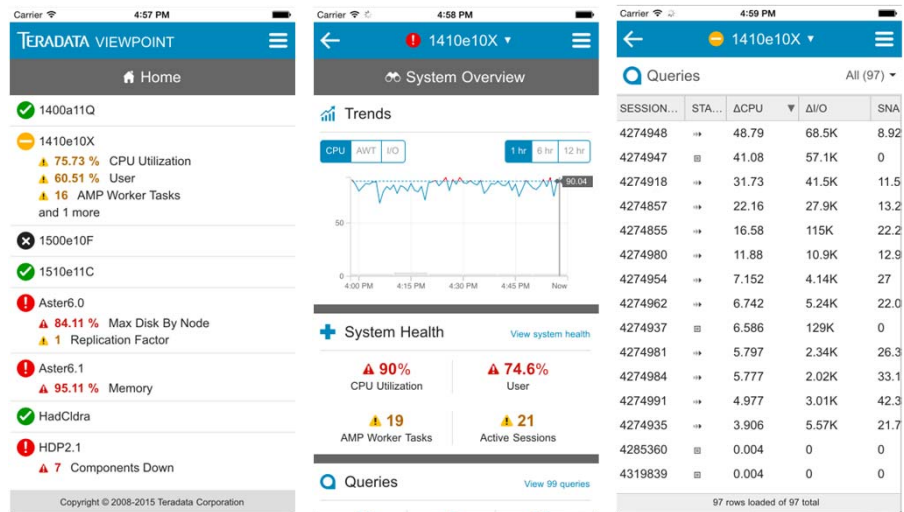
In summary, Dashboard is a new portal feature that provides a high-level view of overall system activity;

- Information on the dashboard:
 - System performance metrics (CPU, I/O, AWT's)
 - System health
 - Workload metrics
 - Query metrics + actions (abort, change workload, release)
 - Alerts
 - Hadoop services
- Permissions
 - Dashboard access is enabled per role
 - Dashboard will only show content to which the user has access in the associated portlets.

Viewpoint Mobile Feature

Web based Viewpoint (15.11) for IOS or Android phones via browser. No download required.

- System performance metrics (CPU, I/O, AWT's)
- System health
- Query metrics + actions (abort, change workload, release)
- Alerts



Imagine you're a DBA on a vacation lying on the beach when your boss calls saying users are complaining the system is running slow. No worries, you have Viewpoint Mobile and via your smartphone you find the issue and correct the problem without getting up from your beach towel.

Introduced in Viewpoint 15.11, Viewpoint Mobile is a new way of monitoring and managing Teradata systems using an interface optimized for smaller devices. Viewpoint Mobile is designed to pull together the most relevant data and features from the *System Health*, *Query Monitor*, *Alert Viewer*, *Metrics Analysis*, and *SQL Scratchpad* portlets in order to support system monitoring and management while on the go. Not just Teradata, but also Aster, or Hadoop systems, Viewpoint Mobile allows easy monitoring of the health of those systems and to perform common management tasks. For example aborting a query or changing the workload for a query on a Teradata DB system.

Viewpoint Mobile is implemented as a web application so it does not require an app download; just open a mobile browser using the same URL normal used. The mobile interface does not simply mirror the standard desktop UI to small devices; rather, it has been tailored to give you the mobile experience you expect:

- A wide range of screens are supported
- Touch based control
- The number and size of files that are sent to the browser have been reduced to increase network performance and to ensure Viewpoint does not eat-up your data plan
- Important data is surfaced making easy to see the information needed without a lot of digging

In Summary, a web based app for IOS or Android phones. No download required.

Summary

Viewpoint is the cornerstone of Vantage Systems Monitoring and Management.

- Provides systems management via a web browser.
- Viewpoint provides a single operational view (SOV) for multiple systems.
- Highly customizable and can be personalized.

The Teradata Viewpoint Management and Self-Service portlets allow the Viewpoint user access to Monitor and interact with System resources. Examples include:

- [Alert Viewer](#) – allows users to view alerts defined for the system
- [Query Manager](#) – allows users to view information about requests
- [System Health](#) – allows users to monitor and display the status of a selected Vantage Advanced SQL Engine system
- [Node Resources](#) – allows users to monitor physical and virtual resources
- [Remote Console](#) – allows users to run many of the Advanced SQL Engine console utilities remotely from within the Teradata Viewpoint portal
- [SQL Scratchpad](#) – allows users to enter queries and retrieve data

This slide summarizes some important concepts regarding this module.



Module 13: Review Questions

1. Which portlet is used to view active queries across multiple systems?
 - a. My Queries
 - b. Query Monitor
 - c. Query Spotlight
 - d. SQL Scratchpad History
2. Which portlet is used to establish TASM filters and throttles?
 - a. Workload Health
 - b. Workload Wizard
 - c. Workload Monitor
 - d. Workload Designer
3. Which portlet can be used to execute the DBSControl utility?
 - a. Productivity
 - b. TVS Monitor
 - c. System Health
 - d. Remote Console
4. Which portlet can be used to display AMP Worker Task usage during different hours?
 - a. Productivity
 - b. System Health
 - c. Metrics Analysis
 - d. Node Resources

Check your understanding of the concepts discussed in this module by completing the review questions as directed by your instructor.



Lab Exercise 13-1 (1 of 2)

Purpose

In this lab, you will use the Teradata Viewpoint "My Queries" portlet and the rewind option.

Tasks

1. Use the viewpoint profile setup to add your Teradata account. Select "Profile" from the Teradata portal menu. Then click the "Teradata Accounts" tab and click the "Add Account" button. Fill in your studentxxx Teradata account information and password.
2. Add "My Queries" portlet.
3. Change displayed columns in "My Queries" portlet to also show "State" via the Table Actions → Configure Columns.
4. Login via Studio and execute the Collect Statistics, SET QUERY_BAND and product join queries on this slide and answer YES to popup regarding 2000 limit. [Make sure your Employee, Department and Job tables are populated before running the query.](#)

```
SET QUERY_BAND = 'ApplicationName=FactoryApp; Version=13.13.13;' FOR SESSION;
SELECT      D.DeptName, E.EmpNumber, E.LastName, E.FirstName, J.JobDesc
FROM        Employee E
INNER JOIN  Department D
ON          E.DeptNumber = D.DeptNumber
OR          E.EmpNumber = D.DeptMgrNumber
INNER JOIN  Job J ON E.JobCode = J.JobCode
ORDER BY    1, 2, 3, 4;
```

The slide contains the start of this lab exercise.

Execute the following commands to ensure your tables are populated.

```
INSERT INTO Employee SELECT * FROM PD.Employee;
INSERT INTO Department SELECT * FROM PD.Department;
INSERT INTO Job SELECT * FROM PD.Job;
```

Once your tables are populated collect statistics by running the following commands.

```
COLLECT STATISTICS COLUMN EmpNumber, COLUMN DeptNumber, COLUMN jobcode on
Employee;

COLLECT STATISTICS COLUMN DeptNumber, COLUMN DeptMgrNumber on Department;

COLLECT STATISTICS COLUMN JobCode on Job;
```

Once your done with the above, execute the SET Query_Band and SELECT statements in the slide within your Studio or Studio Express session. Hint: Make sure that all of the semi-colons are included in the SET QUERY_BAND command.



Lab Exercise 13-1 (2 of 2)

5. Once query has completed wait about 1 more minute.
6. Using the "My Queries" portlet and the Rewind feature go back in time to find your session when it was in the "Active" state for the query you just ran.
7. Then click the session to see the details.
8. Click tabs to see the SQL and EXPLAIN.

Note: Not all queries are captured via Viewpoint. Only the long running ones. Our Viewpoint is configured for "Elapsed seconds > 10, Total CPU seconds > 3)

9. From QUERY BAND tab what is the version number for the FactoryApp? _____
10. Exit rewind.

The slide continues this lab exercise.



Lab Exercise 13-2

Purpose

In this lab, you will use the "Space Usage" portlet to investigate database disk space usage for Teradata Database.

Tasks

1. Add the "Space Usage" portlet.
2. Find your user and determine the Peak Spool % you used. Peak Spool % _____
3. For your user what was the percentage skew for the Employee_2 table that you created and was distributed using LastName as the NUPI? Skew % _____
4. Using "By Vproc" rather than "By Database" which AMP # has the greatest % of space in use and what is that percent? AMP # _____, % _____
5. Is the space usage on our education system well distributed? Yes or No

This slide contains the start of this lab exercise.

Hint: You may need to add a column via the Table Actions → Configure Columns.



Lab Exercise 13-3 (optional)

Purpose

In this lab, you will use the "Metrics Analysis" portlet and rewind back to analyze resource usage trends and export data.

Tasks

1. Add the "Metrics Analysis" portlet.
2. Using portlet settings add these 2 additional graphs, in addition to the default CPU one, "Session Login Count" and "Logical MB/Sec".
3. Modify the default "CPU" graph setting and select "Disk I/O".
4. Set a threshold for "Session Login Count" that is 10 greater than the number of students in the class.
5. Adjust the time interval to the "Last 24 hours".
6. Select the "Session Login Count" metric.
7. Using Rewind, go back in time to locate the Disk I/O and Logical MB/Sec set of graphs - you should that Disk I/O and Logical MB/Sec track each other closely. If the session threshold exceeds the number you set, how is that indicated on the graph? _____
8. Use export option to export the data to a CSV file and then open it in Excel.
9. Exit rewind.

This slide contains the start of this lab exercise.



Module 14: ResUsage – Performance Monitoring

Teradata Vantage Administration

Copyright © 2007–2023 by Teradata. All Rights Reserved.

Objectives

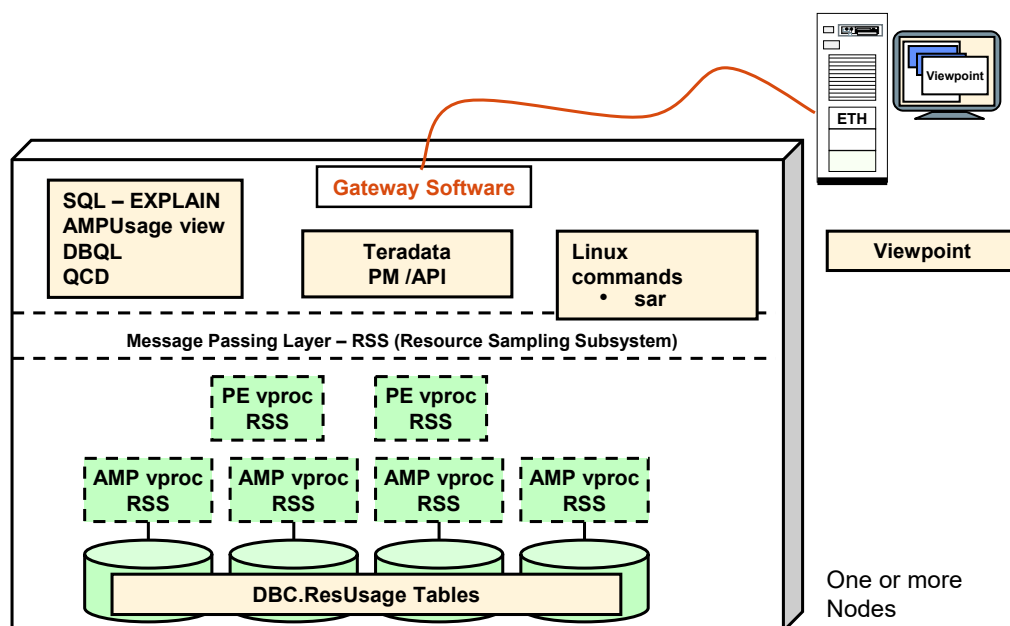
After completing this module, you will be able to:

- Identify the purpose of various ResUsage tables
- List two ways in which ResUsage data can be collected
- Name the two phases of gathering resource usage data and briefly describe them
- Describe the purpose of the Teradata System Emulation Tool

Performance Monitoring Tools

Levels of performance collection:

- System (e.g., ResUsage)
- Session (e.g., Viewpoint)
- Query (e.g., DBQL)



The Teradata database provides several facilities you can use to monitor database performance.

- EXPLAIN statement
- Access logging – record the activity of specific users.
- AMPUsage – a view that provide AMP Usage information for each user and account.
- ResUsage – a set of tables, view, and macros used to monitor system resource usage.
- Teradata PM/API – Performance Monitor / Application Programming Interface
- Teradata Manager – Performance Monitor (formerly PMON)
- Database Console Utilities – (e.g., Showspace displays space utilization)
- Linux tools (e.g., sar)
- TDP User Transaction Monitor (TDPTMON)

For example, you may have to use several of these tools to monitor query performance. You should always have an EXPLAIN report to understand what the query is doing.

SAR and xperfstate

SAR (System Activity Reports) is a UNIX facility to capture performance metrics at the UNIX operating system level.

Xperfstate is a UNIX tool that provides a real-time display of system performance combining some UNIX and Teradata information.

Why Collect Performance Data?

Performance data may be used to:

- Measure system benchmarks.
- Measure component performance.
- Analyze performance degradation and improvement.
- Identify potential performance impact.
- Identify bottlenecks, parallel inefficiencies and other problems.
- Assist on-site job scheduling.
- Plan installations.
- Capacity planning – resource usage data can help determine if system expansion is necessary.

Traditionally, performance data and reports have been the primary diagnostic tool available to analyze performance data on a Teradata system.

This slide shows some uses for ResUsage reports and data.

When considering system expansion and doing capacity planning, some of the activities to consider include:

- Batch windows – how much time is available to perform batch SQL jobs. A larger system with more AMPs usually means that these jobs can run faster. However, if the network and/or channel are the bottleneck, then maybe additional network and/or channel connections may be necessary.
- Backup windows – how much time is available to perform Archive activities? If a disaster occurred and a RESTORE/RECOVERY had to be performed, how long can the system be unavailable before there is negative impact to the business?
- Maintenance Windows - how much time is available to perform batch load/unload activities? A larger system with more AMPs usually means that the load/unload functions will take less time. However, if the network and/or channel is the bottleneck, then maybe additional network and/or channel connections may be necessary.
- Ad-hoc decision support queries – how much spool is needed? When will these be executed

Resource Usage Data

ResUsage data provides historical system-level performance information.

- Data is logged into various ResUsage tables at a specified logging interval.

Two Teradata subsystems work in conjunction with other subsystems to gather ResUsage data:

- Parallel Database Extension (PDE)
- Resource Sampling Subsystem (RSS)

Data collection

- PDE and RSS help to collect data and gather information from the operating system and the Teradata Database software.

Data logging

- RSS writes the collected data to ResUsage tables.

Collections Costs

- Disk Space
- Additional I/Os (minimal)
- Additional CPU overhead (minimal)

ResUsage data gathering is a two-phase process that encompasses data collection and data logging. The ResUsage facility consists of a set of tables, views, and macros to access system metrics.

Two Teradata subsystems work in conjunction with other subsystems to gather ResUsage data:

- Parallel Database Extension (PDE)
- Resource Sampling Subsystem (RSS)

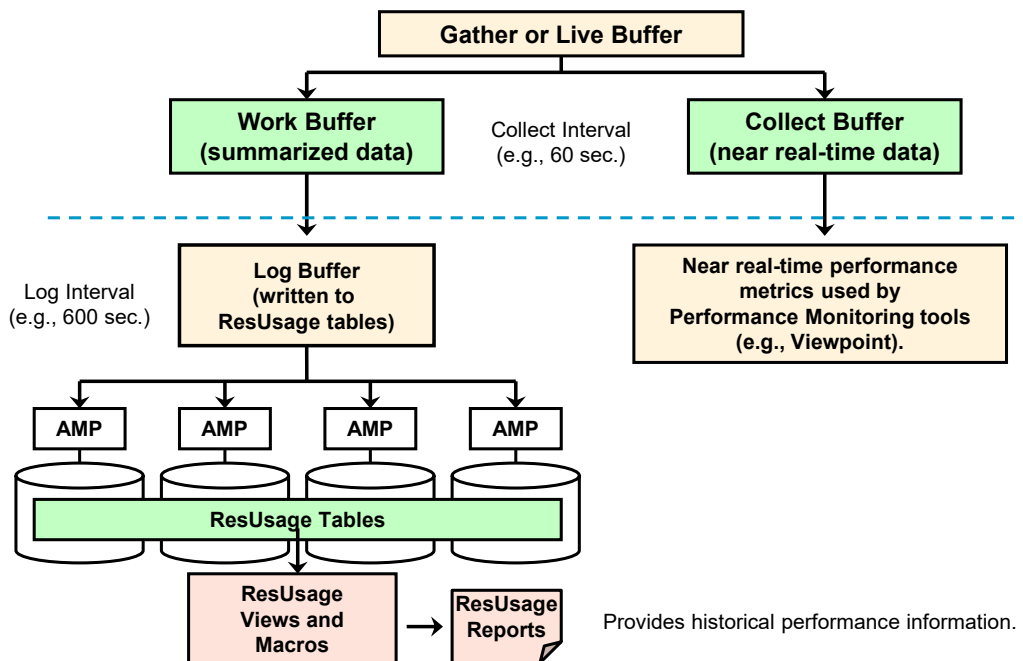
Data Collection - during the data collection phase, both PDE and RSS gather information from the operating system and from Teradata Database. This data is temporarily stored in shared data collection buffers. Data collection continues for a user-specified period of time called the collect interval.

Data Logging - in the logging phase, RSS writes all gathered data to ResUsage tables and reinitializes the shared data collection buffers for the next log interval.

Collection Costs - recording information in the DBC.ResUsage table requires disk space and processing time. Despite the additional resources used in performance monitoring, there are benefits to understanding how your system resources are being used.

Cost - the collection of ResUsage data incurs associated system overhead costs in three areas: I/O capacity, User DBC Perm Space and CPU utilization. Space is usually the largest cost.

Logging ResUsage Data



ResUsage information is gathered in three ways, depending on the nature of the data being collected.

- Counted – the number of times an event happened. The “gather or live” buffer is updated.
- Time monitored – determines how much time was spent in a particular state. The “gather or live” buffer is updated at each state change.
- Tracked data – uses a snap shot of a queue length at the collect period. This information goes directly into the collect buffer.

Collection and Log Buffers

Statistical information is stored in the gather buffer and holds it there until the set collect interval. At that time, the utility moves the data to the work and collect buffers. The collected data is referred to as PM/PC (Performance Monitoring/Production Control) data.

Resource Usage Logging

When you initiate data logging, the system collects a variety of statistics for a period you specify. Teradata stores performance data in the DBC.ResUsage tables. Teradata uses nine tables to gather resource utilization data for a specified time period, and stores this information by node or vproc. The system does not automatically collect resource utilization data. Consequently, you must activate resource collection and logging to gather performance data

Real-time Performance Monitoring

With Viewpoint, you can view and capture data by looking directly into the collect buffer to see what is happening real-time on the system.

Specifying ResUsage Tables and Logging Rates

Tools that can be used to specify the tables to collect and log include:

- Control GDO Editor – `ctl`
- Supervisor commands (e.g., SET LOGTABLE ...) via DB Window

Example of specifying tables to log system data using the "ctl" utility.

```
# ctl
> screen rss
> 1=on

(0) Node Logging Rate : 600 sec

RSS Table Logging Enable
(1) SPMA : On   (2) IPMA : Off   (3) SCPU: On
(4) SVPR : On   (5) IVPR : Off   (6) SLDV: On   (7) SHST: On
(8) SPDSK: On   (9) SVDSK: On   (A) SAWT: On   (B) SPS : On

RSS Summary Mode Enable
Summarize SPMA: Off   Summarize IPMA : Off   (C) Summarize SCPU : Off
(D) Summarize SVPR: Off   (E) Summarize IVPR : Off   (F) Summarize SLDV : Off
(G) Summarize SHST: Off   (H) Summarize SPDSK: Off   (I) Summarize SVDSK: Off
(J) Summarize SAWT: Off   (K) Summarize SPS : Off

> write

CTL: Control GDO successfully written.
```

Various tools can be used to specify which tables to collect and log information into. These include:

- `ctl` utility (Linux command line)
- Supervisor commands (e.g., SET LOGTABLE ...) via DB Window

Note: Using the supervisor commands in the Database Window, you can enable tables and set collection and logging, but you cannot set Summary mode.

- You can display the collection rate as shown below. Using `ctl`, ...

```
> print rss collection rate
RSS Collection Rate=30
> print vproc logging rate
Vproc Logging Rate=0
```

The following command (from root prompt) will set the collection rate to 60 seconds and the logging rate to 600 seconds.

```
ctl -first "RSS Collection Rate = 60; node logging rate=600; screen rss; write; quit"
```

Specifying Tables using Supervisor Commands

You can also set resource logging from the Database Window Supervisor screen. The **set** and **get resource** commands may be used: **set logtable *tablename_or_ALL* ON/OFF**

Resource Usage Tables

General Recommendations for system performance metric collection:

- Enable all of the System(S) tables and nothing should be collected in Summary mode.
- Active row filters should be turned ON for SAWT and SPS tables.

Node Data	ResUsageSpma	System-wide node information	
	ResUsageIpma	System-wide internal node information	Generally not enabled
CPU Data	ResUsageScpu	Information specific to the CPUs in a node	
VProc Data	ResUsageSvpr	Data specific to each virtual processor	
	ResUsageLvpr	System-wide internal vproc information	Generally not enabled
Host and LAN Data	ResUsageShst	Host channel and LAN traffic	
Logical Device Data	ResUsageSldv	Logical Device – information specific to disk I/O	
Additional tables	ResUsageSawt	Collects and reports statistics about the AWTs	
	ResUsageSpdisk	Provides AMP-level Pdisk statistics	
	ResUsageSvdsk	Provides AMP-level Vdisk statistics	
	ResUsageSps	Priority Scheduler details & information	

Teradata stores ResUsage data in a set of system tables. Each ResUsage macro derives its report from one or more of these tables. You must activate logging to produce a useful ResUsage report.

All ResUsage table names begin with ResUsage and have the following extensions.

S: System or I: Internal (of interest mainly to Teradata development personnel)

pma: node information (PMA – Processor Module Assembly)

cpu: cpu-specific information

vpr: vproc information

ldv: logical device information

hst: Channel and LAN host information

The following tables are new starting with Teradata 12.0.

awt: AMP Worker Task information

ps: Priority Scheduler performance group information

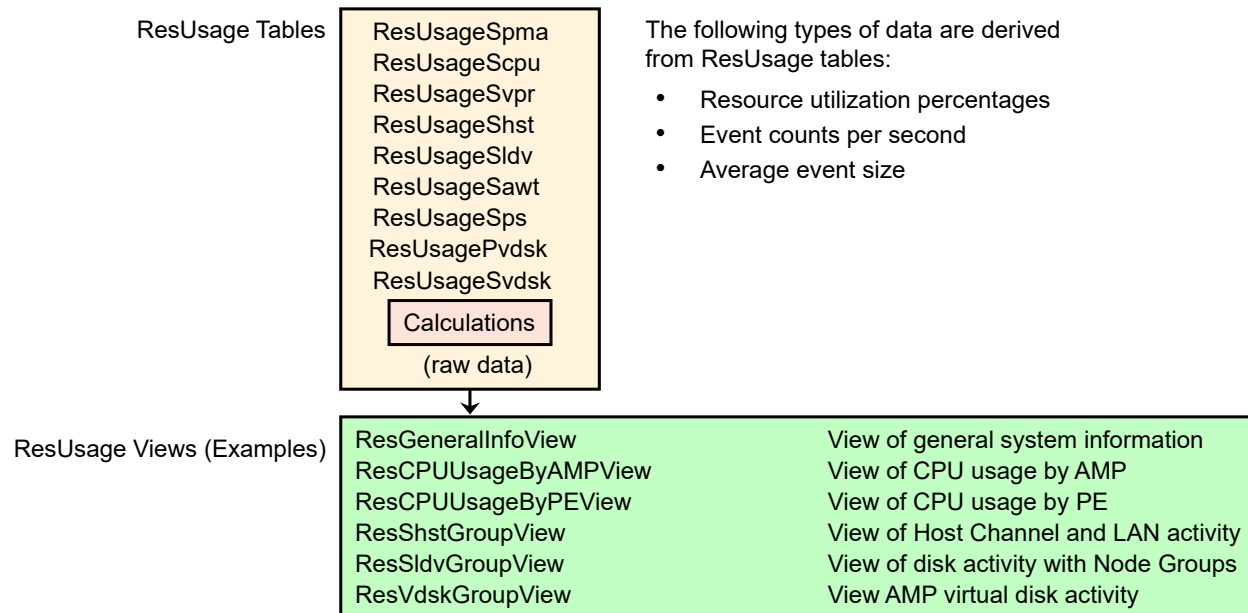
pdisk: AMP pdisk cylinder allocation, migration, and I/O statistics

vdsk: AMP vdisk cylinder allocation, migration, and I/O statistics

Miscellaneous notes:

- All of the ResUsage tables are located in the DBC database.
- The DBC.ResUsageSobj table exists in V2R5, but is currently no used.

Resource Usage Views



Each row in the ResUsage tables represents activity during one logging period; the same is true of each row in the views. The views derive values from data in ResUsage tables.

ResGeneralInfoView

The ResGeneralInfoView provides an overview of system operation. Contains data from ResUsageSpma covering CPUs, disks, and BYNET information.

ResCPUUsageByAMPView

Contains data from ResUsageSvpr detailing the ways the CPUs are used by the AMPs.

ResCPUUsageByPEView

Contains data from ResUsageSvpr detailing the ways the CPUs are used by the PEs.

ResShstGroupView

The ResShstGroupView is based on the ResUsageShst table.

ResSldvGroupView

The ResSldvGroupView is based on the ResUsageSldv table.

ResSvdskGroupView

The ResSvdskGroupView is based on the ResUsageSvdsktable.

Resource Usage Macros

The ResUsage facility provides macros to report information about Teradata.

<u>One-node Macros</u>	<u>Multiple-node Macros</u>	<u>Group-node Macros*</u>	<u>All-node Macros</u>
	ResAwtByAMP		ResAwt
	ResAwtByNode	ResAmpCpuByGroup	
ResCPUByAMPOneNode	ResCPUByAMP	ResPeCpuByGroup	
ResCPUByPEOneNode	ResCPUByPE	ResCPUByGroup	
ResCPUOneNode	ResCPUByNode	ResHostByGroup	
ResHostOneNode	ResHostByLink	ResLdvByGroup	
ResLvdOneNode	ResLdvByNode	ResMemByGroup	
ResMemMgmtOneNode	ResMemMgmtByNode	ResNetByGroup	
ResNetOneNode	ResNetByNode	ResNodeByGroup	ResNode
ResOneNode	ResNodeByNode	ResPdiskByGroup	
ResPdiskOneNode	ResPdiskByNode	ResVdiskByGroup	
ResVdiskOneNode	ResVdiskByNode	ResPsByGroup	
	ResPsByNode		

General Macro Syntax:

EXEC MacroName (FromDate, ToDate, FromTime, ToTime [, additional parameters depend on macro];

Example using ResNode macro:

EXEC DBC.ResNode (Date-7, Date,,);

This generates data from one week ago to today using the ResNode macro.

* Group-node macros are designed for co-existence systems.

Resource usage macros produce reports from data collected in the resource usage tables. You can use the reports to analyze key operational statistics and evaluate the performance of your system. Like other macros, resource usage macros consist of one or more Teradata SQL statements stored in the Teradata database and executed by a single EXECUTE statement.

The macros are installed in the DBC database by DIP. You can run these macros after logging ResUsage data on a specific job or set of jobs.

There are different macros for one node, multiple nodes, a group of nodes, or all nodes. Examples:

Macro

ResCPUByAmp
ResCPUByPE
ResCPUByNode
ResHostByLink
ResLdvByNode

Information Provided

CPU utilization of each AMP vproc.
CPU utilization of each PE vproc.
CPU utilization of the node.
Host traffic for each communication link.
Logical device traffic channeled through the node by totaling its controller links into one summarized node output line.

Examples of ResNode macros that summarize resource usage are:

Macro

ResNode
ResOneNode

Provides summary of ResUsage...

Averaged across all nodes
For a specific node

Example Output from DBC.ResNode

```
EXEC DBC.resnode ('2020-03-27', '2020-03-27', '01:30:00', '02:30:00');
```

Date	Time	CPU Bsy %	CPU Eff %	WIO %	Ldv IOs /Sec	Ldv Eff %	...
3/27/2020	01:30:00	0.74	100.00	0.81	2,364.59	100.00	...
3/27/2020	01:40:00	43.68	100.00	0.67	6,094.12	100.00	...
3/27/2020	01:50:00	94.18	100.00	0.05	3,788.97	100.00	...
3/27/2020	02:00:00	87.88	100.00	0.16	2,433.98	100.00	...
3/27/2020	02:10:00	0.60	100.00	0.62	307.69	100.00	...
3/27/2020	02:20:00	0.78	100.00	0.72	365.05	100.00	...
3/27/2020	02:30:00	0.52	100.00	0.65	233.20	100.00	...

For the DBC.ResNode macro to display data, logging must be enabled on ResUsageSpma.

Notes about columns shown in this output:

- CPU Busy - % of time the CPUs are busy; based on average CPU usage per node.
- CPU Efficiency - parallel efficiency of node CPU usage; parallel efficiency is calculated by dividing average node utilization by maximum node utilization.
- WIO % - Percent of time the CPUs are idle and waiting for completion of an I/O operation.
- Ldv IOs /Sec - average number of logical device (disk) reads and writes per second per node.
- Ldv Eff % - parallel efficiency of the logical device (disk) I/Os.

This slide contains an example of general ResUsage Summary information across all nodes. There are 23 columns (including date and time) with the ResNode report. The first 5 columns (after the date and time) represent:

CPU Bsy%	Percent of time the CPUs are busy, based on average CPU usage per node
CPU Eff%	Parallel efficiency of node CPU usage. Parallel efficiency is the total percent of time nodes are busy. It is the average for all nodes of total busy divided by the total busy time of the busiest node.
WIO %	Percent of time the CPUs are idle and waiting for completion of an I/O operation.
Ldv IOs /Sec	Average number of logical device (disk) reads and writes per second per node.
Ldv Eff %	Parallel efficiency of the logical device (disk) I/Os. It is the average number of I/Os per node divided by the number of I/Os performed by the node with the most I/Os.

If a system's resources are (CPU and I/O) are heavily utilized, it may be necessary to add system resources. The DBC.ResNode macro can be used to provide before and after results.

One of the metrics measured is "skew" (not shown on this slide). "System skew" is calculated as the difference between the max and all AMPs. "Query skew" is calculated as the difference between the max and active AMPs.

The example on this slide was captured for a Teradata 6690 one-node (plus one HSN) system.

PM/API and Viewpoint

The PM/API (Performance Monitor/Application Programming Interface) ...

- is part of Teradata software and has low overhead.
- provides real-time monitoring capability and session information.
- provides the following data:

Processor Data

- Collects/reports node/vproc usage for single period.
- New period overwrites data from previous period.
- Collection is not cumulative.

Session-level Data

- Collects/reports session-level data cumulatively.
- New sampling period increases collected data.
- Accessed via applications such as Teradata Viewpoint.

Teradata Viewpoint

- Web-based application that utilizes portlets to ...
 - provide real-time performance monitoring.
 - show how efficiently the Teradata database is using its resources.
 - identify problem sessions and users.
 - abort sessions and users having a negative impact on system performance.

The PM/API (Performance Monitor/Application Programming Interface) facility provides real-time performance-monitoring that allows an application to collect and return Teradata performance data.

Teradata Viewpoint is a web-based application that uses the PM/API to provide real-time performance and session information (PM/PC) which has been collected.

How PM/API Collects Data

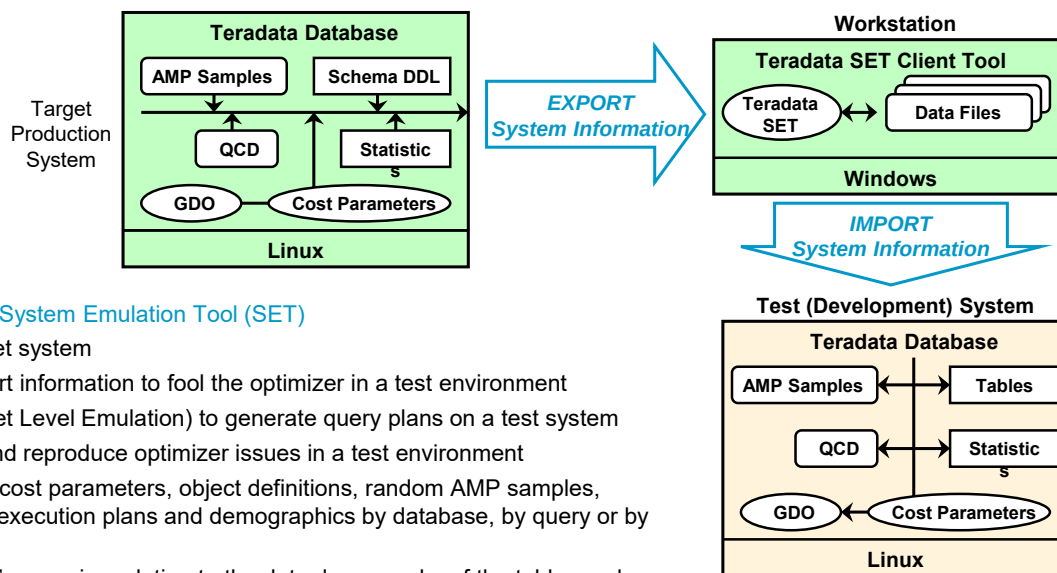
PM/API contains monitoring commands that you issue through a logon partition called MONITOR. MONITOR collects different types of performance data, including the current system configuration; resource usage and status of individual nodes or vprocs; and of individual sessions.

PM/API collects data in memory, not in a spool file on disk. As a result, PM/API routines (except the IDENTIFY command) cannot be blocked and consequently incur low overhead. PM/API stores node and vproc resource usage data and session-level usage data in separate collection areas. The data stored in memory is updated once during each sampling period. All users share the collected data. The MONITOR partition collects and reports resource usage data differently from session-level usage data. To interpret the information that the MONITOR returns, you must understand the difference.

Collecting and Reporting Resource Usage Data

PM/API collects and reports node and vproc usage data for a single sample period. For example, a user sets the sampling period to 120 seconds. Then she issues the MONITOR RESOURCE request. The system collects node and/or vproc usage data during the next 120 seconds.

Teradata System Emulation Tool



Features of Teradata System Emulation Tool (SET)

- Emulates a target system
- Export and import information to fool the optimizer in a test environment
- Uses TLE (Target Level Emulation) to generate query plans on a test system
- Verify queries and reproduce optimizer issues in a test environment
- Capture system cost parameters, object definitions, random AMP samples, statistics, query execution plans and demographics by database, by query or by workload
- Perform "what-if" scenarios relating to the data demography of the tables and system

The Teradata System Emulation Tool simplifies the task of emulating a target system by providing the ability to export and import all of the information necessary to fake out the optimizer in a test environment. This information can be used along with Teradata's Target Level Emulation feature to generate query plans on the test system as if they were run on the target system. This feature is useful for verifying queries and reproducing optimizer related issues in a test environment.

Teradata SET allows the user to capture system cost parameters, object definitions, random AMP samples, statistics, query execution plans and demographics by database, by query or by workload. This tool does not export user data. Upon import the user can customize or edit object definitions, random AMP samples, statistics and cost parameters. The Customize feature allows the user to perform "what-if" scenarios relating to the data demography of the tables and system performance parameters. Teradata SET also has an option to log SQL statements. The user can view the log directly from the Teradata SET window to troubleshoot any failures that occur during export or import operations.

Summary

- Resource usage ([ResUsage](#)) data and reports can help you to improve system performance and management.
- Various tools exist to specify the tables to collect and log as well as setting the collection and log rates.
- You can access the ResUsage statistics stored in the ResUsage tables directly or use supplied views and macros.
- The [PM/API facility](#) is an application programming interface that can be used to provides a real-time monitoring capability and session information.
- [Teradata Viewpoint](#) is a web-based interface that utilizes portlets to assist in monitoring the resources of a system.

This slide summarizes some important concepts regarding this module.

An easy way to enable ResUsage logging using cnstern is the following commands:

- set logtable all on
- set logtable ipma off
- set logtable ivpr off
- set session collect 60
- set resource collection 60 logging 600



Module 14: Review Questions

1. What are two methods of setting the ResUsage logging intervals?

Match the following tools to its description.

- | | |
|-----------------------------|---------------------------------------|
| 2. _____ ResUsage tables | a. Set logging rates |
| 3. _____ Teradata Viewpoint | b. Emulates a target system |
| 4. _____ Teradata SET | c. Provides session level information |
| 5. _____ ctl | d. Holds historical resource data |

Check your understanding of the concepts discussed in this module by completing the review questions as directed by your instructor.



Module 15: System Restarts

Teradata Vantage Administration

Copyright © 2007–2023 by Teradata. All Rights Reserved.

Objectives

After completing this module, you will be able to:

- List three different ways to restart the Teradata database
- Use the RESTART command
- Describe the impact of ...
 - Drive(s) failure
 - Disk array controller(s) failure
 - BYNET(s) failure
 - Node failure
 - SWS failure
 - VPROC failure

Types of Restarts

Scheduled Restarts

- Changing system parameters (e.g., DBS Control parameter is updated)
- Software upgrades
- Configuration changes (addition of new AMPs and/or PEs)

Unscheduled Restarts

- Power failure (e.g., 8/14/2003 – the North East U.S. and parts of Canada)
- Disasters (e.g., 10/22/2007 – Rancho Bernardo fires; 10/28/2012 – Sandy Hurricane)
- Hardware failure
- Software failure
- Accidents

Restart Processes

1. Spool cylinders are returned to free cylinder list (unused cylinder pool).
2. Before logons are enabled, uncommitted work is rolled back.
 - 1st Tables are re-locked for background recovery.
 - 2nd Logons are enabled in cold start.

There are two types of restarts on a Teradata Database:

- Scheduled restarts
- Unscheduled restarts

Scheduled Restart/User-initiated Restart

In a scheduled or user-initiated restart, use the RESTART command from either the DBW Supervisor window or from **vprocmanager** to restart the system.

Unscheduled Restart/Automatic Restart

In an unscheduled restart or automatic restart, the system reboots without user input.

This slide provides examples of when you might need to perform scheduled restarts, and under what conditions you might encounter unscheduled restarts.

Scheduled Restarts

Restart Teradata with	Use this command	Options
Command-line	tpareset <comment>	-f, -x, -y, -d, -l, -Q, -P
Vprocmanager	restart	cold, coldwait
DB Console - Supervisor	restart tpa <comment>	cold, coldwait

Example:

```
# tpareset -f Change of system parameters
```

To see any restarts that have occurred in the last 90 days:

```
.LOGON tdpid/systemfe,service;
EXEC ALLRESTARTS (DATE - 90,);
.LOGOFF;
```

The “tpatrace” command may also be used to see information about restarts.

```
# tpatrace 3    (shows last three restarts)
```

This slide shows the windows and utilities from which you can restart the Teradata Database system, the necessary commands and available restart options.

-f option

The -f option, used with the tpareset command, forces all TPA nodes to participate in the tpareset regardless of their current state, without rebooting Linux.

-x option

The -x option allows you to shut down the Teradata Database on the entire system without shutting down the operating system. This option does not automatically restart Teradata.

-y option

The -y option automatically answers yes to the confirmation prompt.

-d option

Specifies that a DBS dump be taken before doing the restart.

-l delay option

Specifies the delay interval in seconds to wait for other nodes to join the TPA configuration. This parameter controls how long a node will wait during the BYNET configuration phase of PDE initialization for other nodes to reach that point before continuing on without them.

Restart Using the “tpareset” Command

Example of using the tpareset command:

```
# tpareset -f Change of DBSControl parameters
```

```
    You are about to restart the database
    on the system
    'TDLabs'
```

```
    Do you wish to continue (default: n) [y,n]
```

```
tpareset: TPA reset submitted.
```

Example of using the tpatrace command:

```
# tpatrace
```

```
TPA Initialization Trace for System TDLabs Node 001-01
```

```
10/12/20 07:35:35 ----- PDE restarting    TPA cycle = 9
10/12/20 07:35:35.95 (0/0 13c5) ---- PDE starting.
10/12/20 07:35:35.95 (0/0 13c5) State is START/BEGIN.
:
10/12/20 07:35:36.07 (0/0 13c5) State is START/RECONCILE.
10/12/20 07:35:38.03 (0/0 13c5) State is START/NETREADY.
:
10/12/20 07:35:46.66 (0/0 13c5) Crash count/ceiling = 0/3
10/12/20 07:35:46.66 (0/0 13c5) PDE started in 11 seconds.
```

Restart information is logged to Linux and to Teradata:

- Linux messages log – /var/log/messages
- DBC.Sw_Event_Log table – use the following view to select Teradata restart entries

```
SELECT * FROM
DBC.Software_Event_LogV WHERE
TheFunction LIKE 'tpareset%';
```

The **tpareset** command can be used to restart the Teradata database. Common options used with **tpareset** include:

- f option** The -f option, used with the tpareset command, forces all TPA nodes to participate in the tpareset regardless of their current state, without rebooting Linux.
- x option** The -x option allows you to shut down the Teradata Database on the entire system without shutting down the operating system. This command shuts down Teradata without restarting Teradata.

Restart information is logged in numerous locations depending on the operating system. For example, with older UNIX MP-RAS systems, the following locations are utilized.

- SW_Event_Log table – view with Software_EventLog[V] view
- Console Log (e.g., /etc/.osm)
- /var/adm/streams (MP-RAS)

PDE States

To display the DBS and PDE state without opening the vprocmanager utility.

```
# tdatcmd (sets the path for Teradata command-line utilities)
# vprocmanager -s
DBS State: Logons are enabled - Users are logged on
PDE State: RUN/STARTED
```

The pdestate command displays the PDE state for a specific node.

```
# psh pdestate -a
<----- byn001-13 ----->
PDE state is RUN/STARTED.
DBS state is 4: Logons are enabled - Users are logged on
<----- byn001-14 ----->
PDE state: DOWN/STANDBY
```

OpenPDE (Linux/Windows) has five primary operational states (DOWN, START, RUN, RESET, and STOP):

- **DOWN** – no PDE services are currently running,
- **START** – this is a transitory state between DOWN and RUN
- **RUN** – all PDE services are fully available and the TPA is started.

The **/ntos/bin/pdestate** command can be used to check the current state of the PDE and Teradata software. The states have different names depending on the operating system environment.

- Open PDE (Linux/Windows) has five major operational states: DOWN, START, RUN, RESET, and STOP.
- PDE (UNIX MP-RAS) has three major operational states: NULL, NOTPA, and TPA. When PDE starts up, it transitions between them in that order. The sub-states correspond to the different phases of startup.

The major states with Linux include:

DOWN – no PDE services are currently running, other than those that are required to switch out of this state, i.e., startup of the database.

START – this is a transitory state between DOWN and RUN that may be seen for a brief time during database startup.

RUN – all PDE services are fully available and the TPA is started. Once this point has been reached, Teradata RDBMS can go through its initialization.

Unscheduled Restarts

Drive Failures

Drive Failures:

Scenario 1

Failure: One drive in a drive group (assume RAID 1 or RAID 6)
Result: No TPA reset
Resolution: Replace drive – Array Controllers automatically rebuild the drive

Scenario 2

Failure: Two drives in a RAID 1 drive group or three drives in a RAID 6 drive group
Result:

- TPA reset (1-5 minutes)
- AMP taken offline and marked as Fatal
- Fallback tables OK
- Non-fallback tables partially available

Resolution:

- Replace the two drives
- Reformat LUNs or Volumes in the drive group
- Perform a table rebuild
- Restore non-fallback tables

Scenario 3

Failure: Two drives in two different drive groups (RAID 1) associated with AMPs in the same cluster – 2 AMPs fail in a cluster
Result: Machine halts
Resolution: Restore User DBC and tables

Drive Failure

When a drive fails, there may be a loss of data. Tables with fallback protection continue to be 100% available. Tables without fallback protection will only be partially available.

An AMP will attempt 5 retries to a disk array before determining that it cannot access the array or its associated Vdisk.

Unscheduled Restarts (cont.)

BYNET Failures:

Scenario 1

Failure:	One BYNET fails
Result:	– No TPA reset – All traffic auto-switched to remaining BYNET – Impact on system performance
Resolution:	Repair BYNET

Scenario 2

Failure:	Both BYNETs fail
Result:	Teradata halts and is not available
Resolution:	Repair BYNETs

BYNET Failure

If a BYNET fails, processing will resume on the other BYNET. Performance will be impacted.

Unscheduled Restarts (cont.)

Node Failure:

Scenario

Failure: Node Fails (e.g., Linux hangs, 2 power supplies fail, memory fails, etc.)
Result: – TPA restart (1 - 5 minutes) and vprocs migrate to other nodes in clique
 – Possible Linux reboot (3 - 15 minutes)
Resolution: – Repair node and reboot operating system
 – Restart Teradata to allow node to rejoin Teradata configuration

Vproc Software Failure:

Scenario

Failure: AMP or PE Vproc fails
Result: TPA restart (1 - 5 minutes) and vprocs may be marked offline
Resolution: If necessary, run Scandisk, Checktable, and Rebuild utilities

SWS Failure:

Scenario

Failure: SWS fails
Result: No restart of Teradata; SWS is not available to monitor/manage system
Resolution: Reboot or recover SWS

Node Failure

This slide describes a node failure.

VPROC Failure

PE VPROC This type of failure produces very little impact on system performance. It reduces the maximum number of sessions that can be active at one time, however, and logons may take longer when a PE vproc is down.

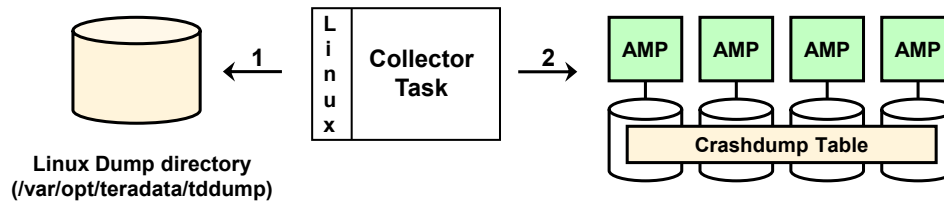
AMP VPROC If a single vproc fails in one or more clusters, the system can continue servicing users with the other AMP vprocs. However, the performance level drops causing a slow down in performance and response time.

If two or more AMP vprocs fail in a single cluster, it halts the database system. All processing stops until the administrator brings at least one of the down vprocs back on-line.

SWS Failure

This slide describes an SWS failure.

TPA Reset – Crashdumps



1. Selective memory and swapped pages are written to “pdedump” space.
2. As part of Teradata restart, a background collector task reads “pdedump” and writes dump information to a Crashdump table in Crashdumps database.
 - If the Crashdumps database is out of perm space, the collector task outputs a warning message and retries every 60 minutes to create a crashdump table.

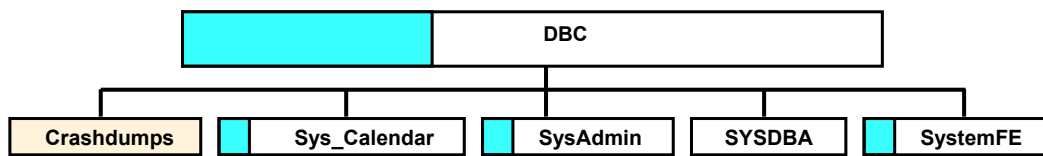
When there is an unscheduled TPA reset, a crashdump is generated (PDE dump).

PDE DUMP

A PDE dump is a selective dump of system memory; including only information that might be needed to analyze a problem within PDE or TPA (the only one currently is the Teradata Database). It can also contain pages read in from swap space that are not in memory at the time of the dump. Exact contents vary depending on the cause of the dump.

PDE dumps, being selective, vary in size depending on the system configuration and the cause of the crash. PDE dumps are much smaller than memory. PDE dumps are taken in parallel on all nodes.

Allocating Crashdumps Space



Use the following formula to estimate the amount of space to allocate:

Example: Four-node system, 96 GB memory/node, and space for 3 Crashdumps.

$$(4 \times 96 \text{ GB}) \times 3 = 1152 \text{ GB without fallback} - \text{double for fallback if needed}$$

MODIFY USER Crashdumps AS PERM = 1152E9;

Note: Normally, a crashdump uses only 50% of node memory; for initial sizing purposes, allocate enough space for an entire memory image per node without fallback.

Example of Crashdump name: Crash_20110930_143249_20
 (Date) (Time) (Segment #)

Help USER Crashdumps;

Table/View/Macro name	Kind	Comment
Crash_20110930_143249_20	T	PDE:13.10.00.05,TDBMS:13.10.00.05,TGTW:13.10.00.05;

Teradata recommends you allocate enough space to this database to hold three crash dumps. Dump size can vary depending on the number of vprocs running, how busy the system was at the time of the crash, and a number of other factors. If a site needs additional space for the Crashdumps database, increase the MaxPerm space by submitting the MODIFY USER statement.

Example

This slide illustrates how to calculate the appropriate amount of permanent space for the Crashdumps database.

A site has a four-node system. The administrator needs to allocate enough space for three crash dumps. The formula is:

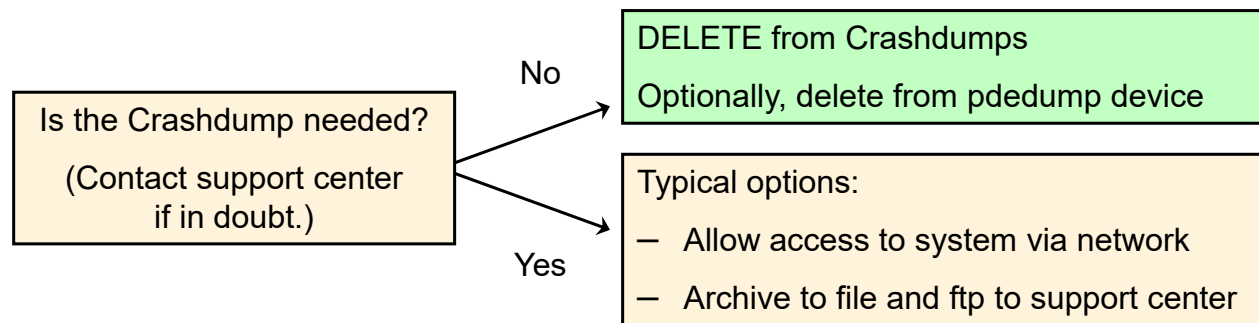
$$\begin{aligned} (4 \times 96 \text{ GB} \times 3) &= 1152 \text{ GB without fallback} \\ (4 \times 96 \text{ GB} \times 3) \times 2 &= 2304 \text{ GB with fallback} \end{aligned}$$

Since a crash dump is normally about 50% of actual memory, the above numbers can be divided in half. You should monitor the actual Crashdumps space usage and adjust it up or down as appropriate, depending on the size of the dumps a site typically gets.

To modify Crashdumps space, you need to log on to the system as user DBC and submit the following SQL statement:

MODIFY USER Crashdumps AS PERM = 1152E9;

TPA Dump Maintenance



This slide describes tasks for maintaining TPA dumps.

Linux Operating (Panic) Dumps

A Linux operating system dump is a complete dump of system memory including PDE and Kernel information, but only for the node(s) that panicked. If Linux panics on multiple nodes, you get a separate dump for each one.

Since this is a complete dump of memory, dump size is equal to the memory size on your system.



Module 15: Review Questions

1. What is the operating system command to restart Teradata? _____
2. What is the DB Window supervisor command to restart Teradata?

3. Which of the following choices will cause a Teradata restart?
 - a. SWS hard drive failure
 - b. Single drive failure in RAID 1 drive group
 - c. Two drive failures in same RAID 1 drive group
 - d. Single power supply failure in a TPA node
 - e. TPA node CPU failure
 - f. One of BYNETs fails
 - g. LAN connection to SMP is lost

Check your understanding of the concepts discussed in this module by completing the review questions as directed by your instructor.



Module 16: System and Maintenance Utilities

Teradata Vantage Administration

Copyright © 2007–2023 by Teradata. All Rights Reserved.

Objectives

After completing this module, you will be able to:

- List two ways in which a system utility can be started
- Explain how to use the following utilities to maintain a Teradata database:
 - DBS Control
 - Ferret Scandisk
 - Checktable
 - Table Rebuild
 - Recovery Manager
 - Showlocks
 - UpdateSpace
 - Vprocmanager
- List the order in which to execute the Checktable and Scandisk utilities

Starting Teradata System Utilities

Typical interfaces to start Teradata System Utilities are:

- Command line
Linux and Windows
- Database Window
Linux (X Windows) and Windows (Database Window)
- Remote Console
Viewpoint
- Host Utility Console (HUTCNS)
Mainframe operating system

Linux	
Command Line	Execute " tdatcmd " – adds the directories to the path to execute command-line utilities.
Database Window	This Database Window is an X client program that requires an X server to be running on the local machine. example: <code>xdbw -display 192.0.2.24:0.0 &</code>

Windows	
Command Line	Start the Teradata Command Prompt in Windows – adds the Teradata directories to the PATH environment variable Start>Programs>Teradata Database>Teradata Command Prompt
Database Window	Open Database Window (DBW) from the Windows Start menu: Start>Programs>Teradata Database>Database Window

The Teradata database offers several user interfaces from which the utilities may be started as shown on this slide. Note that not all utilities support all the user interfaces.

Once a utility is started, some utilities present their own interactive command-line or graphical user interfaces. These utilities let you browse and enter information until you explicitly exit the utility.

Other utilities present their own interactive command-line or graphical user interfaces. These utilities continue running until they are explicitly stopped by the user. Many utilities that present their own command environment are stopped by entering the QUIT command.

Utilities that present a graphical user interface are usually stopped by clicking the Exit or Close command from the graphical menu.

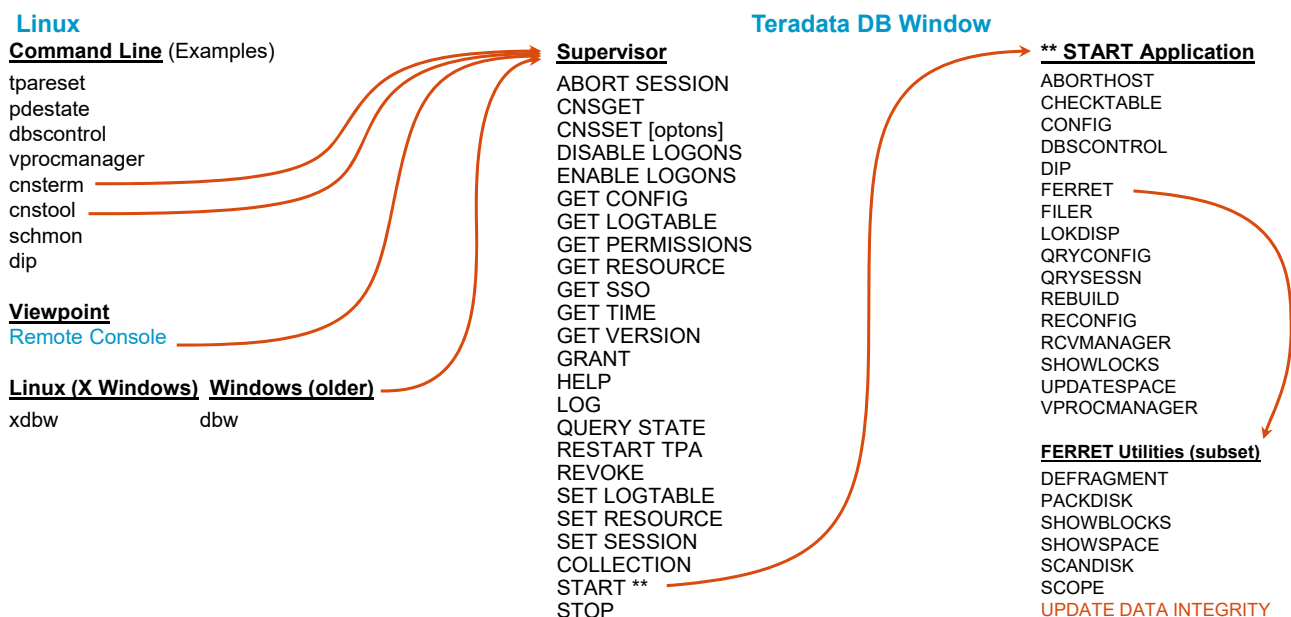
Teradata Console Task

The Teradata Console (CNS) task is responsible for managing the Teradata DB Window.

There are numerous ways in which the CNS task can be invoked.

- xdbw or the Teradata Database window
- Command-line utilities
- cnstern (command-line interface to access supervisor)
- cnstool (command-line interface to access supervisor)

Using the Interfaces to Access Utilities



This slide lists various Teradata control and support utilities and from where they are executed. These functions can be secured through the SMP **cnspers** and **rhosts** files. This **cnspers** file contains 4 fields separated by colons. The format is:

A:B:C:D where A = linux_userid_name@hostname
 B = ALL or a list of allowed supervisor commands
 C = Utilities the user is allowed to start
 D = Utilities the user is NOT allowed to start

List of commands or utilities is space separated. Field D is ignored if field C is present.

This file is maintained by DB Window Supervisor commands.

- GET PERMISSIONS
- GRANT
- REVOKE

This file is automatically copied to all nodes by PDE software. Only execute these commands when all SMP nodes are available so that all copies are updated. Note that the HELP and GET commands require no permissions (these are always accepted).

cnstool and cnstern commands

cnstool and **cnstern** are Linux command-line interfaces to the Teradata DB Console functions.

DBS Control Utility

The DBS Control utility is used to view/modify the DBS Control Record fields which:

- Establish system values
- Tune performance
- Debug/diagnose problems

There are multiple ways to access the DBS Control utility.

1. Command line – `dbsscontrol`
2. Viewpoint Remote Console – select DBS Control
3. Supervisor – `START DBSSCONTROL`

DBS Control fields are divided into categories. Key categories include:

- General
- File System
- Performance
- Checksum
- Compression
- Statistics
- Storage (used if temperature based compression is enabled or if TVS is licensed)

DISPLAY or HELP commands

- `DISPLAY GENERAL | FILESYS | PERFORMANCE | COMPRESSION | ...`
- `HELP GENERAL | FILESYS | PERFORMANCE | COMPRESSION | ...`

The DBS Control utility is used to view/modify the DBS Control record fields which:

- Establish system values
- Tune performance
- Debug/diagnose problems

DBSSControl Record fields are logically grouped based on how the Teradata Database uses them. The physical position of the field in the record is not significant. The group names are defined as follows:

This group...	Contains fields used...
General	For various purposes by the Teradata Database
File System	By Teradata Database V2 File System
Performance	For tuning performance
CheckSum	Set of parameters for CHECKSUM feature
Compression	Used with block level compression
Statistics	Provides default for auto statistics collection
Storage	Used with temperature based compression or TVS
Internal	Used to set internal parameters

There are multiple ways to access the DBSSControl utility:

1. Command line – `dbsscontrol`
2. Viewpoint – Remote Console (select DBS Control)
3. Supervisor – `START DBSSCONTROL`

DBS Control Record – General Fields

1. Version	= 11	23. Export Width Table ID	= 0	45. NewHashBucketSize	= 20 bits
2. SysInit	= TRUE	24. DBQL Log Last Resp	= FALSE	46. MaxLoadAWT	= 0
3. DeadLockTimeOut	= 240 (seconds)	25. DBQL Options (spec. options)	= 0	47. MaxRowHashBlocksPercent	= 50%
4. (Reserved for future use)		26. External Authentication	= 0 (On)	48. MonSesCPUNormalization	= FALSE
5. HashFuncDBC	= 5 (Universal)	27. IdCol Batch Size	= 1000000	49. TempLargePageSize	= 64K Bytes
6. (Reserved for future use)		28. LockLogger Delay Filter	= FALSE	50. RepCacheSegSize	= 512K bytes
7. (Reserved for future use)		29. LockLogger Delay Filter Time	= 0	51. MaxDownRegions	= 6 (default)
8. SessionMode	= 0 (Teradata)	30. ObjectUseCountCollectRate	= 10 min.	52. MPS_IncludePEOnlyNodes	= FALSE
9. LockLogger	= TRUE	31. LockLogSegmentSize	= 64 KB	53. PrimaryIndexDefault	= D (default)
10. RollbackPriority	= FALSE	32. CostProfileId	= 0	54. AccessLockForUncomRead	= FALSE
11. MaxLoadTasks	= 15	33. DBQLFlushRate	= 600 (sec.)	55. EnablNonTempoOp	= TRUE
12. RollForwardLock	= FALSE	34. Memory Limit Per Transaction	= 2 pages		
13. MaxDecimal	= 15 (18, or 38)	35. Client Reset Timeout	= 300 (sec.)		
14. CenturyBreak	= 0	36. Temporary Storage Page Size	= 4K bytes		
15. DateForm	= 0 (IntegerDate)	37. Spill File Path	= /var/tdrsg		
16. System TimeZone Hour	= -8	38. (Reserved for future use)			
17. System TimeZone Minute	= 0	39. ChecktableTableLockRetryLimit	= 0		
18. System TimeZone String	= Not Set	40. EnableCostProfileTLE	= FALSE		
19. (Reserved for future use)		41. EnableSetCostProfile	= 0		
20. RoundHalfwayMagUp	= FALSE	42. (Reserved for future use)	= 0		
21. (Reserved for future use)		43. DisableUDTImplCastForSysFuncOp	= FALSE		
22. Target Level Emulation	= FALSE	44. CurHashBucketSize	= 20 bits		

This page lists some of the General fields that may be of interest.

Field

Purpose

8. SessionMode	This field defines the system default transaction mode, case sensitivity, and character truncation rule for a session. The setting is either 0 (Teradata) or 1 (ANSI).
13. MaxDecimal	Defines max number of decimal digits used in expression typing (valid values are 15, 18, or 38).
14. CenturyBreak	Defines which Teradata dates are specific to the 21st Century. Valid values range from 0 to 100.
15. DateForm	Defines the standard date format – IntegerDate is 0, ANSIDate is 1.
22. Target Level Emulation	Used when a user wants to emulate a Target Level Machine. TRUE means the user wants to run Target Level Emulation. FALSE means the user does NOT. The default is FALSE
53. PrimaryIndexDefault	This field determines whether a primary index is automatically created for a table or join index whose definition does not explicitly include a PRIMARY INDEX, PRIMARY AMP [INDEX], NO PRIMARY INDEX, PRIMARY KEY, or UNIQUE clause. When this value is set to D : Teradata Default (Default Setting). P : first column as the non-unique Primary Index. N : results in tables/join index created without a primary index

DBS Control Record – General Fields (cont.)

55. EnabNonTempoOp = FALSE	77. DisableRecoverableNetProtocol = FALSE	99. DotNotationOnErrorCondition = 0 (Default)
56. IncINTforGrntOrRevokAll = FALSE	78. Default Character Set = 1 (LATIN)	100. Obsolete (XMLWarnThreshold)
57. TimeDateWZControl = 2	79. EnableLogonsMsg = FALSE	101. DefaultPermSkewLimitPercent = 0 (0-10000%)
58. (Reserved for future use)	82. EnableJSON = TRUE	102. DefaultSpoolSkewLimitPercent = 0 (0-10000%)
59. SysInitRelease = 16000000	83. JSON_MaxDepth = 512 (Default 512)	103. DefaultTempSkewLimitPercent = 0 (0-10000%)
60. DefaultCaseSpec = FALSE	84. JSON_AttributeSize = 4096 (Default 4096)	104. GlobalSpaceSoftLimitPercent = 0.00 (0.0-99.99%)
61. PMPC_TimeoutSecs = 60 (Sec.)	85. Reserved for future use	105. ProratedSpaceDistPercent = 0 (0-100%)
62. ExportOrderBySegmentLimi = 500	86. DisableJSONValidation = FALSE	106. MaxSetQueryBandSize = 3 (2-4 KB)
63. MLoadDiscardDupRowUPI = FALSE	87. FixSRandomRange = TRUE	107. LegacySpaceAcctg = FALSE
64. DBQL CPU/IO Collection = 0	88. Temporal Behavior = 1 (ANSI Behavior)	
65. No13dot0Backdown = TRUE	89. (Reserved for future use)	
66. SnapBypassAggrCache = FALSE	90. (Reserved for future use)	
67. RedriveProtection = 1	91. (Reserved for future use)	
68. RedriveDefaultParticipation = 0	92. MLOADXUtilityLimits = FALSE	
69. PMPC_SessionRateThreshold = 60 (Seconds)	93. MaxMLOADXTasks = 30 (Default 30)	
70. RoundNumberAsDec = FALSE	94. MaxMLOADXAWT = 48	
71. DMLStatementShipping = 0 (Disabled)	95. CPUupdate = 3 (0-4, Default 3)	
72. SHOW IN XML Formatted Output = FALSE		
73. XML_MemoryLimit = 4 (MB; def.)	General Fields (New with Teradata 16.00)	
74. DisplacementOnOverlap = 0 (disabled, def)	96. EnableDATASET = TRUE	
75. NameValidationRule = 0 (default)	97. DATASETAttributeSize = 4096 (Default)	
76. EnableEON = TRUE	98. DisableDATASETValidation = FALSE	

This page lists some of the General fields that may be of interest.

Field	Purpose
70. RoundNumberAsDec	If TRUE indicates that the RoundHalfwayMagUp flag will be honored for NUMBER data just like for DECIMAL data. If FALSE (default), NUMBER data will be Rounded away from 0.
78. DefaultCharacterSet	This field specifies the system default for the character set option when it is not specified in a CREATE USER SQL statement. The valid values are 0 (UNICODE) and 1 (LATIN). 0 is default on Japanese Language Support mode systems and 1 is default on Standard Language Support mode systems. Enter 0 for setting to UNICODE and 1 for setting to LATIN. The modification is effective after a write. The modified value is used for new users created and does not impact existing users in the system.
80. COUNT_mode	This field is used to set COUNT_mode to default/BIGINT/NUMBER mode. 0: Default, 1: NUMBER, 2: BIGINT.
82. EnableJSON	This field enables or disables the system-defined JSON type. TRUE - system-defined JSON type enabled (default) FALSE - system-defined JSON type disabled

DBS Control Record – File System Fields

1. FreeSpacePercent	= 0%	28. DisableMergeBlocks	= FALSE (Enabled)
2. MiniCylPackLowCylProd	= 10 (free cylinders)	29. DisableRestartableBLC	= FALSE (Enabled)
3. PermDBSize	= 254 (sectors)	30. SpoolBigDBAllocUnit	= 1 (Sectors; Default value)
4. JournalDBSize	= 254 (sectors)	31. DictionaryDBSize	= 254 (Sectors; Default value)
5. DefragLowCylProd	= 100 (free cylinders)	32. WorkDBSize	= 254 (Sectors; Default value)
6. PermDBAllocUnit	= 1 (sectors)	33. MiniCylPackHighWaterMark	= 100 (Free cylinders; Default value)
7. Cylinders Saved for PERM	= 10 (cylinders)		
8. DisableWALforDBs	= FALSE		
9. DisableWAL	= FALSE		
10. WAL Buffers	= 20 (WAL log buffers)		
11. MaxSyncWALWrites	= 5 (MaxSyncWalWrites)		
12. SmallDepotCylsPerPdisk	= 2 (cylinders)		
13. LargeDepotCylsPerPdisk	= 1 (cylinders)		
14. WAL Checkpoint Interval	= 60 (seconds)		
15. Free Cylinder Cache Size	= 100 (number of cylinders)		
16. Bkgrnd Age Cycle Interval	= 60 (seconds)		
17. DisableAutoCylPack	= FALSE (Enabled)		
18. AutoCylPackColddata	= FALSE (Disabled)		
19. AutoCylPackFSP	= 10 (Percent; Default value)		
20. AutoCylPackStyle	= 2 (PACK / UNPACK)		
21. AutoCylPackThresh	= 5 (Percent; Default value)		
22. AutoCylPackFreeCylThresh	= 20 (Cylinders; Default value)		
23. AutoCylPackInterval	= 60 (Seconds; Default value)		
24. AutoCylPackIOThresh	= 1 (Number; Default value)		
25. (Reserved for future use)			
26. (Reserved for future use)			
27. MergeBlockRatio	= 60 (Percent; Default value)		

* Parameters 8-14 apply to WAL
 * Parameters 15-16 apply to TD 13.0
 * Parameters 17-28 apply to TD 13.10
 * Parameters 29-30 apply to TD 14.10
 * Parameters 31-33 apply to TD 16.00

This page lists some of the File System fields that may be of interest.

Field

Purpose

1. FreeSpacePercent
This field is used by the DBS and the File System to determine the percentage of free space to leave on cylinders during data load operations. The valid range of values is 0 .. 75. The default value is 0 (percent). A table definition (CREATE or ALTER) overrides this value.
2. MiniCylPackLowCylProd
Determines the number of free cylinders below which it will start to perform the "MiniCylPacks" operation in anticipation of their need. Setting this field to 0 indicates that "MiniCylPacks" will only be performed when no free cylinders exist. The default value is 10.
3. PermDBSize
Determines the maximum size of a Permanent Table's multi-row Data Blocks in 512-byte sectors. The valid range of values is 14 .. 255. The default value is 127 (sectors).
A table definition (CREATE or ALTER) overrides this value.
7. Cylinders Saved for PERM
Used to save X number of cylinders for Perm Data only – cannot be used by Spool. Range is 1 to 100; default is 10. If the number of available cylinders falls below this value, spool files will not be allocated space.
- 8-14. WAL parameters
Parameters to setup options for Write-Ahead Logging.

DBS Control Record – Performance Fields

<u>Performance Fields:</u>		<u>Notes</u>	
1. DictionaryCacheSize	= 8192	Size of data dictionary cache for each PE	17. UtilityReadAheadCount = 10
2. DBSCacheCtrl	= TRUE		18. StandAloneReadAheadCount = 20
3. DBSCacheThr	= 10%		19. DisablePeekUsing = FALSE
4. MaxParseTreeSeg	= 2000	Raise for more complex SQL requests	20. IVMaxWorkloadCache = 1 (megabytes)
			21. RevertJoinPlanning = FALSE
			22. MaxJoinTables = 128
5. ReadAhead	= TRUE		23. DBQLXMLPlanMemLimit = 64000 (KB; Def.)
6. StepsSegmentSize	= 1024 (kilobytes)	Maximum size of the plastic steps segment in KB	24. LimitInlistCVal = 1048576
			25. NumStatisticsCacheSegs = 5 (Size of Segment is 1024 KB)
7. RedistBufSize	= 3832 (kilobytes)		26. SHOW IN XML Memory Limit = 64000 (KB; Default value; new in 14.10)
8. DisableSyncScan	= FALSE		27. DictionaryCacheSegmentSize = 16 (* 64 KBs; Default value; new in 14.10)
9. SyncScanCacheThr	= 10%		28. PRPDMaxSkewedVals = 50 (Default value; new in 14.10)
10. HTMemAlloc	= 10%		29. PRPDSkewPct = 80 (Percent; Default value; new in 14.10)
11. SkewAllowance	= 75%		30. DBQLLOBCacheSize = 2 (MB; Default:2, Min:2, Max:16;15.0 fiield)
12. Read Ahead Count	= 1		31. HJ2IMHJ = FALSE (15.10 - Every Hash Join changed to In-memory Hash Join)
13. PPICacheThrP	= 10		
14. ReadLockOnly	= FALSE	(FALSE enables this feature)	
15. IMaxWorkloadCache	= 64 (megabytes)		
16. MaxRequestsSaved	= 600 (default)		

Field

Purpose

1. DictionaryCacheSize This field defines the size of the dictionary cache for each PE Vproc on the system. The valid range of values is 64 .. 16384. 64-bit system default is 3072 (kilobytes).
2. DBSCacheCtrl Enable or disable the performance enhancements associated with Cache Control Page-Release Interface. FALSE cause old caching rules to be used. With old cache rules, data blocks added to spool tables or used in sort operations are NOT cached. With old cache rules, data blocks for user tables are always cached.
3. DBSCacheThr Specifies the percentage value to use to calculate the cache threshold when the DBSCacheCtrl field is enabled. The valid range of values is 0 .. 100. The default value is 10.
4. MaxParseTreeSegs This field defines the maximum number of 64KB tree segments that the parser will allocate while parsing a request. The valid range of values is 12 .. 12000. 64-bit sys default is 2000.
5. ReadAhead Enable or disable the performance enhancements associated Read-Ahead Sequential File Access Workload. The default value is TRUE.
6. StepsSegmentSize Defines the maximum size of the plastic steps segment. Range of values is 64 – 1024 Kbytes. Default is 1024.

Modifying DBS Control Fields

To modify a DBS Control field, use the **Modify** and **Write** commands.

This example changes "general" field #13 to a value of 38 to allow decimal definitions up to 38 digits.

To save the changes, use the **Write** command.

```
102. DefaultSpoolSkewLimitPercent = 0 (0-10000%, ...
103. DefaultTempSkewLimitPercent = 0 (0-10000%, ...
104. GlobalSpaceSoftLimitPercent = 0.00 (0.0-99.99%,...
105. ProratedSpaceDistPercent    = 0 (0-100%, ...
106. MaxSetQueryBandSize         = 3 (2-4 KB, Default 3)
107. LegacySpaceAcctg            = FALSE (default FALSE)
```

Enter a command, HELP, or QUIT:

```
modify general 13=38
```

The MaxDecimal field has been modified from 18 to 38.
NOTE: This change will become effective after the next DBS restart.

Enter a command, HELP, or QUIT:

```
write
```

Locking the DBS Control GDO...
Updating the DBS Control GDO...

Enter a command, HELP, or QUIT:

Example: Set the Century Break value

To change the Century Break value, you need to use the following modify and write commands:

To change the Century Break value:

1. From the command-line prompt (or the Supervisor window of the Database Window):
dbcontrol
2. From the DBS Control window:
display general
3. Use the modify command to change flag 14 (Century Break):
modify general 14 = 50
4. Write changes to the GDO:
write

Note: Some changes may not take place until the next database restart, even though the flag shows the change right away. If you **DISPLAY GENERAL**, you'll see the new setting, but the setting is not really available until the system is restarted.

If a restart is needed, you will be notified with a message from DBS Control.

Checking Data Integrity

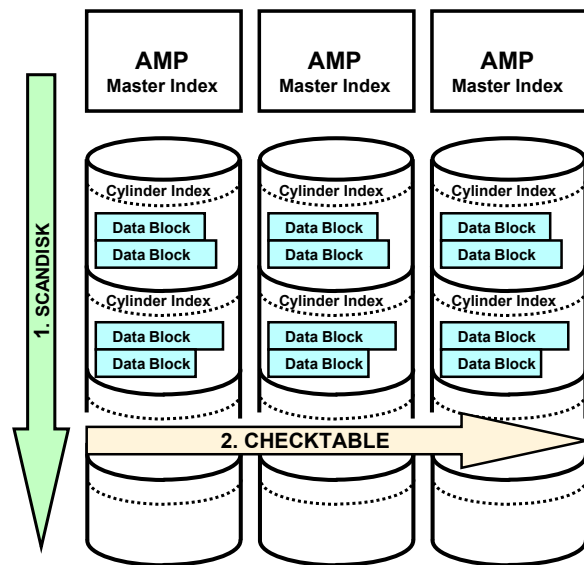
SCANDISK

Checks the AMP's file system structures (CIs and DBs for consistency)

CHECKTABLE

Checks for consistency in internal data structures such as table headers, SI subtables, row identifiers, etc.

Typically, first execute SCANDISK, then CHECKTABLE.



There are two utilities that are used to check data consistency.

- SCANDISK - checks the AMP's file system structures (CIs and DBs for consistency)
- CHECKTABLE - checks for consistency in internal data structures such as table headers, SI subtables, row identifiers, etc.

Ferret – Scandisk Utility

If Scandisk encounters an error for a specific AMP, execute (table) REBUILD for that AMP.

```

ferret localhost
File Help
Ferret ==>
scandisk
Fri Sep 24, 2010 08:37:05 : Scandisk will be started
                        On All AMP vprocs
                        Do you wish to continue based upon this scope?? (Y/N)
y
Fri Sep 24, 2010 08:37:10 : Scandisk has been started
Type 'ABORT' to stop the command before completion
Type 'INQUIRE' to check on progress of command
inquire
Inquire request has been sent
Fri Sep 24, 2010 08:37:48
SCANDISK STATUS :
Slowest vproc      0 is 14% done
Fastest vproc      5 is 15% done
The scandisk is about 15% done
Nobody has reached the SCANFREE stage
Status: Reading
scandisk
y
inquire
Enter a command:

```

The SCANDISK utility/command enables you to determine if there is a problem with the AMP file system and assess its extent. SCANDISK is a diagnostic tool designed to check for inconsistencies between key file system data structures such as the master index, cylinder index, and data blocks.

As an administrator, you can perform this procedure as preventative maintenance to validate the file system, as part of other maintenance procedures, or when users report that there are file system problems. Execute the SCANDISK command in the Ferret utility while the system is operational.

The SCANDISK command:

- Verifies data block content matches the data descriptor.
- Checks that all sectors are allocated to one and only one of the following:
 - Bad sector list
 - Free list
 - A data block
- Ensures that continuation bits are flagged correctly.

If Scandisk discovers a problem with a disk, you must use the Table Rebuild utility to rebuild any tables it reports as having bad data for the particular AMP vproc. (The Table Rebuild utility is discussed later in this lesson.)

To avoid potential TPA resets, run SCANDISK prior to:

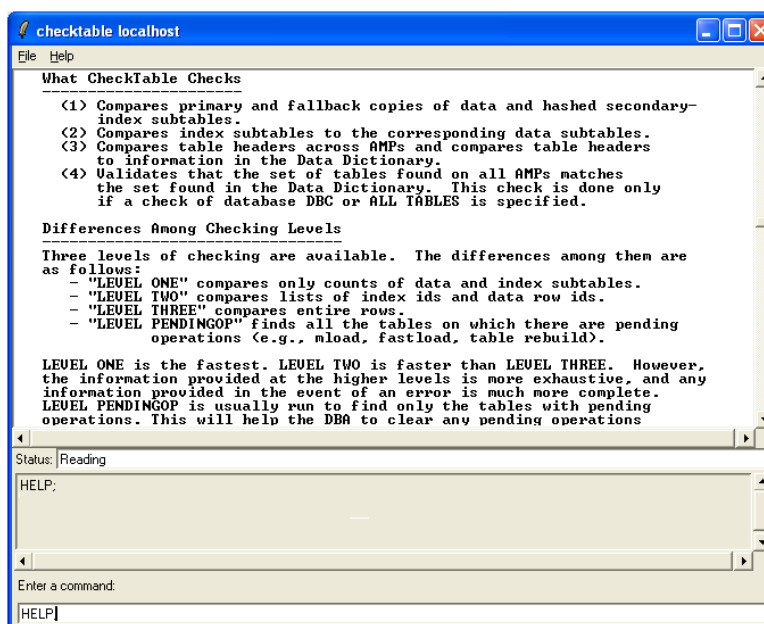
- Running the Checktable utility
- Rebuilding database tables using the Table Rebuild utility

Checktable Utility

Three levels of
typical checking →

From supervisor of DB Window,
START CHECKTABLE

Checktable Command →



Checktable is a diagnostic tool designed to check for inconsistencies in internal data structures such as table headers, row identifiers, and secondary indexes. Checktable can help determine if there is corruption in your system. Normally, Checktable is executed on a system that is quiescent. Use the Checktable utility as both a diagnostic and validation tool. As a diagnostic tool, you can identify problems with data integrity. As a validation tool, you can verify data integrity prior to a reconfiguration or archive. Checktable only identifies inconsistencies; it does not correct them.

Always run Scandisk before you run Checktable. Checktable assumes the underlying structure of the file system is intact. If there are structural errors, Checktable could cause a tpa reset on the database. Scandisk is located within the Ferret utility.

The estimated run time for a Checktable varies depending on the characteristics of the data. The more non-unique secondary indexes defined on the tables, the longer it takes to run Checktable. If you invoke Checktable when users are logged on, the time it takes to process the Checktable will depend on the activity on the system and the amount of resource contentions that it encounters (for example, object locks).

Starting Checktable

To start the utility, enter **start Checktable** from the DBW supervisor.

Stopping Checktable

To stop Checktable, enter **QUIT**;

Checktable – Levels of Checking

Checktable provides three levels of checking that are typically used:

Level 1 Compares only the counts of data and index subtables.

Level 2 Includes level 1 checking; also compares lists of index and data row ids/keys and primary to fallback checksums.

Level 3 Includes levels 1 and 2 checking; also compares entire rows.

While Checktable is running, you may use the following function keys:

- F2** Displays current status.
- F3** Aborts the current table check and continues with the next.
- F4** Aborts the current Checktable command.
- F7** Help

It is recommended that you periodically (e.g., monthly) schedule the following maintenance routines:

- A SCANDISK diagnostic run for all AMPs. This function performs intra-disk integrity checks. Your Customer Engineer can start this diagnostic tool.
- A CHECKTABLE run at level 2. Checktable completes the diagnostic analysis with inter-disk integrity checks.

The Checktable utility provides more than three levels of checking. The first three levels of checking are typically used by customers. Each level is a superset of the lower levels and runs all previous level checks.

Level One Level one checking compares the counts of data and index subtables. Use level-one checking to identify specific tables that contain errors. If errors are detected, perform a more detailed check using level-two or level-three checking.

Level Two Level two compares lists of index and row IDs as well as primary to fallback checksums. This level also verifies that hash codes reflect correct row distribution in any given subtable. Level two checking requires significantly more system resources than level one. You can use spool space to check tables that have secondary indexes.

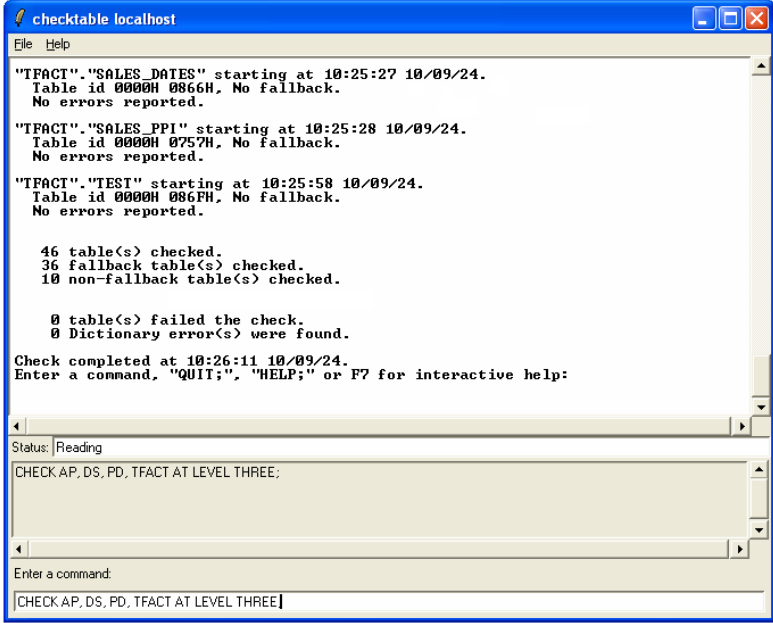
Level Three Use level three checking to obtain detailed diagnostic information. This level provides the most detailed check and requires the most system resources. Level three compares entire rows, byte by byte.

Teradata recommends that you perform the following maintenance routine once a month:

1. Run a Scandisk diagnostic for all Vdisks.
2. Follow Step 1 with a Checktable run at Level 1.

Checktable – Example

CheckTable Command:
CHECK AP, DS, PD,
TFACT AT LEVEL
THREE;



```

checktable localhost
File Help

"TFAC". "SALES_DATES" starting at 10:25:27 10/09/24.
Table id 0000H 0866H, No fallback.
No errors reported.

"TFAC". "SALES_PPI" starting at 10:25:28 10/09/24.
Table id 0000H 0757H, No fallback.
No errors reported.

"TFAC". "TEST" starting at 10:25:58 10/09/24.
Table id 0000H 086FH, No fallback.
No errors reported.

46 table(s) checked.
36 fallback table(s) checked.
10 non-fallback table(s) checked.

0 table(s) failed the check.
0 Dictionary error(s) were found.

Check completed at 10:26:11 10/09/24.
Enter a command, "QUIT;", "HELP;" or F? for interactive help:

Status: Reading
CHECK AP, DS, PD, TFACT AT LEVEL THREE;

Enter a command:
CHECK AP, DS, PD, TFACT AT LEVEL THREE

```

This slide shows the output from executing Checktable against several databases. Additional options in Checktable are:

Error Limit for a Check Command

The error limit is the maximum number of errors that can be found during checking a table. If the number of found errors exceeds the error limit, Checktable stops checking on the current table but continues to check the next table.

To by-pass tables which are locked.

The SKIPLOCKS option is intended to help the user by-pass any contention on tables. Without this option Checktable will block indefinitely on the table to be checked until it has been unlocked. When this option is specified Checktable will automatically skip the in-use (locked) tables.

To check database(s) or table(s) in serial or parallel.

SERIAL/PARALLEL mode allows the user to specify whether the Checktable utility should check the specified databases/tables in SERIAL mode or in PARALLEL mode. Default mode <checkmode> is SERIAL mode.

In SERIAL mode, the Checktable utility checks a single table at a time.

In PARALLEL mode, the Checktable utility checks the specified database(s)/table(s) in parallel.

Table Rebuild Utility

For an AMP that has failed (e.g., a Vdisk failure) and has been repaired, REBUILD will ...

- Rebuild table headers and data for fallback tables.
- Only build table headers for non-fallback tables.

```

rebuild localhost
Release 13.10.00.00 Version 13.10.00.00
TABLE REBUILD Utility (Dec 94)

      {ALL TABLES } {ALL }
REBUILD AMP nnnn {dbname } {PRIMARY } DATA [, Options] ;
      {dbname.tname } {FALLBACK}

REBUILD AMP nnnn FALLBACK TABLES [, Options] ;

RESTART REBUILD ON AMP nnnn ;

Options : LOG INTO logdbase.logtbl
          NO LOCK [ON NO FALLBACK TABLES]
          {DATABASE LOCK}
          WITH {TABLE LOCK}
          {ROWRANGE LOCK}
          [n TABLES] IN PARALLEL

Enter command, "QUIT ;" or F7 for help :
REBUILD AMP 3 AP ALL DATA, WITH DATABASE LOCK;

System time is 10/09/26 02:53:40

D 02:53:40 Rebuilding database AP
02:53:40 Rebuilding table Accounts

Status: Reading
REBUILD AMP 3 AP ALL DATA, WITH DATABASE LOCK;

Enter a command:
  
```

Table Rebuild is a utility that repairs data corruption. It does so by rebuilding tables on a specific AMP vproc based on data located on the other AMP vprocs in the fallback cluster. Table Rebuild can rebuild data in the following subsets:

- The primary or fallback portion of a table
- An entire table (both primary and fallback portions)
- All tables in a database
- All tables that reside on an AMP vproc

Table Rebuild performs differently based on the type of table (e.g., fallback or not, etc.)

Type of Table

Fallback tables

Action

Delete the table header (one row table which defines a user data table)

Delete specified portion of the table being rebuilt

Rebuild the table header

Rebuild the specified portion of the table

Non-fallback tables

Delete the table header

Rebuild the table header

Permanent journal tables

Delete data for the table being rebuilt

Rebuild the table header

Locks the table ("pending rebuild")

Note: You must restore non-fallback data.

Recovery Manager Utility

RCVMANAGER provides a means for the user to interact with the recovery subsystem.

Following a Teradata restart, you can ...

- View the number of rows being rolled back via the Transient Journal (TJ).
- View the number of rows being updated on an AMP via the Recovery Journal (Fallback tables and rows that were updated while the AMP was out of service).
- List the tables that are locked until the rollback completes.
- Change the priority of Rollback and/or Recovery operations.

Other options ...

- View which tables are being rolled back.
- Set the rollback priority for a specific session to a specific performance group.
- For a table or tables, **cancel rollback processing** for an online user requested abort or following a system restart.
 - WARNING: The target table will be unusable after this command is issued.
- View the tables for which rollback processing is not pending cancellation during the online transaction recovery.
 - The table is removed from the list when no more TJ rows exist for the table.

The Rcvmanager (Recovery Manager) utility lets you monitor the backing out of incomplete transactions on tables that may be locked for access. The resulting journal is called the Transaction Journal. Rcvmanager also shows the count of records of data presently in the Down AMP Recovery Journal. This journal represents the data rows an AMP vproc must recover from the other AMP vprocs in the cluster before coming back online to the database.

The Recovery Manager utility runs only when the system is in one of the following states:

- Logon
- Logon/Quiet
- Logoff
- Logoff/Quiet
- Startup (if system has completed voting for transaction recovery).

If the system is not in one of the above states, Recovery Manager will terminate immediately after you start it.

Recovery Manager Commands

From supervisor: **START RCVMANAGER**

Commands are:

LIST STATUS [<proc-id>] ; shows status of transaction and/or down AMP recovery

LIST LOCKS; displays all locks currently held by online transaction recovery

REBUILD PRIORITY [Low | Medium | High] ; sets the table rebuild priority

RECOVERY PRIORITY [Low | Medium | High] ; sets the AMP recovery priority

DEFAULT PRIORITY; sets both priorities back to their default

HELP;

QUIT;

Other commands include:

LIST ROLLBACK TABLES; – view the tables being rolled back for which rollback processing is not pending cancellation

LIST CANCEL ROLLBACK TABLES; view the tables that are pending cancellation

CANCEL ROLLBACK ON TABLE <table-id> [{, <table-id>} ...] ;

ROLLBACK SESSION <host>, <session> **PERFORMANCE GROUP** [<Perf Group Name>];

The **LIST STATUS** command displays information about recovery operations in progress. The processor id option provides additional detailed information about a specific down AMP.

The **LIST LOCKS** command displays a list of all locks currently held by online transaction recovery.

The **LIST ROLLBACK TABLES** command displays the list of tables which are currently being rolled back in the system. Separate listings are generated to distinguish between online transaction recovery and system recovery. Table ids can be selected from this listing for executing the **CANCEL ROLLBACK ON TABLE** command. In case a '*' character follows the table names then they cannot be specified in the **CANCEL ROLLBACK ON TABLE** command.

The **LIST CANCEL ROLLBACK TABLES** command displays the list of tables for which rollback processing is pending cancellation during the online transaction recovery. These tables are removed from the list when all the journal rows corresponding to the tables have been skipped on all the AMPs.

The **CANCEL ROLLBACK ON TABLE** command is used to specify a table for which rollback processing is to be cancelled for an online user requested abort or during system recovery.

Rcvmanager – List Status

After a Teradata restart, the list status command is used.

In this example, there are almost 270K TJ rows remaining to be rolled back.

The list status command is used a second time and the count has decreased.

Note: The AMP with the highest count of TJ rows to roll back is listed.

```

rcvmanager localhost
File Help
Release 13.10.00.00 Version 13.10.00.00
RCVMANAGER Utility (Jan 96)

Enter command, "QUIT;" or "HELP;" :
list status;

ONLINE TRANSACTION RECOVERY JOURNAL COUNTS at 07:31:48 10/09/26

Recovery Session Count AMP W/Count
-----
1 269,584 00002

Enter command, "QUIT;" or "HELP;" :
list status;

ONLINE TRANSACTION RECOVERY JOURNAL COUNTS at 07:32:53 10/09/26

Recovery Session Count AMP W/Count
-----
251,584 00002

Enter command, "QUIT;" or "HELP;" :

Status: Reading
list status;
list status;

Enter a command:

```

The Recovery Manager utility uses two basic commands: the LIST STATUS and LIST LOCKS commands.

The LIST STATUS command generates two reports:

- ONLINE TRANSACTION RECOVERY JOURNAL
- DOWN AMP CATCHUP JOURNAL

ONLINE TRANSACTION RECOVERY JOURNAL

This report pertains to online transaction recovery and displays a list of all active recovery sessions as well as the maximum number of transaction journal rows remaining to be processed for the AMP that has this maximum count.

The entries in this report are:

Recovery Session ID of active recovery session

Count Maximum number of transaction journal rows remaining to be processed for a specific AMP

AMP W/Count The AMP to which the corresponding count applies

Rcvmanager – List Locks

To determine which tables are locked as part of “Online Transaction Recovery”, the list locks command is used.

The list status command is used a third time and the count has decreased even more.

```

rcvmanager localhost
File Help

Enter command, "QUIT;" or "HELP;" :
list locks;

LOCKS HELD BY ONLINE TRANSACTION RECOVERY at 07:38:01 10/09/26
Lock      Lock      Object
Mode      Object     Name
-----
Write     Row hash   0 1033, 0 3 (0000H 0409H, 0000H 0003H)
Write     Table      "DS"."Sales_and_SalesHistory"

Enter command, "QUIT;" or "HELP;" :
list status;

ONLINE TRANSACTION RECOVERY JOURNAL COUNTS at 07:39:31 10/09/26
Recovery  Count      AMP
Session   Count      W/Count
-----
1          68,584      00002

Enter command, "QUIT;" or "HELP;" :

Status: Reading
list status;
list locks;
list status;

Enter a command:

```

The screen display on this slide continues the same example and has an example of using the List Locks command.

The LIST LOCKS command displays all locks currently held by transaction recovery. The command generates a single report called, *Locks Held By Online Transaction Recovery*.

The report is sorted alphabetically by object name. The report does not display information for row range and row hash locks, but does display the table within which the row resides. If Recovery Manager is unable to determine the database name associated with an object, then it displays the database ID in decimal and hexadecimal. The same is true if the table name cannot be determined.

Note: LIST LOCKS displays only those locks currently held by online transaction recovery.

Rcvmanager – List Status (2nd Example)

If an AMP has been offline, use the **list status** command to determine if fallback table rows are being updated from the down AMP recovery journal.

OJ (Ordered System Change Journal)
– DDL changes to fallback tables.

CJ (Changed Row Journal) – DML changes to fallback tables.

```
rcvmanager localhost
File Help
Enter command, "QUIT;" or "HELP;" :
list status;

ONLINE TRANSACTION RECOVERY JOURNAL COUNTS at 12:09:02 10/09/26
Recovery Session Count AMP W/Count

DOWN AMP RECOVERY STATUS at 12:09:02 10/09/26
AMP to be Current Pass Next Pass
caught up Pass OJ CJ OJ CJ
-----
0004 1 0 12,236 0 64
- AMP Status: Offline Catchup
- Change Row Recovery: 32% complete in this pass

Enter command, "QUIT;" or "HELP;" :

Status: Reading
list status;
list locks;
list status;
list status;

Enter a command:
```

AMP CATCHUP JOURNAL COUNTS

This report pertains to offline AMP recovery and displays an entry for every offline AMP. Entries include the following:

AMP to be caught up

Designates which AMP needs to be recovered.

Asterisk (*) indicates that the AMP would be brought into **online catchup** if a restart were to occur (CJ < 3000 and the OJ/TJ = 0).

AMP Status

Describes the recovery mode now in progress for the recovering AMP:

NOT IN RECOVERY — AMP is still down, and has not come up into recovery yet.

OFFLINE CATCHUP -- AMP is offline, and is in catchup mode

ONLINE CATCHUP — The AMP is still catching up, but it is online and will accept new transactions.

Changed Row Journal Count

The number of rows updated in the cluster while an AMP was down. Usually DML changes that impact a row on a single AMP.

Ordered System Change Journal

The number of system or table level changes in the cluster. Usually DDL changes that impact row(s) on all of the AMPs.

Rcvmanager – List Rollback Tables

In this example, the List Rollback Tables lists a table that is being rolled back because of a user aborted transaction.

The number of rows remaining to be rolled back and the estimated time for rollback is provided.

The Table ID and table name are also provided.

```
rcvmanager localhost
File Help

list rollback tables;

TABLES BEING ROLLED BACK AT 17:22:48 10/09/26

ONLINE USER ROLLBACK TABLE LIST
Host Session User ID Performance Group AMP W/Count
-----
1 1001 0000:0407 R 1

TJ Rows Left TJ Rows Done Time Est.
-----
617822 12648 00:28:52

Table ID Name
-----
0000:060D "DS"."Sales_and_SalesHistory"

SYSTEM RECOVERY ROLLBACK TABLE LIST
Host Session TJ Row Count
-----

Enter command, "QUIT;" or "HELP;" :
```

The **LIST ROLLBACK TABLES** command displays the list of tables which are currently being rolled back in the system. Separate listings are generated to distinguish between online transaction recovery and system recovery.

Table Ids can be selected from this listing for executing the CANCEL ROLLBACK ON TABLE command. In the situation where a '*' character follows the table names, then they cannot be specified in the CANCEL ROLLBACK ON TABLE command.

Rcvmanager – Cancel Rollback on Table

CANCEL ROLLBACK should only be used when ...

- rollback will take longer than a restore of the table
- or the table is unimportant (i.e., a temporary table)

Is only executed by the Support Center!!

You will prompted for the DBC password.

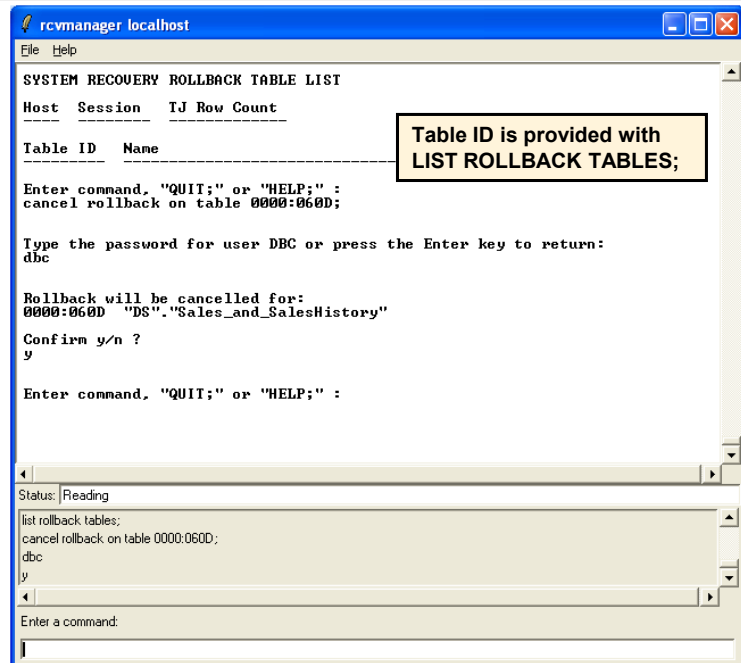
After a cancelled rollback, the table is unusable.

SELECT * FROM ...;

- Error 7562

You can ...

- DROP TABLE tname;
- DELETE tname ALL;



The **CANCEL ROLLBACK ON TABLE** command is used to specify a table for which rollback processing is to be cancelled for an online user requested abort or during system recovery. The DBC password is required to execute this command. Multiple tables can be specified by separating their table ids with commas.

The CANCEL ROLLBACK command should only be used in cases where the rollback will take longer than the restore of the table, or in cases where the table is unimportant (i.e., a temporary table). A single table retrieve operation can be performed on the target table by using the READ OVERRIDE locking modifier on it.

WARNING: The target table will be unusable after this command is issued, and will become usable only when the table is dropped and created again, or when the table is restored from an archived backup, or when a DELETE ALL operation is performed on that table.

For example,

SELECT * FROM Sales_and_SalesHistory;

Error 7562: Invalid operation on table Sales_andSalesHistory

DELETE Sales_and_SalesHistory ALL:

Completed: 483, 350 rows processed (command executes successfully)

Showlocks Utility

SHOWLOCKS displays “host utility” (HUT) locks that ARC has placed on databases and tables during an archive, restore, or recovery operation.

From supervisor: START SHOWLOCKS

The ARCHIVE command places a READ lock on database/table.

The RESTORE/COPY command places an EXCLUSIVE lock on database/table.

This utility displays the host utility locks that are present and terminates. There are no commands to exit this utility.

```

Appl 1 localhost
File Help
Release 13.10.00.00 Version 13.10.00.00
Show Locks Utility (May 95)

This program queries all AMPs and reports all
Host Utility locks which currently exist at both
the data base level and the table level.

For each lock which is found, an entry will
appear on your console which includes the following
information:
--Data Base Name
--Table Name (if applicable)
--User Name of user who placed lock
--Lock Mode
--AMP Number

TFACT.Customer
USER Sysdba    MODE Read    AMP All Amps
TFACT.Daily_Sales
USER Sysdba    MODE Read    AMP All Amps
TFACT.Orders
USER Sysdba    MODE Excl    AMP All Amps
--ShowLocks Processing Complete--

Status:
Enter a command:
  
```

The Showlocks utility enables you to retrieve information about host utility locks the ARC utility places on databases and tables during backup or restore activities. This utility also displays information about locks placed during a system failure.

Host utility locks may interfere with application processing and are normally released after the utility process is complete. If locks interfere with application processing, you can remove them by invoking the RELEASE LOCK statement. It is available through the ARC utility or as an SQL statement. An individual session may be in a "blocked" state due to one of the following situations:

- An ARC operation failed and a database cannot be accessed
- Locks were not implicitly or explicitly released after an ARC operation
- A lock was not released by the user after a system failure occurred

Report Contents

The utility displays the following information for each utility lock:

- Database name that contains lock
- Table name that contains lock (if applicable)
- User name of user who placed the utility lock
- Lock mode (read, write, exclusive, access)
- Read = Dump
- Write = Roll
- Exclusive = Restore/Copy
- Access = Group read lock or Checkpoint
- ID of vproc (all AMPs when lock resides on all AMPs)

Orphan or Phantom Spool Issues

In rare cases, when a query completes (or is aborted), the spool file is not dropped.

Orphan Spool – spool file (tableid) incorrectly remains in existence

Phantom Spool – spool file tableid is gone, but the spool space remains allocated.

The following query can be executed to determine the presence of leftover spool (orphan or phantom).

```
SELECT Databasename, Vproc, CurrentSpool
FROM DBC.DiskSpace
WHERE Databasename NOT IN
      (SELECT UserName FROM DBC.SessionInfo)
AND CurrentSpool > 0
ORDER BY 1, 2
WITH SUM (CurrentSpool);
```

Should this query return rows, the next step is to run the utility "updatespace".

The UpdateSpace utility is also used to correct inconsistencies in the DBC.DatabaseSpace table, which might occur as the result of unusual (rare) types of system failures.

From supervisor: START UPDATESPACE

Like all tables, spool tables require a Table ID. There is a range of tableids exclusively reserved for spool tables (C000 0001 thru FFFF FFFF) and the system cycles through them. If a spool file (table) is incorrectly not dropped, it remains in existence. Eventually, the system will cycle through all the table ids and reassign the tableid which is in use by our left over spool table. Usually, the presence of this table is detected, the query which was going to use the tableid is aborted – even though it is innocent of any wrongdoing – and the following message is returned to the user and put in the error log:

*** FAILURE 2667 Left-over spool table found: transaction aborted.

In rare cases, the leftover spool file (orphan spool) is not detected and the leftover spool is used. Since it was not created by the current transaction, its format is incorrect and the system will crash in an unpredictable way.

A more subtle condition is when a spool table is dropped, but the steps which reduce the tally of spool space currently in use are not. This is phantom spool. The tallies say the table exists, but it does not. Phantom spool does not cause restarts. Unless the space involved is a significant percentage of the total spool reserve, it is just an annoyance.

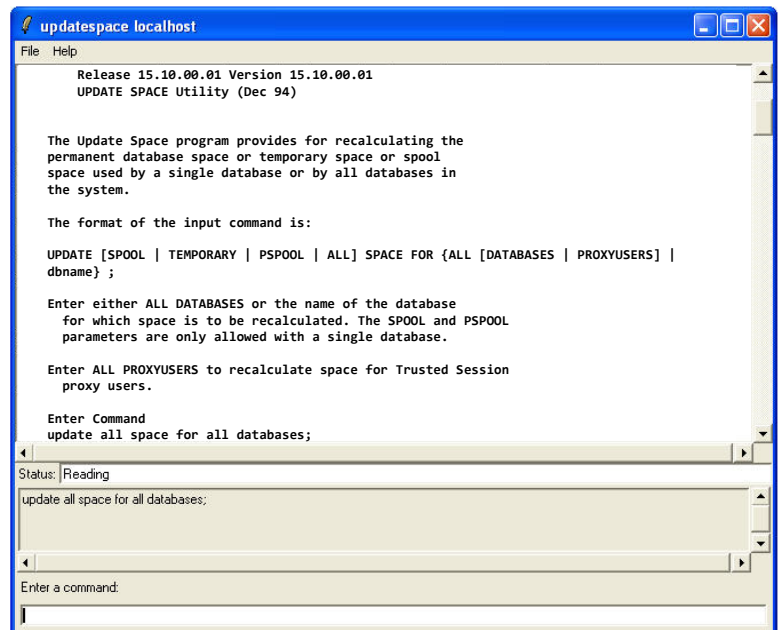
The query on this slide can be run to flag the presence of either variety (real or phantom) of leftover spool.

Should this query return rows, the next step is to run the utility **updatespace**.

Update Space Utility

The Update Space utility, **updatespace**, recalculates the permanent, temporary, or spool space in the DBC.DatabaseSpace table.

Command →



The Update Space utility (**updatespace**) recalculates the permanent, temporary, or spool space used by either of the following:

- A single database and its individual tables
- All databases in a system and their individual tables

The Update Space utility accomplishes this by performing the following:

- Examining storage descriptors and adding up space for each table.
- Setting values in CurrentPermSpace, CurrentTempSpace, or CurrentSpoolSpace in the DBC.DatabaseSpace table for each table and for the containing database as a whole.

A different utility, Update DBC (**updatedbc**), recalculates the maximum allowed values for permanent, temporary, and spool space in the DBC.Dbase and DBC.DatabaseSpace tables.

The following table lists the difference between the Update DBC and Update Space utilities.

- Update DBC recalculates maximum allowed values for permanent, temporary, and spool space.
- Update Space recalculates current usage for permanent, temporary, and spool space.

The only reason to use Update Space is to correct inconsistencies in the DBC.DatabaseSpace table, which might occur as the result of rare types of system failures.

Vprocmanager

Commands:

STATUS – provides status information about vprocs

SET – sets the state (e.g., ONLINE, OFFLINE) of a vproc.

RESTART – restarts Teradata

INITVDISK – initializes the DBS File System on a Vdisk.

BOOT – initializes the File System and boots a specific vproc

Note the large vproc numbers that are available starting in 14.0.

status;

SYSTEM NAME: TDT5B 15/07/04 06:32:46

DBS LOGICAL CONFIGURATION

Vproc Number	Rel. Vproc#	Node ID	Can Move	Crash Count	Vproc State	Config Status	Config Type	Cluster/Host No.	Rcv Jrn1/Type	TVS Vproc
0*	1	1-13	Yes	0	ONLINE	Online	AMP	0	On	28671
1	2	1-13	Yes	0	ONLINE	Online	AMP	1	On	28671
2	3	1-13	Yes	0	ONLINE	Online	AMP	2	On	28671
3	4	1-13	Yes	0	ONLINE	Online	AMP	3	On	28671
:	:	:	:	:	:	:	:	:	:	:
28	29	1-13	Yes	0	ONLINE	Online	AMP	13	On	28670
29	30	1-13	Yes	0	ONLINE	Online	AMP	14	On	28670
26622	-	1-14	No	0	NONODE	N/A	RSG	0	N/A	N/A
26623	34	1-13	No	0	ONLINE	N/A	RSG	0	N/A	N/A
22528	33	1-13	No	0	ONLINE	N/A	GTW	1	COP	N/A
28670	35	1-13	Yes	0	ONLINE	N/A	TVS	0	N/A	N/A
28671	36	1-13	Yes	0	ONLINE	N/A	TVS	0	N/A	N/A
30718	31	1-13	Yes	0	ONLINE	Online	PE	1	COP	N/A
30719	32	1-13	Yes	0	ONLINE	Online	PE	1	COP	N/A

Status: Reading

status;

Enter a command:

What does the NONODE vproc state indicate?

The Vprocmanager utility provides a means to manage/manipulate various vproc attributes. Vprocmanager can be started from the Supervisor (**start vprocmanager**) or command-line (**vprocmanager**).

Valid commands are: STATUS, INITVDISK, RESTART, BOOT, SET, HELP, and QUIT,.

STATUS

The simple format of this command (i.e., STATUS without any options) returns the DBS and PDE status tables in their entirety.

RESTART [TPA] [NODUMP | DUMP = {YES, NO}][<RestartKind> [<comment>]

This command is used to force different flavors of DBS restarts.

- A system dump will not be taken when the NODUMP option is specified. This is the default action.
- The DUMP = YES option causes a system dump to be taken.
- The DUMP = NO option is equivalent to NODUMP.
- Valid RestartKinds are COLD or COLDWAIT.
- The comment specifies the reason for the restart.

SET

Sets the state of a vproc to either ONLINE, OFFLINE, or FATAL.

Summary

- **DBS Control** – used to view/modify the DBS Control Record fields to establish system and default values.
- **Packdisk** – fill (packs) cylinders up to the Free Space Percentage (FSP) with the purpose of freeing up cylinders.
- **Defragment** – combines free sectors and moves them to the end of a cylinder.
- **Scandisk** – identifies and determines the extent of any problems with the AMP file system.
- **Checktable** – checks for inconsistencies in internal structures such as table headers, row identifiers and secondary indexes.
- **Table Rebuild** – repairs data corruption.
- **Recovery Manager** (RCVManager) – enables you to view information about online transaction recovery and AMP recovery.
- **Showlocks** – displays host utility locks.
- **Teradata Locale Definition Utility** (tdlocaledef) – converts a Specification for Data Formatting file (SDF) into an internal, binary format (a GDO) for use by Teradata Database.
 - The SDF file is a textfile that defines how Teradata Database formats numeric, date, time, and currency output. An example of this utility was not provided in this module, but it is important when establishing system formatting defaults.

This slide summarizes some important concepts regarding this module.



Module 16: Review Questions

1. What are two ways that you initiate a Teradata system utility (e.g., dbscontrol)?

2. Identify the purpose of the following DBS Control utility parameters.
CenturyBreak _____
DateForm _____
PrimaryIndexDefault _____
SessionMode _____
3. True or False: The Checktable utility has only two levels of internal table checking.
4. True or False: The Table Rebuild utility rebuilds tables differently depending on whether the table is a fallback or a non-fallback table.
5. The _____ utility does a consistency check within an AMP's file system.
6. The _____ utility does a consistency check for a table across all AMPs.
7. The _____ utility can be used to set an offline AMP to online.

Check your understanding of the concepts discussed in this module by completing the review questions as directed by your instructor.



Module 17: MAPS

Teradata Vantage Administration

Copyright © 2007–2023 by Teradata. All Rights Reserved.

Objectives

After completing this module, you will be able to:

- Explain the two types of Maps (Contiguous and Sparse)
- Have an understanding of how to move tables after a system reconfiguration.
- Describe what is in TDMaps and why this user is important.
- Identify the System Default Map.
- Use Colocation option to insure two tables with same Sparse map are co-located on the same AMPs.

MAPS High Level Description

Starting in TD 16.10, MAPS feature added support for a table to reside in a [subset of AMPs](#).

Two types of maps: contiguous and sparse

- [Contiguous](#) maps include all the AMPs within a range of AMPs
- [Sparse](#) maps include some of the AMPs within a range of AMPs

Benefits:

1. [Eliminates the extended](#) downtime for Reconfig by not moving tables to the new AMPs.
 - You do not have to move every table
 - You do not have to move every table at same time
 - A read lock is used for most of the time while a table is being moved
 - ✓ Upgraded to exclusive lock at the end
2. Improves performance for [small tables](#).
 - Without MAPS, access of small tables can be inefficient because some AMPs do not have rows. An all-AMP access still has to access the AMPs that don't have any rows.

The Teradata MAPS architecture (or simply MAPS) allows tables to be assigned a subset of AMPs in the system. As part of MAPS, the Teradata Database has been enhanced to support multiple different “maps” at the same time. Each map definition points to a range or set of AMPs that support the data for tables assigned to that map. MAPS provide two key benefits:

- Increased availability during system expansion
- Greater efficiency when accessing rows from small tables.

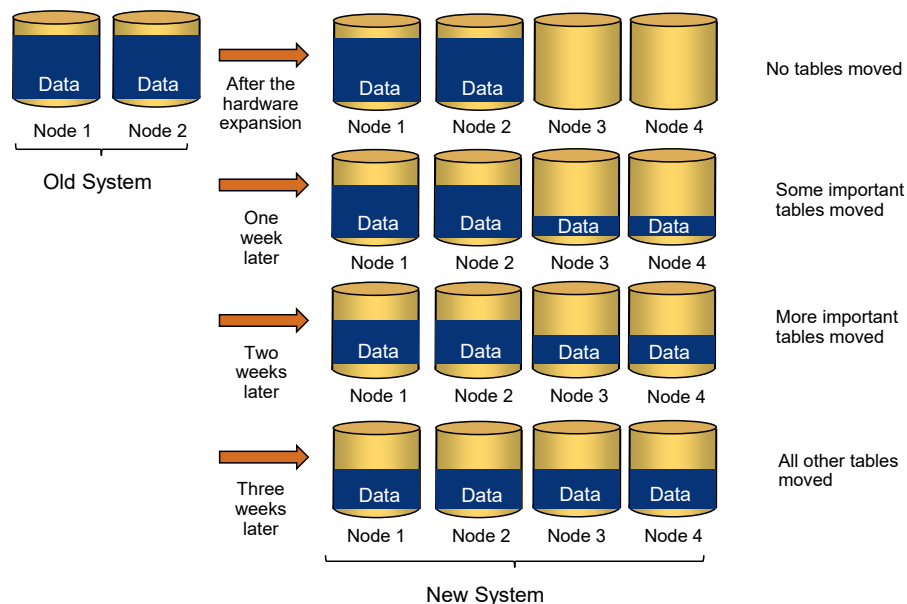
Every table uses a map that specifies which AMPs store the rows of the table. For tables with fallback, the primary and fallback copies of the row are stored on different AMPs in the map.

You can use maps to optimize the placement of tables on the AMPs in your system or to redistribute table rows after a system expansion. Before doing this, it is important to consider some general concepts. Teradata uses two types of maps to track which rows of a table belong on which AMP:

- **Contiguous** maps include all AMPs within a specified range. By default, every system has one contiguous map that includes every AMP in the system. Teradata Database creates contiguous maps during a system initialization, configuration, or reconfiguration.
- **Sparse** maps are smaller maps that contain only a limited number of AMPs. By default, each system that enables Teradata Database MAPS Architecture has a 1-AMP sparse map and an n-AMP sparse map, where n is the number of nodes in the system.

Secondary index tables use the same map as their base (indexed) table. Tables and join indexes are assigned a map either explicitly or by default.

MAPS System Expansion Concept



After you add AMPs or nodes to a system, if the Teradata Database MAPS architecture is enabled, the Reconfiguration utility (Reconfig) creates a new map that reflects the expanded system. A DBA can then choose to move table rows to the new AMPs using either Reconfig or maps. There are several advantages to using maps instead of Reconfig:

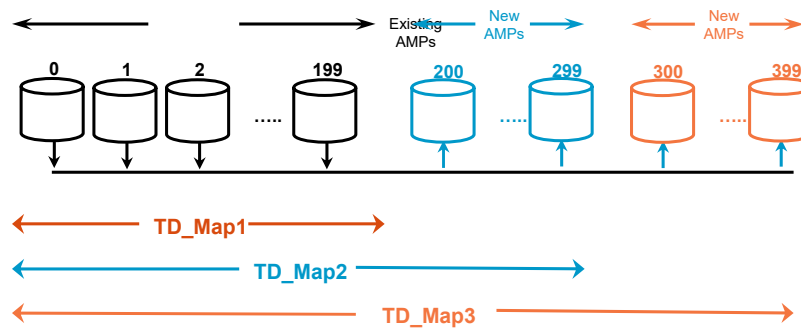
- Reconfig requires more planned downtime.
- Reconfig requires you to move all tables in the system immediately to evenly distribute rows between AMPs. However, if you use MAPS, you can move tables into the new configuration gradually while the system is **online**. Some tables can be in the old map and some tables can be in the new, expanded map.

MAPS feature allows for skipping the table redistribution portion of the reconfiguration process, leaving tables on the old AMPs. Decoupling the movement of data from the hardware expansion shortens the planned downtime to just the time required to connect the new nodes and update a few system files.

Tables can then be redistributed across the new nodes and their AMPs at the convenience of the administrator, and while the system is online. By doing so, data is accessible more quickly and there is less disruption to the business following a hardware expansion.

Prior to MAPS, the old map that pointed rows to AMPs where they were located was discarded after the rows had all been successfully relocated. With MAPS, the old map is retained to manage access to tables that are still in their original location. A new map is created in readiness for the movement of tables to the new nodes, and is immediately available for spools. Thus, a single Teradata system can use multiple maps at the same time.

MAPS Multiple Reconfigurations



The Config/Reconfig process creates new Hash Maps called **Contiguous Maps**. Contiguous maps use every AMP between a range of AMPs.

- Original system is **TD_Map1** contains AMPs 0 – 199.
- After 1st expansion, **TD_Map2** contains AMPs 0 – 299.
- After 2nd expansion, **TD_Map3** contains AMPs 0 – 399.
- Next expansions becomes TD_Map4, TD_Map5... etc.

Note: After an expansion/reconfig the system default map will be set to the new contiguous map.

A contiguous map allows for a range of AMPs to act in combination as the repository for the rows of a set of tables. When a table is accessed, only the AMPs in the table's contiguous map are involved. Prior to MAPS, the Teradata Database had one contiguous map for primary data with a range of AMPs from zero to N-1, where N is the number of AMPs in the system. With just a single contiguous map, queries that executed on the platform were guaranteed to have all the AMPs in the system available when they run. The parallelism is maximized, stable, and predictable.

The key advantage to having more than one contiguous map is to allow some (or all) tables to stay in the original map after a system expansion. This reduces the downtime associated with a hardware expansion because it postpones the effort of moving rows from every table in the old map to the new map.

Whether or not all tables are eventually moved to the new map is now up to the individual site. The movement of tables can be scheduled by the administrator, and performed at convenient times and run as background, low-priority work. Having more than one contiguous map for user tables is best viewed as a temporary state of a few weeks, or a month or two, until such time as the movement of all user tables is complete and all rows are contained within the newest, more parallel-rich contiguous map.

If a site undergoes multiple hardware expansions and the administrator has fallen behind in moving tables to the most current contiguous map, it is possible to end up having user tables located in several, even many different contiguous maps. This situation is best avoided, as it will make query tuning more difficult and could add confusion when analyzing system metrics. Optimal performance is experienced when all units of parallelism are fully available to each query that executes.

Special Maps

- **TD_GlobalMap**
 - Always defined and includes every AMP.
 - Only unhashed tables (e.g., DatabaseSpace) reside in this map.
- **TD_DataDictionaryMap**
 - Always defined, all tables in DBC reside in this map
 - All stored procedures, UDFs, XSPs reside in this map
- **TD_Map1**
 - Can be dropped – however, all tables must be removed from a Map, before the Map itself can be dropped.
 - All other tables reside in this map upon upgrade to 16.xx or sysinit.
- **TD_Map2...**
 - Added by Config/Reconfig when expanding a system
- **DIPMAPS creates two Sparse maps – a 1 AMP and #AMPs = #Nodes**
 - TD_1AmpSparseMap_nNodes, ex. TD_1AmpSparseMap_4Nodes
 - TD_nAmpSparseMap_nNodes, ex. TD_4AmpSparseMap_4Nodes

Note: Every table in the system is assigned to a map.

When the MAPS architecture first becomes available after a software upgrade to Teradata 16.10, several changes will be noticeable, even if none of the options of the MAPS architecture is being currently utilized. The following default maps will have been created for you:

- **TD_GlobalMap** – this contiguous map includes all of the AMPs in the system; it represents a global view of the system across all of the installed hardware.
- **TD_DataDictionaryMap** – this contiguous map contains DBC tables as well as SQL procedures, external procedures, and UDFs.
- **TD_Map1** – this contiguous map contains all other tables, such as user tables and join indexes.
- **TD1AmpSparseMap_nNodes** – a one-AMP sparse map with TD_Map1 as the parent contiguous map. n is the number of nodes in the system.
- **TDnAmpSparseMap_nNodes** – an n-AMP sparse map with TD_Map1 as the parent contiguous map. n is the number of nodes in the system. Note that this map is only created if there are two or more nodes in the system.

Initially, all three of the above contiguous maps have the same scope that includes all the AMPs.

When no additional maps have been created on the system beyond the ones mentioned above, all newly created tables will go into TD_Map1. Every time new hardware is added, a new contiguous map TD_MapK (K is a number chosen by the system) is created and TD_GlobalMap is changed to cover the complete scope of AMPs available to Teradata Database.

Contiguous maps follow a defined naming convention.

Moving a Table to a New Map

ALTER TABLE used to move a table to a new map.

- ALTER TABLE Table1, MAP=TD_Map2;
- Processed like an INSERT SELECT
 - Read lock on the source table
 - Upgraded to Exclusive lock at transaction end.
- Copies the entire table to the new map
- Similar performance (maybe faster) than Reconfig

ALTER TABLE

- Old AMPs:
 - Read all rows and send all rows
 - *Insert old rows*
- New AMPs:
 - *Insert new rows*

Reconfig

- Old AMPs:
 - Read all rows and send some rows to new AMPs
- New AMPs:
 - *Insert new rows*
- Old AMPs:
 - *Delete the rows sent*

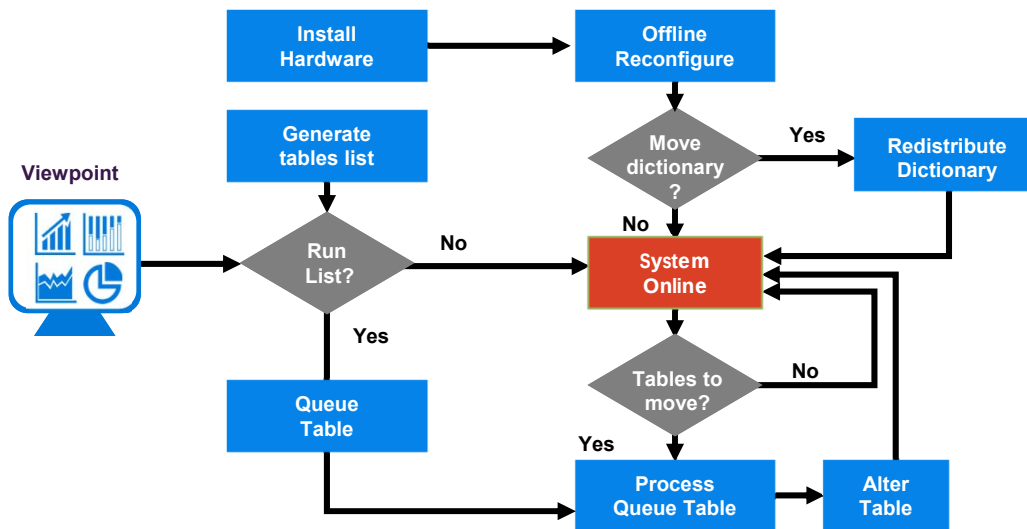
If you decide that a table should use a different map, either issue an ALTER TABLE request to name a different map for the table or use the Mover stored procedure, which is provided with the TDMaps user that we will explain later in this module.

The form of an ALTER TABLE to move a table to another map leaves the original table in its entirety in the source map and performs an INSERT-SELECT into a second instance of the table that resides in the destination map. At the end of a successful INSERT, this new ALTER TABLE updates the map name for the table and instructs the file system to drop the source data. From that point on, the version of the table residing in the new map is used.

The key benefit of this map-specific ALTER TABLE is that all of the data in the source table can continue to be accessed while the move is taking place, as is true of any INSERT-SELECT. A read lock is placed on the source table while rows are being inserted into the destination table, but is then updated very briefly to an exclusive lock at the end of processing. A secondary benefit is the move can be aborted and the rollback is a quick drop of the data in the new map.

One tradeoff, however, is that two versions of the table are in existence while the INSERT-SELECT is executing. Twice as much space is required compared to the traditional ALTER TABLE which makes changes one cylinder at a time.

System Expansion & Table Migration



Do you move the DD tables? There is no compelling need to expand this map and spread the data dictionary tables across a larger number of AMPs at the time you expand hardware. You might, for example, be thinking about getting more parallelism from queries that access the DBC. If you do wish to modify TD_DataDictionaryMap to include all AMPs, it is recommended that you specify that at the start of the Reconfig job, when prompted. But even if you don't decide until after the Reconfig that you want to expand the data dictionary map, you can re-run the Reconfig utility for the sole purpose of expanding TD_DataDictionaryMap and redistributing the data dictionary tables and objects.

Note: You can only expand the data dictionary map and move the contents of the data dictionary during a Reconfig. DBC table can never be moved manually under the control of the administrator.

If you choose to expand TD_DataDictionaryMap and move the DBC tables during a Reconfig, whether during the system expansion or at a later time, all of the data in the data dictionary must be redistributed successfully to the larger version of the map before the system can come back up. There is only a single dictionary map, which means all DBC tables must be spread across that map's AMPs as soon as the map has been modified. Unlike user tables, DBC tables cannot be moved after the Reconfig by means of an ALTER TABLE, at the convenience of the administrator.

A reason to leaving TD_DataDictionaryMap as is coming into the Reconfig: the time to redistribute the data in the dictionary will reduce availability of the data. DBC tables are not the only objects in the dictionary map. Database modules, such as the parser and the optimizer, issue express requests to access data in the DBC tables, and those express requests are mostly composed of streamlined single-AMP requests. Additional AMPs would provide no performance benefit for those types of queries.

What is TDMaps?

TDMaps is a user created for MAPS feature.

- Contains stored procedures to help the movement of tables
 - Since Reconfig won't be used to move tables, how will a site submit ALTER TABLEs for all of their tables?
- The Stored Procedure named `TDMaps.MoveTablesSP` can be utilized.
 - If the logged-on user is TDMaps:
 - Doesn't require the DROP TABLE privilege (special for TDMaps.MoveTablesSP).
 - For example, assume you want to move a table to the TD_1AmpSparseMap_4Nodes map. The GRANT MAP privilege is needed.
 - Since TDMaps created TD_1AmpSparseMap_4Nodes, the TDMaps user automatically has GRANT MAP TD_1AmpSparseMap_4Nodes.
 - If the logged-on user is not TDMaps:
 - The logged-on user needs DROP TABLE privilege on table(s).
 - GRANT MAP for any sparse map is needed for the logged-on user
 - For example, since TDMaps created TD_1AmpSparseMap_4Nodes, the GRANT MAP TD_1AmpSparseMap_4Nodes to logged-on user is necessary.

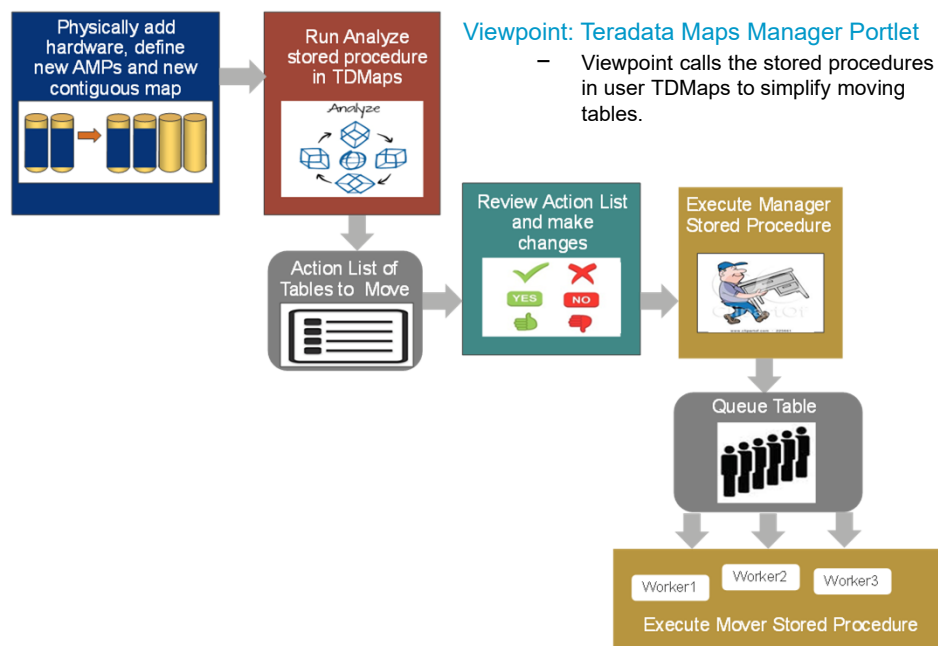
Within the TDMaps system user, there are many SQL stored procedures for automating system expansion and map assignments. The procedures in TDMaps fall into two main categories: Advisor and Mover.

- **Advisor** procedures analyze tables to recommend the best map for each table. The recommendations can optionally be customized and used as input for the Mover.
- **Mover** procedures redistribute the data for a specified list of tables based on new maps. Mover can use multiple worker sessions to achieve the desired level of concurrency. One manager session schedules the execution of groups of related actions and coordinates the worker sessions.

The ALTER TABLE command has been enhanced for use with maps so that when it sees the map name clause, it performs differently from the traditional ALTER TABLE. The TDMaps.MoveTablesSP procedure that is also used by the Viewpoint portlet to perform table moves is basically a packaging of the enhanced ALTER TABLE.

Although they are both manual approaches, one difference between calling the procedure and submitting the ALTER TABLE is that the procedure includes a summary-level statistics collection request against that table after it has been moved. When using the ALTER TABLE command outside of the procedure, remember to recollect summary statistics on the table when the ALTER completes. Another difference is the requirement for the DROP TABLE privilege on the table being moved to the new map. If the procedure is being executed by user TDMaps, the DROP TABLE privilege is not needed. If the procedure is being executed by another user, that user needs the DROP TABLE privilege just as if that user issued the ALTER TABLE MAP statement themselves.

Steps to Move Tables



Use the Teradata Viewpoint portlet MAPS Manager to determine if tables are using optimal maps and, if necessary, switch them to more suitable maps on a flexible schedule.

There are several advantages of using Viewpoint, and it is recommended that the Analysis job be your starting point for the movement of tables across maps. Before a Mover job can be initiated, an Analysis job must evaluate each table undergoing analysis at the same time. The Analysis job may suggest groupings of tables from among those submitted with the recommendation that they be moved together. This might be the case if tables perform primary index joins. The Analyze job also identifies candidate small tables suitable for sparse maps.

An Analysis job produces an action list that becomes input to the subsequent Mover job. This list is materialized in the database as a table named TDMaps.ActionsTbl. The size of the action list depends on how many tables and/or databases were submitted to the Analysis job.

After the Analysis job completes, the administrator can review the recommendations that have been made and make modifications. A single Mover job is then executed, and typically includes all the tables in the action list. The Mover job calls two different procedures:

- A Manager procedure, which inserts all the table names in the actions table to be included in the move into a Mover queue table.
- TDMaps.MoveTablesSP procedure, which consumes from the queue table and initiates an ALTER TABLE for each table in the queue.

More than one table can be moved to the new map by calling more than one TDMaps.MoveTablesSP procedure. If using Viewpoint, this is done by defining multiple “workers

ALTER TABLE Possible Issues

- Join/Hash Index with ROWID will be marked invalid after NoPI table is ALTERed.
- Table that has permanent journaling enabled will cause the ALTER will fail
- Not enough free permanent space to ALTER table (space issues).
 - Depends on number of workers
 - ALTER TABLE uses INSERT-SELECT of the entire table so 2x space is used.
 - CP and NoPI tables use 3x space (spool is needed to generate new RowIDs)
 - Administrator must allocate space, if necessary, before starting job
 - If there isn't enough space available
 1. Then offline RECONFIG is a choice where the table is added to the reconfig order table.
 - Not recommended
 2. View with UNION ALL
 - For a very large table with partitioning based on time, consider moving only the current partition into the new system-default map, and then using a UNION in a view to connect the history partitions which remain in the older map.

Special consideration should be given to join indexes when used on columnar and “No Primary Index” (NoPI) tables if the join index contains the ROWID derived column of a base table. These join indexes are invalidated when the base table is moved to a new map.

Space Issues

For columnar and No Primary Index (NoPI) tables, three times the space is required because an intermediate spool is used to sort the rows. This sort is required so that new row IDs can be generated for those types of tables. An extra step can be seen in the Explain text for the INSERT-SELECT in cases such as this.

The table below delineates the tradeoffs involved in using Reconfig versus the Teradata Viewpoint Maps Manager portlet Mover job (or manual ALTER TABLE) when considering how to get a very large table to a new contiguous map:

Space requirements

- Reconfig: Doesn't require any additional space, a practical choice when there is a shortage of free space on old AMPs.
- Viewpoint Mover / ALTER TABLE: Requires free space on each AMP (including the old AMPs) for the new table rows, total space needs will be double the current table size.

Availability

- Reconfig: Table data, as well as the entire system, is unavailable during the Reconfig process.
- Viewpoint Mover / ALTER TABLE: Table data is available during the move.

GRANT MAP

- Before the MAP word can be used in a SQL statement, the map must first have been granted. An exception is if the map is the default map (TD_Map1 or after 1st expansion TD_Map2).
 - How can a customer restrict/force map usage for a user or group of users?
 - The answer is: A User/Database/Profile can have an optional default map which can be specified and used to force specific map usage.
- Creator of sparse map automatically gets GRANT MAP.
- You can Grant Map to a Role or User.
 - `GRANT MAP TD_1AmpSparseMap_1Node to GL_ACCESS_R;`
- DIPMAPS calls TDMaps.CreateInitialSparseMaps to create two sparse maps.
 - The `TDMaps` user automatically gets the GRANT MAP privilege on these created sparse maps.

Before you can add the clause “MAP = map-name” to a CREATE or an ALTER command, you must have been granted the map. However, this grant is not required when the map that you specify in your request is your default map. So, if you are only creating and moving tables within the scope of your own default map, then you do not have to be concerned with having new privileges established.

Your default map is determined first by the map-name carried in your Profile, if one exists. Secondly, it is determined by the map-name associated with your User record, if one exists. If neither Profile nor User carries a map-name, then the system-default map established by the Reconfig utility becomes your default map.

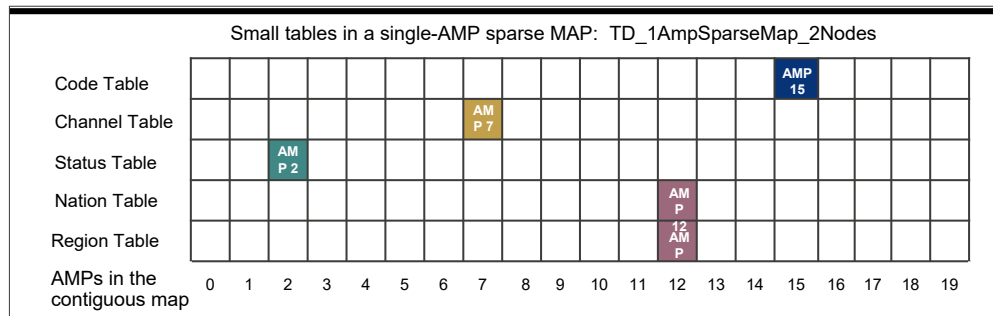
In addition, MAPS feature introduces two privileges: CREATE MAP and DROP MAP. These privileges must be granted for an administrator to be able to create maps and drop maps. These privileges can be revoked.

Additionally, before the MAP clause can be used in a SQL statement such as CREATE TABLE/DATABASE/USER/ PROFILE, the map itself must first be granted via a GRANT MAP statement unless the map specified is the same as the map that would be chosen by default. User tables, join indexes, and hash indexes always have a map, but specifying the map is optional. If the map is not specified, a default map is chosen.

For example, if TD_Map2 is your default map, you can issue SQL without either naming a map or being granted that map. The table is created in your default map, TD_Map2.

Sparse Maps

- Used for small tables (some AMPs don't have any rows)
- **COLOCATION** determines which AMPs by hashing name using parent contiguous map
- AMPs not chosen until CREATE TABLE:



```
CREATE TABLE Nation,  
MAP=TD_1AMPSparseMap_2Nodes  
COLOCATE USING Co_NationRegion  
(NationKey INTEGER,  
  Name      VARCHAR(10),  
  RegionKey INTEGER)  
UNIQUE PRIMARY INDEX (NationKey);
```

```
CREATE TABLE Region,
MAP=TD_1AMPSparseMap_2Nodes
COLOCATE USING Co_NationRegion
  (RegionKey INTEGER,
   Name      VARCHAR(10))
UNIQUE PRIMARY INDEX (RegionKey);
```

A contiguous map covers a range of AMPs, usually across some number of nodes in the system. A sparse map is the opposite; it covers just a single AMP, or under some conditions a small number of AMPs. A sparse map does not exist in isolation within its own dedicated AMP or AMPs, but rather it is defined within a parent contiguous map. The rows of a table targeted to a sparse map coexist with the rows of tables that belong to the parent contiguous map.

A sparse map is intended as a destination for the rows of “small tables”. A small table that would be suitable for a single-AMP sparse map is one that contains 128KB or less, after removing table header overhead. If the table is being considered for a multi-AMP sparse map, the upper limit of suitability is the number of AMPs within the multi-AMP sparse map times 128KB.

For example, if the permanent space of a small table minus table header overhead is 100KB, then that table is suitable for a single-AMP sparse map. If the perm space of a table (minus table header overhead) is between 500KB, it would be suitable for a 4-AMP sparse map ($128\text{KB} * 4 \text{ AMPs} = 512\text{KB}$), and so forth.

Accessing tables with small amounts of data, where some AMPs have no rows, can be inefficient on large systems. Unnecessary resources are spent trying to find rows on AMPs that do not carry them. If that large system is expanded, the number of empty AMPs supporting a small table increases, driving up this overhead. Using the default single-AMP sparse map for a small table allows all-row access to become a single-AMP retrieve step that only runs on the one AMP where all the data is located. While particularly helpful for very large systems with thousands of AMPs, moderate-sized or smaller systems may also benefit from sparse maps, although to a lesser degree.

Colocation Name

Sparse Tables and the Colocation Name

- Tables in sparse maps reside on a subset of AMPs in the parent contiguous map.
- MAPS introduces the *Colocation Name* parameter, the purpose of which is twofold:
 - It is the mechanism used to determine which AMPs a table utilizing a sparse map will reside on.
 - Can be used to insure that two tables with the same sparse map are colocated on the same AMPs.
- The *Colocation Name* parameter is specified as part of a CREATE TABLE statement and can have a specified value or a default value.
 - The default Colocation Name is DatabaseName_TableName. For example,
- The *Colocation Name* can only be specified for tables that utilize sparse maps.
- To view Colocation name, either SHOW TABLE or use the Colocation column in DBC.TablesV.
 - To list the Colocation names in a system,

```
TFACT_Nation_S
```

```
SELECT DISTINCT(ColocationName) FROM DBC.TablesV ORDER BY 1;
```

The user may or may not specify a colocation name at the time the table is created or moved. If one is not specified, the system constructs one from the table's database name, underscore, and the table's name. If you issue a SHOW TABLE command, you can see both the sparse map name and the colocation name associated with the table.

Selecting a Colocation Name

If you have small tables that require colocation, then it is recommended that you come up with a meaningful colocation name to distinctly represent those tables. If you do not have a need for collocating multiple tables, let the system generate the colocation name for you and simply ignore the colocation naming option.

The colocation name is put through the hashing algorithm and the resulting hash bucket is used to identify which AMP(s) will house that small table. This process that selects an AMP or AMPs only considers AMPs that make up the parent contiguous map.

To view all colocation names in a system,

```
SELECT DISTINCT(ColocationName) FROM DBC.TablesV ORDER BY 1;
```

Miscellaneous SQL and Hash Functions

```
SHOW MAP TD_Map1;

CREATE MAP TD_Map1
FROM TD_Map1
CONTIGUOUS
AMP BETWEEN 0 AND 15;
```

```
SHOW MAP TD_1AmpSparseMap_1Node;

CREATE MAP TD_1AmpSparseMap_1Node
FROM TD_Map1
SPARSE
AMPCOUNT = 1;
```

```
SELECT MapName FROM DBC.MapsV WHERE SystemDefault = 'Y';
```

```
SELECT HASHAMP(MAP = TD_MAP1) + 1 AS #AMPs;      OR
SELECT HASHAMP (HASHBUCKET (HASHROW (0)) MAP = TD_MAP1) + 1 AS #AMPs;
```

```
SELECT HASHAMP (HASHBUCKET (HASHROW (0))
  MAP = TD_1AmpSparseMap_1Node
  COLOCATE USING Co_NationRegion) AS AMP#;
```

```
SELECT HASHBAKAMP (HASHBUCKET (HASHROW (0))
  MAP = TD_1AmpSparseMap_1Node
  COLOCATE USING Co_NationRegion) AS FallBackAMP#;
```

Results

MapName
TD_MAP1

#AMPs
16

AMP#
2

FallBackAMP#
3

There are two different designations that may be assigned to a contiguous map, one at the system level and one at the user level. Both these designations use the term “default” and are explained below:

- **System-default:** Reconfig sets what is called the “system-default” map. This is the new, larger contiguous map that is created by Reconfig. SystemDefault is the name of the column in the DBC.Maps table.
- **Default:** An assigned, specified “default” map is chosen by the database based on the Profile→User→System hierarchy.

Recommendation: Rely on the System-Default Map

It is recommended that the system-default map be used as much as possible as a means of associating new tables with maps, rather than coding specific maps for a given set of users. This guarantees that new tables will be located in the most expansive map and benefit from the highest level of parallelism. In addition, when the system expands, any map entries in Database, Profile, or User records will have to be revisited.

HASHAMP / HASHBAKAMP Hash-Related Functions

You can specify an existing contiguous or sparse map for the table. Optionally, you can specify a colocation name so that tables can share a sparse map. When two tables that share the same sparse map are joined on a primary index or primary AMP index, the tables reside on the same AMPs to avoid a redistribution of the rows.

Summary

Why is the MAPS feature useful?

- Eliminates the extended downtime for Reconfig
- Improves performance for small tables by putting them in **Sparse Maps**

TDDMaps:

- Contains stored procedures to help the movement of tables since Reconfig will not be moving the tables.
- Viewpoint: Teradata Maps Manager Portlet
 - Viewpoint calls the stored procedures in user TDDMaps to simplify moving tables.

Additional considerations comparing Reconfig to Viewpoint Mover / ALTER TABLE:

Restart support

- Reconfig: In case of failure, can restart and continue from the cylinder where the job left off.
- Viewpoint Mover / ALTER TABLE: No restart or rollback, will drop the target table, source still available, can restart the job.

Control over execution time

- Reconfig: If runs longer than expected, can't stop the job once it starts.
- Viewpoint Mover / ALTER TABLE: Can abort the job and immediately access the source data.

Special caveats

- Reconfig: Will prolong system down time.
- Viewpoint Mover / ALTER TABLE: Not a viable choice if adequate free space is not available.

If neither of those two options satisfies your needs, there is a third option: Split the table in two and use a UNION within a view. With a very large table that is partitioned by date, you have the option of only moving the current partition of that table to the new contiguous map, and leaving the majority of the partitions in the old map. Over time, more and more current data will be inserted into the version of the table in the new, more expansive map. Also, over time, older partitions in the old map can be dropped when no longer needed and more recent partitions in the old map can be moved to the new map; eventually, the table on the old map will be empty and can be dropped.



Module 17: Review Questions (1 of 2)

1. Which special map is added by Config/Reconfig when expanding the system for the 1st time?
 - a. TD_MAP1
 - b. TD_MAP2
 - c. TD_GlobalMap
 - d. TD_DataDictionaryMap
2. What type of object is TDMAPS?
 - a. User
 - b. View
 - c. Table
 - d. Special Map
3. What are potential ALTER TABLE issues with the MAPS feature?
 - a. Not enough free permanent space to ALTER table
 - b. ALTER TABLE will fail for tables that have permanent journaling enabled
 - c. Join/Hash Index with ROWID will be marked invalid after a NoPI table is ALTERed
 - d. All of the above.

Check your understanding of the concepts discussed in this module by completing the review questions as directed by your instructor.



Module 17: Review Questions (2 of 2)

1. True or False: Before the MAP key word can be used in a SQL statement, the map (other than the default map) must first have been granted to the user.
2. True or False: When creating a table and including the COLOCATE USING option with a Sparse Map the COLOCATE name you pick must first exist.
3. True or False: Every table in the system is assigned to a map.
4. True or False: NoPI, including Column Partition tables, tables need 3x space to move because spool is needed to generate new row ids.
5. True or False: Users, databases, and profiles have an optional default map which can be specified.

Check your understanding of the concepts discussed in this module by completing the review questions as directed by your instructor.



Lab Exercise 17-1 (1 of 2)

Purpose

In this lab, you will populate tables using the sparse MAPS capability.

What you need

Populated TRNG_PD.Nation_C and TRNG_PD.Region_C tables.

Tasks

1. Create two Contiguous tables Nation_C and Region_C from TRNG_PD.Nation_C and TRNG_PD.Region_C. (Note: these tables do not have all of the nations/regions, just a small subset of data)

```
CREATE TABLE Region_C AS TRNG_PD.Region_C WITH DATA;  
CREATE TABLE Nation_C AS TRNG_PD.Nation_C WITH DATA;
```

2. Find the system single AMP (1) Sparse MAP by using the view DBC.MapsV.

```
SELECT MapName FROM DBC.MapsV;
```

Name of Sparse Map for 1 AMP – _____

3. Use SHOW TABLE and create new Nation_S and Region_S sparse tables specifying the 1 AMP sparse MAP name and use a COLOCATE value of Co_NationRegion for both tables (i.e., COLOCATE USING Co_NationRegion).

This lab exercise provides the opportunity to use the Teradata 16.10 MAPS feature using 2 small tables.

```
CREATE SET TABLE TRNG_PD.Nation_C, Fallback, MAP = TD_MAP1  
  (NationKey INTEGER,  
   NationName VARCHAR(25) CHARACTER SET LATIN NOT CASESPECIFIC,  
   RegionKey INTEGER)  
UNIQUE PRIMARY INDEX (NationKey);  
  
CREATE SET TABLE TRNG_PD.Region_C, Fallback, MAP = TD_MAP1  
  (RegionKey INTEGER,  
   RegionName VARCHAR(25) CHARACTER SET LATIN NOT CASESPECIFIC)  
UNIQUE PRIMARY INDEX (RegionKey);
```

To specify a sparse MAP and optionally the COLOCATE option, the following SQL can be used.

```
CREATE TABLE tablename, Fallback,  
  MAP = TD_1AmpSparseMap_1Node COLOCATE USING Co_NationRegion  
  (column1 INTEGER,  
   column2 VARCHAR(25),  
   column3 INTEGER)  
UNIQUE PRIMARY INDEX (column1);
```



Lab Exercise 17-1 (2 of 2)

- Populate the two Sparse tables with INSERT-SELECT from the two contiguous tables.
- Use the DBC.TableSizeV and determine which two AMPs the sparse tables are using.

Why are two AMPs used per sparse table? _____

- Compare the sizes of Nation_S and Region_S to the Nation_C and Region_C tables.

Size of Nation_S = _____

Size of Nation_C = _____

Size of Region_S = _____

Size of Region_C = _____

- Use the DBC.SparseMapAmpsV to determine which AMP has the primary data of your Sparse tables. AMP # = _____

```
SELECT * FROM DBC.SparseMapAmpsV where DatabaseName = DATABASE;
```

- Execute the following EXPLAIN of a join of the Contiguous tables.

Is the data redistributed for the join? Yes/No _____

```
EXPLAIN  SELECT      NationName, RegionName
          FROM        Nation_C
          INNER JOIN   Region_C
          ON           Nation_C.RegionKey = Region_C.RegionKey ;
```

Execute the same EXPLAIN using the Nation_S and Region_S sparse tables. Is the data redistributed for the join? Yes/No _____

- Optional: Create a new Nation_S2 table and a new Region_S2 table without using the COLOCATE option. Use the DBC.SparseMapAmpsV to determine which AMP has your Sparse tables. Are the two tables on the same AMP? Yes/No _____

SUM of Perm space at AMP level using the DBC.TableSizeV view.

```
SELECT DatabaseName, TableName, Vproc, SUM(CurrentPerm)
FROM   DBC.TablesizeV
WHERE  DatabaseName = DATABASE
AND    TableName IN ('Nation_S', 'Region_S')
GROUP BY 1,2,3
ORDER BY 1,2,3;
```

SUM of Perm space for all AMPs using the DBC.TableSizeV view.

```
SELECT DatabaseName, TableName, SUM(CurrentPerm)
FROM   DBC.TablesizeV
WHERE  DatabaseName = DATABASE
AND    TableName IN ('Nation_S', 'Region_S', 'Nation_C', 'Region_C')
GROUP BY 1,2
ORDER BY 1,2;
```

To view the AMPs associated with your tables assigned to a sparse map, use the following view.

```
SELECT * FROM DBC.SparseMapAmpsV where DatabaseName = DATABASE;
```

To view objects that are associated with sparse maps and COLOCATE value, use the following view.

```
SELECT * FROM DBC.ObjectsInSparseMapsV;
```



Module 18: Recap

Teradata Vantage Administration

Copyright © 2007–2023 by Teradata. All Rights Reserved.

Survey Question

How likely are you to recommend this class to a colleague?



Objectives

After completing this module, you will be able to:

- Recap key concepts covered in this course

Teradata DBA Review

Teradata Database Administration

- Creating and using the database environment and hierarchy
- The Data Dictionary/Directory
- Space Allocation and Usage
- Management of Users and Databases
- Controlling access to system via Access Rights, Roles, and Profiles
- Monitoring system activity and logging user access and queries
- Workload Management (TASM)
- Utilization of tools such as Teradata Administrator, Teradata Studio and Viewpoint
- System utilities – administrative, maintenance, and recovery

GOAL – Administration of a Teradata database using DBA commands and utilities

This slide lists some of the key topics that were covered in this course.

Dictionary Tables to Maintain

Reset accumulators and peak values using DBC.AMPUsage view and the ClearPeakDisk macro provided with the software.

DBC.Acctg
Resource usage by Account/user

DBC.DataBaseSpace
Dbase and Table space accounting

Teradata automatically maintains these tables, but good practices can reduce their size.

DBC.AccessRights
User Rights on objects

DBC.RoleGrants
Role rights on objects

DBC.Roles
Defined Roles

DBC.Accounts
Account Codes by user

Archive these logging tables (if desired) and purge information 60-90 days old. Retention depends on customer requirements.

DBC.SW_Event_Log
Database Console Log

DBC.ResUsage
Resource monitor tables

DBC.EventLog
Session logon/logoff history

DBC.AccLogTbl
Logged User-Object events

DBC.DBQL tables
Logged user/SQL activity

Purge these tables when the associated removable media is expired and over-written.

DBC.RCEvent
Archive/Recovery events

DBC.RCConfiguration
Archive/Recovery config

DBC.RCMedia
VolSerial for Archive/Recovery

You need to maintain some dictionary tables. The following pages list these tables and describe the maintenance.

Plan and Follow-up

Review the material you have learned and experienced in this course. From it, develop a checklist of tasks. For example:

1. Set up a job that periodically checks the size of your dictionary tables.
2. Set up a job that periodically checks the size of your application databases. Evaluate them for even data distribution on AMPs. Ensure that users' permanent space is being used efficiently. Reallocate space if necessary.
3. Set up and document the definition of users, roles, profiles, privileges, and an accounting system (if you don't have one already).
4. Establish rules to capture queries (DBQL) for workload and performance analysis.
5. Establish rules to capture access logging (security audit analysis).
6. Capture ResUsage data and install and run the ResUsage macros at regular intervals. Evaluate and review reports for even distribution of processing.
7. Establish plans to move ResUsage, DBQL, etc. data periodically to a separate database (e.g., PDCR).
8. Install and configure Viewpoint for system monitoring.
9. Verify adequate spool and temporary space in the system (e.g., Spool_Reserve).
10. Check the allocation of space for the Crashdumps database.

Establish a set of procedures that will help you administer the Teradata database. Document these procedures and periodically refer to them.

Things You Never Have to do with Teradata

Implementation

1. Create and format data files to hold the data and the indexes.
2. Determine the physical location of each table and index partition or simple tablespace.
3. Write programs to determine how to divide data into partitions.
4. Code the space allocation for each partition or underlying file structures.
5. Embed partitioning assignments into CREATE TABLE statements.
6. Code the definition and allocation for temporary work space.
7. Create, size, and determine the content of tablespaces.
8. Associate tables and/or queries with degrees of parallelism.
9. Add hints or otherwise rewrite SQL.
10. Determining the level of parallelism to be assigned to tables or indexes.
11. Assign and manage special buffer pools for parallel processing.
12. Create rollback segments or log files.
13. Ensure that the data is spread evenly across disks and controllers.
14. Build summary tables before end users can access the data warehouse.
15. Carefully build indexes on tables and summary tables to support index only access for performance, based on known queries – these indexes may or may not aid in ad hoc access.
16. Build and partition materialized view logs.
17. Build and partition indexes on top of materialized views.
18. Determine how materialized views are updated, asynchronously or synchronously.

This slide lists a number of database maintenance functions that you never have to do with the Teradata database.

Things You Never Have to do with Teradata (cont.)

Support

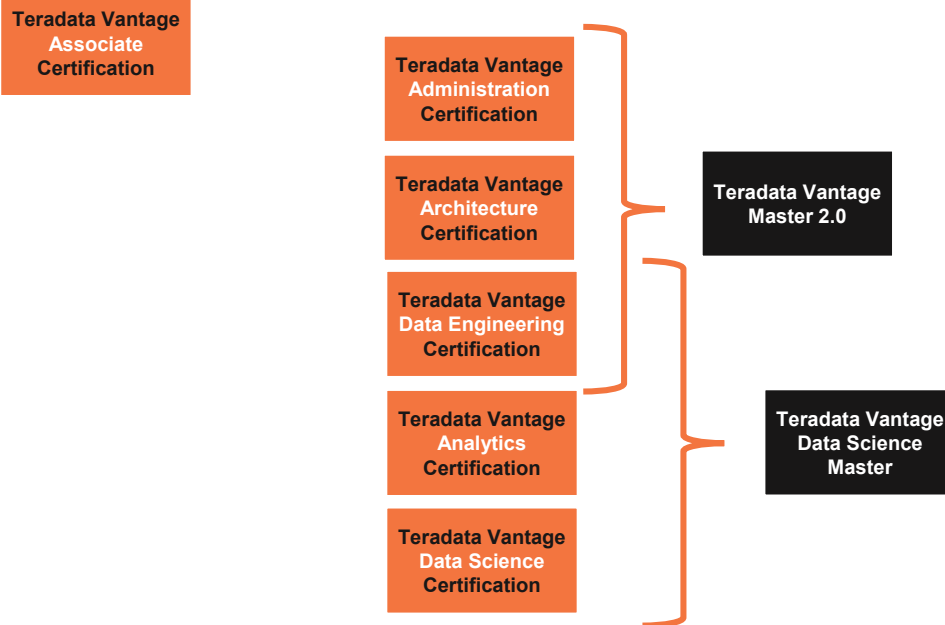
19. Monitor partition size.
20. Monitor and tune temporary work and sort spaces.
21. Monitor and tune buffer pool assignments.
22. Monitor and tune parameters and control blocks that enable parallel execution.
23. Perform periodic table and index reorgs (unloads and reloads, dropping and rebuilding).
24. Convert data types of mainframe data sets prior to a data warehouse load.
25. Setting up multiple load jobs from a mainframe to the data warehouse in order to load a single table in parallel.
26. Manually restart the multi-step load process when failure occurs.
27. Sort and/or split the data before a load job.

Growth and Leverage

28. Alter the parallelism assignments as the number of users or data volume increases.
29. Expand partition boundaries or relocate partition data sets.
30. Add or delete table or index partitions as tables grow.

This slide lists a number of database maintenance functions that you never have to do with the Teradata database.

Teradata Certification Exams



Depending upon the tests that are completed, you can earn various Teradata Certified designations. The Teradata Certification exams require knowledge plus experience with Teradata release 16 and higher. This manual will help you prepare for these Teradata tests, but many of the test questions are scenario-based and Teradata experience is needed to answer these types of questions. The websites listed below provide much more information. The first website listed below can be used to understand the concepts and objectives that are on each exam. These objectives will give you a better idea of what to study.

Teradata Certification (TCPP) Website: www.teradata.com/University/Certification

Bypass Exams_with_Prior_Teradata_14_Certification information:
www.teradata.com/University/Certification/Vantage-Certifications#BypassExams

Registration Information: www.pearsonvue.com/teradata

TCCP Contact Information: email: certification.Teradata@Teradata.com
phone: 1-800-845-2273

Disclaimer: The Teradata Certification tests include questions from a mix of sources and **require experience**, especially on tests other than the Associate Exam.



Appendix A: Review Question Solutions

This Appendix contains answers to the review questions for the course modules.

Copyright © 2007–2023 by Teradata. All Rights Reserved.



Module 1: Review Questions

1. True or False: You should use system user DBC to create application users and databases.

You should create and logon as an administrative user to perform these tasks.

2. True or False: A database or user can have multiple Owners, but only one Creator.

3. True or False: An Owner and a Parent are two different terms that mean the same thing.

4. True or False: An Owner and a Creator are two different terms that mean the same thing.

"Creator" is used to mean the one and only one user who creates a database object.

5. True or False: An administrative user (e.g., Sysdba) will never have more permanent space than DBC.

Sysdba can be allocated more permanent space than DBC.

6. True or False: The GIVE statement transfers a database or user space to a recipient you specify. It does not automatically transfer all child databases.

The GIVE statement automatically transfers all child databases, users, tables, view and macros the transferred object owns.



Module 2: Review Questions

1. Which data interface is available with Teradata Studio?
 - a. CLIV2
 - ☒ b. JDBC
 - c. ODBC
 - d. Teradata .Net
2. Which piece of query information is not available in the History Window for Teradata Studio?
 - a. User name
 - ☒ b. Query band
 - c. Elapsed time
 - d. Data source name
 - e. Number of rows returned
3. What is a technique to execute multiple statements as a multi-statement request in Teradata Studio? Use menu SQL > Execute All



Module 3: Review Questions

1. With a populated table, which two actions are allowed with the ALTER TABLE command.
 - a. Drop all of the ranges
 - b. Add or drop ranges from the partition "ends"
 - c. Change the columns that comprise the primary index
 - d. Add or drop special partitions (NO RANGE, UNKNOWN)
2. What are two advantages of partitioning a table?
 - a. Fast delete of rows in partitions
 - b. Fewer AMPs are involved when accessing data
 - c. Range queries can be executed without doing an all-rows scan
 - d. Faster access for a PI value access than for a non-partitioned table
3. For NoPI tables, what are two ways in which the Random Generator is executed?
 - a. At the PE level for ad hoc SQL requests
 - b. At the TPT Stream client level for array insert operations
 - c. At the PE level with TPT Stream for array insert operations
 - d. At the AMP level for INSERT-SELECT into an empty NoPI table
4. Which two types of usage normally apply to a composite NUSI that is hash-ordered?
 - a. Covering index
 - b. Equality value access
 - c. Inequality value access
 - d. Non-covering range access



Module 4: Review Questions

1. True or **False**: The DBC.DatabasesV view only contains information about databases; users are not included in this view. *Users are included in this view.*
2. **True** or False: The DBC.UsersV view only contains information about users; databases are not included in this view.
3. True or **False**: The DBC.ShowTblChecksV view provides information about table level check constraints, unnamed and named column check constraints. *Unnamed check constraints not provided.*
4. True or **False**: All of the data dictionary tables are Fallback protected.
Non-hashed tables are not fallback protected.
5. If a child table exists and the parent table doesn't, the reference constraint is marked as _____.
 - a. Invalid
 - c. Unresolved**
 - b. Missing
 - d. Inconsistent
6. After executing the ALTER TABLE ... ADD FOREIGN KEY ... statement, Foreign Key values that are missing in the parent table are marked in an error table and are known as _____ rows.
 - a. Invalid**
 - c. Unresolved
 - b. Missing
 - d. Inconsistent



Module 5: Review Questions

1. True or **False**: Space limits are enforced at the table level.
2. True or **False**: When you use the GIVE statement to transfer a user, the SPOOL and TEMPORARY max values are adjusted to the value of the new immediate owner.
3. True or **False**: A child user cannot have a SPOOL limit greater than the immediate parent.
4. The DBC. TablesizeV view provides disk space usage at the table level and excludes table ALL.
5. The DBC. DiskSpaceV view only provides disk space usage at the database level.



Module 6: Review Questions

1. True or False: You can only give the authority to use the CREATE DATABASE and CREATE USER statements to certain types of users.
2. True or False: AMPUsage views can be used to determine which users are consuming CPU and/or logical disk I/O resources.
3. True or False: A user can use the MODIFY USER statement to change their password, default database, and date format.
4. When creating a new user, which option defaults to the creator's value.
 - a. SPOOL
 - b. FALLBACK PROTECTION
 - c. All of the Account_IDs
 - d. DEFAULT DATABASE
5. When creating a new user, which three options are required with the CREATE USER command.
 - a. SPOOL
 - b. PERMANENT
 - c. User name
 - d. PASSWORD



Module 7: Review Questions

Answer the following questions:

1. List two advantages of utilizing profiles. Change common user attributes for large groups of users.
Specify password security controls for groups of users.
2. If a user's profile is set to NULL, which two are immediately affected in the current session?
 - ☒ a. SPOOL value
 - b. Session priority
 - c. Default database
 - ☒ d. TEMPORARY value
3. Which two attributes can be set in a Profile?
 - a. Default Role
 - ☒ b. Query_Band
 - ☒ c. Default Database
 - d. Permanent Space

Match each term to the definition.

- | | |
|-----------------------------------|---|
| 4. <u>b</u> CREATE PROFILE | a. Lists all words that can not be included in passwords |
| 5. <u>c</u> DBC.ProfileInfoV | b. System access right needed to create a profile |
| 6. <u>a</u> DBC.RestrictedWordsV | c. Lists the profiles currently in the system |
| 7. <u>d</u> DBC.SecurityDefaultsV | d. May be used to change password rules at the system level |

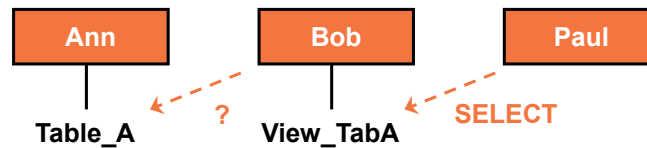


Module 8: Review Questions

1. True or False: There are only two ways used to create new access rights: explicit and implicit.
2. True or False: The primary statements you use to manage access rights are GRANT, REVOKE, and GIVE.
3. The ALL option on the GRANT command grants privileges to a database or user and all of its current and future descendants.
4. The INSERT, REFERENCES, SELECT, and UPDATE access rights can be granted at the column level.
5. Which user name is used when you want to grant an access right to every user? PUBLIC
6. Given the following: Ann owns Table_A, Bob creates View_TabA and grants SELECT on View_TabA to Paul.

What access right does Ann give Bob on Table_A so Paul can use View_TabA to access Table_A?

SELECT WITH GRANT OPTION





Module 9: Review Questions

1. What are two advantages of utilizing Roles? (choose two)
 - ☒ a. Reduces the number of rows in the access rights table.
 - ☐ b. Eliminates the need for the access right checks in the PE.
 - ☐ c. Helps reduce the number of required profiles in a system.
 - ☒ d. Simplifies assigning access rights for an application to a user.
2. If a user executes this command (SET ROLE Applx_R;), which two statements are true? (choose two)
 - ☒ a. Access rights granted to PUBLIC are still available to the user.
 - ☐ b. Access rights granted to the user are not available to the user.
 - ☒ c. Access rights assigned to Applx_R and any nested roles are available to the user.
 - ☐ d. Access rights granted to any role granted to the user are still available to the user.
3. How many levels of role nesting are allowed? 1
4. ☒ True or False: Roles may only be granted to users and other roles.

Match each term to the definition.

- | | |
|---------------------------------|---|
| 5. <u>e</u> WITH ADMIN OPTION | a. Established by the SET ROLE command |
| 6. <u>c</u> CREATE ROLE | b. Lists the roles currently in the system |
| 7. <u>b</u> DBC.RoleInfoV | c. System access right needed to create a role |
| 8. <u>d</u> DBC.UserRoleRightsV | d. Lists all of a user's role rights – including nested roles |
| 9. <u>f</u> DEFAULT ROLE | e. Allows the user to assign other users to the role |
| 10. <u>a</u> Current role | f. Option with the MODIFY USER statement |



Module 10: Review Questions

1. True or False: Although usernames are the basis for identification in the system, username information usually is not protected information.
2. True or False: A REVOKE LOGON command for a user removes a GRANT LOGON row in the DBC.LogonRuleTbl for the that user.
3. True or False: To change the minimum number of characters in a valid password from 6 to 8, you would update the DBC.LogonRules table.
4. What does 1024 represent in the DBC.LogonRules view? All of the hosts
5. Which two choices can be used to view host or mainframe sessions?
 - a. Sessions utility
 - b. QrySessn utility
 - c. Gtwwglobal utility
 - d. DBC.SessionInfoV view
6. Which view is used to determine why a user logon has failed?
 - a. DBC.LogonsV
 - b. DBC.LogOnOffV
 - c. DBC.AccessLogV
 - d. DBC.SessionInfoV
 - e. DBC.LogonEventsV



Module 11: Review Questions

1. In order to use the BEGIN/END LOGGING commands, what is the name of the system macro you need execute permission on? DBC.AccLogRule
2. How is this macro initially created? When the DIP script (DIPACC) is executed.
3. What is a negative impact of the following statement?
BEGIN LOGGING WITH TEXT ON EACH
Potentially a lot of entries are placed in the dictionary and would require a lot of PERM space.
4. With DBQL, what is the size of the default text captured for queries? 200 characters
5. ☒ True or ☐ False: With DBQL, the LIMIT SUMMARY option cannot be used with any other LIMIT.
6. ☐ True or ☒ False: With DBQL, the WITH SQL option only captures a maximum of 10,000 characters.
7. ☐ True or ☒ False: With DBQL, the option WITH ALL ON ALL is typically a good choice.
8. ☒ True or ☐ False: With DBQL, default rows are logged in the DBC.DBQLLogTbl.



Module 12: Review Questions (1 of 2)

1. What type of filter or throttle rule is needed for the following restrictions?

Limit the number of concurrent sessions	<u>Session Throttle</u>
Reject queries based on max processing time	<u>Filter</u>
Reject queries accessing a specific DB	<u>Filter</u>
Limit the number of TPT Load jobs	<u>Load Utility Throttle</u>
Delay more than 20 queries for a specific account	<u>Throttle</u>
2. What is the purpose of the default workload definition name "WD-Default"?
 - a. Default workload for any queries with an enforcement policy of normal.
 - ☒ b. Default workload for any queries that are not associated with a workload.
 - c. Default workload for any queries assigned to Default resource partition.
 - d. Default workload for any queries assigned to Standard resource partition.
3. Which query attribute is not used by the Parsing Engine software to classify a query into a Workload Definition (WD)?
 - a. User name
 - b. Account
 - ☒ c. User Role
 - d. User Profile
 - e. Optimizer estimates



Module 12: Review Questions (2 of 2)

4. Which Teradata application can be used to initially define workload definitions?
- a. Teradata Manager
 - ☒ b. Workload Analyzer
 - c. Dynamic Workload Wizard
 - d. Priority Scheduler Administrator
5. Place the following control options in the proper sequence from 1 to 4 as they are acted upon by Teradata software.
- a. 2 System throttles
 - b. 4 Exception criteria
 - c. 3 Workload throttles
 - d. 1 Filters



Module 13: Review Questions

1. Which portlet is used to view active queries across multiple systems?
 - ☒ a. My Queries
 - b. Query Monitor
 - c. Query Spotlight
 - d. SQL Scratchpad History
2. Which portlet is used to establish TASM filters and throttles?
 - a. Workload Health
 - b. Workload Wizard
 - c. Workload Monitor
 - ☒ d. Workload Designer
3. Which portlet can be used to execute the DBSControl utility?
 - a. Productivity
 - b. TVS Monitor
 - c. System Health
 - ☒ d. Remote Console
4. Which portlet can be used to display AMP Worker Task usage during different hours?
 - a. Productivity
 - b. System Health
 - ☒ c. Metrics Analysis
 - d. Node Resources



Module 14: Review Questions

1. What are two methods of setting the ResUsage logging intervals?

ctl utility

Supervisor SET commands

Match the following tools to its description.

2. d ResUsage tables

a. Set logging rates

3. c Teradata Viewpoint

b. Emulates a target system

4. b Teradata SET

c. Provides session level information

5. a ctl

d. Holds historical resource data



Module 15: Review Questions

1. What is the operating system command to restart Teradata? *tpareset*
2. What is the DB Window supervisor command to restart Teradata?
restart tpa
3. Which of the following choices will cause a Teradata restart?
 - a. SWS hard drive failure
 - b. Single drive failure in RAID 1 drive group
 - ☒ c. Two drive failures in same RAID 1 drive group
 - d. Single power supply failure in a TPA node
 - ☒ e. TPA node CPU failure
 - f. One of BYNETs fails
 - g. LAN connection to SMP is lost



Module 16: Review Questions

1. What are two ways that you initiate a Teradata system utility (e.g., dbscontrol)?
Command-line Viewpoint Remote Console
2. Identify the purpose of the following DBS Control utility parameters.
CenturyBreak determines break point for 21st century; how 2-digit years are interpreted
DateForm sets IntegerDate or ANSIDate default; how dates are displayed and exported
PrimaryIndexDefault determines if tables should be created as PI or NoPI if nothing is specified
SessionMode sets default session mode – BTET or ANSI
3. True or False: The Checktable utility has only two levels of internal table checking.
4. True or False: The Table Rebuild utility rebuilds tables differently depending on whether the table is a fallback or a non-fallback table.
5. The SCANDISK utility does a consistency check within an AMP's file system.
6. The CHECKTABLE utility does a consistency check for a table across all AMPs.
7. The VPROCMANAGER utility can be used to set an offline AMP to online.



Module 17: Review Questions (1 of 2)

1. Which special map is added by Config/Reconfig when expanding the system for the 1st time?
 - a. TD_MAP1
 - ☒ b. TD_MAP2
 - c. TD_GlobalMap
 - d. TD_DataDictionaryMap
2. What type of object is TDMAPS?
 - ☒ a. User
 - b. View
 - c. Table
 - d. Special Map
3. What are potential ALTER TABLE issues with the MAPS feature?
 - a. Not enough free permanent space to ALTER table
 - b. ALTER TABLE will fail for tables that have permanent journaling enabled
 - c. Join/Hash Index with ROWID will be marked invalid after a NoPI table is ALTERed
 - ☒ d. All of the above.



Module 17: Review Questions (2 of 2)

1. True or False: Before the MAP key word can be used in a SQL statement, the map (other than the default map) must first have been granted to the user.
2. True or False: When creating a table and including the COLOCATE USING option with a Sparse Map the COLOCATE name you pick must first exist.
3. True or False: Every table in the system is assigned to a map.
4. True or False: NoPI, including Column Partition tables, tables need 3x space to move because spool is needed to generate new row ids.
5. True or False: Users, databases, and profiles have an optional default map which can be specified.



Appendix B: Solutions to Lab Exercises

This Appendix contains possible solutions to the lab exercises.

Copyright © 2007–2023 by Teradata. All Rights Reserved.



Lab Solutions for Lab 2-1 (1 of 2)

6. Using the Query Window, execute the following query.

```
CREATE TABLE Orders_1Q2021 AS Orders_2021 WITH NO DATA;
```

7. Execute the following query to populate Orders_1Q2021 with orders for January of 2021.

```
INSERT INTO Orders_1Q2021 SELECT * FROM TRNG_DS.Orders_2021  
WHERE orderdate BETWEEN DATE '2021-01-01' AND DATE '2021-01-31';
```

How many rows are in Orders_1Q2021? [12,021](#)

8. Using the History window, recall the query from step #7 in a new SQL editor and modify it to add orders for February and March of 2021 into the Orders_1Q2021 table.

```
INSERT INTO Orders_1Q2021 SELECT * FROM TRNG_DS.Orders_2021  
WHERE orderdate BETWEEN DATE '2021-02-01' AND DATE '2021-03-31';
```

How many rows are now in Orders_1Q2021? [36,075](#)

```
SELECT COUNT(*) FROM Orders_1Q2021;
```

9. Execute the following query by using the drag and drop object feature of the SQL tool you are using.

```
SELECT custid, SUM (ordertotal)  
FROM Orders_1Q2021  
WHERE custid = 21072  
GROUP BY 1  
ORDER BY 1;
```

What is the SUM of the orders (ordertotal) in the 1Q2021 for custid = 21072? [67,125.66](#)



Lab Solutions for Lab 2-1 (2 of 2)

10. Create a view (that utilizes an Access Lock) named "Orders_Jan2021_v" on the Orders_1Q2021 table that includes these columns (orderid, custid, ordertotal, orderdate) and only includes orders for the January of 2021.

```
CREATE VIEW Orders_Jan2021_v
AS LOCKING ROW FOR ACCESS
SELECT orderid, custid, ordertotal, orderdate FROM Orders_1Q2021
WHERE orderdate BETWEEN DATE '2021-01-01' AND DATE '2021-01-31';
```

SELECT the total count of orders from "Orders_Jan2021_v".

```
SELECT COUNT(*) FROM Orders_Jan2021_v;
```

How many orders are available using this view? [12,021](#)

12. Create a parameterized macro named "Orderid_m" which accepts an orderid as a value and returns the single order from "Orders_Jan2021_v".

```
CREATE MACRO Orderid_m (oid_value INTEGER)
AS (SELECT * FROM Orders_Jan2021_v WHERE orderid = :oid_value;);
```

Execute the macro to provide order details for orders 550013 and 551550.

```
EXEC Orderid_m (550013); EXEC Orderid_m (551550);
```

What is the ordertotal for orderid #1781509? [1227.66](#) Orderid #1781939? [1238.66](#)

13. What is shown in photo #7? [DBC 1012](#)



Lab Solutions for Lab 3-1 (1 of 3)

1. INSERT/SELECT all rows from the populated DS.Orders table to your "Orders" table. Verify the number of rows in your table.

```
INSERT INTO Orders      SELECT * FROM TRNG_DS.Orders;
INSERT INTO Orders_2021 SELECT * FROM TRNG_DS.Orders_2021;
SELECT COUNT(*)        FROM Orders;                      Count = 450,000
SELECT COUNT(*)        FROM Orders_2021;                  Count = 150,000
```

2. Use the SHOW TABLE for Orders to help create a new, similar table (same column names and definitions, etc.) named "Orders_RP" that is row partitioned.

```
CREATE TABLE Orders_RP
(orderid          INTEGER NOT NULL,
 custid          INTEGER NOT NULL,
 orderstatus     CHAR(1),
 ordertotal      DECIMAL(9,2) NOT NULL,
 orderdate       DATE FORMAT 'YYYY-MM-DD' NOT NULL,
 orderpriority   SMALLINT,
 clerk          CHAR(16),
 location        SMALLINT,
 shippriority    SMALLINT,
 ordercomment    VARCHAR(79))
PRIMARY INDEX (orderid)
PARTITION BY RANGE_N (orderdate BETWEEN
DATE '2012-01-01' AND DATE '2021-12-31' EACH INTERVAL '1' MONTH, NO RANGE);
```




Lab Solutions for Lab 3-1 (2 of 3)

3. Use the SHOW TABLE for Orders to help create a new, similar table (same column names and definitions, etc.) named "Orders_RP2" that is row partitioned based on the following:

```
CREATE TABLE Orders_RP2
(orderid          INTEGER NOT NULL,
 custid          INTEGER NOT NULL,
 orderstatus     CHAR(1),
 ordertotal      DECIMAL(9,2) NOT NULL,
 orderdate       DATE FORMAT 'YYYY-MM-DD' NOT NULL,
 orderpriority   SMALLINT,
 clerk          CHAR(16),
 location        SMALLINT,
 shippriority    SMALLINT,
 ordercomment    VARCHAR(79))
PRIMARY INDEX (orderid)
PARTITION BY RANGE_N
(orderdate BETWEEN
  CAST ((EXTRACT(YEAR FROM CURRENT_DATE) - 9 ) || '-01-01' AS DATE) AND
  CAST ((EXTRACT(YEAR FROM CURRENT_DATE))      || '-12-31' AS DATE)
  EACH INTERVAL '1' DAY, NO RANGE );
```

4. INSERT/SELECT all of the rows from your Orders and Orders_2021 tables into the Orders_RP table. Verify the number of rows in your table. Count = _____

```
INSERT INTO Orders_RP SELECT * FROM Orders;
INSERT INTO Orders_RP SELECT * FROM Orders_2021;
SELECT COUNT(*) FROM Orders_RP;
```

Count = 600,000



Lab Solutions for Lab 3-1 (3 of 3)

5. INSERT/SELECT all of the rows from the Orders_RP table into the Orders_RP2 table. Verify the number of rows in your table.
Count = _____

```
INSERT INTO Orders_RP2 SELECT * FROM Orders_RP;  
SELECT COUNT(*) FROM Orders_RP2;
```

Count = 600,000



Lab Solutions for Lab 3-2 (1 of 8)

1. Use the SHOW TABLE for Orders to help create a new, similar table (same column names and definitions, etc.) named "Orders_custid" which has custid as the primary index.

```
CREATE TABLE Orders_custid
  (orderid      INTEGER NOT NULL,
   custid       INTEGER NOT NULL,
   orderstatus  CHAR(1),
   ordertotal   DECIMAL(9,2) NOT NULL,
   orderdate    DATE FORMAT 'YYYY-MM-DD' NOT NULL,
   orderpriority SMALLINT,
   clerk        CHAR(16),
   location     SMALLINT,
   shippriority SMALLINT,
   ordercomment VARCHAR(79))
PRIMARY INDEX (custid);
```

Using INSERT/SELECT, populate the Orders_custid table with all rows from your Orders and Orders_2021 tables and verify the count of 600,000 orders in the Orders_custid table.

```
INSERT INTO Orders_custid SELECT * FROM Orders;
INSERT INTO Orders_custid SELECT * FROM Orders_2021;
SELECT COUNT(*) FROM Orders_custid;
```



Lab Solutions for Lab 3-2 (2 of 8)

2. Determine the AMP space usage of the Orders_custid table using DBC.TableSizeV.

```
SELECT      TableName
            ,MIN(CurrentPerm) AS Min_Size
            ,MAX(CurrentPerm) AS Max_Size
            ,(100 - (CAST(AVG(CurrentPerm) AS FLOAT)/
                     NULLIFZERO (MAX(CurrentPerm))* 100)) AS SKEW_FACTOR
FROM        DBC.TableSizeV
WHERE       DatabaseName = DATABASE
AND         TableName = 'Orders_custid'
GROUP BY    1;
```

What is the least amount of permanent space on one of the AMPs? 913,408

What is the greatest amount of permanent space on one of the AMPs? 983,040

What is the skew factor for this table 3.75



Lab Solutions for Lab 3-2 (3 of 8)

3. Use the SHOW TABLE for Orders to help create a new, similar table (same column names and definitions, etc.) named "Orders_NoPI" and does NOT have a primary index.

```
CREATE TABLE Orders_NoPI
  (orderid      INTEGER NOT NULL,
   custid      INTEGER NOT NULL,
   orderstatus  CHAR(1),
   ordertotal   DECIMAL(9,2) NOT NULL,
   orderdate    DATE FORMAT 'YYYY-MM-DD' NOT NULL,
   orderpriority SMALLINT,
   clerk        CHAR(16),
   location     SMALLINT,
   shippriority SMALLINT,
   ordercomment VARCHAR(79))
```

```
NO PRIMARY INDEX;
```

```
INSERT INTO Orders_NoPI SELECT * FROM Orders_custid;
SELECT COUNT(*) FROM Orders_NoPI;
```

```
Count = 600,000
```



Lab Solutions for Lab 3-2 (4 of 8)

4. Determine the AMP space usage of the Orders_NoPI table using DBC.TableSizeV.

```
SELECT  TableName
        ,MIN(CurrentPerm) AS Min_Size
        ,MAX(CurrentPerm) AS Max_Size
        ,(100 - (CAST(AVG(CurrentPerm) AS FLOAT)/
                  NULLIFZERO (MAX(CurrentPerm))* 100)) AS SKEW_FACTOR
FROM    DBC.TableSizeV
WHERE   DatabaseName = DATABASE
AND     TableName = 'Orders_NoPI'
GROUP BY 1;
```

What is the least amount of permanent space on one of the AMPs? 913,408

What is the greatest amount of permanent space on one of the AMPs? 983,040

What is the skew factor for this table? 3.75

Is the distribution similar to the Orders_custid table (yes or no)? Yes

Why or why not? The INSERT-SELECT operation copied blocks within the AMPs.



Lab Solutions for Lab 3-2 (5 of 8)

5. DELETE all the rows in Orders_NoPI.

```
DELETE Orders_NoPI;
```

Using INSERT/SELECT with the `HASH BY RANDOM` option, populate your `Orders_NoPI` with all rows from your `Orders_custid` table.

```
INSERT INTO Orders_NoPI SELECT * FROM Orders_custid HASH BY RANDOM;
```



Lab Solutions for Lab 3-2 (6 of 8)

6. Determine the AMP space usage of your Orders_NoPI table using DBC.TableSizeV.

```
SELECT  TableName
        ,MIN(CurrentPerm) AS Min_Size
        ,MAX(CurrentPerm) AS Max_Size
        ,(100 - (CAST(AVG(CurrentPerm) AS FLOAT)/
                  NULLIFZERO (MAX(CurrentPerm))* 100)) AS SKEW_FACTOR
FROM    DBC.TableSizeV
WHERE   DatabaseName = DATABASE
AND     TableName = 'Orders_NoPI'
GROUP BY 1;
```

What is the least amount of permanent space on one of the AMPs?	937,984
What is the greatest amount of permanent space on one of the AMPs?	946,176
What is the skew factor for this table?	0.43
Is the space distribution the same as the <code>Orders_custid</code> table (yes or no)?	No
Is the table distributed as evenly as expected (yes or no)?	Yes
Why or why not?	This table has enough blocks to be reasonably distributed between the AMPs, even though this system has a small number of AMPs (24).



Lab Solutions for Lab 3-2 (7 of 8)

7. DELETE all the rows in Orders_NoPI.

```
DELETE Orders_NoPI;
```

Using INSERT/SELECT with the `HASH BY RANDOM` option, populate your `Orders_NoPI` with all rows from your `Orders_custid` table.

```
INSERT INTO Orders_NoPI SELECT * FROM Orders_custid
                             HASH BY RANDOM (1,2000000000);
```



Lab Solutions for Lab 3-2 (8 of 8)

8. Determine the AMP space usage of your Orders_NoPI table using DBC.TableSizeV.

```
SELECT  TableName
        ,MIN(CurrentPerm) AS Min_Size
        ,MAX(CurrentPerm) AS Max_Size
        ,(100 - (CAST(AVG(CurrentPerm) AS FLOAT)/ NULLIFZERO (MAX(CurrentPerm))* 100)) AS SKEW_FACTOR
FROM    DBC.TableSizeV
WHERE   DatabaseName = DATABASE
AND     TableName = 'Orders_NoPI'
GROUP BY 1;
```

What is the least amount of permanent space on one of the AMPs? 1,118,208

What is the greatest amount of permanent space on one of the AMPs? 1,126,400

What is the skew factor for this table? 0.36

Is the space distribution the same as the Orders_custid table (yes or no)? No

Is the table distributed as evenly as expected (yes or no)? Yes

Why or why not? The data is evenly distributed because individual rows are randomly distributed between the AMPs.

Was row level vs. block level distribution useful in this case? No



Lab Solutions for Lab 3-3 (1 of 2)

1. Use SHOW TABLE for Orders_RP2 to create a column partitioned table named "Orders_CPPA_RP based on the following:
 - Incorporate row partitioning with orderdate as the partitioning column
 - From '2012-01-01' through '2021-12-31', partition by day using CURRENT_DATE
 - Do not use the NO RANGE or UNKNOWN options
 - Each column is created as a separate partition (COLUMN partitioning is the second level)
 - Use the PRIMARY AMP INDEX option with custid
 - Utilize auto compression for every column

```
CREATE TABLE Orders_CPPA_RP
  (orderid          INTEGER NOT NULL,
   custid           INTEGER NOT NULL,
   orderstatus      CHAR(1),
   ordertotal       DECIMAL(9,2) NOT NULL,
   orderdate        DATE FORMAT 'YYYY-MM-DD' NOT NULL,
   orderpriority    SMALLINT,
   clerk            CHAR(16),
   location         SMALLINT,
   shippriority     SMALLINT,
   ordercomment     VARCHAR(79))
PRIMARY AMP INDEX (custid)
PARTITION BY (RANGE_N (orderdate BETWEEN
  CAST ((EXTRACT(YEAR FROM CURRENT_DATE) - 9 ) || '-01-01' AS DATE) AND
  CAST ((EXTRACT(YEAR FROM CURRENT_DATE)) || '-12-31' AS DATE)
  EACH INTERVAL '1' DAY)
,COLUMN);
```



Lab Solutions for Lab 3-3 (2 of 2)

2. Populate the Orders_CPPA_RP table (via INSERT/SELECT) from the Orders_NoPI table.

```
INSERT INTO Orders_CPPA_RP SELECT * FROM Orders_NoPI;
```

3. Use the PARTITION key word to determine the number of partitions defined in the Orders_CP_RP.

How many partitions actually have data? 3620

```
SELECT COUNT(DISTINCT(PARTITION)) FROM Orders_CPPA_RP;
```



Lab Solutions for Lab 4-1 (1 of 7)

1. Using the DBC.DBCInfoV view, find the release and version of the system on which you are logged on:

```
SELECT *
FROM DBC.DBCInfoV;
```

<u>InfoKey</u>	<u>InfoData</u>	
RELEASE	16.20.31.01	(PDE Release #)
VERSION	16.20.31.01	(Teradata Version #)

2. Using the DBC.ChildrenV view, list your parents' userids:

```
SELECT Parent
FROM DBC.ChildrenV
WHERE Child = USER;
```

<u>Parent</u>
DBC
Sysdba
Students
TT_Class1



Lab Solutions for Lab 4-1 (2 of 7)

3. Using the DBC.DatabasesV view, find your:

```
SELECT OwnerName, CreatorName, AccountName, PermSpace, SpoolSpace, TempSpace
FROM   DBC.DatabasesV
WHERE  Databasename = USER;
```

Immediate parent's name	TT_Class1
Creator's name	TT_Class1
Default account code	\$M00EDUC&D&H
Perm space limit	600,000,000 (600 MB)
Spool space limit	4,000,000,000 (4 GB)
Temp space limit	1,000,000,000 (1 GB)

4. Using the DBC.UsersV view, find your:

```
SELECT DefaultDatabase, DefaultCollation, DefaultDateFormat, CreateTimeStamp, PasswordLastModDate
FROM   DBC.UsersV
WHERE  UserName = USER;
```

Default database name	?
Default collation sequence	H
Default date format	?
Create Time Stamp	2021-01-06 10:43:09
Last password modification date	2021-01-06



Lab Solutions for Lab 4-1 (3 of 7)

4. OPTIONAL: SHOW this view. Note the WHERE conditions. (Remember, this is a restricted view, even though it does not have an [X] suffix.)

```
REPLACE VIEW DBC.UsersV
AS SELECT
    dbase.DatabaseName(NAMED UserName),
    dbase.CreatorName,
    ...
```

5. Using the DBC.TablesV view, find the number of tables in the DD/D (User DBC) that are:

Fallback protected Count is 183

```
SELECT Count(*)
FROM    DBC.TablesV
WHERE   DatabaseName = 'DBC'
AND     TableKind IN ('T', 'O') AND ProtectionType = 'F';
```

Not Fallback protected Count is 18

```
SELECT Count(*)
FROM    DBC.TablesV
WHERE   DatabaseName = 'DBC'
AND     TableKind IN ('T', 'O') AND ProtectionType = 'N';
```



Lab Solutions for Lab 4-1 (4 of 7)

5. (cont.) Modify the query to find the number of tables OTHER THAN DD/D (not DBC tables) that are:

Fallback protected [Count will vary – 2,098](#)

```
SELECT Count(*)
FROM DBC.TablesV
WHERE DatabaseName NE 'DBC'
AND TableKind IN ('T', 'O')
AND ProtectionType = 'F';
```

Not Fallback protected [Count will vary - 0](#)

```
SELECT Count(*)
FROM DBC.TablesV
WHERE DatabaseName NE 'DBC'
AND TableKind IN ('T', 'O')
AND ProtectionType = 'N';
```

How many tables (entire system) are defined as No Primary Index tables? [Count will vary – 26](#)

```
SELECT Count(*)
FROM DBC.TablesV
WHERE TableKind = 'O';
```




Lab Solutions for Lab 4-1 (5 of 7)

6. Using the DBC.PartitioningConstraintsV view, answer the following questions; hint, use the ColumnPartitioningLevel (CP) column to determine if a table is column partitioned or not.

For your user, how many tables have row partitioning and no column partitioning? 5 (CP =0)
For your user, how many tables have column partitioning? 5 (CP > 0)
For the system, how many tables have row partitioning and no column partitioning? 91
For the system, how many tables have column partitioning? 36
For the system, how many tables have column and row partitioning? 6
What is the constraint type for partitioned tables? Q

```
SELECT TableName, ColumnPartitioningLevel AS "CP", PartitioningLevels AS "# Levels", ConstraintText AS  
"Text"  
FROM DBC.PartitioningConstraintsV  
WHERE DatabaseName = USER;
```

```
SELECT COUNT(*)  
FROM DBC.PartitioningConstraintsV  
WHERE ColumnPartitioningLevel = 0; 91 (Answers may vary)
```

```
SELECT COUNT(*)  
FROM DBC.PartitioningConstraintsV  
WHERE ColumnPartitioningLevel <> 0; 36 (Answers may vary)
```

```
SELECT COUNT(*)  
FROM DBC.PartitioningConstraintsV  
WHERE ColumnPartitioningLevel <> 0 AND PartitioningLevels GE 2; 6
```



Lab Solutions for Lab 4-1 (6 of 7)

7. Using the DBC.IndicesV view, find the number of tables OTHER THAN Dictionary tables that have non-unique primary indexes (NUPI):

Number of tables Count will vary – 1,047

```
SELECT  COUNT (DISTINCT(DatabaseName || '.' || TableName))
FROM    DBC.IndicesV
WHERE   IndexType IN ('P', 'Q')
AND     UniqueFlag = 'N'
AND     DatabaseName NE 'DBC';

Count(Distinct(TableName))
1,047
```

Note: The IndexType of 'P' is used for the primary index of non-partitioned tables.
The IndexType of 'Q' is used for the primary index of partitioned tables.



Lab Solutions for Lab 4-1 (7 of 7)

8. Using the DBC.ColumnsV view, find the number of columns in the entire system defined with *default values*:

Number of columns Count will vary – 3,604

```
SELECT  'Column count:'
        ,COUNT (*)
FROM    DBC.ColumnsV
WHERE   DefaultValue IS NOT NULL;

'Column count:'      Count(*)
Column count:        3,604
```

OPTIONAL: Modify the query to find the number of objects that have columns defined with *default values*:

Number of tables Count will vary – 134

```
SELECT  'Object Count:'
        ,COUNT (DISTINCT (DatabaseName || '.' || TableName))
FROM    DBC.ColumnsV
WHERE   DefaultValue IS NOT NULL;

'Object Count:'      Count(Distinct(TableName))
Object Count:        134
```



Lab Solutions for Lab 4-2 (optional) (1 of 2)

1. Use INSERT/SELECT to place all rows from the populated PD tables into your empty tables. Verify the number of rows in your tables.

```
INSERT INTO Employee SELECT * FROM PD.Employee;
SELECT COUNT(*) FROM Employee;                      Count = 150,000
INSERT INTO Department SELECT * FROM PD.Department;
SELECT COUNT(*) FROM Department;                     Count = 17,415
INSERT INTO Job SELECT * FROM PD.Job;
SELECT COUNT(*) FROM Job;                            Count = 11,124
INSERT INTO EmpPhone SELECT * FROM PD.EmpPhone;
SELECT COUNT(*) FROM EmpPhone;                       Count = 350,000
```

2. Use the GRANT statement to GRANT yourself the REFERENCES access rights on the tables.

```
GRANT REFERENCES ON Employee TO studentYYY;
GRANT REFERENCES ON Department TO studentYYY;
GRANT REFERENCES ON Job TO studentYYY;
```



Lab Solutions for Lab 4-2 (optional) (2 of 2)

3. Create a References constraint between the Employee.Dept_Number column and the Department.Dept_Number column.

```
ALTER TABLE Employee ADD CONSTRAINT emp_dept_ref  
FOREIGN KEY (DeptNumber) REFERENCES  
Department (DeptNumber);
```

What is the name of the Employee RI error table? `EMPLOYEE_0`

How many rows are in this table? `1`

Which department is not represented in the department table? `999`

What is the name of the employee with an invalid department number? `Ibee DeBosse`

4. Use the DBC.All_RI_ChildrenV view (qualify the ChildDB to your database) and verify this constraint.

What is the IndexID of this constraint? `0`

```
SELECT Indexid (FORMAT 'z9') AS ID  
      ,IndexName  
      ,ChildTable  
      ,ChildKeyColumn  
      ,ParentTable  
      ,ParentKeyColumn  
FROM DBC.All_RI_ChildrenV  
WHERE ChildDB = USER  
ORDER BY 1;
```

ID	IndexName	ChildTable	ChildKeyColumn	ParentTable	ParentKeyColumn
0	emp_dept_ref	Employee	DeptNumber	Department	DeptNumber



Lab Solutions for Lab 6-1 (1 of 7)

1. Using the DBC.DiskSpaceV view, find the total disk storage capacity of the system on which you are logged on:

Total capacity 571,123,664,144

```
SELECT SUM(MaxPerm) (FORMAT 'zzz,zzz,zzz,999') FROM DBC.DiskSpaceV;
```

Sum(MaxPerm)

571,123,664,144

2. Using the same view, find how much of the space is currently in use:

Current space utilization 31,634,845,696

```
SELECT SUM(CurrentPerm) (FORMAT 'zzz,zzz,zzz,999') FROM DBC.DiskSpaceV;
```

Sum(CurrentPerm)

31,634,845,696

Write a query to show what percentage of system capacity is currently in use.

```
SELECT ((CAST(SUM(currentperm) AS FLOAT)/ NULLIFZERO(SUM(maxperm)) * 100))  
AS "% Used Perm Space" FROM DBC.DiskSpaceV;
```

% Used Perm Space

5.56



Lab Solutions for Lab 6-1 (2 of 7)

3. Write a query to show which databases/users are currently using (current perm) the largest percentage of their max perm space limit (group by database/user).

```
SELECT Databasename
      , ((CAST(SUM(currentperm) AS FLOAT)/ NULLIFZERO(SUM(maxperm)) * 100))
      AS "% Used Perm Space"
FROM   DBC.DiskSpaceV
GROUP BY 1
ORDER BY 2 DESC;
```

<u>Databasename</u>	<u>% Used Perm Space</u>
student103	45.83
student102	44.25
student104	42.98
student408	41.08
student101	39.72
TD_SYSFNLIB	36.12



Lab Solutions for Lab 6-1 (3 of 7)

4. Using the DBC.DatabasesV view, find the total number of databases and users defined in the system.

```
SELECT COUNT(*) AS "Total Count"  
FROM   DBC.DatabasesV;
```

Total Count
254

Total row count (databases and users) 254 (Answers will vary)

How many users are there? 224 (Answers will vary)

```
SELECT COUNT(*) AS "User Count"  
FROM   DBC.DatabasesV  
WHERE  DBKind = 'U';
```

User Count
224

How many databases are there ? 30 (Answers will vary)

```
SELECT COUNT(*) AS "DB Count"  
FROM   DBC.DatabasesV  
WHERE  DBKind = 'D';
```

DB Count
30



Lab Solutions for Lab 6-1 (4 of 7)

4. (cont.)

Who is the creator of AP? [SYSDBA](#)

Who is the owner of AP? [TT_Data](#)

```
SELECT CreatorName, OwnerName  
FROM   DBC.DataBasesV  
WHERE  DatabaseName = 'AP';
```

<u>CreatorName</u>	<u>OwnerName</u>
SYSDBA	TT_Data



Lab Solutions for Lab 6-1 (5 of 7)

5. Using the DBC.TableSizeV view, find the name and size of each table in the DBC user. List the Data Dictionary tables in DESCending order by size.

```
SELECT TOP 6 TableName, SUM(CurrentPerm) (FORMAT 'zzz,zzz,zzz')
FROM DBC.TableSizeV
WHERE DatabaseName = 'DBC'
GROUP BY 1
ORDER BY 2 DESC;
```

<u>TableName</u>	<u>Sum(CurrentPerm)</u>
DBQLogTbl	8,712,699,904
DBQLSqlTbl	4,527,104,000
DBQLExplainTbl	3,812,450,304
ResUsageSvpr	2,639,380,480
DBQLObjTbl	951,877,632
TransientJournal	808,845,312

6. Using the DBC.AMPUsage view, find the number of AMP vprocs defined on your system.

(HINT: Use a WHERE condition to reduce the number of DD/D table rows considered.)

```
SELECT COUNT(DISTINCT(Vproc)) AS "# of AMPS"
FROM DBC.AMPUsageV
WHERE UserName = USER; OR SELECT HASHAMP() + 1;
```

of AMPS
24



Lab Solutions for Lab 6-1 (6 of 7)

7. Using the DBC.AccountInfoV view, list all of your valid account codes.

```
SELECT *
FROM DBC.AccountInfoV
WHERE UserName = USER;

UserName AccountName
student135 $M00EDUC&D&H
student135 $L00EDUC&D&H
```

8. Using the DBC.AMPUsageV view, write a query to show the number of AMP CPU seconds and logical disk I/Os that have been charged to your username.

```
SELECT UserName (CHAR(12))
,CAST (SUM(CPUTime) AS FORMAT 'zzzz.99')
,CAST (SUM(DiskIO) AS FORMAT 'zzz,zzz,zzz')
FROM DBC.AMPUsageV
WHERE UserName = USER
GROUP BY UserName;

UserName Sum(CpuTime) Sum(DiskIO)
STUDENT135 449.78 860,752.00
```



Lab Solutions for Lab 6-1 (7 of 7)

8. Using the DBC.AMPUsageV view, write a query to show the number of AMP CPU seconds and logical disk I/Os that have been charged to all users in your class.

```
SELECT  UserName, SUM (CPUTime), SUM (DiskIO)
FROM    DBC.AMPUsageV
WHERE   UserName LIKE 'student1%' GROUP BY UserName ORDER BY 2 DESC;
```

<u>UserName</u>	<u>Sum(CpuTime)</u>	<u>Sum(DiskIO)</u>
STUDENT135	449.85	861,269
STUDENT102	320.08	2,232,511
STUDENT104	281.18	2,131,182
STUDENT101	257.55	1,425,369
STUDENT103	222.92	1,545,387



Lab Solutions for Lab 7-1 (1 of 2)

1. Create a user profile with a profile name that is the same as your user name (studentxxx_P).

```
CREATE PROFILE studentxxx_P AS
ACCOUNT      = '$M00FACT&D&H',
DEFAULT DATABASE = studentYYY
SPOOL        = 50E6,
TEMPORARY     = 50E6,
PASSWORD      = (MINCHAR = 6, MAXLOGONATTEMPTS = 3, LOCKEDUSEREXPIRE = 5, DIGITS='R',
                  RESTRICTWORDS='Y', SPECCHAR='P')
QUERY_BAND    = 'Group=PS;' (DEFAULT);
```

2. Use the DBC.ProfileInfoV view to display information about profiles in the system.

```
SELECT      ProfileName
            ,DefaultAccount AS "Def Acct"
            ,DefaultDB
            ,SpoolSpace
            ,TempSpace
            ,QueryBand
            ,QuerybandDefault AS QD
FROM        DBC.ProfileInfoV
ORDER BY 1;
```

How many profiles are defined in the system? 28 (Answers will vary.)

```
SELECT COUNT(*) FROM DBC.ProfileInfoV;
```



Lab Solutions for Lab 7-1 (2 of 2)

3. Create two new users in the system as follows:

```
CREATE USER studentxxx_A AS PERM=0, PASSWORD=Temp1234, PROFILE=studentxxx_P;  
CREATE USER studentxxx_B AS PERM=0, PASSWORD=Temp1234, PROFILE=studentxxx_P;
```

4. Logon to Teradata as "studentxxx_A".

As studentxxx_A, use MODIFY USER to change their password to a different value (and remember this password).

```
MODIFY USER studentxxx_A AS PASSWORD=Qwerty123;
```

Logon off as studentxxx_A and logon using the new password that you just created.

As studentxxx_A, issue the following SELECT statement.

```
SELECT COUNT(*) FROM studentxxx.EmpPhone;
```

Note: You should get an access right error (3523) with this SELECT since studentxxx_A should not have access rights on studentxxx.

SELECT Failed. 3523: The user does not have SELECT access to studentxxx.EmpPhone.



Lab Solutions for Lab 9-1 (1 of 4)

1. Using the DBC.AllRightsV view, find the total number of rows in the DBC.AccessRights table assigned to users.

```
SELECT COUNT(*) FROM DBC.AllRightsV;
```

Count(*)

64,535

Total row count (AllRights view)

(Answers will vary)

Using the DBC.AllRoleRightsV view, find the total number of rows in the DBC.AccessRights table assigned to roles.

```
SELECT COUNT(*) FROM DBC.AllRoleRightsV;
```

Count(*)

435

Total row count (AllRoleRights view)

(Answers will vary)

2. Using the DBC.UserRightsV view, take a look at the databases and tables on which you currently hold rights.

```
SELECT COUNT(*) FROM DBC.UserRightsV;
```

Count(*)

462

Total number of rows returned

(Answers will vary)

How do you think most of these privileges were granted?

[These are automatic rights that you received when your user and tables were created.](#)



Lab Solutions for Lab 9-1 (2 of 4)

2. (cont.)

Execute the following SQL command and then recheck the number of Access Rights you have.

```
CREATE TABLE EmpPhone2 AS PD.EmpPhone WITH NO DATA;
```

Total number of user rights returned 474 (Answers will vary)

```
SELECT COUNT(*) FROM DBC.UserRightsV;
```

```
Count(*)  
474
```

How many new access rights were created? 12



Lab Solutions for Lab 9-1 (3 of 4)

3. Use the GRANT command to give the SELECT WITH GRANT OPTION access right on **EmpPhone** to studentxxx_A.

```
GRANT SELECT ON EmpPhone TO studentxxx_A;
```

Use the GRANT command to give the INSERT access right on **EmpPhone2** to studentxxx_A.

```
GRANT INSERT ON EmpPhone2 TO studentxxx_A WITH GRANT OPTION;
```

Check the total number of user rights returned [474](#)

```
SELECT COUNT(*) FROM DBC.UserRightsV;
```

[Count\(*\)](#)

474

Did this count change? [No](#)

If not, why not? [These new rights are associated with users different than yourself.](#)



Lab Solutions for Lab 9-1 (4 of 4)

4. Logon as studentxxx_A and issue the following statement.

```
INSERT INTO studentxxx.EmpPhone2 SELECT * FROM studentxxx.EmpPhone;
```

Does this INSERT complete successfully? [Yes](#)

As studentxxx_A, issue the following command.

```
SELECT COUNT(*) FROM studentxxx.EmpPhone2;
```

Does this SELECT complete successfully? [No](#)



Lab Solutions for Lab 9-2 (1 of 4)

1. Using the DBC.RoleInfoV view, find the total number of roles defined in the system?

```
SELECT COUNT(*) AS "Role Count" FROM DBC.RoleInfoV;
```

Count = 530

Using the DBC.RoleInfoVX view, what is the number of roles that you have created?

```
SELECT COUNT(*) AS "Role Count" FROM DBC.RoleInfoVX;
```

Count = 0

Using the DBC.RoleMembersVX view, what is your (studentxxx) default role?

```
SELECT * FROM DBC.RoleMembersVX;
```

[TT_Access_R](#)

Using the DBC.RoleMembersVX view, which roles do you have the "With Admin" option?

```
SELECT * FROM DBC.RoleMembersVX;
```

Role1_xxx, Role2_xxx, Role3_xxx

2. Grant the following access rights to the specified roles as follows:

GRANT SELECT	ON Orders	TO Role1_xxx;
GRANT SELECT	ON Orders_NoPI	TO Role1_xxx;
GRANT SELECT	ON Orders_RP	TO Role2_xxx;
GRANT INSERT, UPDATE, DELETE	ON Orders_RP	TO Role3_xxx;



Lab Solutions for Lab 9-2 (2 of 4)

3. Grant Role1_xxx to studentxxx_A; GRANT Role1_xxx to studentxxx_A;
Grant Role2_xxx to studentxxx_B; GRANT Role2_xxx to studentxxx_B;
Grant Role2_xxx to Role3_xxx; GRANT Role2_xxx TO Role3_xxx;
Grant Role3_xxx to studentxxx_B; GRANT Role3_xxx TO studentxxx_B;
4. Modify the two users in the system as follows:
- ```
MODIFY USER studentxxx_A AS DEFAULT ROLE = Role1_xxx;
MODIFY USER studentxxx_B AS DEFAULT ROLE = Role2_xxx;
```
5. As "studentxxx\_A", execute the following SQL statements and indicate if SELECT is allowed or not.
- ```
SELECT COUNT(*) FROM Orders;    Permitted or not? Yes  
SELECT COUNT(*) FROM Orders_NoPI;    Permitted or not? Yes  
SELECT COUNT(*) FROM Orders_RP;    Permitted or not? No
```



Lab Solutions for Lab 9-2 (3 of 4)

6. As "studentxxx_A", use the DBC.RoleMembersV and DBC.UserRoleRightsV views to view the current role of the user, any nested roles, and access rights for the roles.

```
SELECT * FROM DBC.RoleMembersVX;

SELECT RoleName, DatabaseName, TableName, ColumnName, AccessRight
FROM DBC.UserRoleRightsV
ORDER BY 1;
```

How many roles are available to studentxxx_A? 1

What is the default role for studentxxx_A? Role1_xxx

Does studentxxx_A have the "With Admin" option on any roles? No

How many user role rights are available to studentxxx_A? 2

OPTIONAL

7. Logon as "studentxxx_B" and set the password if requested.
8. As "studentxxx_B", execute the following SQL statements and indicate if SELECT is allowed or not.

SELECT COUNT(*) FROM Orders;	Permitted or not? <u>No</u>
SELECT COUNT(*) FROM Orders_NoPI;	Permitted or not? <u>No</u>
SELECT COUNT(*) FROM Orders_RP;	Permitted or not? <u>Yes</u>
DELETE Orders_RP;	Permitted or not? <u>No</u>



Lab Solutions for Lab 9-2 (4 of 4)

9. As "studentxxx_B", use the DBC.RoleMembersVX and DBC.UserRoleRightsV views to view the current role of the user, any nested roles, and access rights for the roles.

```
SELECT * FROM DBC.RoleMembersVX;

SELECT RoleName, DatabaseName, TableName, ColumnName, AccessRight
FROM DBC.UserRoleRightsV
ORDER BY 1;
```

How many roles are available to studentxxx_B?

2

What is the default role for studentxxx_B?

Role2_xxx

Does studentxxx_B have the "With Admin" option on any roles?

No

How many user role rights are available to studentxxx_B?

1

10. As "studentxxx_B", use the SET ROLE command to set the current role to "Role3_xxx".

```
SET ROLE Role3_xxx;
```

```
SELECT COUNT(*) FROM Orders;
```

Permitted or not? No

```
SELECT COUNT(*) FROM Orders_NoPI;
```

Permitted or not? No

```
SELECT COUNT(*) FROM Orders_RP;
```

Permitted or not? Yes

```
DELETE Orders_RP;
```

Permitted or not? Yes

11. Log off as "studentxxx_A" and "studentxxx_B". Using your initial user logon name, DROP the two users and the profile you created.

```
DROP USER studentxxx_A; DROP USER studentxxx_B; DROP PROFILE studentxxx_P;
```



Lab Solutions for Lab 11-1 (1 of 2)

1. What system security defaults are in effect for your system?

`SELECT * FROM DBC.SecurityDefaults;`

Number of days to expire password:	<u>0</u>	
Minimum number of characters required:	<u>6</u>	
Maximum number of characters required:	<u>127</u>	
Are digits allowed?	Yes <u>X</u>	No <u> </u>
Are special characters allowed?	Yes <u>X</u>	No <u> </u>
Restrict words in passwords?	Yes <u> </u>	No <u>X</u>
Maximum failed logons permitted (0=never lock):	<u>3</u>	
Minutes to elapse before unlocking:	<u>5</u>	
Days to expire before password reuse:	<u>0</u>	

2. Is a Profile in effect for your username? If so, what is the name of your Profile? Student_P

`SELECT Profile;`

3. If a Profile is being used, which attributes in the Profile override the system security defaults?

`SELECT * FROM DBC.ProfileInfoVX;`

Maximum Character is 15, and Password Special Characters (I)



Lab Solutions for Lab 11-1 (2 of 2)

4. Using the DBC.LogOnOffV view, list the "BAD" logon attempts on your system that have occurred during the last ten days. Qualify the SELECT using LIKE 'BAD%'.

Number of Bad Logons (System) 148 (Answers will vary)

```
SELECT COUNT(*)
FROM DBC.LogonoffV
WHERE EVENT LIKE 'BAD%'
AND LOGDATE > CURRENT_DATE - 10;
```

Number of Bad Logons (Your UserName) 2 (Answers will vary)

```
SELECT COUNT(*)
FROM DBC.LogonoffV
WHERE EVENT LIKE 'BAD%'
AND LOGDATE > CURRENT_DATE - 10
AND UserName = USER;
```

5. Using the DBC.SessionInfoV, list the sessions currently logged on your system.

Total number of Sessions (System) 28 (Answers will vary)

Total number of sessions (Your UserName) 2 (Answers will vary)

```
SELECT COUNT(*) FROM DBC.SessionInfoV;
SELECT COUNT(*) FROM DBC.SessionInfoV WHERE UserName = USER;
```




Lab Solutions for Lab 11-2 (1 of 3)

Tasks

1. Using the DBC.AccLogRulesV view, how many access logging rules are there? [Count = 5](#)

```
SELECT COUNT(*) from DBC.AccLogRulesV;
```

Using the DBC.AccLogRulesV view, complete the following access log information for macros – creating , dropping, and execution of macros.

```
SELECT UserName (CHAR (10)),DatabaseName (CHAR (10)),TVMName (CHAR (15)), AcrCreateMacro, AcrDropMacro,
        AcrExecute
FROM    DBC.AccLogRulesV;
```

or

```
SELECT * from DBC.AccLogRulesV ORDER BY 1, 2, 3;
```

<u>Username</u>	<u>Databasename</u>	<u>TVMName</u>	<u>CMC</u>	<u>DMC</u>	<u>EXE</u>
All	All	All			E +
All	DBC	AccLogRule		E +	E +
All	DBC	All	E -	E -	E -
All	DBC	DBQLAccessMacro		E +	E +
All	DBC	LogonRULE		E +	E +



Lab Solutions for Lab 11-2 (2 of 3)

2. Execute the following statement.

```
CREATE DATABASE Testxxx FROM DBC AS PERM=0; (xxx is your student number)
```

(this command should fail – 3524: access right violation)

3. Using the DBC.AccessLog view, list the access log entries for the last two weeks for your username.

How many entries are in this log have been granted? [102 \(answers will vary\)](#)

```
SELECT COUNT(*) FROM DBC.AccessLog
WHERE AccLogResult = 'G'
AND Username=USER
AND Logdate > Current_Date - 14;
```

How many entries are in this log have been denied? [1 \(at least one; answers will vary\)](#)

```
SELECT COUNT(*) FROM DBC.AccessLog
WHERE AccLogResult = 'D'
AND Username=USER
AND Logdate > Current_Date - 14;
```



Lab Solutions for Lab 11-2 (3 of 3)

4. Using the DBC.DBQLRulesV view, how many query logging rules are there? [Count = 5](#)

```
SELECT COUNT(*) from DBC.DBQLRulesV;
```

Using the DBC.DBQLRulesV view, list the query log rules that are in effect for this system.

```
SELECT * from DBC.DBQLRulesV ORDER BY 1, 2, 3;
```

<u>Username</u>	<u>Application Name</u>	<u>TypeofRule</u>	<u>Explain</u>	<u>Object</u>	<u>SQL</u>	<u>Object Usage</u>
All		Log enabled	T	T	T	T
All	FASTLOAD	With None	F	F	F	F
All	MULTLOAD	With None	F	F	F	F
All	TPTLOAD	With None	F	F	F	F
All	TPTUPD	With None	F	F	F	F

5. Using the DBC.QryLogV view, list the logged queries for your username and how many are there?

```
SELECT COUNT (*) FROM DBC.QryLogV WHERE Username = USER;
```

Count = [605 \(answers will vary\)](#)

Using this view, how many queries are logged for all of the users with usernames like 'student%'?

```
SELECT COUNT (*) FROM DBC.QryLogV WHERE Username LIKE 'student%';
```

Count = [15,815 \(answers will vary\)](#)



Lab Solutions for Lab 13-1, 52-2, and 52-3

Lab Exercise 52-1

9. From Explain tab what is the Estimated Time for step 5? [46.565 sec](#)

Lab Exercise 52-2

2. Find your user and determine the Peak Spool % you used. Peak Spool % [8.436 %](#) (Answers will vary)
3. For your user what was the percentage skew for the "Employee_2" table you made that was distributed using LastName as the NUPI? Skew % [14.21 %](#)
5. Using "By Vproc" rather than "By Database" which AMP # has the greatest % of space in use and what is that percent? AMP # [11](#), % [1.355 %](#) (Answers may vary)
6. Is the space usage on our education system well distributed? [Yes](#) or No

Lab Exercise 52-3

7. Using Rewind, go back in time to locate the Disk I/O and Logical MB/Sec set of graphs - you should that Disk I/O and Logical MB/Sec track each other closely. If the session threshold exceeds the number you set, how is that indicated on the graph?

[Graph line above threshold becomes red and max value bold](#)



Lab Solutions for Lab 17-1 (1 of 5)

1. Create two Contiguous tables Nation_C and Region_C from PD.Nation_C and PD.Region_C. (Note: these tables do not have all of the nations/regions, just a small subset of data)

```
CREATE TABLE Region_C AS TRNG_PD.Region_C WITH DATA;  
CREATE TABLE Nation_C AS TRNG_PD.Nation_C WITH DATA;
```

2. Find the system single AMP (1) Sparse MAP by using the view DBC.MapsV.

```
SELECT MapName FROM DBC.MapsV;
```

Name of Sparse Map for 1 AMP – [TD_1AmpSparseMap_1Node](#)



Lab Solutions for Lab 17-1 (2 of 5)

3. Use SHOW TABLE and create new Nation_S and Region_S sparse tables specifying the 1 AMP sparse MAP name and use a COLOCATE value of Co_NationRegion for both tables (i.e., COLOCATE USING Co_NationRegion).

```
CREATE TABLE Nation_S, FALLBACK,  
  MAP = TD_1AmpSparseMap_1Node COLOCATE USING Co_NationRegion  
  (  
    NationKey  INTEGER,  
    NationName VARCHAR(25),  
    RegionKey  INTEGER )  
  UNIQUE PRIMARY INDEX (NationKey);  
  
CREATE TABLE Region_S, FALLBACK,  
  MAP = TD_1AmpSparseMap_1Node COLOCATE USING Co_NationRegion  
  (  
    RegionKey  INTEGER,  
    RegionName VARCHAR(25) )  
  UNIQUE PRIMARY INDEX (RegionKey);
```

4. Populate the two Sparse tables with INSERT/SELECT from the two contiguous tables.

```
INSERT INTO Nation_S SELECT * FROM Nation_C;  
INSERT INTO Region_S SELECT * FROM Region_C;
```



Lab Solutions for Lab 17-1 (3 of 5)

5. Use the DBC.TableSizeV and determine which 2 AMPs the sparse tables are using.

```
SELECT DatabaseName, TableName, Vproc, SUM(CurrentPerm)
FROM   DBC.TablesizeV
WHERE  DatabaseName = DATABASE AND TableName IN ('Nation_S', 'Region_S')
GROUP BY 1,2,3
ORDER BY 1,2,3;
```

<u>DataBaseName</u>	<u>TableName</u>	<u>Vproc</u>	<u>Sum(CurrentPerm)</u>
student535	Nation_S	2	8,192
student535	Nation_S	3	8,192
student535	Region_S	2	8,192
student535	Region_S	3	8,192

Why are two AMPs used per sparse table? [Primary and Fallback AMPs](#)

6. Compare the sizes of Nation_C and Region_C to the Nation_S and Region_S tables.

```
SELECT DatabaseName, TableName, SUM(CurrentPerm)
FROM   DBC.TablesizeV
WHERE  DatabaseName = DATABASE
AND    TableName IN ('Nation_C', 'Nation_S', 'Region_C', 'Region_S')
GROUP BY 1,2
ORDER BY 1,2,3;
```

Size of Nation_C =	98,304	Size of Nation_S =	16,384
Size of Region_C =	73,728	Size of Region_S =	16,384



Lab Solutions for Lab 17-1 (4 of 5)

7. Use the DBC.SparseMapAmpsV to determine which AMP has your primary data for the Sparse tables. AMP # = 2

```
SELECT * FROM DBC.SparseMapAmpsV where DatabaseName = DATABASE;
```

<u>DatabaseName</u>	<u>ObjectName</u>	<u>ObjectKind</u>	<u>AmpNo</u>	<u>PF</u>
student535	Nation_S	T	2	P
student535	Region_S	T	2	P
student535	Nation_S	T	3	F
student535	Region_S	T	3	F

8. Execute the following EXPLAIN of a join of the Contiguous tables.
Is the data redistributed? Yes/No Yes

```
EXPLAIN SELECT      NationName, RegionName
FROM                Nation_C
INNER JOIN          Region_C
ON                  Nation_C.RegionKey = Region_C.RegionKey;
```

Execute the same EXPLAIN using the Nation_S and Region_S sparse tables. Is the data redistributed for the join? Yes/No No

```
EXPLAIN SELECT      NationName, RegionName
FROM                Nation_S
INNER JOIN          Region_S
ON                  Nation_S.RegionKey = Region_S.RegionKey;
```




Lab Solutions for Lab 17-1 (5 of 5)

9. Optional: Create a new Nation_S2 table and a new Region_S2 table without using the COLOCATE option. Use the DBC.SparseMapAmpsV to determine which AMP has your Sparse tables.

Are the two tables on the same AMP? Yes/No No

```
CREATE TABLE Nation_S2, FALLBACK,  
  MAP = TD_1AmpSparseMap_1Node  
  MAP = TD_1AmpSparseMap_1Node  
  ( NationKey  INTEGER,  
    NationName VARCHAR(25),  
    RegionKey  INTEGER )  
  UNIQUE PRIMARY INDEX (NationKey);
```

```
CREATE TABLE Region_S2, FALLBACK,  
  MAP = TD_1AmpSparseMap_1Node  
  ( RegionKey  INTEGER,  
    RegionName VARCHAR(25) )  
  UNIQUE PRIMARY INDEX (RegionKey);
```

```
SELECT * FROM DBC.SparseMapAmpsV  
WHERE DatabaseName = DATABASE AND ObjectName IN ('Nation_S2', 'Region_S2');
```

<u>DatabaseName</u>	<u>ObjectName</u>	<u>ObjectKind</u>	<u>AmpNo</u>	<u>PF</u>
student535	Nation_S2	T	6	P
student535	Region_S2	T	5	P
student535	Nation_S2	T	7	F
student535	Region_S2	T	4	F



Appendix C: Acronyms

Teradata Vantage Administration

Copyright © 2007–2023 by Teradata. All Rights Reserved.

-Numbers-

2PC Two-phase Commit

-A-

ABF	Adaptive Bandwidth Feature
ABI	Application Binary Interface
ABIOS	Advanced BIOS
ABM	Asynchronous Balance Mode
ABRD	Automatic Baud Rate Detection
AC	Application Component, Alternating Current
ACE	Array Configuration Editor
ACF	Advanced Communications Function
ACF/NCP	Advanced Communication Function for the Network Control Program
ACF/SSP	ACF/Software Support Program
ACF/VTAM	Advanced Communication Function for the Virtual Telecommunications Access Method
ACID	Atomicity, Consistency, Isolation, Durability
ADAM	Application Data Access Manager
ADC	Americas Development Center
ADCOM	Advanced Distributed Communications System
ADCS	Advanced Data Communication System
ASCII	American Standard Code for Information Interchange
ADE	Application Development Environment
ADW	Active Data Warehouse
AF	Advanced Format (drives with 4K, 8K, or 16K sectors)
AFMS	Advanced Function Management System
AI	Artificial Intelligence
AJI	Aggregate Join Index

ALC	Algorithmic Compression
AMP	Access Module Processor
ANSI	American National Standards Institute
ANSWER	Alpha-Numeric Single Wire Electronic Recorder
AOD	Application Output Definition
AOE	Application Operating Environment
AOS	Automated Order System
AP	Application Processor
API	Application Programming Interface
APPC	Advanced Program to Program Communication
AS	Administration Station
ASF	Archive Storage Facility
ASIC	Application Specific Integrated Circuit
ATM	Asynchronous Transfer Module (Networking)
	Automated Teller Machine
AUI	Attachment Unit Interface
AWS	Amazon Web Services or Administration WorkStation

-B-

BAM	BYNET Administrative Menus
BASIC	Beginner's All-purpose Symbolic Instruction Code
BCD	Binary-Coded Decimal
BDL	BYNET Data Link
BIC	BYNET Interface Card (Adapter)
BIM	Business Information Model; Business Impact Model
BIOS	Basic Input/Output System
BIST	Built-In Self-Test
BIU	Battery Interface Unit
BLC	Block Level Compression
BLIP	BYNET Low Level Internet Protocol
BLLI	BYNET Low Latency Interface
BLM	BYNET Link Manager
BLOB	Binary Large Object
BMCA	BYNET Micro Channel Architecture adapter
BNC	Bayonet-Neill-Concelman (Ethernet connector)
BOM	Byte Order Mark (associated with Unicode)
BOOTP	BOOTstrap Protocol
BPCI	BYNET Peripheral Component Interconnect
bps	bits per second
BTAM	Basic Telecommunications Access Method
BTEQ	Basic Teradata Query facility
BYA4G	BYNET A Switch V2 – 4 ports – Gigahertz speed (60 MB/sec.)
BYA4M	BYNET A Switch V2 – 4 ports – Gigahertz speed (60 MB/sec.)
BYA4MS	BYNET A Switch V2 – 4 ports – Gigahertz speed (60 MB/sec.)
BYA4P	BYNET A Switch V1 – 4 ports – 10 MB/sec.
BYA8QX	BYNET A Switch V3 – 8 ports within BYA64GX or BYC64G

BYA16	BYNET A Switch V1 – 16 ports – 10 MB/sec.
BYA16G	BYNET A Switch V2 – 16 ports – Gigahertz speed (60 MB/sec.)
BYA64GX	BYNET A Switch V2 – 64 ports – Expandable - Gigahertz speed
BYAS	Component name for BYA16XS boards
BYB32	BYNET B Switch – 10 MB/sec.
BYB64G	BYNET B Switch V2
BYC64G	BYNET C Switch V3
BYCLK	BYNET Clock
BYNET	Banyan Network - High speed interconnect
BYOX	BYNET Optical Extension

-C-

CA	Cable Adapter
CAD/CAM	Computer-Aided Design/Computer-Aided Manufacturing
CAE	Computer Aided Engineering
CAI	Century Analysis, Inc., Computer-Aided (or Assisted) Instruction
CASE	Computer Aided Software Engineering
CAT	Configuration and Test
CC	Cluster Controller, Common Cable, Common Carrier
CCB	Change Control Board, Character Control Block, Command Control Block
CCC	Common Carrier Communication
CCDLC	Common Carrier Data Link Control
CCITT	Consultative Committee on International Telephone and Telegraph
CCL	Customer Care Link (older software to alert remote support center)
CCU	Cloud Compute Unit
CD	Compact Disk
CD-ROM	Compact Disk - Read Only Memory
CDS	Customer Data Storage
CDT	Complex Data Type
CE	Customer Engineer
CFM	Configuration File Management
CI	Cylinder Index
CICS	Customer Information Control System
CIM	Common Information Model (used as model for WBEM)
CJK	Chinese, Japanese, and Korean
CLAN	Cabinet Local Area Network
CLC	Cabinet Level Controller (5100 component)
CLCX	Cabinet Level Controller eXtended (5100 component)
CLI	Call Level Interface
CLOB	Character Large Object
CM	Communications Manager
CMB	Chassis Management Board
CMIC	Cabinet Management Interface Controller or Chassis Management Interface Controller
CMOS	Complementary Metal Oxide Semiconductor

COBOL	Common Business-Oriented Language
COP	Communications Processor
COS	Corporate Office Server
CP	Column Partitioning (or Partitioned)
CP NoPI	Column-Partitioned No Primary Index
CP NUPI	Column-Partitioned Non-Unique Primary Index
CPPA	Column-Partitioned Primary AMP Index
CPPI	Column-Partitioned Primary Index or Character PPI
CPUPI	Column-Partitioned Unique Primary Index
CRP	Column-partitioned and Row-Partitioned
CRP NoPI	Column-partitioned and Row-Partitioned No Primary Index
CRPPA	Column-partitioned and Row-Partitioned Primary AMP index
CRPPI	Column-partitioned and Row-Partitioned Primary Index
CPU	Central Processing Unit
CR/LF	Carriage Return/Line Feed
CRC	Cyclic Redundancy Check
CRU	Customer Replaceable Unit
CSF	Customer Support Facility
CSI	Complex Service Interface
CTG	Channel Tailgate
CTS	Clear to Send

-D-

DA	Disk Array
DAC	Disk Array Controller
DAMC	Disk Array Module Cabinet
DARDAC	Dual Active Redundant Disk Array Controller
DASD	Direct-Access Storage Device
DAT	Digital Audio Tape, Dynamic Address Translation
DBA	Database Administrator
DBC	Database Computer (Teradata)
DBLT	Disk Bay Locator Table
DBMS	Database Management System
DBQL	Database Query Log
DBS	Database Subsystem
DC	Data Communications, Direct Current
DCE	Distributed Computing Environment
DCL	Data Control Language
DCRAM	Disk Cache Random Access Memory
DD	Derived Data
DDE	Direct Data Exchange
DDI	Device Driver Interface
DDL	Data Definition Language
DES	Data Encryption Standard
DEUI	Dual Ethernet UPS Interface (5400 UPS feature)
DIF	Document Interchange Facility, Data Interchange Format

DKI	Driver Kernel Interface
DIM	Diagnostic Interconnect Module (3600 component
DIMM	Dual In-line Memory Module
DIP	Database Initialization Procedure or Program
DISCO	Disconnect memory
DLPI	Data Link Provider Interface
DMA	Direct Memory Access
DMCA	Dual Micro Channel Architecture
DML	Data Manipulation Language
DMX	EMC2 Direct Matrix Disk Array
DNS	Domain Name Server
DOD	Department of Defense
DOS	Disk Operating System
DP	Diagnostic Processor
DR	Deficiency or Defect Report
DRAM	Dynamic Random-Access Memory
DS	Data Stream or Desk Side
DSA	Data Stream Architecture
DSC	Data Stream Controller
DSS	Decision Support System
DSU	Disk Storage Unit
DSW	Destination Selection Word
DTP	Distributed Transaction Processing
DTR	Data Terminal Ready

-E-

E2I	External-to-Internal
EA	EaseAdvantage
EAFW	EaseAdvantage Framework
EAH	External Access Handler
EBCA	EISA Bus Channel Adapter
EBCDIC	Extended Binary Coded Decimal Interchange Code
ECC	Error Check and Correction
ECL	Emitter Coupler Logic
EDAC	Error Detection and Correction
EDI	Electronic Data Interchange
EDR	Enhanced Data Rate, 100 Gb bandwidth
EEPROM	Electrically Erasable Programmable Read Only Memory
EGA	Enhanced Graphics Adapter (IBM)
EISA	Extended Industry Standard Architecture
EOE	Enhanced Operating Environment
EON	Extended Object Names (or Naming) – 128-character object names
EPROM	Erasable Programmable Read-Only Memory
ESCON	Enterprise System Connection
ESM	Environmental Services Monitor
EUC	Extended UNIX Code

-F-

4GL	Fourth Generation Language
FAX	Facsimile
FC	Fibre Channel
FDDI	Fiber Distributed Data Interface
FDR	Fourteen Data Rate, 56 Gb bandwidth
FDR10	Fourteen Data Rate, 40 Gb bandwidth
FE	Field Engineer
FEU	Front End Unit
FICON	Fibre Connection (IBM)
FIFO	First In, First Out
FIPS	Federal Information Processing Standards
FK	Foreign Key
FMLI	Forms and Menu Language Interpreter
FORTRAN	FORmula TRANslation
FSB	Front Side Bus
FSG	File Segment Cache
FRU	Field Replaceable Unit
FTAM	File Transfer and Access Method (OSI)
FTP	File Transfer Protocol
FUD	Fear, Uncertainty, Doubt
FW	Firmware
FYI	For Your Information

-G-

GB	Gigabyte (one billion bytes)
GCA	General Customer Availability
GCC	Global Consulting Center
GE	Graphic Engine
GOSIP	Government Open Systems Interconnection Profile
GPC	Google Public Cloud
GSC	Global Support Center
GUI	Graphical User Interface

-H-

HASP	Houston Automatic Spooling Program
HATP	High Availability Transaction Processing
HBN	Hash Bucket Number
HCA	Host Channel Adapter
HDA	Home Disk Address (used with Virtual Storage Services)
HDD	Hard Disk Drives
HDLC	High-level Data Link Control
HI	Hash Index
HOLAP	Hybrid On-Line Analytical Processing
HSN	Hot Standby Node

HW	Hardware
Hz	Hertz - cycles per second

-I-

iABI	Intel Application Binary Interface
I2E	Internal-to-External
IB	InfiniBand®
I2C or I2C	Inter-Integrated Circuit
IC	Integrated Circuit
ICI	Initial Certified Installation
ICMB	Intelligent Chassis Management Bus
ICSI	Integrated Complex Service Interface
IDE	Integrated Development Environment
IE	Information Engineering
IEEE	Institute of Electrical and Electronics Engineers
I/L	Memory Inter-Leave controller
IMS	Information Management System, Inventory Management System
IO	Input/Output
IP	Internet Protocol
IPMB	Intelligent Platform Management Bus
IPS	Integrated Peripheral Subsystem
IRQ	Interrupt Request
ISA	Industry-Standard Architecture
ISAM	Indexed Sequential Access Method
ISD	Interactive System Definition
ISDN	Integrated Services Digital Network
ISO	International Standards Organization
ISP	Internet Support Package
ISPF	Interactive System Productivity Facility
ISV	Independent Software Vendor
IT	Information Technology
IWM	Integrated Workload Management

-J-

JBOD	Just a Bunch of Disks
JCL	Job Control Language
JDBC	Java Database Connectivity
JDO	Joint Development Operation (Teradata and NCR)
JES	Job Entry Subsystem
JFS	Journaling File System
JI	Join Index
JIS	Japanese Industrial Standards
JIT	Just-In-Time
JSON	JavaScript Object Notation
JVM	Java Virtual Machine

-K-

Kb	Kilobit
KB	Kilobyte (1,024 bytes)
Kbaud	Kilo Baud
Kbps	Kilobits per second
KBPS	Kilobytes per second
KHz	Kilohertz
KSR	Keyboard Send/Receive

-L-

LAN	Local Area Network
LARC	Limited Address Range Cache
LCMP	Loosely Coupled Multiprocessing
LCD	Liquid Crystal Display
LDAP	Lightweight Directory Access Protocol
LDM	Limited Distance Modem, Local Domain Manager
LED	Light Emitting Diode
LFM	Log File Management
LM	Local Media Module
LOB	Large Object (either a BLOB or CLOB)
LPB	Local Peripheral Board
LPI	Language Processors Inc.
lpm	lines per minute (or line-per-minute)
LSU	Logical Storage Unit
LT/ST	Large Table/Small Table (join)
LUN	Logical Unit (disk array logical unit)

-M-

MA	Modular Array
MAC	Media Access Control, Medium Access Control, Message Authentication Code
MAP	Manufacturing Automation Protocol, Maintenance Analysis Procedures, Master Application Processor
Mb	Megabit
MB	Megabyte
Mbps	Megabits per second
ML	Machine Learning (Engine)
MLAN	Management LAN
MCA	Micro Channel Architecture
MCCA	Micro Channel to Channel Adapter
MCIA	Micro Channel Interface Architecture
MHz	MegaHertz - million cycles per second
MI	Master Index
MIPS	Million Instructions per Second
MIS	Management Information System
MLPPI	Multi-Level Partitioned Primary Index

MO	Method of Operation
MOLAP	Multi-dimensional On-Line Analytical Processing
MPEG	Moving Picture Experts Group
MPL	Message Passing Layer
MPP	Massively Parallel Processing
MSU	Memory Storage Unit
MTBDL	Mean Time Between Data Loss
MTBF	Mean Time Between Failures
MTBR	Mean Time Between Repairs
MTDP	Micro Teradata Director Program
MVC	Multi-value Compression
MVS	Multiple Virtual Storage (IBM mainframe OS)

-N-

NA	Network Agent
ND	No Duplicates
NetBIOS	Network Basic Input/Output System
NFS	Network File System
NI	Network Interface
NIST	National Institute of Standards and Technology
NLQ	Near Letter Quality
NN	No Nulls
NOBOM	No Byte Order Mark (associated with Unicode)
NoPI	No Primary Index Table
NOS	Native Object Store
NPPI	Non-Partitioned Primary Index
NRZ	Non-Return to Zero
Ns	Nanosecond
NSC	NCR Storage Cabinet
NTOS	NCR Teradata Operating System
NUPI	Non-Unique Primary Index
NUSI	Non-Unique Secondary Index
NVRAM	Non-Volatile Random-Access Memory (read and write memory)

-O-

OCC	Open Cooperative Computing
OCCA	Open Cooperative Computing Architecture
OCR	Optical Character Reader, Optical Character Recognition
ODBC□	Open Database Connectivity
ODS	Operational Data Store
OE	Operating Environment
OEM	Original Equipment Manufacturer
OLAP	On-Line Analytical Processing
OLCP	On-Line Complex Processing
OLE	Object Linking and Embedding
OLTP	On-Line Transaction Processing

OMC	ORION Memory Controller chip
OMC-DC	ORION Memory Controller - DRAM Controller
OMC-DP	ORION Memory Controller - Data Path Controller
ONE	Open Network Environment
ONS	Open Networking System
OOP	Object Oriented Programming
OPB	ORION PCI Bridge chip
OPS	Oracle Parallel Server
OS	Operating System
OSA	Open Systems Architecture
OSF	Open Software Foundation
OSI	Open Systems Interconnect
OSI/DTP	Open Systems Interconnect/Distributed Transaction Processing
OSS	Open System Standards
OTOS	Open Teradata Operating System
OUC	Object Use Count

-P-

PA	Primary AMP index
PB	Petabyte (quadrillion bytes)
PBX	Private Branch Exchange
PC	Personal Computer, Printed Circuit
PCEB	PCI EISA Bridge chip
PCI	Peripheral Component Interconnect
PCMCIA	Personal Computer Memory Card International Association
PC/RA	Probable Cause/Recommended Action
PSF	Teradata Priority Scheduler Framework
PCS	Personal Computer Support
PDB	Parallel Data Bus
PDCR	Performance Data and Collection Repository or Reporting
PDE	Parallel Database Extensions
PDN	Public or Private Data Network
PE	Parsing Engine
PEP	Parsing Engine Processor
PIM	Plug-In Module (used with V3 BYNET 32 switch)
PJI	Product Join Indicator (> 5 indicates a bad product join)
PK	Primary Key (or Parent Key)
PL/1	Programming Language I
PLAN	Private LAN
PM/PC	Performance Management/Production Control
PN	Primary Network
POPS	Parallel Object Processing System
POS	Point of Sale (noun), Point-of-Sale (terminal) (adj.)
POSIX	Portable Operating System Interface for Computing Environments
POST	Power-On System Test
PPI	Partitioned Primary Index

PQS	PCI Quad SCSI
PSA	Priority Scheduler Administrator
PSC	Power Shelf Controller
PSF	Teradata Priority Scheduler Framework
PSI	Power Supply Interface
PTB	Pass Through Board
PTI	Primary Time Index
PUT	Parallel Upgrade Tool

-Q-

QA	Quality Assurance
QCD	Query Capture Database
QFC	Quad Fibre Channel
QIC	Quarter Inch Cartridge

-R-

RAID	Redundant Array of Independent (formerly, Inexpensive) Disks
RAM	Random Access Memory
RASUI	Reliability, Availability, Serviceability, Usability, Installability
RC	Remote Client
RCMB	Rack (or Remote) Chassis Management Board
RCP	Row-partitioned and Column-Partitioned
RCP NoPI	Row-partitioned and Column-Partitioned No Primary Index
RCP PA	Row-partitioned and Column-Partitioned Primary AMP index
RCP PI	Row-partitioned and Column-Partitioned Primary Index
RDAC	Redundant Disk Array Controller (software)
RDBMS	Relational Database Management System
RFC	Request for Change
RFP	Request for Proposal
RFS	Remote File System
RGB	Red, Green, Blue
RI	Read Intensive (for SSD drives)
RISC	Reduced Instruction Set Computing
RJE	Remote Job Entry
RLL	Row Level Locking
RLM	Revision Level Manager
RM	Resource Manager (TOP END) or Rack Mount
ROLAP	Relational On-Line Analytical Processing
ROM	Read Only Memory
ROSE	Remote Operation Service Element
RPC	Remote Procedure Call
RPG	Report Program Generator
rpm	revolutions (or rotations) per minute
RPPI	Row-Partitioned Primary-Index
RPO	Recovery Point Objective (DSA – usually when last backup was taken)

RSC	Remote System Call
RSS	Resource Sampling Subsystem (used for performance capture)
RTO	Recovery Time Objective (associated with DSA)
RTS	Ready to Send, Request to Send
RTS-CTS	Request to Send - Clear to Send

-S-

SAN	Storage Area Network
SAR	System Activity Report (sar – UNIX/Linux utility)
SCO	Santa Cruz Operation UNIX
SCSI	Standard (or Small) Computer System Interface
SDL	Screen Definition Language
SDW	Scalable Data Warehouse
SE	Systems Engineer
SEEPROM	Serial Electronically Erasable Programmable Read-Only Memory
SI	Self-Instruction
SIMM	Single In-line Memory Module
SKU	Stock-keeping Unit
SLAN	Service or System LAN (Local Area Network)
SLES	SUSE Linux Enterprise Server
SLC	Second Level Cache
SM3G	Server Management – 3rd Generation
SMB	Server Management Board
SMC	Server Management Chassis
SMP	Symmetrical Multi-Processing
SN	Secondary Network
SNA	Systems Network Architecture
SNAG	SNA Gateway
SNO	SuperNode
SNP	Super Node Processor
SOV	Single Operational View
SP	Stored Procedure
SPI	System Performance Investigator
SQL	Structured Query Language
SQLCA	Structured Query Language Communication Area
SQLDA	Structured Query Language Data Area
SRAM	Static Random-Access Memory
SS	Support Sentinel
SSA	Symbolic Service Address, System-to-System Adapter
SSD	Solid State Drive
SSI	Single System Image, Standard Serial Interface
SSL	Secure Socket Layer (Networking)
SSP	Software Support Program, System Support Program (IBM)
STMP	Simple Mail Transfer Protocol
SUS	Startup Subsystem
SUSE	Software und System Entwicklung (German name which means

Software and Systems Development)

SVID	System V Interface Definition
SVR4	System V Release 4
SW	Software
SWS	Service Workstation (used with 56xx and later systems)
SYSGEN	System Generation

-T-

3GL	Third Generation Language
T2A	Teradata to Aster (QueryGrid)
T2H	Teradata to Hadoop (QueryGrid)
T2T	Teradata to Teradata (QueryGrid)
TAM	Teradata Access Method (used with Replication Services)
TAP	Teradata Application Program
TARA	Teradata Archive/Restore Architecture
TASM	Teradata Active System Management
TAXI	Transparent Asynchronous Xceiver Interface
TB	Terabyte (a trillion bytes)
TBA	To Be Announced
TBD	To Be Determined
TCAM	Telecommunications Access Method
TCP/IP	Transmission Control Protocol/Internet Protocol
TD	Teradata
TDAS	Teradata Dual Active Solution
TDQM	Teradata Dynamic Query Manager
TFTP	Trivial File Transfer Protocol
TIM	Teradata Intelligent Memory
TLI	Transport Layer Interface
TLS	Transport Layer Security (Networking)
TM	Transaction Manager
TMSM	Teradata Multi-System Manager
TOD	Time of Day (noun) or Time-of-Day (adj.)
TOS	Teradata Operating System
TQD	Teradata Query Director
TP	Transaction Processing
TPA	Trusted Parallel Application or Architecture
TPD	Teradata Parallel Database
TPS (tps)	transactions per second
TPT	Teradata Parallel Transporter
TSET	Teradata System Emulation Tool
TSO	Time-Sharing Option
TTL	Transistor-Transistor Logic
TTU	Teradata Tools and Utilities
TTY	Teletype (serial communication protocol)
TVI	Teradata Vital Infrastructure (software to alert remote support)
TVS	Teradata Virtual Storage

TVSS	Teradata Virtual Storage Services
TWA	Teradata Workload Analyzer
TWB	Teradata Warehouse Builder (old name for TPT)

-U-

U	Unit of Height Measurement in cabinet or rack – 1.75” or 4.445 cm
UDF	User Defined Function
UDM	User Defined Method
UDP	Unreliable Data Protocol
UDT	User Defined Type
UFS	UNIX File System
UII	Unnecessary I/O Indicator (>5 may indicate an index is needed)
UMB	Universal Management Board
UPI	Unique Primary Index
UPS	Uninterruptible Power Source
UPS-IS	Uninterruptible Power Source – Input Selector
USI	Unique Secondary Index

-V-

VAR	Value Added Reseller
VCX	Vantage Customer Experience
VDC	Volts Direct Current
VDT	Video Display Terminal
VGA	Video Graphics Array
VLC	Value List Compression
VLF	Very Low Frequency
VLSI	Very Large-Scale Integration
VM	Virtual Machine
VONUSI	Valued-ordered NUSI
VPIX	"DOS under UNIX"
VPROC	Virtual Processor
VSAM	Virtual System Access Method
VSS	Virtual Storage Services
VTAM	Virtual Telecommunications Access Method

-W-

WAL	Write Ahead Logic or Write Ahead Logging
WAN	Wide Area Network
WAWS	Windows AWS (Administration Workstation)
WBEM	Web-Based Enterprise Management
WCI	WAL Cylinder Index
WI	Write Intensive (for SSD drives)
WinDDI	Windows Data Dictionary Interface (e.g., Teradata Administrator)
WIN-TCP	Wollongong Integrated Networking/Transmission Control Protocol
WMI	WAL Master Index
WORM	Write Once - Read Many Optical Disk

WTO	Write to Operator
WYSIWYG	What You See Is What You Get

-X-

XA	Extended Architecture
XMIT	Transmit
XML	Extensible Markup Language
XSP	External Stored Procedure

-Y-

YMCA	Ynet Micro Channel Architecture adapter
YTD	Year-to-Date



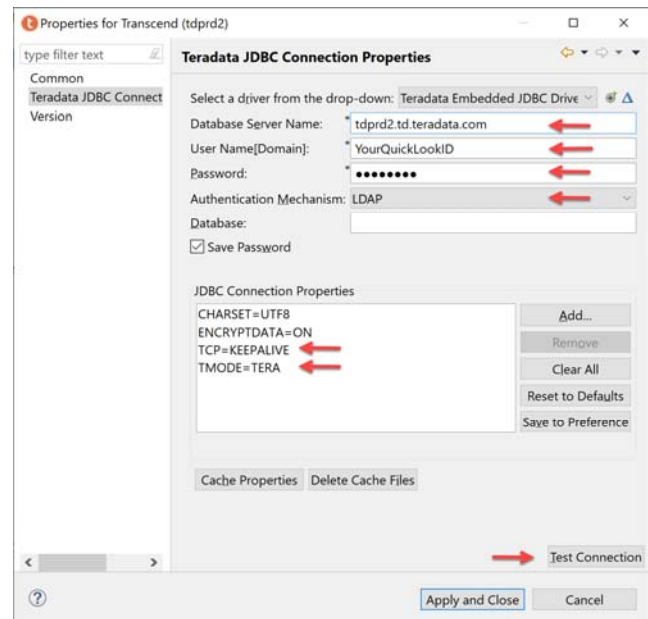
Appendix D: The Lab Environment

Teradata Vantage Administration

Copyright © 2007–2023 by Teradata. All Rights Reserved.

If Using Transcend, Hands-on Setup (1 of 2)

Teradata Vantage system: you can use **tdprd2.td.teradata.com** (preferred for training purposes).



To create a Transcend connection, specify the following:

Database Server Name:	tdprd2.td.teradata.com
User Name:	your Quicklook ID
Password:	your Teradata password
Authentication Mechanism:	LDAP

Also specify the following JDC properties:

CHARSET = UTF8
TCP = KEEPALIVE
TMODE = TERA

Use the Test Connection option to verify the connection with the username and password.

Although there is another Transcend system (tdprd.td.teradata.com), you should use the connection to "tdprd2" for training and education purposes. You have additional privileges on "tdprd2" that do not exist on the "tdprd" system.

If Using Transcend, Hands-on Setup (2 of 2)

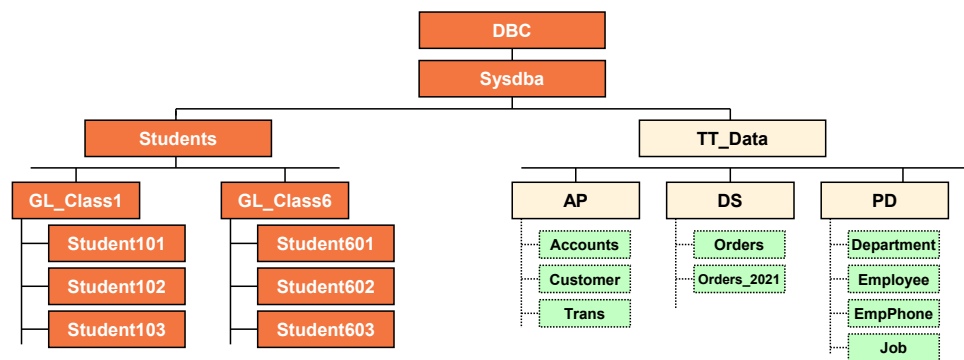
- ▼ TRNG_TDU_FACTORY
 - > Indexes
 - > Macros
 - > Stored Procedures
 - > Tables
 - > User-Defined Functions
 - > Views
 - > TRNG_AP
 - > TRNG_Collaterals
 - > TRNG_DS
 - > TRNG_PD

On the Transcend system (tdprd2.td.teradata.com), [use the following databases and CREATE TABLE statements to set up your user environment.](#)

CREATE SET TABLE Accounts	AS TRNG_AP.Accounts	WITH NO DATA;
CREATE SET TABLE Customer	AS TRNG_AP.Customer	WITH NO DATA;
CREATE SET TABLE Trans	AS TRNG_AP.Trans	WITH NO DATA;
CREATE SET TABLE Department	AS TRNG_PD.Department	WITH NO DATA;
CREATE SET TABLE Employee	AS TRNG_PD.Employee	WITH NO DATA;
CREATE SET TABLE Job	AS TRNG_PD.Job	WITH NO DATA;
CREATE SET TABLE EmpPhone	AS TRNG_PD.EmpPhone	WITH NO DATA;
CREATE SET TABLE Orders	AS TRNG_DS.Orders	WITH NO DATA;
CREATE SET TABLE Orders_2021	AS TRNG_DS.Orders_2020	WITH NO DATA;

Copy and paste the CREATE TABLE statements into your Studio Query window to establish the set of empty tables used in lab exercises in the Teradata database classes.

Lab Environment



Common Teradata SQL Commands:

To populate a newtable:

```
INSERT INTO newtable
SELECT * FROM db.olddtab WITH [NO] DATA;
```

HELP USER username;	- lists objects owned by user
HELP 'SQL sqlcommand';	- help for an SQL command
SHOW TABLE tablename;	- shows table creation syntax
SELECT COUNT(*) FROM tname;	- provides count of rows
DELETE tablename ALL;	- deletes all rows in a table
DROP TABLE tablename;	- drops table from the database

Useful Linux Commands:

ls -l	- long listing of Linux files
more filename	- view contents of text file
vi filename	- create or edit a text file
cp file1 file2	- copies file1 to file2
mv oldname newname	- rename a file
rm filename	- delete/remove a file
exit	- exits Linux

Most access to Teradata will be through Teradata Studio or Teradata Studio Express.

However, you may need to connect from a Windows PC to a Linux system. Linux requires a Secure Shell connection. This opens a terminal window which you can use to logon to Linux and enter Linux commands.

putty is an example of a utility that provides a secure shell connection

If you are executing BTEQ interactively, you cannot enter your password as part of the .LOGON statement. BTEQ will prompt you for your Teradata username password.

AP Tables

AP.Accounts (10,000 Rows)

Account Number	Street Number	Street	City	State	Zip Code	Balance Forward	Balance Current
PK							
UPI							
20024010	123	Harbor Blvd.	Torrance	CA	90323	1000	900.00
20031023	3456	186th St.	Glendale	CA	90451	1500	1700.00
20049873	100	Western Av.	Las Vegas	NV	97345	400	400.00
20031134	10	Heather Rd.	S. Monica	CA	92345	6020	5312.00
Integer	Integer	Char(25)	Char(20)	Char(2)	Integer	Decimal(10,2)	Decimal(10,2)

Note:

Column names that span two lines are connected with an underscore.

Ex: Account_Number

AP.Customer (7,000 Rows)

Customer Number	Last Name	First Name	Social Security
PK			
UPI			
13021	Smith	George	456788765
18765	Jones	Barbara	987453498
11023	Wilson	John	495028367
1123	Omaguchi	Sandra	234904587
Integer	Char(30)	Char(20)	Integer

AP.Trans (15,000 Rows)

Trans Number	Trans Date	Account Number	Trans ID	Trans Amount
PK				
UPI				
4653	2/11/2002	20024020	2009	-50.00
3241	2/8/2002	20034567	DEP	160.00
1298	2/8/2002	20024005	2987	-70.00
11026	2/13/2002	20024020	DEP	20.00
Integer	Date	Integer	Char(4)	Decimal(10,2)

These are populated tables in the AP Database.

DS Tables

Orders (450,000 Rows)

orderid	custid	orderstatus	ordertotal	orderdate	orderpriority	clerk	location	shippriority	ordercomment
PK	FK,NN								
UPI									
100001	1001	C	1,005.00	1/2/2008	10	Jack ..	5	20	In Stock
103501	1451	C	1,005.00	12/1/2009	10	Dee ...	3	20	In Stock
101400	1080	C	1,150.00	2/28/2010	10	Fred ...	10	20	In Stock
Integer	Integer	Char(1)	Decimal(10,2)	Date	SmallInt	Char(16)	SmallInt	SmallInt	Varchar(79)

Orders_2021 (150,000 Rows) – Same Layout

These are populated tables in the DS Database.

PD Tables

Employee (150,000 Rows)

Emp Number	Dept. Number	EmpMgr Number	Job Code	Last Name	First Name	Salary Amount
PK	FK	FK	FK			
UPI						
100001	999	?	8903	DeBosse	Ibee	200000
100797	1048	100791	3017	Myers	Ruth	41000
100002	1001	100001	3001	Smith	Steve	110000
Integer	Integer	Char(25)	Char(20)	Char(2)	Integer	Decimal(10,2)

Job (11,124 Rows)

Job Code	Job Desc
PK	
UPI	
3000	President
3001	Senior Mgmt
3017	Analyst L2
Integer	Char(50)

Department (17,415 Rows)

Dept. Number	Dept Name	DeptMgr Number	Budget Amount
FK	FK	FK	
1048	Design SW	100791	1000000.00
1050	Design Services	100811	1000000.00
1028	Engineering SW	100441	3000000.00
Integer	Char(100)	Integer	Decimal(10,2)

EmpPhone (350,000 Rows)

Emp Number	Area Code	Phone Number	Extension
FK	PK		
NUPI			
100001	937	5100001	1001
100001	937	4100001	1001
100389	858	4852815	815
Integer	SmallInt	Integer	Integer

Note:

Column names that span two lines are NOT connected with an underscore for these tables.

Ex: EmpNumber

These are populated tables in the PD Database.

vi or vim Commands

Input mode	Movement and other	Modification	Colon mode
i insert before cursor	h move cursor to left	r replace 1 character	:w write or save the file
I insert at beginning of line	j move cursor down a line	x delete char under cursor	:w <i>fname</i> write new file name
a insert after cursor	k move cursor up a line	X delete char before cursor	:wq write and quit
A insert at end of line	l move cursor to right	5x delete 5 characters	:q! quit and don't save
o open a new line below	space move cursor to right	dw delete a word	
O open a new line above	w move a word to right	dd delete the line	:e <i>fname</i> edit another file
R Replace mode and insert after <cr>	b move a word to left	d\$ delete the rest of the line	:r <i>fname</i> read in a file
	0 go to start of a line (zero)	dG delete rest of the file	:!cmd run UNIX command
	\$ go to end of a line	cw change a word	:set smd set showmode on
	G go to end of file	C change rest of the line	
	nG go to line n	J join 2 lines together	
	^f scroll forward 1 screen	yy yank or copy a line	
	^b scroll back 1 screen	3yy yank or copy 3 lines	
	^l refresh the screen	p put yanked line(s) after line	
	/str search forward for str	P put yanked line(s) before line	
	. repeat last command	u undo last command	

vim filename

ESC exits input & colon modes

If you are not comfortable or familiar with vi or vim, you can create your script in a Windows Notepad, copy it, and paste it into a vi file.

1. Enter your commands (job/script) in a Notepad file.
2. Highlight the text and use the mouse to choose the **Edit → Copy** function

Switch to your terminal window where Linux is running and ...

3. **vi labx_1.btq** (or whatever filename you wish)
 - enter an **i** by itself (do not press <Enter>)
 - Use the mouse to choose the **Edit → Paste** function
 - Press the ESC key (multiple times does not hurt)
 - Enter **:wq** (this saves the file and exits vi)

Another technique that can be used to create Linux scripts without using vi is to do the following:

1. Enter your commands (job/script) in a Notepad file.
2. Highlight the text and use the mouse to choose the **Edit → Copy** function

Switch to your terminal window where Linux is running and ...

3. **cat > labx_1.btq** (or whatever filename you wish)
 Use the mouse to choose the **Edit → Paste** function.
 To exit the cat command, press either the DELETE key or CNTL C.



Appendix E: Data Dictionary Views

Teradata Vantage Administration

Copyright © 2007–2023 by Teradata. All Rights Reserved.

Data Dictionary Views – Teradata 16.20

DBC.AccessLog[V]

User Type	Columns Selected
Security Administrator	LogDate LogTime LogonDate LogonTime LogicalHostId IFPNo SessionNo UserName AccountName OwnerName AccessType Frequency EventCount AccLogResult DatabaseName TVMName ColumnName StatementType StatementText

DBC.AccessLog[V] (cont.)

User Type	Columns Selected
Security Administrator	QueryBand ProxyUser ConstraintId

Data Dictionary views are part of the Teradata Database Data Dictionary and reside in the space owned by the system user DBC. They provide information about users, their access rights, grants, and logons.

View definitions are stored in the table DBC.TVM. View column information is stored in DBC.TVFields.

The following are the view forms:

- Without the X (for example, DBC.AccountInfo and DBC.AccountInfoV), they display global information.
- With the X (for example, DBC.AccountInfoX and AccountInfoVX), they display information associated with the requesting user only.
- With the V (for example, AccessLogV), they display information associated with the Unicode version, where object name columns have a data type of VARCHAR(128).
- Without the V (for example, DBC.AccountInfo or DBC.AccountInfoX), they display information associated with the Compatibility version, where object name columns have a data type of CHAR(30).

Data Dictionary Views – Teradata 16.20

DBC.AccLogRules[V]

User Type	Columns Selected
Security Administrator	UserName DatabaseName TVMName ConstraintName AcrAlterFunction AcrCheckpoint AcrCreateDatabase AcrCreateFunction AcrCreateMacro AcrCreateTable AcrCreateUser AcrCreateView AcrCreateProcedure AcrCreExtProcedure AcrDelete AcrDropDatabase AcrDropFunction AcrDropMacro AcrDropTable AcrDropUser AcrDropView AcrDropProcedure AcrDump AcrExecute AcrExecuteFunction

DBC.AccLogRules[V] (cont.)

User Type	Columns Selected
Security Admin.	AcrExectuteProcedure AcrGrant AcrIndex AcrInsert AcrReference AcrRestore AcrSelect AcrUpdate AcrCreateTrigger AcrDropTrigger AcrCreateRole AcrDropRole AcrCreateProfile AcrDropProfile AcrAlterProcedure AcrRepControl AcrAlterExtProcedure AcrUDTUsage AcrUDTType AcrUDTMethod AcrCreAuthorization AcrDropAuthorization AcrStatistics AcrShow AcrCreOwnerProcedure

DBC.AccLogRules[V] (cont.)

User Type	Columns Selected
Security Admin.	ArcConnectThrough CreatorName CreateTimeStamp ArcCreateGLOP ArcDropGLOP ArcGLOPMember AcrConstrDef AcrConstrAsgn AcrOverrideIns AcrOverrideSel AcrOverrideUpd AcrOverrideDel AcrOverrideDump AcrOverrideRestore AcrCreateZone AcrDropZone AcrZoneOverride AcrCreateDatasetSchema AcrDropDatasetSchema AcrWithDatasetSchema AcrCreateMap AcrDropMap

Data Dictionary Views – Teradata 16.20

DBC.All_RI_Children[V][X]

User Type	Columns Selected
All Users	IndexID IndexName ChildDB ChildTable ChildKeyColumn ParentDB ParentTable ParentKeyColumn InconsistencyFlag CreatorName CreateTimeStamp

DBC.AccountInfo[V][X]

User Type	Columns Selected
Database Administrator End User Supervisor	UserName AccountName UserOrProfile

Data Dictionary Views – Teradata 16.20

DBC.All_RI_Parents[V][X]

User Type	Columns Selected
All Users	IndexID IndexName ParentDB ParentTable ParentKeyColumn ChildDB ChildTable ChildKeyColumn InconsistencyFlag CreatorName CreateTimeStamp

Data Dictionary Views – Teradata 16.20

DBC.AllGlobalSpaceV[X]

User Type	Columns Selected
All Users	DatabaseName AccountName TableName MaxPerm MaxSpool MaxTemp CurrentPerm CurrentSpool CurrentPersistentSpool CurrentTemp PeakPerm PeakSpool PeakPersistentSpool PeakTemp MaxProfileSpool MaxProfileTemp TrustUserName AppProxyUser AllocatedPerm AllocatedSpool

DBC.AllGlobalSpaceV[X] (cont.)

User Type	Columns Selected
All Users	AllocatedTemp PermSkew SpoolSkew TempSkew

Data Dictionary Views – Teradata 16.20

DBC.AllRoleRights[V]

User Type	Columns Selected
Security Administrator Database Administrator Supervisor	RoleName DatabaseName TableName ColumnName AccessRight GrantorName CreateTimeStamp

DBC.AllRights[V][X]

User Type	Columns Selected
Database Administrator	UserName DatabaseName TableName ColumnName AccessRight GrantAuthority GrantorName AllnessFlag CreatorName CreateTimeStamp

Data Dictionary Views – Teradata 16.20

DBC.AllSpace[V][X]	
User Type	Columns Selected
Database Administrator	Vproc
End User	DatabaseName
Supervisor	AccountName
	TableName
	MaxPerm
	MaxSpool
	MaxTemp
	CurrentPerm
	CurrentSpool
	CurrentPersistentSpool
	CurrentTemp
	PeakPerm
	PeakSpool
	PeakPersistentSpool
	PeakTemp
	MaxProfileSpool
	MaxProfileTemp
	TrustUserName
	AppProxyUser

Data Dictionary Views – Teradata 16.20

DBC.AllSpace[V][X] (cont.)

User Type	Columns Selected
Database Administrator	AllocatedPerm
End User	AllocatedSpool
Supervisor	AllocatedTemp
	PermSkew
	SpoolSkew
	TempSkew

DBC.AllTempTables[V][X]

User Type	Columns Selected
Database Administrator	HostNo
End User	SessionNo
	UserName
	B_ DatabaseName
	B_ TableName
	E_ TableId

Data Dictionary Views – Teradata 16.20

[DBC.AMPUsage\[V\]\[X\]](#)

User Type	Columns Selected
Database Administrator	AccountName UserName CpuTime DiskIO CpuTimeNorm Vproc VprocType Model

Data Dictionary Views – Teradata 16.20

DBC.Association[V][X]

User Type	Columns Selected
Operations Control	DatabaseName TableName EventNum Original_DataBaseName Original_TableName Original_TableKind Original_Version Original_ProtectionType Original_JournalFlag Original_CreatorName Original_CommentString

DBC.ArchiveLoggingObjsV[X]

User Type	Columns Selected
Database Administrator	DatabaseName TVMName LogLevel CreatorName CreateTimeStamp

Data Dictionary Views – Teradata 16.20

[DBC.Authorizations\[V\]\[X\]](#)

User Type	Columns Selected
Database Administrator	DatabaseName AuthorizationName AuthorizationId TableKind Version AuthorizationType AuthorizationSubType OSDomainName OSUserName

Data Dictionary Views – Teradata 16.20

Sys_Calendar.BusinessCalendar

User Type	Columns Selected
DB Administrator	calendar_date
End User	day_of_week
	day_of_month
	day_of_year
	day_of_calendar
	weekday_of_month
	week_of_month
	week_of_quarter
	week_of_year
	week_of_calendar
	month_of_quarter
	month_of_year
	month_of_calendar
	quarter_of_year
	quarter_of_calendar
	year_of_calendar
	WeekBegin
	WeekEnd
	MonthBegin
	MonthEnd
	QuarterBegin

Sys_Calendar.BusinessCalendar (cont.)

User Type	Columns Selected
DB Administrator	QuarterEnd
End User	YearBegin
	YearEnd
	IsBusinessDay
	BusinessWeekBegin
	BusinessWeekEnd
	BusinessMonthBegin
	BusinessMonthEnd
	BusinessQuarterBegin
	BusinessQuarterEnd
	BusinessYearBegin
	BusinessYearEnd

Data Dictionary Views – Teradata 16.20

Sys_Calendar.BusinessCalendarExceptions

User Type	Columns Selected
Database Administrator	CalendarName ExceptionIndicator ExceptionDate ExceptionReason CreatorName CreationTime

Sys_Calendar.BusinessCalendarPatterns

User Type	Columns Selected
Database Administrator	CalendarName DayName Pattern PatternComment CreatorName LastModified

Data Dictionary Views – Teradata 16.20

DBC.CharTranslations[V]

User Type	Columns Selected
End User	CharSetName CharSetId InstallFlag E2I E2IUp I2E I2EUp

DBC.Children[V][X]

User Type	Columns Selected
Supervisor	Child Parent

DBC.CharSets[V]

User Type	Columns Selected
End User	CharSetName

Data Dictionary Views – Teradata 16.20

DBC.Collations[V]

User Type	Columns Selected
End User	CollName CollInstall CollEqvClass CollOrderCS CollOrderUC

DBC.ColumnUseCountV[X]

User Type	Columns Selected
Database Administrator	DatabaseName TableName FieldName UsageType AccessCount

Data Dictionary Views – Teradata 16.20

DBC.ColumnsV[X]

User Type	Columns Selected
DB Administrator End User	DatabaseName
	TableName
	ColumnName
	ColumnFormat
	ColumnTitle
	SPPParameterType
	ColumnType
	ColumnUDTName
	ColumnLength
	DefaultValue
	Nullable
	CommentString
	DecimalTotalDigits
	DecimalFractionalDigits
	ColumnId
	UpperCaseFlag
	Compressible
	CompressValue
	ColumnConstraint
	ConstraintCount

DBC.ColumnsV[X] (cont.)

User Type	Columns Selected
DB Administrator End User	CreatorName
	CreateTimeStamp
	LastAlterName
	LastAlterTimeStamp
	CharType
	IdColType
	AccessCount
	LastAccessTimeStamp
	CompressValueList
	TimeDimension
	VTCheckType
	TTCheckType
	ConstraintId
	ArrayColNumberOfDimensions
	ArrayColScope
	ArrayColElementType
	ArrayColElementUdtName
	PartitioningColumn
	ColumnPartitionNumber
	ColumnPartitionFormat

DBC.ColumnsV[X] (cont.)

User Type	Columns Selected
DB Administrator End User	ColumnPartitionFormat
	ColumnPartitionAC
	PseudoUDTFieldId
	PseudoUDTFieldType
	StorageFormat
	DatasetSchemaName
	InlineLength
	TSColumnType

Data Dictionary Views – Teradata 16.20

DBC.ColumnsqV[X]

User Type	Columns Selected
DB Administrator End User	DatabaseName
	TableName
	ColumnName
	ColumnFormat
	ColumnTitle
	SPPParameterType
	ColumnType
	ColumnUDTName
	ColumnLength
	DefaultValue
	Nullable
	CommentString
	DecimalTotalDigits
	DecimalFractionalDigits
	ColumnId
	UpperCaseFlag
	Compressible
	CompressValue
	ColumnConstraint
	ConstraintCount

DBC.ColumnsqV[X] (cont.)

User Type	Columns Selected
DB Administrator End User	CreatorName
	CreateTimeStamp
	LastAlterName
	LastAlterTimeStamp
	CharType
	IdColType
	AccessCount
	LastAccessTimeStamp
	CompressValueList
	TimeDimension
	VTCheckType
	TTCheckType
	ConstraintId
	ArrayColNumberOfDimensions
	ArrayColScope
	ArrayColElementType
	ArrayColElementUdtName
	PartitioningColumn
	ColumnPartitionNumber
	ColumnPartitionFormat

DBC.ColumnsqV[X] (cont.)

User Type	Columns Selected
DB Administrator End User	ColumnPartitionAC
	PseudoUDTFieldId
	PseudoUDTFieldType
	StorageFormat
	DatasetSchemaName
	InlineLength
	TSColumnType

Data Dictionary Views – Teradata 16.20

DBC.ColumnsjqV[X]

User Type	Columns Selected
DB Administrator End User	DatabaseName
	TableName
	ColumnName
	ColumnFormat
	ColumnTitle
	SPPParameterType
	ColumnType
	ColumnUDTName
	ColumnLength
	DefaultValue
	Nullable
	CommentString
	DecimalTotalDigits
	DecimalFractionalDigits
	ColumnId
	UpperCaseFlag
	Compressible
	CompressValue
	ColumnConstraint
	ConstraintCount

DBC.ColumnsjqV[X] (cont.)

User Type	Columns Selected
DB Administrator End User	CreatorName
	CreateTimeStamp
	LastAlterName
	LastAlterTimeStamp
	CharType
	IdColType
	AccessCount
	LastAccessTimeStamp
	CompressValueList
	TimeDimension
	VTCheckType
	TTCheckType
	ConstraintId
	ArrayColNumberOfDimensions
	ArrayColScope
	ArrayColElementType
	ArrayColElementUdtName
	PartitioningColumn
	ColumnPartitionNumber
	ColumnPartitionFormat

DBC.ColumnsjqV[X] (cont.)

User Type	Columns Selected
DB Administrator End User	ColumnPartitionAC
	PseudoUDTFieldId
	PseudoUDTFieldType
	StorageFormat
	DatasetSchemaName
	InlineLength
	TSColumnType

Data Dictionary Views – Teradata 16.20

DBC.ColumnStatsV

User Type	Columns Selected
All Users	DatabaseName TableName ColumnName StatsName StatsSource ValidStats DBSVersion IndexNumber SampleSignature SampleSizePct ThresholdSignature MaxIntervals MaxValueLength RowCount UniqueValueCount PNullUniqueValueCount NullCount AllNullCount HighModeFreq PNullHighModeFreq

DBC.ColumnStatsV (cont.)

User Type	Columns Selected
All Users	StatsSkipCount CreateTimeStamp LastCollectTimeStamp LastAlterTimeStamp

Data Dictionary Views – Teradata 16.20

DBC.ConnectRules[V]

User Type	Columns Selected
Security Administrator	TrustUser
	ProxyUser
	ProxyUserType
	GrantStatus
	WithoutRole
	ProxyRole1
	ProxyRole2
	ProxyRole3
	ProxyRole4
	ProxyRole5
	ProxyRole6
	ProxyRole7
	ProxyRole8
	ProxyRole9
	ProxyRole10
	ProxyRole11
	ProxyRole12
	ProxyRole13
	ProxyRole14
	ProxyRole15

DBC.ConnectRules[V] (cont.)

User Type	Columns Selected
Security Administrator	CreatorName
	CreateTimeStamp
	TrustOnly
	ProfileName

Data Dictionary Views – Teradata 16.20

DBC.ConstraintFunctionsV

User Type	Columns Selected
Security Administrator	ConstraintName Action DatabaseName FunctionName

DBC.ConstraintValuesV

User Type	Columns Selected
Security Administrator	ConstraintName ValueName ValueConstant ValueIsBitPos

Data Dictionary Views – Teradata 16.20

DBC.CostProfiles_V

User Type	Columns Selected
All Users	ProfileTypeName ProfileName ProfileId ProfileCat ProfileDesc

DBC.CostProfileTypes_V

User Type	Columns Selected
All Users	ProfileTypeName ProfileTypeDesc

Data Dictionary Views – Teradata 16.20

DBC.CostProfileValues_V

User Type	Columns Selected
All Users	ProfileName ProfileId ConstName ConstId ConstCat ConstVal ConstDesc

DBC.CSPSessionInfo[V]

User Type	Columns Selected
Operations Control	SessionNo HostNo StartMBox LogonSource

Data Dictionary Views – Teradata 16.20

DBC.Databases[V][X]

User Type	Columns Selected
Database Administrator End User	DatabaseName CreatorName OwnerName AccountName ProtectionType JournalFlag PermSpace SpoolSpace TempSpace CommentString CreateTimeStamp LastAlterName LastAlterTimeStamp DBKind AccessCount LastAccessTimeStamp DefaultMapName MapOverride

DBC.Databases2[V][X]

User Type	Columns Selected
Database Administrator	DatabaseName DatabaseId UnResolvedRiCount

Data Dictionary Views – Teradata 16.20

[DBC.DBCInfo\[V\]](#)

User Type	Columns Selected
All Users	InfoKey InfoData

[DBC.Database_Default_Journals\[V\]\[X\]](#)

User Type	Columns Selected
Database Administrator End User	DatabaseName Journal_DB JournalName

[DBC.DatabaseUseCountV\[X\]](#)

User Type	Columns Selected
Database Administrator	DatabaseName UsageType AccessCount

Data Dictionary Views – Teradata 16.20

DBC.DBQLRules[V]

User Type	Columns Selected
All Users	UserName AccountString ApplName TypeofRule ExplainFlag ObjFlag SqlFlag StepFlag XMLPlanFlag StatsUsageFlag Verbose DetailedStats NoColumns SummaryFlag ThresholdFlag ObjectUsage ParamFlag FeatureUsage UtilityInfoFlag TextSizeLimit

DBC.DBQLRules[V] (cont.)

User Type	Columns Selected
All Users	SummaryVal1 SummaryVal2 SummaryVal3 LockDelay AlgMode TypeOfCriterion DetailDiag

Data Dictionary Views – Teradata 16.20

DBC.DeleteOldInDoubt[V]

User Type	Columns Selected
Database Administrator	LogicalHostId SessionNumber CoordTaskId RunUnitId LogonUserName ResolvingUserLogonName CommitOrRollback UserLogonDate UserLogonTime CompletionDate CompletionTime Options

DBC.DeleteAccessLog[V]

User Type	Columns Selected
Security Administrator	LogDate LogTime

Data Dictionary Views – Teradata 16.20

DBC.DiskSpace[V][X]

User Type	Columns Selected
Database Administrator	Vproc
End User	DatabaseName
Supervisor	AccountName
	MaxPerm
	MaxSpool
	MaxTemp
	CurrentPerm
	CurrentSpool
	CurrentPersistentSpool
	CurrentTemp
	PeakPerm
	PeakSpool
	PeakPersistentSpool
	PeakTemp
	MaxProfileSpool
	MaxProfileTemp
	TrustUserName
	AppProxyUser
	AllocatedPerm
	AllocatedSpool
	AllocatedTemp

DBC.DiskSpace[V][X] (cont.)

User Type	Columns Selected
Supervisor	PermSkew
	SpoolSkew
	TempSkew

Data Dictionary Views – Teradata 16.20

DBC.DeleteUseCountV[X]

User Type	Columns Selected
Database Administrator	DatabaseName ObjectName UsageType DeleteCount

DBC.DiskSpaceErrorV

User Type	Columns Selected
Database Administrator	Vproc DatabaseName MaxPerm CurrentPerm AllocatedPerm

DBC.DiskGlobalSpaceErrorV

User Type	Columns Selected
Database Administrator	DatabaseName MaxPerm CurrentPerm AllocatedPerm

Data Dictionary Views – Teradata 16.20

DBC.ErrorTblsV[X]

User Type	Columns Selected
Operations Control	ErrTblDbName ErrTblName BaseTblDbName BaseTblName CreatorName CreateTimeStamp

Data Dictionary Views – Teradata 16.20

DBC.Events[V][X]

User Type	Columns Selected
Operations Control	CreateDate CreateTime EventNum EventType UserName DatabaseName ObjectType AllAMPsFlag RestartSeqNum OperationInProgress TableName CheckPointName LinkingEventNum DataSetName LockMode JournalUsed JournalSaved IndexPresent DupeDumpSet

Data Dictionary Views – Teradata 16.20

DBC.Events_Configuration[V][X]	
User Type	Columns Selected
Operations Control	CreateDate CreateTime EventNum EventType UserName LogProcessor PhyProcessor Vproc ProcessorState RestartSeqNum

Data Dictionary Views – Teradata 16.20

DBC.Events_Media[V][X]

User Type	Columns Selected
Operations Control	CreateDate CreateTime EventNum EventType UserName DataSetName VolSerialId VolSequenceNum DupeDumpSet

DBC.ExportWidthV

User Type	Columns Selected
All Users	ExportDefinitionName ExportWidthRuleSet

Data Dictionary Views – Teradata 16.20

DBC.ExpStatsV

User Type	Columns Selected
Database Administrator	DatabaseName TableName ColumnName StatsName StatsSource ValidStats DBSVersion IndexNumber SampleSignature SampleSizePct ThresholdSignature MaxIntervals MaxValueLength RowCount UniqueValueCount PNullUniqueValueCount NullCount AllNullCount HighModeFreq PNullHighModeFreq

DBC.ExpStatsV (cont.)

User Type	Columns Selected
Database Administrator	CreateTimeStamp LastCollectTimeStamp LastAlterTimeStamp

Data Dictionary Views – Teradata 16.20

[DBC.ExternalSPs\[V\]\[X\]](#)

User Type	Columns Selected
Database Administrator	DatabaseName ExternalProcedureName ExternalProcedureID NumParameters ExternalName SrcFileLanguage NoSQLDataAccess ParameterStyle ExecProtectionMode ExtFileReference CharacterType Platform RoutineKind ParameterUDTIds AuthIDUsed AppCategory GLOPSetDatabaseName GLOPSetMemberName

Data Dictionary Views – Teradata 16.20

DBC.Functions[V][X]

User Type	Columns Selected
DB Administrator	DatabaseName FunctionName SpecificName FunctionId NumParameters ParameterDataTypes FunctionType ExternalName SrcFileLanguage NoSQLDataAccess ParameterStyle DeterministicOpt NullCall PrepareCount ExecProtectionMode ExtFileReference CharacterType Platform InterimFldSize RoutineKind

DBC.Functions[V][X] (cont.)

User Type	Columns Selected
DB Administrator	ParameterUDTIds AuthIdUsed MaxOutParameters GLOPSetDatabaseName GLOPSetMemberName RefQueryband ExecMapName ExecMapColocName

Data Dictionary Views – Teradata 16.20

DBC.GlobalDBSpaceV[X]

User Type	Columns Selected
All Users	DatabaseName AccountName MaxPerm MaxSpool MaxTemp AllocatedPerm AllocatedSpool AllocatedTemp PeakAllocatedPerm PeakAllocatedSpool PeakAllocatedTemp MaxProfileSpool MaxProfileTemp PermSkew SpoolSkew TempSkew SpaceMapName TrustUserName AppProxyUser

Data Dictionary Views – Teradata 16.20

DBC.HostsInfo[V]

User Type	Columns Selected
Database Administrator	LogicalHostId
End User	HostName
	DefaultCharSet

DBC.IndexConstraints[V][X]

User Type	Columns Selected
Database Administrator	DatabaseName
End User	TableName
	IndexName
	IndexNumber
	ConstraintType
	ConstraintText
	ConstraintCollation
	CollationName
	CreatorName
	CreateTimeStamp
	CharSetID
	SessionMode
	ResolvedCurrent_Date
	ResolvedCurrent_TimeStamp

Note:

The DBC.PartitioningConstraints views are the preferred views for information about partitioning constraints.

Data Dictionary Views – Teradata 16.20

DBC.IndexStatsV[X]

User Type	Columns Selected
Database Administrator	DatabaseName
End User	TableName
	ColumnName
	StatsName
	StatsSource
	ValidStats
	DBSVersion
	IndexNumber
	SampleSignature
	SampleSizePct
	ThresholdSignature
	MaxIntervals
	MaxValueLength
	RowCount
	UniqueValueCount
	PNullUniqueValueCount
	NullCount
	AllNullCount
	HighModeFreq
	PNullHighModeFreq

DBC.IndexStatsV[X]

User Type	Columns Selected
End User	CreateTimeStamp
	LastCollectTimeStamp
	LastAlterTimeStamp

Data Dictionary Views – Teradata 16.20

DBC.Indices[V][X]

User Type	Columns Selected
Database Administrator	DatabaseName
End User	TableName
	IndexNumber
	IndexType
	UniqueFlag
	IndexName
	ColumnName
	ColumnPosition
	CreatorName
	CreateTimeStamp
	LastAlterName
	LastAlterTimeStamp
	IndexMode
	AccessCount
	LastAccessTimeStamp
	UniqueOrPK
	VTConstraintType
	TTConstraintType
	SystemDefinedJI
	IndexDatabaseName
	LDIType
	RowSizeFormat

Data Dictionary Views – Teradata 16.20

DBC.Indices[V][X] (cont.)

User Type	Columns Selected
Database Administrator End User	TimeZero TimeBucketUnit TimeBucketValue TSFlags

DBC.IndexUseCountV[X]

User Type	Columns Selected
Database Administrator End User	DatabaseName TableName IndexName IndexNumber FieldName UsageType AccessCount

Data Dictionary Views – Teradata 16.20

DBC.InsertUseCountV[X]

User Type	Columns Selected
Database Administrator	DatabaseName ObjectName UsageType InsertCount

DBC.InDoubtLog[V]

User Type	Columns Selected
Database Administrator	LogicalHostId SessionNumber CoordTaskId RunUnitId LogonUserName ResolvingUserLogonName CommitOrRollBack UserLogonDate UserLogonTime CompletionDate CompletionTime Options

Data Dictionary Views – Teradata 16.20

DBC.Journals[V][X]

User Type	Columns Selected
Database Administrator End User	Tables_DB TableName Journals_DB JournalName

DBC.JoinIndicesV

User Type	Columns Selected
Database Administrator	DatabaseName TableName JoinIdxDatabaseName JoinIdxName IndexType

Data Dictionary Views – Teradata 16.20

[DBC.LoadTablesInfoV\[X\]](#)

User Type	Columns Selected
Database Administrator	TableName DatabaseName LoadQueryBand SingleSession ControlSessNum ControlHostNum StartTimeStamp

Data Dictionary Views – Teradata 16.20

[DBC.LogOnOff\[V\]\[X\]](#)

User Type	Columns Selected
Database Administrator	LogDate
Security Administrator	LogTime
Supervisor	UserName
	AccountName
	Event
	LogicalHostId
	IFPNo
	SessionNo
	LogonDate
	LogonTime
	LogonSource
	ClientIpAddress
	ClientProgramName
	ClientSystemUserId
	ClientConnectionType
	ClientCoordName
	ClientEnvName
	ClientJobId
	ClientJobName
	ClientOsName

Data Dictionary Views – Teradata 16.20

DBC.LogOnOff[V][X] (cont.)

User Type	Columns Selected
Database Administrator	ClientProcThreadId
Security Administrator	ClientSecProdGrp
Supervisor	ClientSecProdUserId
	ClientTcpPortNumber
	ClientTdHostName
	ClientTerminalId
	ClientTransactionId
	ClientUserOperId
	ClientVmName
	ClientVmUserId
	MechanismName
	ClientTDPReleasId
	ClientCLlv2ReleasId
	ClientSessionDesc
	ClientWorkload
	ClientJobData
	ClientODBCDriverVersion
	ClientNetDataProviderVersion
	ClientODBCDriverManagerVersion
	ClientNetFrameworkVersion

Data Dictionary Views – Teradata 16.20

DBC.LogOnOff[V][X] (cont.)

User Type	Columns Selected
Database Administrator	ClientAttributesEx
Security Administrator	ClientJDBCVersion
Supervisor	ClientJavaVersion
	RecoverableNetworkProtocol
	LogonRedrive
	ClientIPAddrByClient
	ClientPortByClient
	ServerIPAddrByClient
	ServerPortByClient
	ClientIPAddrByUnity
	ClientPortByUnity
	UnityClientSideIPAddr
	UnityClientSidePort
	UnityServerSideIPAddr
	UnityServerSidePort
	ServerIPAddrByUnity
	ServerPortByUnity
	ServerIPAddrByServer
	ServerPortByServer
	ClientCOPSuffixedHostName
	UnitySessNo
	UnityVersion

Data Dictionary Views – Teradata 16.20

DBC.LogOnOff[V][X] (cont.)

User Type	Columns Selected
Database Administrator	UnityAuthMechName
Security Administrator	UnityMechanismName
Supervisor	UserAuthenticatedBy
	ClientTDSessionPoolName
	UnityTcpPortNumber
	UnityIpAddress
	SecurityPolicy
	UnitySecurityPolicy
	Unity_AuthUser
	DirUserNetConfidentiality
	DirUserNetPolicyLevel
	UnityNetConfidentiality
	UnityNetPolicyLevel
	EffectiveSessionNetConf
	EffectiveSessionNetPolicyLevel
	ProxyLogon
	ErrorCode
	ErrorMsg

Data Dictionary Views – Teradata 16.20

[DBC.LogonRules\[V\]](#)

User Type	Columns Selected
Security Administrator	UserName LogicalHostId LogonStatus NullPassword CreatorName CreateTimeStamp

[DBC.MapGrantsV\[X\]](#)

User Type	Columns Selected
Security Administrator	UserOrRoleName MapName GrantOption GranteeKind CreateTimeStamp GrantorName

Data Dictionary Views – Teradata 16.20

DBC.MapsV[X]

User Type	Columns Selected
Database Administrator	MapName
End User	MapSlot
	MapKind
	SystemDefined
	SystemDefault
	ParentMapName
	NumberOfAMPs
	ContiguousStartAmp
	ContiguousEndAmp
	CreatorName
	ZoneName
	CreateTimeStamp
	CommentString
	FallbackKind
	TotalNumberOfAMPs

Data Dictionary Views – Teradata 16.20

DBC.MultiColumnStatsV

User Type	Columns Selected
Database Administrator	DatabaseName
End User	TableName
	ColumnName
	StatsName
	StatsSource
	ValidStats
	DBSVersion
	IndexNumber
	SampleSignature
	SampleSizePct
	ThresholdSignature
	MaxIntervals
	MaxValueLength
	RowCount
	UniqueValueCount
	PNullUniqueValueCount
	NullCount
	AllNullCount
	HighModeFreq
	PNullHighModeFreq

DBC.MultiColumnStatsV

User Type	Columns Selected
End User	CreateTimeStamp
	LastCollectTimeStamp
	LastAlterTimeStamp

Data Dictionary Views – Teradata 16.20

DBC.MultiExpStatsV

User Type	Columns Selected
Database Administrator	DatabaseName
End User	TableName
	ColumnName
	StatsName
	StatsSource
	ValidStats
	DBSVersion
	IndexNumber
	SampleSignature
	SampleSizePct
	ThresholdSignature
	MaxIntervals
	MaxValueLength
	RowCount
	UniqueValueCount
	PNullUniqueValueCount
	NullCount
	AllNullCount
	HighModeFreq
	PNullHighModeFreq

DBC.MultiExpStatsV (cont.)

User Type	Columns Selected
End User	CreateTimeStamp
	LastCollectTimeStamp
	LastAlterTimeStamp

Data Dictionary Views – Teradata 16.20

DBC.ObjectUseCountV[X]

User Type	Columns Selected
Database Administrator End User	DatabaseName ObjectName UsageType AccessCount

DBC.PartitioningConstraintsV[X]

User Type	Columns Selected
Database Administrator End User	DatabaseName TableName IndexName IndexNumber ConstraintType ConstraintText ConstraintCollation CollationName CreatorName CreateTimeStamp CharSetID SessionMode ResolvedCurrent_Date ResolvedCurrent_TimeStamp DefinedCombinedPartitions MaxCombinedPartitions PartitioningLevels ColumnPartitioningLevel

Data Dictionary Views – Teradata 16.20

DBC.PeriodsV[X]

User Type	Columns Selected
All Users	DatabaseName Table_Name Period_Name Fid Tid DBid START_COLUMN_NAME END_COLUMN_NAME

DBC.ProfileAsgdSecConstraintsV[X]

User Type	Columns Selected
All Users	ProfileName ConstraintName ValueName IsDefault Assignor

Data Dictionary Views – Teradata 16.20

DBC.ProfileInfo[V][X]

User Type	Columns Selected
Database Administrator	ProfileName
End User	DefaultAccount
Security Administrator	DefaultDB
Supervisor	SpoolSpace
	TempSpace
	ExpirePassword
	PasswordMinChar
	PassordMaxChar
	PasswordDigits
	PasswordSpecChar
	PasswordRestrictWords
	MaxLogonAttempts
	LockedUserExpire
	PasswordReuse
	Queryband
	QuerybandDefault
	CommentString
	CreatorName
	CreateTimeStamp
	LastAlterName
	LastAlterTimeStamp

DBC.ProfileInfo[V][X]

User Type	Columns Selected
Supervisor	IgnoreQuerybandValues
	DefaultMapName
	MapOverride

Data Dictionary Views – Teradata 16.20

[DBC.QryLockLogXMLV](#)

User Type	Columns Selected
Database Administrator	ProdID CollectTimeStamp QueryID XMLTextInfo

Data Dictionary Views – Teradata 16.20

DBC.QryLog[V]

User Type	Columns Selected
Database Administrator	ProcID CollectTimeStamp QueryID UserID UserName DefaultDatabase AcctString ExpandAccString SessionID LogicalHostID RequestNum InternalRequestNum LogonDateTime AccStringTime AccStringHour AccStringDate LogonSource AppID ClientID ClientAddr QueryBand ProfileID StartTime FirstStepTime FirstRespTime

DBC.QryLog[V] (cont.)

User Type	Columns Selected
Database Administrator	ElapsedTime NumSteps NumStepswPar MaxStepsInPar NumResultRows TotalIOCount AMPCPUTime ParserCPUTime UtilityRowCount ErrorCode ErrorText WarningOnly AbortFlag CacheFlag StatementType StatementGroup QueryText NumOfActiveAMPs MaxAMPCPUTime MaxCPUAmpNumber MinAmpCPUTime MaxAmpIO MaxIOAmpNumber MinAmpIO SpoolUsage

Data Dictionary Views – Teradata 16.20

DBC.QryLog[V] (cont.)

User Type	Columns Selected
Database Administrator	LSN EstResultRows EstProcTime EstMaxRowCount AMPCPUTimeNorm ParserCPUTimeNorm MaxAMPCPUTimeNorm MaxCPUAmpNumberNorm MinAmpCPUTimeNorm ParserExpReq ProxyUser ProxyRole SessionTemporalQualifer CalendarName CPUDecayLevel IODecayLevel TacticalCPUException TacticalIOException SeqRespTime ReqIOKB ReqPhysIO ReqPhysIOKB DataCollectAlg

DBC.QryLog[V] (cont.)

User Type	Columns Selected
Database Administrator	CallNestingLevel NumRequestCtx KeepFlag QueryRedriven ReDriveKind LastRespTime DisCPUTime Statements DisCPUTimeNorm TxnMode RequestMode DBQLStatus NumFragments VHLogicalIO VHPhysIO VHLogicalIOKB VHPhysIOKB LockDelay TxnUniq LockLevel CheckpointNum UnityTime

Data Dictionary Views – Teradata 16.20

DBC.QryLog[V] (cont.)

User Type	Columns Selected
Database Administrator	UtilityInfoAvailable
	UnitySQL
	ThrottleBypassed
	IterationCount
	TTGranularity
	MaxStepMemory
	TotalServerByteCount
	PersistentSpool
	RemoteQuery
	ProxyUserID
	DelayTime
	MinRespHoldTime
	TotalFirstRespTime
	TDWMEstMemUsage
	NumResultOneMBRows
	MaxOneMBRowSize
	UtilityByteCount
	EstMaxStepTime
	FlexThrottle
	ParamQuery

DBC.QryLog[V] (cont.)

User Type	Columns Selected
Database Administrator	ProfileName
	WDName
	MaxNumMapAMPs
	MinNumMapAMPs
	SysDefNumMapAMPs
	UsedIota
	ImpactSpool
	TDWMMSRCount

Data Dictionary Views – Teradata 16.20

[DBC.QryLogEventHis\[V\]](#)

User Type	Columns Selected
Database Administrator	ProcID CollectTimeStamp EntryTS EntryKind EntryID EntryType EntryName EventValue LastValue Activity ActivityId ActivityName ConfigId SeqNo Spare1 Spare2 Spare3 FlexThrottleID FlexThrottleSeqNo

Data Dictionary Views – Teradata 16.20

[DBC.QryLogEvents\[V\]](#)

User Type	Columns Selected
Database Administrator	ProcID CollectTimeStamp SessionID LogicalHostID WDID OpEnvID SysConID EventTime EventCode EventSubCode EventInfo FlexThrottleID FlexThrottleSeqNo

Data Dictionary Views – Teradata 16.20

[DBC.QryLogExceptions\[V\]](#)

User Type	Columns Selected
Database Administrator	ProcID CollectTimeStamp QueryID UserName SessionID RequestNum LogicalHostID AcctString WDID OpEnvID SysConID ClassificationTime ExceptionTime ExceptionValue ExceptionAction NewWDID ExceptionCode ExceptionSubCode ErrorText ExtraInfo RuleID WarningOnly RejectionCat

Data Dictionary Views – Teradata 16.20

DBC.QryLogExplain[V]

User Type	Columns Selected
Database Administrator	ProcID CollectTimeStamp QueryID ExpRowNo ExplainText

DBC.QryLogExplainDocV

User Type	Columns Selected
Database Administrator	ProcID CollectTimeStamp QueryID ExplainTextDoc

Data Dictionary Views – Teradata 16.20

[DBC.QryLogObjects\[V\]](#)

User Type	Columns Selected
Database Administrator	ProcID CollectTimeStamp QueryID ObjectDatabaseName ObjectTableName ObjectColumnName ObjectID ObjectNum ObjectType FreqofUse TypeOfUse

Data Dictionary Views – Teradata 16.20

DBC.QryLogParam[V]

User Type	Columns Selected
Database Administrator	QueryID RowNum ParamJSON

DBC.QryLogParamJSON

User Type	Columns Selected
Database Administrator	QueryID RowNum ParamJSON

Data Dictionary Views – Teradata 16.20

DBC.QryLogSQLDocV

User Type	Columns Selected
Database Administrator	ProcID CollectTimeStamp QueryID SqlTextDoc

DBC.QryLogSQL[V]

User Type	Columns Selected
Database Administrator	ProcID CollectTimeStamp QueryID SqlRowNo SqlTextInfo

Data Dictionary Views – Teradata 16.20

DBC.QryLogSteps[V]

User Type	Columns Selected
DB Administrator	ProcID CollectTimestamp QueryID StepLev1Num StepLev2Num StepName StepStartTime StepStopTime ElapsedTime EstProcTime EstCPUCost CPUtime IOcount EstRowCount EstRowCountSkew EstRowCountSkewMatch RowCount RowCount2 RowCount3 NumOfActiveAMPs MaxAmpCPUTime

DBC.QryLogSteps[V] (cont.)

User Type	Columns Selected
DB Administrator	MaxCPUAmpNumber MinAmpCPUTime MaxAmpIO MaxIOAmpNumber MinAmpIO SpoolUsage MaxAMPSPool MaxSpoolAmpNumber MinAMPSPool StepWD LSN UtilityTableId RowsWComprColumns EstIOCost EstNetCost EstHRCost CPUTimeNorm MaxAmpCPUTimeNorm MaxCPUAmpNumberNorm MinAmpCPUTimeNorm NumCombinedPartitions

Data Dictionary Views – Teradata 16.20

DBC.QryLogSteps[V] (cont.)

User Type	Columns Selected
DB Administrator	StepInstance StepStatus DispatchSeq StatementNum TriggerNestLevel TriggerKind FragmentNum IOKB VHLogicalIO VHPhysIO VHLogicalIOKB VHPhysIOKB PhysIO PhysIOKB LockDelay SSRReceiverCount DMLLoadId ServerByteCount PersistentSpool NumContexts NumCPReferences

DBC.QryLogSteps[V] (cont.)

User Type	Columns Selected
DB Administrator	OneMBRowCount MaxOneMBRowSize MaxNumMapAMPs MinNumMapAMPs SysDefNumMapAMPs SourceMapNo DestMapNo StepObjectInfo UsedIota PermUsage MaxAMPPerm MaxPermAmpNumber SpaceDelay MaxSpaceDelayAmpNumber InMemBulkQualStatus

Data Dictionary Views – Teradata 16.20

DBC.QryLogSummary[V]

User Type	Columns Selected
Database Administrator	ProcID CollectTimeStamp UserID AcctString LogicalHostID ApplID ClientID ClientAddr ProfileID SessionID QueryCount ValueType QuerySeconds AverageTime TotalIOCount AverageIO AMPCPUTime AverageAmpCPU ParserCPUTime AverageParserCPU

DBC.QryLogSummary[V] (cont.)

User Type	Columns Selected
DB Administrator	AMPCPUTimeNorm AverageAmpCPUNorm ParserCPUTimeNorm AverageParserCPUNorm LowHist HighHist UsedIota ReqPhysIO ReqPhysIOKB StartTime StopTime UserName ProfileName

Data Dictionary Views – Teradata 16.20

DBC.QryLogTDWM[V] (cont.)

User Type	Columns Selected
DB Administrator	ExceptionValue FinalWDID TDWMEstMaxRows TDWMEstLastRows TDWMEstTotalTime TDWMEstMemUsage TDWMAAllAmpFlag TDWMConfLevelUsed TDWMRuleID StatementGroup SessionWDID MinRespHoldTime TotalFirstRespTime MaxNumMapAMPs MinNumMapAMPs SysDefNumMapAMPs ProfileName WDName FinalWDName SessionWDName OpEnvName

DBC.QryLogTDWM[V]

User Type	Columns Selected
DB Administrator	SysConName ProcID CollectTimeStamp QueryID UserID UserName DefaultDatabase AcctString SessionID LogicalHostID RequestNum InternalRequestNum LastStateChange DelayTime WDID OpEnvID SysConID LSN NoClassification WDOVERRIDE ResponseTimeMet

Data Dictionary Views – Teradata 16.20

DBC.QryLogTDWMSum[V]

User Type	Columns Selected
DB Administrator	ProcID CollectTimeStamp WDID OpEnvID SysConID StartColTime Arrivals ActiveCount Completions MinRespTime MaxRespTime AvgRespTime MinCPUTime MaxCPUTime AvgCPUTime DelayedCount AvgDelayTime ExceptionAbCount ExceptionMvCount ExceptionCoCount

DBC.QryLogTDWMSum[V] (cont.)

User Type	Columns Selected
DB Administrator	ExceptionCount MetSLGCount AbortCount ErrorCount RejectedCount MovedInCount IntervalDelayCnt DelayedQueries OtherCount VirtualPartNum AvgIOWaitTime MaxIOWaitTime AvgOtherWaitTime MaxOtherWaitTime AvgCPURunDelay MaxCPURunDelay AvgSeqRespTime MaxSeqRespTime AvgLogicalIO MaxLogicalIO

DBC.QryLogTDWMSum[V] (cont.)

User Type	Columns Selected
DB Administrator	AvgLogicalKBs MaxLogicalKBs AvgPhysicalIO MaxPhysicalIO AvgPhysicalKBs MaxPhysicalKBs ThrottleBypassed FlexActive FlexComplete FlexArrivals

Data Dictionary Views – Teradata 16.20

DBC.QryLogUtilityV

User Type	Columns Selected
DB Administrator	ProcID CollectTimeStamp UtilityName FastExportNoSpool ExtendedMLoad DSASOperation UtilityRequest JobName JobInstanceID LSN UserName SessionID LogicalHostID LogonDateTime WDID FinalWDID SessionWDID TDWMRuleID CPUDecayLevel IODecayLevel

DBC.QryLogUtilityV (cont.)

User Type	Columns Selected
DB Administrator	UserID ZoneID AcctString ExpandAcctString AcctStringTime AcctStringHour AcctStringDate LogonSource AppID ClientID ClientAddr QueryBand ProfileID ProxyUser ProxyRole OpEnvID SysConID NoClassification WDOVERRIDE ResponseTimeMet

DBC.QryLogUtilityV (cont.)

User Type	Columns Selected
DB Administrator	ResponseTimeMet DelayTime JobStartTime JobEndTime RowsInserted RowsUpdated RowsDeleted RowsExported NumSesOrBuildProc MaxDataWaitTime MaxDataWaitTimeID Phase0StartTime Phase0EndTime Phase0TotalCPUTime Phase0TotalCPUTimeNorm Phase0ParserCPUTime Phase0ParserCPUTimeNorm Phase0TotalIO Phase0IOKB Phase0PhysIO

Data Dictionary Views – Teradata 16.20

DBC.QryLogUtilityV (cont.)

User Type	Columns Selected
DB Administrator	Phase0PhysIO Phase0PhysIOKB Phase1StartTime Phase1EndTime Phase1TotalCPUTime Phase1TotalCPUTimeNorm Phase1MaxCPUTime Phase1MaxCPUAmpNumber Phase1MaxCPUTimeNorm Phase1MaxCPUAmpNumberNorm Phase1ParserCPUTime Phase1ParserCPUTimeNorm Phase1RSGCPUTime Phase1RSGCPUTimeNorm Phase1TotalIO Phase1MaxIO Phase1MaxIOAmpNumber Phase1IOKB Phase1PhysIO Phase1PhysIOKB

DBC.QryLogUtilityV (cont.)

User Type	Columns Selected
DB Administrator	Phase1MaxAWTUsage Phase1MaxAMPMemoryUsage Phase1MaxRSGMemoryUsage Phase1RowCount Phase1BlockCount Phase1ByteCount Phase2StartTime Phase2EndTime Phase2TotalCPUTime Phase2TotalCPUTimeNorm Phase2MaxCPUTime Phase2MaxCPUAmpNumber Phase2MaxCPUTimeNorm Phase2MaxCPUAmpNumberNorm Phase2ParserCPUTime Phase2ParserCPUTimeNorm Phase2RSGCPUTime Phase2RSGCPUTimeNorm Phase2TotalIO Phase2MaxIO

Data Dictionary Views – Teradata 16.20

DBC.QryLogUtilityV (cont.)

User Type	Columns Selected
DB Administrator	Phase2MaxIOAmpNumber Phase2IOKB Phase2PhysIO Phase2PhysIOKB Phase2MaxAWTUsage Phase2MaxAMPMemoryUsage Phase2MaxRSGMemoryUsage Phase2RowCount Phase2BlockCount Phase2ByteCount Phase3StartTime Phase3EndTime Phase3TotalCPUTime Phase3TotalCPUTimeNorm Phase3ParserCPUTime Phase3ParserCPUTimeNorm Phase3RSGCPUTime Phase3RSGCPUTimeNorm Phase3TotalIO Phase3IOKB

DBC.QryLogUtilityV (cont.)

User Type	Columns Selected
DB Administrator	Phase3PhysIO Phase3PhysIOKB Phase3MaxAWTUsage Phase3MaxAMPMemoryUsage Phase3MaxRSGMemoryUsage Phase4StartTime Phase4EndTime Phase4TotalCPUTime Phase4TotalCPUTimeNorm Phase4ParserCPUTime Phase4ParserCPUTimeNorm Phase4TotalIO Phase4IOKB Phase4PhysIO Phase4PhysIOKB Phase0UsedIota Phase1UsedIota Phase2UsedIota Phase3UsedIota Phase4UsedIota

DBC.QryLogUtilityV (cont.)

User Type	Columns Selected
DB Administrator	ProfileName WDName FinalWDName SessionWDName OpEnvName SysConName

Data Dictionary Views – Teradata 16.20

[DBC.QryLogXMLV](#)

User Type	Columns Selected
Database Administrator	ProdID CollectTimeStamp QueryID XMLRowNo XMLTextInfo XMLDocType XMLPlanEnabled StatsUsageEnabled VerboseEnabled DetailedStatsEnabled HasXMLPlanData HasStatsUsageData HasVerboseData HasDetailedStatsData

Data Dictionary Views – Teradata 16.20

[DBC.QryLogXMLDocV](#)

User Type	Columns Selected
Database Administrator	ProdID CollectTimeStamp QueryID XMLTextDoc XMLDocType XMLPlanEnabled StatsUsageEnabled VerboseEnabled DetailedStatsEnabled HasXMLPlanData HasStatsUsageData HasVerboseData HasDetailedStatsData

Data Dictionary Views – Teradata 16.20

DBC.RCC_Media[V][X]

User Type	Columns Selected
Operations Control	EventNum VolSerialId VolSequenceNum DupeDumpSet

DBC.RCC_Configuration[V][X]

User Type	Columns Selected
Operations Control	EventNum LogProcessor PhyProcessor Vproc ProcessorState RestartSeqNum

DBC.QueryStatUseCountV[X]

User Type	Columns Selected
Database Administrator	DatabaseName QueryName AccessCount

Data Dictionary Views – Teradata 16.20

DBC.ReconfigDeleteOrderV

User Type	Columns Selected
All Users	OrderNumber DatabaseName TableName CheckTableOption ProcessOffline

Data Dictionary Views – Teradata 16.20

DBC.ReconfigInfoV

User Type	Columns Selected
All Users	ReconfigId Description ReconfigType BeginTimeStamp EndTimeStamp BeginRedistTimeStamp EndRedistTimeStamp BeginDelTimeStamp EndDelTimeStamp Status CurrByteCount CurrTabRedistCount CurrTabDeleteCount EstRemainRedistSecs EstRemainDeleteSecs AddAmpCount DelAmpCount MovAmpCount ModAmpCount NodeCount

DBC.ReconfigInfoV (cont.)

User Type	Columns Selected
All Users	TotTaskCount TotTableCount TotByteCount TotCatchUpByteCount TotJournalByteCount ActualRedistSecs ActualDeleteSecs EstRedistSecs EstDeleteSecs BeginCalcHBTimeStamp EndCalcHBTimeStamp BeginWrSpaceTimeStamp EndWrSpaceTimeStamp BeginWrCfgTimeStamp EndWrCfgTimeStamp BeginWrCfgNewTimeStamp EndWrCfgNewTimeStamp BeginWrBkupldTimeStamp EndWrBkupldTimeStamp BeginInsUpdMapRowTimeStamp

DBC.ReconfigInfoV (cont.)

User Type	Columns Selected
All Users	EndInsUpdMapRowTimeStamp BeginWrBMTimestamp EndWrBMTimestamp BeginVProcCfgTimeStamp EndVProcCfgTimeStamp

Data Dictionary Views – Teradata 16.20

[DBC.ReconfigRedistOrderV](#)

User Type	Columns Selected
All Users	OrderNumber DatabaseName TableName ProcessOffline

Data Dictionary Views – Teradata 16.20

DBC.ReconfigTableStatsV

User Type	Columns Selected
All Users	ReconfigId Phase TableId DatabaseName TableName Status BeginTimeStamp EndTimeStamp TotRowCount TotByteCount TotCPUsecs TotIOCount LowRowCount LowRowCountAmp HighRowCount HighRowCountAmp LowByteCount LowByteCountAmp HighByteCount HighByteCountAmp LowCPUsecsCount

DBC.ReconfigTableStatsV (cont.)

User Type	Columns Selected
All Users	LowCPUsecsCountAmp HighCPUsecsCount HighCPUsecsCountAmp LowIOCount LowIOCountAmp HighIOCount HighIOCountAmp NUSICount FallBackFlag DBlockSize SortFlag ActualRedistSecs ActualDeleteSecs EstRedistSecs EstDeleteSecs FSGIOCount FSysReadCount FSysWriteCount FSysMiscCount MsgRcvCount MsgSendCount

DBC.ReconfigTableStatsV (cont.)

User Type	Columns Selected
All Users	MsgMiscCount MsgWaitRcvTime MsgWaitSendTime MsgWaitMiscTime NoMemFlushCount CkptFlushCount CheckTableError

Data Dictionary Views – Teradata 16.20

DBC.RepTables[V][X]

User Type	Columns Selected
All users	GroupName TableName

DBC.RepCaptureRulesV

User Type	Columns Selected
All users	RuleSetName GroupName ObjectKind DefaultOpt LikePattern LikeEscape NotLikePattern NotLikeEscape

Data Dictionary Views – Teradata 16.20

[DBC.RestrictedWords\[V\]](#)

User Type	Columns Selected
All users	RestrictedWord

[DBC.ResolvedDTS\[V\]\[X\]](#)

User Type	Columns Selected
All users	ResolvedCurrent_Date ResolvedCurrent_TimeStamp TableName DatabaseName

Data Dictionary Views – Teradata 16.20

[DBC.RI_Child_Tables\[V\]\[X\]](#)

User Type	Columns Selected
All users	IndexID IndexName ChildDbID ChildTID ChildKeyFID ParentDbID ParentTID ParentKeyFID InconsistencyFlag CreatorName CreateTimeStamp VTFKType TTFKType VTPKType TTPKType

Data Dictionary Views – Teradata 16.20

[DBC.RI_Distinct_Children\[V\]\[X\]](#)

User Type	Columns Selected
All users	IndexID IndexName ChildDB ChildTable ParentDB ParentTable InconsistencyFlag CreatorName CreateTimeStamp

Data Dictionary Views – Teradata 16.20

[DBC.RI_Distinct_Parents\[V\]\[X\]](#)

User Type	Columns Selected
All users	IndexID IndexName ParentDB ParentTable ChildDB ChildTable InconsistencyFlag CreatorName CreateTimeStamp

Data Dictionary Views – Teradata 16.20

[DBC.RI_Parent_Tables\[V\]\[X\]](#)

User Type	Columns Selected
All users	IndexID IndexName ParentDbID ParentTID ParentKeyFID ChildDbID ChildTID ChildKeyFID InconsistencyFlag CreatorName CreateTimeStamp VTPKType TTPKType VTFKType TTFKType

Data Dictionary Views – Teradata 16.20

DBC.RoleMembers[V][X]

User Type	Columns Selected
Database Administrator End User	RoleName Grantee GranteeKind Grantor WhenGranted DefaultRole WithAdmin

DBC.RoleInfo[V][X]

User Type	Columns Selected
Database Administrator End User	RoleName CreatorName CommentString CreateTimeStamp ExtRole

Data Dictionary Views – Teradata 16.20

DBC.SecConstraintsV[X]

User Type	Columns Selected
All Users	ConstraintName DataType Nullable SizeInBytes AssigneeCount Creator CreateTimeStamp

DBC.SecurityDefaults[V]

User Type	Columns Selected
Security Administrator	ExpirePassword PasswordMinChar PasswordMaxChar PasswordDigits PasswordSpecChar PasswordRestrictWords MaxLogonAttempts LockedUserExpire PasswordReUse

Data Dictionary Views – Teradata 16.20

[DBC.SecurityLog\[V\]\[X\]](#) [Note: Older view – use AccessLogV](#)

User Type	Columns Selected
Security Administrator	LogDate LogTime LogType UserName AccountName DatabaseName TableName Text

[DBC.ServerV\[X\]](#)

User Type	Columns Selected
Database Administrator	ServerID DataBaseName ServerName CreatorName CreateTimeStamp LastAlterName LastAlterTimeStamp AuthorizationName AuthorizationType ExecMapName ExecMapColocName

Data Dictionary Views – Teradata 16.20

DBC.SessionInfo[V][X]

User Type	Columns Selected
Database Administrator	UserName
End User	AccountName
Security Administrator	SessionNo
Supervisor	DefaultDataBase
	IFPNo
	Partition
	LogicalHostId
	HostNo
	CurrentCollation
	LogonDate
	LogonTime
	LogonSequenceNo
	LogonSource
	ExpiredPassword
	TwoPCMode
	Transaction_Mode
	CurrentRole
	ProfileName
	LogonAcct
	LDAP
	AuditTrailID

DBC.SessionInfo[V][X] (cont.)

User Type	Columns Selected
Database Administrator	CurlIsolationLevel
End User	QueryBand
Security Administrator	ProxyUser
Supervisor	ProxyCurRole
	TemporalQualifier
	CalendarName
	ExtendedLogonSource
	ClientIpAddress
	ClientProgramName
	ClientSystemUserId
	ClientConnectionType
	ClientCoordName
	ClientEnvName
	ClientJobId
	ClientJobName
	ClientOsName
	ClientProcThreadId
	ClientSecProdGrp
	ClientSecProdUserId
	ClientTcpPortNumber
	ClientTdHostName

Data Dictionary Views – Teradata 16.20

DBC.SessionInfo[V][X] (cont.)

User Type	Columns Selected
Database Administrator	ClientTerminalId
End User	ClientTransactionId
Security Administrator	ClientUserOperId
Supervisor	ClientVmName
	ClientVmUserId
	MechanismName
	ClientTDPReleaseId
	ClientCLv2ReleaseId
	ClientSessionDesc
	ClientWorkload
	ClientJobData
	ClientODBCDriverVersion
	ClientNetDataProviderVersion
	ClientODBCDriverManagerVersion
	ClientNetFrameworkVersion
	ClientAttributesEx
	ClientJDBCDriverVersion
	ClientJavaVersion
	ExportDefinitionName
	ExportWidthRuleSet
	RecoverableNetworkProtocol

DBC.SessionInfo[V][X] (cont.)

User Type	Columns Selected
Database Administrator	LogonRedrive
End User	ClientIPAddrByClient
Security Administrator	ClientPortByClient
Supervisor	ServerIPAddrByClient
	ServerPortByClient
	ClientIPAddrByUnity
	ClientPortByUnity
	UnityClientSideIPAddr
	UnityClientSidePort
	UnityServerSideIPAddr
	UnityServerSidePort
	ServerIPAddrByUnity
	ServerPortByUnity
	ServerIPAddrByServer
	ServerPortByServer
	ClientCOPSuffixedHostName
	UnitySessNo
	UnityVersion
	UnityAuthMechName
	UnityMechanismName
	UserAuthenticatedBy

Data Dictionary Views – Teradata 16.20

DBC.SessionInfo[V][X] (cont.)

User Type	Columns Selected
Database Administrator	ClientTDSessionPoolName
End User	UnityTcpPortNumber
Security Administrator	UnityIpAddress
Supervisor	TTGranularity
	LoadingOp
	UnicodePassThrough
	DirUserNetConfidentiality
	DirUserNetPolicyLevel
	UnityNetConfidentiality
	UnityNetPolicyLevel
	EffectiveSessionNetConf
	EffectiveSessionNetPolicyLevel
	ProxyLogon

Data Dictionary Views – Teradata 16.20

DBC.ShowColChecks[V][X]

User Type	Columns Selected
Database Administrator Supervisor	DatabaseName TableName ColumnName ColCheck CreatorName CreateTimeStamp TimeDimension VTCheckType TTCheckType

DBC.ShowTblChecks[V][X]

User Type	Columns Selected
Database Administrator Supervisor	DatabaseName TableName CheckName TblCheck CreatorName CreateTimeStamp VTCheckType TTCheckType

Data Dictionary Views – Teradata 16.20

DBC.Software_Event_Log[V]

User Type	Columns Selected
Operations Control	TheDate TheTime Event_Tag Category Severity PMA Vproc Partition Task TheFunction SW_Version Line Text

DBC.SparseMapAmpsV

User Type	Columns Selected
All Users	DatabaseName ObjectName ObjectKind AmpNo PF

Data Dictionary Views – Teradata 16.20

DBC.StatsV

User Type	Columns Selected
Database Administrator End User	DatabaseName TableName ColumnName FieldIdList StatsName ExpressionCount StatsId StatsType StatsSource ValidStats DBSVersion IndexNumber SampleSignature SampleSizePct ThresholdSignature MaxIntervals MaxValueLength RowCount UniqueValueCount PNullUniqueValueCount NullCount

DBC.StatsV (cont.)

User Type	Columns Selected
Database Administrator End User	AllNullCount HighModeFreq PNullHighModeFreq StatsSkipCount CreateTimeStamp LastCollectTimeStamp LastAlterTimeStamp BLCCompRatio

Data Dictionary Views – Teradata 16.20

[DBC.StatUseCountV\[X\]](#)

User Type	Columns Selected
Database Administrator	DatabaseName
End User	TableName
	StatName
	LastAccessTimeStamp
	AccessCount

Data Dictionary Views – Teradata 16.20

[DBC.Table_LevelConstraints\[V\]\[X\]](#)

User Type	Columns Selected
Database Administrator	DatabaseName
End User	TableName
	ConstraintName
	ConstraintText
	CreatorName
	CreateTimeStamp
	VTCheckType
	TTCheckType

Data Dictionary Views – Teradata 16.20

DBC.TableStatsV

User Type	Columns Selected
Database Administrator End User	DatabaseName TableName ColumnName StatsName StatsSource ValidStats DBSVersion IndexNumber SampleSignature SampleSizePct ThresholdSignature MaxIntervals MaxValueLength RowCount UniqueValueCount PNullUniqueValueCount NullCount AllNullCount HighModeFreq PNullHighModeFreq StatsSkipCount CreateTimeStamp

DBC.TableStatsV (cont.)

User Type	Columns Selected
Database Administrator End User	CreateTimeStamp LastCollectTimeStamp LastAlterTimeStamp BLCCompRatio

Data Dictionary Views – Teradata 16.20

DBC.Tables[V][X]

User Type	Columns Selected
DB Administrator End User	DatabaseName
	TableName
	Version
	TableKind
	ProtectionType
	JournalFlag
	CreatorName
	RequestText
	CommentString
	ParentCount
	ChildCount
	NamedTblCheckCount
	UnnamedTblCheckExist
	PrimaryKeyIndexId
	RepStatus
	CreateTimeStamp
	LastAlterName
	LastAlterTimeStamp
	RequestTxtOverflow
	AccessCount
	LastAccessTimeStamp

DBC.Tables[V][X] (cont.)

User Type	Columns Selected
DB Administrator End User	UtilVersion
	QueueFlag
	CommitOpt
	TransLog
	CheckOpt
	TemporalProperty
	ResolvedCurrent_Date
	ResolvedCurrent_Timestamp
	SystemDefinedJI
	VTQualifier
	TTQualifier
	PIColumnCount
	PartitioningLevels
	LoadProperty
	CurrentLoadId
	LoadIdLayout
	DelayedJI
	LastArchiveId
	LastFullArchiveId
	BlockSize
	FreeSpacePercent

DBC.Tables[V][X]] (cont.)

User Type	Columns Selected
DD Administrator End User	MergeBlockRatio
	Checksum
	BlockCompression
	BlockCompressionAlgorithm
	BlockCompressionLevel
	TableHeaderFormat
	RowSizeFormat
	MapName
	ColocationName
	TVMFlavor

Data Dictionary Views – Teradata 16.20

DBC.Tables2[V][X]

User Type	Columns Selected
Database Administrator Supervisor	TVMName TVMId Databaseld ParentCount ChildCount

DBC.Tables3VX

User Type	Columns Selected
Database Administrator Supervisor	DatabaseName TableName FieldName TableId FieldId

Data Dictionary Views – Teradata 16.20

DBC.TableText[V][X]

User Type	Columns Selected
Database Administrator End User	DatabaseName TableName TableKind RequestText LineNo

DBC.TableSize[V][X]

User Type	Columns Selected
Database Administrator End User	Vproc DatabaseName AccountName TableName CurrentPerm PeakPerm

Data Dictionary Views – Teradata 16.20

DBC.TblSrvV[X]

User Type	Columns Selected
Database Administrator End User	ServerName SrvDataBaseName TblOpName TblOpDBName TableOperatorType

DBC.TblSrvInfoV[X]

User Type	Columns Selected
Database Administrator End User	ServerName SrvDataBaseName TblOpName TblOpDataBaseName NameInfo ValueInfo ValueType TableOperatorType

Data Dictionary Views – Teradata 16.20

DBC.TempTableStatsV

User Type	Columns Selected
Database Administrator End User	DatabaseName TableName ColumnName StatsName StatsSource ValidStats DBSVersion IndexNumber SampleSignature SampleSizePct ThresholdSignature MaxIntervals MaxValueLength RowCount UniqueValueCount PNullUniqueValueCount NullCount AllNullCount HighModeFreq PNullHighModeFreq CreateTimeStamp

DBC.TempTableStatsV (cont.)

User Type	Columns Selected
Database Administrator End User	LastCollectTimeStamp LastAlterTimeStamp BLCCompRatio

Data Dictionary Views – Teradata 16.20

[DBC.Triggers\[V\]\[X\]](#)

User Type	Columns Selected
Database Administrator	DatabaseName SubjectTableDataBaseName TableName TriggerName EnabledFlag ActionTime Event Kind OrderNumber TriggerComment RequestText CreatorName CreateTimeStamp LastAlterName LastAlterTimeStamp AccessCount LastAccessTimeStamp CreateTxtOverflow VTEventType TTEventType

Data Dictionary Views – Teradata 16.20

DBC.UDTInfoV

User Type	Columns Selected
All Users	TypeId DatabaseId TypeName TypeKind INSTANTIABLE FINAL Encryption Compression OperatorAll DefaultTransformGroup OrderingForm OrderingCategory OrderingRoutineID CastCount ExtFileReference DefaultNull ArrayNumDimensions ArrayScope

Data Dictionary Views – Teradata 16.20

DBC.UDTTransformV

User Type	Columns Selected
All Users	TypeName GroupName ToSQLRoutineId ToSQLDatabaseName ToSQLRoutineName ToSQLSpecificName ToSQLParameterDataType FromSQLRoutineId FromSQLDatabaseName FromSQLRoutineName FromSQLSpecificName FromSQLReturnDataType

Data Dictionary Views – Teradata 16.20

[DBC.UsrAsgdSecConstraintsV\[X\]](#)

User Type	Columns Selected
All Users	UserName ConstraintName ValueName IsDefault Assignor

[DBC.User_Default_Journals\[V\]\[X\]](#)

User Type	Columns Selected
End User	UserName Journal_DB JournalName

[DBC.UpdateUseCountV\[X\]](#)

User Type	Columns Selected
All Users	DatabaseName ObjectName FieldName UsageType UpdateCount

Data Dictionary Views – Teradata 16.20

[DBC.UserGrantedRights\[V\]](#)

User Type	Columns Selected
End User	DatabaseName TableName ColumnName Grantee AccessRight GrantAuthority AllnessFlag CreatorName CreateTimeStamp

Data Dictionary Views – Teradata 16.20

DBC.UserRoleRights[V]

User Type	Columns Selected
End User	RoleName DatabaseName TableName ColumnName AccessRight GrantorName CreateTimeStamp

DBC.UserRights[V]

User Type	Columns Selected
End User	DatabaseName TableName ColumnName AccessRight GrantAuthority GrantorName CreatorName CreateTimeStamp

Data Dictionary Views – Teradata 16.20

DBC.Users[V]

User Type	Columns Selected
DB Administrator	UserName
End User	CreatorName
Supervisor	PasswordLastModDate
	PasswordLasModTlme
	OwnerName
	PermSpace
	SpoolSpace
	TempSpace
	ProtectionType
	JournalFlag
	StartupString
	DefaultAccount
	DefaultDataBase
	CommentString
	DefaultCollation
	PasswordChgDate
	LockedDate
	LockedTime
	LockedCount

DBC.Users[V] (cont.)

User Type	Columns Selected
DB Administrator	TimeZoneHour
End User	TimeZoneMinute
Supervisor	DefaultDateForm
	CreateTimeStamp
	LastAlterName
	LastAlterTimeStamp
	DefaultCharType
	RoleName
	ProfileName
	AccessCount
	LastAccessTimeStamp
	ExportDefinitionName
	ExportWidthRuleSet
	DBA
	DefaultMapName
	MapOverride

Data Dictionary Views – Teradata 16.20

[DBC.ZonesV\[X\]](#)

User Type	Columns Selected
Security Administrator	ZoneName RootName RootType ZoneDBAName CreatorName CreateTimeStamp

[DBC.ZoneGuestsV\[X\]](#)

User Type	Columns Selected
Security Administrator	ZoneName GuestName GuestKind GrantorName GrantedTimeStamp



Appendix F: Permanent Journals

Teradata Vantage Administration

Copyright © 2007–2023 by Teradata. All Rights Reserved.

Objectives

After completing this module, you will be able to:

- Describe journaling options and the type of recovery each option provides.
- Determine when to use permanent journals instead of (or in addition to) fallback to provide data integrity.
- Create, modify and delete permanent journals for databases and tables.
- Use the DBC.Journals view to associate tables with specific permanent journals.
- Analyze the efficiency of backup procedures after both drives in a mirrored pair have failed.

Automatic Data Protection Mechanisms (Review)

- **Transient Journal**
 - Takes before-image snapshot (UNDO row) of row before change is made
 - Copies before-image of row back to table if transaction fails
 - Maintained within the WAL Log
- **Down AMP Recovery Journal**
 - Automatically used for fallback tables when an AMP is down
 - Other AMP in the cluster identifies rows that have changed for a down AMP
- **Fallback Protection**
 - Optional data protection feature for a table
 - Creates copy of each row on a fallback AMP
- **RAID 1, RAID 6**
 - Provides data redundancy through RAID 1 (disk mirroring) or RAID 6 (dual data parity protection)
 - Provides protection from physical disk failure
- **Cliques**
 - Group of nodes where vproc migration can occur to another node in clique or a HSN
 - Provides protection from node failure(s)

Transient Journal

The Transient journal maintains snapshots of rows in tables before you or other users make changes to them. If the transaction fails or if you abort the request, the Transient Journal copies its snapshot into the existing table which rolls back any changes the failed transaction may have made to the table.

Down AMP Recovery Journal

The Down AMP Recovery Journal supports fallback protection. If a primary AMP fails, the fallback feature allows automatic data recovery using the Down AMP Recovery Journal.

Fallback Protection

Fallback protection is an optional data protection feature that you activate with the CREATE or MODIFY commands. Fallback provides data level protection by automatically creating a copy of each row on a fallback AMP.

RAID 1, RAID 6

RAID 1 provides data redundancy through drive mirroring. RAID 6 protects data with a technique called “data parity protection” using 2 parity drives.

Cliques

Group of SMP nodes sharing a common set of disk arrays. If a node fails, vprocs can migrate to other nodes within the clique. Although Teradata will restart, Teradata continues after a node failure.

Permanent Journals

Permanent journals:

- Optional features that can provide protection for software and hardware failures.
- Store committed, uncommitted and aborted changes.
- Users manage journal tables.

Permanent journal options:

- Single before change image: **BEFORE**
 - Captures images before a change is made
 - Protects against software failures
 - Allows rollback to a checkpoint
- Single after-change image: **AFTER**
 - Captures images after a change is made
 - Protects against hardware failures
 - Allows rollforward to a checkpoint
- Dual image: **DUAL BEFORE** or **DUAL AFTER**
 - Maintains two images copies
 - Protects against loss of journals
- Keyword **JOURNAL** with no other keywords capture single before and after images.

The Teradata system offers a manual method called permanent journals that you can use to protect data. The purpose of a permanent journal is to maintain a sequential history of all changes made to the rows of one or more tables. Permanent Journals help protect user data when users commit, rollback, or abort transactions. A permanent journal can capture a snapshot of rows before a change, after a change, or both. Each database or user space can contain only one journal table.

Existing data tables can write to a journal table defined in its parent or to a journal table located in another database or user. Journal tables require permanent space.

You can create permanent journal tables with the CREATE USER/CREATE DATABASE statement or the MODIFY USER/MODIFY DATABASE statement.

Permanent journal tables exist within a database or user space. Only one permanent journal can be assigned to that user or database. The journal may be located in the same database or user as the tables that use the journal or in a different database.

Location of Change Images

The location of journal rows depends on the image type requested (before or after) and the protection type of the journaled tables.

Fallback Tables

<u>Journal Option</u>	<u>Change Image Location</u>
After images	Primary AMP and fallback AMP
Before images	Primary AMP and fallback AMP

- Dual images are always maintained for fallback tables.
- To determine the fallback AMP for a journal row, the fallback hash map is used.

Non-fallback Tables

<u>Journal Option</u>	<u>Change Image Location</u>
After images	Fallback AMP
Before images	Primary AMP
Dual after images	Fallback AMP and primary AMP
Dual before images	Primary AMP and fallback AMP

- For no fallback tables, you may request either single or dual journal images.
- Since most systems (all new systems) have 2-AMP clusters, the fallback and backup AMP is the same AMP in a cluster.
 - With clusters > 2 AMPs, a backup AMP is another AMP in the same cluster as the primary AMP assigned to journal rows.

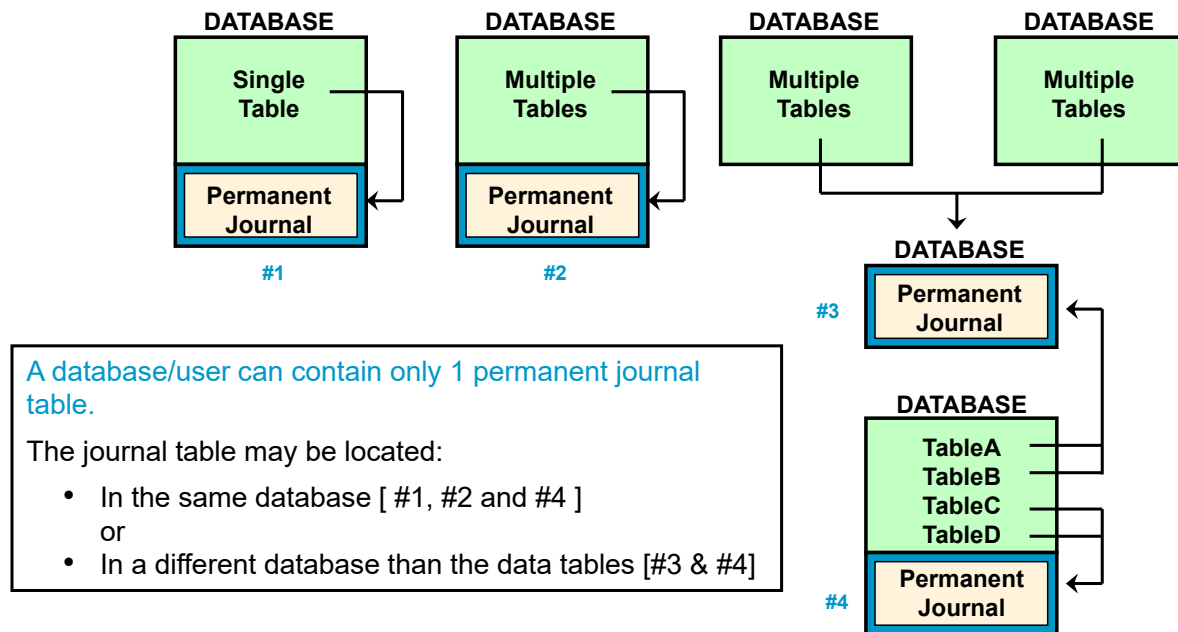
Tables that include fallback and journaling options automatically receive dual image journal protection. Tables with no-fallback protection can request either single or dual permanent journals. The chart on this slide illustrates the location of change-image journals. The placement of permanent journals depends on: requested image type (either before or after) and the protection type (either fallback or non-fallback).

AMP Definitions

The following definitions are used to describe how AMPs are used to store before and/or after images.

Primary AMP	Holds before- and/or after-images for any table with fallback protection. Holds single before images and dual after-images for non-fallback protected tables.
Fallback AMP	Contains before- and/or after-images for tables with fallback protection. The DBC distributes duplicate data rows to fallback processors by assigning the row's hash code to a different AMP in the cluster.
Backup AMP	With most systems, clusters consist of 2 AMPs. Therefore, the Fallback and Backup AMP is the same AMP. For older systems with clusters > 2 AMPs, the Backup AMP holds three types of images for non-fallback tables: single or dual after images, and dual before images. The hashing algorithm is not used for the row distribution. All images for one AMP go to a single backup, which is always in the same cluster. For example, if AMPs 1, 2, 3, and 4 are in the same cluster, 1 backs up 2, 2 backs up 3, 3 backs up 4, and 4 backs up 1. There is no way to predict the backup AMP.

Assigning Tables to a Permanent Journal



When you create a new journal table, there are options you can use to control the type of information the table captures. A permanent journal provides four basic options:

<u>Option</u>	<u>Description</u>
Single Image	Captures/stores one copy of the data.
Dual Image	Captures/stores two separate copies of data: one copy on the primary AMP and one on the fallback AMP or backup AMP.
Before Image	Captures/stores row values before a change occurs.
After Image	Captures/stores row values after a change occurs.

Unlike transient and recovery journals, permanent journal options capture and store all changes whether, committed, uncommitted, or aborted. In addition, journal maintenance and activity are under user control.

Journaling Functions

Journal tables use rollback operations for software failure recoveries. To restore data tables to the state they were in before a software failure; configure a permanent journal to capture before-change information.

Permanent journaling is not a substitute for RAID technology or fallback protection. Both options provide duplicate images of all rows in a table. The journal tables only maintain images for changed rows.

Creating a Permanent Journal

You create permanent journals at the user/database level when you define a new user or database:

```
CREATE DATABASE Payroll_Tab AS PERM = 100E6
      DEFAULT JOURNAL TABLE = Payroll_Jrnl;
```

Or you can create them in an existing user or database:

```
MODIFY DATABASE HR_Tab AS
      DEFAULT JOURNAL TABLE = HR_Jrnl;
```

They are identified in the DD/D as TableKind 'J':

```
SELECT DatabaseName, TableName, TableKind
FROM   DBC.TablesV
WHERE  TableKind = 'J' ;
```

Results:

DatabaseName	TableName	TableKind
Payroll_Tab	Payroll_Jrnl	J
HR_Tab	HR_Jrnl	J

You create permanent journals when you create a user or database. To create permanent journals within an existing user or database, use the MODIFY statement. This slide shows examples of using these statements.

The following restrictions apply to the use of permanent journals:

- If a journal table in another user/database is specified as the default, that other journal table must already exist.
- You can change a DEFAULT JOURNAL for a user or database only if no tables or other databases journal into it.
- Permanent journals are not supported across an AMP configuration change.

Deleting a Permanent Journal

Use the MODIFY USER or MODIFY DATABASE statement to delete a permanent journal. Before you delete the journal, you must use the ALTER TABLE statement to stop the journaling being done to that journal.

```
ALTER [TABLE NAME]
      ,WITH [JOURNAL TABLE=JOURNAL TABLE NAME] ;
      ,NO BEFORE JOURNAL
      ,NO AFTER JOURNAL;
MODIFY DATABASE [DATABASE NAME AS]
      DROP DEFAULT JOURNAL TABLE=[JOURNAL TABLE NAME] ;
```


Assigning a Permanent Journal

Default journal values at the database levels are:

```
{ CREATE USER | CREATE DATABASE } . . .
  [ [ [ NO | DUAL ] [ AFTER | BEFORE ] JOURNAL ] ... ]
  [ DEFAULT JOURNAL TABLE = journal_name ] ;
```

<u>Journal Option</u>	<u>Default</u>
NONE SPECIFIED	NO JOURNAL MAINTAINED
NEITHER AFTER NOR BEFORE	BOTH TYPES IMPLIED
NEITHER DUAL NOR NO	FALLBACK – DUAL IMAGES; NO FALLBACK – SINGLE IMAGES

At the table level, you can indicate journal options with the CREATE statement:

```
CREATE TABLE . . .
  [ [ [ NO | DUAL ] [ AFTER | BEFORE ] JOURNAL ] ... ]
  [ WITH JOURNAL TABLE = journal_name ] ;
```

Default journal values at the table levels are :

<u>Journal Option</u>	<u>Default</u>
NONE SPECIFIED	Defaults to USER/DATABASE
AFTER IMAGE ONLY	Defaults FOR BEFORE IMAGE
BEFORE IMAGE ONLY	Defaults FOR AFTER IMAGE
NEITHER DUAL NOR NO	Defaults to PROTECTION TYPE

[Note:](#) If a database or user that contains a permanent journal runs out of space, all table updates that write to that journal abort.

Permanent journals are optional. You can specify journal options at the database/user level or at the individual table level. You can define a DEFAULT JOURNAL TABLE associated with a user or database. You can associate an individual table within the database with the DEFAULT JOURNAL (by default) or another journal table by specifying that on the CREATE or ALTER TABLE statement.

Users activate permanent journaling by including the JOURNAL option in the CREATE or MODIFY statements for users or databases. The following page illustrates CREATE USER and CREATE TABLE statements that create and assign permanent journals.

If you create a database/user and specify a default journal table, but do not specify any journaling options, the default at the database level is NO BEFORE and NO AFTER journaling for tables created in the database. When creating a table in this database/user and if you want journaling, you must specify the journaling options you want as part of the CREATE TABLE.

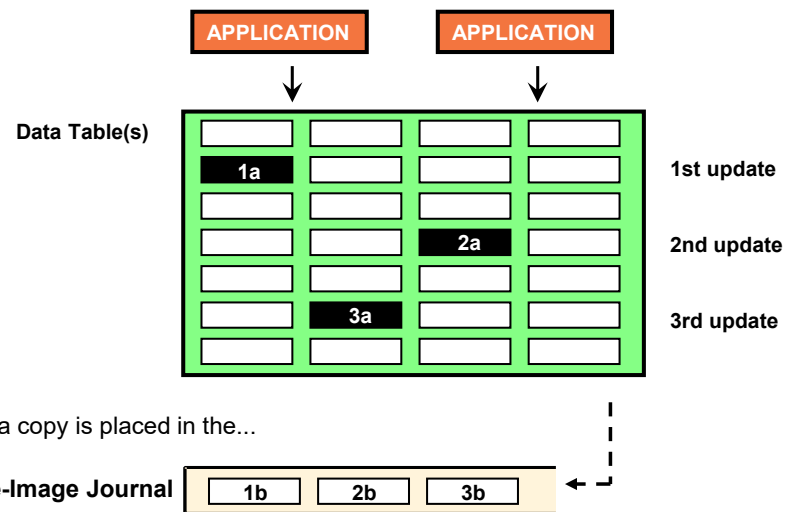
You must allocate sufficient permanent space to a database or user that will contain permanent journals. If a database or user that contains a permanent journal runs out of space, all table updates that write to that journal abort.

Syntax

```
[NO] [BEFORE] [[,] [NO] [AFTER JOURNAL]]
[DUAL] [AFTER] [[,] [DUAL] [BEFORE JOURNAL]]
DEFAULT JOURNAL TABLE = [dbname.] tname
```

Before-Image Journals

Before-images are used to roll back users' changes to one or more tables by returning the data to a previous consistent state.



Before the data is modified, a copy is placed in the...

Before-Image Journal

You can define permanent journals to record:

After Change Images The data in a row after a change has occurred is recorded in the permanent journal.

Before Change Images The data in a row prior to its change is recorded in the permanent journal.

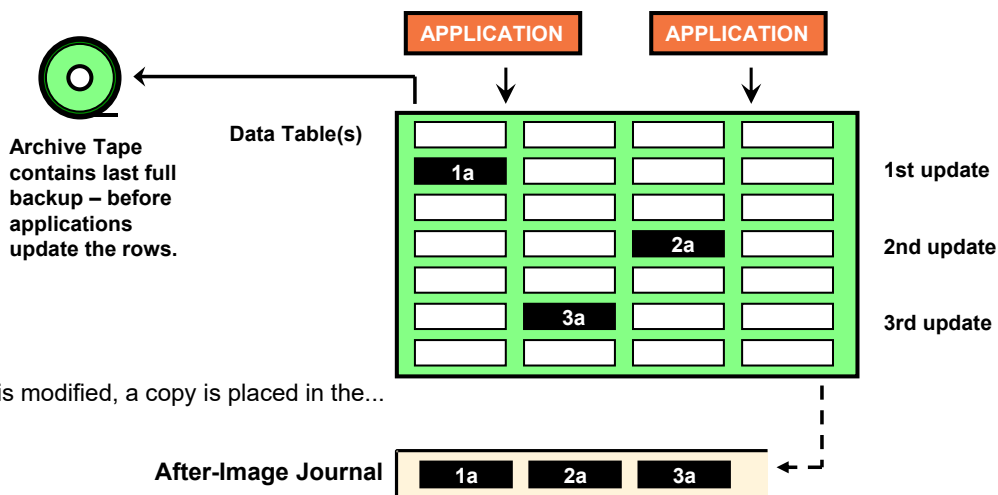
Both (Before Change Images and After Change Images)
The data in the permanent journal is maintained in the internal DBC format and is not accessible to the user through any SQL statements. Users delete permanent journals.

Before Images

Before Images are used for ROLLBACK recovery as shown on the following page. Once a before-image journal is created, a snapshot of an existing row is stored in the journal table before any data is modified. In the event of a software failure, the before-image journal can roll back any unwanted changes. Permanent journals roll back all transactions from a table to a checkpoint. They may not be used to roll back specific transactions.

After-Image Journals

After-images can be used to apply changes users have made since the last full backup.



To recover data that must be restored, use the after-images in the permanent journal to rollforward users' changes since the restored backup was taken.

After you create an after-image journal, a snapshot of a row value is stored in the permanent journal after a change is committed. If a hardware failure occurs, the after-image journal can roll forward any changes made to data tables since the last full system backup.

Site Disaster

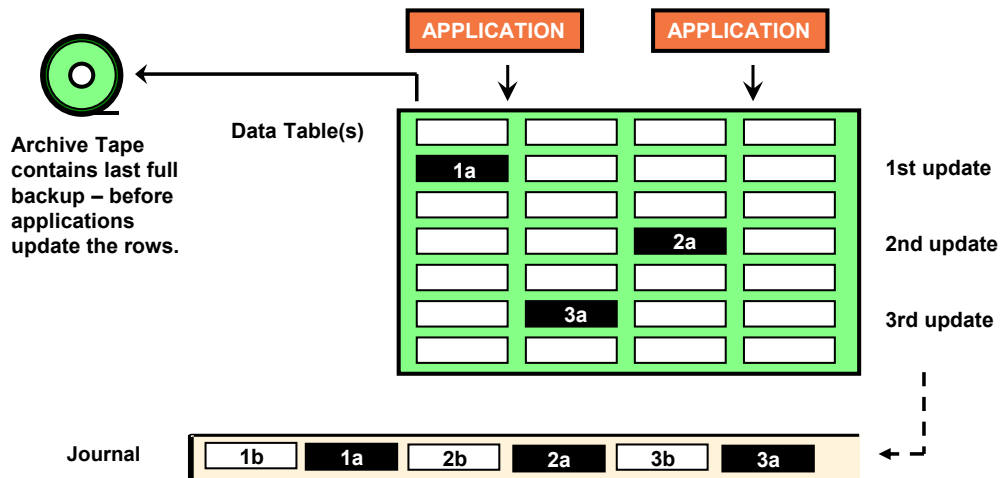
To protect against the loss of data in the event of a site disaster, many applications require that data archives be kept off-site at all times. Ideally, users dump the database to magnetic tape daily and store the tape off-site.

Daily archives may not be practical for very large databases. To solve this problem, you can activate after-change journals and take a daily archive of the journal itself that provides archived copies of all changes made since the last full database archive. The full backup tapes along with the journal backup tapes could restore the entire database.

This slide shows after images in the permanent journal are used for ROLLFORWARD recovery.

Before & After-Images – Same Journal

If both Before and after-images are captured, they are stored in the same journal.



Rollback commands apply Before-Images and Rollforward commands apply After-Images.

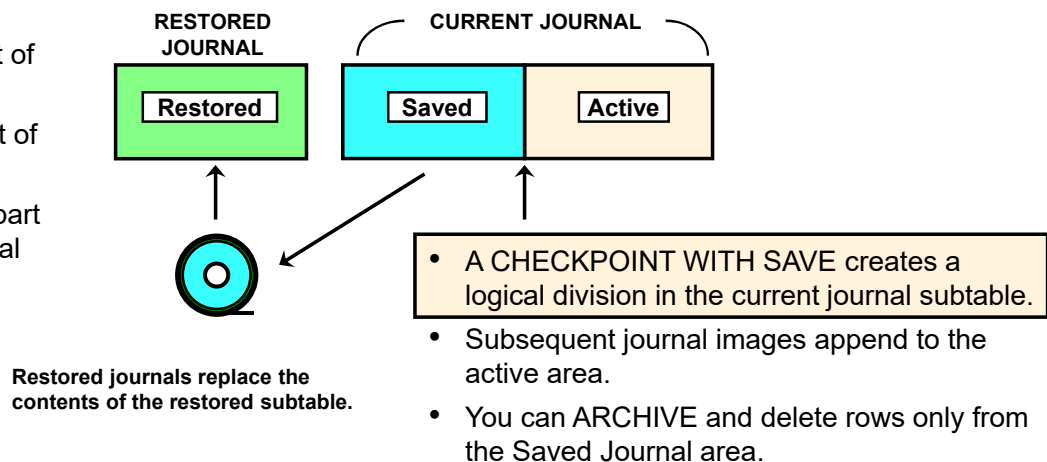
If you capture before and after images for a table, they are stored in the same Permanent Journal table as illustrated on this slide.

Before-images are used with ROLLBACK recovery and after-images are used with ROLLFORWARD recovery.

Journal Subtables

A permanent journal table consists of three areas:

- **Active area** – part of Current journal
- **Saved area** – part of Current journal
- **Restored area** – part of Restored journal



Each journal table consists of three areas:

- Active area (part of current journal subtable)
- Saved area (part of current journal subtable)
- Restored area (part of restored journal subtable)

Current Journal

Each time you update a data table that has an associated journal table; a change image is appended to the active subtable. You cannot archive journal tables while the change images are in the active subtable. Instead, you must move the images to the saved subtable. To move images from active to saved areas, you must submit the Checkpoint With Save statement. A checkpoint places a marker at the chronological end of the active subtable. The database assigns an event number any time a user submits the checkpoint statement. The With Save option of the checkpoint statement inserts a checkpoint in the active subtable and then appends the contents of the active subtable to the end of the saved subtable. After the database appends the contents, it initiates a new active subtable automatically. You can now submit an ARCHIVE JOURNAL TABLE statement. Archiving the journal saves it to tape.

Restored Journal

To restore a journal, move the journal table contents from the portable storage media back to the restored subtable. The information stays there until you invoke roll operations. Permanent journals are maintained in an internal Teradata database format. They are not accessible by SQL statements and cannot be used for audit trail purposes.

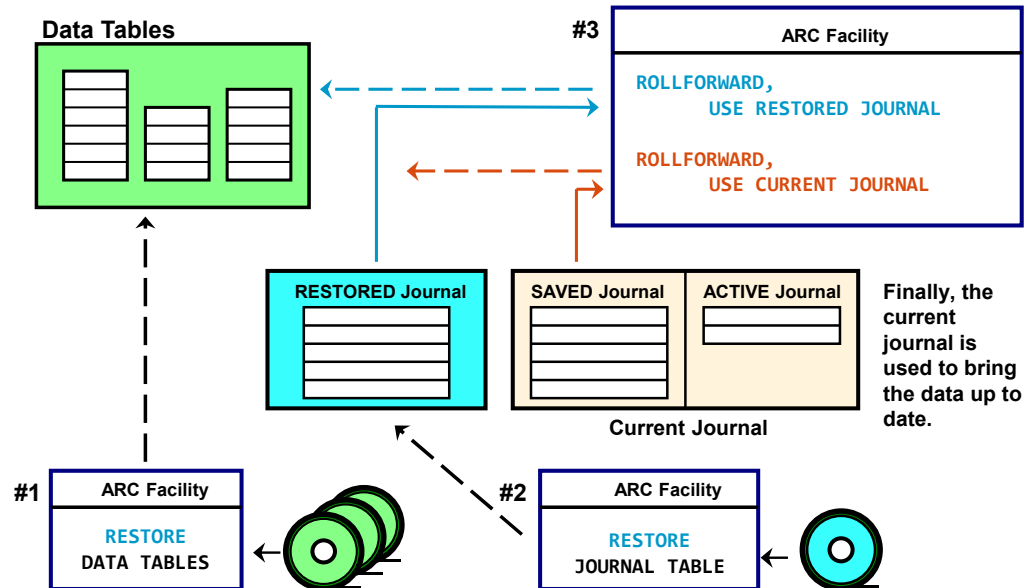
-
- The diagram illustrates the data flow and journaling process in a database system. It shows the following components and their interactions:
- Data Tables:** Represented by a green box containing three tables. It receives **INSERT**, **UPDATE**, and **DELETE** operations (indicated by a blue box with arrows).
 - Current Journal:** A yellow box containing two sections:
 - ACTIVE Journal:** A section for active journaling, containing a table.
 - SAVED Journal:** A section for saved journaling, containing a table.
 - ARC Facility #1:** A blue box labeled "ARC Facility" containing **ARCHIVE DATA TABLES**. It receives data from the Data Tables (indicated by a dashed arrow) and outputs to a stack of green disks (representing archived data tables).
 - ARC Facility #2:** A blue box labeled "ARC Facility" containing **CHECKPOINT WITH SAVE**. It receives data from the SAVED Journal (indicated by an orange arrow) and outputs to a single green disk (representing a checkpoint).
 - ARC Facility #3:** A blue box labeled "ARC Facility" containing **ARCHIVE JOURNAL TABLE**. It receives data from the ACTIVE Journal (indicated by an orange arrow) and outputs to a single cyan disk (representing an archived journal table).

ROLLFORWARD	Replaces a data row by its after-image from the beginning of the journal, to either a checkpoint or to the end of the journal.
ROLLBACK	Replaces a data row by its before change image from the end of the journal, to a checkpoint or to the beginning of the journal.
DELETE	Deletes the contents of either the saved or restored journal areas.

- Archive the data tables onto portable storage media.
- Submit a checkpoint with a SAVE statement to move change images from the active journal to the saved journal.
- Archive the journal tables onto portable storage media.
- Submit the DELETE JOURNAL statement to erase the saved journal rows.

Recovery with Permanent Journals

Tables or databases are restored first. Next, archived journals are restored, one at a time, and then the restored journal is rolled forward.



An example of how to use some of these ARC statements is shown when a batch program is run:

1. Submit an SQL Checkpoint statement as the first statement of the batch job, with or without a Checkpoint name.
2. If required, ROLLBACK to the Checkpoint using either the checkpoint name or the event number supplied by the DBC when you executed the Checkpoint command. Later changes are also backed out.
3. The data table is now in its original condition.

A permanent journal is time-oriented, not transaction-oriented.

Journals View

Associates journals with tables when the user owns or holds rights on the objects referenced.

[DBC.Journals\[V\]\[X\]](#)

Tables_DB	TableName	Journals_DB	JournalName
-----------	-----------	-------------	-------------

Example: List all tables in the system that use a journal and list the names of the journals.

```
SELECT TRIM (Tables_DB) || '.' || TableName AS "Table Name"
      ,TRIM (Journals_DB) || '.' || JournalName AS "Assigned to Journal"
FROM   DBC.JournalsV
ORDER BY 1;
```

Results:

Table Name	Assigned to Journal
HR_Tab.Employee	HR_Tab.HR_Jrnl
HR_Tab.Department	HR_Tab.HR_Jrnl
HR_Tab.Job	HR_Tab.HR.Jrnl
Payroll_Tab.Paycheck	Payroll_Tab.Payroll_Jrnl

The Teradata system provides a system view called DBC.Journals[V][X], that displays links between journal tables and the data tables that journal into them. The DBC.JournalsX View is a restricted view. The restricted version of the view displays only those objects that you own or to which you hold access rights.

The example on the next page uses the SELECT statement to list all of the tables in the system that uses a permanent journal. In addition, it requests to see a list of the journal names.

The response displays the table names first followed by the journal names.

Columns Defined

The Journals view has four different columns. Each one is described below:

Tables_DB	Displays the name of a database where a data table resides that has the journal option activated.
TableName	Displays the name of a data table that records changed images in a journal table.
Journals_DB	Displays the name of a database where a journal table resides.
JournalName	Displays the name of a journal table associated with a listed data table.

Permanent Journal Scenario

A Tale of Three Tables



A user has three data tables:

1. Table X Fallback
Before and
After Image Journals
2. Table Y No Fallback
No Before and
Dual After Image Journals
3. Table Z No Fallback
Single Before and
Single After Image Journals

In the following scenario, assume that a user has three tables in a four AMP system. Each table has its own data protection features stored on all four AMPs. The illustrations on the following pages illustrate data protection features in effect for each table.

In this example, AMPs 1 and 2 are clustered together and AMPs 3 and 4 are clustered together.

Table X

- ✓ Fallback protected data
- ✓ Fallback protected before images
- ✓ Fallback protected after images

Assume AMPs 0 & 1 are in a cluster and AMPs 2 & 3 are in a cluster.

	AMP 0	AMP 1	AMP 2	AMP 3
Data	Table X Primary 1	Table X Primary 2	Table X Primary 3	Table X Primary 4
	Table X Fallback 2	Table X Fallback 1	Table X Fallback 4	Table X Fallback 3
PJ	After 1	After 2	After 3	After 4
	After 2	After 1	After 4	After 3
	Before 1	Before 2	Before 3	Before 4
	Before 2	Before 1	Before 4	Before 3

Table X is defined as having fallback, dual before images, and dual after images.

Table Y

- ✓ No fallback
- ✓ Dual after image

Assume AMPs 0 & 1 are in a cluster and AMPs 2 & 3 are in a cluster.

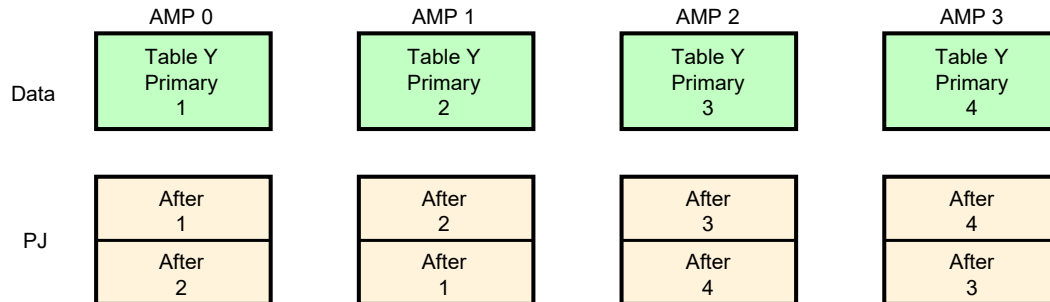


Table Y has no fallback protection, but has dual after image journaling defined.

Table Z

- ✓ No fallback
- ✓ Single before images
- ✓ Single after images

Assume AMPs 0 & 1 are in a cluster and AMPs 2 & 3 are in a cluster.

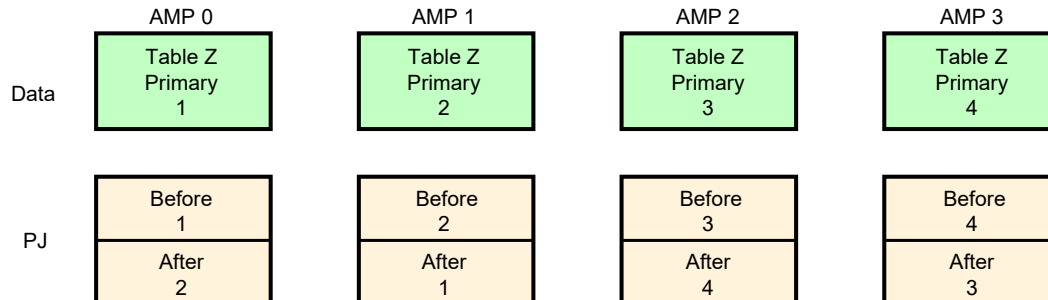


Table Z has no fallback protection. This table has single before and after-images.

Permanent Journals (Putting All Three Tables Together)

AMP 0	AMP 1	AMP 2	AMP 3
Table X Primary 1	Table X Primary 2	Table X Primary 3	Table X Primary 4
Table X Fallback 2	Table X Fallback 1	Table X Fallback 4	Table X Fallback 3
After 1	After 2	After 3	After 4
After 2	After 1	After 4	After 3
Before 1	Before 2	Before 3	Before 4
Before 2	Before 1	Before 4	Before 3
Table Y Primary 1	Table Y Primary 2	Table Y Primary 3	Table Y Primary 4
After 1	After 2	After 3	After 4
After 2	After 1	After 4	After 3
Table Z Primary 1	Table Z Primary 2	Table Z Primary 3	Table Z Primary 4
Before 1	Before 2	Before 3	Before 4
After 2	After 1	After 4	After 3

This slide shows the three tables and all their data protection options.

Archive Policy



The company established an archive policy to cover any data loss in the event of a site disaster. The archive policy has two components:

- Daily archive procedures
- Weekly archive procedures

Daily Archive Procedures

Each day the administrator submits a CHECKPOINT WITH SAVE command for each journal table which appends any changes stored in the active journal subtable to the saved journal subtable. In addition, it initiates a new active journal subtable. Second, the administrator archives each current journal, and then deletes the saved journal subtable from the saved journal.

Only one table is archived each day. By the end of the week, each table has been archived once.

Archive Scenario

Sunday:

Full archive of tables X, Y, and Z
DELETE all the Journals

Monday night:

Archive table X
Archive journals X, Y and Z

Tuesday night:

Archive table Y
Archive journals X, Y and Z

Wednesday night:

Archive table Z
Archive journals X, Y and Z

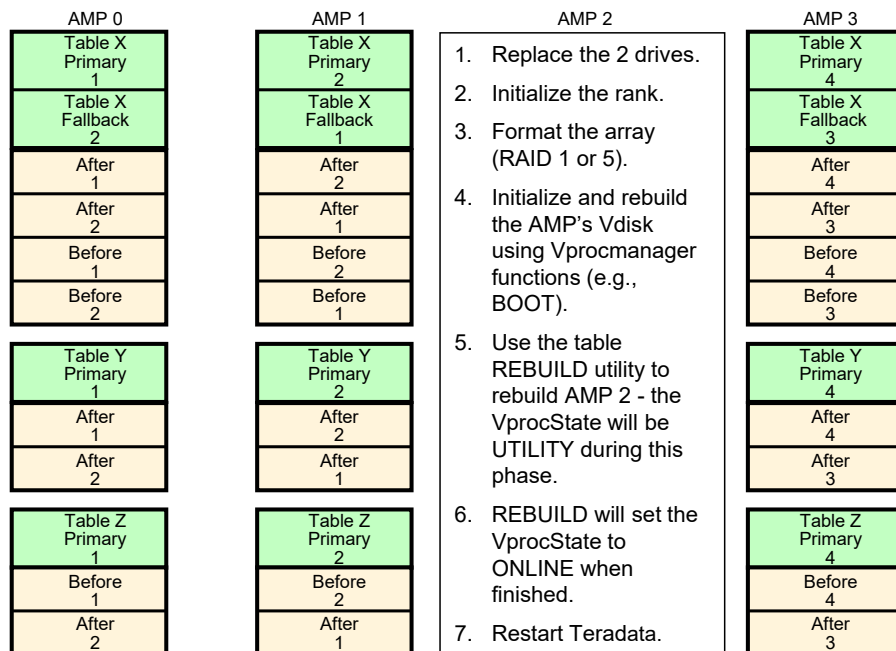
Thursday day:

AMP 2: Two drives fail in a RAID 1 drive group

The company activated its archive policy and implemented daily and weekly backup procedures as scheduled. Each day the administrator archives journals X, Y, and Z.

On Monday, the administrator archived data table X, and on Tuesday archived table Y. On Wednesday, the administrator archived data table Z. On Thursday, two drives failed in a drive group.

(After Teradata Restarts) Steps to Rebuild the Down AMP



After the system restarts, Table X is fully available because it is fallback protected. Tables Y and Z are partially available and any attempts to execute an all-AMP operation against table Y or Z will result in the following error message.

2642 AMP Down: The request against non-fallback Table_Y cannot be done.

The following steps are performed to rebuild the down AMP. The diagram on this slide outlines each of the steps. The steps are as follows:

1. Replace the 2 drives.
2. Initialize the rank.
3. Format the array (RAID 1, 5, or 6).
4. Initialize and rebuild the AMP's Vdisk using Vprocmanager functions (e.g., BOOT or INITVDISK).
5. Use the table REBUILD utility to rebuild AMP 2 - the VprocState will be UTILITY during this phase.
6. REBUILD will set the VprocState to ONLINE when finished.
7. Restart Teradata.

After REBUILD and Restart of Teradata

AMP 0	AMP 1	AMP 2	AMP 3
Table X Primary 1	Table X Primary 2	Table X Primary 3	Table X Primary 4
Table X Fallback 2	Table X Fallback 1	Table X Fallback 4	Table X Fallback 3
After 1	After 2	After 3	After 4
After 2	After 1	After 4	After 3
Before 1	Before 2	Before 3	Before 4
Before 2	Before 1	Before 4	Before 3
Table Y Primary 1	Table Y Primary 2	Table Y (Header)	Table Y Primary 4
After 1	After 2	After 3	After 4
After 2	After 1	After 4	After 3
Table Z Primary 1	Table Z Primary 2	Table Z (Header)	Table Z Primary 4
Before 1	Before 2		Before 4
After 2	After 1		After 3

The diagram on this slide shows the row information that the administrator recovered after the REBUILD and RESTART functions were executed.

Table X is fully recovered. All primary and fallback rows are restored. In addition, all before and after-journal images are recovered as well.

The administrator needs to perform additional recovery measures on table Y and table Z. Only the table headers for table Y and Z have been recovered on AMP 2.

Table X Recovery

- ✓ Fallback protected data
- ✓ Fallback protected before images
- ✓ Fallback protected after images

Fully Recovered

	AMP 0	AMP 1	AMP 2	AMP 3
Data	Table X Primary 1	Table X Primary 2	Table X Primary 3	Table X Primary 4
	Table X Fallback 2	Table X Fallback 1	Table X Fallback 4	Table X Fallback 3
PJ	After 1	After 2	After 3	After 4
	After 2	After 1	After 4	After 3
	Before 1	Before 2	Before 3	Before 4
	Before 2	Before 1	Before 4	Before 3

Table X has primary and fallback rows restored. All journal images are also recovered.

Table Y Recovery

Before Recovery:

1. All AMP RESTORE of Tuesday's ARCHIVE of Table Y.
2. Do NOT release utility locks.
3. RESTORE JOURNAL of Wednesday's ARCHIVE of journal Y.
4. All AMP ROLLFORWARD USE RESTORED journal of Table Y.
5. DELETE RESTORED JOURNAL Y.
6. All AMP ROLLFORWARD USE CURRENT journal of Table Y.
7. RELEASE utility LOCKs.

	AMP 0	AMP 1	AMP 2	AMP 3
Data	Table Y Primary 1	Table Y Primary 2	Table Y Primary (Header)	Table Y Primary 4
PJ	After 1	After 2	After 3	After 4
	After 2	After 1	After 4	After 3

After Recovery:

	AMP 0	AMP 1	AMP 2	AMP 3
Data	Table Z Primary 1	Table Z Primary 2	Table Z Primary 3	Table Z Primary 4
PJ				

The REBUILD utility was able to restore the table header for Table Y on AMP 2, but not the data because the table was not fallback protected. Since a dual-image journal was used, the journal images on AMP 2 were fully recovered. The end result is that the table Y does not have a full set of data. The administrator needs to perform some recovery procedures to fully recover missing data for table Y. Note: Any attempts to access (e.g., SELECT) table Y or Z will be unsuccessful the table header is marked as being in a "rebuild state" and it is not fully rebuilt at this time.

The administrator must perform the following steps to fully recover table Y:

1. Perform an all-AMP RESTORE using Tuesday's ARCHIVE of table Y to restore all data rows stored in the archive file from table Y.
2. Do NOT release the utility locks.
3. Use the RESTORE JOURNAL TABLE command to restore Wednesday's ARCHIVE JOURNAL of journal Y.
4. Perform an all-AMP ROLLFORWARD using the RESTORED journal from table Y. Doing so replaces the existing rows in table Y with any after-change images made since the last backup on Tuesday.
5. Use the DELETE JOURNAL command to delete restored journal Y. This action deletes all stored images from the restored journal.
6. Perform an all-AMP ROLLFORWARD using the CURRENT journal from table Y. This step replaces existing table rows with any after-change images stored in the active and/or saved subtables of the permanent journal.
7. RELEASE all utility locks.

Table Z Recovery

Before Recovery:

1. Specific AMP RESTORE of Wednesday's ARCHIVE of Table Z. Specific AMP operations are not recommended, but needed because a single after-image journal was used.
2. Do NOT release utility locks.
3. Specific AMP ROLLFORWARD USE CURRENT journal of Table Z.
4. Perform all-AMPs ARCHIVE of Table Z.
5. Run CHECKPOINT WITH SAVE and DELETE SAVED JOURNAL.
6. RELEASE utility LOCKs.

	AMP 0	AMP 1	AMP 2	AMP 3
Data	Table Z Primary 1	Table Z Primary 2	Table Z Primary (Header)	Table Z Primary 4
PJ	Before 1	Before 2		Before 4
	After 2	After 1		After 3

After Recovery:

	AMP 0	AMP 1	AMP 2	AMP 3
Data	Table Z Primary 1	Table Z Primary 2	Table Z Primary 3	Table Z Primary 4
PJ				

The REBUILD utility was able to restore the table header for Table Y on AMP 2, but not the data because the table was not fallback protected. Since a dual-image journal was used, the journal images on AMP 2 were fully recovered. The end result is that the table Y does not have a full set of data. The administrator needs to perform some recovery procedures to fully recover missing data for table Y. Note: Any attempts to access (e.g., SELECT) table Y or Z will be unsuccessful the table header is marked as being in a "rebuild state" and it is not fully rebuilt at this time.

The administrator must perform the following steps to fully recover table Y:

1. Perform an all-AMP RESTORE using Tuesday's ARCHIVE of table Y to restore all data rows stored in the archive file from table Y.
2. Do NOT release the utility locks.
3. Use the RESTORE JOURNAL TABLE command to restore Wednesday's ARCHIVE JOURNAL of journal Y.
4. Perform an all-AMP ROLLFORWARD using the RESTORED journal from table Y. Doing so replaces the existing rows in table Y with any after-change images made since the last backup on Tuesday.
5. Use the DELETE JOURNAL command to delete restored journal Y. This action deletes all stored images from the restored journal.
6. Perform an all-AMP ROLLFORWARD using the CURRENT journal from table Y. This step replaces existing table rows with any after-change images stored in the active and/or saved subtables of the permanent journal.
7. RELEASE all utility locks.

After Recovery

AMP 0	AMP 1	AMP 2	AMP 3
Table X Primary 1	Table X Primary 2	Table X Primary 3	Table X Primary 4
Table X Fallback 2	Table X Fallback 1	Table X Fallback 4	Table X Fallback 3
After 1	After 2	After 3	After 4
After 2	After 1	After 4	After 3
Before 1	Before 2	Before 3	Before 4
Before 2	Before 1	Before 4	Before 3
Table Y Primary 1	Table Y Primary 2	Table Y Primary 3	Table Y Primary 4
After 1	After 2	After 3	After 4
After 2	After 1	After 4	After 3
Table Z Primary 1	Table Z Primary 2	Table Z Primary 3	Table Z Primary 4

This slide shows the three tables after recovery. The following summary outlines the effects of permanent journals on recovery from a single disk failure.

Fallback Tables, Dual Image Tables (Table X)

- Processing continues
- Journals play no part in recovery

No Fallback Tables, Dual Image Journals (Table Y)

- Limited processing continues
- Data and journal tables are fully recovered

No Fallback Tables, Single Image Journals (Table Z)

- Limited processing continues
- Data is fully recovered
- Journals are lost

No Fallback Tables, No Journals

- Limited processing continues
- The administrator can only recover data to the point of the last archive

Commands to Support Permanent Journals

LOGON	Begins a session.
LOGOFF	Ends a session.
ARCHIVE JOURNAL TABLE	Archives a copy of a database or table to a host-resident data set/file.
ANALYZE	Reads an archive tape to display information about its content.
RESTORE JOURNAL TABLE	Restores a database or table from an archive file to specified AMPs.
RELEASE LOCK	Releases host utility locks on databases or tables.
CHECKPOINT	Marks a journal for later archive or recovery activities.
ROLLBACK	Recovers a database and tables to a state that existed <i>before</i> some change.
ROLLFORWARD	Recovers a database or table to a state that existed <i>after</i> some change.
DELETE JOURNAL	Deletes SAVED or RESTORED Journal rows.

This slide identifies key commands that we will cover in this module.

The restore statement allows you to move information from archive files back to the Teradata database. The restore operation can restore data or journal tables.

After you execute a RESTORE statement, data tables are ready to use.

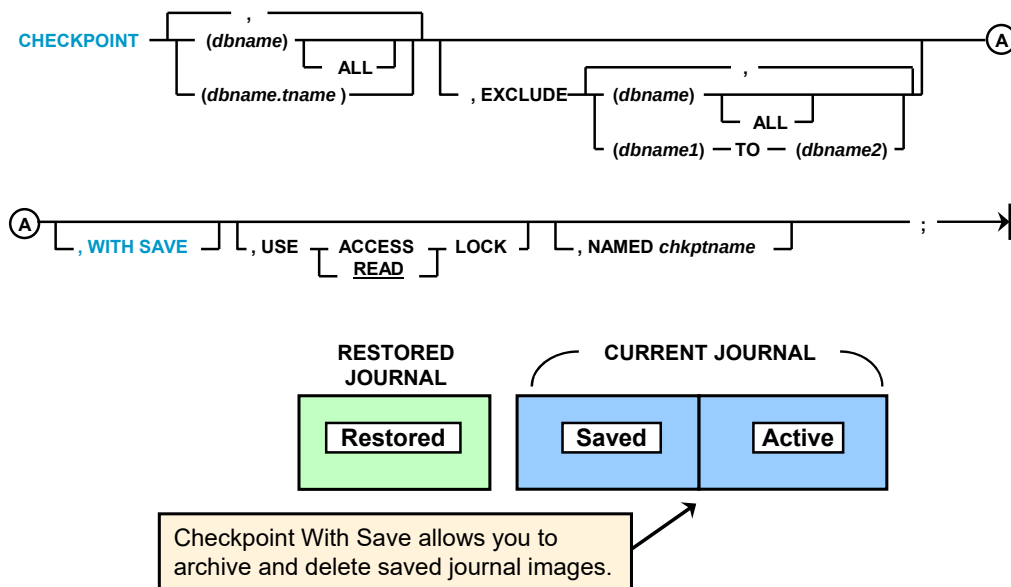
When you restore a journal table, the system restores the information to a permanent journal subtable. Before you can use the tables, you must perform a rollback or rollforward operation to move the journal tables back to the data tables.

Roll operations can use either the current journal or the restored journal. If you specify the current journal, then the ARC utility uses information stored in both the active and saved subtables.

A permanent journal is checkpoint-oriented rather than transaction-oriented. The goal of the journals is to return existing data tables to some previous or subsequent checkpoint. For example, if a batch program corrupted existing data, the rollback operation would return the data to a checkpoint prior to the running of the batch job.

A rollforward operation might occur after an all-AMP restore. After you move the data and journal archive files back to the database, the data tables would only include changes committed since the last full backup. Any intermediate changes would reside in the journal tables. The rollforward operation would replace the existing data with changes from the journal table.

The CHECKPOINT Statement



Use the CHECKPOINT statement to indicate a recovery point in the Journal.

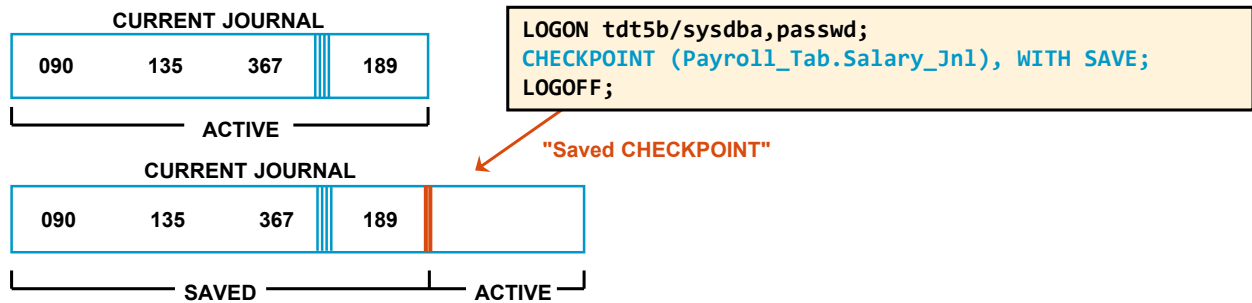
The CHECKPOINT statement places a marker row after the most recent change image row in the active subtable of a permanent journal. The database assigns an event number to the marker row and returns the number in response. You may assign a name to the CHECKPOINT command rather than use the event number in subsequent ARC activities. Use the following options with the CHECKPOINT statement:

WITH SAVE Use this option before you archive saved journal images to a host media to append the active journal subtable to the saved journal subtable. After you archive the saved area of the journal, you can delete this section of the current journal to make space for subsequent saved journal images. The saved journal subtable has no fixed size and can grow to the limit of the database.

USE ACCESS LOCK A checkpoint with save may optionally use an access lock. Without this option, the system must acquire a read lock on all tables assigned to the journal being checkpointed. Updates to tables may continue when you use the access lock option. Updates in progress complete after the checkpoint.

NAMED checkpointname Checkpoint names may be up to 30 characters long and are not case-specific. Teradata software always supplies an event number for each checkpoint. Use the number to reference a checkpoint if a name is not supplied.

CHECKPOINT WITH SAVE Statement



To create a saved checkpoint, archive the saved journal, and delete the images in the permanent journal, you could execute the following script.

```
LOGON tdt5b/sysdba,passwd;
CHECKPOINT (Payroll_Tab.Salary_Jnl), WITH SAVE;
ARCHIVE JOURNAL TABLE (Payroll_Tab.Salary_Jnl), ABORT, RELEASE LOCK, FILE=pay_jr1;
DELETE SAVED JOURNAL (Payroll_Tab.Salary_Jnl);
LOGOFF;
```

The CHECKPOINT WITH SAVE option inserts a marker row and appends any stored images preceding the marker row from the active to the saved subtable. The database automatically initiates a new active subtable. You can archive the contents of the saved subtable to an archive file.

Example

This slide shows two different current journals, before and after a checkpoint operation. The active subtable before checkpoint contains four change image rows. After checkpoint with save, the active subtable is empty, and the saved subtable contains the four change rows and a marker row.

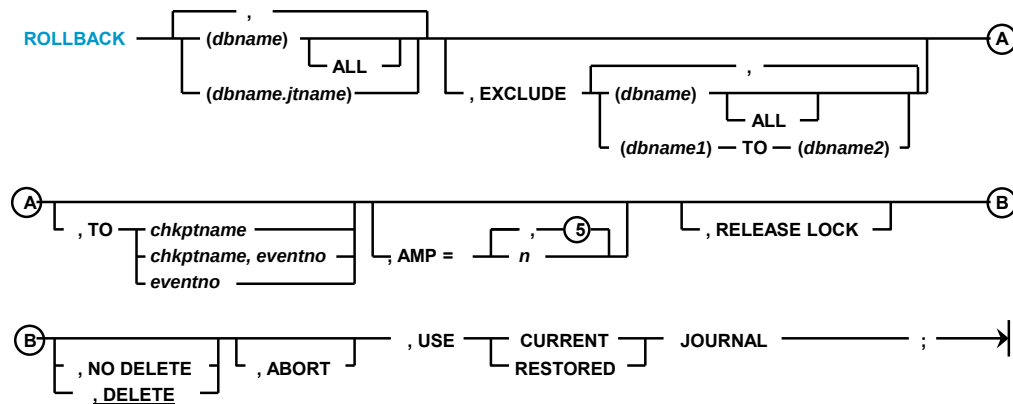
Checkpoint Lock Mechanisms

The default lock mechanism for the checkpoint command is read lock. The read lock suspends update activity for all data tables that might write changes to the journal table during checkpoint. This lock provides a clean point on the journal. The USE LOCK option permits users to assign an access lock rather than a read lock. The access lock accepts all transactions that insert change images to the journal, but it treats them as though they were submitted after the checkpoint was written. The access lock option requires that you also use the WITH SAVE option.

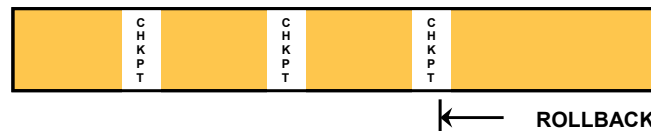
Checkpoint with Offline AMPs

An individual AMP may be off-line when you issue the checkpoint command. In this case, the utility automatically generates a system log entry that marks the checkpoint as soon as the AMP comes back on-line. The system startup process generates the checkpoint and requires no user input.

ROLLBACK Statement



Use ROLLBACK to recover from a transaction (software) error.



To recover from one or more transaction errors, use the ROLLBACK statement. To use this statement, you must define the table with a before-image journal table. The ROLLBACK is performed to a checkpoint or to the beginning of the current or restored journal.

The system uses the before images to replace any changes made to the table or database since a particular checkpoint was taken.

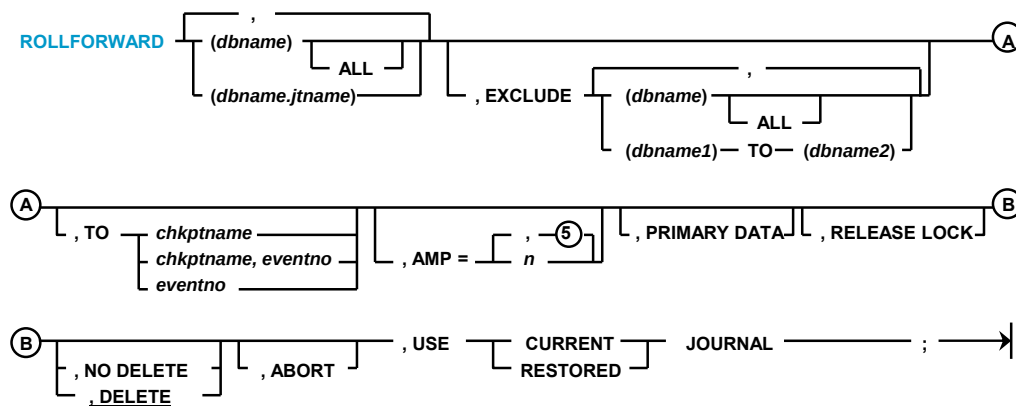
The ROLLBACK command helps you recover from one or more transaction errors. It reverses changes made to a database or table. To accomplish this reversal, it replaces existing data table rows with before-change images stored in a permanent journal. The before-change images must reside in either the restored or current subtables of a permanent journal. If you choose the current subtable for rollback procedures, the database uses the contents of both the active and saved subtables.

Example

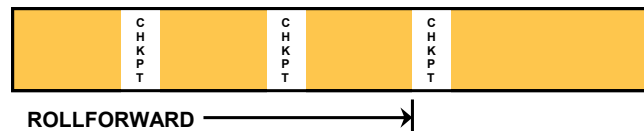
For example, as step 1, activate the ROLLBACK CURRENT JOURNAL statement to rollback any changes made since the journal table was archived. This statement rolls back the current subtable. Next (step 2), run the RESTORE JOURNAL TABLE command to load the appropriate archive file into the restored subtable of the permanent journal.

Finally (step 3), submit the ROLLBACK RESTORED JOURNAL command to reverse the changes by replacing any changed rows with their before-image rows stored in the restored journal. Repeat Steps 2 and 3 as necessary.

The ROLLFORWARD Statement



Use ROLLFORWARD to recover from a hardware failure.



The ROLLFORWARD command helps you recover from a hardware error and changes existing rows in data tables by replacing them with after-change images stored in a permanent journal. The after-change images must reside in either the restored or current subtables of a permanent journal.

The ROLLFORWARD statement is shown on this slide. Some options include:

PRIMARY DATA During a rollforward operation, this option instructs the software to ignore secondary index and fallback row updates. A BUILD operation will rebuild the invalidated fallback copy and indexes.

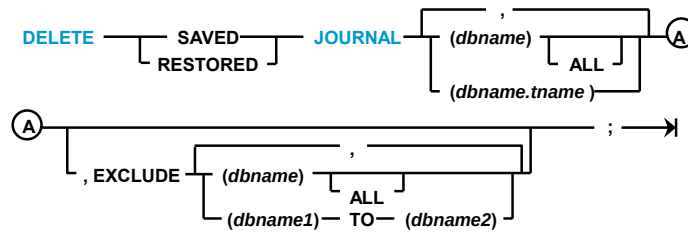
TO checkpointname, eventno

Checkpoint names need to match existing names used with a previous CHECKPOINT statement. An event number is the software-supplied event number of a previous checkpoint. You can supply either one or both of these. To find the checkpoint names or event numbers, select information about the checkpoint from the DBC.Events view. If there are duplicate checkpoint names in the journal and an event number is not supplied, rollforward stops when it encounters the first chronological entry made with a matching name.

AMP-Specific Restore

If you perform a restore operation on a specific AMP rather than on all AMPs, the ROLLFORWARD command does not permit you to use the TO CHECKPOINT NAME option. Following an AMP-specific restore, the system permits a rollforward only to the end of the journal. You must follow up the restore process with a rollforward of the entire journal table.

DELETE JOURNAL Statement



To use the DELETE JOURNAL statement, you must have the RESTORE privilege or own the database that contains the journal. Note: you cannot delete rows from an active journal.

To create a saved checkpoint and delete the images in the permanent journal, you could execute the following script.

```

LOGON sysdba,dbapass;
CHECKPOINT (Payroll_Tab.Salary_Jnl), WITH SAVE;
DELETE SAVED JOURNAL (Payroll_Tab.Salary_Jnl);
LOGOFF;
  
```

The DELETE JOURNAL command enables you to erase the contents of either the restored subtable or the saved subtable of a permanent journal. You cannot delete the contents of the active subtable. You must have the RESTORE privilege to execute this command.

Access Privileges

To delete a journal table, the user name specified in the LOGON statement must have one of the following:

- The RESTORE privilege on the database or journal table being deleted
- Ownership of the database containing the journal table

Restrictions

You cannot delete a saved subtable when all of the following conditions are true:

- A CHECKPOINT statement in the archive utilized an access lock, and
- The journal is not dual image, and
- One or more AMPs are off-line.

Transactions between an all-AMP archive and a single-AMP archive may not be consistent when a journal archive has all three of the above conditions. You cannot delete a saved subtable with an AMP off-line that does not have a dual journal.

The command does not delete the rows in the active journal.

Summary

- Permanent journals maintain a sequential history of all changes made to the rows of one or more tables.
- You create a permanent journal when you CREATE/MODIFY a user/database.
- The CHECKPOINT statement indicates a recovery point in a journal.
 - The **CHECKPOINT WITH SAVE statement** creates a row marker in the current journal and rows before the checkpoint marker are known as saved.
- ROLLBACK commands help you recover from one or more transaction errors and can reverse changes made to a database or table.
- ROLLFORWARD commands help you recover from hardware errors. These commands replace existing row data with after-change images.
- DELETE JOURNAL command erases the contents of either the restored subtable or the saved subtable in the permanent journal.

This slide summarizes some important concepts regarding this module.



Appendix F: Review Questions (1 of 2)

1. True or False: A permanent journal stores committed, uncommitted, and aborted changes to a row in a table.
2. True or False: A database or user can have many permanent journals.
3. True or False: Separate Permanent Journals are required for before and after images.
4. True or False: The Saved and Active areas are both part of the Current Journal.
5. True or False: The CREATE JOURNAL statement may be used to create a permanent journal.
6. True or False: Tables that use a Permanent Journal must be in the same database as the Permanent Journal.
7. After an application error, which two journal options can be used a ROLLBACK command? ____ ____
 - a. Active Journal
 - b. Saved Journal
 - c. Current Journal
 - d. Restored Journal
8. Which two objects can be used with a ROLLFORWARD command? ____ ____
 - a. Database
 - b. User table
 - c. Active Journal
 - d. Permanent Journal

Check your understanding of the concepts discussed in this module by completing the review questions as directed by your instructor.



Appendix F: Review Questions (2 of 2)

9. Which two sets of journal images can be deleted with a DELETE JOURNAL command?
 - a. Active Journal
 - b. Saved Journal
 - c. Current Journal
 - d. Restored Journal

10. Which command is needed to mark active journal images as saved?
 - a. CHECKPOINT with SAVE
 - b. CHECKPOINT without SAVE
 - c. CHECKPOINT with MARK
 - d. CHECKPOINT with MARK ACTIVE

Check your understanding of the concepts discussed in this module by completing the review questions as directed by your instructor.